DevOps:
от бита до кластера

Практическое руководство по Computer Science, сетям, Python, Bash и Go

Октябрь 2026

О книге

Эта книга написана для одного конкретного человека: инженера, который умеет запускать команды в терминале, но хочет понимать, что происходит под капотом — от электрического сигнала в процессоре до пода в Kubernetes. Она построена по принципу нарастания: каждая часть опирается на предыдущую, но каждая глава читается отдельно.

Книга не заменяет документацию. Её задача — дать карту местности и те объяснения, которых обычно не хватает между «прочитал man» и «понял, почему оно падает в три часа ночи».

Кому

- Тем, кто пришёл в DevOps из разработки и хочет закрыть дыры в фундаменте.
- Тем, кто пришёл из администрирования и хочет системно освоить Python и Go.
- Тем, кто готовится к собеседованию, где спрашивают не «что такое Docker», а «почему контейнер

видит меньше памяти, чем есть на хосте».

Как читать

Книга состоит из шести частей.

1. **Основы Computer Science** (главы 1–8) — как считает процессор, что такое память, как устроена ОС.

Без этого дальше будет казаться магией. Здесь же «Алгоритмика» (1a) — короткий вход в оценку сложности и выбор алгоритма, и пять глав про Linux: права и sudo (5a), пакеты и systemd (5b), namespaces и cgroups (5c), eBPF (5d) и SSH (5e). 5c объясняет, из чего физически сделан контейнер.

2. **HTTP и интернет** (главы 9–15) — что реально летит по проводу, когда вы открываете страницу, и

почему «работает у меня» перестаёт работать за прокси. Глава 12a — про то, как пользоваться чужим API: ключи, лимиты, пагинация, вебхуки; глава 13 — как спроектировать своё.

3. **Python** (главы 16–24) — основной язык автоматизации: от venv до asyncio, таймаутов и тестов.

Глава 24a добавляет Makefile — контракт репозитория, с которым работают одинаково все и CI.

4. **Bash** (главы 25–29) — клей, который держит всё вместе, и главный источник инцидентов вида

«скрипт сработал не так».

5. **Go** (главы 30–36) — язык сервисов и инструментов: статические бинарники, горутины, HTTP из

коробки. Здесь же данные: очереди и асинхронная обработка (34a) и SQL — индексы, планы, блокировки (34b).

6. **DevOps-практика** (главы 37–44) — Git, контейнеры, пайплайны, инфраструктура как код,

Kubernetes, наблюдаемость, безопасность и разбор инцидентов. Дополнительно: бэкапы и проверка восстановления (39a), Helm и GitOps (41a), нагрузочное тестирование (42a), секреты в Vault и SOPS (43a).

Четыре приложения — справочная часть: чек-лист продакшн-сервиса, шпаргалки по восьми инструментам, глоссарий на 153 термина со ссылками на главы и ответы ко всем 280 упражнениям (пять частей) — с разбором, а не только с конечным результатом.

Если вы совсем новичок — читайте по порядку. Если нужно быстро закрыть конкретную тему — начинайте с нужной главы, но держите в голове, что все последующие главы ссылаются на понятия из первых двух частей.

Отдельно про линию «Linux → Docker → Kubernetes», которая проведена намеренно: глава 5c разбирает namespaces и cgroups — ровно те два механизма ядра, из которых собран контейнер; глава 38 показывает, как из них получается образ и запущенный процесс; глава 41 — как этим оркестрируют. Читать их подряд полезнее, чем по отдельности.

Обозначения

- Блоки с серым фоном — команды оболочки или код. Их можно копировать.
- Синие термины — ссылки в глоссарий (приложение C): в PDF клик ведёт на определение, в печатной

версии — ищите термин в приложении C по алфавиту.

- Моноширинный текст внутри абзаца — имена команд, файлов, переменных, флагов.
- «На момент написания» — значение может измениться в следующей версии инструмента. Все версии,

приведённые в книге, сверены с первоисточниками на октябрь 2026 года.

- Ссылки вида «(гл. 5c)» указывают, где понятие разобрано подробно.

Чего в книге нет

- Пересказа официальной документации: её нужно читать отдельно и в актуальной версии.
- Привязки к одному облачному провайдеру: примеры построены так, чтобы переноситься.
- Обещания, что «этого достаточно»: инженер отличается не объёмом знаний, а привычкой проверять.

Источники и сверка

Технические утверждения сверялись с первоисточниками: RFC для сетевых протоколов, официальные release notes Python и Go, документация Kubernetes, страницы релизов. Версии инструментов, приведённые в тексте, соответствуют октябрю 2026 года; перед использованием в продакшене их нужно проверить — это нормальная часть работы, а не признак того, что книга устарела.

Числа, которые легко проверить и легко переврать, в книге даны с контекстом: 99,9% доступности — это 43 минуты в месяц, а не «почти всё время»; 1450 знаков на страницу A4 при 12,5 pt — это

замер на готовом PDF, а не оценка. Где утверждение зависит от версии ядра или дистрибутива, это сказано прямо.

Оглавление

Глава 1. Что такое вычисление: автоматы, машина Тьюринга, вычислимость 10

Глава 1a. Алгоритмика: как оценивать алгоритмы и не изобретать велосипед 20

Глава 2. Биты, числа и кодировки 30

Глава 3. Процессор и память: как код превращается в такты . . . 39

Глава 4. Операционная система: процессы, потоки, планировщик 48

Глава 5. Виртуальная память и файловые системы 58

Глава 5a. Linux: пользователи, права, sudo 68

Глава 5b. Linux: пакеты, systemd, журналы 78

Глава 5c. Linux: namespaces, cgroups, диагностика 89

Глава 5d. eBPF: наблюдение за ядром без пересборки 98

Глава 5e. SSH: ключи, конфиг, туннели 107

Глава 6. Сети: от кабеля до IP-пакета 117

Глава 7. TCP и UDP: надёжность, порты, сокеты 126

Глава 8. DNS: как имя становится адресом 136

Глава 9. HTTP/1.1: запросы, ответы, заголовки 144

Глава 10. HTTP/2 и HTTP/3: мультиплексирование и QUIC 154

Глава 11. Куки, сессии, кэширование, CDN 164

Глава 12. TLS и сертификаты 173

Глава 12a. API: что это такое и как им пользоваться 183

Глава 13. REST и проектирование API 193

Глава 14. Прокси, балансировщики, reverse proxy 203

Глава 15. nginx и event loop: почему один процесс держит 10k соединений 213

Глава 16. Python: окружение 223

Глава 17. Синтаксис и типы: коллекции, срезы, comprehension . . 233

Глава 18. Функции, замыкания, декораторы 244

Глава 19. ООП, dataclasses, протоколы, MRO 254

Глава 20. Асинхронность: asyncio 264

Глава 21. Сеть и API на Python 274

Глава 22. Файлы, конфиги, логирование 284

Глава 23. Тесты: pytest 295

Глава 24. CLI и упаковка 305

Глава 24a. Makefile: контракт репозитория 316

Глава 25. Шелл: переменные, кавычки, подстановки 327

Глава 26. Управляющие конструкции и коды возврата 338

Глава 27. Текст: grep, sed, awk, jq, xargs, find 349

Глава 28. Процессы, сигналы, перенаправления 359

Глава 29. Практика: скрипты бэкапа, healthcheck, деплоя, systemd timers 368

Глава 30. Go: начало 378

Глава 31. Синтаксис: срезы, мапы, указатели, интерфейсы, ошибки 388

Глава 32. Конкурентность: горутины, каналы, context 398

Глава 33. HTTP-сервер и клиент 408

Глава 34. Базы данных в Go 418

Глава 34a. Очереди и асинхронная обработка 426

Глава 34b. SQL: индексы, планы, блокировки 436

Глава 35. Тесты, бенчмарки, pprof, race detector 445

Глава 36. Сборка и поставка 453

Глава 37. Git: модель данных, ветки, rebase, хуки 461

Глава 38. Docker: слои, Dockerfile, compose 469

Глава 39. CI/CD: пайплайны, кэш, артефакты 479

Глава 39a. Бэкапы и восстановление 490

Глава 40. IaC: Terraform и Ansible 500

Глава 41. Kubernetes: поды, деплойменты, сервисы, probes 511

Глава 41a. Helm и GitOps 522

Глава 42. Наблюдаемость: метрики, логи, трейсы, SLO 533

Глава 42a. Нагрузочное тестирование 543

Глава 43. Безопасность и supply chain 553

Глава 43a. Секреты: Vault, SOPS, age 562

Глава 44. Инциденты, postmortem, on-call 572

Приложение A. Чек-лист продакшн-сервиса 582

Приложение B. Шпаргалки 591

Приложение C. Глоссарий 602

Приложение D. Ответы и решения — часть 1 (главы 1–8 и Linux) . 613

Приложение D. Ответы и решения — часть 2 (главы 9–20) 625

Приложение D. Ответы и решения — часть 3 (главы 21–32) . . . 636

Приложение D. Ответы и решения — часть 4 (главы 33–44) . . . 647

Приложение D. Ответы и решения — часть 5 (дополнительные главы) 658

Глава 1. Что такое вычисление: автоматы, машина Тьюринга, вычислимость

Инженер, который пишет скрипты развёртывания и правит манифесты Kubernetes, редко задумывается о том, что такое вычисление по своей сути. Между тем теория вычислений отвечает на вопросы, с которыми он сталкивается еженедельно: почему линтер не может доказать, что программа корректна; почему «умная» валидация конфигурации всегда остаётся набором эвристик; почему нельзя написать проверялку, которая заранее скажет, зависнет ли чужой процесс. В этой главе мы пройдём путь от простейшей модели вычислений — конечного автомата — до машины Тьюринга, сформулируем тезис Чёрча—Тьюринга и разберём доказательство неразрешимости проблемы остановки. Читатель научится отличать «мы не знаем, как это сделать» от «это невозможно в принципе» и оценивать, где автоматизация заканчивается.

Конечный автомат: светофор

Конечный автомат (finite state machine, FSM) — это пятёрка: множество состояний `Q`, входной алфавит `Σ`, функция переходов `δ: Q × Σ → Q`, начальное состояние `q₀` и множество принимающих состояний `F`. Всё «знание» автомата о прошлом упаковано в одно число — номер текущего состояния. Собственной памяти у него нет: он не считает, сколько событий пришло, не хранит историю, не накапливает счётчик.

Светофор — канонический пример. У него четыре состояния: красный, красный с жёлтым, зелёный, жёлтый. Единственное внешнее событие — тик таймера. Функция переходов задаётся таблицей, и таблица эта полная: для каждой пары «состояние + вход» определён ровно один следующий шаг.

Состояние	Вход	Следующее состояние
красный	тик	красный + жёлтый
красный + жёлтый	тик	зелёный
зелёный	тик	жёлтый
жёлтый	тик	красный

На Python это отображается почти дословно: словарь, ключом которого служит пара (состояние, вход). Такой код ценен не краткостью, а тем, что он исключает «непредусмотренные» ветки: всё поведение видно в одной структуре данных, и добавление нового состояния — это правка таблицы, а не поиск `if` по всему файлу.

```python
from enum import Enum

class Light(Enum):
    RED = "красный"
    RED_YELLOW = "красный+жёлтый"
    GREEN = "зелёный"
    YELLOW = "жёлтый"

DELTA = {
    (Light.RED, "timer"):        Light.RED_YELLOW,
    (Light.RED_YELLOW, "timer"): Light.GREEN,
    (Light.GREEN, "timer"):      Light.YELLOW,
    (Light.YELLOW, "timer"):     Light.RED,
}

def run(start: Light, events: list[str]) -> list[Light]:
    state, trace = start, [start]
    for e in events:
        state = DELTA.get((state, e), state)  # неизвестный вход игнорируется
        trace.append(state)
    return trace

print([s.value for s in run(Light.RED, ["timer"] * 6)])
```

Вывод:

```text
['красный', 'красный+жёлтый', 'зелёный', 'жёлтый', 'красный', 'красный+жёлтый',
    'зелёный']
```

Обратите внимание на строку `DELTA.get((state, e), state)`: она моделирует важное инженерное решение — реакцию на событие, которого нет в таблице. Автомат, у которого переход не определён, «застревает»; практическая реализация обычно

игнорирует неизвестный вход или переходит в состояние ошибки. Выбор между этими вариантами — не теоретический: именно он определяет, упадёт ли ваш сервис от лишнего поля в вебхуке.

Лексер: автомат, который работает в каждом компиляторе

Второй пример ближе к ежедневной работе: лексический анализатор (lexer, tokenizer). Он читает поток символов и выдаёт поток токенов — чисел, идентификаторов, операторов. Формально это тоже конечный автомат: состояния «начало токена», «внутри числа», «внутри идентификатора», «оператор», «ошибка»; памяти хватает лишь на накопление текущего токена.

python

```
DIGITS = set("0123456789")
LETTERS = set("abcdefghijklmnopqrstuvwxyz_")
OPS = set("+-*/=()")

def lex(src: str) -> list[tuple[str, str]]:
    tokens, i, n = [], 0, len(src)
    while i < n:
        c = src[i]
        if c in " \t\n":
            i += 1                      # пробелы не порождают токенов
            continue
        if c in DIGITS:                 # состояние NUM
            j = i
            while j < n and (src[j] in DIGITS or src[j] == "."):
                j += 1
            tokens.append(("NUM", src[i:j])); i = j
        elif c in LETTERS:              # состояние IDENT
            j = i
            while j < n and (src[j] in LETTERS or src[j] in DIGITS):
                j += 1
            tokens.append(("IDENT", src[i:j])); i = j
        elif c in OPS:                  # состояние OP
            tokens.append(("OP", c)); i += 1
        else:                           # состояние ERR
            raise SyntaxError(f"недопустимый символ {c!r} в позиции {i}")
    return tokens

print(lex("total = price * 1.2 + shipping_fee"))
```

Разбор строки `total = price * 1.2 + shipping_fee` даёт `IDENT total`, `OP =`, `IDENT price`, `OP *`, `NUM 1.2`, `OP +`, `IDENT shipping_fee`. Никакого «понимания» смысла здесь нет: различить `x*2` и `x**2` лексер не в состоянии, для этого нужен следующий уровень — синтаксический анализатор с магазинной памятью. Это первая

важная граница: регулярные языки не умеют считать вложенность произвольной глубины. Скобки, блоки, YAML-отступы — всё, что требует счётчика глубины, выходит за пределы FSM. Поэтому regex не подходит для разбора конфигов, а нужен настоящий парсер.

Машина Тьюринга

Машина Тьюринга (Turing machine, 1936) добавляет к автомату ровно две вещи: бесконечную ленту с ячейками и головку, которая может читать символ, записывать новый и сдвигаться на ячейку влево или вправо. Формально это набор $(Q, \Gamma, b, \delta, q_0, F)$: состояния, ленточный алфавит, пустой символ, функция переходов вида $Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$, начальное и принимающие состояния.

Разница с конечным автоматом принципиальна. FSM с n состояниями — это устройство с n «ячейками памяти», и всё, что оно способно распознать, описывается регулярным выражением. Машина Тьюринга имеет потенциально бесконечную память и поэтому может считать, сравнивать, умножать, интерпретировать другие машины. При этом её конструкция до смешного проста — и именно простота делает её удобной мерой: если задача решается на машине Тьюринга, она вычислима вообще.

Покажем машину, которая прибавляет единицу к двоичному числу. Идея: головка идёт вправо до пустой ячейки, затем влево, выполняя перенос: `1` заменяется на `0` с переносом дальше, `0` — на `1` с остановкой.

python

```
def tm_increment(bits: str, max_steps: int = 10_000) -> tuple[str, int]:
    """Прибавляет 1 к двоичному числу. '_' — пустая ячейка."""
    tape = list(bits) + ["_"]
    head, state, steps = 0, "seek_end", 0
    blank = "_"

    while steps < max_steps:
        steps += 1
        sym = tape[head]

        if state == "seek_end":
            if sym in "01":
```

```
                head += 1
            else:                # дошли до пустой ячейки — шаг назад
                head -= 1
                state = "carry"

        elif state == "carry":
            if sym == "1":       # 1 + 1 = 10: пишем 0, перенос влево
                tape[head] = "0"
                head -= 1
            else:                # '0' или пустая ячейка: пишем 1, готово
                tape[head] = "1"
                return "".join(tape).rstrip(blank), steps

        if head < 0:             # лента влево не бесконечна — расширяем
            tape.insert(0, blank)
            head = 0

    raise RuntimeError("не остановилась за отведённое число шагов")

for n in ("0", "1011", "111", "11111111"):
    result, steps = tm_increment(n)
    print(f"{n:>10} + 1 = {result:<12} шагов: {steps}")
```

Вывод на этой машине:

```text
         0 + 1 = 1           шагов: 3
      1011 + 1 = 1100        шагов: 8
       111 + 1 = 1000        шагов: 8
  11111111 + 1 = 100000000   шагов: 18
```

Параметр `max_steps` появился не из осторожности программиста, а из теории: у машины Тьюринга нет встроенного представления о том, «долго» она работает или «вечно». Любая реальная реализация обязана ввести внешний лимит — определить факт зацикливания изнутри невозможно.

Почему одной машины достаточно

Ключевой результат Тьюринга: существует универсальная машина (universal Turing machine), которая, получив на ленту описание любой другой машины и её вход, воспроизводит её поведение. Описание машины — это строка символов, то есть данные. Отсюда следует идея программы, хранящейся в памяти вместе с данными, — архитектура фон Неймана, на которой работают все современные серверы. Интерпретатор Python и есть универсальная машина, а файл скрипта — описание конкретной машины на ленте.

Тезис Чёрча—Тьюринга

В 1936 году почти одновременно появились две формализации вычислимости: машины Тьюринга и лямбда-исчисление Алонзо Чёрча. Позже выяснилось, что все разумные модели — рекурсивные функции, нормальные алгорифмы Маркова, стековые автоматы с неограниченным стеком, современные языки программирования — определяют один и тот же класс функций. Тезис Чёрча—Тьюринга утверждает: функция вычислима интуитивно (то есть существует механическая процедура её вычисления) тогда и только тогда, когда она вычислима на машине Тьюринга.

Это именно тезис, а не теорема: «интуитивно вычислимая» не имеет формального определения, поэтому доказать эквивалентность нельзя. Зато её можно опровергнуть — достаточно предъявить механическую процедуру, не выразимую на машине Тьюринга. За девяносто лет таких процедур не нашлось, включая квантовые вычисления: квантовый компьютер расширяет класс эффективно решаемых задач, но не класс вычислимых функций. Практический смысл тезиса для инженера простой: любой язык, любая программа, любой скрипт — это ровно машина Тьюринга по своим возможностям. Никакой «более мощный» инструмент не появится, и, что важнее, ограничения, доказанные для машин Тьюринга, действуют и для вашего Bash-скрипта.

Неразрешимость проблемы остановки

Проблема остановки (halting problem): по описанию программы `P` и её входа `x` определить, завершится ли `P(x)` за конечное время. Тьюринг доказал, что такая программа-анализатор не существует.

Доказательство ведётся от противного. Пусть есть функция `halts(p, x)`, которая всегда завершается и возвращает `True`, если `p(x)` останавливается, и `False` иначе. Построим на её основе программу `troll`:

python

```
def troll(p: str) -> bool:
    if halts(p, p):          # если p(p) останавливается...
        while True:          # ...то troll(p) не останавливается никогда
```

```
            pass
    return True              # иначе — немедленно возвращает True
```

Теперь спросим: останавливается ли `troll(troll)`? Если `halts(troll, troll)` вернула `True`, то `troll` уходит в бесконечный цикл — противоречие. Если `False` — `troll` немедленно возвращает `True`, то есть останавливается, — снова противоречие. Значит, `halts` не может существовать: любая её реализация либо даёт неверный ответ на некоторой программе, либо сама зависает.

Обратите внимание, чего доказательство не утверждает. Оно не говорит, что нельзя определить остановку для конкретной программы — для `print("hi")` это очевидно. Оно говорит, что нет универсального метода, работающего для всех программ сразу. И это не «пока не придумали», а доказанный предел: любое расширение анализатора можно обойти тем же диагональным трюком.

Где это встречается в работе

Следствия видны в трёх местах.

Первое — статические анализаторы. Линтеры, `mypy`, `golangci-lint` работают на консервативных приближениях: они либо пропускают часть ошибок (false negative), либо ругаются на корректный код (false positive). Обещать «полную проверку» невозможно не из-за слабости инструмента, а по теореме. Отсюда и культура `# type: ignore` — это не костыль, а плата за неразрешимость.

Второе — таймауты. Любой внешний вызов и любой CI-job должны иметь лимит времени: «дождёмся завершения» — не стратегия. `timeout 30 curl ...`, `context.WithTimeout` в Go, `timeoutSeconds` в Kubernetes — следствия отсутствия оракула остановки.

Третье — эквивалентность программ. Доказать автоматически, что две реализации одной функции идентичны, нельзя, поэтому рефакторинг проверяется тестами, а не анализатором.

Границы автоматизации

Из сказанного вытекает рабочая классификация задач, полезная при планировании автоматизации.

Класс задачи	Пример	Что ожидать
Разрешимая, эффективная	парсинг JSON, проверка схемы	автоматизируется полностью, детерминированно
Разрешимая, но дорогая	точное покрытие всех путей, SAT	автоматизируется с оговорками по времени
Полуразрешимая	«найдётся ли баг» перебором	ответ «да» рано или поздно придёт, «нет» — никогда
Неразрешимая	halt(P, x), эквивалентность программ	только эвристики и приближения

Инженерный вывод прост: если инструмент обещает «гарантированно найти все проблемы», проверьте, не подменил ли он неразрешимую задачу разрешимой, сузив область (например, анализируя только типы, а не значения). Приём нормальный и полезный — но обещание надо формулировать честно.

Отдельно проговорим разницу между неразрешимостью и сложностью. Задача может быть разрешимой, но требовать экспоненциального времени; тогда практический предел наступает задолго до теоретического. Поэтому, прежде чем строить «умную» систему, оценивают не только «можно ли», но и «за сколько».

Типичные ошибки

1. **Считать, что регулярное выражение разбирает вложенные структуры.** FSM не имеет счётчика глубины, поэтому regex «работает» на двух уровнях и ломается на трёх. Для произвольной вложенности нужен парсер со стеком.
2. **Доверять «полной» статической проверке.** Анализатор решает неразрешимую задачу, значит, он где-то приближает: либо пропускает случаи, либо ошибается на корректном коде. Настройка строгости — это выбор, в какую сторону ошибаться, а не путь к идеалу.
3. **Путать «программа не зависла» и «программа работает».** Отсутствие ошибки в логе не означает прогресса: процесс может быть в бесконечном цикле без вывода. Нужны

внешние признаки живости (health-check, метрика прогресса, watchdog), а не наблюдение за тишиной.

4. **Вводить таймаут только на клиенте.** Таймаут клиента не останавливает работу на сервере; без `context`/дедлайна, проброшенного вглубь, отменённый запрос продолжает жечь CPU и соединения к базе.
5. **Пытаться доказать эквивалентность двух реализаций автоматически.** Проверка эквивалентности — неразрешимая задача в общем виде; тесты и дифференциальное тестирование дают уверенность, но не доказательство.

Чек-лист

- Можете ли вы нарисовать таблицу состояний и переходов для конечного автомата в вашем сервисе (соединение, задача, статус заказа)?
- Есть ли в этой таблице явное поведение для неизвестного события, и задокументировано ли оно?
- Для каждой задачи автоматизации вы понимаете, разрешима она, полуразрешима или неразрешима — и сформулированы ли обещания инструмента соответственно?
- У всех внешних вызовов и фоновых задач есть таймаут и способ отмены, проброшенный вглубь?
- Есть ли у долгих операций внешний признак прогресса, отличимый от «процесс жив»?
- Знаете ли вы, какие проверки в вашем CI приблизительные, и готовы ли принять их false positive?

Упражнения

1. **Автомат.** Опишите светофор с кнопкой вызова для пешеходов: добавьте событие `button` и состояние «зелёный для пешеходов». Реализуйте таблицей переходов на Python и проверьте возврат в исходное состояние.
2. **Лексер.** Расширьте лексер из главы: добавьте строковые литералы в двойных кавычках с поддержкой

escape-последовательности `\"` и комментарии `# до конца строки`. Объясните, почему задача всё ещё решается конечным автоматом.

3. **Машина Тьюринга.** Реализуйте машину, которая по строке из `0` и `1` определяет, является ли число чётным, заменяя ленту на `Y`/`N`. Оцените число шагов как функцию длины входа.
4. **Практика.** Возьмите любой сервис из вашего окружения и постройте для него автомат жизненного цикла (запуск, готовность, деградация, остановка). Найдите переходы, которые сейчас обрабатываются неявно, и опишите, какой инцидент может из них вырасти.

Ответы и разбор к этим упражнениям — в приложении D, часть 1.

Глава 1a. Алгоритмика: как оценивать алгоритмы и не изобретать велосипед

Инженер, который обслуживает сервисы, редко пишет сортировки — он их вызывает. Но каждый день он принимает решения, которые по сути алгоритмические: парсить лог построчно или загрузить в память, искать в конфиге перебором или по индексу, хранить сессии в списке или в словаре, обходить граф зависимостей в ширину или в глубину. Ошибка здесь стоит не «некрасиво», а «джоб встал на шесть часов» или «сервис съел 8 ГБ на ровном месте».

Эта глава — короткий вход в алгоритмическое мышление: без доказательств и без олимпиадных трюков, но с числами, которые позволяют на глаз отличить решение, которое масштабируется, от решения, которое работает только на тестовом стенде. Формат намеренно простой: одна идея — один пример — одна оценка стоимости.

Бинарный поиск: разница между 128 и 7

Задача: найти имя в отсортированном списке из 128 элементов. Перебор в худшем случае — 128 сравнений. Бинарный поиск делит список пополам и отбрасывает половину: 128 → 64 → 32 → 16 → 8 → 4 → 2 → 1. Семь шагов вместо ста двадцати восьми.

python

```
def binary_search(items: list[int], target: int) -> int:
    low, high = 0, len(items) - 1
    while low <= high:
        mid = low + (high - low) // 2      # не (low + high) // 2 — см. ниже
        if items[mid] == target:
            return mid
        if items[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Вывод: `binary_search(list(range(128)), 100)` → `100`, семь итераций. Для миллиона элементов — двадцать итераций против миллиона.

Почему `low + (high - low) // 2`, а не `(low + high) // 2`: в языках с фиксированной разрядностью сумма двух больших индексов переполняется и становится отрицательной — знаменитый баг, который прожил в стандартной библиотеке Java двадцать лет. В Python переполнения нет, но привычка полезная: тот же код переживёт перенос на Go.

Условие применимости одно и жёсткое: **массив отсортирован**. Если данные меняются каждый день, дешевле пересортировать и искать бинарно, чем перебирать; если меняются каждую секунду — нужен индекс (B-дерево, хеш-таблица), а не бинарный поиск.

В эксплуатации это встречается буквально: `journalctl --since` работает по отсортированному по времени журналу, `git bisect` ищет коммит, сломавший сборку, делением истории пополам, а `sorted` + `bisect` в Python — стандартный способ найти окно в таймсерии без полного прохода.

O-большое: как разговаривать о скорости, не запуская код

O-большое описывает, как растёт время работы при росте входа, и отбрасывает всё, что не влияет на рост: константы и младшие слагаемые. $3n + 100$ — это $O(n)$: при n = миллион сто сравнений не значат ничего. $n^2 + n$ — это $O(n^2)$: квадрат съест линейное слагаемое.

Практическая таблица для $n = 1\,000\,000$ (порядок величины, не бенчмарк):

Класс	Что это	Операций при n = 10^6	Пример
O(1)	не зависит от n	1	доступ к `dict[key]`
O(log n)	деление пополам	~20	бинарный поиск, B-дерево
O(n)	один проход	10^6	чтение файла, `in` по списку
O(n log n)	проход с сортировкой	~$2 \cdot 10^7$	`sorted()`, `sort -k`
O(n^2)	вложенный цикл	10^{12}	сравнение всех пар
O(2^n)	перебор подмножеств	—	точное покрытие, брутфорс

Два вывода, которые экономят годы. Первый: разница между O(n) и O(log n) на миллионе — пять порядков, и именно она отделяет «грепнул лог за секунду» от «грепал сутки». Второй: O(n^2) на десяти тысячах элементов — это сто миллионов операций, то есть секунды в Python и мгновения в Go; на миллионе — уже сутки, и никакой оптимизацией констант это не лечится.

O-большое говорит о худшем случае, если не оговорено иное. Средний случай может быть лучше: quicksort в среднем O(n log n), но на неудачном входе деградирует до O(n^2).

Массив или связный список

Классическая таблица, которую полезно помнить наизусть:

Операция	Массив	Связный список
Чтение по индексу	O(1)	O(n)
Вставка в начало	O(n)	O(1)
Вставка в конец	O(1)*	O(1)
Поиск значения	O(n)	O(n)
Память на элемент	компактно	+указатель

*амортизированно, с редкой переаллокацией.

На бумаге список выигрывает на вставках, на практике — почти никогда. Причина не в асимптотике, а в железе: элементы массива лежат подряд, и процессор тянет их в кэш пачками по 64 байта; узлы списка разбросаны по куче, и каждое обращение —

промах кэша (глава 3). Замер из главы 3 на этой же машине: обход массива быстрее обхода связного списка в 91 раз при одинаковой O(n).

Отсюда рабочее правило: по умолчанию массив (в Python — `list`, в Go — слайс). Связный список оправдан, когда элементы вставляются и удаляются в середину потоком и узел уже найден, — например, LRU-кэш на двусвязном списке.

Рекурсия: база, шаг и стек

Рекурсивная функция — это функция, которая вызывает себя с меньшей задачей, плюс условие остановки. Без остановки — бесконечность и `RecursionError` (в CPython по умолчанию около 1000 кадров).

python

```
def factorial(n: int) -> int:
    if n <= 1:          # база: дальше некуда
        return 1
    return n * factorial(n - 1)   # шаг: задача меньше на единицу
```

Каждый вызов кладёт кадр в стек: аргументы, локальные переменные, адрес возврата. Глубина рекурсии — это глубина стека; десять тысяч вложенных вызовов в Python упадут раньше, чем закончится память. Поэтому обход дерева каталогов пишут итеративно с явным стеком или через `os.walk`, а не рекурсией «в лоб».

Рекурсия хорошо ложится на задачи, где структура данных сама рекурсивна: дерево JSON, файловая иерархия, разбор выражения, обход графа зависимостей.

Сортировки: почему `sorted()` — почти всегда правильный ответ

Сортировка выбором — учебный минимум: найти минимум, поставить первым, повторить. $O(n^2)$, для десяти тысяч элементов — сто миллионов сравнений.

Быстрая сортировка (quicksort) выбирает опорный элемент, разбивает массив на меньшие и большие, и рекурсивно сортирует половины. В среднем O(n log n), но при неудачном выборе

опорного (например, первого элемента в уже отсортированном массиве) — $O(n^2)$ и глубокая рекурсия.

Сортировка слиянием стабильно даёт O(n log n) и сохраняет порядок равных элементов, но требует дополнительной памяти под слияние.

Практический вывод: не писать сортировку. В Python `sorted()` и `list.sort()` — Timsort, гибрид слияния и вставок, O(n log n) в худшем случае, стабильный, с оптимизацией под почти отсортированные данные. В Go `sort.Slice` и `slices.Sort` — тоже O(n log n). Своя сортировка нужна ровно тогда, когда меняется критерий, а не алгоритм: например, «сначала по severity, потом по времени» — это ключ сортировки, а не новая сортировка.

```python
events.sort(key=lambda e: (severity_rank[e["sev"]], e["ts"]))
```

Хеш-таблицы: O(1), которое иногда становится O(n)

Словарь (`dict` в Python, `map` в Go) хранит пары «ключ — значение» в массиве бакетов: хеш ключа превращается в индекс, коллизии разрешаются цепочкой или открытой адресацией. В среднем чтение, запись и удаление — O(1). В худшем (все ключи в один бакет) — O(n), но это требует либо плохой хеш-функции, либо целенаправленной атаки.

Что важно эксплуатационно:

- **Загрузка (load factor).** Когда таблица заполнена примерно на две трети, она вырастает вдвое и

пересчитывает все хеши. Отсюда «неожиданные» всплески latency при массовой вставке в кэш.

- **Ключ должен быть неизменяемым.** В Python `list` нельзя положить ключом: его хеш меняется при

мутации. Кортеж — можно.

- **Порядок обхода.** В Python 3.7+ `dict` сохраняет порядок вставки, в Go порядок обхода мапы

намеренно случайный — код, который на него полагается, ломается нестабильно.

- **Память.** Словарь на миллион ключей — это десятки мегабайт, а не «пара мегабайт», как кажется.

Графы: BFS, DFS и порядок деплоя

Граф — это узлы и связи: сервисы и их зависимости, хосты и маршруты, пакеты и их требования. Два обхода решают почти всё:

- **BFS (в ширину)** — очередь, обходим слой за слоем. В невзвешенном графе даёт кратчайший путь в

рёбрах: «через сколько сервисов проходит запрос».

- **DFS (в глубину)** — стек (или рекурсия), идём до упора и откатываемся. Нужен для обнаружения

циклов, топологической сортировки, обхода дерева каталогов.

python

```
from collections import deque

def bfs_distance(graph: dict[str, list[str]], start: str, goal: str) -> int | None:
    seen, queue, dist = {start}, deque([(start, 0)]), {}
    while queue:
        node, d = queue.popleft()
        if node == goal:
            return d
        for nxt in graph.get(node, []):
            if nxt not in seen:
                seen.add(nxt)
                queue.append((nxt, d + 1))
    return None
```

Почему очередь даёт кратчайший путь: мы обрабатываем узлы в порядке удаления от старта, поэтому, когда цель встречена, короче пути не существует. Стек такого свойства не даёт: DFS найдёт какой-то путь, но не обязательно короткий.

Прикладной пример, знакомый каждому, кто катил Kubernetes: граф зависимостей сервисов и топологическая сортировка — это DFS с запоминанием порядка выхода, и именно она отвечает на вопрос «в каком порядке поднимать сервисы и где цикл, из-за которого деплой не сходится».

Жадные алгоритмы и цена жадности

Жадный алгоритм на каждом шаге берёт локально лучшее решение и не оглядывается назад. Иногда это оптимально (задача о выборе монет в канонической системе, минимальное остовное дерево), часто — нет. Классический контрпример: покрытие множествами. Есть радиостанции, каждая покрывает набор городов, нужно покрыть все города минимальным числом станций. Жадность «взять станцию, покрывающую больше всех непокрытых» даёт решение не хуже ln(n) от оптимума, но не обязательно оптимальное.

Практический смысл: когда планировщик раскладывает задачи по нодам «самой свободной», а кэш вытесняет «самый давний» элемент, это жадные эвристики. Они быстрые и почти всегда достаточно хорошие — и именно поэтому LRU-кэш иногда вытесняет элемент, который понадобится через секунду.

Динамическое программирование: запомнить подзадачу

Динамическое программирование — это рекурсия без повторного счёта: результат подзадачи запоминается и переиспользуется. Наивное `fib(n)` — $O(2^n)$ и минуты на `fib(35)`; с мемоизацией — O(n) и микросекунды.

```python
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n: int) -> int:
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

Идея шире, чем числа Фибоначчи: если задача разбивается на пересекающиеся подзадачи, кэш результатов превращает экспоненту в полином. В эксплуатации это встречается как «посчитать стоимость запроса по звеньям цепочки вызовов без повторного обхода общего хвоста» и как мемоизация в компиляторах и планировщиках.

Что из этого реально нужно в DevOps

- **Топ-K по частоте** (самые частые IP, коды ответа): `heapq.nlargest(k, counter.items(), key=...)` —

O(n log k) вместо полной сортировки O(n log n). На гигабайтном логе разница заметна.

- **Бинарный поиск по времени** в отсортированном логе: `bisect` вместо линейного прохода.
- **Множества для дедупликации**: `set` даёт O(1) на проверку, список — O(n); дедуп миллиона строк

списком — квадратичное время и часы ожидания.

- **BFS по зависимостям** — порядок запуска и поиск циклов.
- **LRU-кэш** — словарь плюс двусвязный список (или `OrderedDict.move_to_end`) для O(1) на чтение и

вытеснение.

Типичные ошибки

1. **Оптимизация констант вместо алгоритма.** Ускорять Python-цикл на 20%, когда сложность $O(n^2)$ и

нужно O(n log n), — это неделя работы ради процентов вместо часа ради порядка.

2. `in` **по списку внутри цикла.** `if x in list_` — это O(n) на каждую проверку, то есть $O(n^2)$ в

сумме. Замена списка на `set` обычно даёт ускорение в сотни раз на реальных данных.

3. **Сортировка внутри цикла.** `sorted(...)` на каждой итерации вместо одного прохода с накоплением —

типовая ошибка в отчётах и health-check-скриптах.

4. **Рекурсия без базы или на глубоких данных.** Обход файловой системы рекурсией падает на

`RecursionError` там, где итеративный стек работает.

5. **Полагаться на порядок обхода** `map` **в Go.** Он рандомизирован специально; код, который «работает

локально», начнёт мигать в проде.

6. **Считать O(1) у словаря гарантией.** При плохом хеше или атаке коллизиями деградация до O(n)

превращает кэш в тормоз.

7. **Путать «работает на 1000 строк» с «работает».** Квадратичный алгоритм на тестовых данных

незаметен, на проде — стена.

Чек-лист

- [ ] Для каждой горячей операции известна сложность и размер входа в проде, а не на фикстурах.
- [ ] Поиск по коллекции — в `set`/`dict`/индексе, а не `in` по списку в цикле.
- [ ] Сортировка выполняется один раз, а не внутри цикла.
- [ ] Для топ-K используется куча, а не полная сортировка.
- [ ] Рекурсия имеет базу и ограничение глубины; на глубоких данных есть итеративный вариант.
- [ ] Ключи словаря неизменяемы; порядок обхода не используется как контракт (кроме Python 3.7+).
- [ ] Обход графа зависимостей начинается с проверки на цикл.

Упражнения

1. Дан отсортированный список из миллиона чисел и миллион запросов «есть ли число». Сравните два

подхода: бинарный поиск на каждый запрос и загрузка в `set`. Что быстрее и во сколько раз при ваших оценках памяти?

2. В логе 2 ГБ нужно найти 20 самых частых IP. Опишите алгоритм с O(n log k) и объясните, почему

`sorted(Counter.items())` на этом объёме — плохая идея.

3. Напишите функцию, которая находит цикл в графе зависимостей сервисов и печатает сам цикл, а не

только факт его наличия. Оцените сложность.

4. Реализуйте LRU-кэш фиксированного размера на `dict` + двусвязном списке (или `OrderedDict`) и

докажите, что чтение и вставка — O(1).

5. Возьмите любой свой скрипт с вложенным циклом по спискам и перепишите его на множества или словарь.

Замерьте время до и после на реальных данных и объясните разницу через сложность.

Ответы и разбор к этим упражнениям — в приложении D, часть 1.

Глава 2. Биты, числа и кодировки

Любая программа, любой протокол, любой файл в конечном счёте — это последовательность битов, и большинство загадочных багов рантайма растёт из несовпадения представлений: сервер отдал 4 байта в одном порядке, клиент прочитал их в другом; в базе лежала строка в cp1251, а в JSON ушла как UTF-8; сумма в бухгалтерии не сошлась на копейку, потому что деньги считали во `float`. В этой главе разбирается, как из битов получаются числа: позиционные системы счисления, беззнаковые и знаковые целые, дополнительный код, переполнение, формат IEEE-754 с его знаменитым `0.1 + 0.2 != 0.3`, а затем — как из чисел получаются тексты: ASCII, Windows-1251, UTF-8. Читатель научится читать дампы байтов, понимать вывод `hexdump` и не удивляться, увидев `ÐŸÑ€Ð¸Ð²ÐµÑ`, вместо `Привет`.

Бит и позиционные системы счисления

Бит хранит одно из двух значений. Байт — это 8 бит, он же октет (octet) в сетевых стандартах; восьмибитный байт стал стандартом де-факто потому, что его хватало на один символ ASCII и на два шестнадцатеричных знака. `n` бит дают 2^n различных комбинаций: 8 бит — 256 значений, 16 бит — 65 536 (ровно столько портов в TCP и UDP), 32 бита — около 4,29 млрд (адрес IPv4), 128 бит — число с 39 цифрами (адрес IPv6).

В позиционной системе счисления вес разряда — это степень основания. В двоичной основание 2, поэтому $1011_2 = 1\cdot8 + 0\cdot4 + 1\cdot2 + 1\cdot1 = 11$. В шестнадцатеричной основание 16, цифры `0–9` и `a–f`. Шестнадцатеричная запись удобна не традицией, а арифметикой: одна hex-цифра — ровно 4 бита, две — ровно байт. Перевод между двоичной и hex-записью делается глазами, без вычислений, а между двоичной и десятичной — нет. Поэтому дампы памяти, хэши, цвета `#ff8800`, MAC-адреса и UUID печатают в hex.

python

```
n = 0xB4                       # hex-литерал, 180 в десятичной
print(n, bin(n), hex(n))       # 180 0b10110100 0xb4
print(int("10110100", 2))      # 180 — разбор строки в заданном основании
print(bin(0xB4 | 0x0F))        # 0b10111111 — побитовое ИЛИ
print(bin(0xB4 & 0x0F))        # 0b100 — маска младших 4 бит
print(0xB4 >> 4, 0xB4 & 0x0F) # 11 4 — старший и младший полубайты
```

Операции `>>`, `&`, `|`, `^` — это не «низкоуровневая экзотика»: на них построены флаги прав доступа (`0o755`), маски подсетей (`/24` — это `255.255.255.0`), битовые наборы признаков в заголовках TCP, сжатие (`gzip`), контрольные суммы и хэш-функции.

Знаковые числа и дополнительный код

Для беззнаковых целых из `n` бит диапазон — от `0` до $2^n - 1$. Для знаковых нужен способ кодировать минус. Наивный вариант «старший бит — знак, остальные — модуль» (sign-magnitude) неудобен: получается два нуля (`+0` и `−0`), а сложение требует отдельной логики для разных знаков. Вариант «обратный код» (ones' complement) тоже даёт два нуля.

Победил дополнительный код (two's complement): отрицательное число `−x` кодируется как `~x + 1`, то есть побитовая инверсия плюс единица. Старший бит получает вес -2^{n-1}, остальные — обычные положительные веса. Для 8 бит:

Биты	Беззнаковое	Знаковое
`0000 0000`	0	0
`0111 1111`	127	127
`1000 0000`	128	−128
`1111 1111`	255	−1

Главное свойство: сложение не зависит от знака. `−1 + 1` в 8 битах — это `1111 1111 + 0000 0001 = 1 0000 0000`, старший бит переноса теряется, остаётся `0000 0000`. Одна и та же схема сумматора работает для всех чисел, поэтому дополнительный код стоит в каждом процессоре.

python

```
print(bin(-5 & 0xFF))     # 0b11111011 — -5 в 8 битах
print((~5) & 0xFF)        # 250, инверсия пяти
print((~5 & 0xFF) + 1)    # 251 — то же число по формуле ~x + 1
print((-5 + 1) & 0xFF)    # 252 — сложение идёт по модулю 2^8
```

В Python целые произвольной точности, поэтому переполнения нет: `2**100` — обычное число. В C, Go, Java, Rust типы фиксированной ширины, и там граница реальна.

python

```
import ctypes

x = ctypes.c_int32(2**31 - 1)   # максимум для int32
print(x.value)                  # 2147483647
x.value = x.value + 1           # переполнение
print(x.value)                  # -2147483648
```

Переполнение знакового целого в C — неопределённое поведение (undefined behavior), и компилятор вправе выбросить проверку `if (x + 1 < x)`: он исходит из того, что переполнения не бывает. Отсюда классические уязвимости в проверках размера буфера. В Go переполнение определено и просто заворачивается по модулю 2^n, в Rust в debug-сборке — паника, в release — заворачивание. В Python безопасно, но за счёт другого представления чисел и скорости.

Порядок байтов

Многобайтовое число в памяти можно разложить двумя способами: старший байт первым (big-endian, «сетевой порядок») или младший первым (little-endian, x86-64 и ARM по умолчанию). `0x12345678` в little-endian лежит в памяти как `78 56 34 12`.

python

```
import struct
print(struct.pack("<I", 0x12345678).hex(" "))  # 78 56 34 12
print(struct.pack(">I", 0x12345678).hex(" "))  # 12 34 56 78
```

Символ `<` задаёт little-endian, `>` — big-endian, `!` — сетевой порядок. Все целые в заголовках IP, TCP, UDP передаются big-endian, поэтому код, который читает бинарный протокол, обязан указывать порядок явно. Если данные читаются «правильно» на одной машине и мусором — на другой, причина почти всегда здесь.

IEEE-754: почему 0.1 + 0.2 != 0.3

Дробные числа хранятся в формате IEEE-754 как `(−1)^s · 1.m · 2^e`. В `float64` (double) 1 бит знака, 11 бит экспоненты со смещением

1023 и 52 бита мантиссы; в `float32` — 1, 8 и 23. Ненулевое нормализованное число имеет неявную ведущую единицу, поэтому точность `float64` — около 15–17 значащих десятичных цифр, а `float32` — около 7.

Двоичная дробь представляет только суммы степеней двойки: `0.5`, `0.25`, `0.75` точны, а `0.1` — это `1/10`, и в двоичной системе она превращается в бесконечную периодическую дробь, как `1/3` в десятичной. Хранится ближайшее приближение. Отсюда знаменитый результат:

```python
import struct

def bits(v: float) -> str:
    return format(struct.unpack("<Q", struct.pack("<d", v))[0], "064b")

b = bits(0.1)
print(b[:1], b[1:12], b[12:])   # знак, экспонента, мантисса
print(struct.pack(">d", 0.1).hex())  # 3fb999999999999a
print(repr(0.1 + 0.2))          # 0.30000000000000004
print(0.1 + 0.2 == 0.3)         # False
print((0.1 + 0.2) - 0.3)        # 5.551115123125783e-17
```

Ошибка `5.55e-17` — это примерно 2^{-54}, половина шага сетки в районе 0.3. Она не «баг Python»: тот же результат дадут C, Go, JavaScript и калькулятор с двоичной арифметикой. `float32` ошибается заметнее:

```python
import struct
print(struct.unpack("<f", struct.pack("<f", 0.1))[0])  # 0.10000000149011612
```

Практические правила. Деньги считать в целых минимальных единицах (копейки, центы) или в `decimal.Decimal`; `Decimal("0.1") + Decimal("0.2")` даёт ровно `0.3`, но `Decimal(0.1)` — уже нет, потому что во `Decimal` попадёт двоичное приближение. Вещественные числа сравнивать с допуском: `math.isclose(a, b, rel_tol=1e-9)`. Никогда не проверять `if x == 0.0` после серии вычитаний — вместо нуля получится `1e-17`, а деление на него даст бесконечность. И помнить, что `float` не хранит `NaN`-семантику по правилам сравнения: `nan != nan` — это `True`.

Кодировки: от ASCII до UTF-8

ASCII и однобайтовые кодировки

ASCII (1963) закрепляет 128 кодов: управляющие символы `0x00–0x1F`, цифры `0x30–0x39`, заглавные латинские буквы `0x41–0x5A`, строчные `0x61–0x7A`. Семь бит, старший бит свободен — и именно это обстоятельство позже породило хаос: разные страны использовали освободившиеся 128 кодов под свои алфавиты. Так появились Windows-1251 (кириллица), KOI8-R, ISO-8859-1 (latin-1), CP866. Все они однобайтовые: один символ — один байт, `256` возможных символов.

Проблема не в самих кодировках, а в том, что байт не говорит, какой алфавит применять. Файл `cf f0 e8 e2 e5 f2` — это `Привет` в cp1251 и мусор в UTF-8. Пока весь мир писал внутри одной страны, это работало; с появлением веба любой текст мог прийти в любой кодировке, и угадывание превратилось в отдельную индустрию.

python

```
s = "Привет"
print(len(s), s.encode("utf-8")[:6].hex(" "))   # 6 d0 9f d1 80 d0 b8
print(len(s.encode("cp1251")), s.encode("cp1251").hex(" "))  # 6 cf f0 e8 e2 e5 f2
```

Обратите внимание: `len(s) == 6` — это шесть символов, а в UTF-8 получается 12 байт. Строка и её байтовое представление — разные вещи, и путаница между ними даёт баги в валидации длины полей и в обрезке сообщений.

Как устроен UTF-8

UTF-8 кодирует кодовые точки Unicode последовательностями от 1 до 4 байт по строгой схеме: ведущие биты первого байта говорят, сколько байт в последовательности, а каждый следующий байт начинается с `10`.

Диапазон кода	Байт	Первый байт	Остальные
U+0000–U+007F	1	`0xxxxxxx`	—
U+0080–U+07FF	2	`110xxxxx`	`10xxxxxx`
U+0800–U+FFFF	3	`1110xxxx`	2 × `10xxxxxx`
U+10000–U+10FFFF	4	`11110xxx`	3 × `10xxxxxx`

Взгляните на байты `Привет`: `d0 9f` — это `П`. `d0` = `11010000`, значит, два байта, полезных бит пять: `10000`. `9f` = `10011111`, шесть бит: `011111`. Склеиваем: `10000 011111` = `0x041F` = U+041F — заглавная «П». Проверим на настоящем вызове:

python

```
raw = "Привет".encode()
print(raw.hex(" "))              # d0 9f d1 80 d0 b8 d0 b2 d0 b5 d1 82
print(raw[:2].hex(" "))          # d0 9f
print(int("10000" + "011111", 2))  # 1055 = 0x41F
print(chr(0x41F))                # П
print("□".encode().hex(" "))     # f0 9f 9a 80 — четыре байта
```

Схема даёт три свойства, которых не было ни у одной однобайтовой кодировки. Первое: ASCII — подмножество UTF-8, поэтому старые протоколы, regex и `grep` по латинице работают без изменений. Второе: самосинхронизация — по любому байту можно определить, начало это символа или продолжение (`10xxxxxx` — продолжение), поэтому потерю одного байта в потоке можно обнаружить и восстановиться на следующем символе. Третье: отсутствие `0x00` внутри многобайтовых последовательностей, поэтому UTF-8 безопасен в C-строках с нулевым терминатором. Именно эти свойства, а не чья-то политическая воля, сделали UTF-8 стандартом де-факто: по разным оценкам, более 98% веб-страниц сегодня в UTF-8.

Ловушки текста

Первая — обрезка по байтам. `"Привет".encode()[:5]` даёт `b'\xd0\x9f\xd1\x80\xd0'` — последний символ разрезан, и `decode()` выбросит `UnicodeDecodeError`. Правильно резать строку по символам, а если нужна длина в байтах — считать `len(s.encode())`, а не `len(s)`.

Вторая — «кракозябры» (mojibake): байты UTF-8, прочитанные как cp1251. `"Привет".encode("utf-8").decode("cp1251")` возвращает `ĐŸÑ€Đ¸Đ²ĐµÑ‚`, — узнаваемый признак того, что где-то в цепочке «БД → бэкенд → HTTP → шаблон» потеряли кодировку или дважды её применили.

Третья — нормализация Unicode. Символ `й` существует в двух формах: одна кодовая точка U+0439 и композиция `и` + U+0306 (комбинирующая бревис). Визуально идентичны, при сравнении — разные строки, `len()` даёт 1 и 2. Решение:

`unicodedata.normalize("NFC", s)` перед сравнением, сохранением в ключ или поиском.

Четвёртая — BOM. В UTF-8 маркер `EF BB BF` не нужен, но некоторые редакторы его пишут; в начале файла он превращается в невидимый символ U+FEFF и ломает парсинг YAML, JSON и shell-скриптов (`#!/bin/bash` с BOM не исполняется). В UTF-16 BOM обязателен и задаёт порядок байтов.

Типичные ошибки

1. **Считать деньги во `float`.** Двоичная дробь не представляет 0.1 точно, ошибки накапливаются при каждом сложении, и на тысяче операций сумма расходится с бухгалтерской на копейки. Помогает переход на целые минимальные единицы или `Decimal` с явным контекстом округления.
2. **Сравнивать вещественные числа на равенство.** `0.1 + 0.2 == 0.3` ложно из-за округления, а результат зависит от порядка операций и уровня оптимизации. Нужен допуск (`math.isclose`) или целочисленная арифметика.
3. **Оценивать размер строки через `len(s)`.** В Python это число символов, в Go — байт, в C — байт до нуля. Валидация «не больше 255 символов» по разным метрикам даёт разные ответы для кириллицы и эмодзи, а лимит в БД обычно задан в байтах.
4. **Резать текст по байтам или по индексу байта.** Разрезанная многобайтовая последовательность не декодируется вовсе. Обрезка должна идти по границам символов (или графем, если важны эмодзи с модификаторами).
5. **Полагаться на «угадывание» кодировки.** Автодетект работает статистически и ошибается на коротких строках. Кодировку нужно фиксировать в протоколе (`Content-Type: text/html; charset=utf-8`), в схеме БД (`COLLATE`) и в настройках клиента.
6. **Игнорировать порядок байтов в бинарном протоколе.** `struct.pack("<I")` на x86 и чтение `>I` на другой стороне дают другое число, а не ошибку: молчаливое искажение данных

опаснее падения.

7. **Путать переполнение и насыщение.** При переполнении счётчика запросов `int32` значение уходит в минус и алерт «превышен порог» перестаёт срабатывать. Счётчики и таймеры надо держать в 64 битах и уметь насыщать, а не заворачивать.

Чек-лист

- Деньги и любые точные дробные величины хранятся в целых единицах или `Decimal`; в схеме БД тип `NUMERIC`, а не `FLOAT`?
- Все сравнения вещественных чисел идут через допуск, а не через `==`?
- В бинарных протоколах порядок байтов указан явно и одинаков на обеих сторонах?
- Кодировка задана в каждом звене: HTTP-заголовок, `Content-Type` письма, схема БД, `LANG`/`LC_ALL` в контейнере?
- Валидация длины строки считает байты, а не символы (или наоборот — осознанно)?
- Обрезка текста выполняется по границам символов, а не по индексам байтов?
- Строки перед сравнением нормализуются (`NFC`), если они приходят из разных источников?
- Текстовые файлы сохраняются без BOM, а чтение конфигов устойчиво к BOM на случай чужого редактора?

Упражнения

1. **Счисление.** Переведите `0xDEADBEEF` в двоичную, восьмеричную и десятичную запись без калькулятора, затем проверьте в Python. Объясните, почему перевод в hex делается «глазами», а в десятичную — нет.
2. **Дополнительный код.** Для 8-битных чисел постройте таблицу всех 256 значений и найдите, при каких операциях беззнаковая и знаковая интерпретация одного и того же байта дают разные результаты сравнения.

3. **IEEE-754.** Напишите функцию, которая печатает знак, экспоненту и мантиссу для `float32` и `float64`, и проверьте на числах `0.1`, `1/3`, `1e300`, `5e-324`. Найдите, при каком значении `float32` перестаёт различать соседние целые.
4. **UTF-8.** Реализуйте декодер UTF-8 вручную: на вход байты, на выходе список кодовых точек и позиция первой ошибки. Проверьте на корректной строке, на обрезанной последовательности и на байтах `c0 af` (перекодировка нуля — она запрещена стандартом).
5. **Практика.** Возьмите лог-файл вашего сервиса, найдите в нём не-ASCII символы и определите, в какой кодировке он записан. Проверьте, что происходит с ним при `iconv -f utf-8 -t cp1251` и обратно, и опишите, как отличить «двойное кодирование» от «неверной кодировки».

Ответы и разбор к этим упражнениям — в приложении D, часть 1.

Глава 3. Процессор и память: как код превращается в такты

Программа на любом языке в конечном счёте превращается в поток инструкций, которые процессор исполняет по тактам, и в обращения к памяти, которые уходят в кэш или в DRAM. Понимание этой пары — инструкции и данные — объясняет вещи, которые иначе выглядят магией: почему обход массива в девяносто раз быстрее обхода связного списка с теми же данными; почему добавление одного поля в структуру увеличивает её с 16 до 24 байт; почему сервис на 8 ядрах работает медленнее, чем на 4, если память «неправильная». В главе разбираются конвейер, предсказание переходов, иерархия кэшей с латентностями в тактах, выравнивание структур и NUMA. Все замеры в главе сделаны на реальной машине: Xeon E3-1230 v3, 3.3 ГГц, L1 32 КиБ на ядро, L2 256 КиБ, L3 8 МиБ — цифры на вашем железе будут отличаться, порядок величин — нет.

Такт и конвейер

Такт — минимальный интервал, за который процессор продвигает работу на один шаг. На 3.3 ГГц такт длится около 0.3 нс; за это время свет проходит 10 см. Тактовая частота давно перестала быть главной характеристикой: с середины 2000-х она упёрлась в тепловыделение, и рост производительности пошёл за счёт параллелизма внутри ядра и кэшей.

Современное ядро — конвейер (pipeline) из десятка стадий: выборка инструкции, декодирование, переименование регистров, планирование, исполнение, доступ к памяти, запись результата. Пока инструкция №1 исполняется, №2 декодируется, №3 выбирается из памяти. В идеале за такт завершается одна инструкция, на практике мешают три вида конфликтов (hazards): структурные (один и тот же блок занят), по данным (нужен результат предыдущей инструкции) и по управлению (неизвестно, куда прыгать).

Конфликты по данным процессор обходит пересылкой результатов (forwarding) и внеочередным исполнением (out-of-order): пока ждём данные из памяти, исполняются независимые инструкции, а окно переупорядочивания на современных ядрах — сотни инструкций. Конфликт по управлению решается предсказанием переходов (branch prediction). Ядро угадывает, пойдёт ли ветвление `if` или цикл на следующую итерацию, и спекулятивно исполняет угаданную ветку. Угадало — ветка уже готова; не угадало — конвейер сбрасывается, теряется 15–25 тактов.

Точность современных предсказателей — 95–99% на обычном коде, поэтому средний штраф невелик. Но у предсказания есть цена и цена эта видна в двух местах. Первое: на данных, которые ведут себя непредсказуемо, штраф платится постоянно. Классический пример — фильтрация массива, где условие зависит от случайных данных: тот же код на отсортированных данных работает в разы быстрее не потому, что меньше работы, а потому что предсказатель перестаёт ошибаться. Второе: спекулятивное исполнение — вектор атаки (Spectre и родственники), из-за чего компиляторы и ядра получили барьеры и оговорки, а часть оптимизаций стала недоступной.

Практический вывод для прикладного кода скромен: писать ветвления так, чтобы они были предсказуемы (сначала частый случай), не бояться «лишних» вычислений ради устранения ветки в горячем цикле и не пытаться перехитрить компилятор без замеров.

Иерархия памяти

Процессор за такт может выполнить несколько инструкций, но данные из DRAM идут сотни тактов. Разрыв компенсируется кэшами: маленькие и быстрые ближе к ядру, большие и медленные — дальше.

Уровень	Типичный объём	Латентность	Что это в тактах
Регистры	~1 КБ	< 1 такта	0
L1d	32–48 КиБ	~1 нс	4–5
L2	256 КиБ – 2 МиБ	~4 нс	12–20
L3	8–64 МиБ	~15 нс	40–75
DRAM	десятки–сотни ГиБ	~70–100 нс	200–350
NVMe SSD	терабайты	20–100 мкс	десятки тысяч
Сеть (RTT в ДЦ)	—	0.1–0.5 мс	сотни тысяч

Числа — порядок величин для настольных и серверных x86; на ARM и на новых поколениях они сдвигаются, но соотношения сохраняются. Разрыв между L1 и DRAM — примерно два порядка, между L1 и SSD — четыре-пять.

Кэш работает не с байтами, а с линиями (cache line) по 64 байта: читая один `int`, процессор тащит всю линию. Отсюда два следствия. Первое: последовательный доступ почти бесплатен — восемь подряд идущих `long` приходят одной линией. Второе: случайный доступ по одному байту из разных линий тратит 64 байта трафика на 8 полезных. Именно поэтому структуры данных, разложенные в памяти подряд, выигрывают у указателей даже при одинаковой алгоритмической сложности.

Ещё один механизм — аппаратный префетчер (hardware prefetcher). Он замечает последовательный или ступенчатый шаблон адресов и заранее подтягивает следующие линии. Префетчер спасает линейный обход: даже когда массив не влезает в кэш, загрузка идёт «пакетом» и латентность прячется. Но он бессилен против зависимых обращений: если адрес следующего элемента неизвестен, пока не прочитан текущий (связный список, обход дерева), предсказать нечего, и каждое обращение ждёт полную латентность DRAM.

Замер: массив против связного списка

Проверим это на практике. Программа ниже суммирует 4 194 304 числа двумя способами: по массиву подряд и по связному списку, узлы которого лежат в той же выделенной области, но связаны в случайном порядке. Объём данных одинаковый — 32

МиБ, работы одинаково — 4 миллиона сложений. Ниже приведён ключевой фрагмент файла `bench.c`; полная программа (~80 строк) компилируется как `gcc -O2 -o bench bench.c`.

c

```
typedef struct Node {
    struct Node *next;   // 8 байт
    long value;          // 8 байт
} Node;

// ... перемешивание индексов Фишера–Йетса ...
for (size_t i = 0; i < N; i++) nodes[i].value = (long)i;
for (size_t i = 0; i < N; i++)
    nodes[perm[i]].next = &nodes[perm[(i + 1) % N]];   // один цикл по всем узлам

// последовательный проход
for (size_t i = 0; i < N; i++) s1 += arr[i];

// обход списка: каждое обращение зависит от предыдущего
for (size_t i = 0; i < N; i++) { s2 += p->value; p = p->next; }
```

Сборка и запуск:

bash

```
gcc -O2 -o bench bench.c && ./bench
```

Вывод на тестовой машине:

text

```
массив:     3.9 ms    8.60 ГБ/с  0.930 нс/элемент
список:   356.2 ms    0.09 ГБ/с  84.919 нс/элемент
разница: 91x (суммы 8796090925056 и 8796090925056)
```

Девяносто один раз — при одинаковом количестве сложений и одинаковом объёме данных. Контрольные суммы совпадают, значит, оба цикла прочитали ровно одно и то же. Причина — не в указателях как таковых, а в том, что разыменование `p->next` создаёт цепочку зависимостей: адрес следующего узла неизвестен, пока не завершено текущее чтение. 84.9 нс на элемент при 3.5 ГГц — это около 300 тактов, ровно латентность DRAM. Массив же читается по 0.93 нс на элемент, потому что префетчер идёт впереди цикла.

Вторая часть вывода того же прогона показывает, где кончается кэш. Обход массива разного размера, лучшее из 20 прогонов:

text

```
размер    мс        нс/элемент
```

```
   64 КБ    0.003   0.3499
 1024 КБ    0.046   0.3500
 4096 КБ    0.184   0.3500
16384 КБ    1.334   0.6360
65536 КБ    6.346   0.7564
```

До 4 МиБ производительность не меняется — данные помещаются в L2/L3, дальше начинается DRAM, и время на элемент удваивается. Обратите внимание: рост всего вдвое, а не в сто раз. Это работа префетчера: последовательный доступ к DRAM конвейеризуется. Именно поэтому главный враг производительности — не «медленная память», а непредсказуемый доступ к ней.

Выравнивание и размер структуры

Компилятор размещает поля структур с выравниванием (alignment): поле типа `long` должно начинаться с адреса, кратного 8. Чтобы это выполнялось, между полями вставляются невидимые байты — паддинг, а размер структуры округляется до кратности её выравнивания.

c

```
struct A { char a; long b; char c; };   // sizeof = 24
struct B { long b; char a; char c; };   // sizeof = 16
```

На тестовой машине `sizeof(A)` равен 24, `sizeof(B)` — 16, хотя полей в них поровну и данные те же. В `A` после `char a` вставлено 7 байт паддинга, чтобы `long b` лёг на границу 8, и ещё 7 байт в конце — чтобы размер был кратен 8. Перестановка полей по убыванию размера экономит треть памяти на структуре. На массиве из миллиона записей это 8 МиБ трафика и заметно меньше промахов кэша.

На x86 невыровненный доступ обычно работает, но стоит дороже: обращение, пересекающее границу линии, требует двух обращений к кэшу. На ARM часть невыровненных обращений приводит к исключению. Упаковка структур через `#pragma pack(1)` или атрибут `packed` убирает паддинг, но создаёт ровно эту проблему — и часто вдобавок запрещает векторизацию. Упаковывать имеет смысл только форматы на диске и в сети, а не структуры в памяти.

Отдельный случай — ложное разделение (false sharing). Если два потока пишут в разные поля, попавшие в одну 64-байтовую линию, линия «прыгает» между ядрами, и оба потока тормозят, хотя данные не пересекаются. Лечится выравниванием разделяемых структур по границе линии (в C — `alignas(64)`, в Go — паддинг до 64 байт, в Java — `@Contended`), и это одна из немногих оптимизаций, которая даёт эффект без изменения алгоритма.

NUMA кратко

На многосокетных серверах память физически привязана к сокетам: часть планок подключена к первому процессору, часть — ко второму. Доступ к «своей» памяти занимает условные 80 нс, к чужой — 130–150 нс плюс ограничение пропускной способности межсокетной шины. Linux распределяет страницы по принципу первого касания (first touch): страница физически выделяется на том узле, чей поток первым в неё записал. Отсюда два практических правила: инициализировать большие массивы в том же потоке, который будет их обрабатывать, и привязывать процессы к узлам (`numactl --cpunodebind=0 --membind=0`). В контейнерах на многосокетном хосте без привязки планировщик легко разложит потоки по разным сокетам, и приложение будет вдвое медленнее «на пустом месте».

Что из этого следует для кода

Первое: структура данных важнее микрооптимизаций. Массив структур (AoS) хорош, когда нужны все поля записи; массив отдельных полей (SoA) — когда нужен один признак по всем записям: он читает 1/8 трафика и векторизуется. Колоночные СУБД (ClickHouse, Parquet) построены на этом же принципе.

Второе: избегать цепочек указателей в горячем пути. `[]*Item` в Go, `list.List`, словарь объектов в Python — все они превращают линейный обход в набор зависимых обращений. Замена на `[]Item` или на индексы вместо указателей часто даёт больше, чем любой профайлер.

Третье: батчить. Один запрос к БД на 1000 строк дешевле тысячи запросов по одной строке не из-за сети, а из-за локальности: драйвер собирает строки в один буфер.

Четвёртое: не оптимизировать без замеров. Кэш-эффекты нелинейны: программа может ускоряться при добавлении работы (лучше шаблон доступа) и замедляться при её сокращении. Мерить надо на реальном объёме данных, а не на тестовом массиве из десяти элементов.

Типичные ошибки

1. **Оптимизировать на языке с интерпретатором, глядя на алгоритм.** В Python накладные расходы интерпретатора (десятки-сотни нс на операцию) перекрывают латентность кэша, поэтому «локальность» в чистом Python почти не видна; она проявляется в numpy/pandas, где цикл уходит в C. Замер на CPython-цикле покажет одно, а на реальной нагрузке — другое.
2. **Считать, что связный список медленный из-за аллокаций.** В замере выше все узлы лежали в одном `malloc`, аллокаций во время обхода не было — и всё равно 91-кратная разница. Причина в зависимых обращениях, а не в `malloc`.
3. **Игнорировать паддинг при проектировании структур.** Порядок полей и лишний `bool` в середине структуры увеличивают её на 30–50%, а на массивах это прямо превращается в промахи кэша. Проверять `sizeof` (или `unsafe.Sizeof` в Go) дешевле, чем потом разбираться с памятью.
4. **Упаковывать всё через `packed`.** Экономия байтов оборачивается невыровненными обращениями, падением скорости и потенциальными исключениями на ARM. Упаковка уместна для внешних форматов, а не для рабочих структур.
5. **Держать «горячие» счётчики потоков рядом.** Два атомарных счётчика в одной линии превращают масштабирование в ping-pong; на 8 потоках суммарная пропускная способность падает в разы. Разносить по линиям, а не по структурам.

6. **Забывать про NUMA в контейнерах.** Лимит CPU и memory limit ничего не говорят о топологии; приложение может половину обращений делать в удалённый узел. Проверять `numactl --hardware` и профиль деградации при росте потоков.
7. **Верить одному замеру.** Первый прогон включает прогрев кэшей, page faults и JIT; результат скачет в разы. Нужны прогресс, несколько итераций, минимум или медиана, и фиксация частоты CPU.

Чек-лист

- Горячие структуры данных лежат в памяти подряд, без цепочек указателей на каждый элемент?
- Порядок полей в часто используемых структурах проверен через `sizeof`/`unsafe.Sizeof`, лишнего паддинга нет?
- Для разделяемых между потоками счётчиков исключено ложное разделение (выравнивание по 64 байтам)?
- Обход больших наборов данных идёт последовательно (по индексу, по колонке), а не в случайном порядке?
- Замеры сделаны на объёме данных, близком к продакшену, с прогревом и несколькими итерациями?
- На многосокетном хосте проверена топология и, если нужно, сделана привязка процессов к NUMA-узлу?
- В коде на интерпретируемом языке тяжёлые циклы вынесены в векторизованные библиотеки, а не написаны на самом языке?
- Известно, какие ветвления в горячем пути зависят от случайных данных (и нельзя ли их устранить)?

Упражнения

1. **Кэш-лестница.** Повторите замер из главы на своей машине для размеров от 4 КиБ до 256 МиБ. Определите по графику «время на элемент от размера» объёмы L1, L2 и L3 и сравните с выводом `lscpu`.

2. **Структуры.** Для структуры `{char, int, char, double, char}` посчитайте `sizeof` вручную, затем проверьте компилятором. Найдите перестановку полей с минимальным размером и объясните, почему меньше определённого значения размер быть не может.
3. **Ложное разделение.** Напишите программу с двумя потоками, каждый из которых инкрементирует свой счётчик миллион раз: сначала счётчики в одной структуре, затем разнесённые на разные линии. Сравните время и объясните разницу.
4. **Указатели против индексов.** Возьмите обход бинарного дерева на 10 миллионов узлов, реализованный через указатели и через индексы в массиве. Сравните время поиска и объясните, почему разница зависит от высоты дерева.
5. **NUMA.** На машине с двумя сокетами запустите один и тот же процесс с `numactl --cpunodebind` и без него, замерив пропускную способность чтения памяти. Объясните, почему для однопоточного кода разница может быть незаметна, а для многопоточного — двукратной.

Ответы и разбор к этим упражнениям — в приложении D, часть 1.

Глава 4. Операционная система: процессы, потоки, планировщик

Операционная система делит процессор между десятками задач, прячет от программы физическую память и устройства и решает, кому достанется следующая микросекунда. Инженеру, который деплоит сервисы и разбирает инциденты, нужно понимать три вещи: чем процесс отличается от потока; как ядро выбирает, что запустить дальше, и что означает `nice`; что происходит при системном вызове и при смерти процесса. В главе — реальные наблюдения за ядром Linux 6.12: состояние зомби в `/proc`, счётчик добровольных переключений контекста, цена 500 потоков и 500 процессов, лимиты дескрипторов. Из этого собирается поведение контейнеров и Kubernetes.

Процесс и поток

Процесс (process) — программа в состоянии исполнения со своим адресным пространством: код, данные, куча, стек, таблица открытых дескрипторов, идентификатор PID. Ядро описывает его структурой `task_struct`: состояние, указатели на память (`mm_struct`), дескрипторы (`files_struct`), учёт времени, приоритет, маски сигналов. Поток (thread) в Linux — та же `task_struct`, но с общей памятью и общей таблицей дескрипторов: создание потока — это `clone()` с флагами `CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND`, а не отдельная сущность ядра.

Практическая разница сводится к четырём пунктам. Потоки одного процесса видят одну память и могут гоняться за одни данные — нужны блокировки. Падение потока по умолчанию валит весь процесс, поэтому «изоляция через потоки» не работает. Создание и переключение потоков дешевле, потому что не меняется адресное пространство. Процессы дают изоляцию отказов и отдельные лимиты, потоки — масштабирование внутри одного адресного пространства.

Свойство	Процесс	Поток
Адресное пространство	своё	общее с процессом
Дескрипторы файлов	своя таблица	общая
Стоимость создания (замер)	≈ 0.38 мс	≈ 0.11 мс
Изоляция отказа	полная	нет
Обмен данными	pipe, сокет, shared memory	обычные переменные

Цифры — замер на тестовой машине (Xeon E3-1230 v3, 8 логических ядер): 500 потоков создаются за 55.6 мс, 500 процессов — за 192.5 мс. Разница в четыре раза, а не в сто: `fork` в Linux не копирует память, а помечает страницы копированием при записи (copy-on-write, COW), поэтому остаётся дешёвым даже для больших процессов.

python

```
import os, threading, time

N = 500
t0 = time.perf_counter()
threads = [threading.Thread(target=lambda: None) for _ in range(N)]
for t in threads: t.start()
for t in threads: t.join()
print(f"потоки: {(time.perf_counter()-t0)*1e3:.1f} мс")

t0 = time.perf_counter()
pids = []
for _ in range(N):
    p = os.fork()
    if p == 0:
        os._exit(0)          # ребёнок не должен вернуться в цикл родителя
    pids.append(p)
for p in pids:
    os.waitpid(p, 0)          # обязательно, иначе останутся зомби
print(f"процессы: {(time.perf_counter()-t0)*1e3:.1f} мс")
```

Состояния процесса

Ядро ведёт процесс через набор состояний, видных в поле `State` файла `/proc/<pid>/status` и в колонке `STAT` вывода `ps`:

Код	Состояние	Смысл
`R`	running / runnable	выполняется или в очереди на процессор
`S`	interruptible sleep	ждёт события: `read()`, сокета, таймера
`D`	uninterruptible sleep	ждёт ввода-вывода в ядре, сигналы не доставляются
`T`	stopped	остановлен `SIGSTOP` или отладчиком
`Z`	zombie	завершился, но родитель не забрал код возврата

Состояние `D` — источник самой неприятной диагностики. Процесс в `D` не реагирует ни на `SIGTERM`, ни на `SIGKILL`: сигнал доставляется при возврате в пространство пользователя, а процесс сидит в ядре и ждёт ответа от зависшего NFS или диска с отказавшим контроллером. Убить его нельзя — можно починить причину или перезагрузиться. Если в `D` висят процессы, load average растёт, а CPU простаивает — ищите проблему в хранилище.

Контекстное переключение

Переключение контекста (context switch) — смена задачи на ядре: сохранить регистры и состояние текущего потока, выбрать следующего, восстановить его состояние. Прямые затраты — 1–3 мкс. Косвенные больше: новый поток приходит с «холодными» кэшами и TLB, и первые тысячи обращений идут в DRAM.

Переключения бывают добровольными (voluntary) и вынужденными. Добровольное происходит, когда поток сам уходит в сон: на `read()`, `recv()`, `sleep()`, на ожидании мьютекса. Вынужденное — когда планировщик отбирает процессор по истечении кванта. Оба счётчика есть в `/proc/<pid>/status`. Проверим, что блокирующий вызов стоит ровно одного добровольного переключения:

```python
import os, time

def field(pid, name):
    with open(f"/proc/{pid}/status") as f:
        for line in f:
```

```
            if line.startswith(name):
                return int(line.split(":")[1])

r, w = os.pipe()
pid = os.fork()
if pid == 0:
    os.close(w)
    for _ in range(2000):
        os.read(r, 1)        # блокируется до появления байта
    os._exit(0)
os.close(r); time.sleep(0.1)
before = field(pid, "voluntary_ctxt_switches")
for _ in range(2000):
    os.write(w, b"x")
    time.sleep(0.0005)       # даём читателю заснуть
time.sleep(0.2)
after = field(pid, "voluntary_ctxt_switches")
print("переключений:", after - before)
os.waitpid(pid, 0)
```

Вывод на тестовой машине: `переключений: 2000` — ровно одно на каждое блокирующее чтение. Отсюда правило: «дешёвый» по CPU код может быть дорогим по переключениям. Миллион мелких `read()` по одному байту — это миллион переключений и секунды времени, тогда как чтение буфером 64 КиБ — одно переключение.

Планировщик: CFS и EEVDF

До версии 6.6 Linux использовал полностью справедливый планировщик CFS (Completely Fair Scheduler). У каждой задачи есть виртуальное время выполнения `vruntime`, и планировщик выбирает задачу с наименьшим `vruntime`: задача, которая поспала, накапливает «долг» и получает процессор первой, что обеспечивает интерактивность. Вклад в `vruntime` обратно пропорционален весу: при `nice 0` вес 1024, при `nice -20` — 88761, при `nice 19` — 15. Отношение весов задаёт долю процессора.

С версии 6.6 (октябрь 2023) по умолчанию работает EEVDF (Earliest Eligible Virtual Deadline First). Идея та же, но вместо «кто дольше ждал, тот и прав» планировщик считает «лаг» (насколько задача отстала от идеальной доли) и виртуальный дедлайн среза; процессор получает задача с ближайшим дедлайном. Модель приоритетов не изменилась: `nice`, веса и cgroup работают как раньше.

Классы планирования: `SCHED_OTHER` (обычный), `SCHED_BATCH`, `SCHED_IDLE` (только когда процессор свободен), `SCHED_FIFO` и `SCHED_RR` для задач реального времени.

Приоритеты и nice

`nice` принимает значения от −20 (наибольший приоритет) до +19 (наименьший); понижать может любой пользователь, повышать — только root. Команды: `nice -n 10 ./batch.sh`, `renice -n 5 -p 1234`, `ionice -c 3` для дискового ввода-вывода. В контейнерах главный инструмент — cgroup: `cpu.weight` задаёт долю при конкуренции, `cpu.max` — жёсткий потолок (`quota period`, например `200000 100000` — два ядра). В Kubernetes это `resources.requests.cpu` (вес) и `resources.limits.cpu` (квота с троттлингом).

Разница принципиальна: вес — «доля при дележе», он ничего не ограничивает, когда процессор свободен; квота — «потолок», после которого контейнер тормозится до конца периода, даже если ядра простаивают. Отсюда типичный инцидент: сервис с `limits.cpu: 1` и четырьмя потоками троттлится при средней загрузке 0.5 ядра.

Системные вызовы

Системный вызов (system call) — единственный способ попросить ядро что-то сделать: открыть файл, прочитать сокет, выделить память, создать процесс. Переход стоит 100–300 нс на современном x86 — в сотни раз дороже вызова функции. Библиотеки сокращают число вызовов: `printf` буферизует, `gettimeofday` обслуживается через vDSO вообще без перехода в ядро.

Инструмент наблюдения — `strace`: он печатает имя вызова, аргументы и результат, а с `-c` даёт сводку по времени:

bash

```
strace -f -e trace=openat,read,write -o trace.log ./app
strace -c -f python3 -c "import socket; socket.create_connection(('1.1.1.1', 443))"
```

Первая команда пишет только файловые операции в лог, вторая даёт сводку по каждому вызову. Типичная картина для «тормозящего» сервиса: десятки тысяч `futex` — борьба за

мьютекс, а не работа; тысячи `openat` на запрос — шаблон читается с диска каждый раз. `strace` замедляет процесс в разы и в контейнерах часто запрещён seccomp, поэтому параллельно смотрят в `/proc`:

bash

```
cat /proc/1234/syscall     # номер syscall, аргументы, адрес возврата
cat /proc/1234/wchan       # где ядро держит задачу, например ep_poll
grep -E 'State|ctxt_switches' /proc/1234/status
cat /proc/1234/io          # read_bytes / write_bytes
```

fork и exec

`fork()` создаёт копию процесса: страницы помечаются copy-on-write, копирование происходит только при записи. Ребёнок получает копию адресного пространства, но не копию потоков — в новом процессе живёт только вызвавший поток, и это классический источник дедлоков, если после `fork` в ребёнке трогают мьютекс, захваченный другим потоком.

`execve()` заменяет образ процесса: загружает новый бинарник, сбрасывает память, сохраняет PID и дескрипторы (кроме помеченных `FD_CLOEXEC`). Пара `fork` + `exec` — стандартный способ запустить программу: между вызовами остаётся окно, в котором ребёнок настраивает окружение (`chdir`, `dup2` для перенаправлений, лимиты, смену пользователя). Так работают `sudo`, `systemd` и shell. В прикладном коде явный `fork` опасен: в многопоточном процессе состояние библиотек (аллокатор, логгер, DNS-резолвер) может быть несогласованным, поэтому используют `posix_spawn` или `subprocess`, где между созданием и `exec` нет пользовательского кода.

Зомби и сироты

Завершившийся процесс не исчезает сразу: ядро хранит его структуру с кодом возврата, пока родитель не вызовет `wait()`/`waitpid()`. В этом состоянии процесс называется зомби (zombie) и занимает только запись в таблице процессов.

python

```
import os, time

pid = os.fork()
if pid == 0:
```

```
    os._exit(0)              # ребёнок сразу завершается
time.sleep(0.2)              # родитель ещё не сделал wait
with open(f"/proc/{pid}/status") as f:
    print([l.strip() for l in f if l.startswith("State")][0])
os.waitpid(pid, 0)           # только теперь запись освобождается
```

Программа печатает `State: Z (zombie)`. Зомби не потребляет ни памяти, ни процессора, но занимает слот в таблице процессов. Если они накапливаются, исчерпывается `pid_max` (`/proc/sys/kernel/pid_max`, на тестовой машине 4 194 304) и новые процессы перестают создаваться. Убить зомби нельзя — он уже мёртв; лечится родитель, который обязан вызывать `waitpid` по сигналу `SIGCHLD`.

Сирота (orphan) — процесс, чей родитель умер раньше. Ядро передаёт его «усыновителю» (`prctl(PR_SET_CHILD_SUBREAPER)`) или процессу с PID 1. В контейнере PID 1 — это ваша команда: если она не собирает зомби, они копятся внутри контейнера, поэтому нужен `tini` или `--init`. Вторая проблема PID 1 — игнорирование сигналов по умолчанию: `SIGTERM` не убивает процесс без обработчика, и `docker stop` ждёт 10 секунд, после чего следует `SIGKILL`.

Лимиты

Лимиты задаются через `setrlimit` и наследуются потомками. Смотреть — `ulimit -a` или `/proc/<pid>/limits`, где видны мягкий (soft) и жёсткий (hard) пределы; мягкий можно поднять до жёсткого, жёсткий — только root. На тестовой машине мягкий лимит дескрипторов 4096, жёсткий 524288, лимит процессов 31282. Ключевые лимиты: `RLIMIT_NOFILE` (файлы и сокеты), `RLIMIT_NPROC` (процессы и потоки пользователя), `RLIMIT_AS` (виртуальная память), `RLIMIT_CORE`, `RLIMIT_STACK`. Классическая ошибка — сервис под systemd, которому не хватает дескрипторов:

bash

```
ulimit -n                                  # мягкий лимит оболочки: 4096
cat /proc/$(pgrep -f myservice)/limits | grep 'open files'
systemctl show myservice -p LimitNOFILE    # лимит, с которым процесс запущен
ls /proc/$(pgrep -f myservice)/fd | wc -l  # сколько открыто на самом деле
```

В systemd лимит задаётся в юните (`LimitNOFILE=65536`), в Docker — `--ulimit nofile=65536:65536`, в Kubernetes — через настройку

рантайма. Ошибка `Too many open files` означает именно лимит, но утечку дескрипторов всё равно надо исключить.

Почему 10 000 потоков — плохая идея

Поток дешевле процесса, но не бесплатен. Первое — стек: по умолчанию 8 МиБ виртуального адресного пространства на поток (`ulimit -s`), то есть 10 000 потоков — это 80 ГиБ виртуальной памяти. Второе — резидентная память и структуры ядра: в замере 500 потоков добавили 97 МиБ `VmSize` и 1.8 МиБ `VmRSS`, то есть около 3.5 КиБ реальной памяти на поток плюс запись в таблице задач и стек ядра (8–16 КиБ). Третье — планировщик: переключение между 10 000 потоков означает промахи кэша и TLB, и пропускная способность падает, хотя процессор «занят». Четвёртое — синхронизация превращается в борьбу за мьютексы.

Отсюда модель nginx, Redis, Node.js, asyncio и горутин Go: не поток на соединение, а событийный цикл с неблокирующими сокетами (`epoll` в Linux, `kqueue` в BSD). Горутина Go начинается с 2 КиБ и растёт по необходимости, поэтому 100 000 горутин — норма, а 100 000 потоков — авария. Правило: потоков — примерно по числу ядер плюс запас на блокирующие операции.

Типичные ошибки

1. **Не вызывать `wait` за детьми.** Каждый завершившийся ребёнок остаётся зомби, пока родитель не заберёт код возврата; при потоке задач таблица процессов забивается и `fork` падает с `EAGAIN`. Опрос «раз в секунду» лишь сужает окно. Правильно — обработчик `SIGCHLD` с `waitpid(-1, ..., WNOHANG)` в цикле.
2. **Запускать приложение как PID 1 в контейнере.** Оно не собирает зомби и часто игнорирует `SIGTERM`, поэтому `docker stop` всегда ждёт таймаут и завершается `SIGKILL`. Нужен `--init`/`tini` или явная обработка сигналов.
3. **Путать `requests.cpu` и `limits.cpu`.** Requests — вес при дележе, limits — жёсткий потолок с троттлингом на период. Сервис с лимитом в одно ядро и четырьмя рабочими потоками троттлится при средней загрузке 0.5.

4. **Считать `D`-состояние зависанием приложения.** Процесс ждёт ядро (диск, NFS) и не реагирует на сигналы; `kill -9` не поможет. Диагностика — `wchan`, `/proc/<pid>/stack`, состояние хранилища.
5. **Лечить `Too many open files` перезапуском.** Лимит — конфигурация, а не сбой: его поднимают в юните или в `--ulimit`, иначе проблема вернётся после деплоя. Заодно проверяют утечку по `/proc/<pid>/fd`.
6. **Масштабировать потоками «на всякий случай».** 10 000 потоков дают не параллелизм, а соревнование за кэш и планировщик.

Чек-лист

- У сервиса, запускающего дочерние процессы, есть обработчик `SIGCHLD` и цикл `waitpid`?
- PID 1 в контейнере собирает зомби и корректно обрабатывает `SIGTERM`?
- Лимит дескрипторов задан явно и соответствует числу ожидаемых соединений (`/proc/<pid>/limits`)?
- Для каждого деплоя понятно, что задано: доля CPU (`requests`/`cpu.weight`) или потолок (`limits`/`cpu.max`)?
- Число рабочих потоков сопоставлено с числом ядер, а ввод-вывод не блокирует их без нужды?
- При росте задержки проверяются `voluntary_ctxt_switches`, `wchan` и load average, а не только CPU?
- Буферизация не даёт делать системный вызов на каждый байт?

Упражнения

1. **Состояния.** Проведите программу через состояния `R`, `S`, `T`, `Z` и снимите для каждого `ps -o pid,stat,wchan`. Объясните, почему в `T` процесс не потребляет процессор, но и не завершается.
2. **nice и cgroup.** Запустите два одинаковых счётчика с разными `nice` и замерьте распределение процессорного времени. Повторите в двух контейнерах с разными `cpu.weight` и объясните,

почему результат не совпадает с nice.

3. **Переключения.** Замерьте добровольные переключения при чтении из сокета по одному байту и буфером 64 КиБ и объясните разницу во времени.
4. **Зомби и лимиты.** Доведите сервис до `EMFILE` в контейнере, поднимите лимит через `--ulimit` и через юнит systemd и объясните, почему `ulimit` в оболочке не влияет на уже запущенный процесс. Затем создайте 100 зомби и покажите, как их убирает обработчик `SIGCHLD`.

Ответы и разбор к этим упражнениям — в приложении D, часть 1.

Глава 5. Виртуальная память и файловые системы

Процесс не видит физическую память напрямую — он получает ровное адресное пространство, которого в железе не существует. Файл на диске — тоже не «имя с содержимым», а inode плюс запись в каталоге. Эти две абстракции объясняют большинство загадок эксплуатации: почему контейнер убит с кодом 137, почему `df` показывает 100%, а `du` — 40%, почему `write()` вернул успех, а после отключения питания файла нет. Ниже — страничная адресация, page fault, swap и OOM-killer, затем inode, журнал ext4 и fsync, и в конце — практика с реальными командами.

Адресное пространство

На x86-64 с четырёхуровневой таблицей страниц пользовательскому коду доступно 128 TiB — адреса от нуля до `0x00007fff_ffff_ffff`; верхняя половина отдана ядру и отображена во всех процессах одинаково, но недоступна из user space. Отсюда два следствия: адрес `0x55d4c2e0d000` в двух процессах указывает на разные физические ячейки, а «память процесса» и «память машины» — разные величины.

Ядро описывает занятые диапазоны структурами VMA (virtual memory areas): отрезок адресов плюс права и то, что под ними лежит.

bash

```
cat /proc/self/maps
```

```
55d4c2e0d000-55d4c2e2e000 r--p 00000000 08:11 1310742   /usr/bin/cat
55d4c2e2e000-55d4c2e52000 r-xp 00021000 08:11 1310742   /usr/bin/cat
55d4c2e6b000-55d4c2e6c000 rw-p 00000000 00:00 0         [heap]
7ffd8b3f6000-7ffd8b418000 rw-p 00000000 00:00 0         [stack]
7ffd8b4a1000-7ffd8b4a5000 r--p 00000000 00:00 0         [vvar]
```

Поля слева направо: начало и конец, права (`p` — private, `s` — shared), смещение в файле, устройство, inode, путь. Код отображён как `r-xp`, стек и куча — анонимные (`00:00 0`). Этот файл

смотрят, когда процесс «съел гигабайт»: анонимная память — это диапазоны `rw-p` без пути.

Страничная адресация, MMU и TLB

Память делится на страницы по 4 KiB. Виртуальный адрес режется на две части: младшие 12 бит — смещение внутри страницы ($4096 = 2^{12}$), старшие — номер виртуальной страницы. Переводит его в физический номер аппаратный блок MMU (memory management unit) по таблице страниц, которую строит ядро.

На x86-64 таблица четырёхуровневая: 48 бит адреса — это четыре индекса по 9 бит плюс 12 бит смещения. Каждый уровень — 512 записей по 8 байт, ровно страница 4 KiB. Запись PTE (page table entry) содержит адрес следующей таблицы или страницы и флаги: `P` (present), `RW`, `US` (доступно из user space), `NX` (исполнение запрещено), `A/D` (были обращения / изменения — на них завязаны swap и NUMA).

Разберём адрес вручную — лучший способ убедиться, что магии нет:

python

```
addr = 0x7ffd8b3f6120
print("offset ", hex(addr & 0xFFF))
print("индексы", [hex((addr >> (12 + 9 * i)) & 0x1FF) for i in range(4)])
```

```
offset  0x120
индексы ['0x1f6', '0x59', '0x1f6', '0xff']
```

Читается справа налево: `0xff` — запись в корневой таблице, дальше уровни вниз, `0x120` — смещение внутри страницы. Четыре обращения к памяти на каждый доступ слишком дорого, поэтому рядом с MMU стоит TLB (translation lookaside buffer) — кэш на несколько сотен готовых переводов. Попадание стоит единицы тактов, промах — обход таблиц, десятки тактов, а если таблиц нет в кэше процессора — сотни.

Отсюда практика: приложение, прыгающее по большому массиву, страдает и от промахов TLB. Лечится huge pages — страницами по 2 MiB, покрывающими тот же объём одной записью:

bash

```
grep -i huge /proc/meminfo
```

```
AnonHugePages:     14336 kB
Hugepagesize:       2048 kB
```

Ненулевой `AnonHugePages` означает, что ядро уже склеивает анонимную память (THP, transparent huge pages). У СУБД с собственной работой с памятью THP иногда отключают: фоновый `khugepaged` тратит CPU, а задержки на выделение растут.

Page fault: minor, major и copy-on-write

Страницы выделяются лениво: `malloc` большого блока превращается в `mmap`, ядро создаёт VMA и ничего не выделяет. Первое обращение вызывает исключение — page fault, — и ядро решает:

- **minor fault** — страница уже в памяти (page cache или свободный фрейм), надо лишь прописать PTE;
- **major fault** — данных нет, нужен ввод-вывод: дочитать с диска или из swap;
- **SIGSEGV** — адреса нет в VMA или идёт запись в страницу только для чтения; это уже падение.

Классический пример ленивости — `fork()`. Копировать гигабайты при каждом форке безумно, поэтому ядро помечает страницы обоих процессов read-only и при записи копирует одну страницу: copy-on-write. Так `fork()` в nginx или gunicorn стоит микросекунды, пока потомок не начнёт писать.

Измерить это можно без профилировщика:

bash

```
/usr/bin/time -v python3 -c "import numpy as np; a=np.zeros(10**8); a.sum()"
```

```
    Maximum resident set size (kbytes): 812344
    Minor (reclaiming a frame) page faults: 198432
    Major (requiring I/O) page faults: 0
```

`Maximum resident set size` — пик RSS, реально занятые физические страницы. Сотни тысяч minor fault — норма для выделения 800 МБ: ядро отдаёт заранее обнулённые страницы. Major fault в

тысячах означает старт с холодного кэша — первый запуск после деплоя всегда медленнее второго.

Swap и OOM-killer

Swap — место на диске, куда ядро выгружает давно не используемые страницы. Ядро не «переносит память при заполнении», а держит баланс между анонимными страницами и page cache; агрессивность задаёт `vm.swappiness` (по умолчанию 60, для серверов с БД ставят 1–10).

```
               total        used        free      shared  buff/cache   available
Mem:            7878        4817         148          12        2913        2781
Swap:          10172        7231        2941
```

Смотреть надо на `available`: `buff/cache` — page cache, который отдаётся приложениям мгновенно. `free` в 148 МБ при `available` 2.7 ГБ — нормальное состояние здоровой машины.

Когда кончился и swap, вступает OOM-killer: выбирает процесс с наибольшим `oom_score` и убивает его через SIGKILL, который нельзя перехватить.

```
[41231.882014] python3 invoked oom-killer: gfp_mask=0x140cca(GFP_HIGHUSER_MOVABLE),
    order=0, oom_score_adj=0
[41231.882102] CPU: 3 PID: 48211 Comm: python3 Not tainted 6.12.0-15-generic #1
[41231.882183] active_anon:412304 inactive_anon:98122 active_file:8120
    inactive_file:15231
[41231.883001] oom-kill:constraint=CONSTRAINT_NONE,cpuset=/,mems_allowed=0,
    global_oom,
             task_memcg=/system.slice/app.service,task=python3,pid=48211,uid=1000
[41231.883044] Out of memory: Killed process 48211 (python3) total-vm:8123456kB,
    anon-rss:7890120kB,
             file-rss:0kB, shmem-rss:0kB, UID:1000 pgtables:15872kB oom_score_adj:0
```

Ключевые поля: `task_memcg=/system.slice/app.service` — cgroup, в которой жил процесс (именно по ней в Kubernetes выставляется OOMKilled: контейнер убивает контроллер памяти cgroup v2 при исчерпании `memory.max`, а не глобальный OOM); `anon-rss:7890120kB` — почти 8 ГБ анонимной памяти, типичная утечка; `total-vm` в разы больше RSS и о реальном потреблении ничего не говорит.

Снять критичный процесс с первых позиций очереди можно через `oom_score_adj` (от −1000 до 1000). Важно взять ровно один PID: `pgrep -f postgres` на живой базе вернёт и мастер, и воркеров, а путь вида `/proc/123\n456/oom_score_adj` не существует.

bash

```
pid=$(pgrep -o -f postgres)     # -o: только самый старый процесс
echo -900 | sudo tee "/proc/$pid/oom_score_adj"
```

Это не бронирует память — при тотальной нехватке кого-то всё равно убьют, — но защищает от «первого кандидата».

mmap: файл как часть адресного пространства

`mmap` отображает файл в адресное пространство: чтение и запись идут через обычные обращения к памяти, ядро подгружает страницы по требованию. Так работают динамические библиотеки и почти все СУБД.

python

```
import mmap

with open("index.bin", "r+b") as f:
    mm = mmap.mmap(f.fileno(), 0)      # 0 — весь файл
    print(mm[0:16])                    # чтение без read()
    mm[0:4] = b"IDX2"                  # запись в page cache
    mm.flush()                         # msync
    mm.close()
```

`flush()` выталкивает грязные страницы, но не гарантирует подтверждения от диска — для durability нужен `os.fsync(f.fileno())`. При `MAP_SHARED` изменения видны другим отображениям, при `MAP_PRIVATE` копируются при записи, и файл не меняется.

inode

Имя файла — запись в каталоге, ссылающаяся на inode. В inode лежит всё, кроме имени: тип, права, владелец, размер, временные метки, счётчик ссылок, указатели на блоки (в ext4 — extents).

bash

```
stat /etc/hosts
```

```
  File: /etc/hosts
  Size: 221         Blocks: 8          IO Block: 4096   regular file
Device: 8,17    Inode: 1310742     Links: 1
Access: (0644/-rw-r--r--)  Uid: (0/root)   Gid: (0/root)
Modify: 2026-10-20 18:02:44.000000000 +0500
Change: 2026-10-20 18:02:44.000000000 +0500
```

Три времени различаются: `Modify` — содержимое, `Change` — сам inode (права, владелец, переименование), `Access` — чтение (с `relatime` не на каждое). `Links: 1` — жёстких ссылок одна: `ln file hard` создаёт второе имя для того же inode, и данные исчезнут только при `Links: 0`; `ln -s` создаёт отдельный файл с путём.

Inode-ы расходуются отдельно от места:

bash

```
df -i /mnt/data
```

```
Filesystem      Inodes  IUsed    IFree IUse% Mounted on
/dev/sda1     61022208 412330 60609878    1% /mnt/data
```

Отсюда классика: «свободно 40 ГБ, а `touch` пишет `No space left on device`» — кончились inode на миллионе мелких файлов (сессии, кэш, `node_modules` в образе).

ext4: журнал и его гарантии

ext4 журналируемая: перед изменением метаданных она пишет намерение в журнал, затем применяет изменение, затем помечает транзакцию завершённой. Если питание пропадёт посередине, при монтировании журнал «докрутится», и структура останется согласованной — возможно, с потерей последних данных, но без разрушенных каталогов.

Режим	Что в журнале	Скорость	Риск при сбое
`data=journal`	данные и метаданные	медленно	минимальный
`data=ordered` (умолчание)	метаданные, данные — до коммита	средне	старые данные, не мусор
`data=writeback`	только метаданные	быстро	в файле могут быть чужие блоки

bash

```
findmnt -no SOURCE,FSTYPE,OPTIONS /
tune2fs -l /dev/sdb1 | grep -E 'Filesystem features|Block size|Journal size'
```

```
/dev/sdb1 ext4 rw,relatime
Filesystem features:  has_journal ext_attr resize_inode dir_index filetype needs_recovery extent
    64bit
Block size:           4096
Journal size:         1024M
```

`needs_recovery` означает, что журнал не чист и будет проигран при монтировании — нормальное состояние после жёсткого выключения.

fsync: почему «запись прошла» ничего не значит

`write()` не пишет на диск. Он копирует данные в page cache и возвращает управление. Долговечными (durable) данные становятся, когда ядро отправит их на устройство и устройство подтвердит запись. Между этими моментами — грязные страницы, которые сбрасываются по таймеру (у ext4 `commit=5` секунд) или при нехватке памяти. Потеря питания в этом окне — потеря записи, причём приложение уже получило «успех».

`fsync(fd)` сбрасывает данные и метаданные и ждёт подтверждения; `fdatasync(fd)` — без метаданных, не нужных для чтения. Корректная запись файла:

```python
import json, os, tempfile

def atomic_write(path: str, payload: dict) -> None:
    d = os.path.dirname(path) or "."
    fd, tmp = tempfile.mkstemp(dir=d, prefix=".tmp-")
    try:
        with os.fdopen(fd, "w") as f:
            json.dump(payload, f)
            f.flush()
            os.fsync(f.fileno())     # 1. содержимое временного файла на диске
        os.replace(tmp, path)        # 2. атомарная подмена имени
        dfd = os.open(d, os.O_DIRECTORY)
        try:
            os.fsync(dfd)            # 3. запись каталога на диске
        finally:
            os.close(dfd)
    except BaseException:
        if os.path.exists(tmp):
            os.unlink(tmp)
        raise
```

Все три шага обязательны. `rename` атомарен: читатель видит либо старый файл, либо новый, но не половину. Но сам `rename` — изменение каталога, и без `fsync` на его дескрипторе после сбоя имя может откатиться, а данные остаться. Это и есть разница между «файл не побьётся» и «файл точно есть».

Цена durability — задержка: `fsync` на SSD — от десятых долей до нескольких миллисекунд, на сетевой ФС — десятки. Поэтому Postgres с `synchronous_commit=on` делает порядка тысячи транзакций в секунду, а не сто тысяч.

df vs du

`df` спрашивает у файловой системы, сколько блоков занято; `du` обходит дерево и складывает размеры файлов. Расхождение дают три механизма.

Удалённый, но открытый файл. Дескриптор жив, inode жив, блоки не освобождены, имени в дереве нет:

bash

```
sudo lsof +L1 | awk 'NR==1 || $7 > 0'
```

```
COMMAND  PID     USER   FD   TYPE DEVICE  SIZE/OFF NLINK     NODE NAME
nginx   2210 www-data    4w   REG   8,17 10737418240     0  1843321 /var/log/nginx/access.log (deleted)
```

`NLINK 0` и `(deleted)` — приметы. Лечится перезапуском процесса или `: > /proc/2210/fd/4`.

Разреженные файлы. Логический размер 10 ГБ, физически — единицы блоков:

bash

```
truncate -s 10G sparse.img
du -h sparse.img; du -h --apparent-size sparse.img
```

```
0   sparse.img
10G sparse.img
```

Зарезервированные блоки. ext4 держит 5% под root, поэтому `Avail` меньше, чем `Size − Used`; меняется `tune2fs -m 1 /dev/sdb1`.

Типичные ошибки

1. **Смотреть на `free` вместо `available`.** Ядро держит в page cache всё прочитанное и отдаёт при первой нужде. 148 МБ «свободной» памяти на здоровом сервере — норма.
2. **Ставить `vm.swappiness=0`, «чтобы не было swap».** При нуле ядро всё равно вытесняет анонимные страницы под давлением, но перестаёт освобождать память заранее — OOM приходит раньше и резче. Разумный диапазон 1–10.

3. **Считать `write()` гарантией сохранности.** Возврат означает лишь копирование в page cache; без `fsync` сбой питания забирает последние секунды записей.
4. **Писать «в тот же файл» без rename.** Запись не атомарна — читатель увидит обрезанный файл. Схема «временный + fsync + rename» обязательна.
5. **Верить, что `du` показывает занятое место.** Удалённые открытые файлы и разреженность в него не попадают.
6. **Swap-файл на btrfs без `nodatacow`.** CoW ломает требование фиксированных блоков, ядро откажется его использовать.

Чек-лист

- [ ] `free -m`: смотрим `available`, тревога — стабильно ниже 10% от `total`.
- [ ] Для каждого сервиса известны `oom_score_adj` и лимит cgroup `memory.max`.
- [ ] Важные файлы пишутся через `fsync` + `rename` + `fsync` каталога.
- [ ] Есть алерты на `df -h` и `df -i` выше 85%.
- [ ] Известно, что покажет `lsof +L1` на машине с «исчезнувшим» местом.
- [ ] После каждого OOM разбирается `dmesg`, а не только `kubectl describe`.
- [ ] Для БД проверено отношение к THP и `vm.swappiness`.

Упражнения

1. В `cat /proc/self/maps` найдите код, кучу и стек. Объясните, почему у стека `rw-p`, а у кода `r-xp`.
2. Сравните `du -h`, `du -h --apparent-size` и `ls -l` для обычного и разреженного файла на 500 МБ.
3. Напишите два скрипта — запись 10 000 строк без `fsync` и с ним. Замерьте время и объясните разницу в порядке величины.
4. Воспроизведите OOM: процесс, добавляющий по 10 МБ в список, при `memory.max=512M`. Разберите строку `oom-kill` в `dmesg`.

5. Откройте файл, удалите его и освободите место, не перезапуская процесс.

Ответы и разбор к этим упражнениям — в приложении D, часть 1.

Глава 5a. Linux: пользователи, права, sudo

Права доступа — та часть Linux, которую инженер трогает каждый день и чаще всего понимает наполовину. Симптомы одинаковы годами: «nginx отдаёт 403», «сервис не может писать в лог», «после `chmod -R 777` заработало, а через месяц нас взломали», «sudo просит пароль root, а мы его не знаем». Все они родом из одной модели: у процесса есть uid и gid, у файла — девять бит прав и владелец, и ядро при каждом `open()` сверяет одно с другим. Ниже эта модель собирается целиком: от `/etc/passwd` до capabilities, с разбором `ls -l` по колонкам.

uid, gid и /etc/passwd

Для ядра субъект — это число: идентификатор пользователя (uid) и группы (gid). Имена существуют для людей и лежат в текстовой базе `/etc/passwd`, которую читают все, а пишут через `vipw` и `useradd` (параллельная правка двумя редакторами портит файл).

```
nextdoor:x:1000:1000:Next Door,,,:/home/nextdoor:/bin/bash
```

Семь полей: имя, место пароля (`x` — «смотри в `/etc/shadow`»), uid, gid основной группы, комментарий, домашний каталог, shell входа. Диапазоны заданы в `/etc/login.defs`: 0 — root, 1–99 — статические системные, 100–999 — системные, создаваемые пакетами, от 1000 — люди. Поэтому `www-data` с uid 33 — системная учётка без shell (`/usr/sbin/nologin`) и без пароля: под ней не логинятся, под ней работает сервис.

Группы — в `/etc/group`, членство в последнем поле: `adm:x:4:syslog,nextdoor`.

У процесса ровно один основной gid и произвольный набор дополнительных; проверка прав идёт по всем, но только по тем, что были на момент запуска. Добавили себя в `docker` через `usermod -aG docker $USER` и не перелогинились — текущая оболочка всё ещё без группы. Смотреть фактический набор — `id`, обновлять — новым входом.

/etc/shadow и хеши паролей

`/etc/passwd` читаем всем, значит хешу там не место. Пароли уехали в `/etc/shadow` с режимом `640 root:shadow`: группа нужна ровно для того, чтобы `passwd` мог писать, а обычный пользователь — нет.

```
root:$y$j9T$Fv3s...:20345:0:99999:7:::
```

Поля: имя, хеш, дата последней смены (дни с 1 января 1970), минимум и максимум дней жизни пароля, сколько предупреждать, сколько дней после истечения вход ещё разрешён, дата блокировки, резерв. `!` или `*` вместо хеша — вход запрещён (так выглядят системные учётки), `!!` — учётка заблокирована.

Хеш — односторонняя функция с солью, не шифрование. Префикс говорит об алгоритме: `$y$` — yescrypt (по умолчанию в Debian и Ubuntu последних лет), `$6$` — SHA-512 (долгий стандарт, в том числе в семействе RHEL), `$2b$` — bcrypt, `$1$` — MD5, взломанный и недопустимый. Соль у каждого своя, поэтому одинаковые пароли дают разные хеши, а радужные таблицы бесполезны. Стоимость подбирают так, чтобы проверка одного пароля занимала десятки миллисекунд: незаметно при входе и смертельно при переборе. Отсюда правило: `$1$` в системе — инцидент, а не мелочь.

Права rwx и восьмеричная запись

У файла три набора по три бита: владелец (user), группа (group), остальные (other). Для каталога смысл другой, и это главный источник путаницы:

Бит	Для файла	Для каталога
`r`	прочитать содержимое	список имён (`ls`)
`w`	изменить содержимое	создавать, удалять, переименовывать
`x`	запустить как программу	войти и обращаться к файлам внутри

Каталог с `r` без `x` — `ls` работает, `cat file` внутри даёт «Permission denied». Поэтому `/var/www` делают `755`.

Восьмеричная запись — упаковка трёх бит в цифру: `r`=4, `w`=2, `x`=1. Отсюда `7`=rwx, `6`=rw-, `5`=r-x, `4`=r--, `0`=---.

```
644  rw- r-- r--   владелец пишет, остальные читают: конфиг
755  rwx r-x r-x   владелец пишет, все читают и входят: каталоги, скрипты
600  rw- --- ---   только владелец: приватный ключ, .env, база SQLite
640  rw- r-----     владелец пишет, группа читает: лог с чувствительными данными
777  rwx rwx rwx   все могут всё: почти всегда ошибка
```

`777` — зло не из-за эстетики. `w` на каталоге означает право удалять и переименовывать чужие файлы, `w` на файле — право подменить содержимое: конфиг, скрипт, шаблон. Любой локальный пользователь и любой процесс под сервисной учёткой (а взламывают первыми именно веб-приложения и парсеры) получает запись в ваш конфиг. Многие демоны ещё и отказываются работать с world-writable файлами: ssh молча игнорирует слишком открытый `authorized_keys`.

Разбор `ls -l` по колонкам

```
-rwsr-xr-x 1 root root  118168 Apr 19  2025 /usr/bin/passwd
drwxrwxrwt 13 root root     900 Oct 29 19:54 /tmp
-rw-r----- 1 root adm  1048576 Oct 28 03:12 /var/log/syslog
lrwxrwxrwx 1 root root      27 Sep  1  2025 /etc/localtime -> /usr/share/zoneinfo/Europe/Moscow
```

Первая колонка — тип и права. Первый символ: `-` файл, `d` каталог, `l` ссылка, `b`/`c` устройства, `s` сокет, `p` FIFO. Дальше девять бит; `s` на месте `x` — setuid (в групповой тройке setgid), `t` — sticky, `+` после битов — есть ACL, `.` — контекст SELinux.

Вторая колонка — число жёстких ссылок (для каталога — число подкаталогов плюс два). Третья и четвёртая — владелец и группа. Пятая — размер в байтах; для каталога это размер его inode, а не сумма файлов. Шестая — время модификации: если вместо года стоят часы и минуты, файл изменён менее полугода назад (поведение GNU coreutils). Последняя — имя, для ссылки ещё и цель.

umask: маска, которая вычитает права

Программа при создании файла просит режим (обычно `0666` для файла и `0777` для каталога), ядро вычитает из него маску процесса. Файл никогда не получает `x` из umask — только явным `chmod`.

umask	Файлы (из 666)	Каталоги (из 777)	Когда уместна
022	644	755	дефолт системных файлов
002	664	775	групповая работа
027	640	750	серверы: чужие не читают
077	600	700	приватные данные

`umask` печатает текущую маску (`0002` в Debian для людей, `0022` для root), `umask 027` меняет её для текущей оболочки. Системно — `UMASK` в `/etc/login.defs` и `UMask=` в systemd-юните (глава 5b): сервис наследует маску от демона, а не от вашей оболочки, и это объясняет половину «файлы создаются 644, хотя я поставил 600».

chmod, chown, chgrp

bash

```
chmod 640 /etc/app/config.yml          # восьмерично: rw-r-----
chmod u+x deploy.sh                    # символьно: добавить x владельцу
chmod g-w,o-rwx secrets.env            # снять w у группы, у остальных всё
chmod -R g+rX /srv/www                 # X: x только каталогам и уже исполняемому
chmod 2750 /srv/shared                 # setgid на каталог
chown app:app /srv/app                 # владелец и группа
chgrp adm /var/log/app.log
```

`X` (заглавная) ставит `x` каталогам и тем файлам, где `x` уже у кого-то есть, и не делает исполняемыми данные. Поэтому `chmod -R 755` на дерево с картинками вредно, а `chmod -R a+rX` — почти всегда то, что имелось в виду.

Смена владельца требует root или `CAP_CHOWN`: отдать свой файл другому нельзя, иначе можно подсунуть root'у setuid-бинарник. Смена группы разрешена, если вы член целевой группы.

Специальные биты: setuid, setgid, sticky

setuid (`4000`, в `ls` — `s` на месте `x` владельца) даёт процессу euid владельца файла. Классика — `/usr/bin/passwd`: запускается обычным пользователем, но пишет в `/etc/shadow`, потому что владелец — root. Это же делает setuid главным подарком атакующему: setuid-бинарник с ошибкой = root. На скриптах

ядро setuid игнорирует, а список в системе стоит знать: `find / -perm -4000 -type f`.

setgid (`2000`) на файле даёт egid владельца группы; на каталоге — новые файлы и подкаталоги наследуют группу каталога. Стандартный способ сделать общий каталог: `chgrp dev /srv/shared`, `chmod 2775` — всё внутри автоматически принадлежит `dev`, и бегать с `chgrp` не нужно.

sticky (`1000`, `t`) на каталоге с `w` для всех запрещает удалять и переименовывать чужие файлы — это может владелец файла, владелец каталога и root. Ровно поэтому `/tmp` имеет режим `1777`:

```
drwxrwxrwt 13 root root 900 Oct 29 19:54 /tmp
```

Без sticky в мире, где `/tmp` открыт на запись всем, любой процесс мог бы удалить чужой сокет или подменить чужой временный файл. Тот же приём — на общих каталогах загрузок и в спуле печати.

ACL: права для тех, кто не владелец и не в группе

Девяти бит не хватает, когда нужно дать чтение одному человеку, не добавляя его в группу. Для этого — списки контроля доступа (ACL), в Debian пакет `acl`.

bash

```
setfacl -m u:deploy:rx /srv/app            # пользователю deploy — чтение и вход
setfacl -m g:qa:r /srv/app/report.csv      # группе qa — только чтение
setfacl -m d:u:deploy:rx /srv/app          # default ACL: наследуется новыми файлами
getfacl /srv/app
```

```
# file: srv/app
# owner: app
# group: app
user::rwx
user:deploy:r-x
group::r-x
mask::r-x
other::---
```

Строка `mask` — верхняя граница прав для именованных записей и группы-владельца; `setfacl` пересчитывает её сам, но при ручной правке легко получить «права есть, а доступа нет» — применяются именно эффективные права из `effective:`. В `ls -l` ACL

отмечается плюсом: `drwxr-x---+`. Default-ACL работает только на каталоги и только для новых объектов.

sudo и /etc/sudoers.d

`su` переключает пользователя и требует пароль целевой учётки — для `su -` это пароль root. `sudo` проверяет пароль вызывающего и сверяет его с политикой: кто, от кого, какие команды, нужен ли пароль. Root-пароль не нужен вообще — поэтому в Ubuntu учётка root заблокирована, а первый пользователь входит в группу `sudo`.

Политика — `/etc/sudoers` плюс подключаемые файлы; правят только через `visudo`, потому что синтаксическая ошибка означает потерю доступа к root.

bash

```
visudo -f /etc/sudoers.d/deploy

# файл должен иметь режим 0440 и принадлежать root:root
deploy ALL=(root) NOPASSWD: /usr/bin/systemctl restart app.service
%devops ALL=(ALL) ALL
Defaults:%devops timestamp_timeout=15
```

Формат: `кто где=(от кого) что`; первое поле — пользователь или `%группа`, `ALL` — любые хосты. Голое `ALL` в последнем поле означает и `sudo vi /etc/sudoers`, то есть полный root. Опасны и «безобидные» записи: `NOPASSWD: /usr/bin/systemctl` без юнита даёт root через `systemctl edit`; `journalctl`, `less`, `find`, `awk`, `tar` дают shell или чтение-запись файлов.

`NOPASSWD` снимает вторую аутентификацию: любой процесс под этим пользователем (взломанное приложение, cron-скрипт, чужой код в CI) получает привилегии без секрета. В контейнерах `NOPASSWD: ALL` — обычная практика, и её цена в том, что граница «root» внутри образа стирается.

Помнить: `sudo` по умолчанию сбрасывает окружение (`env_reset`) и подставляет `secure_path`, поэтому `PATH` из вашей оболочки не доезжает до root-команды — это защита, а не баг. Каждый вызов попадает в журнал через PAM/journald, и «кто, когда, какую команду» — главный аргумент против общей учётки root.

Capabilities: привилегии по частям

Root — не «всё», а набор из нескольких десятков возможностей, которые ядро проверяет по отдельности: `CAP_NET_BIND_SERVICE` (порты ниже 1024), `CAP_NET_ADMIN` (сеть), `CAP_SYS_ADMIN` (монтирование), `CAP_CHOWN`. Можно выдать одну.

Именно поэтому nginx от root слушает 80-й порт, а потом сбрасывает привилегии: мастер держит сокет, рабочие процессы работают под `www-data`. Альтернатива без root:

bash

```
sudo setcap cap_net_bind_service=+ep /usr/sbin/nginx
getcap /usr/sbin/nginx
```

```
/usr/sbin/nginx cap_net_bind_service=ep
```

`e` — effective (действует сразу), `p` — permitted (можно включить). Цена: capability привязана к файлу, а не к пользователю; кто может заменить бинарник, наследует право; обновление пакета сбрасывает `setcap`. В systemd то же описывается декларативно: `AmbientCapabilities=`, `CapabilityBoundingSet=`. Третий путь — `sysctl net.ipv4.ip_unprivileged_port_start=0`: Docker именно так разрешает контейнерам любые порты.

Практика: «сервис не может писать в /var/log»

Приложение работает под `app`, пишет в `/var/log/app/app.log`, и после переустановки в журнале появляется:

```
app[2417]: PermissionError: [Errno 13] Permission denied: '/var/log/app/app.log'
```

Диагностика — три команды, каждая отвечает на свой вопрос.

bash

```
id app
ls -ld /var/log/app /var/log/app/app.log
namei -l /var/log/app/app.log
```

```
uid=995(app) gid=995(app) groups=995(app)
drwxr-x--- 2 root root 4096 Oct 28 10:02 /var/log/app
-rw-r----- 1 root root  812 Oct 28 10:02 /var/log/app/app.log
```

Читается так: каталог `root:root 750` — `app` не в группе `root` и даже войти в каталог (`x`) не может; файл тоже чужой. Приложение не «сломало права», оно их никогда не имело: каталог создали руками от root, а пакет и юнит ожидали `app:adm 0750`.

Лечение — не `chmod 777`, а явное владение:

bash

```
install -d -o app -g adm -m 0750 /var/log/app
touch /var/log/app/app.log
chown app:adm /var/log/app/app.log && chmod 0640 /var/log/app/app.log
```

`install -d` создаёт каталог с нужными владельцем и режимом за один шаг. В systemd то же делает `LogsDirectory=app` с `LogsDirectoryMode=0750`: демон сам создаст каталог с владельцем из `User=`/`Group=`. Если же включён `ProtectSystem=strict`, запись запрещена вообще, пока каталог не добавлен в `ReadWritePaths=`, и текст ошибки будет `Read-only file system`, а не `Permission denied` — это разные поломки.

Проверять доступ нужно от имени сервиса:

bash

```
sudo -u app sh -c 'echo probe >> /var/log/app/app.log' && tail -1
    /var/log/app/app.log
```

```
probe
```

`test -w` от вашей учётки ничего не доказывает: uid другой.

Типичные ошибки

1. `chmod -R 777` **как универсальное средство.** Права «починились», потому что запись получили все, включая скомпрометированный сервис; причина — неверный владелец, неверная группа или отсутствующий `x` у каталога — осталась.
2. **Путаница** `r` **и** `x` **на каталоге.** Файл читается, а `cd` не работает. Механизм: путь резолвится по компонентам, и каждому нужен `x`; `r` нужен только для листинга.
3. **Добавили в группу и не перелогинились.** `id` в новой сессии группу показывает, а старый процесс — нет. Механизм: набор дополнительных групп фиксируется при входе.
4. `umask` **из оболочки не действует на сервис.** Механизм: маска наследуется от родителя, у systemd-сервиса это демон; нужен `UMask=` в юните.
5. `sudo echo ... > /etc/file` **не работает.** Механизм: редирект открывает файл ваша оболочка, а не `sudo`. Лечится `sudo tee` или `sudo sh -c '...'`.

6. **Оставили setuid на самописном бинарнике.** Любая ошибка в нём — локальное повышение привилегий, а обновление пакета может вернуть бит: euid root получает весь процесс, включая разбор входных данных.

Чек-лист

- `ls -l` разбирается по колонкам: тип, девять бит, s/t/+/., ссылки, владелец, группа, размер, время.
- Для любого «Permission denied» проверены `id` процесса, `ls -ld` на каждый компонент пути и `namei -l`.
- Нет world-writable файлов вне `/tmp`-подобных каталогов: `find / -xdev -perm -0002 -type f`.
- Список setuid известен и объяснён: `find / -xdev -perm -4000 -type f`.
- В `/etc/sudoers.d` нет лишних `ALL` и `NOPASSWD` на интерпретаторы; файлы `0440 root:root`.
- Приватные ключи и `.env` — `600`, каталоги `700`.
- Сервисные каталоги создаются декларативно (`install -d`, `LogsDirectory=`), а доступ проверен от имени сервиса.

Упражнения

1. Создайте `/srv/shared` так, чтобы файлы внутри всегда принадлежали группе `dev`, любой из группы мог их править, а удалять чужие не мог никто, кроме владельца. Объясните, какие три бита за это отвечают.
2. Найдите все setuid-файлы и для трёх объясните, зачем бит нужен и что сломается, если его снять.
3. Настройте ACL: `deploy` читает `/srv/app/config.yml`, но не видит остальных файлов каталога. Объясните строку `mask` в `getfacl`.
4. Разрешите nginx слушать 80 без root двумя способами (capabilities и `ip_unprivileged_port_start`) и сравните, что сломается при обновлении пакета и при переносе на другой хост.

5. Дайте `ci` право без пароля перезапускать ровно один юнит и докажите, что через эту запись нельзя получить root.

Ответы и разбор к этим упражнениям — в приложении D, часть 1.

Глава 5b. Linux: пакеты, systemd, журналы

Сервер живёт не сам по себе: сотня пакетов, десятки юнитов, гигабайты логов. Три навыка покрывают почти всю работу с этим хозяйством — откуда взялся файл и какая у него версия; как описать сервис, чтобы он перезапускался сам и не висел под root; как найти в журнале момент, когда всё сломалось. Ниже — apt и dpkg, unit-файлы systemd целиком, journald и logrotate, и рабочий юнит с таймером для ночного бэкапа. Примеры на Debian 13 (systemd 257); отличия RHEL отмечены, где меняют практику.

Пакеты: репозитории, приоритеты, зависимости

Пакет — архив с файлами плюс метаданные: имя, версия, зависимости, контрольные суммы, скрипты установки. Apt держит локальную базу (`/var/lib/dpkg`) и индексы репозиториев из `/etc/apt/sources.list` и `/etc/apt/sources.list.d/`; в Debian 13 и Ubuntu 24.04 преобладает формат deb822 — файл `*.sources` с полями `Types`, `URIs`, `Suites`, `Components`.

bash

```
apt-cache policy nginx
```

```
nginx:
  Installed: (none)
  Candidate: 1.26.3-3+deb13u10
  Version table:
     1.26.3-3+deb13u10 500
        500 http://security.debian.org/debian-security trixie-security/main amd64
            Packages
     1.26.3-3+deb13u7 500
        500 http://deb.debian.org/debian trixie/main amd64 Packages
```

Число после версии — приоритет (pin priority). `500` — обычный репозиторий, `100` — установленный пакет или архив без явной политики, `990` — обычно backports, `1000` и выше — версия, закреплённая вручную. Итоговый кандидат выбирается по формуле: приоритет, затем версия. Политику задают в `/etc/apt/preferences.d/`:

```
Package: nginx
```

```
Pin: version 1.26.3-3+deb13u7
Pin-Priority: 1001
```

Это и есть «пин» — способ не пустить в продакшн версию, пока её не проверили. Обратная сторона: запиненный пакет перестаёт получать исправления безопасности.

Зависимости разрешает apt: `Depends` (без пакета нельзя), `Recommends` (ставится по умолчанию, отключается `--no-install-recommends`), `Suggests` (не ставится). Когда граф не сходится, apt предлагает варианты — и слепо соглашаться на «remove 40 packages» не стоит. В RHEL то же делают `dnf repoquery --requires`, приоритеты задаёт `priority=` в `.repo`, а версии фиксирует `versionlock`.

Версии, владельцы файлов и hold

bash

```
apt list --installed 2>/dev/null | head -3
```

```
Listing...
7zip/stable,now 25.01+dfsg-1~deb13u2 amd64 [installed,automatic]
adduser/stable,now 3.152 all [installed]
```

Пометка `automatic` означает, что пакет притянут зависимостью и уйдёт `apt autoremove`, когда станет не нужен; `apt-mark manual nginx` говорит «поставлен осознанно».

Главный инструмент — «какой пакет владеет файлом»:

bash

```
dpkg -S /usr/bin/ls
dpkg -L coreutils | head -3
```

```
coreutils: /usr/bin/ls
/.
/usr
/usr/bin
```

Если файл не принадлежит ни одному пакету, `dpkg -S` ответит `no path found` — сигнал, что он собран вручную и потеряется при переустановке. Для файлов из неустановленных пакетов — `apt-file search`, в RHEL — `dnf provides */nginx.conf` и `rpm -qf /etc/nginx/nginx.conf`.

Версии держат `apt-mark hold nginx` (снять — `unhold`), в RHEL — `dnf versionlock add nginx`. Hold — мера на время инцидента, не стратегия: без обновлений пакет копит уязвимости.

Обновления: upgrade, full-upgrade и цена спешки

bash

```
apt update            # обновить индексы, не трогая пакеты
apt upgrade           # обновить всё, что можно без удаления пакетов
apt full-upgrade      # разрешить удаление и установку ради зависимостей
```

Разница принципиальна. `apt upgrade` не удаляет пакеты: если новая версия требует убрать зависимость, пакет останется старым. `apt full-upgrade` (ранее `dist-upgrade`) удаляет и добавляет что угодно ради консистентного графа — он и сносит половину системы, если подключён сторонний репозиторий с другой libc. Порядок: `apt update && apt upgrade`, глазами `apt list --upgradable`, и только потом `full-upgrade` с чтением списка `REMOVING`.

В Debian автообновления безопасности делает `unattended-upgrades`, в RHEL — `dnf-automatic`. Полезен и `needrestart` (`dnf needs-restarting`): обновление библиотеки не меняет уже запущенный процесс — сервис работает со старой версией в памяти, пока его не перезапустят.

Unit-файлы: три секции

Юнит — текстовое описание объекта для systemd. Каталоги поиска по убыванию приоритета: `/etc/systemd/system` (администратор), `/run/systemd/system` (временные), `/usr/lib/systemd/system` (пакеты). Файл с тем же именем в `/etc` перекрывает пакетный целиком; `systemctl edit` создаёт drop-in `name.service.d/override.conf` — правильный способ правки чужого юнита.

ini

```
[Unit]
Description=Example API
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=app
Group=app
WorkingDirectory=/srv/app
Environment=APP_ENV=production
EnvironmentFile=/etc/app/env
ExecStart=/srv/app/bin/api --listen :8080
```

```
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

`[Unit]` — метаданные и зависимости, `[Service]` — как запускать и что делать при падении, `[Install]` — при какой цели включать (`WantedBy` создаёт ссылку в `/etc/systemd/system/multi-user.target.wants/` при `systemctl enable`). Без `[Install]` юнит запускается только вручную или по зависимости.

Типы сервисов: simple, oneshot, notify

`Type=` определяет, когда systemd считает сервис запущенным.

Тип	Когда «запущен»	Для чего
`simple` (по умолчанию)	сразу после `fork/exec`	обычные демоны, не сообщающие о готовности
`exec`	после успешного `execve`	когда важен провал запуска (нет файла, нет прав)
`oneshot`	после завершения процесса	скрипты, задачи; нужен `RemainAfterExit=yes`, если сервис «активен» после выхода
`notify`	после `sd_notify(READY=1)`	приложения с поддержкой systemd (Go, Python через sdnotify)
`forking`	после выхода родителя	старые демоны, демонизирующиеся сами

Разница видна на практике: `Type=simple` с битым `ExecStart` даёт `active (running)`, а через секунду — `failed`, и в мониторинге это «моргание». `Type=exec` поймает отсутствующий бинарник сразу. `Type=notify` убирает гонку «порт ещё не слушается, а systemd уже говорит „готово“».

Падения, лимиты и окружение

`Restart=on-failure` перезапускает при ненулевом коде выхода, сигнале или таймауте; `always` — и после чистого выхода. `RestartSec=` по умолчанию 100 мс, и это почти всегда слишком быстро: падающий на старте сервис крутится в tight loop. После

`StartLimitBurst` попыток за `StartLimitIntervalSec` (по умолчанию 5 за 10 секунд) systemd сдаётся и пишет `start request repeated too quickly` — сбросить счётчик можно `systemctl reset-failed`.

Остальное, что чаще всего правят:

- `User=`/`Group=` — под кем работать; без них сервис работает под root, и это главная ошибка самописных юнитов;
- `Environment=` — переменные в юните (строка `KEY=VALUE`), `EnvironmentFile=` — файл с правами `600`; префикс `-` разрешает его отсутствие. Синтаксис не shell: никаких `export`, кавычек и подстановок;
- `LimitNOFILE=` — лимит дескрипторов; в юнитах по умолчанию 1024, и для сервиса с тысячами соединений это «слишком много открытых файлов» в самый неподходящий момент. `LimitNOFILE=65536` — обычная практика;
- `UMask=` — маска создания файлов (глава 5a): сервис наследует маску демона, а не вашей оболочки;
- `TimeoutStopSec=` (по умолчанию 90 с) — сколько ждать корректной остановки перед `SIGKILL`; сервис, игнорирующий `SIGTERM`, будет убит жёстко и потеряет незаписанные данные;
- `NoNewPrivileges=yes`, `PrivateTmp=yes`, `ProtectSystem=strict`, `ReadWritePaths=` — песочница без контейнера: с `strict` ФС только для чтения, запись разрешают точечно.

Зависимости: After, Wants, Requires

Порядок и необходимость — разные вещи, и их путают чаще всего:

- `Wants=` — «запусти вместе со мной, но если он упал, мне всё равно»;
- `Requires=` — «без него нельзя»: падение зависимости останавливает и юнит;
- `After=`/`Before=` — только порядок, без требования запускать;
- `BindsTo=` — жёстче `Requires`: остановка зависимости останавливает юнит;

- `PartOf=` — остановка/перезапуск распространяется в обратную сторону.

`Wants=network-online.target` без `After=network-online.target` не даёт ничего, кроме «цель тоже будет запущена»: сервис стартует параллельно и, возможно, раньше сети. Нужны оба. `After=network.target` — «стек инициализирован», а не «есть адрес и маршрут»; реальную готовность даёт `network-online.target`.

После правки любого юнита — `systemctl daemon-reload`: без него systemd работает со старым описанием, и «изменения не применились» — забытая команда. Перезагрузка нужна и после изменения `[Install]`, но не после правки `EnvironmentFile`: файл читается при каждом старте.

Таймеры вместо cron

Таймер — юнит `.timer` рядом с `.service` того же имени. Выигрыш перед cron: логи в journald, зависимости и песочница как у сервиса, `Persistent=true` — догоняющая работа после простоя машины.

ini

```
[Unit]
Description=Nightly backup

[Timer]
OnCalendar=*-*-* 03:30:00
Persistent=true
RandomizedDelaySec=10min
AccuracySec=1min
Unit=backup.service

[Install]
WantedBy=timers.target
```

`OnCalendar` понимает `daily`, `Mon..Fri 09:00` и полный формат `год-месяц-день часы:минуты:секунды`; `systemd-analyze calendar '*-*-* 03:30:00'` печатает нормализованную форму и следующие срабатывания. `Persistent=true` хранит отметку запуска в `/var/lib/systemd/timers` и догоняет пропущенное после загрузки. `RandomizedDelaySec` разводит сотню машин, чтобы они не пришли на бэкап-сервер одной секундой.

bash

```
systemctl enable --now backup.timer
systemctl list-timers backup.timer
```

```
NEXT                         LEFT     LAST                         PASSED  UNIT
    ACTIVATES
Fri 2026-10-30 03:30:00 +05 7h 31min Thu 2026-10-29 03:31:12 +05 16h ago
    backup.timer  backup.service
```

`enable` нужен таймеру, а не сервису: `[Install]` в юните сервиса заставит его запускаться ещё и при загрузке.

journald и journalctl

systemd собирает stdout/stderr сервисов, ядро и syslog в один бинарный журнал:

bash

```
journalctl -u backup.service --since today      # по юниту и времени
journalctl -u docker.service -f                 # следить в реальном времени
journalctl -b -1 -p err                         # прошлая загрузка, только ошибки
journalctl --since "2026-10-29 03:00" --until "2026-10-29 04:00" -o short-iso
journalctl -u app.service -o json-pretty | head -40
journalctl --disk-usage
```

```
окт 29 03:30:02 nextpc backup[2417]: dumping pg: 128 MB in 4.1s
окт 29 03:30:06 nextpc backup[2417]: upload: 1.2 GB in 3.9s
```

Приоритеты — от `0` (emerg) до `7` (debug); `-p err` — «err и выше», `-p warning..err` — диапазон. `-b -1` — предыдущая загрузка, `--list-boots` покажет все. Фильтры по полям: `_PID=2417`, `_SYSTEMD_UNIT=`.

По умолчанию `Storage=auto`, и журнал **не переживает перезагрузку**: он в `/run`, пока нет каталога `/var/log/journal`. `mkdir /var/log/journal && systemctl restart systemd-journald` делает его постоянным. Размер задают в `/etc/systemd/journald.conf` (`SystemMaxUse=2G`), чистят `journalctl --vacuum-size=500M`.

Ротация: logrotate

Если сервис пишет в файл сам, journald ему не указ — нужен logrotate. Конфиг в `/etc/logrotate.d/`, проверка без изменений — `logrotate -d`.

```
/var/log/app/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 app adm
```

```
    sharedscripts
    postrotate
        systemctl reload app.service >/dev/null 2>&1 || true
    endscript
}
```

`rotate 14` — хранить 14 архивов, `delaycompress` — не сжимать самый свежий (его ещё может держать процесс), `create` задаёт права нового файла. `postrotate` обязателен, если процесс держит дескриптор: без переоткрытия он продолжит писать в переименованный файл, и диск забьётся «удалённым» логом. Альтернатива `copytruncate` теряет строки при копировании. На Debian logrotate запускается таймером.

systemd-analyze blame

bash

```
systemd-analyze blame | head -4
systemd-analyze critical-chain multi-user.target
```

```
2min 9.094s backup-weekly.service
    42.396s health-check.service
     5.405s systemd-modules-load.service
     2.477s gpu-setup.service
```

`blame` показывает время активации юнитов, а не «тормоза загрузки»: `Type=oneshot` с двухминутным скриптом законно занимает две минуты и попадает в топ, хотя загрузку не задерживает, если от него никто не зависит. Реальное время определяет `critical-chain`.

Полный рабочий пример: бэкап по таймеру

Скрипт `/usr/local/bin/backup.sh` (проверка места, дамп, rsync) и два юнита.

ini

```
# /etc/systemd/system/backup.service
[Unit]
Description=Nightly backup of /srv/app to /mnt/backup
RequiresMountsFor=/mnt/backup
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
User=backup
Group=backup
EnvironmentFile=/etc/backup.env
ExecStart=/usr/local/bin/backup.sh --src /srv/app --dst /mnt/backup
```

```
Nice=10
IOSchedulingClass=best-effort
IOSchedulingPriority=7
TimeoutStartSec=2h
UMask=0027
PrivateTmp=yes
ProtectSystem=strict
ProtectHome=read-only
ReadWritePaths=/mnt/backup
NoNewPrivileges=yes
```

ini

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Run nightly backup at 03:30

[Timer]
OnCalendar=*-*-* 03:30:00
Persistent=true
RandomizedDelaySec=10min
Unit=backup.service

[Install]
WantedBy=timers.target
```

bash

```
systemctl daemon-reload
systemd-analyze verify /etc/systemd/system/backup.service
systemctl start backup.service
systemctl enable --now backup.timer
journalctl -u backup.service -n 20 --no-pager
```

Разбор решений: `Type=oneshot` — скрипт завершается, сервис не «висит»; `RequiresMountsFor` гарантирует, что `/mnt/backup` смонтирован, иначе бэкап уйдёт в корневую ФС; `TimeoutStartSec=2h` — дефолтных 90 секунд не хватит; `Nice` и `IOSchedulingClass` снижают давление на диск. `systemd-analyze verify` ловит опечатки в путях до первого запуска.

Типичные ошибки

1. **Правка юнита без** `daemon-reload`**.** Изменения не применяются, в статусе старый `ExecStart`: systemd держит разобранные юниты в памяти.
2. `Wants=` **без** `After=`**.** Зависимость запускается, но порядок не гарантирован — падение на гонке «сети ещё нет»: `Wants` — необходимость, `After` — порядок.

3. **Сервис под root по умолчанию.** Нет `User=` — демон получает полные права; systemd не наследует пользователя из вашей оболочки.
4. `LimitNOFILE=1024`**.** Сервис упирается в дескрипторы при росте соединений: это дефолт юнита, а не хоста, и `ulimit -n` в сессии о сервисе ничего не говорит.
5. `Type=simple` **для скрипта.** `systemctl start` возвращает успех, хотя скрипт уже упал: «запущен» значит «процесс создан», а не «работает».
6. `apt full-upgrade` **без чтения списка удалений.** При сторонних репозиториях сносит нужные пакеты: `full-upgrade` разрешает удаление ради консистентности графа.

Чек-лист

- `dpkg -S $(command -v nginx)` отвечает, какой пакет владеет бинарником; `dpkg -V пакет` не показывает изменённых файлов.
- `apt upgrade` выполняется регулярно, `full-upgrade` — осознанно, с чтением `REMOVING`.
- У каждого самописного юнита есть `[Install]`, `User=`, `Restart=`, `RestartSec≥1s`.
- `systemd-analyze verify` проходит для всех новых юнитов, таймер включён, а сервис — нет.
- `/var/log/journal` существует, `SystemMaxUse` задан; для файловых логов есть конфиг в `/etc/logrotate.d/` с `postrotate`.

Упражнения

1. Найдите пакет, которому принадлежит `/usr/bin/ss`, и все файлы этого пакета. Объясните, что означает `ii` в `dpkg -l`.
2. Запинните nginx на текущую версию, обновите индексы и докажите, что кандидат не изменился. Затем снимите пин и объясните риск.
3. Напишите юнит для Python-сервиса с `Type=notify`, ограничением `LimitNOFILE=65536` и песочницей, проверьте его `systemd-analyze verify`.

4. Сделайте таймер для healthcheck каждые 5 минут так, чтобы пропущенный запуск выполнился после перезагрузки.
5. Ограничьте журнал 500 МБ, включите постоянное хранение и найдите все ошибки предыдущей загрузки одной командой.

Ответы и разбор к этим упражнениям — в приложении D, часть 1.

Глава 5c. Linux: namespaces, cgroups, диагностика

Контейнер — не «маленькая виртуалка», а обычный процесс Linux, которому ядро показало урезанную картину мира и ограничило аппетит. Второй ОС внутри нет: тот же планировщик, тот же сетевой стек, то же ядро. Отсюда и странности, которые ломают интуицию: `free` в контейнере показывает память хоста, `sysctl -w` не работает, `docker kill` возвращает код 137, а `top` в поде врёт про CPU. В этой главе — семь видов namespaces, cgroups v2 с реальными файлами, veth и bridge, overlayfs в двух словах, набор диагностических инструментов и разбор двух самых частых вопросов эксплуатации. В конце — прямой мост к Docker (глава 38) и Kubernetes (глава 41).

Namespaces: что именно изолируется

Namespace (пространство имён) — это подмена глобальной таблицы ядра на локальную для группы процессов. Процесс в своём namespace видит только «свои» объекты и не подозревает о существовании остальных.

Namespace	Что изолирует	Что видно снаружи
`pid`	номера процессов	процесс в контейнере — обычный PID на хосте
`net`	интерфейсы, адреса, маршруты, порты, iptables	свой `lo`, свой порт 80
`mnt`	точки монтирования	свой корень `/`, свои тома
`uts`	hostname и домен NIS	своё имя хоста
`ipc`	очереди сообщений, семафоры, разделяемая память	свои очереди POSIX/SysV
`user`	uid/gid и capabilities внутри	uid 0 внутри — не uid 0 на хосте
`cgroup`	видимая иерархия cgroup	свой `/sys/fs/cgroup`

Отдельно стоит `user`: он позволяет процессу быть root внутри namespace, оставаясь непривилегированным снаружи. Именно на нём держатся rootless-контейнеры (Podman, Docker в

rootless-режиме), и именно без него «root в контейнере» означает настоящий root на хосте. Есть ещё `time` (смещение монотонных часов, с ядра 5.6), но в разговоре о контейнерах он почти не встречается.

Каждый namespace — объект ядра с номером inode. Посмотреть, в каких namespace живёт процесс:

bash

```
ls -l /proc/self/ns/
```

```
lrwxrwxrwx 1 nextdoor nextdoor 0 окт 29 20:14 cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 nextdoor nextdoor 0 окт 29 20:14 ipc -> 'ipc:[4026531839]'
lrwxrwxrwx 1 nextdoor nextdoor 0 окт 29 20:14 mnt -> 'mnt:[4026531841]'
lrwxrwxrwx 1 nextdoor nextdoor 0 окт 29 20:14 net -> 'net:[4026531992]'
lrwxrwxrwx 1 nextdoor nextdoor 0 окт 29 20:14 pid -> 'pid:[4026531836]'
lrwxrwxrwx 1 nextdoor nextdoor 0 окт 29 20:14 user -> 'user:[4026531837]'
lrwxrwxrwx 1 nextdoor nextdoor 0 окт 29 20:14 uts -> 'uts:[4026531838]'
```

Совпадающие номера означают общий namespace. Это самый быстрый способ проверить, «в контейнере ли» процесс: если `net` у двух процессов одинаковый — они в одной сети.

unshare, lsns, nsenter

Создать namespace вручную — `unshare`. Полноценный «контейнер на коленке» выглядит так:

bash

```
sudo unshare --pid --net --mount --uts --ipc --fork --mount-proc /bin/bash
```

```
root@host:/# ps aux
    PID TTY          TIME CMD
      1 pts/1    00:00:00 bash
      7 pts/1    00:00:00 ps
```

Внутри — два процесса и `bash` с PID 1. Снаружи это обычные PID хоста. `--fork` обязателен: без него `unshare` меняет namespace себе, а `--mount-proc` перемонтирует `/proc`, иначе `ps` покажет процессы хоста, потому что старый `/proc` остался от родителя.

Обратные инструменты — `lsns` (кто где) и `nsenter` (зайти внутрь):

bash

```
lsns -t net -o NS,TYPE,NPROCS,PID,COMMAND
sudo nsenter -t 4213 -n -p ss -tlnp
```

```
        NS TYPE NPROCS   PID COMMAND
4026531840 net      24   742 python /srv/bot/bot.py
```

`nsenter -t PID -n` входит в сетевой namespace процесса и позволяет увидеть его интерфейсы и порты, не заходя в контейнер через `docker exec` — полезно, когда контейнер «почти мёртв», но процесс ещё есть. Так же работают `-m` (корень ФС), `-p`, `-u`.

cgroups v2: иерархия и лимиты

Контрольные группы (cgroups) отвечают за вторую половину — сколько ресурсов процессу можно. В cgroups v2 иерархия одна, смонтирована в `/sys/fs/cgroup`, а контроллеры (cpu, memory, io, pids) включаются поштучно на каждом уровне через `cgroup.subtree_control`. Посмотреть, что доступно и что включено:

bash

```
cat /sys/fs/cgroup/cgroup.controllers
cat /sys/fs/cgroup/cgroup.subtree_control
```

```
cpuset cpu io memory hugetlb pids rdma misc
cpuset cpu io memory pids
```

Лимиты — это файлы, и значение `max` в них означает «без ограничения».

bash

```
C=/sys/fs/cgroup/system.slice/docker-0fce964f4258.scope
cat $C/memory.max $C/memory.current $C/cpu.max $C/pids.max
```

```
536870912
418734080
200000 100000
512
```

Читается так: памяти не больше 512 МиБ (536870912 = 512×1024²), занято около 400 МиБ, CPU — 200000/100000, то есть два ядра (квота 200 мс на период 100 мс), процессов не больше 512. `cpu.weight` (по умолчанию 100) задаёт не потолок, а долю при конкуренции — это разница между «не больше двух ядер» и «получить половину при перегрузке».

`memory.low` защищает память от вытеснения, `memory.high` — мягкий порог с торможением, `memory.max` — жёсткий, за которым наступает OOM. `cgroup.procs` — PID группы, `cgroup.freeze=1` — заморозка, `cgroup.kill=1` — убить всю группу разом (ядро 5.14+), что решает проблему «убил родителя — остались внуки».

OOM: memory.events и код 137

Когда группа упирается в `memory.max`, ядро сначала пытается освободить страницы (reclaim), а если не выходит — вызывает OOM-killer и убивает процесс внутри группы. Счётчики лежат рядом:

bash

```
cat $C/memory.events
```

```
low 0
high 0
max 14
oom 2
oom_kill 2
```

`max 14` — сколько раз группа упиралась в потолок, `oom_kill 2` — сколько процессов убито. Это единственный надёжный способ отличить «приложение утекло память» от «приложению не дали памяти»: если `oom_kill` растёт, виноват лимит, а не код.

Снаружи убийство выглядит как код `137` — это `128 + 9` (SIGKILL) по соглашению Unix. В `dmesg` остаётся `Memory cgroup out of memory: Killed process 2417 (python)`. Отсюда практика: `137` в Kubernetes — почти всегда `limits.memory`, `143` (128+15) — корректная остановка по SIGTERM.

Сеть контейнера: veth и bridge

Сетевой namespace сам по себе пуст: внутри только `lo`. Чтобы дать контейнеру сеть, создают пару виртуальных интерфейсов (veth pair): один конец остаётся на хосте, второй переносят в namespace контейнера. Всё, что попадает в один конец, выходит из другого.

bash

```
ip -br link | head -5
ip -br addr show docker0
```

```
lo               UNKNOWN        00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
enp3s0           UP             d0:17:c2:93:74:8d <BROADCAST,MULTICAST,UP,LOWER_UP>
docker0          UP             f6:14:2a:c3:f8:9b <BROADCAST,MULTICAST,UP,LOWER_UP>
veth7f46a3a@if2  UP             aa:28:b2:95:54:19 <BROADCAST,MULTICAST,UP,LOWER_UP>

docker0          UP             172.17.0.1/16 fe80::f414:2aff:fec3:f89b/64
```

`veth7f46a3a@if2` — хостовый конец пары; `@if2` означает «парный интерфейс с индексом 2 в другом namespace». `docker0` —

программный мост (bridge): он коммутирует veth-концы между собой, как обычный свитч, и держит адрес шлюза по умолчанию для контейнеров. Выход в мир — через NAT: правило `MASQUERADE` в `iptables`/`nftables` подменяет адрес контейнера на адрес хоста. Публикация порта `-p 8080:80` — это DNAT на хосте плюс запись в цепочке `DOCKER`.

Типичные симптомы: контейнер не видит соседа — разные bridge-сети; `ping` работает, а `curl` нет — MTU или NAT. Диагностика — в namespace контейнера: `nsenter -t PID -n ip addr`, `ss -tlnp`.

overlayfs в двух словах

Корневая ФС контейнера собрана из слоёв: нижние — read-only (образ), верхний — пустой каталог для записи. overlayfs показывает их как один каталог `merged`. При записи в файл из нижнего слоя ядро сначала копирует его наверх (copy-up) и только потом меняет копию; удаление ставит в верхнем слое «белую метку» (whiteout), чтобы файл не появился снова. Изменения в контейнере живут в верхнем слое и исчезают вместе с ним, а образ остаётся неизменным; поэтому `rm` внутри не уменьшает образ (глава 38).

Диагностика: что и где смотреть

Набор инструментов делится на три вопроса: кто ест ресурсы, кто держит файл, что говорит ядро.

bash

```
cat /proc/loadavg                 # 0.84 1.15 0.64 1/555 553079
cat /proc/meminfo | head -3        # MemTotal, MemFree, MemAvailable
ps -eo pid,ppid,stat,pcpu,pmem,rss,etime,cmd --sort=-rss | head
vmstat 1 3                         # r b swpd free buff cache si so bi bo in cs us sy id wa
iostat -x 1 3                      # %util и await по дискам
ss -tlnp                           # кто слушает порты
lsof -p 2417 | head                # какие файлы и сокеты открыл процесс
strace -f -p 2417 -e trace=openat,connect
dmesg --since "1 hour ago" | tail  # OOM, ошибки драйверов, segfault
```

Load average — самый непонятый показатель. Это среднее число задач в состоянии `R` (выполняется или ждёт CPU) и `D` (uninterruptible sleep — обычно ждёт диск или сеть) за 1, 5 и 15 минут. Поэтому load 8.0 на восьмиядерной машине с `%id` 80 — это

не «процессор загружен», а «восемь процессов висят в ожидании I/O». Смотреть надо в паре: `vmstat` показывает колонку `wa` и очередь `b`, и если `wa` растёт вместе с load, а `us`/`sy` низкие — узкое место диск, а не CPU. В контейнере `/proc/loadavg` общий с хостом: load — величина не namespaced, и это одна из причин, почему метрики пода надо брать из cgroup (`cpu.stat`), а не из `top`.

`%st` (steal) в `top` показывает, сколько времени гипервизор отобрал у вашей виртуалки — на «железном» сервере он всегда 0, а в облаке легко объясняет «медленно, хотя CPU свободен».

Три инструмента ломаются в контейнерах предсказуемо: `dmesg` требует `CAP_SYSLOG` и упирается в `kernel.dmesg_restrict=1`; `strace` требует `CAP_SYS_PTRACE` и часто запрещён seccomp; `lsof` показывает пути внутри namespace, а не хоста. Поэтому диагностируют снаружи — через `/proc/PID` и `nsenter`.

Вопрос 1: почему контейнер видит память хоста

bash

```
docker run --rm -m 512m alpine free -m
```

```
              total        used        free      shared  buff/cache   available
Mem:          15887        9214        1174         412        5498        6337
Swap:             0           0           0
```

Лимит 512 МиБ, а `free` показывает 15,5 ГиБ. Причина в том, что `/proc/meminfo` собирается из глобальных счётчиков памяти ядра, а не из cgroup. Namespaces покрывают PID, сеть, монтирования, IPC, UTS, user и cgroup — память в этот список не входит, потому что ядро одно на всех. В cgroup namespaced только дерево `/sys/fs/cgroup`, где лежит настоящий лимит.

Правильный ответ на вопрос «сколько памяти у контейнера»:

bash

```
cat /sys/fs/cgroup/memory.max
cat /sys/fs/cgroup/memory.current
```

```
536870912
418734080
```

`memory.current` — фактическое потребление группы вместе с page cache, поэтому оно всегда больше «памяти приложения» и

уменьшается после записи на диск. В Kubernetes те же цифры приходят в метриках cAdvisor как `container_memory_working_set_bytes`, и именно на них смотрит планировщик и OOM.

Обойти вранье `free` можно через lxcfs (FUSE-слой, подменяющий `/proc/meminfo` и `/proc/cpuinfo` значениями cgroup) — так делают LXC и некоторые managed-кластеры. В Docker/Kubernetes по умолчанию этого нет, и приложение, читающее `/proc/meminfo` для автонастройки (JVM, PostgreSQL, Elasticsearch), берёт память хоста и падает по OOM. Лечится явными лимитами в конфиге (`-Xmx`, `shared_buffers`) или переменными окружения — это одна из самых частых причин «контейнер убит сразу после старта».

Вопрос 2: почему в контейнере не работают sysctl

bash

```
docker run --rm alpine sh -c 'echo 4096 > /proc/sys/net/core/somaxconn'
sh: can't create /proc/sys/net/core/somaxconn: Read-only file system
```

Причин две, и они разные. Первая: `/proc/sys` в контейнере смонтирован read-only — Docker делает это намеренно, и запись отбивается на уровне монтирования, хотя `net.*` вообще-то namespaced и принадлежит сети контейнера. Вторая: параметры `kernel.*`, `vm.*`, `fs.*` не namespaced вовсе — они относятся к ядру, а ядро у всех общее. Даже с `--privileged` запись в `kernel.shmmax` изменит настройку для всего хоста.

Практика: `net.*` задают при создании контейнера (`docker run --sysctl net.core.somaxconn=4096`) или на уровне пода (`securityContext.sysctls` в Kubernetes, где разрешён только список «безопасных» параметров). Всё остальное меняют на хосте через `/etc/sysctl.d/` и `sysctl --system` — и это, кстати, ещё один аргумент против «контейнер как замена виртуалке»: граница владения ядром проходит по хосту.

Мост: как из этого получается Docker и зачем это Kubernetes

Сложите два механизма. Namespaces дают процессу иллюзию отдельной машины: свой PID 1, своя сеть, свой корень. Cgroups

дают ему бюджет: не больше 512 МиБ, два ядра, 512 процессов. Добавьте overlayfs как корневую ФС из слоёв и `pivot_root` вместо `chroot` — получится ровно то, что делает `runc` по команде Docker: `clone(CLONE_NEWPID|CLONE_NEWNET|...)`, запись лимитов в `/sys/fs/cgroup`, монтирование overlay, `execve`.

Docker в главе 38 — обёртка над этими вызовами: образы и слои, bridge-сети, тома, рестарт-политики. Магии нет, и когда контейнер «не стартует», вопрос почти всегда один из трёх: не тот namespace, упёршийся cgroup-лимит, перекрытый слой.

Kubernetes в главе 41 добавляет оркестрацию, но опирается на то же самое: `limits.memory` — это `memory.max` в cgroup пода, `requests.cpu` — `cpu.weight`, `shareProcessNamespace` — общий pid-namespace, `hostNetwork` — отказ от net-namespace, а `terminationGracePeriodSeconds` — это `TimeoutStopSec` из главы 5b. Инженер, который знает `/sys/fs/cgroup` и `nsenter`, отлаживает поды быстрее того, кто знает только `kubectl describe`.

Типичные ошибки

1. **Верить `free` и `top` внутри контейнера.** Память и load берутся с хоста. Механизм: `/proc/meminfo` и `/proc/loadavg` не namespaced; лимит живёт в `/sys/fs/cgroup`.
2. `sysctl -w` **в контейнере без** `--sysctl`**.** Ответ — `Read-only file system`. Механизм: `/proc/sys` смонтирован только для чтения, а `kernel.*`/`vm.*` вообще глобальны.
3. `strace`/`dmesg` **не работают в поде.** Механизм: нет `CAP_SYS_PTRACE`/`CAP_SYSLOG`, срабатывает seccomp.
4. `unshare --pid` **без** `--fork`**.** Внутри видно процессы хоста. Механизм: namespace применяется к самому `unshare`, а `ps` читает старый `/proc`.
5. **Код 137 трактуют как «упало приложение».** Механизм: `128+9` — это SIGKILL, и чаще всего его прислал OOM-killer по `memory.max`; смотреть `memory.events`.
6. **Лимит CPU как гарантия.** `cpu.max` ограничивает, но не резервирует. Механизм: квота работает только когда есть конкуренция, а `cpu.weight` — это доля, не абсолют.

Чек-лист

- `ls -l /proc/PID/ns/` показывает, в каких namespace живёт процесс и совпадает ли `net` с ожидаемым.
- Для любого лимита известно имя файла в cgroup v2: `memory.max`, `cpu.max`, `pids.max`.
- `memory.events` прочитан перед выводами об утечке: `oom_kill` отличает лимит от бага.
- Код возврата расшифрован: 137 — SIGKILL, 143 — SIGTERM, 126/127 — про запуск, а не про приложение.
- Сеть проверена изнутри namespace (`nsenter -n ip addr`, `ss -tlnp`), а не только с хоста.
- Для диагностики снаружи доступны `/proc/PID`, `nsenter`, `dmesg` на хосте.

Упражнения

1. Создайте через `unshare` окружение с собственным PID 1, hostname и пустым сетевым namespace. Докажите, что `ps` внутри показывает один процесс, а снаружи — все.
2. Найдите в `/sys/fs/cgroup` группу работающего контейнера и объясните каждое значение в `memory.max`, `cpu.max`, `pids.max`, `memory.events`.
3. Запустите процесс под лимитом 64 МиБ, который аллоцирует 256 МиБ, и определите по `memory.events` и `dmesg`, что именно произошло.
4. Сравните `free -m` внутри контейнера с `memory.max`/`memory.current` и объясните расхождение в двух предложениях.
5. Остановите сервис внутри контейнера и объясните разницу между `SIGTERM` и `SIGKILL` с точки зрения `terminationGracePeriodSeconds`.

Ответы и разбор к этим упражнениям — в приложении D, часть 1.

Глава 5d. eBPF: наблюдение за ядром без пересборки

Диагностика долго делилась на два лагеря: «снаружи» — `strace`, `perf`, логи — и «внутри» — правка кода, пересборка, новый модуль ядра. eBPF убирает выбор: небольшая программа загружается в работающее ядро, проверяется верификатором, компилируется в машинный код и срабатывает на нужных событиях, не меняя ни ядро, ни приложение. В этой главе — что такое eBPF и почему это не модуль ядра, какие бывают точки прикрепления, как читать однострочники `bpftrace`, чем это отличается от `perf` и `strace`, где пределы метода и почему код в ядре опаснее обычной программы. К концу вы сможете ответить, кто открывает файл, сколько длится syscall и почему процесс ждёт, хотя CPU свободен.

Что такое eBPF

eBPF (extended Berkeley Packet Filter) — виртуальная машина внутри ядра с собственным набором инструкций: 11 регистров по 64 бита (R10 — указатель на кадр стека), стек 512 байт на программу, запрет произвольных переходов. Программа пишется на подмножестве C, компилируется в байт-код (clang с целью `bpf`), загружается системным вызовом `bpf()` и прикрепляется к событию. Исполняется она интерпретатором или JIT-компилятором в машинный код хоста — по умолчанию вторым, и тогда срабатывание стоит десятки-сотни наносекунд.

Отличие от модуля ядра не в мощности, а в способе попадания внутрь. Модуль собран против заголовков конкретного ядра, исполняется с полным доверием, и ошибка в нём означает панику. Программа eBPF не содержит машинного кода платформы: до запуска её разбирает верификатор (verifier), который статически проходит все ветки и либо доказывает безопасность, либо отказывает. Загрузка занимает миллисекунды, выгрузка мгновенна (закрыли дескриптор —

программа снята), заголовки ядра не нужны, а худший исход ошибки — сообщение верификатора, а не kernel panic.

Верификатор требует, чтобы программа завершалась (циклы с доказуемым пределом, с ядра 5.3), обращалась к памяти только через проверенную арифметику указателей или хелперы вроде `bpf_probe_read_kernel` и не читала неинициализированный стек. Лимит сложности — около миллиона инструкций; всё, что не доказуемо, отклоняется.

Точки прикрепления

Тип программы определяет, что ей доступно и какой контекст придёт в аргументах.

Тип	Что перехватывает	Стабильность	Когда брать
`kprobe` / `kretprobe`	вход и возврат функции ядра по имени	символ могут переименовать или заинлайнить	нет подходящего tracepoint
`uprobe` / `uretprobe`	функция в бинарнике или библиотеке user space	ломается при обновлении пакета	нет исходников, а поведение нужно
`tracepoint`	статические точки трассировки ядра	стабильный ABI, поля описаны	есть подходящая точка
`fentry` / `fexit`	вход и выход функции через BTF	привязано к типу функции	ядро 5.5+, нужна типизация
`XDP`	пакет на входе драйвера, до `sk_buff`	стабильно	фильтрация и DDoS на линии

`kprobe` работает так: ядро заменяет первые байты функции инструкцией останова, по срабатыванию выполняет вашу программу и восстанавливает оригинал. Поэтому перехватывать можно только реально существующие символы (статическая функция, вкомпилированная в вызывающего, символа не имеет), и на горячем пути замена стоит недешево. `tracepoint` дешевле и надёжнее: точка уже вкомпилирована, формат аргументов зафиксирован, обновление ядра программу не ломает.

bash

```
sudo bpftrace -l 'kprobe:vfs_*' | head -3
```

```
kprobe:vfs_cancel_lock
kprobe:vfs_clean_context
kprobe:vfs_cleanup_quota_inode
```

Всего на рабочем ядре 6.12 доступно больше ста тысяч точек — без карты список бесполезен.

bpftrace: анатомия однострочника

`bpftrace` компилирует высокоуровневый язык в eBPF. Синтаксис один: `проба /фильтр/ { действие }`. Доступны `comm` (имя процесса), `pid`, `tid`, `nsecs` (монотонное время), `args->` (аргументы пробы), `retval`, `str(ptr)` и агрегаты `count()`, `sum()`, `hist()`. Карты (maps) объявляются знаком `@`: `@start[tid] = nsecs` — хеш-таблица в ядре, живущая между срабатываниями. В `bpftrace` 0.23 аргументы берутся через стрелку (`args->filename`); синтаксис `args.filename` из старых статей даёт ошибку компиляции.

Кто запускает процессы

Точка входа `execve` отвечает на вопрос «что вообще происходит на машине»:

bash

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_execve { printf("%-16s %-6d %s\n",
    comm, pid, str(args->filename)); }'
```

```
bash             589529 /usr/bin/docker
bash             589555 /usr/bin/ls
containerd       589561 /usr/bin/containerd-shim-runc-v2
containerd-shim  589580 /usr/bin/runc
```

`comm` — имя процесса, усечённое до 15 символов, `str()` копирует строку из памяти ядра. Видно, что на машине живёт containerd и что `runc` создаёт процессы: кто-то запускает контейнер, а не «непонятно что тормозит».

Кто открывает конкретный файл

Фильтр в косых чертах отсекает лишнее до того, как событие попадёт в карту, — главный приём экономии.

bash

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat /str(args->filename) ==
    "/etc/shadow"/ { printf("%-16s pid=%-6d\n", comm, pid); }'
```

```
unix_chkpwd      pid=590146
```

```
cat              pid=590147
```

`cat` — это мы, проверяли себя. `unix_chkpwd` — вспомогательный процесс PAM: сам `passwd`/`su`/`sudo` не может прочитать `/etc/shadow` и просит об этом setuid-помощника. Ограничение видно сразу: при относительном пути в `args->filename` будет только имя, а `openat2` приходит в отдельную точку `sys_enter_openat2`.

Сколько длится syscall

Латентность меряется парой «вход — выход», разница кладётся в гистограмму. Ключ `tid` обязателен: syscall может быть вытеснен, и одно глобальное значение времени перепутает потоки.

bash

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @start[tid] = nsecs; }
tracepoint:syscalls:sys_exit_openat /@start[tid]/ { @us = hist((nsecs -
    @start[tid]) / 1000); delete(@start[tid]); }
interval:s:6 { exit(); }'
```

```
@us:
[1]                 2785 |@@@
```

bash

```
sudo bpftrace -e 'tracepoint:sock:inet_sock_set_state /args->newstate == 2/ {
    @start[args->skaddr] = nsecs; @who[args->skaddr] = comm; }
tracepoint:sock:inet_sock_set_state /args->oldstate == 2 && args->newstate == 1/ {
    @ms[@who[args->skaddr]] = hist((nsecs - @start[args->skaddr]) / 1000000);
    delete(@start[args->skaddr]); }'
```

```
@ms[curl]:
[32, 64)               1 |@@@
```

Инструмент	Механизм	Накладные расходы	Что показывает
`strace`	`ptrace`, остановка на каждом syscall	в 10–100 раз медленнее	аргументы одного процесса
`perf`	сэмплирование по таймеру или PMU	единицы процентов	где горит CPU, стеки, off-CPU
`bpftrace`	событие и агрегация в ядре	зависит от частоты, обычно < 5 %	распределения по всему хосту

Один процесс, несколько секунд, нужны точные аргументы — `strace`. Непонятно, кто ест CPU, и нужен профиль на минуту — `perf record`. Нужно распределение по всей машине, и вопрос сформулирован как «кто/сколько/как часто» — `bpftrace`. На нагруженном сервере `strace` не «немного замедляет»: каждый syscall превращается в два переключения контекста, и сервис с сотней тысяч вызовов в секунду просто встаёт. BCC (`execsnoop`, `opensnoop`, `biolatency`, `tcplife`) — те же программы, но уже написанные: начинать разумно с них.

Ограничения метода

Загружать программы может только root: с ядра 5.8 полномочия разделены на `CAP_BPF` (карты и программы) и `CAP_PERFMON` (прикрепление к событиям), плюс `CAP_SYS_ADMIN` для части типов. Что разрешено остальным:

```bash
cat /proc/sys/kernel/unprivileged_bpf_disabled
2
```

`2` (норма для Debian, Ubuntu, RHEL) — «запрещено и не переключается без перезагрузки», `1` — «запрещено, но можно вернуть», `0` — «разрешено». В контейнерах `bpf()` дополнительно режется seccomp, поэтому `bpftrace` в поде без `CAP_BPF` не запустится вовсе.

Накладные расходы складываются из срабатываний: программа на `kprobe` функции, вызываемой миллион раз в секунду, добавляет работу на каждый вызов, а `printf` умножает это на копирование в буфер. Лечится фильтром в ядре, агрегатом вместо печати и

ограничением сбора по `interval`, а проверяется сравнением throughput до и после.

Версии: `bpftrace` требует ядра 4.9+ (комфортно с 5.x), BTF — 5.2+, `fentry` — 5.5+, `CAP_BPF` и `ringbuf` — 5.8+. И перехватывается не всё: нет символа — нет `kprobe` (статические функции инлайнятся, модули без BTF не типизированы), не видно чистой арифметики в user space и содержимого TLS до расшифровки. А `kprobe` на внутреннюю функцию — обещание, которое ломается при обновлении ядра.

Безопасность: почему eBPF-программа опаснее обычной

Обычная программа ограничена адресным пространством и правами пользователя: чтобы навредить, ей нужен syscall, а syscall проверяет ядро. Программа eBPF исполняется в режиме ядра, и единственная граница между ней и всей памятью машины — верификатор, то есть статический анализ, выполненный один раз. Дальше изоляции нет: хелпер `bpf_probe_read_kernel` читает произвольную память ядра, а программа на `kprobe` перед `sys_getdents64` может отфильтровать записи и спрятать процесс или файл от `ps` и `ls` — готовый rootkit, невидимый в списке модулей.

Отсюда три правила. Считать `CAP_BPF` эквивалентом root: контейнер с этим правом — контейнер с доступом к ядру хоста. Не запускать чужие однострочники «из статьи», не прочитав их. Смотреть, что загружено (`sudo bpftool prog list`), и держать `unprivileged_bpf_disabled=2`: история CVE в самом верификаторе показывает, что «песочница» здесь не абсолютна.

Мост: к namespaces и к наблюдаемости

С главой 5c связь прямая и часто неожиданная. `bpf_get_current_pid_tgid()` возвращает PID в корневом namespace, а не тот, который видит процесс внутри контейнера: namespaces подменяют таблицы для процессов, но не для программ ядра. Поэтому `bpftrace` с хоста показывает процессы контейнеров с хостовыми PID — это то, что нужно для отладки, и то, что ломает

фильтры «по PID из `docker top`». Ограничить сбор одним контейнером можно по `cgroup_id`, что и делают Cilium и Tetragon.

С главой 42 связь ещё прямее: eBPF — способ получить телеметрию без инструментирования кода. Время в syscall, off-CPU, ретрансмиты, латентность диска и HTTP собираются на уровне ядра одинаково для Go, Python и байтового кода JVM. Дальше — обычная работа с SLO: распределение вместо среднего, хвосты вместо «в целом нормально».

Типичные ошибки

1. **Синтаксис `args.filename` из старых статей.** В `bpftrace` 0.23 аргументы — указатель, нужен `args->filename`, иначе ошибка компиляции. Механизм: контекст пробы типизирован через BTF, обращение к полям идёт как к структуре по указателю.
2. **Латентность `connect()` через вход и выход syscall.** Получаются нули. Механизм: неблокирующий `connect()` возвращает `EINPROGRESS` сразу, рукопожатие идёт в softirq; нужен `sock:inet_sock_set_state`.
3. **Запуск без root.** Ответ — `ERROR: bpftrace currently only supports running as the root user`. Механизм: нет `CAP_BPF`/`CAP_PERFMON`, а в контейнере ещё и seccomp режет `bpf()`.
4. **Проба на горячей функции без фильтра.** Сервис теряет десятки процентов throughput. Механизм: `kprobe` подменяет первые байты функции точкой останова, каждое срабатывание — исключение плюс JIT-код.
5. **`@start[tid]` без `delete()`.** Карта растёт, память ядра кончается, события теряются. Механизм: карта — хеш-таблица с лимитом элементов, а запись без удаления на выходе накапливается.
6. **`kprobe` на функцию, которой нет.** `Error: failed to attach probe`. Механизм: символ заинлайнен, переименован или это макрос; проверять `bpftrace -l 'kprobe:имя*'` до написания программы.

Чек-лист

- Известно, что eBPF — байт-код с верификатором и JIT, а не модуль: загрузка без пересборки и перезагрузки.

- Перед программой выполнен `bpftrace -l` по нужному шаблону и точка существует, а не «должна быть».
- В измерительных пробах ключ включает `tid` или `skaddr`, а карта очищается `delete()`.
- Фильтр стоит в косых чертах: лишние события не доходят до карты.
- Для латентности сети используется `sock:inet_sock_set_state`, а не вход/выход `connect`.
- Сбор ограничен `interval:s:N { exit(); }`, а накладные расходы сняты `perf stat` до и после.
- Проверены `unprivileged_bpf_disabled` и состав программ (`bpftool prog list`).

Упражнения

1. Покажите все процессы, запускаемые за минуту, с родителем (`args->filename`, `pid`, `ppid`). Объясните, почему `comm` не всегда совпадает с именем бинарника.
2. Постройте гистограмму латентности `openat` для одного каталога и сравните до и после `echo 3 > /proc/sys/vm/drop_caches`. Какие бакеты изменились и почему?
3. Измерьте время установки TCP-соединения до трёх хостов и разделите его на DNS-этап и рукопожатие. Какие точки нужны для второй части?
4. Снимите off-CPU распределение для процесса, ждущего ввода, и для `dd`, ждущего диск. Объясните разницу в форме гистограммы.
5. Напишите программу, считающую `execve` по имени родителя, и оцените накладные расходы через `perf stat`. При какой частоте событий она перестаёт быть безопасной для продакшена?

Ответы и разбор к этим упражнениям — в приложении D, часть 5.

Глава 5e. SSH: ключи, конфиг, туннели

SSH — протокол, которым DevOps-инженер пользуется каждый день и чаще всего не понимает до конца: подключение «просто работает», пока не появится `Permission denied (publickey)`, не перестанет открываться база за бастионом или не выяснится, что ключ с продакшена унесли вместе с ноутбуком. В этой главе — что происходит за секунду рукопожатия, почему ed25519 лучше rsa 4096, как устроены `authorized_keys` и `~/.ssh/config`, чем отличаются три вида туннелей, как закрыть `sshd` и как диагностировать отказ, не перебирая наугад. Команды и вывод проверены на OpenSSH 10.0p2 с реального sshd.

Что происходит при подключении

Полсекунды между Enter и приглашением шелла — это четыре этапа, и каждый отказывает по-своему.

Сначала TCP на порт 22 и обмен версиями: обе стороны присылают строку `SSH-2.0-OpenSSH_10.0p2`.

Затем обмен ключами (key exchange, KEX) по RFC 4253: стороны договариваются об алгоритмах и вырабатывают общий секрет, из которого выводятся ключи шифрования и MAC для каждого направления. Асимметричная криптография нужна для двух вещей — договориться о симметричном ключе и подтвердить, что сервер тот, за кого себя выдаёт (подпись host key, сверяемая с `known_hosts`). OpenSSH по умолчанию берёт гибридную схему `mlkem768x25519-sha256`: постквантовый ML-KEM-768 плюс X25519.

Дальше аутентификация клиента (RFC 4252): при входе по ключу клиент не отправляет ключ «как пароль», а подписывает приватным ключом сессионный хэш, а сервер проверяет подпись по строке из `authorized_keys`. Приватный ключ не покидает машину никогда — в этом смысл схемы. После этого открывается канал (RFC 4254): по одному TCP мультиплексируются сессии, `exec`, `sftp`, проброс портов.

bash

```
ssh -vv -p 2222 -i ~/.ssh/id_ed25519 nextdoor@127.0.0.1 'uptime -p'
```

```
debug1: Remote protocol version 2.0, remote software version OpenSSH_10.0p2
    Debian-7+deb13u4
debug1: kex: algorithm: mlkem768x25519-sha256
debug1: kex: host key algorithm: ssh-ed25519
debug1: Server host key: ssh-ed25519 SHA256:MVp668g1RL1ExRB+/+H47RhyC+3gO07TbgKQifG
    408A
debug1: Offering public key: id_ed25519 ED25519
    SHA256:mmhapd3hr8pdHItCgsxvivdaofR8DNuddHVEwllea58
debug1: Server accepts key: id_ed25519 ED25519 SHA256:mmhapd...
Authenticated to 127.0.0.1 ([127.0.0.1]:2222) using "publickey".
```

Порядок строк и есть порядок этапов: `kex` — чем шифруется канал, `Server host key` — чем сервер доказал, что он сервер, `Offering`/`Server accepts` — какой ключ он согласился принять, `Authenticated` — итог. При отказе полезная строка всегда между `Offering` и `Authenticated`.

Ключи: ed25519 против rsa

Ключ ed25519 — 32 байта, подпись — 64 байта. Ключ RSA-4096 — 512 байт модуля и 512 байт подписи, и подпись заметно дороже. Стойкость при этом сопоставима: ed25519 даёт около 128 бит, RSA-4096 — порядка 140, и 12 бит разницы не компенсируют ни десятикратный размер, ни чувствительность RSA к выбору схемы дополнения.

Историческая ловушка: с OpenSSH 8.8 алгоритм `ssh-rsa` (подпись по SHA-1) отключён по умолчанию, и старый сервер, который умеет только его, отвечает `no matching host key type found`. Включать `PubkeyAcceptedAlgorithms +ssh-rsa` ради такого сервера — сознательный шаг назад по безопасности, а не «настройка совместимости».

bash

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -C 'nextdoor@workstation'
ssh-keygen -l -f ~/.ssh/id_ed25519.pub
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa -C 'nextdoor@workstation'
ssh-keygen -l -f ~/.ssh/id_rsa.pub
```

```
256 SHA256:mmhapd3hr8pdHItCgsxvivdaofR8DNuddHVEwllea58 nextdoor@workstation
    (ED25519)
4096 SHA256:nniMFPIZtEzEStb/IgvSuNsfhE3LvP8lL4GLwMqa44Q nextdoor@workstation (RSA)
```

Отпечаток (fingerprint) — SHA256 от тела ключа в base64, одинаковый у публичной и приватной части. Права на файлы не

косметика: `ssh` отказывается использовать ключ, если файл читаем группой или другими, — сообщение `Permissions 0644 for '...' are too open` означает ровно это.

Passphrase, ssh-agent и authorized_keys

Парольная фраза защищает файл: без неё украденный `id_ed25519` — готовый доступ, с ней — бесполезный зашифрованный блок. Плата — ввод фразы при каждом подключении, и здесь появляется агент.

bash

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
ssh-add -l
```

```
256 SHA256:mmhapd3hr8pdHItCgsxvivdaofR8DNuddHVEwllea58 nextdoor@workstation
    (ED25519)
```

Агент держит расшифрованный ключ в памяти и подписывает запросы по UNIX-сокету `SSH_AUTH_SOCK`. Отсюда правило, которое нарушают чаще всего: **проброс агента (`-A`) на чужой сервер равен передаче ключа**. На бастионе сокет доступен процессам вашей учётной записи, а root бастиона может подключиться к нему и подписать что угодно вашим ключом. Нужен «прыжок» — используйте `ProxyJump`: трафик идёт через бастион, ключ остаётся дома.

`authorized_keys` — не список ключей, а список правил: перед типом ключа ставятся опции, ограничивающие то, что этим ключом можно делать.

```
command="echo forced-command-ok; id -un",no-pty,no-port-forwarding ssh-ed25519
    AAAAC3NzaC1lZDI1NTE5AAAAIFwC7txsUAuXxHM6EpTThcSTgrne5x0qDErkefBG+d7J ci@build
```

bash

```
ssh -p 2222 -i ~/.ssh/id_ed25519 nextdoor@127.0.0.1 'ls /'
```

```
forced-command-ok
nextdoor
```

`ls /` не выполнилась: `command=` подменяет любую запрошенную команду на фиксированную, и это стандартный способ дать CI одну узкую операцию вместо шелла. Другие опции: `from="10.20.0.0/16"` (только с этих адресов), `restrict` (выключить всё необязательное — PTY, forwarding, X11), `permitopen="db.internal:5432"`

(проброс только к одной цели), `expiry-time="20270101"`. Ограничения проверяет сервер, поэтому они работают, даже если клиент не настроен.

~/.ssh/config

Блоки `Host` применяются по порядку, первый совпавший выигрывает по каждому параметру, `Host *` в конце задаёт значения по умолчанию.

```
Host bastion
    HostName 203.0.113.10
    User deploy
    IdentityFile ~/.ssh/id_ed25519

Host db
    HostName 10.20.0.15
    User postgres
    ProxyJump bastion
    LocalForward 15432 127.0.0.1:5432

Host *
    ServerAliveInterval 30
    ControlMaster auto
    ControlPath ~/.ssh/cm/%r@%h:%p
    ControlPersist 10m
```

Проверить результат подстановок можно без подключения:

```bash
ssh -G db | grep -E '^(hostname|user|port|proxyjump|localforward)'
```

```
user postgres
hostname 10.20.0.15
port 22
localforward 15432 [127.0.0.1]:5432
proxyjump bastion
```

`ssh -G` печатает итоговую конфигурацию со всеми значениями по умолчанию — единственный надёжный способ узнать, какой ключ и порт будут использованы.

`ProxyJump` (короткая форма `-J`) — SSH внутри SSH: клиент подключается к бастиону и просит его открыть TCP-канал до целевого хоста. Целевой хост видит соединение с бастиона, бастион не видит ваших ключей.

`ControlMaster`/`ControlPersist` убирают рукопожатие на каждое подключение: первое соединение становится мастером и держит сокет, последующие идут через него.

```
без мультиплексирования: 91 ms, 92 ms, 98 ms
с ControlMaster:         106 ms, 10 ms, 8 ms
```

На канале с RTT 80 мс экономия — сотни миллисекунд на команду, а `rsync` из тысячи файлов превращается из минуты в секунды. Цена: сокет живёт после выхода, а при смене сети мастер «висит» — `ssh -O exit db` закрывает его.

Туннели: -L, -R, -D

Три ключа решают три разные задачи.

Ключ	Что делает	Где слушает	Типичная задача
`-L`	локальный порт → адрес с точки зрения сервера	ваша машина	достучаться до БД за бастионом
`-R`	порт на сервере → адрес с вашей машины	удалённая машина	показать локальный сервис наружу
`-D`	SOCKS5-прокси через SSH	ваша машина	пустить браузер или `curl` через сервер

Локальный проброс: соединение к `127.0.0.1:19080` уходит в SSH-канал, а TCP к `127.0.0.1:8080` открывает уже удалённая сторона.

bash

```
python3 -m http.server 8080 --bind 127.0.0.1 &
ssh -f -N -p 2222 -i ~/.ssh/id_ed25519 -L 19080:127.0.0.1:8080 nextdoor@127.0.0.1
ss -tlnp | grep 19080
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:19080/
```

```
LISTEN 0 128 127.0.0.1:19080 0.0.0.0:* users:(("ssh",pid=595634,fd=5))
200
```

`-N` — не выполнять команду, только проброс, `-f` — уйти в фон после аутентификации. Слушает именно `ssh`, а не удалённый сервис, и по умолчанию только на loopback.

Шаблон «база за бастионом» — тот же механизм: `ssh -N -L 15432:db.internal:5432 bastion`, затем `psql -h 127.0.0.1 -p 15432`. Важно: `db.internal` резолвится **на бастионе**, поэтому локальный DNS о такой записи может не знать — это нормально.

Динамический проброс превращает SSH в SOCKS5-прокси:

bash

```
ssh -f -N -D 11080 nextdoor@127.0.0.1
curl -s --socks5 127.0.0.1:11080 -o /dev/null -w '%{http_code}\n'
    https://example.com
```

```
200
```

DNS по умолчанию резолвится на стороне прокси (`--socks5-hostname` в curl) — это и нужно, когда внутренние имена видны только изнутри.

Обратный проброс (`-R`) нужен, когда сервис у вас, а смотреть на него будут с сервера: `ssh -R 8080:127.0.0.1:3000 user@remote` открывает порт на удалённой машине и ведёт к вашему `localhost:3000`. По умолчанию он слушает только loopback удалённой машины; снаружи — только при `GatewayPorts yes`.

scp, sftp, rsync

`scp` с OpenSSH 9.0 работает поверх SFTP, а не собственного legacy-протокола; старый режим остался под флагом `-O`. Разница практическая: современный `scp` умеет то же, что `sftp`, но не умеет докачивать и синхронизировать.

bash

```
scp -P 2222 ./dump.sql nextdoor@127.0.0.1:/tmp/
rsync -avz --progress -e 'ssh -p 2222' ./site/ nextdoor@127.0.0.1:/srv/site/
```

`scp` копирует целиком и всегда; `rsync` сравнивает и передаёт разницу, умеет `--partial` (докачка), `--delete` (удалить лишнее на приёмнике) и `--dry-run` (показать план, ничего не делая). Для больших каталогов и повторяющихся выкладок `rsync` быстрее на порядок, но требует установки на обеих сторонах. Косая черта в конце источника (`./site/`) означает «содержимое каталога», без неё — «каталог целиком»; это самая частая ошибка первой выкладки.

sshd_config: что менять

Настройки сервера — `/etc/ssh/sshd_config` (в Debian и Ubuntu бывает ещё `/etc/ssh/sshd_config.d/*.conf`, подключаемый через `Include`). Минимальный набор для машины в интернете:

```
PermitRootLogin no
PasswordAuthentication no
```

```
KbdInteractiveAuthentication no
MaxAuthTries 3
LoginGraceTime 20
AllowUsers deploy
```

`PermitRootLogin no` запрещает вход под root полностью (`prohibit-password` — только по ключу, значение по умолчанию с OpenSSH 7.0). `PasswordAuthentication no` убирает подбор пароля как класс атаки и требует, чтобы ключи работали у всех, кому доступ нужен. `AllowUsers` — белый список: всё, что не перечислено, не входит, даже с правильным ключом.

Проверять конфиг нужно до перезапуска, иначе опечатка оставит вас без доступа:

bash

```
sudo sshd -t
sudo sshd -T | grep -E '^(permitrootlogin|passwordauthentication|kbdinteractiveauth
    entication)'
```

```
permitrootlogin no
passwordauthentication no
kbdinteractiveauthentication no
```

`sshd -t` — синтаксическая проверка, `sshd -T` — дамп итоговой конфигурации: он печатает и то, чего в файле нет, подставляя значения по умолчанию, поэтому правду показывает именно `-T`. Изменения применяются через `systemctl reload ssh`, а не `restart`: reload перечитывает конфиг, не разрывая текущие сессии.

`Match`-блоки делают исключения без второго демона:

```
Match Address 10.20.0.0/16
    PasswordAuthentication yes

Match User backup
    ForceCommand /usr/local/bin/backup-only.sh
```

Первый блок оставляет пароли для внутренней сети, второй запирает служебную учётную запись в одну команду. `Match` действует до конца файла или до следующего `Match`, поэтому порядок блоков — часть логики.

`fail2ban` в одном абзаце: демон читает журнал (`/var/log/auth.log` или журнал systemd) по шаблону `[sshd]`, считает неудачные попытки с одного адреса и на время банит его правилом nftables или iptables. С ключами и выключенным паролем он не столько защищает, сколько уменьшает шум и трафик ботов;

обязательный параметр — `ignoreip` со своими сетями, иначе ошибка в конфиге клиента блокирует вам доступ.

Диагностика

Уровни подробности: `-v` — этапы подключения, `-vv` — плюс алгоритмы и попытки ключей, `-vvv` — всё, включая побайтовый обмен. Начинать всегда со второго.

bash

```
ssh -T git@github.com
git@github.com: Permission denied (publickey).
```

`ssh -T` отключает PTY (так требует GitHub, который шелла не даёт). `Permission denied (publickey)` означает, что сервер не принял ни один ключ: ключа нет в `authorized_keys`, не тот файл или не те права. При успехе GitHub печатает приветствие с логином и отдельно сообщает, что шелл не предоставляется, — это нормальный результат.

Отпечаток ключа сервера проверяется без подключения:

bash

```
ssh-keyscan -t ed25519 github.com | ssh-keygen -lf -
256 SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU github.com (ED25519)
```

В `known_hosts` записи по умолчанию хэшированы, и `ssh-keygen -F host` находит строку, не выдавая, к каким ещё серверам вы подключались. При смене ключа хоста правильная реакция — `ssh-keygen -R host` и сверка нового отпечатка, а не отключение `StrictHostKeyChecking`.

Типичные ошибки

1. **Права 0644 на приватном ключе.** `ssh` отвечает `Permissions 0644 for '...' are too open` и ключ не использует. Механизм: клиент требует, чтобы файл не читался группой и остальными, иначе любой пользователь машины мог бы подписывать вашим ключом.
2. `PermitRootLogin no` **без второго администратора.** После reload доступа нет ни у кого. Механизм: настройка применяется к новым сессиям, а текущая не спасает после выхода; лечится

`sshd -t`, отдельной учётной записью и открытой сессией до reload.

3. **Проброс агента на бастион (`-A`).** Ключ фактически доступен root бастиона. Механизм: сокет агента принадлежит вашей учётной записи, а root читает чужие сокеты, и подпись выполняется без вашего участия.
4. `command=` **без** `no-pty`**.** Получается «полушелл»: команда подменена, но PTY выдан. Механизм: `command=` ограничивает запуск, но не запрещает запрос псевдотерминала — его запрещает отдельная опция.
5. `-L 5432:localhost:5432` **при базе на другом хосте.** Туннель открывается, а соединение не идёт. Механизм: адрес назначения резолвится и открывается на удалённой стороне, поэтому `localhost` — это бастион.
6. **Проверка отпечатка тем же каналом.** Отпечаток берут `ssh-keyscan` с того же сервера, который подозревают. Механизм: подмена хоста даёт согласованный ответ в обоих запросах; сверять нужно с консолью или документацией провайдера.

Чек-лист

- Приватные ключи ed25519, с парольной фразой, права 600.
- У служебных ключей в `authorized_keys` есть `from=`, `command=` или `restrict`.
- Проброс агента `-A` не используется; для прыжков настроен `ProxyJump`.
- В `~/.ssh/config` есть `ServerAliveInterval`, `IdentitiesOnly yes` и блоки `Host` для каждого окружения.
- `sshd -t` проходит, `sshd -T` показывает ожидаемые `PasswordAuthentication no` и `AllowUsers`.
- Вход по паролю выключен, root закрыт, для изменений есть вторая учётная запись с sudo.
- Отказ диагностирован по `ssh -vv`: видно, какой ключ предложен и принят ли он.

Упражнения

1. Сгенерируйте ключ ed25519, добавьте его на тестовый sshd и добейтесь входа. Затем закройте вход по паролю и объясните по `ssh -vv`, почему подключение с неверным ключом отклонено.
2. Настройте в `~/.ssh/config` два окружения (`staging`, `prod`) с разными ключами и `IdentitiesOnly yes`. Проверьте через `ssh -G`, что для каждого используется свой ключ.
3. Поднимите HTTP-сервер на `127.0.0.1:8080`, пробросьте его на `19080` через `ssh -L` и объясните, почему `ss -tlnp` показывает процесс `ssh`, а не сервер.
4. Ограничьте ключ в `authorized_keys` так, чтобы он мог только читать файл через `command=`, и проверьте, что произвольная команда не выполняется и PTY не выдаётся.
5. Настройте `ControlMaster` с `ControlPersist 10m`, измерьте время первой и второй команды, затем добавьте `ProxyJump` через второй sshd и посчитайте, сколько соединений в цепочке.

Ответы и разбор к этим упражнениям — в приложении D, часть 5.

Глава 6. Сети: от кабеля до IP-пакета

Сеть начинается с двух медных пар и заканчивается номером порта в ядре. Между ними — четыре преобразования: байты превращаются в кадр Ethernet, кадр несёт IP-пакет, пакет несёт TCP-сегмент, и всё это на каждом шаге переписывается заново. Понимание этих уровней отвечает на вопросы, которые иначе решаются гаданием: почему домашний сервер не виден из интернета, почему `ping` проходит, а `curl` висит, почему после подключения VPN перестал открываться сайт. В этой главе — MAC и ARP, IPv4 и IPv6, разбор маски на живом примере, MTU и фрагментация, таблица маршрутов и чтение `tcpdump` по байтам.

Инкапсуляция

Данные приложения (HTTP-запрос) попадают в TCP-сегмент, тот — в IP-пакет, пакет — в кадр Ethernet, кадр — в электрические сигналы. Каждый уровень добавляет свой заголовок и ничего не знает о содержимом предыдущего. Коммутатор читает только Ethernet-заголовок, маршрутизатор — только IP-заголовок, и это разделение труда позволяет менять Wi-Fi на витую пару, не трогая ни строчки прикладного кода.

Важно, что адресация на каждом уровне своя: MAC-адрес отвечает на вопрос «какому устройству в этом сегменте», IP-адрес — «на каком хосте в интернете», порт — «какому процессу на этом хосте». Пакет за время пути меняет MAC-адреса на каждом маршрутизаторе, но IP-адреса источника и назначения остаются неизменными (кроме NAT).

Ethernet и MAC

Кадр Ethernet (Ethernet II) выглядит так: 6 байт MAC-адрес получателя, 6 байт отправителя, 2 байта EtherType (0x0800 — IPv4, 0x86DD — IPv6, 0x0806 — ARP), полезная нагрузка от 46 до 1500 байт и 4 байта контрольной суммы FCS. Итого минимум 64

байта: если данных меньше 46, кадр добивается нулями (padding). Максимум 1518 байт — отсюда и MTU 1500.

MAC-адрес — 48 бит, обычно записан как `d0:17:c2:93:74:8d`. Первые три байта — идентификатор производителя (OUI), последние три — серийный номер. Адрес `ff:ff:ff:ff:ff:ff` — широковещательный (broadcast): его принимают все устройства в сегменте. Коммутатор (switch) ведёт таблицу «MAC → порт» и пересылает кадр только в нужный порт; хаб, если он ещё где-то жив, повторял бы всем.

ARP: как узнать MAC по IP

Отправить IP-пакет в локальной сети можно только зная MAC получателя. Отображение «IP → MAC» даёт протокол ARP (Address Resolution Protocol): хост кричит в broadcast «кто такой 192.168.0.240?», владелец отвечает своим MAC. В IPv6 ту же роль играет NDP (Neighbor Discovery) поверх ICMPv6.

Реальный захват запроса и ответа на машине с адресом 192.168.0.75:

bash

```
sudo tcpdump -ni enp3s0 -c 2 -e 'arp'
```

```
19:49:21.115527 d0:17:c2:93:74:8d > ff:ff:ff:ff:ff:ff, ethertype ARP (0x0806),
    length 42:
    Ethernet (len 6), IPv4 (len 4), Request who-has 192.168.0.240 tell
        192.168.0.75, length 28
19:49:22.120679 d0:17:c2:93:74:8d > ff:ff:ff:ff:ff:ff, ethertype ARP (0x0806),
    length 42:
    Ethernet (len 6), IPv4 (len 4), Request who-has 192.168.0.240 tell
        192.168.0.75, length 28
```

Ответа нет — адрес свободен, и вторая строка через секунду показывает, что ARP повторяется. Кадр при этом 42 байта: 14 байт Ethernet + 28 байт ARP, и до минимума в 64 его добивает padding на уровне драйвера — в `tcpdump` padding обычно не виден.

Таблица соседей — это и есть ARP-кэш:

```
192.168.0.1 dev enp3s0 lladdr 48:29:e4:51:49:ae REACHABLE
192.168.0.5 dev enp3s0 lladdr 2a:11:e2:02:01:1f STALE
fe80::1 dev enp3s0 lladdr 48:29:e4:51:49:ae router STALE
```

Состояния: `REACHABLE` — недавно подтверждён, `STALE` — запись есть, но давно не проверялась (при отправке ядро сначала

перепроверит), `FAILED` — ответа нет. Отсюда типичный симптом: «пингую новый хост — первые пакеты теряются» — это уходит ARP-запрос, а `ping` уже считает время.

IPv4: заголовок и адресация

IPv4-адрес — 32 бита, четыре октета. Заголовок — минимум 20 байт. Разберём настоящий SYN-пакет, снятый на той же машине:

bash

```
sudo tcpdump -ni enp3s0 -c 1 -e -vvv -X 'tcp[tcpflags] & tcp-syn != 0'
```

```
19:49:24.487959 d0:17:c2:93:74:8d > 48:29:e4:51:49:ae, ethertype IPv4 (0x0800),
    length 74:
    (tos 0x0, ttl 64, id 11413, offset 0, flags [DF], proto TCP (6), length 60)
    192.168.0.75.42528 > 149.154.167.220.443: Flags [S], seq 313029685, win 64240,
    options [mss 1460,sackOK,TS val 582142566 ecr 0,nop,wscale 7], length 0
    0x0000:  4500 003c 2c95 4000 4006 0fbd c0a8 004b
    0x0010:  959a a7dc a620 01bb 12a8 7435 0000 0000
    0x0020:  a002 faf0 fe98 0000 0204 05b4 0402 080a
    0x0030:  22b2 ca66 0000 0000 0103 0307
```

Первые 20 байт — IP-заголовок, и каждый читается глазами:

Байты	Значение	Смысл
`45`	version 4, IHL 5	IPv4, заголовок 5×4 = 20 байт
`00`	TOS	приоритет/класс обслуживания, обычно 0
`003c`	total length 60	весь пакет 60 байт: 20 IP + 40 TCP
`2c95`	id 11413	идентификатор для сборки фрагментов
`4000`	flags DF	«не фрагментировать»
`40`	TTL 64	сколько маршрутизаторов ещё можно пройти
`06`	protocol	6 = TCP (17 = UDP, 1 = ICMP)
`0fbd`	checksum	контрольная сумма заголовка
`c0a8 004b`	192.168.0.75	источник
`959a a7dc`	149.154.167.220	назначение

Дальше `a620 01bb` — порты 42528 и 443, `12a8 7435` — sequence number, `a002` — data offset 10 (40 байт TCP-заголовка с опциями) и

флаг SYN, `faf0` — окно 64240. Опции в конце: `0204 05b4` — MSS 1460, `0402` — SACK permitted, `080a` + два timestamp, `0103 0307` — window scale 7 (окно умножается на 128).

Строка `cksum 0xfe98 (incorrect -> 0x32d3)` в таких захватах — норма: контрольную сумму считает сетевая карта при отправке (checksum offload), и `tcpdump` видит ещё не посчитанное поле. Это частая причина ложной паники при разборе дампов.

Подсети и CIDR

Маска отделяет сетевую часть адреса от хостовой. CIDR-запись `/26` означает «26 старших бит — сеть, 6 младших — хосты». Разберём на примере `192.168.10.77/26`:

```python
import ipaddress

net = ipaddress.ip_network("192.168.10.77/26", strict=False)
print(net)                              # сеть
print(net.netmask, net.hostmask)        # маска и обратная маска
print(net.broadcast_address)            # широковещательный
print(net.num_addresses, net.num_addresses - 2)
print(list(net.hosts())[0], list(net.hosts())[-1])
```

```
192.168.10.64/26
255.255.255.192 0.0.0.63
192.168.10.127
64 62
192.168.10.65 192.168.10.126
```

Что произошло: 26 бит маски — это 24 бита «192.168.10» плюс два бита последнего октета. Последний октет 77 = `01001101`. Два старших бита — `01`, то есть подсеть номер 1 из четырёх: сети идут шагом 64 (0, 64, 128, 192). Отсюда `192.168.10.64` — адрес сети (все хостовые биты нули), `192.168.10.127` — broadcast (все единицы), а usable-диапазон — 65–126, 62 адреса. Сам `.77` попадает внутрь, сеть определена верно.

Полезно помнить шпаргалку по последнему октету:

Префикс	Маска	Шаг сетей	Хостов
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14

Частные диапазоны (RFC 1918): `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. Они не маршрутизируются в интернете — на этом и строится NAT.

IPv6: 128 бит и никакого NAT

IPv6-адрес — 128 бит, записывается восемью группами по четыре hex-цифры, с сокращениями: `2001:0db8:0000:0000:0000:0000:0000:0001` → `2001:db8::1` (одна группа `::` заменяет последовательность нулей). Адресов хватает всем, поэтому NAT в IPv6 не нужен — каждый хост получает глобальный адрес, а фильтрация делается файрволом.

Три вещи, которые чаще всего ломают IPv6 в эксплуатации: link-local адреса `fe80::/10` требуют указания интерфейса (`ping6 fe80::1%eth0`), минимальный MTU для IPv6 — 1280 байт (меньше нельзя, и это ломает туннели), а автоконфигурация (SLAAC) выдаёт адрес без DHCP, поэтому «нет адреса» и «нет маршрута» — разные диагнозы.

MTU, фрагментация и PMTUD

MTU (maximum transmission unit) — максимальный размер полезной нагрузки кадра: 1500 байт для Ethernet. Из него вычитаются заголовки: 20 байт IPv4 + 20 байт TCP = MSS 1460 (для IPv6 — 1440). Если приложение пишет больше, ядро режет данные на сегменты по MSS.

Фрагментация IP — крайняя мера: маршрутизатор с меньшим MTU может разбить пакет, но собирать его будет получатель, и потеря одного фрагмента убивает весь пакет. Поэтому современный подход — выставлять флаг DF и узнавать MTU

через ICMP «fragmentation needed» (PMTUD, path MTU discovery). Проверить путь руками:

bash

```
ping -c 1 -M do -s 1472 192.168.0.1
```

```
PING 192.168.0.1 (192.168.0.1) 1472(1500) bytes of data.
1480 bytes from 192.168.0.1: icmp_seq=1 ttl=64 time=0.716 ms
```

`-s 1472` — размер данных, плюс 8 байт ICMP и 20 байт IP дают ровно 1500. `-M do` запрещает фрагментацию: если бы MTU был меньше, пакет не ушёл бы, и мы увидели бы `Frag needed and DF set`. Это самый быстрый способ найти реальный MTU в туннеле.

Классическая «чёрная дыра MTU»: TCP-соединение устанавливается (мелкие пакеты проходят), `curl` зависает на загрузке большого ответа, потому что ICMP-сообщение о нужном MTU где-то фильтруется, и PMTUD не работает. Симптом диагностируется именно `ping -M do -s` с постепенным уменьшением размера: 1472, 1452 (IPsec), 1412 (OpenVPN), 1392 (VXLAN) — типичные значения для туннелей.

Маршрутизация: таблица маршрутов

Решение «куда отправить пакет» принимается по таблице маршрутов, по правилу longest prefix match: выбирается маршрут с самым длинным совпавшим префиксом.

bash

```
ip route
```

```
default via 192.168.0.1 dev enp3s0 proto dhcp src 192.168.0.75 metric 1002
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1
172.18.0.0/16 dev br-20bb41b6f2ce proto kernel scope link src 172.18.0.1
192.168.0.0/24 dev enp3s0 proto dhcp scope link src 192.168.0.75 metric 1002
```

Читается так: пакет на `192.168.0.5` совпадёт с `/24` (32 бита префикса) — уйдёт напрямую в `enp3s0`, минуя шлюз. Пакет на `172.17.0.2` совпадёт с docker-маршрутом и уйдёт в bridge. Всё остальное — только с `default`, через 192.168.0.1. `proto kernel` означает, что маршрут создан ядром при поднятии интерфейса, `proto dhcp` — получен от DHCP-клиента, `scope link` — «получатель доступен напрямую, шлюз не нужен».

Проверить решение для конкретного адреса:

bash

```
ip route get 203.0.113.10
203.0.113.10 via 192.168.0.1 dev enp3s0 src 192.168.0.75 uid 1000
    cache
```

Одна команда отвечает на вопрос «через какой интерфейс и с каким source IP пойдёт трафик» — это первое, что нужно проверить, когда сервис «не видит» внешний адрес из-за политики маршрутизации или второго интерфейса.

NAT: почему домашний сервер не виден извне

NAT (network address translation) подменяет адрес источника при выходе в интернет: пакет от `192.168.0.75` уходит с публичным адресом роутера, а роутер хранит в таблице conntrack запись «внутренний 192.168.0.75:42528 ↔ внешний 88.99.12.4:51820». Обратный пакет переписывается в обратную сторону.

Следствие: снаружи не существует адреса, по которому можно постучаться внутрь. Соединение может начать только внутренний хост, потому что запись в conntrack создаётся исходящим пакетом. Чтобы сервер был виден из интернета, нужно либо пробросить порт (DNAT: правило «порт 443 на роутере → 192.168.0.75:443»), либо использовать обратный туннель, либо иметь публичный IPv4/IPv6.

Отдельная напасть — CGNAT у провайдера: абонент получает `100.64.0.0/10`, за которым сидит ещё один NAT. Проброс порта там невозможен в принципе, и именно это ломает домашние видеозвонки и self-hosted сервисы. Проверка простая: если адрес на интерфейсе роутера попадает в `100.64.0.0/10`, вы за CGNAT.

Ещё одна деталь: NAT — это состояние. Перезагрузка роутера рвёт все соединения, потому что таблица conntrack потеряна. Именно поэтому TCP keepalive и переподключения в клиентах — не роскошь.

tcpdump: читаем пакет

Правила фильтрации пишутся в синтаксисе BPF: `host`, `port`, `net`, `tcp[tcpflags]`.

bash

```
sudo tcpdump -ni any -c 5 'tcp port 443 and host 203.0.113.10'
sudo tcpdump -ni eth0 -w /tmp/cap.pcap 'port 53'      # в файл для Wireshark
sudo tcpdump -r /tmp/cap.pcap -nn -A 'port 53'        # чтение из файла
```

Практический порядок разбора: сначала `-nn` (не резолвить имена и порты — резолв сам ходит в DNS и врёт в таймингах), затем `-e` (показать MAC), затем `-vvv -X` (детали и hex). Если видно `Flags [S]` без ответного `[S.]` — проблема на стороне получателя или в фильтре; если видно `[S.]` и повторные `[S]` — потерян ACK или переполнена очередь accept (об этом в следующей главе).

Типичные ошибки

1. **Путать MAC и IP при отладке.** `tcpdump` показывает MAC-адреса шлюза, а не конечного получателя: пакет в интернет уходит на MAC роутера. «Не тот MAC» в дампе — это норма, а не подмена.
2. **Считать, что NAT — это файрвол.** NAT скрывает хосты, но не защищает сервисы: проброшенный порт открыт всем, а UPnP может открыть его без спроса.
3. **Забывать про MTU в туннелях.** Инкапсуляция (IPsec, VXLAN, WireGuard) съедает 40–80 байт, и пакеты 1500 перестают проходить. Лечится уменьшением MTU или MSS clamping, а не «перезагрузкой».
4. **Игнорировать ICMP.** Блокировка ICMP ломает PMTUD и диагностику: соединения «висят» на больших передачах, а `ping` молчит, хотя хост жив.
5. **Держать два default route с одинаковой метрикой.** Ядро выберет один, и трафик будет уходить «не туда» непредсказуемо; вторая половина соединений умрёт после переключения.
6. **Диагностировать по имени хоста.** Пока идёт отладка сети, используйте IP: имя добавляет DNS в уравнение и удваивает число гипотез.

Чек-лист

`[ ]` `ip -br a`, `ip route`, `ip neigh` сняты и сохранены до инцидента.

- [ ] Для каждого сервиса известно, слушает ли он `0.0.0.0` или `127.0.0.1`.
- [ ] Проверено, нет ли CGNAT (`100.64.0.0/10`) перед планированием проброса портов.
- [ ] Для всех туннелей зафиксированы MTU и MSS clamp.
- [ ] ICMP не заблокирован полностью: разрешены `destination-unreachable` и `packet-too-big`.
- [ ] `tcpdump` запускается с `-nn`, иначе тайминги врут.
- [ ] Подсети спланированы с запасом: /26 на 30 устройств — плохая идея.

Упражнения

1. Снимите ARP-запрос к свободному адресу своей сети и объясните, почему кадр уходит на `ff:ff:ff:ff:ff:ff`.
2. Для `10.8.130.200/22` определите адрес сети, broadcast и число хостов — сначала вручную, потом через `ipaddress`.
3. Найдите реальный MTU до внешнего хоста через `ping -M do` с уменьшением размера. Сравните с MTU интерфейса.
4. Снимите `tcpdump -X` одного TCP SYN и разберите все поля заголовка по hex-дампу.
5. Соберите схему своей домашней сети: адреса, шлюзы, где NAT, какие порты проброшены.

Ответы и разбор к этим упражнениям — в приложении D, часть 1.

Глава 7. TCP и UDP: надёжность, порты, сокеты

IP умеет доставить пакет «куда-нибудь и как-нибудь»: он не гарантирует ни порядка, ни доставки, ни целостности потока. TCP поверх этого делает вид, что между двумя процессами проложен надёжный двунаправленный канал, хотя под ним — сеть, теряющая пакеты. Почти все «странные» сетевые симптомы в эксплуатации объясняются деталями этой надстройки: соединение висит 127 секунд при недоступном хосте, приложение копит CLOSE_WAIT, сервер под нагрузкой теряет SYN, а `TIME_WAIT` на клиенте не даёт открыть новый сокет. В этой главе — рукопожатие, окна и congestion control, состояния из `ss`, порты, backlog, UDP и по одному сокету на Python и Go.

Что именно гарантирует TCP

TCP (RFC 9293) даёт четыре вещи: доставку без потерь (потерянный сегмент будет передан снова), порядок байт, обнаружение дубликатов и управление потоком. Модель данных — поток байт (byte stream), а не сообщения: `send()` дважды по 100 байт может прийти одним `recv()` на 200. Поэтому в прикладных протоколах нужен свой фрейминг — длина в заголовке или разделитель.

Нумерация идёт по байтам: sequence number (seq) — номер первого байта сегмента, acknowledgement (ack) — номер следующего ожидаемого байта. Начальный номер выбирается случайно (ISN) — это защита от подмены и от старых дубликатов, блуждающих по сети. Отсюда «надёжность»: получив сегмент с дыркой, получатель продолжает подтверждать последний непрерывный байт, а отправитель по таймеру (RTO, retransmission timeout) пересылает потерянное.

Трёхстороннее рукопожатие

Соединение открывается тремя пакетами:

1. клиент → сервер: `SYN`, seq = x, опции (MSS, window scale, SACK, timestamps);
2. сервер → клиент: `SYN-ACK`, seq = y, ack = x+1;
3. клиент → сервер: `ACK`, ack = y+1. С этого момента соединение ESTABLISHED у обеих сторон.

Зачем три, а не два: обе стороны должны подтвердить и свой ISN, и чужой. Два пакета не позволяют одновременно синхронизировать номера в обе стороны.

Если сервер недоступен (пакеты уходят в никуда), клиент повторяет SYN с экспоненциальной задержкой. При `tcp_syn_retries=6` (значение по умолчанию) попытки идут примерно через 1, 2, 4, 8, 16, 32 и 64 секунды — итого около 127 секунд до `ETIMEDOUT`. Именно поэтому «сайт не открывается две минуты» — не баг приложения, а таймер ядра: `connect()` без своего таймаута ждёт столько, сколько скажет TCP.

Рукопожатие видно в захвате: в прошлой главе мы снимали `Flags [S]` и через секунду — повторный `[S]` с тем же seq. Для сервера важен другой счётчик: полуоткрытые соединения (SYN queue) и завершённые, ожидающие `accept()` (accept queue). Если accept queue переполнена, ядро по умолчанию молча выбрасывает SYN — клиент видит ретрансмиссии, а в `nstat` растёт `TcpExtListenOverflows`.

Окно и скользящее окно

Получатель не может бесконечно принимать данные: у него есть буфер. Размер свободного места объявляется в поле window каждого ACK — это receive window (rwnd). Отправитель держит «окно» неподтверждённых байт и может отправить не больше, чем разрешает минимум из rwnd и своего congestion window (cwnd).

Скользящее окно (sliding window) означает, что окно не фиксировано: по мере подтверждения старых байт оно сдвигается вправо, открывая место новым. Если приложение не читает из сокета, буфер заполняется, окно становится нулевым (zero window), и отправитель останавливается. Это самая частая

причина «соединение живо, но данные не идут»: сервер принял запрос и завис в обработке, не вызывая `recv()`.

Изначально окно — 16 бит (максимум 65535), чего мало для быстрых каналов: при RTT 100 мс это 5 Мбит/с. Расширение (RFC 7323) добавляет множитель: в нашем дампе была опция `wscale 7`, то есть окно умножается на $2^7 = 128$, и максимум становится 8 МБ.

Congestion control: коротко о главном

Потери в сети — не всегда вина получателя; чаще это перегрузка на пути. Поэтому отправитель держит ещё один лимит — cwnd, и растит его осторожно:

- **slow start** — cwnd начинается с 10 сегментов и удваивается каждый RTT, пока не упрётся в порог;
- **congestion avoidance** — дальше растёт на один сегмент за RTT (аддитивно), а при потере уменьшается вдвое (мультипликативно). Это AIMD — та самая «аддитивная скорость, мультипликативное снижение».

Алгоритмы отличаются реакцией на потери: `cubic` (по умолчанию в Linux) считает cwnd функцией от времени с момента последней потери и хорошо ведёт себя на длинных толстых каналах; `bbr` измеряет пропускную способность и RTT и не реагирует на потери как на сигнал перегрузки — на каналах с потерями (Wi-Fi, мобильные сети, дальние маршруты) даёт заметно больше.

Посмотреть всё это вживую:

bash

```
ss -tin state established
```

```
0  0  192.168.0.75:35576  149.154.167.220:443
    cubic wscale:10,7 rto:280 rtt:77.215/17.871 mss:1228 pmtu:1500 cwnd:10
    bytes_sent:2652 bytes_acked:2653 bytes_received:6182 segs_out:13 segs_in:11
    send 1272292bps pacing_rate 2544576bps delivery_rate 355488bps app_limited
    busy:160ms rcv_space:14480 snd_wnd:73728 rcv_wnd:79488
```

Читается так: `rto:280` — текущий таймер ретрансмиссии в миллисекундах, `rtt:77.215/17.871` — средний RTT и отклонение, `cwnd:10` — десять сегментов в полёте (соединение только что открылось, slow start ещё не разогнался), `app_limited` —

приложение само не даёт больше данных, `snd_wnd:73728` — сколько разрешил получатель. Если `cwnd` большой, а `delivery_rate` низкий, узкое место не в TCP; если `snd_wnd` близок к нулю — тормозит получатель.

Состояния TCP и ss

Соединение проходит 11 состояний. Практически важны шесть:

Состояние	Что означает	Когда это проблема
`LISTEN`	сокет ждёт подключений	не тот адрес (`127.0.0.1` вместо `0.0.0.0`)
`SYN-SENT`	клиент отправил SYN	много таких — сервер не отвечает
`ESTAB`	соединение работает	много — норма для нагруженного сервиса
`CLOSE-WAIT`	получили FIN, но `close()` не вызван	растёт — приложение не закрывает сокеты
`TIME-WAIT`	закрыли первыми, ждём 60 с	тысячи на клиенте — кончаются порты
`FIN-WAIT-2`	отправили FIN, ждём FIN от второй стороны	висит — вторая сторона не закрывает

bash

```
ss -tan
```

```
State      Recv-Q Send-Q Local Address:Port     Peer Address:Port
LISTEN     0      4096       0.0.0.0:8080          0.0.0.0:*
LISTEN     0      128          0.0.0.0:22            0.0.0.0:*
ESTAB      0      0       192.168.0.75:35576  149.154.167.220:443
ESTAB      0      0       192.168.0.75:51422  170.246.55.162:9230
TIME-WAIT  0      0       192.168.0.75:42512  149.154.167.220:443
```

Для `LISTEN` колонка `Send-Q` — это размер accept queue (текущий backlog), для остальных — байты в буфере отправки. Именно поэтому `ss -lnt` с ненулевым `Send-Q` на слушающем сокете — признак, что приложение не успевает вызывать `accept()`.

Диагностическое правило: **CLOSE-WAIT — всегда баг приложения**, оно получило FIN и не закрыло дескриптор; ядро тут ничего не сделает, пока не кончится лимит файловых дескрипторов. **TIME-WAIT — норма протокола**, и бороться с ним настройками ядра нужно очень осторожно.

bash

```
ss -s

Total: 320
TCP:   151 (estab 49, closed 85, orphaned 0, timewait 77)
```

TIME_WAIT и почему он болит

Сторона, закрывшая соединение первой, уходит в `TIME_WAIT` на 2×MSL — в Linux это 60 секунд (`tcp_fin_timeout` относится к другому, FIN-WAIT-2). Причина двойная: нужно дождаться запоздавших сегментов старого соединения, чтобы они не попали в новое с теми же номерами портов, и нужно, чтобы последний ACK дошёл, если он потерялся.

Боль возникает не от самого состояния, а от исчерпания портов. Сокет однозначно определяется четвёркой (src IP, src port, dst IP, dst port). Клиент, делающий тысячи коротких соединений к одному серверу, тратит эфемерные порты:

bash

```
cat /proc/sys/net/ipv4/ip_local_port_range

32768   60999
```

Это 28 232 порта. При 500 соединениях в секунду и 60 секундах в TIME_WAIT нужно 30 000 одновременно занятых портов — пределов больше нет, и `connect()` начинает падать с `Cannot assign requested address`. Правильные лечения по порядку: переиспользовать соединения (keep-alive в HTTP-клиенте, пул соединений), увеличить диапазон портов, и только потом `net.ipv4.tcp_tw_reuse=1` (безопасно для исходящих соединений при включённых timestamps). `tcp_tw_recycle` в современных ядрах удалён — он ломал NAT и был опасен.

Серверную сторону TIME_WAIT обычно не трогают: там закрывает клиент, а если закрывает сервер — это признак, что приложение рвёт соединение первым (например, `Connection: close` вместо keep-alive).

Порты, сокеты и backlog

Порт — 16 бит, 0–65535. Порты ниже 1024 требуют привилегий в Unix (либо `CAP_NET_BIND_SERVICE`). Диапазоны: 0–1023 системные,

1024–49151 регистрируемые, 49152–65535 динамические; Linux использует для исходящих свой диапазон из `ip_local_port_range`.

Сокет в ядре — структура, привязанная к четвёрке адресов. Сервер делает `socket()` → `bind()` → `listen(backlog)`. Аргумент `listen()` задаёт длину очереди завершённых соединений, но реальный лимит — минимум из backlog и `net.core.somaxconn` (в наших системах 4096). Отдельно есть очередь полуоткрытых (`tcp_max_syn_backlog`, 512 по умолчанию).

Что происходит при переполнении accept queue, зависит от `tcp_abort_on_overflow`: по умолчанию (0) сервер молча выбрасывает ACK, клиент считает соединение установленным и отправляет данные, они теряются, и клиент видит загадочный таймаут при полностью «живом» сервере. Счётчики:

bash

```
nstat -az | grep -E 'ListenOverflows|ListenDrops'
```

```
TcpExtListenOverflows           0                  0.0
TcpExtListenDrops               0                  0.0
```

Ненулевой `ListenOverflows` — прямое указание: либо приложение медленно принимает, либо backlog мал. В nginx это лечится `listen 80 backlog=4096;` и увеличением числа worker-процессов.

UDP: где он уместен

UDP (RFC 768) даёт только мультиплексирование по портам и контрольную сумму. Нет рукопожатия, нет подтверждений, нет порядка, нет управления потоком — есть датаграмма, которая либо доедет, либо нет. Заголовок — 8 байт против 20 у TCP.

Уместен он там, где задержка важнее гарантий, либо где надёжность реализуют выше:

- DNS — один вопрос, один ответ, переспросить дешевле, чем держать соединение;
- QUIC (основа HTTP/3) — надёжность и управление перегрузкой сделаны в user space поверх UDP;
- голос и видео — опоздавший пакет бесполезен, повторять его незачем;

- метрики (StatsD), логи, syslog — потеря одной точки не критична;
- DHCP, NTP, SNMP, WireGuard.

Две ловушки UDP. Первая — размер: датаграмма больше MTU фрагментируется, а потеря одного фрагмента убивает всю датаграмму; практический безопасный размер — около 1200 байт. Вторая — отсутствие управления перегрузкой: приложение, шлющее UDP в цикле, само создаёт перегрузку, и сеть отвечает потерями; отсюда и ограничение скорости на стороне приложения.

Keepalive и таймауты

Молчащее TCP-соединение может быть мёртвым: партнёр ушёл, а FIN не пришёл (обрыв Wi-Fi, рестарт NAT, падение процесса без `close()`). Ядро обнаруживает это keepalive-пробами: первый запрос через `tcp_keepalive_time` (7200 с), затем каждые `tcp_keepalive_intvl` (75 с), до `tcp_keepalive_probes` (9) попыток — итого больше двух часов.

Для сервиса это неприемлемо долго, поэтому:

```python
import socket

s = socket.create_connection(("203.0.113.10", 5432), timeout=3)
s.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIDLE, 30)     # первая проба через 30 с
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPINTVL, 10)    # далее каждые 10 с
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPCNT, 3)       # 3 промаха — соединение мертво
s.settimeout(5)                                               # таймаут на recv/send
s.sendall(b"PING\n")
print(s.recv(64))
```

Важная деталь: `settimeout()` в Python — не «таймаут соединения», а таймаут каждой операции ввода-вывода. Без него `recv()` висит вечно, даже если TCP-соединение полумёртвое. И `timeout` в `create_connection` ограничивает только установку.

В Go принято управлять дедлайнами явно:

```go
func fetch(addr string) ([]byte, error) {
    conn, err := net.DialTimeout("tcp", addr, 3*time.Second)
    if err != nil {
        return nil, fmt.Errorf("dial: %w", err)
```

```
    }
    defer conn.Close()

    conn.SetDeadline(time.Now().Add(5 * time.Second)) // и на чтение, и на запись
    if _, err := conn.Write([]byte("PING\n")); err != nil {
        return nil, fmt.Errorf("write: %w", err)
    }
    buf := make([]byte, 64)
    n, err := conn.Read(buf) // вернётся по дедлайну, даже если данных нет
    if err != nil {
        return nil, fmt.Errorf("read: %w", err)
    }
    return buf[:n], nil
}
```

Дедлайн в Go — абсолютное время, а не длительность, поэтому его нужно пересчитывать перед каждой операцией; иначе второй `Read` упадёт мгновенно с `i/o timeout`.

Типичные ошибки

1. **Ждать `connect()` без своего таймаута.** Ядро ждёт до ~127 секунд при `tcp_syn_retries=6`, и всё это время пул потоков или event loop занят. Таймаут прикладного уровня обязателен и всегда меньше внешнего.
2. **Считать TCP потоком сообщений.** Два `send()` не равны двум `recv()`: границы размываются, особенно при ретрансмиссиях. Нужен фрейминг.
3. **Лечить CLOSE_WAIT настройками ядра.** Это всегда незакрытый дескриптор в коде; `tcp_fin_timeout` тут ни при чём.
4. **Бороться с TIME_WAIT через `tcp_tw_recycle`.** В современных ядрах опции нет; попытки включить её через sysctl молча ничего не делают, а старые рецепты из интернета опасны для NAT.
5. **Ставить `listen(128)` и забыть про `somaxconn`.** Реальный backlog — минимум из двух; при переполнении SYN выбрасывается, и клиенты видят ретрансмиссии вместо ошибки.
6. **Гнать UDP-датаграммы больше MTU.** Фрагментация умножает вероятность потери: при 1% потерь на пакет датаграмма из трёх фрагментов теряется почти в 3% случаев.
7. **Игнорировать `ss` при отладке «медленного» сервиса.** Растущая accept queue (`Send-Q` в `LISTEN`) объясняет задержки

быстрее, чем профайлер.

Чек-лист

- [ ] У всех клиентов есть явные таймауты: connect, чтение, общий дедлайн запроса.
- [ ] HTTP-клиенты используют keep-alive или пул соединений, а не новое соединение на запрос.
- [ ] `ss -s` и `nstat -az` снимаются при каждом инциденте с задержками.
- [ ] Метрика `TcpExtListenOverflows` под наблюдением и равна нулю.
- [ ] `somaxconn` и backlog приложения согласованы и проверены под нагрузкой.
- [ ] Для UDP-сервисов размер датаграммы ≤ 1200 байт.
- [ ] Keepalive включён на уровне приложения или ОС для долгоживущих соединений.
- [ ] `TIME_WAIT` на клиенте измерен, а не предполагаем: `ss -tan state time-wait | wc -l`.

Упражнения

1. Захватите `tcpdump` рукопожатие к внешнему HTTPS-хосту и найдите ISN, MSS, window scale. Сравните ISN с предыдущим соединением.
2. Запустите сервер, который принимает соединение и не читает данные; залейте в него мегабайт и посмотрите `ss -tin` — найдите zero window.
3. Воспроизведите CLOSE_WAIT: сервер, не вызывающий `close()` после получения FIN. Убедитесь, что состояние живёт минутами.
4. Сгенерируйте 30 000 коротких соединений в секунду к одному хосту и получите `Cannot assign requested address`. Объясните арифметику и почините пулом соединений.
5. Напишите UDP-эхо-сервер на 50 строк и потеряйте 10% пакетов через `tc netem`. Сравните с TCP на том же канале.

Ответы и разбор к этим упражнениям — в приложении D, часть 1.

Глава 8. DNS: как имя становится адресом

DNS — самая недооценённая часть инфраструктуры: пока он работает, его никто не замечает, а когда ломается, «падает всё». При этом сбои почти всегда выглядят одинаково — «сайт не открывается», — хотя причин ровно четыре: не тот сервер, не та зона, не тот кэш или не тот поисковый домен. Эта глава объясняет иерархию и рекурсию, типы записей и TTL, а затем разбирает реальные аварии: split-horizon, приоритет `/etc/hosts`, `search`-домены и `ndots`. В конце — пошаговая диагностика, которую можно выполнять сверху вниз, не гадая.

Иерархия: корень, TLD, авторитативные серверы

DNS — распределённая база данных, организованная как дерево (RFC 1034, RFC 1035). Точка в конце имени — корень; `.com`, `.org`, `.ru` — домены верхнего уровня (TLD); дальше — домены второго уровня и поддомены. Каждый узел дерева обслуживается своими авторитативными серверами (authoritative nameservers): они не кэшируют, а хранят факты о зоне.

Корневых серверов 13 логических имён (`a.root-servers.net` … `m.root-servers.net`), и каждое развёрнуто как сеть anycast-инстансов по миру — поэтому «13 серверов» не значит 13 машин. Корень знает только одно: кто обслуживает каждый TLD. Сервер `.com` знает, кто обслуживает `example.com`. Авторитативный сервер `example.com` знает адреса своих хостов.

Такое разделение означает: ни один сервер не содержит всю базу, и любое имя находится за конечное число шагов — обычно три-четыре.

Рекурсия и итерация

Клиент почти никогда не общается с корнем. Он спрашивает свой резолвер (recursive resolver) — и просит «дай готовый ответ» (рекурсивный запрос, флаг `RD` — recursion desired). Дальше

резолвер сам обходит дерево итеративно: спросил корень → получил адрес сервера `.com` → спросил `.com` → получил адрес авторитативного сервера → спросил его → получил `A`-запись. Каждый ответ по пути кэшируется на время TTL.

Разница принципиальна для эксплуатации:

- рекурсивный запрос делает клиент, и за него платит резолвер (нагрузка, кэш, приватность);
- итеративный запрос резолвер делает к каждому серверу по очереди, и именно эти пакеты видит `dig +trace`.

Публичные рекурсивные резолверы (например, 1.1.1.1 или 8.8.8.8) — это сервисы, которые делают итеративную работу за вас. Внутренний резолвер компании делает то же самое плюс знает про внутренние зоны.

Типы записей

Тип	Что содержит	Пример использования
`A`	IPv4-адрес	`api.example.com → 203.0.113.10`
`AAAA`	IPv6-адрес	тот же хост по IPv6
`CNAME`	псевдоним (другое имя)	`www → lb.example.net`
`MX`	почтовый сервер + приоритет	`10 mail.example.com`
`TXT`	произвольный текст	SPF, DKIM, верификация домена
`SRV`	хост + порт + приоритет + вес	`_https._tcp.example.com`
`NS`	авторитативные серверы зоны	делегирование
`SOA`	первичный сервер, serial, таймеры	служебная запись зоны

Правила, которые чаще всего нарушают: `CNAME` нельзя ставить в apex-узел зоны (`example.com`), если там есть `MX`, `NS` или `SOA`; `CNAME` не может указывать на другой `CNAME` в ответе (клиент получит цепочку, которую должен развернуть сам резолвер); `MX` без приоритета не существует — число обязательно.

`TXT` сейчас — рабочая лошадка проверок владения доменом и почтовой аутентификации: SPF ограничен 10 DNS-lookup'ами, и «слишком много SPF-записей» — реальная причина, по которой письма попадают в спам.

TTL и кэширование

TTL (time to live) — сколько секунд ответ можно держать в кэше. Значение выбирается между свежестью и нагрузкой: 300 секунд — типично для сервисов, где адрес может смениться при деплое; 3600–86400 — для стабильной инфраструктуры; 60 — для аварийного переключения.

Практическое следствие: **вы не можете мгновенно изменить DNS**. Если TTL был 86400, старый адрес будет жить в кэшах сутки, и никакие правки в зоне этого не отменят. Поэтому перед миграцией TTL понижают заранее — за время, равное старому TTL.

Кэшируются и отрицательные ответы (NXDOMAIN, NODATA) — на время из поля `minimum` записи `SOA` зоны, но не более суток по рекомендациям. Это объясняет странность «записи нет, а резолвер упорно говорит, что её нет, хотя я её только что создал»: он закэшировал отказ.

Резолвер: путь от `getaddrinfo` до ответа

Приложение редко вызывает DNS напрямую: `curl`, `ping`, `requests` идут через `getaddrinfo()`, а тот — через NSS (name service switch). Порядок источников задан в `/etc/nsswitch.conf`:

```
hosts:          files dns
```

`files` — это `/etc/hosts`, и он проверяется **раньше** DNS. Одна строка в hosts перекрывает любую зону, и это самая частая причина «у меня работает, у коллег нет» (или наоборот).

Дальше `dns` читает `/etc/resolv.conf`: список `nameserver`, `search`-домены и `options ndots`. В системах с systemd-resolved там обычно `nameserver 127.0.0.53` — это локальный stub-резолвер, который занят портом 53 на loopback и проксирует запросы в реальный upstream (и

умеет кэш, DNSSEC, split-DNS по линкам). Проверить, кто именно слушает:

bash

```
ss -lunp | grep ':53'
```

Если в ответе `127.0.0.53%lo:53`, DNS обслуживает systemd-resolved, и правки в `/etc/resolv.conf` бессмысленны — файл перезапишется. Настраивать нужно через `resolvectl` или drop-in для `systemd-resolved.service`.

dig: читаем ответ

`dig` — единственный инструмент, который показывает DNS целиком: вопрос, ответ, флаги, время. Реальный вывод на машине, где stub-резолвер слушает только IPv4:

bash

```
dig example.com A
```

```
;; communications error to ::1#53: connection refused
;; communications error to ::1#53: connection refused
;; communications error to ::1#53: connection refused
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 21055
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;example.com.           IN  A

;; ANSWER SECTION:
example.com.        300 IN  A   104.20.23.154
example.com.        300 IN  A   172.66.147.243

;; Query time: 623 msec
;; SERVER: 127.0.0.1#53(127.0.0.1) (UDP)
;; MSG SIZE  rcvd: 72
```

Разбор. Три строки `communications error to ::1#53` — не ошибка DNS, а следствие того, что резолвер в `/etc/resolv.conf` числится и как `::1`, и как `127.0.0.1`, а слушает только IPv4: `dig` честно пытается по IPv6, получает отказ и переходит к следующему адресу. Итог — 623 мс вместо 5 мс, и это ровно тот случай, когда «DNS тормозит» означает «резолвер настроен на два адреса, один из которых мёртв».

`status: NOERROR` — имя существует; `NXDOMAIN` означал бы, что не существует; `SERVFAIL` — что зона сломана или сервер не отвечает. Флаги: `qr` — это ответ, `rd` — рекурсия запрошена, `ra` — сервер её поддерживает, `ad` — данные аутентифицированы DNSSEC. `ANSWER: 2` — две A-записи, TTL 300 у обеих. `SERVER: 127.0.0.1#53` — кто ответил (важно: это резолвер, а не авторитативный сервер).

Короткие формы для скриптов:

bash

```
dig +short example.com A
dig +noall +answer example.com AAAA
dig @1.1.1.1 example.com A +norecurse     # спросить конкретный сервер без рекурсии
dig -x 8.8.8.8 +short                     # обратная зона
getent hosts example.com                  # то, что увидит приложение (через NSS)
```

`getent hosts` — единственная команда, которая показывает результат ровно так, как его получит приложение: с учётом `/etc/hosts` и `nsswitch.conf`. Если `dig` и `getent` расходятся — виноват hosts или NSS, а не DNS.

DoH и DoT

Классический DNS идёт по UDP/53 открытым текстом: провайдер и любой посредник видит, какие имена вы спрашиваете. Два протокола закрывают это:

- **DoT** (DNS over TLS, RFC 7858) — TLS на порту 853, отдельный порт, легко блокируется и легко детектируется;
- **DoH** (DNS over HTTPS, RFC 8484) — запросы внутри HTTPS на порту 443, неотличимы от обычного веб-трафика, поэтому переживает блокировки, но и фильтровать его сложнее.

Цена — потеря видимости для корпоративной сети: внутренние зоны, split-horizon и блокировки рекламы перестают работать, если браузер ушёл на публичный DoH мимо корпоративного резолвера. Поэтому в компаниях DoH часто отключают политикой, а DoT оставляют для своих резолверов с пином сертификата.

Типичные аварии

Split-horizon. Одна и та же зона отдаётся по-разному изнутри и снаружи: внутри `api.example.com → 10.0.5.20`, снаружи — публичный адрес. Ломается, когда внутренний клиент спрашивает публичный резолвер: он получает внешний адрес, и трафик уходит через NAT наружу и обратно. Симптом — соединение устанавливается, но медленно и через «не тот» путь; проверяется `dig @внутренний_резолвер` против `dig @1.1.1.1`.

`/etc/hosts` **важнее DNS.** Классика при переезде: админ когда-то добавил запись для теста, и теперь один сервер из десяти ходит на старый IP. `getent hosts` показывает правду, `dig` — нет.

`search`**-домены и** `ndots`**.** В `resolv.conf` есть `search corp.example.com` и по умолчанию `ndots:1` (в Kubernetes — `ndots:5`). Имя `api` без точки сначала пробуется как `api.corp.example.com`, потом как `api.` — то есть несколько запросов подряд. Отсюда «первый запрос всегда медленный» и лишние NXDOMAIN в логах резолвера. Лечится завершающей точкой (`api.example.com.`) или настройкой `ndots`.

Кэш живёт дольше, чем правда. TTL 86400 и переключение на новый сервер — это сутки переходного периода, в течение которых часть клиентов ходит на старый адрес. Планировать заранее.

CNAME в apex. Попытка поставить `example.com CNAME lb.provider.com` рядом с `MX` даёт либо ошибку загрузки зоны, либо тихо отброшенную запись — в зависимости от провайдера.

Диагностика по шагам

Порядок важен: каждый следующий шаг исключает предыдущий класс причин.

bash

```
getent hosts api.example.com                          # 1. что видит приложение (hosts + nsswitch)
dig api.example.com A +noall +answer +stats           # 2. что отвечает локальный резолвер
cat /etc/resolv.conf; ss -lunp | grep ':53'           # 3. кто настроен и не сломан ли stub
dig @ns1.example.net api.example.com A +norecurse     # 4. отвечает ли авторитативный сервер
dig @1.1.1.1 api.example.com A +short                 # 5. не кэш ли виноват
dig NS example.com +short; dig SOA example.com +short # 6. делегация у родителя
```

Логика чтения: если шаг 1 даёт один адрес, а шаг 2 — другой, виноват `/etc/hosts`. Если шаг 2 отвечает, а шаг 4 — нет, сломан авторитативный сервер или его сеть. Если шаг 4 отвечает

правильно, а шаг 2 — нет, вы упёрлись в кэш или в split-horizon. Если не отвечает ничего, начиная с шага 2, — проблема в резолвере или в фильтрации UDP/53.

Ещё один приём: `dig +trace` выполняет итеративный обход с вашей машины и показывает, на каком уровне делегирование расходится с ожиданием. Если родитель отдаёт `NS` на сервер, которого нет, — это видно сразу, без ожидания TTL.

Типичные ошибки

1. **Диагностировать через `ping` по имени.** `ping` смешивает ICMP, DNS и маршрутизацию: три гипотезы вместо одной. Сначала `dig`, потом `ping` по IP, потом `curl`.
2. **Забыть про `/etc/hosts`.** Он проверяется раньше DNS, и `dig` его не видит. Расхождение `dig` и `getent` — почти всегда hosts.
3. **Снижать TTL после инцидента, а не до.** Кэши с TTL 86400 не обновятся от вашей правки; понижать TTL нужно минимум за старый TTL до переключения.
4. **Ставить `ndots:5` без нужды.** Каждое короткое имя превращается в пять запросов; на Kubernetes это заметная доля трафика и задержек.
5. **Использовать `CNAME` для apex.** Ломает `MX`, `NS` и `SOA`; для apex нужны `A`/`AAAA` или ALIAS/ANAME у провайдера.
6. **Считать `SERVFAIL` проблемой клиента.** Это ответ сервера: зона не загрузилась, DNSSEC не сошёлся или авторитативный сервер недоступен. Клиент тут ничего не исправит.
7. **Пускать DoH в обход корпоративного резолвера.** Внутренние имена перестают резолвиться, а split-horizon — работать; браузер при этом выглядит «сломанным» только в офисе.

Чек-лист

- [ ] TTL для критичных записей ≤ 300, и это проверено перед каждым переключением.

- [ ] Внутренние зоны резолвятся только через внутренние резолверы; split-horizon проверен с обеих сторон.
- [ ] `/etc/hosts` на всех серверах проверен на «временные» записи.
- [ ] `ndots` и `search` осознанно заданы; в Kubernetes проверено их влияние на latency.
- [ ] Есть мониторинг: время ответа резолвера, доля `SERVFAIL`, доступность авторитативных NS.
- [ ] Настроен и проверен DNSSEC, а `ad`-флаг в `dig` действительно появляется.
- [ ] DoH/DoT политика определена: либо запрещены, либо идут через контролируемый резолвер.
- [ ] Для каждого домена известно, кто держит зону и как её править в 3 часа ночи.

Упражнения

1. Сравните `dig +short`, `getent hosts` и `nslookup` для одного имени. Найдите случай, где они расходятся, и объясните почему.
2. Понизьте TTL до 60, подождите старый TTL, смените A-запись и замерьте, через сколько её увидят разные резолверы.
3. Организуйте split-horizon на своём резолвере: один и тот же домен отдаёт разные адреса изнутри и снаружи.
4. Включите `ndots:5` и замерьте число DNS-запросов при обращении к десяти коротким именам.
5. Разберите `dig +trace` для домена и найдите точку, где делегирование уходит на неожиданный сервер.

Ответы и разбор к этим упражнениям — в приложении D, часть 1.

Глава 9. HTTP/1.1: запросы, ответы, заголовки

В главе 7 мы разобрали TCP как байтовый поток, в главе 8 — как имя превращается в адрес. Теперь над этим фундаментом появляется протокол, с которым DevOps-инженер сталкивается ежедневно. После этой главы вы сможете прочитать сырой HTTP-обмен по байтам, объяснить разницу между 401 и 403, 301 и 308, и воспроизвести любой запрос руками через `nc` и `curl -v`.

Сообщение как текст

HTTP/1.1 (RFC 9112) — текстовый протокол. Сообщение состоит из четырёх частей: стартовая строка, блок заголовков, пустая строка, тело. Разделитель строк — CRLF (`\r\n`, 0x0D 0x0A), но многие клиенты шлют одиночный LF, а серверы терпят: расхождение опасно, если один узел считает разделителем `\r\n\r\n`, а другой — `\n\n`, и они по-разному нарежут поток на сообщения.

Стартовая строка

```
GET /api/v1/users?limit=20&offset=40 HTTP/1.1
```

Три элемента: метод, request-target, версия. Формы request-target: origin-form (`/path?query`), absolute-form — для forward-прокси, authority-form — для `CONNECT`, asterisk-form (`*`) — для `OPTIONS *`. Ответ выглядит как `HTTP/1.1 200 OK`; причина декоративна — клиенты ориентируются на код.

Заголовки

Заголовок — строка `Имя: значение`. Имя регистронезависимо, значение — нет. Порядок не значим, кроме повторяющихся имён: несколько заголовков с одним именем эквивалентны одному со значениями через запятую, но `Set-Cookie` исключён. Перенос значения на следующую строку (obs-fold) устарел: сервер должен ответить 400.

Тело и его границы

TCP не хранит границ сообщений, поэтому конец тела задаётся явно: `Content-Length` (ровно столько байт), `Transfer-Encoding: chunked` (блоки с размером в hex до нулевого блока) или закрытие соединения (тело до EOF, только в ответах с `Connection: close`). Если признаков нет, тело пустое.

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Transfer-Encoding: chunked

1a
This is the first chunk.
15
And the second one.
0
```

`1a` — 26 в hex: 24 символа плюс CRLF; `15` — 21 байт. Ноль закрывает тело.

Отдельный случай — заголовок `Expect: 100-continue`. Клиент, собирающийся отправить большое тело, сначала спрашивает разрешения: сервер отвечает промежуточным `100 Continue` и только потом клиент льёт мегабайты. Если сервер сразу видит, что запрос будет отвергнут (нет авторизации, превышен лимит), он отвечает `417 Expectation Failed` или финальным 4xx, и трафик не тратится. Большинство клиентов ждут ответа не дольше секунды и потом отправляют тело всё равно — поэтому полагаться на экономию нельзя, но для загрузок в сотни мегабайт механизм заметен.

Ключевое правило: `Transfer-Encoding` **приоритетнее** `Content-Length`. Сообщение с обоими заголовками — признак контрабанды запроса (request smuggling): frontend читает `Content-Length`, backend — `Transfer-Encoding`, и второй запрос прячется внутри первого. Прокси обязан ответить 400 и разорвать соединение.

Методы и идемпотентность

Безопасный метод не меняет состояние сервера. **Идемпотентный** при повторе даёт тот же эффект, что при однократном вызове. Второе важнее: на нём держатся автоматические ретраи.

Метод	Тело	Безопасный	Идемпотентный	Кэшируется
GET	нет	да	да	да
HEAD	нет	да	да	да
OPTIONS	опц.	да	да	нет
PUT	да	нет	да	нет
DELETE	опц.	нет	да	нет
POST	да	нет	нет	нет
PATCH	да	нет	нет	нет

Идемпотентность — про эффект на сервере, а не про ответ: два `DELETE /users/42` дадут `204` и `404`, но состояние системы то же, что после одного. Поэтому потерявший соединение клиент вправе повторить `DELETE`, а `POST` — нет: получится дубль. Обходной путь — заголовок `Idempotency-Key`: сервер запоминает ключ и на повтор возвращает сохранённый результат (де-факто стандарт, черновик IETF). `PATCH` идемпотентным быть не обязан: `{"op":"increment"}` при повторе увеличит счётчик дважды.

Три метода стоят особняком и почти не встречаются в прикладном коде. `OPTIONS` запрашивает список поддерживаемых методов и возможностей — его использует CORS-предпроверка (preflight), когда браузер перед кросс-доменным POST спрашивает разрешение. `TRACE` возвращает запрос обратно в теле ответа и в продакшене должен быть выключен: он позволяет прочитать заголовки, добавленные промежуточными узлами. `CONNECT` превращает соединение в туннель — так работают HTTPS через прокси и `ssh -o ProxyCommand`; в HTTP/2 он устроен иначе, через расширение `CONNECT`.

Коды состояний

Первая цифра задаёт класс: 1xx — информирование, 2xx — успех, 3xx — перенаправление, 4xx — ошибка клиента, 5xx — ошибка сервера. Класс важнее конкретного числа.

301, 302, 307, 308

```
HTTP/1.1 301 Moved Permanently
Location: https://example.com/new-path
Content-Length: 0
```

301 обещает постоянство, 302 — временность, но важнее другое: старые коды разрешают клиенту **изменить метод**. Браузер, получив 301 или 302 на POST, повторяет запрос как GET и теряет тело. Коды 307 и 308 менять метод запрещают, поэтому перевод http→https — это 308. По RFC 9111 коды 301 и 308 кэшируются по умолчанию, 302 и 307 — только с явным `Cache-Control`.

304 Not Modified

Ответ на условный запрос: клиент прислал `If-None-Match: "9f2c1ab7e4"`, сервер подтвердил, что версия та же, и вернул `304 Not Modified` с `ETag` и `Cache-Control`, но без тела — в этом смысл. Заголовки, которые были бы в 200, повторяются; 304 допустим только для GET и HEAD.

401 и 403

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer realm="api", error="invalid_token"
Content-Type: application/json
Content-Length: 57

{"error":"token_expired","message":"renew via /auth/refresh"}
```

`401` — «я не знаю, кто вы»: данных нет, они просрочены или неверны; обязательный спутник — `WWW-Authenticate`. `403` — «знаю, кто вы, но нельзя»: токен валиден, роль не та, повторная аутентификация не поможет. Приём безопасности — отдавать 404 вместо 403 там, где секретен сам факт существования объекта.

409 и 422

`409 Conflict` — запрос противоречит текущему состоянию: `PUT` с устаревшей версией (оптимистичная блокировка) или занятый email. `422 Unprocessable Content` (в RFC 9110 переименован из «Unprocessable Entity») — синтаксис корректен, JSON разобран, но семантика неверна: `{"age": -5}`, пустое обязательное поле. 400 — «не смог разобрать», 422 — «разобрал и отказываюсь».

429 Too Many Requests

`429` (появился в RFC 6585) — превышен лимит запросов; в теле обычно `limit`, `window`, `retry_after`, а в заголовках — `Retry-After` (а также поля `RateLimit-*` из черновика IETF, поддержанные пока не везде).

Без `Retry-After` клиент начнёт ретраить немедленно и усилит перегрузку. Тот же `Retry-After` обязателен для `503`: сервис, который отдаёт при перегрузке `500`, не даёт балансировщику понять, что узел временно плох, и трафик продолжает литься.

5xx: 500, 502, 503, 504

Коды 5xx — единственный класс, который говорит «виноват сервер», и различать их важно, потому что они указывают на разные узлы.

`500 Internal Server Error` — исключение в приложении. Ответ пришёл, но приложение не смогло обработать запрос: упал ORM-запрос, не распарсился конфиг, сработал `assert`. Лечится логами приложения, а не прокси.

`502 Bad Gateway` — прокси не смог получить корректный ответ от бэкенда: соединение отклонено (`connection refused`), разорвано посередине (`upstream prematurely closed connection`) или ответ не является HTTP. Это почти всегда признак того, что процесс не слушает порт или падает при старте.

`503 Service Unavailable` — бэкенд жив, но не может обслуживать: пул соединений исчерпан, идёт graceful shutdown, балансировщик исключил все узлы из ротации (в nginx — `no live upstreams`).

`504 Gateway Timeout` — бэкенд не ответил за отведённое время. Запрос мог выполняться и завершиться успешно уже после того, как прокси отдал клиенту 504, — именно поэтому долгие операции нельзя делать синхронными.

Разбор инцидента начинается с вопроса «какой именно код»: 502 и 504 генерирует прокси, 500 — приложение, 503 — либо приложение, либо балансировщик. По одному коду видно, в какой половине системы искать.

Ключевые заголовки

`Host` — обязателен, ровно один: выбирает виртуальный хост, сертификат и конфиг. Нет `Host` — 400; два разных `Host` — тоже 400, иначе кэш можно отравить, подсунув прокси один хост, а бэкенду другой.

`Content-Length` — длина тела **в байтах**: «Привет» — 12 байт в UTF-8, а не 6.

`Accept-Encoding` — список алгоритмов сжатия (`gzip`, `br`, `zstd`). Сервер выбирает один, указывает в `Content-Encoding` и обязан добавить `Vary: Accept-Encoding`, иначе кэш отдаст gzip-версию клиенту, просившему brotli (`zstd` — RFC 8878).

`Content-Type` — тип тела с необязательным `charset`; `application/json; charset=utf-8` избыточен (JSON всегда UTF-8 по RFC 8259). `Date` участвует в свежести: расхождение часов ломает кэширование незаметно.

Соединения: keep-alive и head-of-line blocking

В HTTP/1.0 соединение обслуживало один запрос: TCP-рукопожатие, два-три round-trip на TLS, запрос, ответ, закрытие — и так на каждую картинку. HTTP/1.1 сделал постоянные соединения поведением по умолчанию: при RTT 40 мс каждый ресурс стоил бы около 120 мс только на установку.

Но у постоянного соединения жёсткий предел: **в один момент обрабатывается один запрос**, ответы возвращаются в порядке отправки. Если первый запрос — отчёт на 3 секунды, остальные ждут, хотя сервер мог бы отдать их мгновенно. Это блокировка начала очереди (head-of-line blocking). Pipelining не прижился, и браузеры обходят предел силой — до шести соединений на origin и шейдинг доменов.

Вторая проблема — гонка при закрытии. Сервер держит idle-соединение 60–75 секунд (в nginx `keepalive_timeout` по умолчанию 75 с) и закрывает его; запрос, отправленный в этот момент, уходит в закрывающийся сокет, и пользователь видит `connection reset`, хотя сервер здоров. Лечится тем, что серверный таймаут больше клиентского, плюс ретрай идемпотентных запросов.

Практика: разбор сырого запроса

Через nc

`nc` отправляет в сокет ровно те байты, что вы дали, — лучший способ увидеть протокол без прослоек.

bash

```
printf 'GET / HTTP/1.1\r\nHost: example.com\r\nUser-Agent:
    curl/8.5.0\r\nConnection: close\r\n\r\n' | nc example.com 80
```

`\r\n` в конце каждой строки обязателен (иначе многие серверы ответят `400`), пустая строка `\r\n\r\n` закрывает заголовки, а `Connection: close` заставляет сервер закрыть соединение, чтобы `nc` завершился сам.

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1256
Connection: close
Server: ECAcc (nyd/D13E)
Date: Wed, 28 Oct 2026 09:12:44 GMT

<!doctype html><html><head><title>Example Domain</title>...
```

`Content-Length: 1256` — дальше ровно 1256 байт HTML. С `HTTP/1.0` в стартовой строке сервер ответит `HTTP/1.0 200 OK` и закроет соединение без заголовка `Connection`.

Через curl -v

Строки с `>` — что ушло, с `<` — что пришло, с `*` — диагностика curl.

bash

```
curl -v --http1.1 -o /dev/null https://httpbin.org/status/429
```

```
*   Trying 34.160.111.145:443...
* Connected to httpbin.org (34.160.111.145) port 443
* using HTTP/1.1
> GET /status/429 HTTP/1.1
> Host: httpbin.org
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 429 Too Many Requests
< Date: Wed, 28 Oct 2026 09:12:44 GMT
< Content-Type: text/html; charset=utf-8
< Content-Length: 0
< Server: gunicorn
<
* Connection #0 to host httpbin.org left intact
```

`Connection #0 ... left intact` — соединение осталось открытым и будет переиспользовано для следующего URL; версия в `User-Agent` соответствует вашей сборке curl. Флаги: `-I` (HEAD, только заголовки), `-L` (следовать редиректам), `-w '%{time_connect}`

`%{time_starttransfer}\n'` — расхождение метрик отделяет сетевую задержку от медленного приложения.

Практика: чтение access-лога

Код ответа бесполезен без контекста: тот же 502 означает «бэкенд не ответил» или «таймаут», и различить их можно только по времени upstream. Формат лога стоит расширить сразу.

nginx

```
log_format main '$remote_addr - $host "$request" $status $body_bytes_sent '
                'rt=$request_time urt=$upstream_response_time '
                'ua="$http_user_agent"';
access_log /var/log/nginx/access.log main;
```

```
203.0.113.24 - api.example.com "GET /v1/orders?limit=50 HTTP/1.1" 200 8421
    rt=0.043 urt=0.041 ua="curl/8.5.0"
203.0.113.24 - api.example.com "POST /v1/orders HTTP/1.1" 502 157 rt=0.008
    urt=0.007 ua="myapp/2.4"
203.0.113.51 - api.example.com "GET /v1/report HTTP/1.1" 504 176 rt=60.002
    urt=60.001 ua="Mozilla/5.0"
```

Три строки читаются по-разному. Первая — норма: 43 мс всего, 41 мс на бэкенде. Вторая — 502 за 7 мс: бэкенд отказал мгновенно, значит процесс не слушает порт или упал. Третья — 504 ровно на 60 секундах: сработал `proxy_read_timeout`, бэкенд продолжал считать отчёт. Одинаковой реакции на эти три случая быть не может.

`$request_time` включает время чтения тела запроса от клиента, `$upstream_response_time` — только время ответа бэкенда. Если первое сильно больше второго, медленный клиент или большая загрузка; если наоборот — проблема в приложении.

Типичные ошибки

1. `Content-Length` **в символах вместо байт.** В Python `len("Привет")` равен 6, а тело занимает 12 байт: клиент читает половину строки, парсер падает. Нужно `len(s.encode('utf-8'))`.
2. `Content-Length` **и** `Transfer-Encoding` **одновременно.** Приложение ставит длину, прокси добавляет chunked, узлы режут поток по-разному, и хвост трактуется как начало следующего запроса.

3. `200 OK` **с телом** `{"error": "..."}`**.** Мониторинг считает 2xx успехом, SLO не деградирует, клиентские библиотеки не возбуждают исключение: ошибка видна только по жалобам.
4. `302` **на POST в API.** Клиент меняет метод на GET и отбрасывает тело: операция молча не выполняется.
5. **Игнорирование** `Retry-After`**.** Ретрай через 100 мс не даёт серверу восстановиться — шторм ретраев усиливает исходную проблему.
6. **Чтение тела без лимита.** `Content-Length: 10000000000` или отправка по байту в секунду занимают воркер навсегда.
7. **Кэш ответа с** `Authorization` **без** `Vary` **и** `private`**.** Промежуточный кэш считает ключом только URL и отдаёт ответ одного пользователя другому. Механизм: заголовок `Authorization` не делает ответ приватным автоматически.

Чек-лист

- [ ] Запросы без `Host` и с несколькими `Host` получают 400 на периметре.
- [ ] Запросы с `Content-Length` и `Transfer-Encoding` одновременно отклоняются.
- [ ] Лимиты строки запроса и заголовков заданы (в nginx по умолчанию 8 КБ на строку).
- [ ] Таймауты чтения/записи меньше таймаута балансировщика — иначе 502 приходят от него.
- [ ] Редиректы, сохраняющие метод, отдают 307/308.
- [ ] Ошибки клиента возвращаются 4xx, а не 200 с полем `error`.
- [ ] Ответы 429 и 503 содержат `Retry-After`.

Упражнения

1. Отправьте через `nc` запрос без `Host` и запрос с двумя `Host`. Сравните коды и объясните поведение сервера.
2. Декодируйте chunked-тело вручную, отмечая границы блоков, и сверьте результат с `curl -s` без `--raw`.

3. Настройте в nginx `return 302` и `return 308` на один POST-эндпоинт. Проверьте `curl -v -X POST -d 'a=1'` и объясните судьбу метода и тела.
4. Сравните 100 запросов к одному хосту с переиспользованием соединения и без; объясните разницу числом рукопожатий.
5. Соберите из `access.log` статистику по кодам и `$upstream_response_time` за сутки через `awk` и объясните, какие эндпоинты дают 502 и почему.

Ответы и разбор к этим упражнениям — в приложении D, часть 2.

Глава 10. HTTP/2 и HTTP/3: мультиплексирование и QUIC

В главе 9 выяснилось, что HTTP/1.1 обрабатывает по одному запросу на соединение и заставляет браузер открывать до шести TCP-соединений к одному хосту. Каждое соединение — это рукопожатие, TLS, отдельный congestion window и отдельная очередь. HTTP/2 убрал это ограничение, но остался на TCP, а TCP сохранил собственную блокировку начала очереди. HTTP/3 переносит протокол на QUIC поверх UDP и решает уже транспортную проблему. После этой главы вы сможете объяснить, что именно изменилось на каждом уровне, проверить поддержку HTTP/2 и HTTP/3 в curl и nginx, и понять, почему в корпоративной сети HTTP/3 иногда «не работает».

Что не так с HTTP/1.1

Три проблемы HTTP/1.1 определяют весь дизайн последующих версий. Первая — блокировка начала очереди (head-of-line blocking) на уровне приложения: ответы обязаны идти в порядке запросов, поэтому один медленный отчёт задерживает сотню картинок. Вторая — накладные расходы на текст: каждый запрос несёт 500–800 байт заголовков открытым текстом, и на странице с сотней ресурсов это десятки килобайт. Третья — стоимость соединения: шесть параллельных TCP-соединений означают шесть congestion window, которые конкурируют друг с другом и в сумме используют канал хуже, чем одно соединение с правильным окном.

HTTP/2 (RFC 9113) атакует первые две проблемы и оставляет TCP как есть. HTTP/3 (RFC 9114) заменяет транспорт на QUIC (RFC 9000) и добивает третью.

Бинарный фрейминг

Главное изменение HTTP/2 — отказ от текста. Сообщение разбивается на фреймы (frame), каждый с заголовком фиксированной длины в 9 байт: 24 бита длины полезной

нагрузки, 8 бит типа, 8 бит флагов, 1 зарезервированный бит и 31 бит идентификатора потока. Типов фреймов десять: `HEADERS`, `DATA`, `SETTINGS`, `WINDOW_UPDATE`, `RST_STREAM`, `PRIORITY`, `PING`, `GOAWAY`, `CONTINUATION`, `PUSH_PROMISE`.

Двоичный формат означает, что парсер не ищет `\r\n`, не разбирает строки и не спотыкается о перенос строки внутри значения заголовка. Заголовки сжимаются целиком (см. HPACK), а тело передаётся фреймами `DATA` произвольного размера, обычно до 16 КБ.

Побочный эффект: HTTP/2 невозможно набрать руками в `nc`. Для отладки нужны инструменты, умеющие говорить на бинарном протоколе, — `curl`, `nghttp`, DevTools, `h2load`.

Мультиплексирование потоков

Поток (stream) — логический канал внутри одного TCP-соединения, у него свой идентификатор и своя последовательность фреймов. Клиент открывает потоки с нечётными номерами, сервер — с чётными; номера только растут. В одном соединении одновременно живут десятки потоков: браузер шлёт запросы на HTML, CSS, JS и картинки параллельно, а сервер отвечает в том порядке, в каком успевает.

Ограничения задаются через `SETTINGS`: `MAX_CONCURRENT_STREAMS` (типично 100–128), `INITIAL_WINDOW_SIZE` (по умолчанию 65 535 байт на поток). Управление потоком (flow control) двухуровневое — отдельно на поток и на соединение, и это частая причина «зависших» загрузок: сервер упёрся в окно, клиент не прислал `WINDOW_UPDATE`, передача встала, хотя TCP-канал свободен.

Важно понимать: мультиплексирование убирает блокировку **на уровне HTTP**, но не на уровне TCP. Если один TCP-сегмент потерян, ядро держит все последующие байты до повторной передачи, а в них лежат фреймы всех потоков сразу. На сетях с потерями 1–2 % HTTP/2 может работать хуже HTTP/1.1 с шестью соединениями, потому что там потеря блокировала только одно из шести. Именно этот аргумент и привёл к QUIC.

HPACK

Заголовки в HTTP/2 сжимаются алгоритмом HPACK (RFC 7541). Он использует три механизма: статическую таблицу из 61 частого заголовка (`:method: GET`, `:path: /`, `user-agent`), динамическую таблицу, которую обе стороны пополняют по ходу обмена, и кодирование Хаффмана для литеральных значений.

Результат: повторяющиеся заголовки занимают единицы байт вместо сотен. Но у динамической таблицы есть цена — состояние. Она общая для соединения и должна обновляться строго синхронно; при потере синхронизации соединение приходится разрывать (`GOAWAY` с `COMPRESSION_ERROR`). Это причина, по которой HTTP/2 плохо переносит балансировку без привязки к соединению: если запросы одного соединения попадают на разные бэкенды, динамические таблицы расходятся.

Псевдозаголовки `:method`, `:path`, `:scheme`, `:authority` заменяют стартовую строку. Заголовки `Connection`, `Keep-Alive`, `Transfer-Encoding` в HTTP/2 запрещены — соединением управляет сам протокол.

Приоритеты и server push

Приоритеты в первой редакции HTTP/2 задавались деревом зависимостей: каждый поток указывал родителя и вес. Идея хорошая, реализация — нет: браузеры строили разные деревья, серверы реализовывали приоритизацию по-разному, и на практике выигрыш был нестабильным. В RFC 9113 дерево зависимостей и фрейм `PRIORITY` исключены; остался только `Priority`-заголовок и механизм `PRIORITY_UPDATE`, а основная работа перешла к серверу, который сам решает, что отдать первым, ориентируясь на размер и тип ресурса.

Server push (`PUSH_PROMISE`) позволял серверу отправить ресурс до того, как клиент его попросил. На практике выигрыш съедался промахами: сервер пушил то, что уже было в кэше браузера, трафик рос, а кэш не использовался. Chrome убрал поддержку push в версии 106 в 2022 году, и в RFC 9113 механизм уже отсутствует. Современная замена — заголовок `103 Early Hints` со ссылками `preload`: сервер подсказывает, что понадобится, но решение остаётся за клиентом.

QUIC и HTTP/3

QUIC — транспортный протокол поверх UDP, который забирает у TCP функции надёжности и добавляет то, чего TCP не умеет. Главное: потоки в QUIC независимы. Потеря пакета с данными одного потока не блокирует остальные, потому что у каждого потока свой счётчик последовательности и своя повторная передача.

TLS 1.3 (RFC 8446) встроен в QUIC, а не лежит сверху: рукопожатие транспорта и рукопожатие TLS объединены, поэтому установка соединения занимает один round-trip вместо двух у TCP+TLS. Отсюда знакомые цифры: соединение по HTTP/3 устанавливается за 1 RTT, а при повторном подключении возможен 0-RTT — клиент отправляет данные в первом же пакете, используя сохранённый тикет.

0-RTT требует осторожности: ранние данные не защищены от повторного воспроизведения (replay). Поэтому в 0-RTT допустимо отправлять только идемпотентные запросы (GET, HEAD), и серверы часто ограничивают объём ранних данных.

Ещё две возможности, которых нет у TCP: миграция соединения (connection migration) — при смене IP (Wi-Fi → LTE) соединение не рвётся, потому что идентифицируется по Connection ID, а не по четвёрке адрес-порт; и шифрование всех заголовков, включая служебные, что затрудняет цензуру и инспекцию.

HTTP/3 (RFC 9114) — тонкая надстройка над QUIC: те же фреймы и потоки, но сжатие заголовков через QPACK (RFC 9204), где динамическая таблица вынесена в отдельные потоки, чтобы не блокировать данные при потере пакета.

Как проверить поддержку

Первое — версия протокола в curl. Флаг `--http2` требует HTTP/2, `--http3` — HTTP/3, а `-w '%{http_version}'` печатает фактически согласованную версию.

bash

```
curl -sS -o /dev/null --http2 -w 'HTTP/%{http_version} code=%{http_code}\n'
    https://www.cloudflare.com/
```

```
HTTP/2 code=200
```

Если сервер не поддерживает HTTP/2, curl с `--http2` не упадёт: по умолчанию он согласует протокол через ALPN и откатится на 1.1. Строгий режим — `--http2-prior-knowledge`, он говорит без ALPN сразу по HTTP/2 и падает, если сервер не готов.

bash

```
curl -sS -o /dev/null --http3 -w 'HTTP/%{http_version}\n'
    https://cloudflare-quic.com/
```

```
HTTP/3
```

HTTP/3 curl пробует только по явному флагу: по умолчанию он ходит по TCP. Проверить, что сервер анонсирует HTTP/3, можно по заголовку `alt-svc`:

bash

```
curl -sSI https://cloudflare-quic.com/ | grep -i alt-svc
```

```
alt-svc: h3=":443"; ma=86400
```

`ma=86400` — сколько секунд клиент может помнить анонс. Современные браузеры узнают о HTTP/3 ещё раньше — из DNS-записи типа HTTPS (RFC 9460), где параметр `alpn` перечисляет поддерживаемые протоколы.

Второе — конфигурация nginx. HTTP/2 включается директивой `http2 on;` в блоке `server` (в старых версиях — параметром `http2` у директивы `listen`):

nginx

```
server {
    listen 443 ssl;
    http2 on;
    server_name example.com;

    ssl_certificate     /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;

    location / {
        proxy_pass http://backend;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
    }
}
```

`proxy_http_version 1.1` и пустой `Connection` обязательны: без них nginx пойдёт к бэкенду по HTTP/1.0 и закроет соединение после

каждого ответа. HTTP/2 в nginx работает только с TLS — h2c (HTTP/2 без шифрования) не поддерживается.

Третье — ALPN на уровне TLS:

bash

```
openssl s_client -connect example.com:443 -alpn h2 -servername example.com
    </dev/null 2>/dev/null | grep "ALPN protocol"
```

```
ALPN protocol: h2
```

Если в ответе `ALPN protocol: http/1.1` или строка отсутствует, сервер HTTP/2 не предложил.

Практика: как выглядит HTTP/2 в curl -v

`curl -v` по HTTP/2 показывает псевдозаголовки и номер потока — по ним видно, что запрос ушёл не текстом, а фреймом `HEADERS` в потоке 1.

bash

```
curl -sv -o /dev/null https://www.cloudflare.com/ 2>&1 | sed -n '/ALPN/,$p'
```

```
* ALPN: offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* SSL certificate verify ok.
* using HTTP/2
* [HTTP/2] [1] OPENED stream for https://www.cloudflare.com:443/
* [HTTP/2] [1] [:method: GET]
* [HTTP/2] [1] [:scheme: https]
* [HTTP/2] [1] [:authority: www.cloudflare.com]
* [HTTP/2] [1] [:path: /]
* [HTTP/2] [1] [user-agent: curl/8.5.0]
* [HTTP/2] [1] [accept: */*]
> GET / HTTP/2
> Host: www.cloudflare.com
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/2 200
< date: Wed, 28 Oct 2026 09:12:44 GMT
< content-type: text/html; charset=UTF-8
< server: cloudflare
<
```

`ALPN: offers h2,http/1.1` — клиент предложил оба протокола, сервер выбрал `h2`. `OPENED stream 1` — открытие потока; `:method`, `:scheme`, `:authority`, `:path` — псевдозаголовки, заменяющие стартовую строку. В блоке `<` видно, что версия в статусной строке — `HTTP/2`, а имена заголовков пришли в нижнем регистре, как требует спецификация.

Обратите внимание, чего в выводе нет: `Transfer-Encoding`, `Connection`, `Keep-Alive`. Эти заголовки в HTTP/2 запрещены, и сервер, который их присылает, нарушает протокол — обычно это прокси, вставший посередине и не понимающий, что говорит по h2.

Подводные камни

UDP блокируется. В корпоративных сетях и части гостевых Wi-Fi исходящий UDP/443 закрыт политикой. Клиент попробует QUIC, не дождётся ответа и вернётся на TCP — ценой задержки в несколько секунд на первом запросе. Диагностика: `curl --http3` зависает, `curl --http2` работает.

MTU и фрагментация. Минимальный размер датаграммы для QUIC — 1200 байт, при MTU 1500 запас невелик. Если в пути есть туннель (VPN, GRE, IPsec) с меньшим MTU и ICMP «fragmentation needed» фильтруется, крупные пакеты пропадают. Симптом: рукопожатие проходит, передача данных встаёт.

Кэш и CDN отстают. Не все промежуточные узлы умеют кэшировать HTTP/2 и HTTP/3 корректно; часть корпоративных прокси до сих пор терминирует TLS и ходит дальше по HTTP/1.1. Проверять версию протокола нужно на каждом хопе, а не только на клиенте.

Один поток не значит быстрее. На локальной сети с нулевыми потерями HTTP/2 и HTTP/3 проигрывают HTTP/1.1 по CPU: сжатие заголовков и шифрование каждого пакета стоят cycles. Выигрыш появляется на больших RTT и потерях.

`GOAWAY` **при деплое.** Перезапуск сервера рвёт все потоки открытого соединения; чем их больше, тем заметнее всплеск ошибок у клиентов. Что с этим делать — в разделе «Что это меняет для эксплуатации» ниже.

Что это меняет для эксплуатации

Меньше сокетов. Один клиент держит одно соединение вместо шести, и файловых дескрипторов нужно меньше. Обратная сторона: соединение живёт долго и «прилипает» к одному бэкенду, а балансировка L4 распределяет соединения, а не

запросы. Отсюда практика: терминировать TLS на L7-балансировщике и раскидывать запросы, а не соединения.

Терминация TLS решает всё. Версия протокола определяется на узле, где заканчивается TLS. Если балансировщик терминирует TLS и ходит к бэкенду по HTTP/1.1, клиент получает h2, но приложение об этом не знает и не увидит ни `:authority`, ни номера потока — исходную версию стоит передавать заголовком `X-Forwarded-Proto-Version`.

`GOAWAY` **при деплое.** Перезапуск узла рвёт все потоки в соединении, а их может быть сотня. Клиент увидит пачку ошибок на ресурсах, которые уже были в полёте. Правильная схема — двухфазный `GOAWAY`: сначала с `last-stream-id`, чтобы клиент перестал открывать новые потоки, затем закрытие через несколько секунд. В nginx и Envoy это настраивается, но по умолчанию выключено.

Метрики по версии протокола. Если в логах и метриках нет `$server_protocol`, регресс после включения h3 не виден: доля успешных запросов та же, а p95 вырос, потому что часть клиентов ушла в UDP-таймаут и вернулась на TCP.

Одно соединение — одна точка отказа. При HTTP/1.1 потеря одного из шести соединений стоила 17 % пропускной способности; при HTTP/2 потеря единственного останавливает всё. Поэтому `PING`-фреймы и таймауты важнее, чем кажется: молчащий NAT выбрасывает состояние, и клиент узнаёт об этом только по таймауту.

Типичные ошибки

1. **Считать, что HTTP/2 решает проблему потерь пакетов.** Мультиплексирование живёт внутри одного TCP-соединения, и потеря сегмента блокирует все потоки. Механизм: TCP гарантирует порядок байтов, а не порядок сообщений.
2. **Отдавать HTTP/2 без** `proxy_http_version 1.1`**.** nginx по умолчанию общается с бэкендом по HTTP/1.0, соединения не переиспользуются, и выигрыш съедается ростом числа connect. Механизм: HTTP/1.0 закрывает соединение после

ответа, если не указано иное.

3. **Включать HTTP/3 и не оставлять fallback.** Часть клиентов за UDP-блокировкой получит таймаут вместо быстрого TCP-соединения. Механизм: QUIC не имеет автоматического быстрого отказа без таймера.
4. **Терминировать TLS на прокси и удивляться, что клиент видит 1.1.** Версия протокола заканчивается на том узле, где терминируется TLS; дальше идёт другой протокол. Механизм: HTTP/2 требует TLS с ALPN, а прокси общается с бэкендом plain-text.
5. **Полагаться на server push для ускорения.** Push не учитывает кэш клиента и в современных браузерах не поддерживается; трафик растёт, эффекта нет.
6. **Держать `MAX_CONCURRENT_STREAMS` маленьким «для безопасности».** При значении 10 браузер упрётся в лимит и начнёт ставить запросы в очередь — то же head-of-line blocking, только внутри h2.

Чек-лист

- [ ] `curl -w '%{http_version}'` показывает `2` или `3` на внешнем периметре.
- [ ] `openssl s_client -alpn h2` подтверждает, что сервер предлагает `h2`.
- [ ] В nginx заданы `http2 on;` (или параметр `listen`), `proxy_http_version 1.1` и пустой `Connection`.
- [ ] Для HTTP/3 есть рабочий fallback на TCP и проверено поведение за UDP-блокировкой.
- [ ] `GOAWAY` при деплое двухфазный, время слива соединений согласовано с балансировщиком.
- [ ] MTU на пути проверен, ICMP не фильтруется полностью.
- [ ] Метрики разбиты по версии протокола — иначе регресс h3 не виден.

Упражнения

1. Сравните `curl --http2` и `curl --http1.1` на одном URL с задержкой (`--limit-rate`), замерив `time_starttransfer`. Объясните разницу.
2. Включите HTTP/2 в nginx и проверьте `openssl s_client -alpn h2`. Затем уберите `http2 on;` и сравните вывод.
3. Заблокируйте UDP/443 через `iptables` на тестовой машине и посмотрите, за сколько секунд `curl --http3` откатится на TCP.
4. Соберите страницу из 50 мелких ресурсов и сравните время полной загрузки по h1.1 (6 соединений) и h2 при потерях 2 % (`tc netem loss 2%`).

Ответы и разбор к этим упражнениям — в приложении D, часть 2.

Глава 11. Куки, сессии, кэширование, CDN

HTTP не хранит состояния: каждый запрос самодостаточен, и сервер по умолчанию не знает, что два запроса пришли от одного человека. Куки (cookies) — механизм, который это исправляет, и одновременно главный источник утечек и странных багов. Кэширование — второй механизм, который экономит трафик и деньги, и вторая по частоте причина инцидентов вида «пользователь видит чужой профиль».

После этой главы вы сможете разобрать цепочку «браузер → CDN → балансировщик → приложение» и понять, на каком узле теряется кука или подменяется ответ, объяснить разницу между серверной сессией и JWT, настроить `Cache-Control` так, чтобы кэш не отдавал приватные данные, и проверить всё это через `curl`.

Куки: как сервер узнаёт клиента

Сервер отправляет заголовок `Set-Cookie`, браузер сохраняет пару «имя=значение» и прикладывает её к последующим запросам через `Cookie`. Больше в протоколе ничего нет: идентификация целиком построена на том, что клиент честно вернёт то, что ему дали.

```
HTTP/1.1 200 OK
Set-Cookie: session_id=8f14e45fceea167a5a36dedd4bea2543; Path=/; HttpOnly; Secure;
    SameSite=Lax; Max-Age=604800
Set-Cookie: theme=dark; Path=/; Max-Age=31536000
Content-Type: text/html; charset=utf-8
```

Атрибуты определяют область видимости и защиту. `Domain` задаёт домен: если он не указан, кука становится host-only и уходит только на точное имя, поставившее её. `Path` ограничивает путь. `Max-Age` (в секундах) или `Expires` (дата) делают куку постоянной; без них она сессионная и умирает с браузером.

`HttpOnly` запрещает доступ из JavaScript — это защита от кражи сессии через XSS. `Secure` разрешает передачу только по HTTPS. `SameSite` управляет отправкой при кросс-сайтовых запросах: `Strict` не отправляет куку никогда, `Lax` (по умолчанию в современных

браузерах) — только при переходе по ссылке методом GET, `None` — отправляет всегда, но требует `Secure`.

Отдельно стоит запомнить префиксы: `__Host-` требует `Secure`, `Path=/` и отсутствие `Domain`, то есть гарантирует, что куку нельзя подсунуть с поддомена; `__Secure-` требует только `Secure`.

Ограничения жёсткие: одна кука — не больше 4 КБ вместе с атрибутами, на домен браузеры держат порядка 50 кук. Складывать в куку весь профиль пользователя — плохая идея не только из-за размера: она уходит в каждом запросе, включая картинки.

CSRF и SameSite

Кука уходит с любым запросом к своему домену, включая запрос, инициированный чужой страницей. На этом построена CSRF-атака: сайт злоумышленника содержит форму с `action` на ваш банк, браузер автоматически прикладывает сессионную куку, и перевод выполняется от имени пользователя.

`SameSite=Lax`, ставший поведением по умолчанию в современных браузерах, закрывает основной сценарий: кука не отправляется при кросс-сайтовом POST. Но `Lax` пропускает переход по ссылке методом GET — поэтому изменяющие операции не должны быть GET ни при каких обстоятельствах, а `GET /logout` — плохая идея даже как компромисс.

`SameSite=Strict` надёжнее, но ломает переходы по внешним ссылкам: пользователь приходит из поиска или из письма и видит форму входа, уже имея сессию. Практика — `Lax` плюс CSRF-токен для форм (синхронизирующий токен в скрытом поле или double-submit cookie), плюс проверка `Origin`/`Referer` на стороне сервера для небезопасных методов.

`SameSite=None` нужен только для кросс-сайтовых встраиваний (виджет оплаты в iframe) и обязан идти вместе с `Secure`. Без `Secure` браузер просто отбросит куку, и виджет «сломается» без внятной ошибки в консоли.

Сессии против JWT

Серверная сессия — самая простая схема: в куке лежит случайный идентификатор, а данные — в Redis или в таблице БД. Плюсы: сервер контролирует срок жизни, может отозвать сессию мгновенно, размер куки минимален. Минусы: состояние нужно хранить и реплицировать, а каждый запрос требует обращения к хранилищу.

JWT (RFC 7519) — самодостаточный токен: полезная нагрузка с утверждениями (`sub`, `exp`, `iat`, `scope`) и подпись. Сервер проверяет подпись и не ходит в хранилище вообще. Плюсы: горизонтальное масштабирование без общего состояния, удобно для межсервисных вызовов. Минусы существеннее, чем кажется на старте: токен нельзя отозвать до истечения срока, он заметно больше идентификатора (легко 500–900 байт) и уходит в каждом запросе, а любая утечка ключа подписи компрометирует всё сразу.

Практический компромисс, который используют чаще всего: короткоживущий access token (5–15 минут) плюс refresh token в `HttpOnly`-куке. Отзыв реализуется через версию пользователя или чёрный список refresh-токенов, а не через отзыв access-токена.

Куку с сессией нельзя хранить в `localStorage`: любой XSS прочитает её. `HttpOnly` — не украшение, а базовая гигиена.

Кэширование: Cache-Control и валидаторы

Кэш бывает приватным (браузер) и общим (CDN, прокси). Правила задаёт `Cache-Control` (RFC 9111). Ключевые директивы:

Директива	Смысл
`max-age=3600`	свежесть в секундах для любого кэша
`s-maxage=600`	то же, но только для общих кэшей
`public` / `private`	можно ли кэшировать ответ с авторизацией
`no-store`	не сохранять вообще
`no-cache`	сохранять, но перепроверять перед каждым использованием
`must-revalidate`	после истечения свежести перепроверять, не отдавать устаревшее

`no-cache` и `no-store` путают чаще всего. `no-store` — «не храни», `no-cache` — «храни, но спрашивай разрешения». Для ответов с персональными данными нужен именно `no-store` (плюс `private`, если ответ всё-таки проходит через CDN).

Валидаторы позволяют не передавать тело повторно. `ETag` — версия ресурса, обычно хеш содержимого; `Last-Modified` — время изменения. Клиент присылает `If-None-Match: "9f2c1ab7e4"` или `If-Modified-Since`, сервер отвечает `304 Not Modified` без тела. Слабый валидатор `W/"9f2c1ab7e4"` означает «семантически то же», сильный — «байт в байт».

`Vary` перечисляет заголовки, влияющие на содержимое. `Vary: Accept-Encoding` обязателен, если сервер отдаёт gzip или brotli; `Vary: Cookie` — если ответ зависит от пользователя. `Vary: *` означает «не кэшировать никогда» и обычно является ошибкой конфигурации.

Практика: куки и условные запросы через curl

`curl` хранит куки в файле-«банке» и умеет их переиспользовать.

bash

```
curl -sS -c /tmp/jar.txt -o /dev/null -w '%{http_code}\n' -X POST -d
    'user=alice&pass=secret' https://httpbin.org/cookies/set/session/abc123
grep session /tmp/jar.txt
```

```
200
#HttpOnly_httpbin.org   FALSE   /   FALSE   0   session abc123
```

Пять полей через табуляцию — домен, флаг для поддоменов, путь, `Secure`, срок, имя, значение. Дальше `-b /tmp/jar.txt` отправит куку обратно, и её видно в `-v` в строке `> Cookie: session=abc123`.

Проверка условного запроса выглядит так:

bash

```
curl -sSI https://example.com/ | grep -i etag
curl -sS -o /dev/null -w '%{http_code} %{size_download}\n' -H 'If-None-Match:
    "9f2c1ab7e4"' https://example.com/
```

```
etag: "9f2c1ab7e4"
304 0
```

Код 304 и ноль байт тела — кэш работает. Если сервер вернул `200` с телом, значит `ETag` не совпал или его нет вовсе.

Практика: кэш на стороне nginx

Обратный прокси умеет кэшировать ответы бэкенда, и это дешёвый способ снять нагрузку.

nginx

```
proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=api:20m max_size=2g
    inactive=60m;

server {
    listen 443 ssl;
    http2 on;

    location /api/public/ {
        proxy_pass http://backend;
        proxy_cache api;
        proxy_cache_valid 200 10m;
        proxy_cache_valid 404 1m;
        proxy_cache_key "$scheme$request_method$host$request_uri";
        proxy_cache_use_stale error timeout updating http_500 http_502 http_503;
        proxy_cache_background_update on;
        add_header X-Cache-Status $upstream_cache_status always;
    }

    location /api/private/ {
        proxy_pass http://backend;
        proxy_no_cache 1;
        proxy_cache_bypass 1;
        add_header Cache-Control "no-store" always;
    }
}
```

`X-Cache-Status` показывает `HIT`, `MISS`, `EXPIRED` или `BYPASS` — без него отладка кэша превращается в гадание. `proxy_cache_use_stale` отдаёт устаревший ответ, пока бэкенд недоступен: это лучше, чем 502. Для приватных эндпоинтов кэш отключается полностью, иначе ответ одного пользователя уйдёт другому.

Ключ кэша по умолчанию не включает `Cookie` и `Authorization` — именно поэтому приватные ответы и утекают. Если ответ зависит от пользователя, его нельзя класть в общий кэш ни при каких настройках `max-age`.

Практика: воспроизводим утечку кэша

Утечка «кэш отдал чужое» воспроизводится двумя запросами с разными куками. Достаточно, чтобы ответ зависел от пользователя, а ключ кэша — нет.

bash

```
curl -sS -b 'session=alice-token' -D- -o /dev/null https://shop.example.com/cart |
    grep -i 'cache-control\|age'
```

```
cache-control: public, max-age=300
age: 0
```

Директива `public` разрешает общему кэшу хранить ответ, хотя он собран под конкретную сессию. Второй запрос с другой кукой получает тот же объект:

bash

```
curl -sS -b 'session=bob-token' https://shop.example.com/cart
```

json

```
{"items":[{"sku":"A-17","qty":2}],"owner":"alice@example.com"}
```

Боб видит корзину Алисы. Причина не в CDN, а в заголовке: `public` плюс отсутствие `Vary: Cookie` означает, что вариант один на всех. Лечение — `Cache-Control: private, no-store` для всего, что зависит от сессии, и правило на CDN «не кэшировать запросы с `Cookie`», кроме явно публичных путей.

Есть и юридическая деталь: по RFC 9111 общий кэш не должен переиспользовать ответ на запрос с `Authorization`, если в ответе нет `public`, `s-maxage` или `must-revalidate`. Именно поэтому утечки чаще случаются через куки, а не через токены: для `Authorization` защита встроена в стандарт, для `Cookie` — нет.

CDN: кэш на краю сети

CDN — это сеть кэширующих прокси, расставленных ближе к пользователю. Запрос попадает на ближайший узел (edge); при промахе узел идёт к источнику (origin), забирает ответ и хранит его по правилам `Cache-Control`. Выигрыш двойной: меньше RTT и меньше нагрузки на origin.

Ключ кэша на CDN почти всегда настраиваемый: хост, путь, query-строка (целиком или в виде белого списка параметров), иногда заголовки. Ошибка «кэш отдал чужое» почти всегда означает, что в ключ не попал параметр, влияющий на содержимое: `?lang=en` и `?lang=de` дали один и тот же ключ, или CDN проигнорировал `Vary: Accept-Language`.

Инвалидация — вторая половина задачи. TTL в 24 часа означает, что после деплоя сутки живёт старая версия. Варианты: сократить TTL для HTML и оставить длинный для хешированных

статики (`app.9f2c1a.js`), сбрасывать кэш по URL (`purge`) или по тегам (surrogate keys), либо менять имя файла при каждом релизе — самый надёжный подход, потому что не требует инвалидации вообще.

Управлять кэшем CDN отдельно от браузера позволяют `CDN-Cache-Control` (RFC 9213) и `Surrogate-Control`: origin отдаёт браузеру `max-age=60`, а CDN — `max-age=3600`, и оба живут по своим правилам.

`stale-while-revalidate` (RFC 5861) даёт CDN право отдать чуть устаревший ответ, пока он в фоне обновляет его у origin. Для каталогов и лент это ощутимо снижает время ответа при всплесках трафика.

Сессии в кластере

Как только приложение запущено в двух экземплярах, серверная сессия превращается в распределённую задачу. Три рабочих схемы.

Первая — липкие сессии (sticky sessions): балансировщик по куке направляет клиента на тот же узел. Просто, но хрупко: при деплое или падении узла сессии теряются, а перекос трафика никуда не уходит.

Вторая — общее хранилище: сессии лежат в Redis, идентификатор в куке, любой узел читает любую сессию. Это стандарт для Kubernetes, но добавляет сетевой хоп на каждый запрос (обычно 0,3–1 мс внутри кластера) и точку отказа, которую надо реплицировать.

Третья — подписанные куки (JWT или аналог): состояние у клиента, узлы не общаются. Плата — невозможность отзыва и рост размера куки.

Отдельная ловушка — домен куки. Приложение на `app.example.com` и API на `api.example.com` — это один сайт (same-site) для `SameSite`, но разные хосты для `Domain`. Кука, поставленная без `Domain`, на `api.example.com` не уйдёт, и вход будет «слетать» ровно на кросс-доменных запросах. Лечится либо `Domain=example.com`, либо

проксированием API через тот же хост (`/api`), что предпочтительнее: меньше поверхности для подделки кук.

Типичные ошибки

1. **Кэширование ответа с `Set-Cookie`.** CDN сохраняет ответ вместе с заголовком и отдаёт его следующему пользователю, который получает чужую сессию. Механизм: `Set-Cookie` не входит в ключ кэша и не запрещает кэширование сам по себе; нужен явный `Cache-Control: private` или `no-store`.
2. **Кука «потерялась» на поддомене.** Кука, поставленная на `app.example.com` без `Domain`, не уходит на `api.example.com` — она host-only. Механизм: браузер сравнивает домен строго; чтобы разделить куку между поддоменами, нужен `Domain=example.com`, но тогда её видит и любой другой поддомен, что расширяет поверхность атаки.
3. `SameSite=Strict` **на SSO-переходе.** Пользователь приходит по ссылке с другого сайта, кука не отправляется, и он видит форму логина, уже будучи залогиненным. Механизм: `Strict` блокирует куку при любом кросс-сайтовом переходе, включая обычный клик по ссылке.
4. `no-cache` **вместо** `no-store` **для приватных данных.** Ответ сохраняется на диск браузера и промежуточных кэшей. Механизм: `no-cache` разрешает хранение, запрещая лишь использование без перепроверки.
5. **Инвалидация «по всем URL» после каждого деплоя.** Origin получает шквал запросов, CDN пуст, latency растёт в разы. Механизм: сброс кэша превращает распределённую систему в один сервер.
6. `Vary: Accept-Encoding` **забыт при сжатии.** Кэш хранит gzip-версию и отдаёт её клиенту без поддержки gzip. Механизм: без `Vary` ключ кэша не различает варианты одного ресурса.
7. **JWT в** `localStorage` **«потому что удобно».** Любой XSS читает токен и уносит его. Механизм: `localStorage` доступен всему JavaScript на странице, `HttpOnly`-кука — нет.

Чек-лист

- [ ] Кука сессии имеет `HttpOnly`, `Secure` и осмысленный `SameSite`.
- [ ] Для критичных кук используются префиксы `__Host-` или `__Secure-`.
- [ ] Ответы с персональными данными отдают `Cache-Control: private, no-store`.
- [ ] Статика отдаётся с хешем в имени файла и `max-age` в год, HTML — с коротким TTL.
- [ ] `Vary` содержит `Accept-Encoding`, а при языковой версионности — `Accept-Language`.
- [ ] `X-Cache-Status` (или аналог) есть в ответах и в метриках.
- [ ] Процедура инвалидации описана и проверена на стейдже: по URL, по тегу, через смену имени.
- [ ] Есть алерт на долю промахов кэша (hit ratio) — падение видно раньше, чем пользователи.

Упражнения

1. Снимите `Set-Cookie` через `curl -i`, разберите все атрибуты и объясните, какие из них защищают от XSS, а какие — от CSRF.
2. Поставьте куку на `app.example.com` без `Domain` и с `Domain=example.com`, затем проверьте `curl -b` к `api.example.com`. Объясните результат.
3. Настройте `proxy_cache` в nginx для публичного эндпоинта и добейтесь последовательности `MISS → HIT → EXPIRED` в `X-Cache-Status`.
4. Возьмите любой публичный сайт и определите, кэшируется ли HTML: сравните `Cache-Control`, `ETag` и ответ на `If-None-Match`. Предложите, что стоит изменить.
5. Включите `SameSite=Strict`, пройдите по внешней ссылке на сайт и объясните, почему кука не отправилась. Затем верните `Lax` и добавьте CSRF-токен в форму.

Ответы и разбор к этим упражнениям — в приложении D, часть 2.

Глава 12. TLS и сертификаты

TLS — слой, на котором держится вся современная сеть: без него нет ни HTTPS, ни HTTP/2, ни HTTP/3. При этом именно TLS даёт самую частую аварию в эксплуатации — истёкший сертификат, который никто не заметил. После этой главы вы сможете разобрать рукопожатие TLS 1.3 по шагам, объяснить, зачем нужны и симметричное, и асимметричное шифрование одновременно, прочитать цепочку доверия сертификата, выпустить и обновить сертификат через ACME, и диагностировать проблему через `openssl s_client` без обращения в поддержку.

Три задачи TLS

TLS решает ровно три задачи и делает это одновременно. Конфиденциальность — трафик нельзя прочитать по дороге; её обеспечивает симметричное шифрование. Целостность — трафик нельзя незаметно изменить; за неё отвечает код аутентификации сообщения (AEAD). Аутентификация — клиент уверен, что говорит с тем сервером, чьё имя набрал; за это отвечает сертификат, подписанный удостоверяющим центром.

Без третьей задачи первые две бесполезны: шифрование с неизвестным собеседником защищает канал, который ведёт прямо к атакующему.

Симметрика и асимметрика

Симметричное шифрование использует один ключ для шифрования и расшифровки. Оно быстрое — гигабайты в секунду на современном процессоре, особенно с AES-NI. Асимметричное использует пару ключей: открытый шифрует, закрытый расшифровывает (или наоборот, для подписи). Оно медленное — на порядки.

Поэтому TLS гибридный: асимметричная криптография применяется только на этапе рукопожатия, чтобы договориться о сессионном ключе, а дальше весь трафик идёт симметричным

шифром. В TLS 1.3 обязательны AEAD-шифры: `TLS_AES_128_GCM_SHA256`, `TLS_AES_256_GCM_SHA384`, `TLS_CHACHA20_POLY1305_SHA256`. Первые два выигрывают на процессорах с AES-NI, третий — на мобильных без аппаратного ускорения AES.

Ключевой обмен в TLS 1.3 — только (EC)DHE, обычно X25519. Это даёт свойство прямой секретности (forward secrecy): компрометация долговременного ключа сервера не позволит расшифровать записанный ранее трафик, потому что сессионные ключи уже уничтожены.

Рукопожатие TLS 1.3 по шагам

TLS 1.2 (RFC 5246) требовал двух round-trip до первых данных. TLS 1.3 (RFC 8446) уложился в один:

1. **ClientHello.** Клиент отправляет список поддерживаемых шифров, версий, расширение `server_name` (SNI), `alpn` и, главное, свой `key_share` — открытую часть для обмена ключами.
2. **ServerHello.** Сервер выбирает параметры и отправляет свой `key_share`. С этого момента обе стороны могут вычислить общий секрет.
3. Всё дальнейшее уже шифруется: `EncryptedExtensions` (в том числе выбранный ALPN), `Certificate`, `CertificateVerify` (подпись закрытым ключом сервера, доказывающая владение ключом), `Finished`.
4. Клиент проверяет цепочку и подпись, отправляет свой `Finished`. Соединение готово.

Один round-trip — потому что клиент угадывает группу для обмена ключей и присылает `key_share` заранее. Если угадал не ту, сервер пришлёт `HelloRetryRequest` и добавит ещё один round-trip.

Повторное подключение может уложиться в 0-RTT: клиент использует сохранённый тикет (PSK) и отправляет данные в первом же пакете. Ранние данные не защищены от повторного воспроизведения, поэтому в 0-RTT допустимы только идемпотентные запросы.

Сертификат и цепочка доверия

Сертификат X.509 связывает имя (или имена) с открытым ключом и подписан издателем. Браузер не знает ваш сертификат; он знает корневые сертификаты удостоверяющих центров, вшитые в систему и браузер.

Цепочка выглядит так: корневой сертификат (root, в trust store) → промежуточный (intermediate) → ваш (leaf). Сервер обязан отдавать leaf и все промежуточные; корневой клиент найдёт сам. Самая частая ошибка конфигурации — отдать только leaf: браузеры с кэшем промежуточных иногда спасают, а `curl`, мобильные приложения и Java — нет.

Что проверяется: подпись каждого звена, срок действия (`notBefore`/`notAfter`), соответствие имени из `subjectAltName` запрошенному хосту, назначение ключа, отзыв. Поле `CN` устарело — современные клиенты смотрят только на SAN, и сертификат без SAN не будет принят.

Отзыв реализуется через CRL и OCSP; и то, и другое в последние годы вытесняется короткоживущими сертификатами и Certificate Transparency (CT) — публичными логами, куда CA обязаны записывать каждый выпущенный сертификат.

SNI и ALPN

SNI (Server Name Indication) — расширение, в котором клиент передаёт имя хоста в открытом виде ещё до установки шифрования. Без него сервер не знает, какой из сотен сертификатов показать, и пришлось бы держать отдельный IP на каждый сайт. Имя в SNI не шифруется: провайдер видит, к какому сайту вы идёте, даже не видя содержимого. Расширение ECH (Encrypted Client Hello) призвано это закрыть, но на момент написания поддержано не везде.

ALPN (Application-Layer Protocol Negotiation) — согласование прикладного протокола внутри TLS: `h2`, `http/1.1`, `h3`. Именно поэтому HTTP/2 требует TLS: без ALPN клиент и сервер не договорятся о версии.

ACME и Let's Encrypt

ACME (RFC 8555) автоматизирует выпуск: клиент доказывает контроль над доменом, получает сертификат и обновляет его сам. Let's Encrypt выдаёт короткоживущие сертификаты (90 дней на момент написания, с курсом на дальнейшее сокращение срока) — это вынуждает автоматизировать обновление, и это правильно.

Самый распространённый клиент — certbot. Проверка домена идёт по HTTP-01 (файл в `/.well-known/acme-challenge/`) или DNS-01 (TXT-запись, единственный вариант для wildcard-сертификатов).

bash

```
sudo certbot --nginx -d example.com -d www.example.com --redirect
```

```
Requesting a certificate for example.com and www.example.com

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/example.com/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/example.com/privkey.pem
This certificate expires on 2027-01-07.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate.
```

`--redirect` добавляет редирект с HTTP на HTTPS. Обновление проверяется прогоном без реального выпуска:

bash

```
sudo certbot renew --dry-run
```

```
Congratulations, all simulated renewals succeeded:
  /etc/letsencrypt/live/example.com/fullchain.pem (success)
```

`--dry-run` не трогает боевые файлы и ходит на staging-сервер ACME — именно поэтому его стоит ставить в CI раз в сутки, а не полагаться на «certbot же настроил таймер».

Практика: диагностика через openssl s_client

bash

```
openssl s_client -connect example.com:443 -servername example.com </dev/null
```

```
CONNECTED(00000003)
depth=2 C = US, O = DigiCert Inc, OU = www.digicert.com, CN = DigiCert Global Root
    CA
verify return:1
depth=1 C = US, O = DigiCert Inc, CN = DigiCert TLS RSA SHA256 2020 CA1
verify return:1
depth=0 CN = example.com
verify return:1
---
```

```
Certificate chain
 0 s:CN = example.com
   i:C = US, O = DigiCert Inc, CN = DigiCert TLS RSA SHA256 2020 CA1
 1 s:C = US, O = DigiCert Inc, CN = DigiCert TLS RSA SHA256 2020 CA1
   i:C = US, O = DigiCert Inc, CN = DigiCert Global Root CA
---
Server certificate
subject=CN = example.com
issuer=C = US, O = DigiCert Inc, CN = DigiCert TLS RSA SHA256 2020 CA1
---
SSL handshake has read 3214 bytes and written 347 bytes
Verification: OK
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
```

Читать этот вывод нужно сверху вниз. Блок `depth=0/1/2` — цепочка, которую построил клиент: если depth=0 не совпадает с ожидаемым именем, вы попали на другой сервер (или SNI не передан). `Certificate chain` показывает, что отдал сервер: если там одно звено, промежуточный отсутствует — часть клиентов получит ошибку.

`Verification: OK` и `Verify return code: 0 (ok)` — цепочка и срок в порядке. `TLSv1.3` и имя шифра подтверждают современный стек. `No ALPN negotiated` — признак того, что клиент не просил протокол; с `-alpn h2` здесь появится `ALPN protocol: h2`.

Флаг `-servername` критичен при проверке виртуальных хостов: без него openssl возьмёт сертификат по умолчанию, и вы получите ложную ошибку несоответствия имени.

Быстрая проверка срока годности на сервере:

bash

```
echo | openssl s_client -connect example.com:443 -servername example.com
    2>/dev/null \
  | openssl x509 -noout -subject -issuer -enddate
```

```
subject=CN = example.com
issuer=C = US, O = DigiCert Inc, CN = DigiCert TLS RSA SHA256 2020 CA1
notAfter=Jan  7 09:12:44 2027 GMT
```

Что не так с TLS 1.2

TLS 1.2 (RFC 5246) работает до сих пор и всё ещё встречается в старых интеграциях, но его недостатки хорошо изучены.

Рукопожатие требует двух round-trip, а не одного. Разрешён обмен ключами по RSA, когда сессионный ключ шифруется открытым ключом сервера: удобно, но нет прямой секретности — украденный закрытый ключ расшифровывает весь записанный ранее трафик.

Список шифров в 1.2 настраивается вручную и включает CBC-режимы, для которых годами находили атаки на паддинг (BEAST, Lucky13; POODLE была про SSL 3.0, но показала цену режимов с дополнением). Сжатие убрали после CRIME, пересогласование ограничили отдельным RFC.

TLS 1.3 убрал всё перечисленное: остались только AEAD-шифры, только (EC)DHE, нет сжатия, нет пересогласования, нет статических RSA-ключей в обмене. Поэтому правильная конфигурация на 2026 год — `TLSv1.3` и `TLSv1.2` с ограниченным набором AEAD-шифров; 1.0 и 1.1 отключены везде, где есть хоть какой-то аудит.

Проверить, что старые версии отключены, а не просто «не предлагаются браузером», можно явным указанием версии:

bash

```
openssl s_client -connect example.com:443 -servername example.com -tls1_1
    </dev/null
```

```
1407B7F505C77F0000:error:0A00018E:SSL routines::unsupported protocol
```

Ошибка `unsupported protocol` — правильный ответ. Если же в выводе появился `Protocol : TLSv1.1` и `Cipher`, сервер принимает устаревший протокол, и это находка для аудита.

HSTS и pinning

`Strict-Transport-Security` говорит браузеру, что к этому домену можно обращаться только по HTTPS, и подменять схему нельзя даже при явном `http://`.

nginx

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

`max-age` — сколько секунд браузер помнит правило. `includeSubDomains` распространяет его на поддомены, и это ловушка: если какой-то поддомен живёт только на HTTP, он станет недоступен. `preload`

добавляет домен в список, вшитый в браузеры, и выйти из него можно месяцами. Включать `preload` стоит только после того, как все поддомены годами работают по HTTPS.

Certificate pinning — привязка к конкретному ключу или сертификату. В браузерах механизм HPKP провалился и удалён: ошибка в пине ломает сайт на месяцы. В мобильных приложениях и в межсервисном общении pinning применяется, но требует плана ротации: пин haleaf-сертификат гарантирует аварию при каждом обновлении, поэтому пинят промежуточный CA или пару ключей.

Mutual TLS

В обычном TLS сертификат предъявляет только сервер. В mutual TLS (mTLS) сервер требует сертификат и от клиента: в рукопожатии появляется `CertificateRequest`, клиент присылает свой сертификат, сервер проверяет его против своего набора доверенных CA.

Применяется там, где нет пользователя с паролем: между сервисами в service mesh (sidecar-прокси аутентифицируют друг друга), в B2B-интеграциях, для внутренних API. Проверить вручную можно так:

bash

```
curl -sS --cert client.pem --key client.key https://internal.example.com/health
```

Без сертификата тот же запрос вернёт ошибку рукопожатия, а не 401: соединение не устанавливается вовсе, потому что клиент не прошёл аутентификацию на уровне TLS.

Главная проблема mTLS — жизненный цикл клиентских сертификатов: выпуск, ротация и отзыв ложатся на эксплуатацию. В Kubernetes это решают через cert-manager и SPIFFE, где сертификат живёт часы, а не годы. Ручной mTLS с годовыми сертификатами — почти гарантированная авария при первом истечении.

Почему «сертификат истёк» — самая частая авария

Причины прозаичны. Автообновление настроено, но не проверено: certbot не смог пройти HTTP-01, потому что `.well-known` перехватывается редиректом или закрыт в WAF, и падал молча в логах, которые никто не читает. Сертификат выпущен вручную и положен в Docker-образ, а образ не пересобирался год. Или домен переехал за CDN, и обновляется сертификат CDN, а не ваш.

Симптомы различаются: браузер показывает страницу-заглушку с ошибкой `NET::ERR_CERT_DATE_INVALID`, `curl` пишет `SSL certificate has expired`, а Android может не показать ошибку вовсе, если приложение игнорирует проверку.

Лечится тремя вещами: мониторингом даты `notAfter` с алертами за 21, 14 и 7 дней; автоматическим обновлением с явным хуком перезагрузки (`--deploy-hook "systemctl reload nginx"`, потому что nginx держит сертификат в памяти и не перечитает файл сам); и проверкой результата — `curl` к своему же домену из внешней сети, а не только `systemctl status certbot`.

Проверка срока встраивается в CI и в cron одной командой:

bash

```
end=$(openssl s_client -connect example.com:443 -servername example.com </dev/null
    2>/dev/null \
      | openssl x509 -noout -enddate | cut -d= -f2)
days=$(( ( $(date -d "$end" +%s) - $(date +%s) ) / 86400 ))
echo "days left: $days"
if [ "$days" -lt 21 ]; then exit 1; fi
```

```
days left: 71
```

Код возврата 1 при остатке меньше 21 дня — пайплайн краснеет, и дежурный узнаёт о проблеме за три недели, а не за три минуты.

Типичные ошибки

1. **Отдавать только leaf-сертификат.** Часть клиентов (curl, Java, мобильные) не имеют кэша промежуточных и падает на проверке цепочки. Механизм: сервер обязан передать все звенья, кроме корневого.
2. **Сертификат без SAN.** Имя в `CN` современные клиенты игнорируют, и проверка хоста не проходит. Механизм: RFC

6125 и все браузеры требуют `subjectAltName`.

3. `includeSubDomains` **в HSTS при живом HTTP-поддомене.** Поддомен мгновенно становится недоступен для всех, кто уже получил заголовок. Механизм: браузер применяет политику до истечения `max-age` и не спрашивает сервер.
4. **Ручной выпуск и ручная установка.** Через 90 дней сертификат истекает, потому что человек помнит о нём ровно до следующего релиза. Механизм: короткий срок жизни требует автоматизации, а не дисциплины.
5. **Перезагрузка не выполнена после обновления.** Файлы на диске новые, nginx в памяти держит старые. Механизм: сертификат читается при старте воркеров; нужен reload по хуку.
6. **Часы сервера уехали.** Клиент видит `certificate is not yet valid` или просрочку на живом сертификате. Механизм: проверка срока идёт по локальным часам клиента, и дрейф в минуты уже критичен для короткоживущих сертификатов.
7. **Проверка «работает» только изнутри.** Внутренний curl идёт напрямую к origin, минуя CDN и WAF, и не видит, что наружу отдаётся другой сертификат.

Чек-лист

- [ ] Сервер отдаёт полную цепочку, кроме корневого сертификата.
- [ ] В сертификате есть SAN со всеми используемыми именами.
- [ ] TLS 1.3 включён, TLS 1.0/1.1 отключены, слабые шифры запрещены.
- [ ] Автообновление работает, `--dry-run` проходит, `--deploy-hook` перезагружает nginx.
- [ ] Алерт на остаток срока (21/14/7 дней) идёт в дежурного, а не в общий чат.
- [ ] Внешняя проверка (`curl` снаружи) подтверждает сертификат на периметре, включая CDN.

- [ ] HSTS выставлен без `preload`, пока не проверены все поддомены.
- [ ] Время синхронизировано (NTP), дрейф контролируется.

Упражнения

1. Разберите вывод `openssl s_client` для трёх сайтов и сравните: версия протокола, шифр, число звеньев цепочки, наличие ALPN.
2. Выпустите сертификат через `certbot --nginx` на тестовом домене, затем сломайте HTTP-01 (закройте `/.well-known` в конфиге) и добейтесь, чтобы `--dry-run` упал с понятной ошибкой.
3. Настройте мониторинг срока: скрипт вычисляет дни до `notAfter` и отдаёт метрику, алерт срабатывает на 21 день. Проверьте на сертификате с укороченным сроком.
4. Включите HSTS с `max-age=60` на тестовом домене, убедитесь, что HTTP-переход больше не происходит, и объясните, как откатить политику.

Ответы и разбор к этим упражнениям — в приложении D, часть 2.

Глава 12a. API: что это такое и как им пользоваться

Большая часть работы DevOps — не писать сервисы, а соединять чужие: снять список инстансов у облака, положить образ в registry, завести тикет, забрать счёт из биллинга. Всё это HTTP API, и почти каждый инцидент «интеграция сломалась» сводится к пяти причинам: не тот заголовок аутентификации, игнорирование `429`, пагинация через `offset`, повтор неидемпотентного `POST` и ключ, закоммиченный в репозиторий. Глава — про потребление чужого API: из чего состоит вызов, как доказать, кто ты, как не упереться в лимиты, как прочитать документацию за десять минут. Проектирование своего API — в главе 13, клиент с ретраями — в главе 21.

API — контракт, а не страница

Страница и API могут отдавать одни и те же данные из одной базы, но это разные продукты. Страница — представление для человека: вёрстка и тексты меняются свободно, потребитель смотрит глазами. API — интерфейс для программы: формат зафиксирован схемой, поля названы, коды означают конкретные вещи, проверка автоматическая.

Контракт — набор обещаний: какие пути есть, какие поля обязательны, какие типы у значений, что означает каждый код, сколько запросов в минуту разрешено. Обещания публикуются (документация, OpenAPI) и считаются обязательствами: нарушил сервер — это баг сервера.

Машиночитаемость — ответ разбирает код. Поэтому `200 OK` с телом `{"success": false}` — ловушка: мониторинг увидит успех, ретраи не сработают. Единственный сигнал, который видит вся инфраструктура по пути (прокси, балансировщик, APM), — код ответа.

Стабильность — клиенты выпускаются реже серверов, чужие интеграции живут годами. «Переименуем поле, это же мелочь» в API не работает; работает версия. Практическое следствие: с API

нельзя договориться. Сказано «`limit` не больше 100» — при 500 вы получите `400` или молча усечённую страницу.

Из чего состоит вызов

Вызов — шесть частей, и ошибка в любой даёт свой класс проблем.

Часть	Пример	Типичная ошибка
Метод и путь	`GET /v1/tasks/1042`	`GET` вместо `POST`
Заголовки	`Authorization`, `Accept`	ключ в query-строке
Параметры	`?status=open&limit=50`	`offset` вместо курсора
Тело	`{"title": "..."}`	тело у `GET`
Код ответа	`200`, `404`, `429`	проверка только на `200`
Заголовки ответа	`Retry-After`, `Content-Type`	их не читают

```http
GET /v1/tasks?status=open&limit=2 HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Accept: application/json
X-Request-Id: 7f3c1a9e-2b44-4c01-9a11-5d0e6b8f2c33

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
X-RateLimit-Limit: 600
X-RateLimit-Remaining: 597

{"items": [{"id": 1042, "title": "Починить CI"}], "has_more": true,
 "next_cursor": "eyJ0cyI6MTc2MTc0NDAwMCwiaWQiOjEwNDJ9"}
```

Ставьте всегда `Accept`, `User-Agent` с версией клиента, `X-Request-Id` и `Content-Type` при теле: без последнего запрос с телом получает `415` на ровном месте.

Аутентификация: четыре способа

API-ключ в заголовке (`X-API-Key: 9f2c...`) опознаёт приложение, а не человека: живёт годами, отзывается в панели. Ключ в query (`?api_key=...`) — плохая практика: query попадает в access-логи, в `Referer`, в историю и в трейсы APM.

Bearer-токен — `Authorization: Bearer <token>` (RFC 6750). Токен либо непрозрачный (сервер ищет его в базе), либо JWT (RFC 7519) с

подписанными claims. Bearer значит «кто предъявил, тот и есть»: только TLS и короткий TTL.

Подпись запроса — секрет не передаётся: клиент считает HMAC от канонизированных частей (метод, путь, дата, хеш тела), так работает AWS Signature Version 4. Подпись привязана к запросу и не переиспользуется; цена — сложная канонизация и чувствительность к расхождению часов (окно 5–15 минут).

OAuth 2.0 client credentials — обмен `client_id`/`client_secret` на access-токен (RFC 6749, §4.4). Выигрыш: короткоживущий токен вместо вечного секрета и единый механизм с остальными клиентами провайдера.

Способ	Что летит в сеть	Ротация	Когда брать
API-ключ	сам ключ	вручную	утилиты, простые SaaS
Bearer/JWT	токен	по TTL	сервис уже выдаёт токены
Подпись	HMAC, не секрет	мгновенная	платежи, банки, AWS
client credentials	токен	5–60 минут	server-to-server

bash

```
curl -sS --max-time 10 https://auth.example.com/oauth/token \
  -d grant_type=client_credentials -d client_id="$CLIENT_ID" \
  -d client_secret="$CLIENT_SECRET" -d scope="tasks:read tasks:write"
```

В ответ приходит `access_token` и `expires_in`. Токен кэшируют до истечения минус запас в минуту, а не запрашивают на каждый вызов: иначе вы упираетесь в лимит эндпоинта авторизации ещё до первой полезной операции.

Лимиты и квоты

Лимит (rate limit) — запросов в единицу времени; квота (quota) — всего за период. В лимит упираются на минуту, в квоту — до конца расчётного месяца, и ожиданием это не лечится.

Сервер сообщает три вещи: `429 Too Many Requests`, `Retry-After` (секунды или HTTP-дата, RFC 9110) и `X-RateLimit-Limit` / `Remaining` / `Reset`. Имена `RateLimit-*` без префикса — интернет-драфт IETF, в природе встречаются оба варианта.

Как не быть забаненным: считать `Remaining` до запроса, уважать `Retry-After` буквально, ограничивать параллелизм (двадцать потоков с ретраями — это двадцать запросов в секунду, то есть атака), не ретраить `4xx` кроме `429` и `408`, размазывать фоновые обходы (600 запросов на 10 минут — 1 rps).

```python
import time, httpx

def get_with_budget(client: httpx.Client, url: str, attempts: int = 4) ->
    httpx.Response:
    for attempt in range(attempts):
        r = client.get(url)
        if r.status_code != 429:
            return r
        wait = float(r.headers.get("Retry-After", 2 ** attempt))
        time.sleep(min(wait, 60))
    return r
```

Задержка берётся из ответа сервера, а не из головы клиента, и ограничена сверху: `Retry-After: 86400` иначе повесит воркер на сутки.

Пагинация

Offset/limit (`?limit=50&offset=100`) позволяет прыгнуть на любую страницу, но позиция нестабильна: пока вы листаете, кто-то создаёт запись в начале — все сдвигаются, и на следующей странице вы видите дубль или пропуск. Плюс `OFFSET 100000` заставляет базу прочитать и выбросить 100 000 строк — это уже про стоимость.

Cursor (`?cursor=...`) — «дай 50 записей после этой». Внутри закодированы значение поля сортировки и id; сервер читает по индексу `WHERE (created_at, id) < (:ts, :id)`, вставки его не сдвигают. Минус: нельзя прыгнуть на страницу 7 и показать общее число страниц.

Link-заголовок (RFC 8288) сообщает URL следующей страницы, не требуя своего поля в теле:

```http
Link: <https://api.example.com/v1/tasks?limit=50&cursor=eyJ0cyI6MTc2MTc0NDAwMH0>;
    rel="next"
```

Клиенту достаточно идти по `rel="next"`, пока заголовок есть. Условие выхода — отсутствие `Link` или `has_more: false`, а не пустой `items`: пустая страница при живом курсоре означает, что фильтр съел всё.

Проверяйте, что курсор относится к тем же фильтрам: курсор от `status=open` в запросе `status=done` даст либо `400`, либо тихо неверный результат.

Идемпотентность и Idempotency-Key

Сеть теряет ответы. Сервер выполнил `POST`, ответ не дошёл, клиент повторил по таймауту — две задачи. Для `PUT` и `DELETE` это не проблема: повтор не меняет результат. Для `POST` вводится `Idempotency-Key: 5b1e0f7a-...` — уникальная строка на **логическую операцию**; сервер помнит ключ с ответом (обычно 24 часа) и на повтор возвращает тот же ответ.

Два частых нарушения. Ключ генерируется внутри функции повтора — тогда каждый ретрай приходит с новым ключом и создаёт новый объект. Ключ не привязан к телу: тот же ключ с другим содержимым сервер вправе отклонить (`422`). Ответы `409` и `425` на знакомый ключ означают «операция ещё идёт»: ждать и повторять с тем же ключом.

Версионирование и deprecated

Версия в пути (`/v1/`) видна в логах, конфиге и curl. Версия в заголовке (`Accept: application/vnd.api+json;version=2`) чище теоретически, но в отладке хуже: одинаковые URL ведут себя по-разному. Когда эндпоинт объявляют устаревшим:

1. Найти сигнал: `Deprecation`, `Sunset` (RFC 8594) с датой, changelog. Дата в `Sunset` — дедлайн, а не пожелание.
2. Зафиксировать версию в конфиге, а не «брать последнюю».
3. Понять, что ломается: переименование поля, смена типа (`1` → `"1"`), новое обязательное поле, другой код.
4. Сделать миграцию обратимой: обе версии по флагу, пока обе живы, плюс тест на контракт.

Разбор ответа

Проверять `Content-Type: text/html` значит captive-прокси или страницу ошибки балансировщика, и `json.loads` соврёт про синтаксис вместо причины. Смотреть код, потом тело: успех — это `2xx`, даже если в теле `"status": "ok"`. Ошибки `problem+json` (RFC 9457) читать по `type` — он стабилен, `detail` — для лога:

python

```
if r.headers.get("content-type", "").startswith("application/problem+json"):
    p = r.json()          # p["type"] — стабильный код класса, p["detail"] —
        конкретика
```

Ответ чужого API — недоверенные данные: поле, вчера бывшее числом, сегодня может быть строкой, поэтому схема на входе (pydantic, JSON Schema) ловит это на границе.

Webhooks и подпись тела

Webhook — обратное направление: API зовёт вас. Endpoint публичен, отвечает `2xx` за 5–10 секунд и терпит повторы: доставка «хотя бы один раз» — норма, дубли обязательны к обработке. Порядок: проверить подпись → сохранить сырое событие в очередь → ответить `200` → обработать асинхронно. Дедупликация по id события, а не по содержимому.

Подпись — единственный способ отличить настоящий webhook от запроса любого, кто узнал URL: HMAC-SHA256 от `timestamp + "." + raw_body`, сравнение за постоянное время.

python

```
import hashlib, hmac, time

def verify(raw_body: bytes, header: str, secret: bytes, tolerance: int = 300) ->
    bool:
    parts = dict(p.split("=", 1) for p in header.split(","))
    ts, sig = parts.get("t", "0"), parts.get("v1", "")
    expected = hmac.new(secret, f"{ts}.".encode() + raw_body,
        hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, sig) and abs(time.time() - int(ts)) <=
        tolerance
```

Два нюанса: подписывается **сырое** тело (распарсить и сериализовать заново нельзя — изменится порядок ключей), и метка времени обязательна, иначе перехваченный запрос повторяют вечно.

Как читать документацию за 10 минут

Нужны шесть вещей, в этом порядке: аутентификация (заголовок, где взять, TTL), лимиты (сколько и по чему считают), один пример запроса целиком (не работает пример — дальше читать бессмысленно), формат ошибок, пагинация, changelog с ближайшими deprecation. Плюс два вопроса, экономящие дни: есть ли sandbox и есть ли OpenAPI-спецификация — первое даёт прогнать сценарий без риска, второе избавляет от ручных моделей.

Первый запрос: curl, потом httpx

Начинать с curl: он показывает ровно то, что ушло и пришло, без слоёв библиотеки. Ключ берётся из окружения, чтобы не осесть в истории шелла.

bash

```
export TASKSVC_TOKEN="$(cat ~/.config/tasksvc/token)"
curl -sS -i --max-time 10 \
  -H "Authorization: Bearer $TASKSVC_TOKEN" \
  -H "Accept: application/json" \
  "https://api.example.com/v1/tasks?status=open&limit=2"
```

`-i` печатает заголовки (`Retry-After` и `X-RateLimit-Remaining` видно сразу), `--max-time 10` не даёт зависнуть, `-sS` убирает прогресс, но оставляет ошибки.

python

```
import os, time, httpx

def fetch_open_tasks() -> list[dict]:
    headers = {"Authorization": f"Bearer {os.environ['TASKSVC_TOKEN']}",
               "Accept": "application/json"}
    tasks, cursor = [], None
    with httpx.Client(base_url="https://api.example.com/v1",
                      headers=headers, timeout=httpx.Timeout(10.0, connect=3.0))
                as c:
        for _ in range(1000):                       # предохранитель от зацикливания
            params = {"status": "open", "limit": 50}
            if cursor:
                params["cursor"] = cursor
            r = c.get("/tasks", params=params)
            if r.status_code == 429:
                time.sleep(min(float(r.headers.get("Retry-After", 5)), 60))
                continue
            r.raise_for_status()
            page = r.json()
            tasks.extend(page["items"])
```

```
            cursor = page.get("next_cursor")
            if not page.get("has_more") or not cursor:
                break
    return tasks
```

Чего обычно не хватает самописным клиентам: единого `Client` (пул соединений и один TLS-handshake вместо одного на запрос), явного таймаута, предохранителя в цикле и `Retry-After` из ответа. Ретраи на таймауты и `5xx` строятся так же; полная версия с четырьмя таймаутами и джиттером — в главе 21.

Где хранить ключ

Ключ в коде — это ключ в git, а значит в форке, CI-логе, бэкапе и чьём-то ноутбуке. Ключ живёт в переменной окружения или файле вне репозитория (права `600`), `.env` — только локально и в `.gitignore`, в проде — secret manager. У каждого окружения свой ключ: общий делает ротацию невозможной, а логи бесполезными. Ротация — по расписанию.

bash

```
git log -S "sk_live_" --oneline --all        # поиск по истории коммитов
gitleaks detect --source . --no-banner       # рабочее дерево и история
```

Если ключ уже в истории, удалить строку новым коммитом недостаточно: значение остаётся в старых объектах — ключ отзывают.

Куда двигаться дальше

Потребление и проектирование API — две стороны одного контракта: половина проблем потребления это недоделанный контракт на той стороне. Как его строить — глава 13; как собрать клиент с четырьмя таймаутами, джиттером и лимитом параллелизма — глава 21.

Типичные ошибки

1. **Ключ в query-строке.** Query — часть URL, а URL логируется везде, где есть HTTP-сервер: access-логи, `Referer`, трейсы APM. Лечится заголовком и ротацией.
2. **Проверка только на `200`.** `201` и `204` — успех, но код падает в ветку ошибки; `500` с телом `{"ok": true}` наоборот проходит. Статус

воспринимается как булево, а не как класс.

3. **Ретрай всего подряд.** `401` не исправит ключ, зато попадёт в счётчик брутфорса; `422` детерминирован. Повторяют только временное: таймауты, `502/503/504`, `429`.
4. **Новый `Idempotency-Key` на каждом повторе.** UUID генерируется внутри функции ретрая, а не до неё — и дубли создаются ровно там, где ключ должен был их предотвратить.
5. **Offset на живых данных.** На тестах с 200 записями работает; на проде вставка в начало сдвигает набор, и страница 2 начинается с уже виденной записи.
6. **Игнорирование `Retry-After` и разбор ошибки как текста.** Клиент ждёт «свои» 2 секунды вместо указанных 30, жжёт квоту до блокировки аккаунта, а причину отказа вытаскивает регуляркой из `{"message": "Bad request"}` — тексты меняются каждый релиз.

Чек-лист

- [ ] Ключ приходит из окружения или secret manager; `gitleaks detect` чист.
- [ ] У каждого запроса явный таймаут.
- [ ] `429` обрабатывается с учётом `Retry-After`, число попыток ограничено.
- [ ] `X-RateLimit-Remaining` читается, батчи разбиты под остаток квоты.
- [ ] Пагинация завершается по `has_more: false` или отсутствию `Link`.
- [ ] Создающие `POST` отправляют `Idempotency-Key`, один на логическую операцию.
- [ ] Ошибки разбираются по `type`, webhook — по подписи сырого тела.

Упражнения

1. Возьмите публичное API и за десять минут выпишите шесть пунктов: заголовок аутентификации, лимит и его единицу,

формат ошибки, способ пагинации, версия, дата ближайшего deprecation.

2. Выполните один запрос через `curl -i` и через httpx с `raise_for_status()`. Сравните, какие заголовки видит каждый способ.
3. Реализуйте обход коллекции курсором и offset. Вставьте запись между второй и третьей страницей и покажите, где offset даёт дубль.
4. Напишите проверку подписи webhook с окном 5 минут. Продемонстрируйте два отказа: изменённое тело и подпись недельной давности.
5. Опишите процедуру ротации ключа по шагам, которые можно выполнить в три часа ночи: где лежит, кто читает, что делать при утечке.

Ответы и разбор к этим упражнениям — в приложении D, часть 5.

Глава 13. REST и проектирование API

API — контракт, который живёт дольше кода, его реализующего. Клиенты выпускаются реже серверов, мобильные приложения годами остаются на старых версиях, чужие интеграции ломаются молча. Поэтому проектирование API — дисциплина ограничений: какие методы допустимы, что считается ошибкой, как получить следующую страницу, что будет при повторе запроса. Дальше — REST как набор соглашений поверх HTTP, идемпотентность и ключи идемпотентности, пагинация, версионирование, `problem+json`, rate limiting и полный дизайн API сервиса задач.

Ресурсы, а не глаголы

REST (Representational State Transfer) — архитектурный стиль, описанный Роем Филдингом в 2000 году. В индустрии от него осталась прежде всего одна идея: URL адресует **ресурс** (существительное), а действие выражается HTTP-методом. Сервер не хранит состояние клиента между запросами — каждый запрос несёт всё необходимое для его обработки.

Сравните два подхода к одной операции — отмене заказа:

```http
POST /api/cancelOrder?id=42
POST /api/orders/42/cancel
```

Первый вариант — RPC в обёртке из HTTP: метод всегда POST, действие зашито в путь, кэширование и повторные запросы невозможны осмысленно. Второй лучше, но «отмена» — это не удаление: заказ остаётся в базе с новым статусом. Часто честнее изменить состояние явно:

```http
PATCH /api/orders/42
Content-Type: application/merge-patch+json

{"status": "cancelled"}
```

Методы и их семантика

Семантика методов зафиксирована в RFC 9110, и от неё зависит поведение посредников: кэшей, прокси, браузеров.

Метод	Смысл	Безопасный	Идемпотентный	Тело ответа
GET	прочитать ресурс	да	да	да
POST	создать / выполнить действие	нет	нет	обычно да
PUT	заменить ресурс целиком	нет	да	нет
PATCH	частично изменить	нет	нет (по умолчанию)	нет
DELETE	удалить	нет	да	нет
HEAD	заголовки без тела	да	да	нет

«Безопасный» (safe) значит «не меняет состояние сервера» — по этому признаку роботы и префетчеры решают, можно ли дёргать URL без спроса. «Идемпотентный» — «повторение даёт тот же результат, что один вызов». GET обязан быть и тем, и другим: `GET /orders/42/cancel`, который что-то отменяет, отменит это по вине случайного краулера.

Идемпотентность и ключи идемпотентности

Идемпотентность — способ выжить в сети, где ответ теряется после того, как сервер уже выполнил работу. По таймауту нельзя отличить «сервер умер до записи» от «сервер записал, но ответ не вернулся». Повтор — единственная разумная стратегия, и она безопасна ровно настолько, насколько операция идемпотентна.

Для PUT и DELETE проблема решена самой семантикой: `PUT /orders/42` с одним и тем же телом десять раз даёт одно состояние, `DELETE` во второй раз вернёт 404 или 204, но состояние не изменится. Для POST так не выйдет: два `POST /orders` создадут два заказа. Здесь вводится **ключ идемпотентности** (idempotency key) — заголовок `Idempotency-Key` с уникальным значением, которое клиент генерирует один раз на логическую операцию.

Сервер хранит таблицу: ключ, хеш тела запроса, код и тело ответа, время жизни (обычно 24 часа). Алгоритм обработки:

1. Ключа `K` нет в таблице — вставить со статусом «в работе», выполнить операцию, сохранить ответ и вернуть его.
2. `K` есть, операция завершена — вернуть сохранённый ответ, не выполняя работу повторно.
3. `K` есть, операция идёт — вернуть `409 Conflict` или `425 Too Early`, чтобы клиент подождал.
4. `K` есть, но хеш тела другой — вернуть `422 Unprocessable Content`: тот же ключ с другим содержимым почти всегда баг клиента.

Ключ должен быть непредсказуемым (годится UUIDv4), а таблица — с уникальным индексом по ключу: без него гонка двух одновременных запросов создаст дубль, потому что между SELECT и INSERT помещается второй запрос.

Повторные запросы на клиенте

Клиентская сторона тоже требует дисциплины. Повторять имеет смысл только потенциально временные ошибки: таймауты, обрывы, `502`, `503`, `504`, `429`. Повторять `400` или `422` бессмысленно — ответ не изменится. Задержка должна расти экспоненциально и включать случайный разброс (jitter), иначе все клиенты, упавшие одновременно, вернутся одновременно и добьют сервер.

python

```
import random, time
import httpx

RETRY_STATUS = {429, 502, 503, 504}

def post_with_retry(url: str, payload: dict, key: str, attempts: int = 4) ->
    httpx.Response:
    headers = {"Idempotency-Key": key}
    for attempt in range(attempts):
        try:
            r = httpx.post(url, json=payload, headers=headers, timeout=5.0)
        except (httpx.ConnectError, httpx.ReadTimeout):
            r = None
        if r is not None and r.status_code not in RETRY_STATUS:
            return r
        if attempt == attempts - 1:
            break
        delay = min(2 ** attempt, 8) + random.uniform(0, 0.5)
        if r is not None and r.headers.get("Retry-After", "").isdigit():
```

```
            delay = max(delay, int(r.headers["Retry-After"]))
        time.sleep(delay)
    raise RuntimeError(f"не удалось выполнить POST после {attempts} попыток")
```

Пагинация: offset против cursor

Коллекция редко помещается в один ответ. Два канонических способа отдать её частями различаются тем, что именно запоминает клиент.

Offset-пагинация (`?limit=50&offset=100` или `?page=3`) опирается на позицию в упорядоченном наборе. Её плюс — возможность прыгнуть на произвольную страницу и простота. Минус — стоимость и нестабильность: `OFFSET 100000` заставляет базу прочитать и выбросить 100 000 строк, а вставка новой записи в начало сдвигает весь набор, поэтому клиент на странице 2 увидит дубль или пропуск.

Cursor-пагинация (`?limit=50&cursor=eyJpZCI6MTIzfQ`) передаёт непрозрачный маркер позиции — обычно закодированную пару «значение поля сортировки + id». Сервер читает `WHERE (created_at, id) < (:ts, :id) ORDER BY created_at DESC, id DESC LIMIT 50`, что попадает в индекс и не зависит от вставок. Цена — нельзя перескочить на страницу 7 и нельзя посчитать общее число страниц (если не делать отдельный `COUNT(*)`).

Свойство	offset	cursor
Прыжок на страницу N	да	нет
Стабильность при вставках	низкая	высокая
Стоимость глубокой страницы	растёт линейно	постоянная
Номер страницы в UI	есть	нет
Сложность реализации	минимальная	средняя

Ответ всегда содержит признак продолжения — иначе клиент не отличит конец данных от забытого курсора:

```json
{
  "items": [{"id": 1042, "title": "Починить CI"}],
  "next_cursor": "eyJ0cyI6IjIwMjYtMTAtMjlUMTA6MDA6MDBaIiwiaWQiOjEwNDJ9",
  "has_more": true
}
```

Курсор непрозрачен для клиента: опубликованную структуру потом нельзя менять. Внутрь стоит класть версию схемы и параметры фильтра, иначе курсор от чужого запроса даст бессмысленный результат.

Фильтры, сортировка, выбор полей

Правила простые, но нарушаются постоянно. Фильтры — параметры запроса с именами полей: `?status=open&assignee=42`. Значения для «ИЛИ» передаются повтором параметра (`?status=open&status=in_progress`) или через запятую — выберите одно и задокументируйте. Сортировка — `?sort=-created_at,title`, минус означает убывание. Выбор полей — `?fields=id,title,status`; это спасает мобильные клиенты от мегабайтных ответов.

Договоритесь о поведении по умолчанию: без фильтра — все записи или только активные? Без сортировки порядок не определён, и cursor-пагинация невозможна. Правило: **сортировка по умолчанию детерминирована**, всегда с `id` последним ключом.

Версионирование

Ломающее изменение (breaking change) рано или поздно понадобится, и единственный способ не сломать старых клиентов — выпустить новую версию рядом со старой.

Подход	Пример	Плюсы	Минусы
URL-префикс	`/v2/orders`	видно в логах и в curl	URL перестаёт быть идентификатором ресурса
Свой заголовок	`Accept: application/vnd.api+json;version=2`	чистые URL, версия — часть представления	не видно в логах, сложнее для отладки
Параметр	`?version=2`	просто	легко забыть, плохо кэшируется
Дата	`Stripe-Version: 2026-10-01`	гранулярность по изменениям	требует инфраструктуры выбора схемы

Для внутренних и большинства публичных API практичнее URL-префикс: он однозначен при разборе инцидента, работает в любом прокси и не требует объяснений. Заголовочное версионирование выигрывает в чистоте и проигрывает в эксплуатации.

Ломает клиентов удаление или переименование поля, смена типа (`"id": 1` → `"id": "1"`), ужесточение валидации, новые обязательные параметры, изменение кода ответа. Не ломает добавление необязательного поля, нового значения в enum или нового параметра. Явно пропишите в контракте, что клиент игнорирует неизвестные поля, — это разрешает расширять ответы без смены версии.

Ошибки: коды и problem+json

Клиент должен уметь обработать ошибку, не читая документацию. Для этого есть три уровня информации: HTTP-статус (класс проблемы), машиночитаемый код (конкретная причина) и человекочитаемое сообщение (для логов и разработчика).

Формат `application/problem+json` описан в RFC 9457 (он заменил RFC 7807) и даёт стандартный каркас:

```http
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "https://api.example.com/problems/validation",
  "title": "Ошибка валидации",
  "status": 422,
  "detail": "Поле due_date не может быть в прошлом",
  "instance": "/v1/tasks/1042",
  "errors": [
    {"field": "due_date", "code": "in_past", "message": "дата уже прошла"}
  ]
}
```

`type` — URI, идентифицирующий **класс** проблемы и ведущий на страницу с описанием; `title` — короткое стабильное название класса; `detail` — конкретика случая; `instance` — путь запроса. Расширения вроде `errors` допустимы и полезны для форм.

Выбор статуса — часть контракта. `400` — битый синтаксис, `401` — не аутентифицирован, `403` — прав нет, `404` — ресурса нет, `409` — конфликт состояния, `422` — синтаксис верный, семантика нет, `429` — слишком много запросов, `500` — баг сервера, `503` — временная недоступность, `504` — не дождались бэкенда. Путать `401` и `403` — классика: клиент с верным токеном получит `401`, пойдёт перелогиниваться и зациклится.

Rate limiting и заголовки

Ограничение частоты защищает от исчерпания ресурсов, брутфорса и перерасхода квоты. Алгоритмы: фиксированное окно (просто, но пропускает двойной всплеск на границе), скользящее окно (точнее, дороже), token bucket (разрешает короткие всплески — обычно лучший выбор), leaky bucket (сглаживает поток).

Клиенту нужно сообщать не только факт отказа, но и когда возвращаться:

```http
HTTP/1.1 429 Too Many Requests
Retry-After: 30
RateLimit-Limit: 100
RateLimit-Remaining: 0
RateLimit-Reset: 30
Content-Type: application/problem+json

{"type": "https://api.example.com/problems/rate-limit", "title": "Слишком много
    запросов", "status": 429}
```

`Retry-After` стандартизован в RFC 9110 и принимает либо число секунд, либо HTTP-дату. Поля `RateLimit-*` на момент написания остаются интернет-драфтом IETF, поэтому их стоит дублировать собственными заголовками (`X-RateLimit-*`), если клиенты уже написаны под них. Важнее другое: лимит нужно считать по субъекту (ключ API, пользователь, IP — в таком порядке предпочтения), а не только по IP, иначе за NAT один шумный клиент заблокирует всех.

Пример: API сервиса задач

Спроектируем контракт для сервиса задач. Ресурсы: `tasks`, `projects`, `comments`.

Метод и путь	Назначение	Успех	Ошибки
`GET /v1/tasks`	список с фильтрами и курсором	200	400, 429
`POST /v1/tasks`	создать задачу	201 + `Location`	409, 422
`GET /v1/tasks/{id}`	одна задача	200	404
`PATCH /v1/tasks/{id}`	частичное изменение	200	404, 409, 422
`DELETE /v1/tasks/{id}`	удалить	204	404
`POST /v1/tasks/{id}/comments`	добавить комментарий	201	404, 422
`GET /v1/tasks/{id}/comments`	комментарии, cursor	200	404

Создание задачи с защитой от дублей:

```http
POST /v1/tasks HTTP/1.1
Host: api.example.com
Authorization: Bearer t_9f3c
Content-Type: application/json
Idempotency-Key: 5b1e0f7a-2c4d-4a91-9f60-1d2b3c4e5f60

{"title": "Обновить nginx до 1.29", "project_id": 7, "due_date": "2026-11-15"}
```

```http
HTTP/1.1 201 Created
Location: /v1/tasks/1042
Content-Type: application/json

{"id": 1042, "title": "Обновить nginx до 1.29", "status": "open", "project_id": 7,
 "due_date": "2026-11-15", "created_at": "2026-10-29T12:00:00Z"}
```

Повтор с тем же `Idempotency-Key` вернёт тот же `201` и тот же `id`, а не создаст второй task. Повтор без ключа создаст дубль — поэтому клиентские SDK обязаны подставлять ключ автоматически.

Список запрашивается как `GET /v1/tasks?status=open&assignee=42&sort=-due_date&limit=50&cursor=...` и возвращает `items`, `next_cursor`, `has_more`. При `has_more: false` клиент прекращает цикл; сочетание `has_more: true` с пустым `next_cursor` — баг сервера и нарушение контракта.

Типичные ошибки

1. **Глаголы в URL.** `POST /getUser`, `POST /updateTask` появляются, когда автор мыслит функциями, а не ресурсами. Следствие: не работает кэш, нельзя отдать `Location`, `PATCH` и `DELETE` неприменимы, а прокси видит только `POST`.
2. **POST без ключа идемпотентности на платёжных и создающих операциях.** Клиент ретраит по таймауту, сервер уже создал заказ — получаются два заказа и два списания. Ошибка проектирования, а не клиента: если операция не идемпотентна по природе, ключ обязателен в контракте.
3. **Offset-пагинация на растущей таблице.** Работает на тестах с 200 записями, разваливается на проде: пользователь листает, видит дубли, база греет диск на `OFFSET 500000`. Проявляется не сразу, поэтому чинить приходится при живых клиентах.
4. **200 OK с `{"success": false}`.** Так делают, чтобы «упростить клиент», и ломают всё: мониторинг не видит ошибок, ретраи не срабатывают, CDN кэширует ошибку как успех. Статус-код — единственный машинно-читаемый сигнал, который видит вся инфраструктура по пути.
5. **Неверсионированный контракт с «мелкими» правками.** Переименование поля кажется безобидным, пока не выясняется, что на нём держится отчёт в чужой системе. Правило: любое изменение, ломающее парсер, — новая версия.
6. **Ошибки без машиночитаемого кода.** `{"message": "Bad request"}` заставляет клиента парсить текст, а тексты меняются. Код ошибки — часть контракта, текст — нет.

Чек-лист

- [ ] Каждый URL — существительное во множественном числе; действия выражены методами.
- [ ] GET и HEAD не меняют состояние; проверено тестом, который дёргает все GET дважды.
- [ ] Все создающие POST принимают `Idempotency-Key`, таблица ключей имеет уникальный индекс и TTL.

- [ ] Коллекции пагинированы, сортировка детерминирована, cursor непрозрачен, `has_more` присутствует.
- [ ] Версия API зафиксирована в контракте, политика ломающих изменений описана.
- [ ] Ошибки отдаются как `application/problem+json` с полями `type`, `title`, `status`, `detail`.
- [ ] Коды 401/403/409/422/429 используются по назначению, а не «на всякий случай».
- [ ] Для 429 и 503 отдаётся `Retry-After`, лимиты считаются по ключу, а не только по IP.

Упражнения

1. Возьмите API любого публичного сервиса (например, GitHub) и найдите три эндпоинта, где ресурс назван глаголом или действие выражено через POST. Предложите REST-альтернативу и объясните, что потеряется при переходе.
2. Спроектируйте эндпоинты для «перевести задачу в другой проект» и «схлопнуть дубликаты задач». Для каждого укажите метод, путь, тело, успешный статус и идемпотентность.
3. Реализуйте middleware ключа идемпотентности на любом языке: таблица в PostgreSQL, уникальный индекс, сохранение ответа, обработка конфликта тела. Проверьте двумя параллельными запросами с одним ключом.
4. Переведите список с `offset` на `cursor` и сравните `EXPLAIN ANALYZE` на глубине 100 000 строк.
5. Опишите API задач в OpenAPI и сгенерируйте клиент; проверьте, что он разбирает `problem+json` и ретраит только разрешённые статусы.

Ответы и разбор к этим упражнениям — в приложении D, часть 2.

Глава 14. Прокси, балансировщики, reverse proxy

Почти ни один продакшн-запрос не идёт от клиента прямо в приложение. По пути стоят посредники: TLS-терминатор, балансировщик, кэширующий прокси, WAF, sidecar. Они решают, куда отправить соединение, сколько ждать ответа и что делать, если бэкенд молчит. Отсюда большинство «магических» ошибок эксплуатации: откуда взялся `502`, почему клиент видит IP балансировщика, почему после включения sticky sessions один под получает всю нагрузку. Дальше — прямой и обратный прокси, L4 и L7, алгоритмы балансировки, health checks, таймауты, `X-Forwarded-For` и рабочая конфигурация nginx.

Прямой и обратный прокси

Разница между ними — в том, чьи интересы представляет прокси.

Прямой прокси (forward proxy) стоит на стороне клиента и действует от его имени. Клиент настраивает его в браузере или переменной `HTTPS_PROXY`; прокси выходит в интернет, скрывает клиентский адрес, применяет политики (кэш, запреты, аудит). Сервер видит соединение от прокси и не знает, кто за ним.

Обратный прокси (reverse proxy) стоит на стороне сервера и представляет сервер для клиента. Клиент думает, что говорит с приложением, а говорит с nginx, HAProxy, Envoy или ALB. Он берёт на себя задачи, которые неудобно или опасно решать в приложении:

- терминация TLS: сертификат и шифры в одном месте, приложение слушает открытый HTTP в приватной сети;
- маршрутизация по путям и хостам: `/api` в один сервис, `/static` в объектное хранилище;
- сжатие, кэш статики, ограничение размера тела, защита от медленных клиентов;

- единая точка для логов, метрик и трассировки, сокрытие внутренней топологии.

Обратный прокси — ещё и точка отказа: если он один, N независимых сервисов превращаются в одну общую зависимость. Поэтому в продакшне их минимум два, а перед ними — DNS с несколькими A-записями или облачный L4-балансировщик.

L4 и L7: что именно балансируем

Балансировщик L4 (transport) работает с TCP-соединениями: видит IP, порт, флаги, но не содержимое; бэкенд выбирается при установке соединения, дальше байты перекладываются прозрачно. Уровень L7 (application) разбирает HTTP — метод, путь, заголовки, куки — и решает по каждому запросу, а не по соединению.

Свойство	L4	L7
Что видит	IP, порт, TCP-сессия	метод, путь, заголовки, тело
TLS	проброс или терминация без разбора	терминация, SNI-маршрутизация, mTLS
Маршрутизация по пути	нет	да
Переиспользование соединений к бэкенду	нет (1:1)	да, пул keepalive
Стоимость на запрос	минимальная	выше: разбор и буферизация
Примеры	IPVS/LVS, AWS NLB	nginx, HAProxy, Envoy, AWS ALB

L4 дешевле и быстрее, L7 даёт управляемость. Типичная схема: облачный L4 принимает трафик и держит «грязную» часть интернета (SYN-флуд, медленные клиенты), за ним L7 занимается маршрутизацией и TLS, дальше — приложение.

Тонкость с TLS: при пробросе зашифрованного потока бэкенд видит IP балансировщика и не может применить `X-Forwarded-For` — заголовки внутри шифра. Решения два: терминация TLS на L7, после которой заголовки добавляются, или протокол PROXY, передающий исходный адрес бинарным заголовком в начале TCP-соединения (требует поддержки на бэкенде).

Алгоритмы балансировки

Алгоритм определяет, какой бэкенд получит запрос. Универсально лучшего нет.

Алгоритм	Как выбирает	Когда уместен
Round robin	по кругу	однородные бэкенды, однородные запросы
Weighted round robin	по кругу с весами	разная мощность машин, канареечный деплой
Least connections	у кого меньше активных соединений	запросы разной длительности
Least response time	меньше соединений и меньше среднее время	чувствительные к задержке API
IP hash	хеш адреса клиента	нужна привязка клиента без куки
Consistent hash	хеш ключа (URL, user id) по кольцу	распределённый кэш, шардирование

Round robin предполагает, что все запросы равны. В реальности один читает 10 строк, другой запускает отчёт на 40 секунд: длинные запросы копятся на случайных бэкендах, и распределение перекашивается даже при равном их числе. Least connections исправляет это, учитывая незавершённые запросы, а не выданные.

Consistent hashing нужен, когда за балансировщиком узлы с локальным состоянием — кэш в памяти. При `hash(key) % N` добавление одного узла меняет отображение почти всех ключей и инвалидирует кэш целиком; кольцо консистентного хеширования переносит только долю, пропорциональную числу узлов.

Health checks

Балансировщик должен знать, жив ли бэкенд, иначе он шлёт трафик в мёртвый процесс.

Пассивная проверка (passive) не шлёт дополнительных запросов, а наблюдает за реальным трафиком: после нескольких ошибок соединения или `5xx` бэкенд помечается недоступным на время. В nginx это `max_fails` и `fail_timeout` в директиве `server`. Плюс

— нулевые накладные расходы; минус — бэкенд получает реальные запросы пользователей, пока его не исключат, а без трафика его состояние не проверяется вовсе.

Активная проверка (active) шлёт свой запрос на эндпоинт с фиксированным интервалом, например `GET /healthz` каждые 2 секунды, и снимает бэкенд после трёх неудач. Так делают HAProxy, Envoy, Kubernetes (`readinessProbe`/`livenessProbe`). Плюс — отказ обнаруживается до того, как пострадал пользователь; минус — дополнительная нагрузка и риск, что проверка проверяет не то.

Ключевое различие — **liveness против readiness**. Liveness отвечает на «жив ли процесс или его надо перезапустить», и зависимостей в ней быть не должно: если проба ходит в базу, при падении базы оркестратор перезапустит все поды, превратив деградацию в полную недоступность. Readiness отвечает на «готов ли принимать трафик» — здесь проверки зависимостей уместны, потому что под просто выведут из балансировки.

Проверка должна быть дешёвой и честной: `GET /healthz`, всегда возвращающий `200`, показывает лишь, что event loop жив, а тяжёлый `SELECT` каждые 2 секунды с сотни подов сам становится нагрузкой.

Sticky sessions и почему они вредны

Sticky sessions (session affinity) — привязка клиента к бэкенду кукой, хешем IP или полем заголовка. Смысл — попадать в процесс, который хранит состояние клиента в памяти. Соблазн понятен: сессия лежит в словаре, и без привязки пользователь «разлогинивается» после каждого запроса. Проблем у привязки больше, чем пользы:

- **Неравномерность.** Клиенты приходят пачками: один офис за NAT — один IP — один бэкенд. При хеше по IP распределение перекошено, при куке — ещё и по времени жизни сессии.
- **Деплой ломает сессии.** Rolling update заменяет поды, сессии на них теряются; привязка маскирует проблему до первого релиза.

- **Скейл вниз и канареечный деплой невозможны.** Уменьшение реплик выбрасывает живых пользователей, а привязка фиксирует клиента на старой версии, пока жива кука. Отладка тоже страдает: «у меня работает» перестаёт что-либо значить.

Правильное решение — вынести состояние из процесса: сессии в Redis или другой общий store, либо stateless-аутентификация на подписанных токенах (JWT, RFC 7519), где состояние несёт клиент. Тогда любой бэкенд обслуживает любой запрос, деплой безопасен, масштабирование линейно. Привязка допустима как временная мера при миграции и для кэша с локальным состоянием — но и там лучше консистентное хеширование по ключу, а не по клиенту.

Таймауты и откуда берутся 502 и 504

Таймаут — не одно число, а набор, и ошибки возникают на стыках слоёв. Цепочка: клиент → облачный балансировщик → nginx → приложение → база.

```text
клиент —(1)—> LB —(2)—> nginx —(3)—> app —(4)—> БД
```

На каждом переходе свои параметры:

- **(1) клиент → LB.** Таймаут клиента и idle timeout балансировщика: если LB рвёт простой через 60 с, а клиент ждёт 90 с, он получит обрыв, хотя сервер ещё работает.
- **(2) LB → nginx.** Connect timeout (ожидание TCP) и response timeout (ожидание первого байта).
- **(3) nginx → app.** `proxy_connect_timeout`, `proxy_read_timeout`, `proxy_send_timeout`. Последний — таймаут **между чтениями**, а не общий: медленный сервис может отдавать ответ минутами, если исправно присылает байты.
- **(4) app → БД.** Таймаут пула, statement timeout, сетевой таймаут драйвера.

Главное правило: **внешний таймаут больше внутреннего**. Если приложение ждёт базу 30 с, а nginx рвёт соединение через

10 с, пользователь видит ошибку, а приложение продолжает работу и занимает соединения: клиент ретраит, запросы копятся, пул исчерпан.

Отсюда два разных кода ошибки:

- **502 Bad Gateway** — прокси не соединился с бэкендом или получил невалидный ответ: приложение упало, порт закрыт, соединение сброшено.
- **504 Gateway Timeout** — соединение установлено, ответа не дождались. Бэкенд жив, но медленный.

Третий, невидимый в логах приложения, — **499** (нестандартный код nginx): клиент закрыл соединение, не дождавшись ответа. Массовые 499 означают, что клиентский таймаут короче времени обработки, и править надо не сервер, а контракт по времени.

Приём — бюджет таймаутов: на входе фиксируется допустимое время ответа (например, 2 с для API), и каждый внутренний вызов получает долю бюджета. Тогда времена складываются предсказуемо, а не «30 с на каждом слое, итого 4 минуты».

X-Forwarded-For и доверие заголовкам

Когда запрос проходит через прокси, `remote_addr` на бэкенде — это адрес прокси. Чтобы приложение узнало клиента, прокси дописывает заголовок `X-Forwarded-For` (и часто `X-Forwarded-Proto`, `X-Forwarded-Host`, `X-Real-IP`).

`X-Forwarded-For` — список адресов через запятую; каждый прокси добавляет адрес того, от кого получил запрос:

```http
X-Forwarded-For: 203.0.113.7, 10.0.0.12, 10.0.0.31
```

Читать нужно **справа налево**: последний элемент добавлен вашим доверенным прокси и достоверен, первый — то, что заявил клиент, и подделывается тривиально:

```bash
curl -H 'X-Forwarded-For: 1.2.3.4' https://api.example.com/v1/tasks
```

Если приложение берёт первый адрес, любой подставит нужный IP — и обойдёт rate limiting, подделает геолокацию, отравит логи. Правильно: доверять заголовку только от известных адресов прокси и отбрасывать всё, что пришло снаружи. В nginx для этого есть модуль `realip`:

```nginx
upstream tasks_api {
    least_conn;
    server 10.0.1.11:8080 max_fails=3 fail_timeout=10s;
    server 10.0.1.12:8080 max_fails=3 fail_timeout=10s;
    server 10.0.1.13:8080 max_fails=3 fail_timeout=10s backup;
    keepalive 64;
}

server {
    listen 443 ssl;
    server_name api.example.com;
    # ssl_certificate и ssl_certificate_key — терминация TLS здесь

    location /v1/ {
        proxy_pass http://tasks_api;
        proxy_http_version 1.1;
        proxy_set_header Connection          "";
        proxy_set_header Host                $host;
        proxy_set_header X-Real-IP           $remote_addr;
        proxy_set_header X-Forwarded-For     $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto   $scheme;

        proxy_connect_timeout 2s;
        proxy_send_timeout    10s;
        proxy_read_timeout    30s;

        proxy_next_upstream error timeout http_502 http_503;
        proxy_next_upstream_tries 2;
        proxy_buffering on;
    }
}
```

Каждая директива здесь — решение с последствиями.

`least_conn` выбирает бэкенд с наименьшим числом активных соединений: разумный дефолт для API с запросами разной длительности. `max_fails=3 fail_timeout=10s` — пассивная проверка: после трёх неудач за окно бэкенд выводится из пула на 10 секунд, затем получает пробный запрос; слишком малые значения дают «мигание» при единичных сбоях, слишком большие — долгое обслуживание мёртвого узла. `backup` помечает резервный сервер, который получает трафик только при

недоступности основных: дешёвый способ пережить потерю зоны.

`keepalive 64` держит пул из 64 простаивающих соединений к каждому бэкенду. Без него nginx открывает TCP-соединение на каждый запрос — лишний handshake, лишние `TIME_WAIT`, лишняя нагрузка. Для пула обязательны `proxy_http_version 1.1` и пустой `Connection: ""`: при HTTP/1.0 соединение закрывается после каждого ответа.

`proxy_set_header Host $host` сохраняет исходный хост, иначе бэкенд с несколькими доменами выберет не тот виртуальный хост. `X-Forwarded-Proto $scheme` сообщает приложению о HTTPS, иначе абсолютные ссылки и редиректы уедут на `http://`. `proxy_read_timeout 30s` — не общий таймаут запроса, а пауза между чтениями: для стриминга и SSE его поднимают осознанно, иначе поток оборвётся на паузе в данных.

`proxy_next_upstream error timeout http_502 http_503` разрешает повтор на другом бэкенде. Для GET это спасение, для **неидемпотентных POST** — источник дублей: если первый бэкенд создал заказ и упал, повтор создаст второй. Включайте `http_502`/`http_503` только вместе с ключами идемпотентности из главы 13 либо ограничьте повтор методами `GET`/`HEAD`. `proxy_buffering on` заставляет nginx принять ответ целиком и отдавать клиенту в его темпе, чтобы медленный клиент не занимал worker бэкенда; для стриминга буферизацию выключают.

Проверить и применить (`nginx -t` при ошибке печатает файл и строку):

```bash
nginx -t
systemctl reload nginx
```

`reload` не рвёт существующие соединения: старые worker-процессы дорабатывают запросы, новые подхватывают слушающие сокеты. Поэтому reload, а не restart.

Типичные ошибки

1. **Таймаут снаружи меньше, чем внутри.** nginx рвёт соединение через 10 с, приложение ждёт базу 30 с. Пользователь получает 504, приложение продолжает работу вхолостую, клиент ретраит, нагрузка растёт. Механизм: таймауты настраивались по слоям разными людьми и никогда не сверялись.
2. **Liveness-проба проверяет базу.** База недоступна 20 секунд — оркестратор перезапускает все поды, и восстановление занимает минуты. Механизм: смешение liveness и readiness; перезапуск не лечит внешнюю зависимость.
3. **Доверие** `X-Forwarded-For` **откуда угодно.** Приложение читает первый адрес заголовка, и rate limiting обходится одним `curl -H`. Механизм: заголовок выглядит как данные, но является утверждением клиента, пока не доказано обратное.
4. **Sticky sessions вместо общего хранилища сессий.** Работает на двух репликах в тесте, разваливается на десяти в проде: перекос, потеря сессий при деплое, невозможность скейла вниз. Механизм: состояние оставили в памяти процесса, потому что «так проще», а привязку приняли за решение.
5. `proxy_next_upstream` **на все методы.** Повтор POST на другом бэкенде создаёт дубли заказов при таймауте первого. Механизм: директива выглядит как «сделать надёжнее», но надёжность требует идемпотентности.
6. **Один обратный прокси.** Любой его рестарт — полная недоступность всех сервисов. Механизм: прокси воспринимается как инфраструктурная мелочь, а не как единая точка отказа.

Чек-лист

- [ ] Таймауты на каждом слое выписаны в таблицу и убывают снаружи внутрь.
- [ ] Для 502 и 504 настроены разные алерты: причина у них разная.

- [ ] Liveness-проба не трогает внешние зависимости, readiness — трогает.
- [ ] `set_real_ip_from` ограничен доверенными сетями, IP клиента берётся справа налево.
- [ ] `proxy_next_upstream` не включает методы, которые не идемпотентны.
- [ ] Пул `keepalive` к бэкендам включён, `proxy_http_version 1.1`, `Connection` пустой.
- [ ] Сессии хранятся вне процесса; привязка если и есть, то осознанно и с планом отказа.
- [ ] Конфиг проверяется `nginx -t` в CI и применяется через `reload`, а не `restart`.

Упражнения

1. Нарисуйте путь запроса в вашем проекте от клиента до базы и выпишите все таймауты. Найдите места, где внешний таймаут меньше внутреннего.
2. Поднимите nginx с двумя бэкендами и `max_fails=2`, погасите один и снимите поведение: сколько запросов упадёт до исключения узла, когда он вернётся в пул.
3. Сравните round robin и least_conn на запросах двух классов (10 мс и 2 с) и объясните разницу в p99.
4. Настройте `real_ip` и докажите с помощью `curl -H 'X-Forwarded-For: ...'`, что подделка игнорируется; затем сломайте конфиг и покажите, как подделка проходит.
5. Переведите сервис со sticky sessions на Redis без простоя: опишите шаги и точки отката.

Ответы и разбор к этим упражнениям — в приложении D, часть 2.

Глава 15. nginx и event loop: почему один процесс держит 10k соединений

«nginx держит десять тысяч соединений на процесс» звучит как маркетинг, пока не поймёшь, чего это стоит: один поток, один `epoll`, ни одного блокирующего вызова. За цифрой стоит разница между двумя моделями конкурентности — «поток на соединение» и «цикл событий на все соединения». Она объясняет, почему Apache prefork падает под нагрузкой, где nginx теряет запросы, зачем нужна буферизация и как устроен asyncio, работающий по тому же принципу. Дальше — блокирующий и неблокирующий ввод-вывод, `select`/`poll`/`epoll`, C10K, worker-процессы и accept-мьютекс, конфиг nginx с разбором директив и сравнение с Apache и asyncio.

Блокирующий ввод-вывод

Сокет — файловый дескриптор. Когда процесс вызывает `read()` на дескрипторе, а данных в буфере ядра нет, процесс засыпает, планировщик выбирает другую задачу, а приход пакета будит спящего. Для программы это выглядит как «вызов занял 40 миллисекунд», хотя процессор в это время работал на кого-то другого.

Модель проста: код читается сверху вниз, ошибки очевидны. Её цена — **поток или процесс на каждое соединение**. Пока клиент медленно присылает запрос, поток заблокирован, ничего не делает, но занимает память: стек в Linux по умолчанию 8 МБ виртуальной памяти, плюс структуры планировщика, ядра и буферы сокета. Практическая оценка — от десятков до сотен килобайт resident на соединение, то есть гигабайты на 10 000 соединений.

Вторая проблема — переключение контекста: единицы микросекунд на смену задачи, и при 10 000 соединений, каждое из которых просыпается по нескольку раз в секунду, процессор тратит заметную долю времени на смену задач вместо работы.

Именно поэтому Apache в режиме prefork, где запрос обслуживает отдельный процесс, упирается в потолок задолго до исчерпания сети или диска.

Неблокирующий ввод-вывод и мультиплексирование

Альтернатива — не ждать в вызове. Дескриптор переводится в неблокирующий режим (`O_NONBLOCK`), и `read()` на пустом сокете сразу возвращает `-1` с `errno == EAGAIN`. Программа не спит, а делает что-то другое. Но перебор «опросить все 10 000 сокетов» — busy loop, сжигающий ядро впустую; нужен механизм, который сам скажет, что готово. Это и есть мультиплексирование.

`select()` — первый интерфейс: три битовых набора (чтение, запись, исключения) и максимальный номер дескриптора. Ограничение `FD_SETSIZE` (обычно 1024) делает его непригодным для C10K, а линейный скан на каждый вызов — неэффективным и ниже предела.

`poll()` снимает ограничение: передаётся массив дескрипторов с масками событий, но сложность остаётся O(n) — ядро проверяет весь массив, даже если готов один сокет.

`epoll` (Linux) меняет постановку задачи: дескриптор один раз регистрируется в наборе (`epoll_ctl`), а `epoll_wait` возвращает только готовые — O(k) по числу готовых, без копирования массива. `kqueue` в BSD и macOS — то же самое идеологически.

Механизм	Ограничение	Сложность вызова	Копирование	Платформа
`select`	`FD_SETSIZE`, обычно 1024	O(n)	наборы каждый раз	везде
`poll`	нет жёсткого	O(n)	массив каждый раз	везде
`epoll`	нет	O(k) готовых	нет	Linux
`kqueue`	нет	O(k) готовых	нет	BSD, macOS

У `epoll` есть два режима. **Level-triggered** (по умолчанию) сообщает о готовности, пока в буфере есть данные: не прочитал всё — получишь событие снова. **Edge-triggered** (`EPOLLET`)

сообщает один раз при изменении состояния: не вычитал всё до `EAGAIN` — событие больше не придёт, и соединение «залипнет». Edge-режим быстрее (меньше пробуждений), но требует дисциплины: читать в цикле до `EAGAIN`.

Ключевое свойство всей модели: **пока обработчик события не делает блокирующих вызовов, один поток обслуживает сколько угодно соединений**. Стоит в обработчике сделать блокирующий `read()` с диска или синхронный запрос в базу — и встанет весь цикл, все 10 000 соединений. Это главная ловушка event loop, и она одинаково работает в nginx, Node.js и asyncio.

C10K и почему модель важна

Задача C10K — «обслужить 10 000 одновременных соединений на одной машине» — сформулирована в конце 1990-х как инженерный вызов: железо позволяло, архитектура серверов нет. 10 000 процессов по 8 МБ стека — это 80 ГБ виртуальной памяти и десятки тысяч переключений контекста в секунду.

Решение — комбинация трёх идей: неблокирующие сокеты, `epoll`/`kqueue` и небольшое число потоков, каждый из которых обслуживает много соединений. nginx построен ровно так, и его производительность объясняется не хитрым кодом на C, а отсутствием лишних сущностей: 10 000 простаивающих keepalive-соединений — это 10 000 записей в структуре событий и ни одного потока.

Worker-процессы и accept-мьютекс

nginx запускается как master-процесс и N worker-процессов. Master не обслуживает клиентов: он читает конфиг, открывает слушающие сокеты, порождает воркеров и управляет ими по сигналам (`SIGHUP` — reload, `SIGTERM` — graceful shutdown, `SIGQUIT` — быстрый). Worker делает всю работу: принимает соединения, разбирает HTTP, ходит в upstream, пишет в лог.

Слушающий сокет наследуется воркерами от master, поэтому все они ждут на одном и том же дескрипторе. Кто именно получит новое соединение — вопрос реализации:

- **accept-мьютекс.** Воркеры по очереди захватывают общий мьютекс, принимают пачку соединений и отпускают. Это защищает от «thundering herd» — ситуации, когда все воркеры просыпаются на одном событии, а соединение достаётся одному, и остальные впустую тратят процессорное время. Мьютекс включается директивой `accept_mutex` и особенно полезен при низкой нагрузке, когда новых соединений мало, а воркеров много.
- `SO_REUSEPORT`**.** Каждый воркер открывает собственный слушающий сокет на том же порту, и ядро само распределяет входящие соединения между ними по хешу четвёрки. Мьютекс не нужен вовсе, распределение равномерное, но при reload старые и новые воркеры могут одновременно держать сокеты — это надо учитывать при выкатке.

Директива `worker_processes` задаёт число воркеров. Значение `auto` подставляет число доступных ядер. Логика простая: воркер — это цикл событий, он выполняет код на CPU, и лишние воркеры не добавят параллелизма, а добавят переключений контекста и борьбы за кэш.

Буферизация

Прокси может либо перекладывать байты потоком, либо принять ответ upstream целиком и только потом отдавать клиенту. Второй режим — `proxy_buffering on`, включённый по умолчанию, — важен именно в событийной архитектуре.

Причина — непредсказуемая скорость клиента. Отдавая ответ мобильному клиенту напрямую, nginx держит соединение с upstream открытым всё время медленной передачи, а у upstream соединений конечное число. С буферизацией nginx быстро вычитывает ответ, освобождает upstream и отдаёт клиенту из памяти или временного файла в его темпе.

Параметры:

- `proxy_buffers 16 16k` — число и размер буферов на одно соединение (по умолчанию 8 × 4k или 8 × 8k в зависимости от платформы);

- `proxy_buffer_size` — размер буфера для первой части ответа, где лежат заголовки;
- `proxy_busy_buffers_size` — сколько буферов можно отдавать клиенту, пока остальные ещё заполняются;
- `proxy_max_temp_file_size` — предел файла на диске для ответов, не влезших в память; `0` отключает запись на диск.

Буферизацию выключают для стриминга, SSE и больших файлов: задержка на заполнение буфера воспринимается как «сайт тормозит», а временный файл — как лишняя нагрузка на I/O. Обратная сторона — `client_body_buffer_size` и `client_max_body_size`: медленный клиент, присылающий тело по байту, занимает соединение, и ограничивают его `client_body_timeout` и `client_header_timeout` (защита от slowloris).

Когда воркеров больше, чем ядер

Стандартная рекомендация — `worker_processes auto`, то есть по числу ядер. Но есть случаи, когда воркеров ставят больше:

- **Много ожиданий.** При `proxy_pass` к медленному upstream без буферизации воркер проводит время в ожидании, а не на CPU, и дополнительные воркеры повышают утилизацию ядер. Это компромисс: лечится симптом, а причина — отсутствие асинхронности — остаётся.
- **Дисковый I/O.** Чтение статики с медленного диска или сетевой ФС блокирует воркер на время syscall; на NVMe это незаметно.
- **Изоляция.** Разные воркеры могут обслуживать разные `server`-блоки, чтобы падение одного не задело остальные.

Цена — переключения контекста и худшая локальность кэша. Практическое правило: начинать с `auto`, смотреть на метрики и добавлять воркеры только при доказанном простое на I/O; больше двух-трёхкратного числа ядер почти никогда не помогает.

Конфигурация с пояснениями

nginx

```
worker_processes auto;
```

```
worker_rlimit_nofile 65535;

events {
    worker_connections 8192;
    multi_accept on;
    use epoll;
}

http {
    sendfile on;
    tcp_nopush on;
    keepalive_timeout 65s;
    client_header_timeout 10s;
    client_body_timeout 10s;

    upstream app {
        server 10.0.1.11:8080;
        keepalive 32;
    }

    server {
        listen 80 backlog=4096;

        location / {
            proxy_pass http://app;
            proxy_http_version 1.1;
            proxy_set_header Connection "";
            proxy_buffers 16 16k;
            proxy_busy_buffers_size 32k;
            proxy_max_temp_file_size 0;
            proxy_connect_timeout 2s;
            proxy_read_timeout 30s;
        }
    }
}
```

`worker_rlimit_nofile 65535` поднимает лимит дескрипторов воркера: каждый клиент — минимум один дескриптор, плюс соединения к upstream и лог-файлы. При лимите 1024 (частый дефолт systemd) `worker_connections 8192` бессмысленны — воркер упрётся в ядро и начнёт отдавать `Too many open files`.

`worker_connections 8192` — сколько соединений обслуживает воркер. Потолок клиентов — `worker_processes × worker_connections`, но при проксировании каждому клиенту нужно ещё соединение к upstream, так что реальный предел примерно вдвое ниже: 4 воркера × 8192 дают около 16 000 клиентов, с запасом к заявленным 10k.

`multi_accept on` разрешает принять все ожидающие соединения за одно пробуждение — экономит пробуждения при всплесках. `use`

`epoll` задаёт механизм явно; nginx и сам выбирает лучший, но явное указание снимает вопросы при переносе конфига.

`sendfile on` заставляет ядро копировать файл из page cache в сокет без промежуточного копирования в пользовательское пространство, `tcp_nopush on` собирает заголовки и начало файла в один пакет. `keepalive_timeout 65s` — сколько держать простаивающее соединение: экономит TLS-handshake, но занимает слот воркера.

`client_header_timeout` и `client_body_timeout` — защита от slowloris: соединение, не приславшее заголовки или тело вовремя, закрывается. Это не таймаут запроса, а ограничение на бездействие при передаче.

`listen 80 backlog=4096` — очередь необработанных соединений: при SYN-всплеске и малой очереди ядро отбрасывает новые соединения, и клиенты видят таймаут вместо отказа.

Сравнение с Apache prefork и asyncio

Модель	Единица конкурентности	Память на соединение	Изоляция сбоя	Цена переключения
Apache prefork	процесс	высокая	полная	высокая
Apache worker/event	поток / событие	средняя / низкая	частичная	средняя
nginx	событие в воркере	десятки КБ	по воркерам	минимальная
asyncio	корутина	единицы КБ	общая (один процесс)	минимальная

Apache prefork даёт максимальную изоляцию: падение обработчика не задевает соседей, а модули, не умеющие работать в многопоточном режиме, работают. Цена — память и медленный старт под нагрузкой. `mpm_event` в Apache — попытка совместить пул потоков с асинхронным приёмом соединений, и он ближе к nginx по возможностям, но исторически сложнее в настройке.

asyncio в Python устроен по той же схеме, что и nginx: один поток, `epoll` (через `selectors`), набор задач, которые добровольно отдают управление на `await`. Разница в том, что nginx написан на C и не

имеет GIL, а Python-корутины выполняются в одном потоке и не дают параллелизма на CPU. Для ввода-вывода это неважно: пока корутина ждёт сеть, другие работают.

python

```
import asyncio

async def handle(reader: asyncio.StreamReader, writer: asyncio.StreamWriter) ->
    None:
    request = await reader.readuntil(b"\r\n\r\n")   # await = отдать управление
        циклу
    body = b"HTTP/1.1 200 OK\r\nContent-Length: 2\r\n\r\nok"
    writer.write(body)
    await writer.drain()
    writer.close()
    await writer.wait_closed()

async def main() -> None:
    server = await asyncio.start_server(handle, "127.0.0.1", 8080)
    async with server:
        await server.serve_forever()

asyncio.run(main())
```

`await reader.readuntil(...)` — точка, где задача засыпает и цикл переключается на другие соединения. Ошибка новичка — блокирующая функция внутри обработчика (`time.sleep`, `requests.get`, тяжёлый `json.loads` на мегабайтном теле): цикл встанет целиком, как воркер nginx. Правило одно и то же: **в обработчике события не должно быть блокирующих вызовов**.

Типичные ошибки

1. **Блокирующий вызов в обработчике.** `requests.get` или `time.sleep` внутри async-функции останавливает весь цикл: все соединения ждут, RPS падает до нуля, а профилировщик показывает «всё в системных вызовах». Механизм: код выглядит асинхронным, но внутри `await` нет.
2. `worker_connections` **больше, чем** `worker_rlimit_nofile`**.** Конфиг обещает 8192 соединения на воркер, ядро даёт 1024 дескриптора, и nginx пишет `Too many open files`, отдавая 500. Механизм: два лимита в разных местах, второй обычно задаётся в systemd-юните и забывается.
3. `sendfile on` **без учёта сжатия.** Файл отдаётся из page cache, но `gzip` требует прочитать его в пользовательское пространство, и

`sendfile` тихо отключается. Механизм: директива выглядит как «ускорение», но несовместима с обработкой тела.

4. **Буферизация выключена «для скорости».** Медленные клиенты держат соединения с upstream, пул исчерпывается, здоровые запросы получают 504. Механизм: путаница между задержкой первого байта и пропускной способностью.
5. **Воркеров ставят «побольше».** 64 воркера на 8 ядрах дают переключения контекста и промахи кэша; throughput не растёт, latency растёт. Механизм: интуиция «больше процессов — больше параллелизма» не работает для CPU-bound цикла.
6. `proxy_read_timeout` **понимают как общий таймаут запроса.** Ставят 5 с, и долгие отчёты рвутся на паузе между байтами: директива ограничивает паузу, а не запрос целиком.
7. **Один воркер на многоядерной машине.** Остальные ядра простаивают. Механизм: `worker_processes 1` осталось с отладки или с однопроцессорной ВМ.

Чек-лист

- [ ] `worker_processes auto`, число воркеров обосновано числом ядер, а не привычкой.
- [ ] `worker_rlimit_nofile` не меньше, чем `worker_connections × 2`, и продублирован в systemd-юните.
- [ ] `worker_connections` рассчитано с учётом соединений к upstream, а не только клиентских.
- [ ] Ни в одном обработчике нет блокирующих вызовов; проверено нагрузочным тестом с профилем.
- [ ] `proxy_buffering on` для API, выключен для стриминга и SSE.
- [ ] `client_header_timeout` и `client_body_timeout` заданы явно, `client_max_body_size` ограничен.
- [ ] `backlog` слушающего сокета достаточен для пиковой скорости приёма соединений.
- [ ] Reload проверен под нагрузкой: старые воркеры дорабатывают, новые принимают.

Упражнения

1. Напишите на Python два сервера — блокирующий «поток на соединение» и asyncio-вариант — и сравните потребление памяти на 1000 простаивающих соединений.
2. Поднимите nginx с `worker_connections 8192` и `worker_rlimit_nofile 1024`, доведите до `Too many open files` и почините, не меняя конфиг nginx.
3. Измерьте `ab` или `wrk` на статике с `sendfile on` и `off` при включённом gzip. Объясните результат.
4. Сравните level-triggered и edge-triggered `epoll` на своём коде: сделайте клиента, который присылает данные порциями, и покажите, где edge-режим теряет события.
5. Нагрузите nginx с 4 и с 32 воркерами на 8 ядрах, снимите RPS и p99 и объясните разницу через `vmstat` и `pidstat`.

Ответы и разбор к этим упражнениям — в приложении D, часть 2.

Глава 16. Python: окружение

Половина проблем «у меня не работает, а у тебя работает» решается не кодом, а окружением. Разные версии интерпретатора, глобально установленные пакеты, забытый `pip install`, конфликт зависимостей двух проектов на одной машине — всё это исчезает, когда окружение описано файлами и воспроизводится одной командой. В этой главе — что такое CPython и байткод, зачем нужен GIL (и чего он не делает), как устроен `venv`, чем отличаются pip, uv и poetry, как выглядит современный `pyproject.toml`, зачем нужен src-layout, как обращаться с переменными окружения и как добиться воспроизводимых сборок. Актуальная ветка на момент написания — Python 3.14 (3.14.0 вышла 7 октября 2025, последний патч 3.14.8 — 30 сентября 2026).

Интерпретатор и байткод

CPython — эталонная реализация Python на C. Она не исполняет текст программы напрямую: при импорте модуль компилируется в **байткод** (bytecode) — компактные инструкции для виртуальной машины CPython, и результат складывается в `__pycache__/module.cpython-314.pyc`. Байткод кэшируется и переиспользуется, пока не изменится исходник или версия интерпретатора.

Посмотреть, во что превращается код, можно модулем `dis`:

bash

```
python3 -c "import dis; dis.dis('sum(x * 2 for x in range(3))')"
```

text

```
  0 RESUME                   0
  2 LOAD_GLOBAL              1 (NULL + sum)
 12 LOAD_CONST               0 (<code object <genexpr> ...>)
 14 MAKE_FUNCTION            0
 16 LOAD_GLOBAL              3 (NULL + range)
 28 LOAD_CONST               1 (3)
 30 CALL                     1
 32 GET_ITER
 34 CALL                     1
 36 RETURN_VALUE
```

Байткод — не машинный код и не «полукомпиляция для скорости»: он нужен для переносимости и быстрого старта. Виртуальная машина читает инструкции, работает со стеком значений и обращается к объектам Python. Отсюда две вещи, важные на практике.

Первая: `.pyc` — кэш, а не артефакт. Его не коммитят, не переносят между версиями и спокойно удаляют; при следующем импорте он создастся заново. Второе: байткод привязан к минорной версии интерпретатора — файл от 3.13 не подхватится на 3.14, что видно по имени `cpython-314`.

Python 3.14 принёс изменения, заметные и в рантайме: t-strings (шаблонные строки, PEP 750), отложенное вычисление аннотаций (PEP 649/749) и субинтерпретаторы в стандартной библиотеке. Первое меняет работу с шаблонами, второе — импорт модулей с аннотациями, третье открывает путь к параллелизму без GIL. Для окружения важно другое: **версия интерпретатора — часть контракта проекта**, и её надо фиксировать так же, как зависимости.

GIL в двух словах

GIL (Global Interpreter Lock) — мьютекс, который позволяет исполнять байткод только одному потоку одновременно. Даже на 16 ядрах восемь потоков Python не ускорят вычисление в восемь раз: они будут по очереди захватывать GIL.

Зачем он нужен: управление памятью в CPython — подсчёт ссылок. Каждое присваивание меняет счётчик объекта, и без глобальной блокировки два потока могли бы одновременно испортить счётчик и освободить живую память. GIL — грубое, но дешёвое решение этой проблемы.

Что важно понимать:

- GIL не мешает вводу-выводу. Пока поток ждёт сеть или диск, он отпускает GIL, и другие потоки работают. Поэтому `ThreadPoolExecutor` для сотни HTTP-запросов даёт реальное ускорение.

- GIL мешает CPU-bound задачам. Парсинг, сжатие, численные расчёты в потоках не ускоряются; нужны процессы (`multiprocessing`, `ProcessPoolExecutor`) или код на C/Rust, отпускающий GIL.
- Экспериментальная сборка без GIL (PEP 703) появилась в Python 3.13 и на момент написания не является вариантом по умолчанию: у неё своя цена по памяти и скорости однопоточного кода. Планировать архитектуру «GIL скоро исчезнет» пока рано.

venv: зачем нужно виртуальное окружение

Пакеты, установленные глобально, лежат в одном каталоге `site-packages` и доступны всем проектам сразу. Два проекта, которым нужны разные версии одной библиотеки, в такой схеме несовместимы: обновив зависимость для одного, вы сломали другой. Плюс глобальная установка на системный Python в Linux конфликтует с пакетами дистрибутива.

`venv` решает это копированием структуры: создаётся каталог с собственным интерпретатором (симлинком или копией), собственным `site-packages` и скриптами активации.

bash

```
python3.14 -m venv .venv
source .venv/bin/activate
python -V
which python
```

text

```
Python 3.14.8
/home/nextdoor/project/.venv/bin/python
```

Активация — это всего лишь правка `PATH`: каталог `.venv/bin` ставится первым, поэтому `python` и `pip` резолвятся в окружение. Никакой магии и никакой изоляции процессов: если запустить `/usr/bin/python3`, он увидит системные пакеты.

Отсюда правило, которое экономит часы отладки: **не полагайтесь на активацию в скриптах и CI**. Вызывайте интерпретатор по пути или через `uv run` / `poetry run`. Активация

удобна интерактивно, но в cron, systemd-юните и CI её нет, и `python` там окажется системным.

`--system-site-packages` при создании позволяет видеть глобальные пакеты — иногда нужно для доступа к собранным из дистрибутива библиотекам, но ломает изоляцию. `--upgrade-deps` обновляет `pip` и `setuptools` внутри окружения сразу после создания.

pip, uv, poetry

Инструмент	Что делает	Скорость установки	Lock-файл	Мнение
pip	ставит пакеты по требованиям	базовая	нет (нужен pip-tools)	минимализм
uv	ставит, разрешает, собирает, запускает	очень высокая	`uv.lock`	быстрый, один бинарь
poetry	управление проектом и зависимостями	средняя	`poetry.lock`	своя экосистема
pip-tools	компилирует requirements	средняя	`requirements.txt` с пинами	привычный формат

`pip` — стандарт де-факто и всегда доступен, но он не умеет разрешать зависимости в воспроизводимый набор: `pip install requests` поставит то, что актуально сегодня, и через месяц набор будет другим. Для фиксации используют `pip freeze > requirements.txt` или `pip-compile` из pip-tools.

`uv` написан на Rust и на момент написания заметно быстрее pip на холодной установке: он кэширует колеса, параллелит загрузку и разрешает зависимости целиком, а не пошагово. Он же умеет ставить нужную версию Python (`uv python install 3.14`), что снимает половину проблем с «а у меня 3.11».

`poetry` исторически дал первое удобное управление проектом: `pyproject.toml`, lock-файл, виртуальное окружение, публикация. Он остаётся разумным выбором для библиотек, но его разрешение

зависимостей медленнее, а собственная схема `pyproject.toml` (`[tool.poetry]`) отличается от стандарта PEP 621.

Практический выбор для нового проекта: `uv` для приложений и сервисов, `poetry` или `uv` для библиотек, `pip` — когда нужен только стандартный инструмент (например, в минимальном Docker-образе).

requirements.txt и pyproject.toml

`requirements.txt` — плоский список требований, по одному на строку, с необязательными маркерами окружения:

```text
httpx==0.28.1
uvicorn[standard]>=0.34,<0.40
pydantic==2.11.3 ; python_version >= "3.10"
```

`pyproject.toml` — стандартизованный файл конфигурации проекта (PEP 518/621). Он описывает метаданные, зависимости, точки входа и настройки инструментов в одном месте:

```toml
[project]
name = "tasks-api"
version = "0.3.0"
requires-python = ">=3.12"
dependencies = ["httpx>=0.28,<1.0", "pydantic>=2.11,<3.0"]

[project.optional-dependencies]
postgres = ["asyncpg>=0.30"]

[dependency-groups]
dev = ["pytest>=8.3", "ruff>=0.11", "mypy>=1.15"]
```

Разница принципиальная. `requirements.txt` описывает **что установить в это окружение**; `pyproject.toml` — **что представляет собой проект** и от чего он зависит. Из `pyproject.toml` генерируется lock-файл с точными версиями, а `requirements.txt` в старых проектах играет роль и того, и другого одновременно, что и порождает путаницу.

Два разных набора зависимостей стоит держать раздельно: `dependencies` — то, без чего пакет не работает (нужно пользователям), `dev` — тесты, линтеры, отладчики (не нужно).

Группы разработки описаны в PEP 735 (`[dependency-groups]`), extras — в `[project.optional-dependencies]`.

Структура проекта: src-layout

```text
tasks-api/
├── pyproject.toml
├── uv.lock
├── README.md
├── .python-version
├── src/
│   └── tasks_api/
│       ├── __init__.py
│       ├── __main__.py
│       ├── api.py
│       └── storage.py
└── tests/
    ├── conftest.py
    └── test_api.py
```

Пакет лежит в `src/`, а не рядом с `pyproject.toml`. Причина не в эстетике: если пакет в корне, то `import tasks_api` в тестах находит каталог по текущей директории, а не установленный пакет. Тесты начинают проходить на неработающей конфигурации упаковки, а ошибки `package_data`, `include` и точек входа обнаруживаются только после публикации.

С `src`-layout импорт возможен только после установки (`uv sync`, `pip install -e .`), и это сразу проверяет, что метаданные корректны. Установка в редактируемом режиме (`-e`, editable) создаёт ссылку на исходники: правки видны без переустановки, но сама установка нужна.

`__main__.py` даёт `python -m tasks_api`, а `[project.scripts]` — консольную команду:

```toml
[project.scripts]
tasks-api = "tasks_api.__main__:main"
```

После установки в `PATH` окружения появляется `tasks-api`, которая вызывает `main()`.

Переменные окружения и .env

Конфигурация, которая меняется между средами (адреса баз, ключи, уровни логов), не должна лежать в коде. Принцип из «12-factor»: конфигурация — в переменных окружения.

python

```
import os

DATABASE_URL = os.environ["DATABASE_URL"]          # упадёт, если нет — и это хорошо
LOG_LEVEL = os.environ.get("LOG_LEVEL", "info")    # дефолт для необязательного
DEBUG = os.environ.get("DEBUG", "0") == "1"
```

`os.environ[...]` без дефолта — сознательный выбор: отсутствие обязательной настройки должно ломать старт сразу, а не через час в проде. Обязательно приводите типы: всё, что пришло из окружения, — строка, и `DEBUG="false"` в `bool()` даст `True`.

Локально удобно держать `.env` рядом с проектом и загружать его библиотекой `python-dotenv` или встроенной поддержкой фреймворка. Файл **никогда не коммитится**: в репозиторий идёт `.env.example` с именами переменных и без значений. Иначе ключи утекают в историю git, а оттуда — в форки и сканеры.

Ещё одно правило: не подмешивайте `.env` в продакшн. Там переменные приходят от оркестратора или секрет-хранилища, и лишний файл создаёт неопределённость «какое значение победило».

Воспроизводимость

Воспроизводимость — свойство, при котором установка из репозитория через год даёт тот же набор пакетов. Достигается тремя вещами.

Пины версий. `httpx>=0.28` — диапазон, `httpx==0.28.1` — точная версия. В lock-файле пины есть всегда, включая транзитивные зависимости; в `pyproject.toml` — только диапазоны для прямых.

Фиксация версии Python. Файл `.python-version` (его читают uv, pyenv, asdf) и `requires-python` в `pyproject.toml`. Первое выбирает интерпретатор, второе запрещает установку на неподходящей версии.

Хеши и запрет неявных обновлений. `pip install --require-hashes -r requirements.txt` откажется ставить пакет, хеш которого не

совпал, — защита от подмены на зеркале. `uv sync --frozen` не будет обновлять lock при расхождении, а упадёт.

bash

```
uv sync --frozen    # поставить всё строго по uv.lock
uv add httpx        # добавить зависимость, обновить pyproject.toml и uv.lock
uv run pytest -q    # выполнить команду в окружении проекта
uv lock             # пересчитать lock
```

Эквивалент на чистом pip:

bash

```
python3.14 -m venv .venv
.venv/bin/pip install --upgrade pip
.venv/bin/pip install -e ".[dev]"
.venv/bin/pip freeze > requirements.lock.txt
```

Разница в поведении: `pip freeze` фиксирует то, что реально стоит, включая случайные транзитивные версии; `uv lock`/`poetry lock` разрешает зависимости детерминированно и хранит граф, поэтому обновление одной библиотеки не тянет за собой половину стека.

Типичные ошибки

1. **Глобальная установка вместо venv.** `pip install` без активированного окружения кладёт пакеты в системный `site-packages` (или падает с `externally-managed-environment` на современных дистрибутивах). Механизм: окружение не создано, а pip резолвится в системный.
2. **Активация вместо явного интерпретатора.** В cron или CI `source .venv/bin/activate` не выполняется, и запускается системный Python. Механизм: активация — это только `PATH`, а не свойство каталога.
3. `pip freeze` **как источник зависимостей.** В lock попадают пакеты, стоявшие в окружении случайно, включая dev-инструменты и локальные сборки, а транзитивные версии зафиксированы без графа. Механизм: freeze снимает снимок диска, а не описание проекта.
4. **Пакет в корне репозитория.** `import tasks_api` работает из текущей директории, тесты проходят, а после `pip install` пакет не находится или не содержит данных. Механизм: текущая

директория в `sys.path` маскирует ошибки упаковки.

5. **Секреты в `.env`, закоммиченном в git.** Утечка обнаруживается через месяцы, ключи приходится ротировать. Механизм: файл удобно коммитить «для примера», а `.gitignore` добавляют позже.
6. `DEBUG="false"` **вместо** `0`**.** Строка непустая, `bool("false")` — `True`, и прод работает в режиме отладки с трассировками в ответах. Механизм: переменные окружения всегда строки, приведение типов забыто.
7. **Незафиксированная версия Python.** Локально 3.14, в CI 3.11, синтаксис t-strings падает на этапе импорта. Механизм: версия интерпретатора не описана ни в одном файле проекта.

Чек-лист

- [ ] Версия Python зафиксирована: `.python-version`, `requires-python`, версия в CI-образе.
- [ ] Окружение создаётся одной командой из чистого клона, без ручных шагов.
- [ ] Lock-файл в репозитории; CI ставит по нему, а не по диапазонам.
- [ ] Зависимости разделены: runtime и dev не смешаны в одном списке.
- [ ] Проект использует src-layout, установка проверяется `pip install -e .` в чистом окружении.
- [ ] Секреты не в git; есть `.env.example` с именами переменных.
- [ ] Обязательные переменные читаются без дефолта и падают на старте.
- [ ] `python -m build` или сборка колеса проходит без предупреждений об отсутствующих файлах.

Упражнения

1. Создайте venv на Python 3.14, установите `httpx` и покажите через `sys.path` и `sys.prefix`, чем окружение отличается от системного интерпретатора.

2. Переведите проект с `requirements.txt` на `pyproject.toml` + lock: сравните набор транзитивных зависимостей до и после и объясните расхождения.
3. Разберите вывод `dis.dis` для функции с циклом и с генераторным выражением. Найдите, где создаётся объект-генератор.
4. Соберите колесо своего пакета, распакуйте его как zip и проверьте, что все нужные файлы внутри; найдите минимум одну ошибку упаковки, которую скрывал запуск из корня репозитория.
5. Настройте CI, который ставит зависимости по lock-файлу с `--frozen`, запускает тесты на двух версиях Python и падает при расхождении `pyproject.toml` и lock.

Ответы и разбор к этим упражнениям — в приложении D, часть 2.

Глава 17. Синтаксис и типы: коллекции, срезы, comprehension

Python — язык с динамической типизацией: переменная не имеет типа, тип имеет объект, на который она ссылается. Из этого вытекает почти всё, что раздражает новичков и экономит время опытным: изменяемость объектов, разделяемые ссылки, хешируемость, поведение `is` и `==`. В DevOps-коде Python чаще всего работает с чужими данными — JSON из API, выводом `kubectl`, конфигами, метриками, — и большинство инцидентов «скрипт сделал не то» сводится к непониманию устройства `list`, `dict` и `str`.

Глава даёт рабочую модель типов: числа и их точность, строки и байты, четыре базовые коллекции и их стоимость, распаковка, генераторы, аннотации. Все примеры проверены на CPython 3.14.9; вывод приведён дословно.

Числа: int без переполнения, float с потерей точности

`int` в Python — длинная арифметика: переполнения нет, но есть цена. `2 ** 100` даёт `1267650600228229401496703205376` — корректный результат, а не мусор. Обратная сторона — запрет на гигантские преобразования в строку: с Python 3.11 действует лимит 4300 цифр для `int` ↔ `str`, чтобы не подвешивать процесс.

python

```
import sys
print(sys.get_int_max_str_digits())      # 4300
str(10 ** 5000)
```

text

```
ValueError: Exceeds the limit (4300 digits) for integer string conversion;
use sys.set_int_max_str_digits() to increase the limit
```

Это ровно тот класс проблем, который в проде выглядит как «процесс завис на обработке ответа». Лимит поднимается через

`sys.set_int_max_str_digits()`, но чаще правильнее не считать факториалы в веб-обработчике.

`float` — IEEE-754 двойной точности, 53 бита мантиссы, отсюда `0.1 + 0.2 == 0.30000000000000004`. Для денег и квот, где важна десятичная точность, берут `decimal.Decimal` или `fractions.Fraction`: `Decimal("0.1") + Decimal("0.2")` даёт ровно `0.3`, а `Fraction(1, 3) + Fraction(1, 6)` — `1/2`.

Деление `/` всегда возвращает `float`, целочисленное — `//`, остаток — `%`, а `divmod` отдаёт оба сразу:

python

```
print(7 / 2, 7 // 2, 7 % 2, divmod(7, 2))
```

text

```
3.5 3 1 (3, 1)
```

Отдельный сюрприз — `round()`: это банковское округление (half to even). `round(2.5)` → `2`, `round(3.5)` → `4`. Для метрик неважно, для биллинга критично.

Строки: неизменяемые последовательности Unicode

`str` — последовательность кодовых точек Unicode, неизменяемая. Методы возвращают новые строки; `s.upper()` без присваивания результата — типичный no-op.

f-строки (formatted string literals) — основной инструмент форматирования: внутри `{}` допустимы выражения, спецификация формата после `:` и отладочный суффикс `=`:

python

```
name, p95 = "api-gateway", 128.4567
print(f"{name}: p95={p95:.1f} ms")
print(f"{name=} {p95=}")
print(f"{255:#06x} {0.834:>8.1%} {1234567:,}")
```

text

```
api-gateway: p95=128.5 ms
name='api-gateway' p95=128.4567
0x00ff    83.4% 1,234,567
```

Суффикс `=` печатает выражение вместе со значением — быстрый способ отладки. Формат после `:` читается как

`[[fill]align][sign][width][,][.precision][type]: >8.1%` — вправо в поле 8 символов, один знак после запятой, умножение на 100.

Срезы работают по схеме `s[start:stop:step]`, где `stop` не включается, а отрицательные индексы считаются с конца:

```python
s = "cluster"
print(s[0], s[-1], s[2:5], s[::-1], s[::2])
```

```text
c r ust retsulc cutr
```

Срез не бросает `IndexError` при выходе за границы — он усекается: `s[100:]` вернёт пустую строку, и ошибка всплывёт позже, там, где пустое значение не ожидали.

Байты (`bytes`) — отдельный тип. `len("кластер")` равно 7 (символы), а `len("кластер".encode())` — 14 (байты UTF-8, по два на кириллическую букву). Смешивать их нельзя: конкатенация `str` и `bytes` бросает `TypeError`. Чужой поток декодируют с явной политикой: `raw.decode("utf-8", errors="replace")` не упадёт на битом байте, но подменит символ.

Четыре коллекции и их стоимость

Коллекция	Упорядоченность	Изменяемость	Поиск	Хешируемость
`list`	да, по индексу	да	O(n)	нет
`tuple`	да, по индексу	нет	O(n)	да, если элементы хешируемы
`dict`	да, порядок вставки	да	O(1) в среднем	нет
`set`	нет	да	O(1) в среднем	нет

`dict` с Python 3.7 гарантирует порядок вставки — это часть спецификации языка, а не деталь реализации. `set` порядок не гарантирует: `list({"b": 1, "a": 2})` даёт `['b', 'a']`, а порядок элементов множества может меняться между запусками.

Разница в стоимости принципиальна. Замер на 100 000 элементов:

python

```
import timeit
setup = "d = {i: i for i in range(100_000)}; l = list(range(100_000))"
print(timeit.timeit("d[99_999]", setup, number=1_000_000))
print(timeit.timeit("99_999 in l", setup, number=1_000))
```

text

```
0.0379
1.0921
```

Миллион обращений к словарю — 38 мс, около 40 нс на операцию. Тысяча проверок `in` по списку — 1,09 с, примерно 1,1 мс на проверку. Отношение — четыре порядка. Вывод: если в цикле есть `if item in collection` и `collection` — список длиннее нескольких десятков элементов, это `set`.

Проверка `x in list` выполняет `==` поэлементно, `x in set` — хеширование и поиск в таблице, поэтому `set` требует хешируемых элементов: `{[1, 2]}` бросает `TypeError: unhashable type: 'list'`. Кортеж из хешируемых элементов хешируем: `(1, 2) in {(1, 2), (3, 4)}` → `True`.

python

```
a = {"web-1", "web-2", "web-3"}
b = {"web-2", "web-3", "web-4"}
print(sorted(a & b), sorted(a - b), sorted(a | b))
```

text

```
['web-2', 'web-3'] ['web-1'] ['web-1', 'web-2', 'web-3', 'web-4']
```

Сортировка в примере не случайна: хеши строк рандомизируются при старте процесса (`PYTHONHASHSEED`), поэтому порядок обхода множества меняется от запуска к запуску и делает `diff` двух выгрузок бессмысленным.

Изменяемость, алиасы и ловушка аргумента по умолчанию

Присваивание в Python не копирует объект, а связывает имя со ссылкой. `b = a` для списка — два имени на один объект; изменение через любое имя видно через оба. Копия — это `a[:]`, `a.copy()`, `list(a)`, а для вложенных структур — `copy.deepcopy(a)`.

python

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)          # [1, 2, 3, 4]
c = a[:]
c.append(5)
print(a, c)       # [1, 2, 3, 4] [1, 2, 3, 4, 5]
```

Та же модель объясняет классическую ловушку: значение по умолчанию вычисляется **один раз**, при определении функции, и сохраняется в объекте функции.

python

```
def add_tag(tag, tags=[]):
    tags.append(tag)
    return tags

print(add_tag("a"), add_tag("b"))
```

text

```
['a', 'b'] ['a', 'b']
```

Оба вызова вернули один и тот же список: первый дописал `"a"`, второй — `"b"`. В продакшене это выглядит как «накопление» меток, заголовков, соединений между запросами — утечка состояния через общий объект. Идиома — `None` как сигнал «не передали»:

python

```
def add_tag_ok(tag, tags=None):
    tags = [] if tags is None else tags
    tags.append(tag)
    return tags

print(add_tag_ok("a"), add_tag_ok("b"))
```

text

```
['a'] ['b']
```

Правило шире, чем список: любой изменяемый объект в значении по умолчанию (`list`, `dict`, `set`, экземпляр класса) разделяется между вызовами. Неизменяемые (`None`, числа, строки, кортежи из неизменяемых) безопасны.

Распаковка и слияние

Распаковка заменяет индексацию и одновременно служит валидацией формы данных: несовпадение количества элементов даёт `ValueError` сразу, а не через пять строк.

python

```
host, port = ("10.0.0.5", 5432)
first, *rest = [1, 2, 3, 4]
print(host, port, first, rest)
```

text

```
10.0.0.5 5432 1 [2, 3, 4]
```

Звёздочка собирает «остаток» в список, и такая распаковка допустима один раз в выражении. Для словарей работает `**`, а с Python 3.9 есть оператор `|`:

python

```
cfg = {"host": "db", "port": 5432}
print({**cfg, "port": 6432})        # {'host': 'db', 'port': 6432}
print({**{"timeout": 3}, **cfg})    # {'timeout': 3, 'host': 'db', 'port': 5432}
print(cfg | {"port": 6432})
```

Порядок важен: правый словарь перекрывает левый. Это стандартный способ собрать конфиг «значения по умолчанию + переопределения», но он поверхностный — вложенные словари заменяются целиком, а не сливаются рекурсивно.

Comprehension и генераторы

List comprehension — выражение, а не цикл: оно строит список целиком и сразу. Generator expression строит ленивый итератор, который вычисляет элементы по требованию.

python

```
import tracemalloc

tracemalloc.start()
data = [i * i for i in range(1_000_000)]
print(f"{tracemalloc.get_traced_memory()[1] / 1024 / 1024:.1f} MiB")
del data
tracemalloc.reset_peak()
tracemalloc.stop()

tracemalloc.start()
gen = (i * i for i in range(1_000_000))
print(f"{tracemalloc.get_traced_memory()[1] / 1024:.0f} KiB")
tracemalloc.stop()
```

text

```
38.6 MiB
0 KiB
```

Список занимает 38,6 МиБ, генератор — 0 КиБ по счётчику (меньше блока учёта): он хранит только текущий элемент и позицию. Если данные не нужны все сразу (сумма, максимум, потоковая запись), генератор предпочтителен; если нужен произвольный доступ или повторный проход — список.

Тело генераторной функции не выполняется до первого `next()`; каждый `yield` приостанавливает выполнение и сохраняет локальное состояние. Отсюда класс ошибок: генератор одноразовый — второй проход по исчерпанному генератору не даст ничего. Подробно `yield` разобран в главе 18.

Comprehension бывает не только для списков: `{k: v for ...}` даёт словарь, `{x for ...}` — множество. Условие `if` и вложенные циклы допустимы, но два условия и более делают выражение нечитаемым — тогда нужен обычный цикл.

`is` против `==`

`==` сравнивает значения через `__eq__`, `is` — идентичность объектов (один и тот же адрес). Для `None` правильно `is None`: это одна константа на процесс, и `is` нельзя переопределить. Для чисел и строк `is` даёт неопределённый результат, зависящий от кеширования:

python

```
a = int("257"); b = int("257")
print(a == b, a is b)
print(256 is 256)
```

text

```
True False
True
```

Малые целые от −5 до 256 кешируются интерпретатором, поэтому `256 is 256` истинно, а результат для 257 зависит от того, создан ли объект один раз (константа в коде) или дважды (вызов `int`). Python 3.14 к тому же выдаёт `SyntaxWarning` на `is` с литералом. Правило: `is` — только для `None`, `True`, `False` и явных сигнальных объектов.

Хешируемость и собственные ключи

Ключ словаря или элемент множества обязан иметь стабильный `__hash__`, согласованный с `__eq__`: если `a == b`, то `hash(a) == hash(b)`. По умолчанию класс наследует `__eq__` (сравнение по идентичности) и `__hash__` (по адресу) от `object`, что согласовано. Как только вы определяете `__eq__`, Python обнуляет `__hash__`, и экземпляр становится нехешируемым — защита от нарушения контракта.

python

```
class Point:
    __slots__ = ("x", "y")

    def __init__(self, x, y):
        self.x, self.y = x, y

    def __eq__(self, other):
        return (self.x, self.y) == (other.x, other.y)

    def __hash__(self):
        return hash((self.x, self.y))

print(Point(1, 2) in {Point(1, 2)})
```

text

```
True
```

Без `__hash__` последняя строка бросила бы `TypeError`. И обратное правило: объект-ключ не должен меняться после вставки — иначе хеш «уплывёт» и запись станет недостижимой. `__slots__` здесь про память: у экземпляра нет `__dict__`, экономия на миллионах объектов заметна, но появляются ограничения.

Аннотации типов

Аннотации — синтаксис, а не проверка времени выполнения: интерпретатор их сохраняет и не проверяет. Проверяют внешние инструменты — `mypy`, `pyright`, `pyre`. В DevOps-коде аннотации документируют контракты обработчиков и ловят несовпадение `dict`/`list` до запуска.

python

```
from typing import Literal, Optional

def retry(times: int, backoff: Literal["linear", "exp"] = "exp") -> list[float]:
    return [0.0] * times

def find_node(name: str) -> Optional[str]:
```

```
    return name if name.startswith("node-") else None

print(Optional[int], list[int])
```

```
text

int | None list[int]
```

С Python 3.9 встроенные generic-типы (`list[int]`, `dict[str, int]`, `tuple[int, ...]`) пишутся без `typing.List`. С Python 3.10 `Optional[X]` и `Union[X, Y]` заменяются на `X | None` и `X | Y`, и именно эту форму показывает `repr`. `Optional[T]` означает «T или None», а не «необязательный аргумент»: необязательность задаётся значением по умолчанию, а не типом.

`Literal` фиксирует допустимый набор значений и превращает опечатку в ошибку типизации, а не в `KeyError` в проде. Для словарей с известной схемой используют `TypedDict` или `dataclass`; для «словаря произвольных ключей» — `dict[str, object]`, и это честно говорит, что дальше нужна валидация на границе системы.

Отдельная деталь: `from __future__ import annotations` больше не нужен — в 3.14 отложенное вычисление аннотаций работает по умолчанию (PEP 649/749), а `typing.get_type_hints()` разрешает строки в реальные объекты, когда это требуется.

Типичные ошибки

1. **Изменяемое значение по умолчанию.** `def f(x, acc=[])` создаёт список один раз при определении функции; все вызовы делят его. Механизм: значение по умолчанию вычисляется при выполнении `def` и живёт в `f.__defaults__`. Лечение — `None` плюс создание внутри.
2. `is` **вместо** `==` **для значений.** Кеш малых целых и интернирование строк делают результат `is` зависящим от контекста; сравнение «работает» на тестах и ломается на данных из сети. `is` — только для синглтонов.
3. **Список как ключ словаря.** `{[1, 2]: "x"}` бросает `TypeError`: список изменяем, его хеш не может быть стабильным. Решение — кортеж или `frozenset`.

4. `a = b` **вместо копии.** Для списков, словарей и множеств это алиас; правка «копии» меняет оригинал, потому что присваивание связывает имена с объектом. Копия — `copy()`, срез или `deepcopy`.
5. **Генератор, прочитанный дважды.** После исчерпания он не перезапускается и молча даёт пустоту; сумма по нему оказывается нулём. Механизм — итератор хранит позицию и не имеет операции сброса.
6. `str` **и** `bytes` **вперемешку.** Чтение файла в бинарном режиме и передача результата в функцию, ожидающую `str`, даёт `TypeError` или «кракозябры». Механизм — разные типы и разные единицы длины.

Чек-лист

- [ ] Для проверок вхождения в коллекцию больше ~50 элементов используется `set` или `dict`, а не `list`.
- [ ] Ни одна функция не имеет изменяемого значения по умолчанию; проверено `grep` по `def .*=\[\]` и `={}`.
- [ ] Все сравнения с `None` — через `is None` / `is not None`; `is` не применяется к числам и строкам.
- [ ] Копирование конфигов и вложенных структур — явное (`copy`, `deepcopy`), а не присваивание.
- [ ] Генераторы не переиспользуются: если нужен второй проход, материализован список.
- [ ] Кодировка при `open()` и `decode()` указана явно, политика `errors` выбрана осознанно.
- [ ] Публичные функции имеют аннотации, и `mypy --strict` (или `pyright`) проходит без новых ошибок.
- [ ] `Literal`/`TypedDict` использованы там, где набор значений или ключей фиксирован, вместо «голого» `str` и `dict`.

Упражнения

1. **Разминка.** Дан список словарей с метриками (`{"service": ..., "p95": ...}`). Верните словарь «сервис → максимальный p95»

через comprehension и объясните, зачем здесь промежуточный `set`.

2. **Ловушка.** Напишите функцию `batch(items, size=10, acc=[])`, которая группирует элементы, и воспроизведите баг с повторными вызовами. Исправьте, не меняя сигнатуру вызова у существующих клиентов.
3. **Типы.** Опишите аннотациями функцию, принимающую `dict[str, str | int]` и режим `Literal["fast", "safe"]`, возвращающую кортеж `(str, int)`. Добейтесь, чтобы `mypy --strict` ругался на вызов с `mode="turbo"`.
4. **Своя коллекция.** Реализуйте неизменяемый класс `Resource(kind, name)` с `__eq__`, `__hash__` и `__slots__`, положите 100 000 экземпляров в `set` и сравните потребление памяти с вариантом без `__slots__`.
5. **Стоимость.** Сравните время `x in list` и `x in set` для 10, 1 000 и 1 000 000 элементов, постройте таблицу и объясните, при каком размере список перестаёт быть приемлемым.

Ответы и разбор к этим упражнениям — в приложении D, часть 2.

Глава 18. Функции, замыкания, декораторы

Функция в Python — не просто блок кода, а объект: у неё есть атрибуты, её можно передать, вернуть, обернуть и сохранить в словаре. Из этого следуют три инструмента, на которых держится почти весь инфраструктурный код: декораторы (повторные попытки, замер времени, проверка прав, кэш), замыкания (фабрики обработчиков и конфигураторов) и генераторы (потоковая обработка логов и ответов API без загрузки в память целиком).

Глава разбирает сигнатуры и правила передачи аргументов, области видимости и `nonlocal`, устройство замыканий и их классическую ловушку с поздним связыванием, декораторы с параметрами и без, `functools.wraps` и `lru_cache`, а также итераторы и `yield`. Финал — рабочий декоратор `retry` с экспоненциальной задержкой, тот самый, что в главе 21 будет расширен до HTTP-клиента. Примеры проверены на CPython 3.14.9.

Сигнатуры: позиционные, keyword-only и «всё остальное»

Полная форма сигнатуры выглядит так: `def f(pos_only, /, normal, *args, kw_only, **kwargs)`. Слэш отделяет параметры, которые можно передать только позиционно, звёздочка — начало секции, где аргументы передаются только по имени, `*args` собирает лишние позиционные в кортеж, `**kwargs` — лишние именованные в словарь.

python

```
def deploy(env, /, service, replicas=2, *, dry_run=False, **labels):
    print(env, service, replicas, dry_run, labels)

deploy("prod", "api", 3, dry_run=True, team="core", tier="web")
deploy(env="prod", service="api")
```

text

```
prod api 3 True {'team': 'core', 'tier': 'web'}
TypeError: deploy() missing 1 required positional argument: 'env'
```

Второй вызов падает, потому что `env` объявлен позиционным. Сообщение выглядит странно — «не хватает `env`», хотя он передан, — но смысл точный: именованный аргумент `env` не может заполнить позиционный параметр, а `**labels` его не ловит. Позиционные параметры нужны, когда имя — деталь реализации и вы не хотите, чтобы его использовали как контракт (частая практика в стандартной библиотеке).

Значения по умолчанию вычисляются один раз, при выполнении `def`, — отсюда ловушка изменяемого аргумента из главы 17. Здесь важно следствие для API: значение по умолчанию должно быть неизменяемым или `None`. Порядок параметров тоже регламентирован: параметры без значения по умолчанию идут до параметров со значением, иначе получается двусмысленность при вызове.

Вызов, в свою очередь, поддерживает распаковку: `f(*args, **kwargs)` передаёт содержимое последовательности и словаря. Это основа универсальных обёрток — декоратор не знает сигнатуры оборачиваемой функции и просто пробрасывает всё насквозь.

Области видимости и `nonlocal`

Имена разрешаются по правилу LEGB: локальная область (Local), затем объемлющие функции (Enclosing), затем модуль (Global), затем встроенные (Builtins). Чтение работает наружу автоматически, запись — нет: присваивание внутри функции создаёт локальное имя, если оно не объявлено `global` или `nonlocal`.

python

```
counter = 0

def bump():
    global counter
    counter += 1

bump(); bump()
print(counter)
```

text

```
2
```

Без `global` строка `counter += 1` бросила бы `UnboundLocalError` (чтение локальной переменной до присваивания), потому что присваивание делает имя локальным на всю функцию. Это одна из самых частых ошибок при правке чужого кода: достаточно добавить присваивание, и функция перестаёт видеть глобальную переменную.

`nonlocal` делает то же для ближайшей объемлющей функции:

python

```
def make_counter():
    n = 0

    def inc(step=1):
        nonlocal n
        n += step
        return n

    return inc

c = make_counter()
print(c(), c(5), c())
```

text

```
1 6 7
```

Глобальное состояние в продакшене — источник гонок и «плавающих» тестов: два запроса делят один счётчик. Замыкание предпочтительнее, потому что состояние создаётся на каждый экземпляр функции и не видно снаружи.

Замыкания и позднее связывание

Замыкание (closure) — функция, которая помнит переменные из области, где была создана, даже после выхода из неё. В примере выше `inc` держит ячейку (cell) с `n`: `make_counter()` завершилась, а её локальная переменная жива, пока жива `inc`.

Классическая ловушка — замыкание в цикле захватывает переменную, а не значение:

python

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])
funcs = [lambda i=i: i for i in range(3)]
print([f() for f in funcs])
```

text

```
[2, 2, 2]
[0, 1, 2]
```

В первом случае все три лямбды смотрят в одну ячейку `i`, которая к моменту вызова равна 2. Во втором значение фиксируется значением по умолчанию — оно вычисляется при создании каждой лямбды. Третий способ — `functools.partial` — читается лучше, чем `lambda i=i:`, когда аргументов несколько.

Практический смысл: в цикле по обработчикам, хендлерам или таймерам нельзя регистрировать замыкание, которое читает переменную цикла, если она изменится до вызова. Это регулярно всплывает в коде, где цикл навешивает колбэки на события.

Декораторы

Декоратор — функция, которая принимает функцию и возвращает функцию. Синтаксис `@decorator` — просто сахар: `def f(): ...` плюс `f = decorator(f)`.

python

```
import functools

def logged(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"-> {func.__name__}{args}{kwargs}")
        result = func(*args, **kwargs)
        print(f"<- {result!r}")
        return result
    return wrapper

@logged
def add(a, b):
    return a + b

add(2, 3)
print(add.__name__, add.__doc__)
```

text

```
-> add(2, 3){}
<- 5
add None
```

`functools.wraps` копирует на обёртку `__name__`, `__doc__`, `__module__`, `__qualname__`, `__dict__` и `__wrapped__`. Без него `add.__name__` стало бы `wrapper`, сломались бы логи, трассировки, автодокументация и `inspect.signature`. Правило простое: любой декоратор оборачивает через `functools.wraps`, исключений нет.

Декоратор с параметрами — это функция, возвращающая декоратор: три уровня вложенности. Первый уровень принимает настройки, второй — функцию, третий — аргументы вызова.

python

```
import functools, time

def retry(attempts=3, base=0.1, factor=2.0, exceptions=(Exception,)):
    def decorate(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            delay = base
            for attempt in range(1, attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as exc:
                    if attempt == attempts:
                        raise
                    print(f"  attempt {attempt}: {exc}; sleep {delay:.2f}s")
                    time.sleep(delay)
                    delay *= factor
        return wrapper
    return decorate
```

Разбор по частям. `delay` начинается с `base` и умножается на `factor` после каждой неудачи: 0,05 → 0,1 → 0,2 с. На последней попытке исключение пробрасывается наружу — декоратор не глотает ошибку, а лишь даёт шанс. `exceptions` ограничивает, что вообще считается временной ошибкой: ретраить `ValueError` из-за кривого конфига бессмысленно, ретраить `ConnectionError` — обязательно. `functools.wraps` сохраняет имя, поэтому в логах и метриках видно `fetch_config`, а не `wrapper`.

Проверка на функции, которая падает дважды:

python

```
calls = {"n": 0}

@retry(attempts=4, base=0.05)
def fetch_config(url):
    calls["n"] += 1
    if calls["n"] < 3:
        raise ConnectionError("connection reset by peer")
    return {"url": url, "ok": True}

print(fetch_config("https://cfg.internal/app"), fetch_config.__name__)
```

text

```
  attempt 1: connection reset by peer; sleep 0.05s
  attempt 2: connection reset by peer; sleep 0.10s
{'url': 'https://cfg.internal/app', 'ok': True} fetch_config
```

Три вызова, суммарная пауза 0,15 с, результат получен. В реальном коде сюда добавляют джиттер (случайную добавку к задержке), чтобы после падения общего ресурса клиенты не били по нему синхронно, и общий дедлайн: если бюджет запроса 10 с, четыре попытки с растущей задержкой могут его превысить. Ещё одна тонкость: ретрай допустим только для идемпотентных операций — повтор `POST`, который уже создал ресурс, создаст второй.

Декоратор может быть и классом с `__call__`, что удобно, когда нужно состояние (счётчики, конфиг). Минус — теряется прозрачность для `inspect`, поэтому классы-декораторы чаще применяют к методам с явным `__get__`, а это уже отдельный уровень сложности.

`lru_cache` и мемоизация

`functools.lru_cache` кэширует результаты по аргументам: ключ — кортеж аргументов, значение — результат.

python

```
@functools.lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

t0 = time.perf_counter()
print(fib(80), f"{time.perf_counter() - t0:.4f}s")
print(fib.cache_info())
```

text

```
23416728348467685 0.0000s
CacheInfo(hits=78, misses=81, maxsize=None, currsize=81)
```

Без кэша `fib(80)` потребовал бы порядка 2^80 вызовов — астрономическое время; с кэшем это 81 вычисление и 78 попаданий, доли миллисекунды. `maxsize=None` означает неограниченный кэш; `maxsize=128` ограничивает и вытесняет давно использованные записи (LRU — least recently used).

Ограничения существенные. Аргументы должны быть хешируемыми: список или словарь в аргументах даст `TypeError: unhashable type`. Кэш живёт в памяти процесса и не разделяется между воркерами, поэтому после деплоя каждый процесс прогревается заново. Кэш не знает об изменении внешнего

состояния: если функция читает файл или ходит в API, закэшированный ответ устареет. Для «вычислить один раз» есть `functools.cache` — тот же `lru_cache(maxsize=None)` с более коротким именем. И помните: декоратор на методе экземпляра держит ссылку на `self`, что мешает сборке мусора, — для методов нужен `lru_cache` на уровне класса или кэш по значимым полям.

Генераторы и итераторы

Итератор — объект с `__next__`, который выдаёт элементы по одному и по исчерпании бросает `StopIteration`. Генератор — функция с `yield`, компилируемая в итератор: вызов не выполняет тело, а возвращает объект, хранящий кадр со всеми локальными переменными.

python

```
def read_lines(path):
    with open(path, encoding="utf-8") as fh:
        for line in fh:
            yield line.rstrip("\n")

def parse(lines):
    for line in lines:
        key, _, value = line.partition("=")
        if value:
            yield key.strip(), value.strip()

print(dict(parse(read_lines("app.env"))))
```

text

```
{'host': 'db', 'port': '5432'}
```

Файл `app.env` содержит три строки: `host=db`, `broken`, `port=5432`. Строка без `=` отброшена условием `if value`. Конвейер обрабатывает файл построчно: в памяти одновременно одна строка, а не весь файл, — на гигабайтном логе это разница между работой и `MemoryError`. Генераторы компонуются: выход одного становится входом другого, а вычисления начинаются только при первом `next()`.

`yield from` делегирует итерацию вложенному итератору и пробрасывает `send`/`throw`:

python

```
def chain(*iterables):
    for it in iterables:
        yield from it
```

```
print(list(chain([1, 2], (3,), "ab")))
```

```
text
```

```
[1, 2, 3, 'a', 'b']
```

Генератор одноразовый: после исчерпания повторный проход даёт пустоту. Если данные нужны дважды, материализуйте их (`list(...)`) или создайте генератор заново. Генератор также умеет `close()` — при этом в него «вбрасывается» `GeneratorExit`, и блоки `finally` выполняются, что позволяет освобождать ресурсы.

Типичные ошибки

1. **Изменяемое значение по умолчанию.** `def f(x, acc=[])` — один список на все вызовы, потому что значение вычисляется при `def`. Лечение — `None` и создание внутри.
2. **Забытый** `functools.wraps`**.** Обёртка получает имя `wrapper`; логи, метрики и трассировки указывают не на ту функцию, а `inspect.signature` показывает `(*args, **kwargs)`. Механизм — декоратор буквально подменяет объект функции.
3. **Замыкание в цикле.** `[lambda: i for i in range(3)]` даёт три функции, читающие одну ячейку, — все вернут последнее значение. Лечение — параметр по умолчанию или `partial`.
4. `global` **вместо возврата значения.** Функция, меняющая модульскую переменную, не тестируется изолированно и ломается при параллельных вызовах. Механизм — состояние живёт дольше вызова и делится между потоками.
5. **Ретрай неидемпотентной операции.** Повтор `POST /orders` после таймаута создаёт второй заказ. Механизм — таймаут не означает, что сервер не выполнил операцию.
6. `lru_cache` **на функции с побочными эффектами.** Кэшируется результат чтения файла или API, и данные «замерзают» до перезапуска процесса. Механизм — ключ кэша не включает внешнее состояние.
7. **Исчерпанный генератор.** Второй `for` по тому же объекту молча не выполняется ни разу; сумма равна нулю. Механизм — итератор хранит позицию и не перезапускается.

Чек-лист

- [ ] У каждого декоратора в проекте есть `@functools.wraps(func)`.
- [ ] Ретраи ограничены числом попыток, списком исключений и общим дедлайном; операции идемпотентны.
- [ ] Задержка ретрая растёт экспоненциально и содержит джиттер; максимальная пауза ограничена.
- [ ] Ни одна функция не пишет в глобальные переменные без явной необходимости; состояние — в замыкании или объекте.
- [ ] Замыкания в циклах фиксируют значение (`i=i`, `partial`), а не переменную.
- [ ] `lru_cache` применён только к чистым функциям с хешируемыми аргументами; размер кэша задан.
- [ ] Большие файлы и ответы обрабатываются генераторами, а не `read()` целиком.
- [ ] Публичные функции аннотированы, включая `*args: int` и `**kwargs: str`.

Упражнения

1. **Разминка.** Напишите `def tag(name, /, *, prefix="app", **attrs)`, которая собирает строку `prefix:name` и список атрибутов. Добейтесь, чтобы вызов `tag(name="x")` падал с `TypeError`, и объясните сообщение.
2. **Замыкание.** Реализуйте `make_limiter(rate)` — фабрику функций, каждая из которых помнит своё состояние и возвращает `True` не чаще `rate` раз в секунду (используйте `time.monotonic`).
3. **Декоратор.** Добавьте к `retry` параметр `jitter` (случайная добавка к задержке) и общий дедлайн `deadline`. Проверьте, что при исчерпании дедлайна возвращается последняя ошибка, а не бесконечное ожидание.
4. **Кэш.** Оберните функцию, читающую конфиг из файла, в `lru_cache` и покажите, как получить устаревшие данные. Предложите схему инвалидации по `mtime`.

5. **Генераторы.** Напишите конвейер «чтение лога → фильтр по уровню → агрегация по коду ответа», который обрабатывает файл 1 ГБ с постоянным потреблением памяти. Измерьте пик через `tracemalloc`.

Ответы и разбор к этим упражнениям — в приложении D, часть 2.

Глава 19. ООП, dataclasses, протоколы, MRO

Python-код в инфраструктуре редко начинается с классов, но почти всегда ими заканчивается: конфиг превращается в объект с валидацией, HTTP-клиент — в класс с пулом соединений, набор политик ретраев — в иерархию. Проблема в том, что ООП в Python устроено иначе, чем в Java или C++: наследование множественное, интерфейсы не объявляются, а проверяются структурно, а `dataclass` убирает большую часть шаблонного кода.

Глава разбирает модель классов (атрибуты класса против атрибутов экземпляра), порядок разрешения методов и кооперативный `super()`, свойства, `dataclass` с `frozen` и `slots`, структурную типизацию через `Protocol`, абстрактные базовые классы и границу, за которой класс становится лишним. Отдельная тема — композиция против наследования: в DevOps-коде она решает больше проблем, чем любая иерархия. Примеры проверены на CPython 3.14.9.

Классы и атрибуты: класс против экземпляра

Класс — это пространство имён и фабрика экземпляров. Атрибут, объявленный в теле класса, живёт в словаре класса и виден всем экземплярам; присваивание `self.x = ...` создаёт запись в словаре экземпляра. Разница принципиальна, когда значение изменяемо:

```python
class Queue:
    items = []

q1, q2 = Queue(), Queue()
q1.items.append(1)
print(q1.items, q2.items)
```

```text
[1] [1]
```

`q1.items.append(1)` не создаёт атрибут экземпляра — атрибут читается из класса, а список изменяется на месте, поэтому «вторая очередь» видит те же данные. Это та же ловушка, что и

изменяемое значение по умолчанию, только на уровне класса. Лечение — инициализация в `__init__`: `self.items = []`.

Два подводных камня рядом. Первый: `__init__` — не конструктор, а инициализатор; объект уже создан `__new__`, и возвращать из `__init__` ничего нельзя. Второй: имена с двумя подчёркиваниями (`self.__secret`) проходят через name mangling и превращаются в `_ClassName__secret` — это защита от случайного перекрытия в подклассах, а не приватность. Одно подчёркивание (`self._secret`) — соглашение «внутреннее, не трогать снаружи», которое соблюдают инструменты и люди, но не интерпретатор.

Наследование, MRO и `super()`

При множественном наследовании Python строит линейный порядок разрешения методов (MRO, Method Resolution Order) алгоритмом C3: класс, затем его базовые классы слева направо, с сохранением порядка появления базовых классов. Порядок виден в атрибуте `__mro__`.

python

```
class Base:
    def handle(self) -> list[str]:
        return ["Base"]

class WithRetry(Base):
    def handle(self) -> list[str]:
        return ["retry"] + super().handle()

class WithCache(Base):
    def handle(self) -> list[str]:
        return ["cache"] + super().handle()

class Gateway(WithRetry, WithCache):
    def handle(self) -> list[str]:
        return ["gw"] + super().handle()

print([c.__name__ for c in Gateway.__mro__])
print(Gateway().handle())
```

text

```
['Gateway', 'WithRetry', 'WithCache', 'Base', 'object']
['gw', 'retry', 'cache', 'Base']
```

Ромб разрешён: `Base` встречается в цепочке один раз, хотя наследуется дважды. `super()` в `WithRetry` — это не «родитель», а «следующий класс в MRO экземпляра»: для `Gateway()` он ведёт в

`WithCache`, а не в `Base`. Именно поэтому такой код называют кооперативным: каждый слой делает свою работу и передаёт управление дальше, и все слои выполняются ровно по одному разу. Если бы `WithRetry` вызывал `Base.handle(self)` напрямую, `WithCache` был бы пропущен, и порядок выполнения зависел бы от того, какой класс вызвал метод.

Практическое следствие: любой переопределённый метод в иерархии должен либо вызывать `super()`, либо явно документировать, что он терминальный. Иначе появляются «дыры», в которых не выполняется инициализация или не снимается блокировка.

`@property`: вычисляемый атрибут с валидацией

Свойство (property) позволяет обращаться к методу как к атрибуту — без скобок. Типичное применение — валидация при присваивании и вычисляемые значения.

python

```
class RetryPolicy:
    def __init__(self, attempts: int) -> None:
        self.attempts = attempts

    @property
    def attempts(self) -> int:
        return self._attempts

    @attempts.setter
    def attempts(self, value: int) -> None:
        if not 1 <= value <= 10:
            raise ValueError(f"attempts must be 1..10, got {value}")
        self._attempts = value

p = RetryPolicy(3)
print(p.attempts)
try:
    RetryPolicy(0)
except ValueError as exc:
    print("ValueError:", exc)
print(p.__dict__)
```

text

```
3
ValueError: attempts must be 1..10, got 0
{'_attempts': 3}
```

Валидация срабатывает и в `__init__`, потому что там тоже идёт присваивание `self.attempts`. Это важный приём: правило проверки

живёт в одном месте и не дублируется. Обратите внимание на `_attempts` в `__dict__` — декоратор `@property` создал в классе объект-дескриптор, а данные лежат в отдельном поле. Если назвать поле так же (`self.attempts = ...` внутри сеттера), получится бесконечная рекурсия.

Свойство без сеттера — способ сделать атрибут только для чтения. Минус `property` в том, что `dataclass` не умеет с ним дружить напрямую: поле, объявленное как `property`, не попадает в `__init__`. Поэтому в новых проектах валидацию чаще выносят в `__post_init__` или в отдельный конструктор вроде `pydantic`.

`dataclass`: данные без шаблонного кода

`@dataclass` генерирует `__init__`, `__repr__` и `__eq__` по аннотациям полей. Это лучший выбор для значений конфигурации, результатов и DTO.

```python
from dataclasses import dataclass, field

@dataclass(frozen=True, slots=True)
class Endpoint:
    host: str
    port: int = 8080
    tags: tuple[str, ...] = ()

e = Endpoint("api.internal")
print(e)
print(e == Endpoint("api.internal"), hash(e) == hash(Endpoint("api.internal")))
```

```text
Endpoint(host='api.internal', port=8080, tags=())
True True
```

`frozen=True` запрещает присваивание полей (бросает `FrozenInstanceError`), делает объект неизменяемым и заодно генерирует `__hash__` — такой объект можно класть в `set` и использовать ключом словаря. `slots=True` создаёт `__slots__` и убирает `__dict__`: на сотнях тысяч экземпляров экономия памяти заметна, а опечатка `e.hots = "x"` сразу даёт `AttributeError` вместо тихого создания нового атрибута.

Изменяемые значения по умолчанию `dataclass` запрещает осознанно: `headers: dict = {}` — ошибка при импорте. Правильно —

`field(default_factory=dict)`, тогда каждая копия получает свой словарь:

python

```
@dataclass
class Route:
    path: str
    headers: dict[str, str] = field(default_factory=dict)

r1, r2 = Route("/a"), Route("/b")
r1.headers["x"] = "1"
print(r1.headers, r2.headers)
```

text

```
{'x': '1'} {}
```

Полезные переключатели: `order=True` добавляет сравнения (`<`, `>`), `kw_only=True` делает все поля именованными (удобно, когда полей много и легко перепутать порядок), `eq=False` отключает генерацию `__eq__`, если нужна идентичность. `__post_init__` вызывается после `__init__` — там валидируют и вычисляют производные поля.

`Protocol`: структурная типизация

`Protocol` описывает не «от кого наследоваться», а «какие методы должны быть». Класс, у которого есть нужные методы, удовлетворяет протоколу без единого упоминания о нём — это структурная типизация, «утиная типизация» с проверкой статическим анализатором.

python

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Store(Protocol):
    def get(self, key: str) -> str | None: ...
    def put(self, key: str, value: str) -> None: ...

class MemoryStore:
    def __init__(self) -> None:
        self._d: dict[str, str] = {}
    def get(self, key: str) -> str | None:
        return self._d.get(key)
    def put(self, key: str, value: str) -> None:
        self._d[key] = value

print(isinstance(MemoryStore(), Store), issubclass(MemoryStore, Store))
```

text

```
True True
```

Это главное преимущество перед ABC: `MemoryStore` не обязан импортировать `Store`, поэтому протокол можно объявить в вашем модуле для чужого класса — например, для `httpx.Client` или `redis.Redis`. Тесты подставляют фейк, реализующий те же два метода, и никакой иерархии не требуется.

`@runtime_checkable` нужен только для `isinstance`/`issubclass`, и проверяет он лишь наличие имён методов, а не их сигнатуры: класс с `get(self)` без аргументов пройдёт проверку, но упадёт при вызове. Поэтому `isinstance` по протоколу — грубый фильтр, а настоящая проверка типов происходит в `mypy`. Без `runtime_checkable` `isinstance` бросает `TypeError`.

`ABC`: явный контракт с реализацией по умолчанию

Абстрактный базовый класс задаёт каркас: часть методов обязательна, часть реализована. Экземпляр ABC с нереализованными абстрактными методами создать нельзя.

python

```
from abc import ABC, abstractmethod

class Backend(ABC):
    @abstractmethod
    def send(self, payload: bytes) -> None: ...

    def send_many(self, items: list[bytes]) -> int:
        for item in items:
            self.send(item)
        return len(items)

class NullBackend(Backend):
    def send(self, payload: bytes) -> None:
        print("drop", payload)

print(NullBackend().send_many([b"a", b"b"]))
```

text

```
drop b'a'
drop b'b'
2
```

`Backend()` бросит `TypeError: Can't instantiate abstract class Backend without an implementation for abstract method 'send'`. Выбор между ABC и Protocol простой: если нужен общий код (как `send_many`) или

жёсткая регистрация через `register()`, берите ABC; если нужен лишь контракт для типизации и подстановки в тестах — Protocol, он не создаёт иерархии и не тянет импорты.

Композиция против наследования

Наследование говорит «является»; композиция — «содержит» и «умеет через». В инфраструктурном коде второе почти всегда точнее: клиент содержит политику ретраев, а не является ею.

python

```
@dataclass(frozen=True)
class Policy:
    attempts: int = 3
    timeout: float = 2.0

class Client:
    def __init__(self, policy: Policy) -> None:
        self.policy = policy

    def call(self) -> str:
        return f"call with {self.policy.attempts} attempts"

print(Client(Policy(attempts=5)).call())
```

text

```
call with 5 attempts
```

Политика — значение, её можно сравнить, сериализовать, подменить в тесте, не заводя подклассов `ClientWithFiveAttempts`. Наследование здесь дало бы комбинаторный взрыв: политика × транспорт × кэш.

Когда класс вообще лишний? Если нужен один метод — это функция. Если нужна структура данных без поведения — `dataclass` или `NamedTuple`. Если нужно пространство имён — модуль. Класс оправдан, когда есть состояние, инварианты и несколько методов, работающих с этим состоянием, — например, пул соединений с `acquire`/`release` и счётчиком занятых слотов. «Класс на всё» в скрипте деплоя — самый частый источник лишних 300 строк.

Типичные ошибки

1. **Изменяемый атрибут класса.** `items = []` в теле класса делится всеми экземплярами. Механизм — чтение идёт из

словаря класса, а список меняется на месте. Лечение — `__init__` или `field(default_factory=...)`.

2. **Прямой вызов базового метода вместо `super()`.** `Base.handle(self)` ломает кооперативную цепочку MRO, и часть слоёв не выполняется. Механизм — `super()` ищет следующий класс в MRO конкретного экземпляра, а не в иерархии класса.
3. `self.attempts = ...` **внутри сеттера** `attempts`**.** Бесконечная рекурсия, потому что присваивание снова попадает в сеттер. Механизм — property перехватывает доступ по имени; хранить нужно в `_attempts`.
4. `isinstance` **по** `Protocol` **как проверка контракта.** `runtime_checkable` смотрит только на имена методов, не на сигнатуры; ошибка всплывает при вызове. Механизм — проверка структурная и поверхностная.
5. `frozen=True` **и мутабельное поле.** Заморозка не глубокая: список внутри замороженного `dataclass` меняется спокойно, и хеш «плывёт». Механизм — `frozen` запрещает присваивание атрибутов, а не изменение объектов.
6. **Наследование ради переиспользования кода.** Подкласс `Client(BaseClient)` ради одного метода тянет за собой все атрибуты и ломается при изменении родителя. Механизм — наследование связывает классы сильнее, чем композиция.
7. **Инициализация в** `__init__` **тяжёлых ресурсов.** Создание пула или соединения в конструкторе делает класс непроверяемым и опасным при импорте. Механизм — конструктор вызывается там, где об этом не думают (например, в тестах).

Чек-лист

- [ ] Ни один изменяемый объект не объявлен атрибутом класса; всё в `__init__` или `default_factory`.
- [ ] Все переопределённые методы вызывают `super()` либо явно задокументированы как терминальные.

- [ ] Конфигурационные и «значимые» типы — `@dataclass(frozen=True)`; для больших серий добавлен `slots=True`.
- [ ] Валидация входа живёт в `__post_init__` или сеттере, а не размазана по вызывающему коду.
- [ ] Для подменяемых зависимостей объявлен `Protocol`, а не ABC, если нужен только контракт.
- [ ] ABC используется там, где есть общий код или обязательная регистрация реализаций.
- [ ] Проверено `pytest`-ом, что фейк для протокола действительно реализует все методы (mypy, не `isinstance`).
- [ ] Каждый класс отвечает на вопрос «зачем он», а не «куда его положили»; одноразовые методы вынесены в функции.

Упражнения

1. **Разминка.** Напишите `dataclass(frozen=True, slots=True)` `Resource(kind, name, namespace="default")` с методом `fqn`, возвращающим `namespace/kind/name`. Проверьте, что объект хешируется и попадает в `set`.
2. **MRO.** Постройте иерархию из четырёх классов-«слоёв» (таймаут, ретрай, метрики, логи) так, чтобы `__mro__` давал порядок «логи → метрики → ретрай → таймаут». Объясните, почему обратный порядок базовых классов ломает ожидания.
3. **Property.** Реализуйте класс `Backoff(base, factor, max_delay)` со свойством `delays`, возвращающим кортеж задержек, и валидацией всех трёх полей. Добейтесь, чтобы некорректное значение падало при создании, а не при использовании.
4. **Protocol.** Опишите протокол `Notifier` с методом `send(text: str) -> bool` и две реализации — в лог и в HTTP. Напишите функцию, принимающую `Notifier`, и проверьте её `mypy`-ом с обеими реализациями.
5. **Рефакторинг.** Возьмите класс на 150 строк с пятью методами, три из которых не используют `self`, и разделите его на `dataclass` + функции. Оцените, насколько уменьшилось число тестовых фикстур.

Ответы и разбор к этим упражнениям — в приложении D, часть 2.

Глава 20. Асинхронность: asyncio

Сетевой клиент, который опрашивает сотню хостов, почти всё время ждёт: ответа DNS, TLS-рукопожатия, тела ответа. Потоковое ожидание — самая дорогая часть работы, и `asyncio` позволяет ждать тысячу вещей одновременно в одном потоке, не платя за тысячу стеков. Это не «Python стал многопоточным» и не ускорение вычислений: это способ не простаивать на вводе-выводе.

Глава объясняет, как устроен цикл событий (event loop), чем корутина отличается от задачи, как правильно запускать конкурентную работу и ограничивать её, как работают таймауты и отмена, что делать с блокирующими вызовами и почему `asyncio` бесполезен для CPU-bound задач. Отдельный раздел — типичные ошибки, из-за которых асинхронный код работает медленнее синхронного. Примеры проверены на CPython 3.14.9, все замеры времени — реальный вывод на тестовой машине.

Цикл событий: один поток, много ожиданий

Event loop — планировщик кооперативных задач. Он держит очередь готовых к выполнению корутин и набор файловых дескрипторов с таймерами; когда корутина доходит до `await` на операции ввода-вывода, управление возвращается в цикл, а цикл тем временем запускает другие готовые корутины. Поток всё это время один.

Отсюда два следствия. Хорошее: переключение между задачами стоит сотни наносекунд, а не микросекунды, как переключение потока ОС, и память на задачу — килобайты, а не мегабайт стека. Плохое: любая операция, которая блокирует поток, блокирует **все** задачи. Синхронный `time.sleep(1)` внутри корутины останавливает весь сервис на секунду.

Кооперативность означает, что переключение происходит только в точках `await`. Между ними корутина выполняется непрерывно, и гонок в привычном смысле нет — но и вытеснения тоже: цикл не

отнимет управление у корутины, которая считает что-то в цикле `while True`.

Корутины, `await` и точки приостановки

`async def` создаёт корутину. Вызов такой функции не выполняет тело, а возвращает объект-корутину; тело начнёт выполняться, когда кто-то сделает `await` или обернёт её в задачу.

python

```
import asyncio

async def fetch(name: str, delay: float) -> str:
    await asyncio.sleep(delay)
    return f"{name} ok"
```

`await` означает «приостанови меня здесь, пока результат не готов, и дай поработать другим». `asyncio.sleep` — асинхронный аналог `time.sleep`: он не блокирует поток, а регистрирует таймер в цикле. Всё, что вы ждёте через `await`, должно быть асинхронным: `await httpx.AsyncClient.get(...)`, `await asyncio.open_connection(...)`, `await queue.get()`.

Разница между последовательным и конкурентным ожиданием измерима:

python

```
async def main() -> None:
    t0 = time.perf_counter()
    for name in ("a", "b", "c"):
        await fetch(name, 0.2)
    print(f"sequential {time.perf_counter() - t0:.2f}s")

    t0 = time.perf_counter()
    res = await asyncio.gather(fetch("a", 0.2), fetch("b", 0.2), fetch("c", 0.2))
    print(f"gather {time.perf_counter() - t0:.2f}s {res}")

asyncio.run(main())
```

text

```
sequential 0.60s
gather 0.20s ['a ok', 'b ok', 'c ok']
```

Три запроса по 0,2 с: последовательно — 0,6 с, конкурентно — 0,2 с. Ускорение равно числу одновременных ожиданий, а не числу ядер, — и на тысяче запросов разница будет тысячекратной по времени ожидания. `asyncio.run()` создаёт цикл, выполняет

корутину до конца и закрывает цикл; это единственная точка входа, которую стоит использовать в приложении.

Задачи: `gather` и `TaskGroup`

`await coro()` выполняет корутину, но не даёт её отменить или запустить отдельно. Для этого нужна задача (Task) — обёртка, зарегистрированная в цикле: `asyncio.create_task(coro)`.

`asyncio.gather` запускает несколько awaitable и возвращает список результатов в порядке аргументов. Важная деталь — обработка ошибок: по умолчанию первое исключение пробрасывается наружу, но остальные задачи **продолжают работать**. С `return_exceptions=True` ошибки возвращаются как значения:

```python
async def boom() -> None:
    raise ValueError("bad response")

res = await asyncio.gather(boom(), asyncio.sleep(0.01, result="ok"),
    return_exceptions=True)
print(res)
```

```text
[ValueError('bad response'), 'ok']
```

С Python 3.11 предпочтителен `asyncio.TaskGroup` — структурная конкурентность: если одна задача падает, остальные отменяются, а исключения собираются в `ExceptionGroup`.

```python
async def ok(i: int) -> int:
    await asyncio.sleep(0.01 * i)
    return i

async with asyncio.TaskGroup() as tg:
    tasks = [tg.create_task(ok(i)) for i in range(3)]
print([t.result() for t in tasks])
```

```text
[0, 1, 2]
```

Правило выбора: `TaskGroup`, когда нужен «все или никто» (собрать конфиг из трёх источников), `gather(return_exceptions=True)`, когда нужен отчёт по каждому элементу (проверка 50 хостов, где падение одного не должно рушить остальные).

Таймауты

С Python 3.11 таймаут задаётся контекстным менеджером `asyncio.timeout`, который отменяет всё внутри блока и превращает отмену в `TimeoutError`:

python

```
async def slow() -> str:
    await asyncio.sleep(1.0)
    return "late"

t0 = time.perf_counter()
try:
    async with asyncio.timeout(0.2):
        await slow()
except TimeoutError:
    print(f"timed out after {time.perf_counter() - t0:.2f}s")
```

text

```
timed out after 0.20s
```

`TimeoutError` здесь — встроенное исключение (в 3.11+ `asyncio.TimeoutError` — его псевдоним), поэтому ловить нужно именно его, а не `asyncio.CancelledError`. Старый `asyncio.wait_for(coro, timeout)` всё ещё работает и полезен, когда нужно обернуть один вызов, но он отменяет корутину и на этом всё.

Таймаут обязателен на каждом сетевом вызове. Без него одна зависшая зависимость останавливает воркер навсегда, а очередь задач растёт. Типичный бюджет: 2–5 с на запрос к внутреннему API, 10–30 с на внешний, и общий дедлайн на весь обработчик, чтобы 10 последовательных вызовов по 3 с не превратились в 30 с.

Отмена и `CancelledError`

Отмена в asyncio — это исключение `CancelledError`, которое «вбрасывается» в корутину в точке `await`. Перехватывать его можно и нужно для очистки, но нельзя глотать: если не пробросить исключение дальше, задача будет считаться успешно завершённой, а вызывающая сторона не узнает об отмене.

python

```
async def worker() -> None:
    try:
        await asyncio.sleep(5)
```

```
    except asyncio.CancelledError:
        print("cleanup: releasing lock")
        raise

task = asyncio.create_task(worker())
await asyncio.sleep(0.05)
task.cancel()
try:
    await task
except asyncio.CancelledError:
    print("cancelled", task.cancelled())
```

text

```
cleanup: releasing lock
cancelled True
```

Блок `except asyncio.CancelledError` без `raise` — распространённая ошибка: она превращает отмену в «нормальное завершение» и ломает `TaskGroup`, который ждёт, что отменённая задача действительно отменится. Если нужно гарантировать очистку при любой причине выхода, используйте `try/finally`. Если нужно защитить критическую секцию от отмены — `asyncio.shield(coro)`, но помните: `shield` защищает вложенную операцию, а не внешнюю корутину, и после отмены внешней задачи вложенная продолжит работу.

Ограничение параллелизма

Запустить 10 000 задач разом — верный способ получить `Too many open files` или 429 от API. Ограничитель — `asyncio.Semaphore`: счётчик, который пропускает не больше N одновременных входов.

python

```
async def main() -> None:
    sem = asyncio.Semaphore(2)
    active = peak = 0

    async def job(i: int) -> None:
        nonlocal active, peak
        async with sem:
            active += 1
            peak = max(peak, active)
            await asyncio.sleep(0.1)
            active -= 1

    await asyncio.gather(*(job(i) for i in range(6)))
    print("peak concurrency", peak)

asyncio.run(main())
```

text

```
peak concurrency 2
```

Шесть задач, максимум две одновременно. Семафор берётся **внутри** задачи, а не вокруг `gather`: иначе вы ограничите создание задач, но не работу. Для потоковой обработки очереди удобнее `asyncio.Queue(maxsize=N)` плюс фиксированное число воркеров — это ещё и даёт backpressure: продюсер засыпает, когда очередь полна.

Блокирующие вызовы: `to_thread` и `run_in_executor`

`requests`, `psycopg2`, `boto3`, чтение файла, `subprocess.run` — синхронные библиотеки. Вызов такой функции внутри корутины блокирует цикл целиком. Решение — вынести вызов в пул потоков: `await asyncio.to_thread(func, *args)` (3.9+) или `loop.run_in_executor(None, func, *args)`.

python

```
def blocking_io() -> str:
    time.sleep(0.2)
    return "file read"

t0 = time.perf_counter()
await asyncio.gather(asyncio.to_thread(blocking_io), asyncio.sleep(0.2))
print(f"to_thread {time.perf_counter() - t0:.2f}s")
```

text

```
to_thread 0.20s
```

Два ожидания по 0,2 с заняли 0,2 с: блокирующая функция ушла в поток, цикл продолжил работать. Пул по умолчанию ограничен `min(32, os.cpu_count() + 4)` потоками — это не бесконечный ресурс, и 500 одновременных `to_thread` выстроятся в очередь. Для тяжёлых синхронных драйверов БД пул задают явно, чтобы не упираться в лимит соединений.

`to_thread` — это компромисс, а не решение: потоковое ожидание всё равно расходует память и переключается дороже. Долгосрочно синхронную библиотеку заменяют асинхронной (`httpx` вместо `requests`, `asyncpg` вместо `psycopg2`).

GIL и почему `asyncio` не про CPU

Глобальная блокировка интерпретатора (GIL) не даёт нескольким потокам одновременно выполнять байткод Python. Для ввода-вывода это неважно: GIL отпускается на время системных вызовов, поэтому потоки и `asyncio` отлично распараллеливают ожидание. Для вычислений — важно.

Замер на CPU-bound функции (два вызова по 3 млн итераций):

text

```
serial 0.36s
threads 0.36s
```

Два потока не дали ничего: они по очереди держали GIL. `asyncio` в такой задаче тоже бесполезен — `await` негде поставить, вычисление не приостанавливается. Для CPU-bound работы нужны процессы: `concurrent.futures.ProcessPoolExecutor`, `multiprocessing` или внешний воркер (Celery, RQ). Отдельно стоит знать, что в Python 3.13+ существует экспериментальная сборка без GIL (free-threaded), но по умолчанию интерпретатор по-прежнему с GIL, и полагаться на отсутствие блокировки в общем коде нельзя.

Практический вывод: asyncio выбирают, когда задача — ждать (сеть, диск, внешние API). Когда задача — считать (парсинг гигабайта JSON, ресайз картинок), выбирают процессы или выносят работу из процесса приложения.

Отладка: кто блокирует цикл

Блокирующий вызов в корутине легко не заметить: код работает, тесты проходят, а под нагрузкой время ответа растёт. У `asyncio` есть встроенный детектор — режим отладки, который замеряет, сколько времени занял каждый колбэк, и печатает предупреждение, если дольше 0,1 с.

python

```
async def main() -> None:
    time.sleep(0.6)          # блокирующий вызов вместо await
    await asyncio.sleep(0)

asyncio.run(main(), debug=True)
```

text

```
WARNING Executing <Task pending name='Task-1' coro=<main() running at ...> took
    0.600 seconds
```

Сообщение прямо указывает на виновника: корутина `main` держала поток 0,6 с. Включать режим стоит в тестах (`asyncio.run(main(), debug=True)`) и на стенде через `PYTHONASYNCIODEBUG=1` или `python -X dev`. Цена — заметное замедление, поэтому в продакшене его держат выключенным, а ловят проблему нагрузочным тестом с включённой отладкой.

Типичные ошибки

1. **Забытый `await`.** Вызов корутины без `await` ничего не выполняет и возвращает объект-корутину; в логах появляется `RuntimeWarning: coroutine 'fetch' was never awaited`. Механизм — корутина ленива, тело запускается только при `await` или создании задачи.
2. `time.sleep` **вместо** `asyncio.sleep`**.** Блокирует весь цикл: останавливаются все задачи, включая health-check и таймеры. Механизм — синхронный вызов не возвращает управление планировщику.
3. **Блокирующая библиотека в корутине.** `requests.get` внутри `async def` даёт тот же эффект, что и `time.sleep`, но незаметнее: код «асинхронный», а сервис держит один запрос за раз. Лечение — `to_thread` или асинхронный клиент.
4. **Задача без ссылки.** `asyncio.create_task(coro)` без сохранения ссылки может быть собрана сборщиком мусора до завершения; задача тихо исчезает. Механизм — цикл хранит только слабые ссылки на задачи. Лечение — `TaskGroup`, `asyncio.gather` или именованный набор задач.
5. `except asyncio.CancelledError: pass`**.** Отмена проглатывается, задача считается успешной, `TaskGroup` и таймауты перестают работать. Механизм — отмена реализована исключением, и его нужно пробрасывать.
6. `gather` **без ограничения параллелизма.** 10 000 задач открывают 10 000 соединений и получают отказ от ОС или API. Механизм — `gather` запускает всё сразу, лимит нужно ставить семафором.

7. **Таймаут только на уровне клиента.** Библиотека без таймаута ждёт вечно; общий дедлайн обработчика не задан, и один медленный ответ растягивает весь запрос. Механизм — таймаут по умолчанию часто отсутствует или равен «бесконечности».

Чек-лист

- [ ] В асинхронном коде нет `time.sleep`, `requests`, `subprocess.run` без `to_thread`.
- [ ] Каждый сетевой вызов имеет таймаут; у обработчика есть общий дедлайн.
- [ ] Параллелизм ограничен `Semaphore` или размером пула; число одновременных соединений известно.
- [ ] Все созданные задачи имеют владельца: `TaskGroup`, `gather` или явный набор с `await`.
- [ ] Обработчики `CancelledError` пробрасывают исключение; очистка — в `finally`.
- [ ] Для CPU-bound работы используются процессы, а не корутины и не потоки.
- [ ] `asyncio.run` вызывается один раз, на верхнем уровне; вложенных циклов нет.
- [ ] Есть тест, который проверяет таймаут и отмену, а не только успешный путь.

Упражнения

1. **Разминка.** Напишите `async def fetch_all(urls)` через `gather` с таймаутом 2 с на каждый запрос и верните список `(url, ok, elapsed)`. Сравните время с последовательным вариантом на 20 URL.
2. **Таймаут.** Реализуйте функцию с общим дедлайном 5 с, внутри которой три последовательных `await` по 2 с. Добейтесь, чтобы при превышении возвращался частичный результат, а не исключение.

3. **Семафор.** Ограничьте конкурентность до 5 для 100 задач и постройте гистограмму числа одновременно активных задач. Убедитесь, что пик не превышает 5.
4. **Отмена.** Напишите воркер, который держит блокировку и обновляет метрику; отмените его через 100 мс и проверьте, что блокировка освобождена, а метрика не «залипла».
5. **GIL.** Сравните время обработки 10 млн чисел в одном потоке, в четырёх потоках и в четырёх процессах (`ProcessPoolExecutor`). Объясните результат и оцените накладные расходы на передачу данных между процессами.

Ответы и разбор к этим упражнениям — в приложении D, часть 2.

Глава 21. Сеть и API на Python

Почти любой сервис на Python рано или поздно становится HTTP-клиентом: сходить в API облака за списком инстансов, в биллинг за балансом, в чужой сервис за справочником. И почти любой крупный инцидент с такими сервисами выглядит одинаково: не «упал Python», а «мы повесили все воркеры на одном медленном ответе» или «после 503 начали долбить чужой API и получили бан». В этой главе разбирается, как устроен цивилизованный HTTP-клиент: чем `requests` отличается от `httpx`, зачем нужны четыре разных таймаута, как делать ретраи, которые не превращаются в атаку на соседа, и как ограничить параллелизм. В конце мы соберём класс клиента целиком — с джиттером, лимитами и обработкой 429.

Клиент — это компонент, а не вызов функции

В стандартной библиотеке есть `urllib.request`, в экосистеме — `requests` и `httpx`. Выбор между ними определяется тем, что именно вы строите.

Возможность	`urllib.request`	`requests`	`httpx`
Синхронный API	да	да	да
Асинхронный API	нет	нет	да, `AsyncClient`
HTTP/2	нет	нет	да, `http2=True`
Таймауты	один (`timeout=`)	connect + read	connect, read, write, pool
Пул соединений	нет	`Session`	`Client`
Подмена транспорта в тестах	вручную	через сторонние библиотеки	`MockTransport` из коробки

Практическое правило: `requests` — для скриптов, миграций и административных утилит. `httpx` — если в проекте уже есть `asyncio` (синхронный клиент внутри event loop останавливает весь цикл) или нужен HTTP/2 и единый код для sync и async.

Почему не голый urlopen

Соблазн велик: `urllib` уже установлен. Но посмотрим, что происходит на реальном запросе.

python

```
import json
import urllib.request

with urllib.request.urlopen("https://api.example.internal/v1/services") as resp:
    data = json.load(resp)
print(len(data["items"]))
```

Проблемы начинаются ещё до первого ответа.

1. **Таймаут по умолчанию.** `urlopen` берёт значение из `socket.getdefaulttimeout()`, а оно равно `None` — «ждать столько, сколько ждёт ядро». Установка TCP-соединения в Linux по умолчанию растягивается примерно на две минуты (зависит от `tcp_syn_retries`), чтение ответа не ограничено. Один зависший upstream занимает все воркеры.
2. **Разнородные исключения.** На 4xx/5xx летит `HTTPError`, на проблемы с соединением — `URLError`, на таймаут — `socket.timeout`. Единой ветки обработки нет, и код обрастает `except Exception`.
3. **Нет пула соединений.** Каждый вызов — новый TCP- и TLS-handshake, то есть минимум один лишний round-trip на запрос.
4. **Нет места для политики.** Ретраи, лимиты, разбор JSON и проверка статуса пишутся руками поверх.

Проверим на локальном сервере, который отвечает через три секунды, а на неизвестный путь отдаёт 404:

python

```
import urllib.error, urllib.request

urllib.request.urlopen("http://127.0.0.1:8765/slow")          # без timeout
urllib.request.urlopen("http://127.0.0.1:8765/nope", timeout=5)
```

text

```
urllib.error.HTTPError: HTTP Error 404: Not Found
```

Первый вызов вернулся через 3.0 с — ровно столько спал сервер.

С сервером, который принял соединение и молчит, тот же код ждёт минутами — обычное поведение балансировщика, потерявшего бэкенд. `urlopen` допустим в одноразовом скрипте с явным таймаутом; в сервисе — нет.

Таймауты: четыре разных числа

«Поставить таймаут» — недостаточно: в httpx это четыре независимых значения, каждое отвечает за свой этап.

Параметр	Что ограничивает	Разумное значение
`connect`	установку TCP+TLS	2–3 с внутри ДЦ
`read`	ожидание следующей порции данных	5–15 с
`write`	отправку тела запроса	10 с
`pool`	ожидание свободного слота в пуле	1–2 с

Умолчание httpx — 5 секунд на всё сразу. Явная настройка выглядит так:

```python
import httpx

timeout = httpx.Timeout(10.0, connect=3.0, pool=2.0)
print(timeout)
```

```text
Timeout(connect=3.0, read=10.0, write=10.0, pool=10.0)
```

Первый позиционный аргумент — значение по умолчанию для всех четырёх. `pool=2.0` важен: исчерпанный пул не должен держать запрос в очереди дольше двух секунд, за это время upstream всё равно не уложится в бюджет.

И главное, чего в httpx нет: **общего дедлайна**. `read` сбрасывается на каждой прочитанной порции, поэтому сервер, отдающий по байту в секунду, формально таймаут не нарушает, а запрос может идти час. Дедлайн задаётся снаружи:

```python
import asyncio

async with asyncio.timeout(15):
    data = await client.get_json("/v1/report")
```

Правило бюджета: таймаут клиента меньше таймаута вызывающего. Если ingress рвёт запрос через 30 секунд, клиенту незачем ждать 60 — результат всё равно не успеет, зато соединение и память будут заняты.

Ретраи: что повторять, когда и сколько раз

Ретрай — компромисс: шанс на успех растёт ценой нагрузки и задержки. Поэтому сначала решается, **что** повторять.

Код/ситуация	Повторять	Почему
ошибка соединения, `ReadTimeout`	да	сеть могла моргнуть
408, 425	да	сервер сам сказал «слишком рано»
429	да, с `Retry-After`	исчерпана квота, но она восстановится
500, 502, 503, 504	да	типичные transient-ошибки
400, 401, 403, 404, 409, 422	нет	результат не изменится

Повтор 401 — способ получить блокировку учётной записи, повтор 404 — превратить быструю ошибку в медленную.

Backoff и джиттер

Задержка между попытками растёт экспоненциально и ограничивается сверху:

```python
delay = min(0.5 * 2 ** (attempt - 1), 8.0)   # 0.5, 1, 2, 4, 8, 8...
await asyncio.sleep(random.uniform(0, delay))
```

`random.uniform(0, delay)` — это full jitter: пауза равномерно распределена от нуля до потолка. Без джиттера двести подов одновременно получают 503, одновременно ждут одинаковые 0.5 с и одновременно бьют по восстановившемуся сервису — тот падает снова. С джиттером моменты повторов разные, и нагрузка размазывается. Четыре попытки с базой 0.5 с и потолком 8 с дают около 3.5 с ожидания (0.5 + 1 + 2); дальнейшие попытки почти не повышают шанс, но гарантированно съедают время запроса.

Идемпотентность

Повтор безопасен, только когда повторное выполнение не меняет результат. По RFC 9110 `GET`, `HEAD`, `OPTIONS` безопасны, `PUT` и `DELETE`

идемпотентны, а `POST` — нет: два одинаковых POST создадут две записи или спишут деньги дважды.

Для `POST` идемпотентность делается явно: клиент один раз на операцию генерирует ключ, сервер его запоминает.

python

```
import uuid
headers = {"Idempotency-Key": str(uuid.uuid4())}
resp = await client.post("/v1/payments", json=payload, headers=headers)
```

Сервер, увидев знакомый ключ, возвращает сохранённый результат первой операции вместо повторного списания. Без такого контракта `POST` не ретраится — ошибка идёт наверх.

Ограничение параллелизма

Ретраи и таймауты бесполезны, если клиент запускает тысячу задач разом. Ограничение ставится на двух уровнях: `httpx.Limits` режет число открытых соединений, `asyncio.Semaphore` — число одновременных задач, включая время на ретраи и разбор ответа.

python

```
sem = asyncio.Semaphore(4)

async def one(client: httpx.AsyncClient, i: int) -> int:
    async with sem:
        resp = await client.get(f"https://api.example.internal/items/{i}")
        return resp.status_code

async with httpx.AsyncClient(
    transport=transport,
    limits=httpx.Limits(max_connections=4, max_keepalive_connections=4),
) as client:
    codes = await asyncio.gather(*(one(client, i) for i in range(20)))
```

С транспортом, который спит 50 мс на запрос:

text

```
20 {200} 0.25s
```

Двадцать запросов по четыре в параллель — пять волн по 50 мс, всего 0.25 с. Без семафора все двадцать уйдут одновременно, и сервер получит всплеск, которого никто не обещал. Нужны оба ограничителя: `Limits` не даёт открыть лишние сокеты, но задачи копятся в очереди пула и занимают дескрипторы; семафор отсекает работу раньше.

Клиент целиком

Соберём всё в один класс. Первая часть — конструктор: таймауты, лимиты, заголовки, семафор и подменяемый транспорт (пригодится в главе про тесты).

python

```
from __future__ import annotations

import asyncio
import random
from typing import Any

import httpx

RETRY_STATUS = {408, 425, 429, 500, 502, 503, 504}
NETWORK_ERRORS = (httpx.ConnectError, httpx.ReadTimeout, httpx.RemoteProtocolError)


class ApiError(RuntimeError):
    pass


class ApiClient:
    def __init__(self, base_url: str, *, token: str, concurrency: int = 8,
                 max_attempts: int = 4,
                 transport: httpx.AsyncBaseTransport | None = None) -> None:
        self._client = httpx.AsyncClient(
            base_url=base_url,
            timeout=httpx.Timeout(10.0, connect=3.0, pool=2.0),
            limits=httpx.Limits(max_connections=concurrency,
                                max_keepalive_connections=concurrency),
            headers={"Authorization": f"Bearer {token}",
                     "User-Agent": "deploy-bot/1.4"},
            transport=transport,
        )
        self._sem = asyncio.Semaphore(concurrency)
        self._max_attempts = max_attempts

    async def __aenter__(self) -> "ApiClient":
        return self

    async def __aexit__(self, *exc: object) -> None:
        await self._client.aclose()
```

Вторая часть — сам запрос с циклом попыток:

python

```
    async def get_json(self, path: str, **params: Any) -> dict:
        return await self._request("GET", path, params=params)

    async def _request(self, method: str, path: str, **kw: Any) -> dict:
        last: Exception | None = None
        for attempt in range(1, self._max_attempts + 1):
            async with self._sem:
                try:
```

```
                    resp = await self._client.request(method, path, **kw)
                except NETWORK_ERRORS as exc:
                    last = exc
                else:
                    if resp.status_code not in RETRY_STATUS:
                        return self._unwrap(resp)
                    last = ApiError(f"HTTP {resp.status_code} на {path}")
            if attempt == self._max_attempts:
                break
            delay = min(0.5 * 2 ** (attempt - 1), 8.0)
            await asyncio.sleep(random.uniform(0, delay))
        raise ApiError(f"{path}: сдался после {self._max_attempts} попыток") from
            last

    @staticmethod
    def _unwrap(resp: httpx.Response) -> dict:
        if resp.status_code >= 400:
            raise ApiError(f"HTTP {resp.status_code}: {resp.text[:200]}")
        if resp.status_code == 204 or not resp.content:
            return {}
        return resp.json()
```

Три решения стоит отметить. Семафор берётся **внутри** цикла: иначе клиент, ждущий паузы перед повтором, держал бы слот занятым. Ошибки копятся в `last` и попадают в `raise ... from last`, так что в трейсбеке видна исходная причина. Ответ `204` не разбирается как JSON.

Проверка на подменённом транспорте, где первые два вызова отвечают 503:

python

```
calls = {"n": 0}

def handler(request: httpx.Request) -> httpx.Response:
    calls["n"] += 1
    if calls["n"] < 3:
        return httpx.Response(503, headers={"Retry-After": "1"})
    return httpx.Response(200, json={"ok": True, "attempt": calls["n"]})

async def demo() -> None:
    async with ApiClient("https://api.example.internal", token="t",
                         transport=httpx.MockTransport(handler)) as api:
        print(await api.get_json("/v1/services"))
    print("всего вызовов:", calls["n"])

asyncio.run(demo())
```

text

```
{'ok': True, 'attempt': 3}
всего вызовов: 3
```

Клиент пережил два 503 и вернул данные с третьей попытки. Тот же транспорт позволяет воспроизводить 429, таймауты и битый JSON в тестах — об этом глава 23.

JSON: разбор и его пределы

`resp.json()` вызывает `json.loads` и бросает `JSONDecodeError` (подкласс `ValueError`), если тело не JSON. Это происходит чаще, чем кажется: прокси отдаёт HTML с кодом 200, балансировщик — пустое тело, upstream — обрезанный ответ.

```python
try:
    payload = resp.json()
except ValueError:
    raise ApiError(f"не JSON от {resp.request.url}: {resp.text[:200]}") from None
```

Дальше — доверие к структуре. `payload["items"][0]["id"]` упадёт с `KeyError` или `IndexError` при первом изменении контракта, причём в глубине обработчика, где это выглядит как баг вашего кода. Для критичных интеграций схему описывают явно (dataclass или pydantic) и валидируют на границе. Для больших ответов `resp.json()` опасен: он материализует весь документ в памяти, поэтому выгрузки читают потоком:

```python
with client.stream("GET", "/v1/export") as resp:
    for line in resp.iter_lines():
        process(json.loads(line))
```

429 и 503: чего хочет сервер

429 означает «превышена квота», 503 — «временно не могу». Оба сопровождаются `Retry-After` и оба требуют, чтобы клиент замедлился. При 429 пауза не меньше указанной в заголовке, иначе — backoff с джиттером; после трёх подряд 429 попытки лучше прекратить: квота обычно считается за минуту и за оставшиеся попытки не восстановится. При 503 помогает размыкание цепи (circuit breaker): после N подряд неудач клиент какое-то время не ходит в сервис вовсе, затем пропускает один пробный запрос и при успехе возвращается в норму. Это дешевле сотен таймаутов, каждый из которых стоит секунды.

Типичные ошибки

1. **Запрос без таймаута.** `requests` без `timeout=` ждёт бесконечно, `urlopen` наследует `None` из сокета. Один медленный ответ занимает воркер навсегда, очередь растёт, health-check перестаёт отвечать, оркестратор убивает под — и после рестарта всё повторяется.
2. **Ретрай POST без ключа идемпотентности.** Клиент не знает, дошёл ли первый запрос: сеть порвалась после отправки, до ответа. Повтор создаёт вторую запись, деньги списываются дважды.
3. **Ретрай на 4xx.** 401 и 404 детерминированы, повтор даёт тот же результат. Серия 401 приводит к блокировке учётной записи, серия 404 — к штрафу за лишние запросы.
4. `except Exception` **вокруг вызова.** Под него попадают `TypeError` и `AttributeError` из вашего же кода. Ошибка программиста маскируется под сетевую и ретраится четыре раза.
5. **Новый** `httpx.get` **на каждый вызов.** Модульные функции создают и закрывают клиент, пул не переиспользуется, на каждый запрос — TCP и TLS handshake, а под нагрузкой растёт число сокетов в `TIME_WAIT`.
6. **Игнорирование** `Retry-After` **и отсутствие джиттера.** Все экземпляры сервиса повторяют синхронно и создают рукотворный всплеск ровно в момент восстановления.
7. **Слепой** `resp.json()` **без проверки статуса.** На 502 прокси отдаёт HTML, `json.loads` падает, и в логах остаётся «ошибка парсинга» вместо «upstream вернул 502» — диагностика уходит не туда.

Чек-лист

- [ ] У каждого сетевого вызова задан явный таймаут, включая `pool`, и он меньше таймаута вызывающего.
- [ ] Ретраи включаются только для сетевых ошибок, 408/425/429 и 5xx.

- [ ] Пауза между попытками экспоненциальная, с джиттером и потолком.
- [ ] `POST` ретраится только с `Idempotency-Key`, иначе ошибка идёт наверх.
- [ ] `Retry-After` учитывается как минимальная пауза.
- [ ] Параллелизм ограничен семафором и `httpx.Limits`, числа обоснованы.
- [ ] Клиент один на процесс: создаётся при старте, закрывается при остановке.
- [ ] Статус проверяется до разбора JSON: ошибка парсинга не маскирует 502.

Упражнения

1. Оберните `urllib.request.urlopen` в функцию с таймаутом 5 с, возвращающую `(status, body)` без исключений наружу. Сравните объём кода с вызовом httpx.
2. Реализуйте `retry_after(resp)`: секунды из заголовка `Retry-After`, поддержав и число, и HTTP-дату по RFC 1123.
3. Добавьте в `ApiClient` бюджет ретраев: не более 10% запросов за минуту. Продумайте поведение при исчерпании.
4. Напишите circuit breaker на 5 подряд неудач с паузой 30 с и одним пробным запросом; проверьте на `MockTransport`, отдающем 503 первые 10 вызовов.

Ответы и разбор к этим упражнениям — в приложении D, часть 3.

Глава 22. Файлы, конфиги, логирование

Сервис на Python почти всё время говорит не по HTTP, а с локальными сущностями: читает конфиг, пишет состояние, ведёт лог. Именно здесь прячутся самые некрасивые инциденты — обрезанный на полуслове JSON после перезапуска пода, «кракозябры» в отчёте, потому что файл читали в кодировке локали, и токен доступа, попавший в лог и уехавший в общий Elastic. Эта глава о трёх бытовых вещах, которые в проде оказываются критичными: работа с путями и кодировками, атомарная запись и форматы конфигов, и логирование, которое помогает, а не вредит.

Пути: pathlib вместо строк

Склейка путей через `+` и `os.path.join` работает, но не даёт ни типов, ни проверок. `pathlib` даёт объект `Path`, который умеет нужное без вызовов `os`:

```python
from pathlib import Path

root = Path("/srv/app")
conf = root / "conf" / "app.toml"
report = Path("report.csv")

print(conf)
print(report.suffix, report.stem, report.with_suffix(".json"))
```

```text
/srv/app/conf/app.toml
.csv report report.json
```

Раньше для этого требовался `os.path`: `p.parent`, `p.name`, `p.suffix`, `p.with_suffix(".json")`, `p.exists()`, `p.mkdir(parents=True, exist_ok=True)`. Последняя строка — способ создать дерево каталогов, не проверяя каждый уровень: `parents=True` создаёт цепочку, `exist_ok=True` не падает на существующем каталоге.

`Path` — это `os.PathLike`, его принимают `open`, `shutil`, `subprocess`. Тильду `open` не раскрывает — это делает шелл, поэтому нужен

`Path("~/app.conf").expanduser()`. А `.resolve()` приводит путь к абсолютному и раскрывает симлинки: если приложение читает конфиг по относительному пути, а systemd запускает его с другим `WorkingDirectory`, получатся два разных файла. Пути из аргументов командной строки разрешаются сразу, пути по умолчанию строятся от известного каталога, а не от текущего.

Кодировки и переводы строк

В Python 3.14 `open()` без аргумента `encoding` использует кодировку локали. В контейнере это обычно UTF-8, на старом сервере с `LANG=C` — ASCII или latin-1, и тогда кириллица в имени пользователя ломает запись с `UnicodeEncodeError`. UTF-8 по умолчанию запланирован в следующих ветках (PEP 686), но полагаться на это в 3.14 нельзя.

python

```
Path("out.txt").write_text("привет", encoding="utf-8")
```

Правило простое: `encoding="utf-8"` пишется явно в каждом `open`, `read_text`, `write_text`. Найти пропуски помогает запуск с `python -X warn_default_encoding` — он печатает `EncodingWarning` на каждый неявный выбор кодировки.

Второй источник сюрпризов — переводы строк. В текстовом режиме Python делает универсальный перевод: при чтении `\r\n` становится `\n`, при записи `\n` превращается в `os.linesep`. Для обычного текста это удобно, для форматов с собственными правилами — вредно.

python

```
from pathlib import Path
p = Path("a.txt")
p.write_text("a\r\nb\r\n", encoding="utf-8", newline="")
print(repr(p.read_bytes()))
print(p.read_text(encoding="utf-8").splitlines())
```

text

```
b'a\r\nb\r\n'
['a', 'b']
```

`newline=""` отключает преобразования — это важно для CSV, где модуль `csv` сам управляет разделителями и кавычками. На

Windows без него в файле появляется `\r\r\n`, и потребители видят лишние пустые строки.

python

```
import csv
with open("t.csv", "w", encoding="utf-8", newline="") as f:
    csv.writer(f).writerows([["host", "note"], ["n1", "a\nb"]])
```

text

```
b'host,note\r\nn1,"a\nb"\r\n'
```

Модуль написал `\r\n`, как требует RFC 4180, и закавычил поле с переводом строки внутри.

Атомарная запись

Запись «в лоб» — `path.write_text(...)` — не атомарна: файл сначала усекается до нуля, потом наполняется. Между этими моментами есть окно, в котором другой процесс (или вы сами после `SIGKILL`) увидит пустой или наполовину записанный файл. Для состояния, которое читается при старте, это означает потерю данных или падение на разборе.

Надёжный приём: писать во временный файл в том же каталоге, сбрасывать на диск и переименовывать. `os.replace` на POSIX атомарен в пределах одной файловой системы — читатель видит либо старую версию целиком, либо новую целиком.

python

```
import os, tempfile
from pathlib import Path

def write_atomic(path: Path, data: bytes) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    fd, tmp = tempfile.mkstemp(dir=path.parent, prefix=f".{path.name}.",
        suffix=".tmp")
    try:
        with os.fdopen(fd, "wb") as f:
            f.write(data)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, path)
    except BaseException:
        Path(tmp).unlink(missing_ok=True)
        raise

target = Path("/tmp/deploy/state.json")
write_atomic(target, b'{"replicas": 3}')
print(target.read_text(encoding="utf-8"), target.stat().st_size)
```

```
text

{"replicas": 3} 15
```

Три детали, без которых приём не работает. Временный файл создаётся **в том же каталоге**: `os.replace` через границу файловых систем не атомарен и падает с `EXDEV`. `fsync` вызывается до переименования, иначе после отключения питания на диске будет запись каталога без содержимого. `mkstemp` создаёт файл с правами `0600`, поэтому права группы выставляются явно через `os.chmod` перед заменой. Для критичных данных добавляют ещё `fsync` каталога — он фиксирует саму запись о переименовании.

Чтение построчно и память

`readlines()` и `read()` загружают файл целиком: для лога на 4 ГБ это 4 ГБ памяти плюс накладные расходы списка строк, и процесс убивает OOM killer.

Итерация по файлу читает по строке:

```
python

from pathlib import Path

errors = 0
with Path("/var/log/nginx/access.log").open(encoding="utf-8", errors="replace") as
    f:
    for line in f:
        if " 500 " in line:
            errors += 1
print(errors)
```

Память ограничена размером самой длинной строки, а не файла. `errors="replace"` спасает от падения на битом байте: без него одна повреждённая строка гигабайтного лога выбрасывает `UnicodeDecodeError` и обнуляет работу. Обратная сторона — испорченные символы молча становятся `�`, поэтому для точных данных лучше `errors="strict"` и явная обработка.

Если строки нужны пачками (например, для `executemany`), читают блоками:

```
python

def chunks(path, size=1000):
    batch = []
    with open(path, encoding="utf-8") as f:
        for line in f:
```

```
            batch.append(line.rstrip("\n"))
            if len(batch) == size:
                yield batch
                batch = []
    if batch:
        yield batch
```

Форматы конфигов: JSON, YAML, TOML

Формат	Комментарии	Типы	В стандартной библиотеке	Когда уместен
JSON	нет	строки, числа, bool, null, массивы, объекты	`json`	обмен с API, машинное состояние
YAML	да	богаче JSON (якоря, даты)	нет, нужен PyYAML	конфиги Kubernetes, CI
TOML	да	даты, числа с разделителями, таблицы	`tomllib` (чтение)	конфиги приложений

JSON не поддерживает комментарии и требует кавычек в ключах — для конфига, который правит человек, это неудобно. TOML читается стандартной библиотекой с 3.11, но только на чтение: для записи нужен `tomli-w`. YAML гибок, и именно гибкость делает его опасным:

python

```
import yaml
cfg = yaml.safe_load(Path("deploy.yaml").read_text(encoding="utf-8"))
```

`yaml.safe_load` обязателен. `yaml.load` с `Loader=yaml.Loader` и `yaml.full_load` умеют конструировать произвольные объекты Python по тегам вида `!!python/object/apply`, то есть конфиг из ненадёжного источника превращается в выполнение кода. `safe_load` поддерживает только базовые типы — этого хватает для конфигурации.

TOML читается так:

python

```
import tomllib
cfg = tomllib.loads('[db]\nhost = "10.0.0.5"\nport = 5432\n')
print(cfg, type(cfg["db"]["port"]))
```

text

```
{'db': {'host': '10.0.0.5', 'port': 5432}} <class 'int'>
```

Обратите внимание: порт — уже `int`, а не строка. TOML типизирован, и это снимает целый класс ошибок вида `"5432" + 1`.

Конфиг из окружения с валидацией

Двенадцатифакторное приложение (12-factor) хранит конфигурацию в переменных окружения: один образ работает в dev и prod, различаясь только env. Ключевое слово — **валидация**: конфиг разбирается один раз при старте, и приложение падает сразу, а не через час на первом запросе.

python

```
import os
from dataclasses import dataclass

@dataclass(frozen=True)
class Settings:
    db_host: str
    db_port: int = 5432
    workers: int = 4
    debug: bool = False

def get(env, name, default=None, cast=str):
    raw = env.get(name, default)
    if raw is None:
        raise SystemExit(f"config error: {name} is required")
    try:
        return cast(raw)
    except ValueError:
        raise SystemExit(f"config error: {name}={raw!r} is not {cast.__name__}")
            from None

def load(env=os.environ) -> Settings:
    return Settings(
        db_host=get(env, "DB_HOST"),
        db_port=get(env, "DB_PORT", "5432", int),
        workers=get(env, "WORKERS", "4", int),
        debug=get(env, "DEBUG", "0", lambda s: s.lower() in {"1", "true", "yes"}),
    )

print(load({"DB_HOST": "db.internal", "WORKERS": "16"}))
for bad in ({"DB_PORT": "5432"}, {"DB_HOST": "x", "DB_PORT": "54o2"}):
    try:
        load(bad)
    except SystemExit as exc:
        print("err:", exc)
```

text

```
Settings(db_host='db.internal', db_port=5432, workers=16, debug=False)
err: config error: DB_HOST is required
err: config error: DB_PORT='54o2' is not int
```

`frozen=True` не даёт переписать настройку в рантайме, аргумент `env` делает функцию тестируемой без патча процесса. `SystemExit` с понятным сообщением даёт ненулевой код возврата — оркестратор увидит неудачный старт и не пошлёт трафик в неработающий pod. Читать `os.environ` в глубине кода не стоит: зависимость не видна в сигнатурах, тесты требуют глобальных патчей, а забытая переменная всплывает в самом неудачном месте. Секреты в конфиг-файлы тоже не пишутся: файл попадает в образ и в git.

Логирование

`print` не умеет уровней, не ставит метку времени и не разделяет потоки. `logging` умеет всё это, но требует настройки на старте. Уровни по возрастанию: `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`. Логгер берут по имени модуля — `logging.getLogger(__name__)` — тогда видно, кто сделал запись, а фильтровать можно по префиксу.

python

```
import logging, logging.handlers
from pathlib import Path

fmt = logging.Formatter("%(asctime)s %(levelname)-7s %(name)s %(message)s",
                        "%Y-%m-%dT%H:%M:%S%z")
log = logging.getLogger("deploy")
log.setLevel(logging.INFO)
handler = logging.handlers.RotatingFileHandler(
    Path("/var/log/deploy/app.log"), maxBytes=64_000_000, backupCount=5,
    encoding="utf-8")
handler.setFormatter(fmt)
log.addHandler(handler)
log.info("started workers=%d", 4)
```

text

```
2026-10-29T19:48:20+0500 INFO    deploy started workers=4
```

Каталог для файла должен существовать и быть доступен на запись: сам обработчик его не создаёт. Формат `%Y-%m-%dT%H:%M:%S%z` даёт время с часовым поясом — без него логи из разных зон не складываются в общую картину. Аргументы передаются через `%`-подстановку, а не f-строкой: строка форматируется только если запись действительно будет выведена, и `log.debug("%s", huge_object)` не тратит время на `repr` при выключенном DEBUG.

`RotatingFileHandler` режет файл по размеру, сохраняя `backupCount` архивов с суффиксами `.1`, `.2`. Проверим на маленьком лимите:

```python
h = logging.handlers.RotatingFileHandler(d / "app.log", maxBytes=200,
    backupCount=2)
for i in range(12):
    log.info("deploy step %02d finished ok", i)
print(sorted(p.name for p in d.iterdir()))
```

```text
['app.log', 'app.log.1']
```

Для ротации по времени есть `TimedRotatingFileHandler(when="midnight")`. Важная оговорка: файловая ротация безопасна только когда писатель один. Несколько процессов, пишущих в один файл, ротируют его независимо и теряют записи — в таких случаях логируют в stdout, а ротацией занимается среда (systemd, Docker, сборщик логов).

JSON-логи

Для машинного разбора текстовый формат неудобен: regex по строке ломается на многострочных сообщениях. JSON-форматтер решает это одной функцией:

```python
import json, logging

class JsonFormatter(logging.Formatter):
    def format(self, record: logging.LogRecord) -> str:
        entry = {"ts": self.formatTime(record, "%Y-%m-%dT%H:%M:%S%z"),
                 "level": record.levelname, "logger": record.name,
                 "msg": record.getMessage()}
        if record.exc_info:
            entry["exc"] = self.formatException(record.exc_info)
        return json.dumps(entry, ensure_ascii=False)
```

```text
{"ts": "2026-10-29T19:48:20+0500", "level": "INFO", "logger": "deploy.json",
    "msg": "deployed payments to eu-west-1"}
```

`ensure_ascii=False` оставляет кириллицу читаемой, а не `\u043f...`. Трейсбек попадает в поле `exc` и не разрывает JSON на несколько строк.

Что не логировать

Логи уезжают в общий кластер, реплицируются, попадают в бэкапы и доступны гораздо большему числу людей, чем база. Поэтому в лог не пишут:

- пароли, токены, `Authorization`, cookie и ключи API — даже на DEBUG;
- номера карт, CVV, полные паспортные данные;
- персональные данные, которые не нужны для отладки: email, телефон, адрес;
- полные тела запросов и ответов платежных API.

Вместо значения логируют его идентификатор или хвост: `token=…a91f`, `user_id=88123`. Если маскирование нужно системно, ставят фильтр на логгер, который заменяет значения по регулярному выражению — но фильтр это страховка, а не замена дисциплине.

Ещё один нюанс — инъекция в лог: пользовательская строка с `\n` создаёт фальшивую запись, и в логе появляется «успешный вход», которого не было. Значения из внешнего мира экранируют или логируют через `repr`/JSON.

Типичные ошибки

1. `open` **без** `encoding`**.** Кодировка берётся из локали, а она различается между машиной разработчика, CI и продом. Файл в UTF-8 читается как latin-1: вместо текста — «кракозябры» или `UnicodeDecodeError` на первом не-ASCII символе.
2. **Запись состояния без атомарности.** `write_text` усекает файл перед записью; перезапуск в этот момент оставляет пустой файл, и сервис стартует с нуля.
3. `readlines()` **на большом файле.** Список строк материализует весь файл в памяти; на гигабайтном логе процесс убивает OOM killer.
4. `yaml.load` **вместо** `safe_load`**.** Загрузчик конструирует объекты по тегам, и конфиг из репозитория превращается в выполнение произвольного кода.

5. **Чтение env в глубине кода.** Зависимость не видна, тесты патчат `os.environ` глобально, а отсутствие переменной обнаруживается в момент запроса, а не на старте.
6. **Секреты и PII в логах.** Строка попадает в агрегатор, реплицируется и хранится месяцами; удалить её почти невозможно, а доступ к логам шире, чем к базе.
7. `print` **вместо логов в сервисе.** Ни времени, ни уровня, ни имени модуля; при нескольких воркерах строки перемешиваются, фильтровать нечего.

Чек-лист

- [ ] Во всех вызовах `open`, `read_text`, `write_text` указан `encoding="utf-8"`, для CSV — `newline=""`.
- [ ] Файлы состояния пишутся через временный файл, `fsync` и `os.replace`.
- [ ] Большие файлы читаются итерацией, а не `read()`/`readlines()`.
- [ ] YAML загружается только через `safe_load`.
- [ ] Конфиг валидируется на старте, при ошибке процесс завершается с понятным сообщением.
- [ ] Логи структурированы, содержат время с зоной и имя логгера.
- [ ] В логах нет токенов, паролей, номеров карт и лишних персональных данных.
- [ ] Ротация настроена, и в одном файле не пишет несколько процессов.

Упражнения

1. Напишите `tail(path, n)`, возвращающую последние `n` строк, не читая файл целиком; проверьте на файле в 1 ГБ.
2. Реализуйте `merge_configs(base, override)` для TOML-словарей с рекурсивным слиянием, где списки заменяются.
3. Добавьте в JSON-логгер фильтр, маскирующий значения ключей `password`, `token`, `authorization` на любой вложенности.

4. Сделайте ротацию по времени с архивированием в gzip и удалением файлов старше 14 дней без сторонних библиотек.

Ответы и разбор к этим упражнениям — в приложении D, часть 3.

Глава 23. Тесты: pytest

Тесты — это единственный способ менять код в production и не бояться. Не потому, что они доказывают отсутствие ошибок, а потому, что позволяют за минуту проверить сотню предположений, которые иначе проверялись бы неделей наблюдения за графиками. В Python де-факто один инструмент — `pytest`. В этой главе разбирается его практическая часть: как устроен набор тестов, как работают фикстуры и их области, как подменять окружение и HTTP, что именно мокать, почему покрытие в 100% ничего не гарантирует и как выглядит цикл TDD на коротком примере.

Структура тестов

Тесты лежат отдельно от кода, в каталоге `tests/`, и повторяют структуру пакета:

```text
proj/
├── booking/
│   ├── __init__.py
│   ├── api.py
│   └── duration.py
├── pyproject.toml
└── tests/
    ├── conftest.py
    ├── test_api.py
    └── test_duration.py
```

`pytest` сам находит файлы `test_*.py` и функции `test_*`. Запуск:

```bash
python -m pytest -q
```

```text
..............                                                  [100%]
14 passed in 0.05s
```

Точки — по одному символу на тест; `-q` убирает шапку, `-v` печатает имена, `-x` останавливается на первой ошибке, `-k duration` фильтрует по подстроке, `--lf` перезапускает только упавшие в прошлый раз. Последний флаг экономит минуты при отладке.

Имя теста — это документация. `test_parse_ok` бесполезен, `test_parse_rejects_trailing_garbage` сразу объясняет, что проверяется. Схема `test_<что>_<условие>_<ожидание>` покрывает 90% случаев.

`conftest.py` — файл, который `pytest` импортирует до тестов и из которого фикстуры доступны без импорта. Он же место для общих настроек: маркеров, плагинов, хуков.

assert и почему сообщения понятные

В `pytest` не нужны `self.assertEqual` и подобное — достаточно `assert`. Причина в переписывании утверждений (assertion rewriting): перед выполнением `pytest` заменяет байткод `assert`, чтобы показать значения подвыражений.

```python
def test_ttl_for_hold():
    ttl = parse_duration("1h30m")
    assert ttl == 1800, "TTL заявки должен быть полчаса"
```

```text
    def test_ttl_for_hold():
        ttl = parse_duration("1h30m")
>       assert ttl == 1800, "TTL заявки должен быть полчаса"
E       AssertionError: TTL заявки должен быть полчаса
E       assert 5400 == 1800

tests/test_fail.py:6: AssertionError
```

Видно и сообщение, и фактическое значение. Без переписывания было бы просто `AssertionError`. Отсюда практическое правило: `assert result == expected` лучше, чем `assert result == expected, "не совпало"` — второе сообщение перекрывает информативный вывод `pytest`.

Для исключений есть `pytest.raises` с проверкой текста:

```python
with pytest.raises(DurationError, match="неразобранный хвост"):
    parse_duration("1h30")
```

`match` — регулярное выражение, а не точное совпадение; злоупотреблять не стоит, привязка к тексту сообщения ломается при его переформулировке.

Фикстуры и их области

Фикстура — функция с декоратором `@pytest.fixture`, которая готовит данные или ресурс. Тест запрашивает её по имени параметра.

python

```
import httpx
import pytest

from booking.api import Api, build_client

@pytest.fixture
def seen():
    return []

@pytest.fixture
def api(seen):
    def handler(request):
        seen.append((request.method, request.url.path))
        return httpx.Response(201, json={"hold_id": "h-42"})
    return Api(build_client("https://booking.internal",
                            transport=httpx.MockTransport(handler)))
```

Фикстуры могут зависеть друг от друга — как `api` зависит от `seen`. Область видимости задаётся параметром `scope`:

Область	Когда создаётся	Типичное применение
`function` (по умолчанию)	на каждый тест	изменяемое состояние, соединения
`class`	на класс тестов	общий контекст группы
`module`	на файл	дорогая подготовка данных
`session`	на весь запуск	поднятый контейнер, тестовая БД

Правило: чем шире область, тем выше скорость и тем выше риск протечки состояния между тестами. `scope="session"` для контейнера с БД оправдан, `scope="session"` для списка, который тесты мутируют, — источник «мигающих» падений, воспроизводимых только при полном прогоне.

Ресурс, требующий закрытия, отдают через `yield`:

python

```
import sqlite3

@pytest.fixture
def db(tmp_path):
    conn = sqlite3.connect(tmp_path / "test.db")
    yield conn
```

```
    conn.close()
```

Код после `yield` выполняется всегда, даже если тест упал. `autouse=True` делает фикстуру обязательной для всех тестов в области — этим пользуются для настройки логов или переменных окружения, но злоупотребление прячет зависимости.

Параметризация

Один тест — много входов. Вместо копипаста используется `parametrize`:

python

```
@pytest.mark.parametrize("text,expected", [
    ("1s", 1),
    ("90s", 90),
    ("15m", 900),
    ("1h30m", 5400),
    ("2d", 172800),
])
def test_parse_ok(text, expected):
    assert parse_duration(text) == expected
```

Пять наборов данных — пять независимых тестов в отчёте, каждый со своим именем вида `test_parse_ok[1h30m-5400]`. Именно независимых: падение на одном наборе не мешает остальным, и в выводе сразу видно, какой именно вход сломался.

Для проверки ошибок удобно параметризовать «плохие» входы отдельно:

python

```
@pytest.mark.parametrize("bad", ["", "10x", "1h 30m", "m10", "1h30"])
def test_parse_rejects(bad):
    with pytest.raises(DurationError):
        parse_duration(bad)
```

Если наборов много и они громоздкие, задают `ids=[...]` — короткие имена вместо автоматических.

Временные файлы и окружение

Жёстко заданных путей в тестах быть не должно: `/tmp/test.db` не существует в чужом CI, а параллельный запуск двух прогонов даёт конфликт. `pytest` даёт фикстуру `tmp_path` — уникальный каталог на каждый тест, который удаляется автоматически.

python

```
def test_state_roundtrip(tmp_path):
    path = tmp_path / "seats.json"
    save_state(path, {"12A": "alice"})
    assert load_state(path) == {"12A": "alice"}
    assert path.read_text(encoding="utf-8").startswith("{")
```

`tmp_path` — это `pathlib.Path`; `tmp_path_factory` нужен, когда каталог готовится на уровне модуля или сессии.

Окружение меняется через `monkeypatch` — он автоматически откатывает изменения после теста:

python

```
def test_token_from_env(monkeypatch):
    monkeypatch.setenv("BOOKING_TOKEN", "s3cret")
    monkeypatch.delenv("HTTP_PROXY", raising=False)
    assert load_settings().token == "s3cret"
```

`monkeypatch` умеет `setenv`, `delenv`, `setattr`, `setitem`, `chdir` и `syspath_prepend`. Ключевое свойство — откат: без него `os.environ` протекает в следующие тесты, и порядок запуска начинает влиять на результат.

Что мокать и что нет

Мок — подмена зависимости объектом, поведение которого задаёт тест. Мокать стоит **на границах системы**: HTTP, база, файловая система, часы, генератор случайных чисел, платёжный провайдер. Всё это медленное, недетерминированное или недоступное в CI.

Мокать не стоит собственную логику. Если тест подменяет функцию, которую сам же и проверяет, он проверяет мок. Классический антипример — мок `httpx.Client.get`, который возвращает подготовленный `Response`: тест зелёный, а в проде падает на неверном URL, потому что URL проверялся не в том слое.

Отдельно — не мокать то, чем вы не владеете, на слишком глубоком уровне. Подмена приватных методов библиотеки ломается при обновлении и создаёт ложное чувство защищённости.

Два инструмента: `monkeypatch` (встроен, для атрибутов и окружения) и `mocker` из `pytest-mock` (обёртка над `unittest.mock`, даёт

`assert_called_once_with`, `return_value`, `side_effect`). Когда нужен именно мок объекта:

python

```
def test_notifier_called(mocker):
    send = mocker.patch("booking.notify.send_email")
    confirm_booking("12A")
    send.assert_called_once_with("12A", template="confirm")
```

Путь в `patch` — тот, **откуда** импортируется имя (`booking.notify.send_email`), а не там, где оно определено. Это самая частая ошибка при моках: патч ставится на оригинал, а модуль под тестом держит свою ссылку.

HTTP в тестах: MockTransport

`httpx` даёт транспорт, который перехватывает запросы без сети. Это лучше мока метода: настоящий `Client` строит URL, сериализует JSON, применяет таймауты и заголовки, а подменяется только сокет.

python

```
import json
import httpx
import pytest
from booking.api import Api, build_client

@pytest.fixture
def seen():
    return []

@pytest.fixture
def api(seen):
    def handler(request: httpx.Request) -> httpx.Response:
        seen.append((request.method, request.url.path,
                     json.loads(request.content)))
        return httpx.Response(201, json={"hold_id": "h-42"})
    return Api(build_client("https://booking.internal",
                            transport=httpx.MockTransport(handler)))

def test_hold_sends_ttl_in_seconds(api, seen):
    assert api.hold("12A", ttl="1h30m") == {"hold_id": "h-42"}
    assert seen == [("POST", "/holds", {"seat": "12A", "ttl_s": 5400})]
```

Тест проверяет не «вызвался ли метод», а контракт: правильный путь, правильный метод и правильное тело. Ошибки тоже воспроизводятся тривиально:

python

```
def test_hold_raises_on_conflict():
```

```
    transport = httpx.MockTransport(
        lambda request: httpx.Response(409, json={"error": "seat_taken"}))
    api = Api(build_client("https://booking.internal", transport=transport))
    with pytest.raises(httpx.HTTPStatusError) as exc:
        api.hold("12A")
    assert exc.value.response.status_code == 409
```

Покрытие и его пределы

bash

```
python -m pytest --cov=booking --cov-branch --cov-report=term-missing -q
```

text

```
Name                   Stmts   Miss Branch BrPart  Cover   Missing
------------------------------------------------------------------
booking/api.py            12      0      0      0   100%
booking/duration.py       18      0      8      0   100%
------------------------------------------------------------------
TOTAL                     30      0      8      0   100%
```

Покрытие отвечает на вопрос «какие строки выполнились», а не «какие сценарии проверены». Строка `if resp.status_code == 409: raise Conflict` считается покрытой, если условие было ложным: интерпретатор в неё заходил при вычислении. Ветвевое покрытие (`--cov-branch`) частично лечит это, требуя пройти обе ветки, но не проверяет, что результат проверен `assert`.

Практическая позиция: покрытие — индикатор, а не цель. Падение с 80% до 40% означает, что новый код пришёл без тестов. Гонка за 100% приводит к тестам, которые вызывают функции ради строчки в отчёте. Ориентир — 70–85% по критичным модулям, при этом обязательны тесты на отказные пути: таймаут, 429, битый JSON, пустой список.

Пирамида тестов

Уровень	Что проверяет	Скорость	Доля
Модульные	функции и классы в изоляции	миллисекунды	~70%
Интеграционные	связка с БД, HTTP, очередью	сотни мс	~20%
Сквозные	весь сервис снаружи	секунды и минуты	~10%

Пирамида отражает экономику: модульные тесты дёшевы в написании и мгновенны в запуске, сквозные дороги и

нестабильны. Перевёрнутая пирамида — сотни сквозных тестов через браузер — даёт прогон на сорок минут, который все перестают запускать локально, а падения списывают на «флаки».

TDD на коротком примере

Цикл: красный — зелёный — рефакторинг. Сначала тест, который падает, потом минимальный код, который его проходит.

Красный: пишем тест на ещё не реализованную функцию.

```python
from rate import parse_duration

def test_minutes():
    assert parse_duration("15m") == 900
```

```text
    def test_minutes():
>       assert parse_duration("15m") == 900
E       AssertionError: assert 0 == 900
E        +  where 0 = parse_duration('15m')

1 failed in 0.01s
```

Заглушка `return 0` уже даёт падение по правильной причине — не `ImportError`, а неверное значение. Теперь зелёный:

```python
_UNITS = {"s": 1, "m": 60, "h": 3600, "d": 86400}

def parse_duration(text: str) -> int:
    return int(text[:-1]) * _UNITS[text[-1]]
```

```text
1 passed in 0.01s
```

Рефакторинг начинается с нового падающего теста: `1h30m` текущая реализация не разберёт. Так появляется регулярное выражение и проверка хвоста. Ценность цикла не в «тестах после кода», а в том, что каждая строка реализации появляется потому, что её потребовал конкретный падающий тест, — и мёртвого кода не остаётся.

Типичные ошибки

1. **Тест зависит от порядка запуска.** Механизм: фикстура с `scope="session"` возвращает изменяемый объект, тесты его мутируют. По отдельности всё зелёное, в полном прогоне падает — и воспроизвести можно только полным набором.
2. **Мок не той цели.** Механизм: `patch("httpx.Client.get")` подменяет метод класса, но модуль под тестом держит свою ссылку или создаёт клиент иначе. Тест проходит мимо реального вызова и проверяет собственную заглушку.
3. `sleep` **в тестах.** Механизм: тест с `time.sleep(1)` в наборе из тысячи тестов добавляет минуты, а на медленном CI всё равно падает. Время подменяют, а не ждут.
4. **Проверка реализации вместо поведения.** Механизм: `assert mock.call_count == 3` фиксирует внутреннее устройство. После безобидного рефакторинга тест падает, хотя поведение не изменилось, и его отключают.
5. **Один тест на десять утверждений.** Механизм: первое падение скрывает остальные девять, а имя теста не говорит, что сломалось. Диагностика начинается с чтения кода теста.
6. **Тесты без отказных сценариев.** Механизм: проверены только успешные ответы, поэтому обработка 429, таймаута и битого JSON не выполняется ни разу — и первый реальный сбой уходит в прод.
7. **Сеть в модульных тестах.** Механизм: тест ходит в настоящий API, падает при недоступности и медленный; в CI он либо отключён, либо «флакует» и приучает игнорировать красное.

Чек-лист

- [ ] Тесты лежат в `tests/`, имена файлов и функций начинаются с `test_`.
- [ ] Общие фикстуры вынесены в `conftest.py`, их области обоснованы.
- [ ] Каждый тест проверяет одно поведение и имеет говорящее имя.

- [ ] Временные файлы создаются через `tmp_path`, окружение — через `monkeypatch`.
- [ ] HTTP подменяется `MockTransport`, а не моком метода клиента.
- [ ] Проверены отказные пути: 4xx/5xx, таймаут, битый JSON, пустой ответ.
- [ ] Набор проходит без сети и без `sleep`.
- [ ] Покрытие измеряется с `--cov-branch`, но не является единственной метрикой.

Упражнения

1. Покройте `parse_duration` параметризованными тестами, включая граничные случаи: `0s`, очень большое значение, повторяющиеся единицы (`1h1h`).
2. Напишите фикстуру `fake_clock` с `scope="function"`, которая подменяет `time.monotonic`, и проверьте на её основе логику таймаута без реального ожидания.
3. Добавьте в набор тест, который проверяет, что клиент делает ровно три попытки при 503 и не спит реальные секунды.
4. Доведите покрытие модуля до 100% по ветвям, затем найдите строку, покрытую, но не проверенную по смыслу, и напишите тест, который её действительно ловит.

Ответы и разбор к этим упражнениям — в приложении D, часть 3.

Глава 24. CLI и упаковка

У каждого сервиса рано или поздно появляется свой инструмент командной строки: миграция, чинилка состояния, генератор конфигов, обёртка над API. Сначала это скрипт на тридцать строк, потом в него добавляют флаги, потом его запускают из CI и из чужого терминала, а через полгода выясняется, что он не умеет возвращать нормальный код возврата, печатает данные в stderr и ставится только копированием файла по scp. Эта глава о том, как сделать CLI, который ведёт себя как нормальная утилита: с подкомандами, осмысленными кодами возврата, разделением потоков и установкой через `pip`.

argparse: стандартная библиотека

`argparse` входит в Python, ничего ставить не нужно, и для большинства внутренних утилит его достаточно: парсер, аргументы, подкоманды, `--help`.

Что даёт `argparse` бесплатно: приведение типов (`type=Path`), проверку обязательных аргументов, `--help`, сообщение об ошибке и код возврата 2 при неверном использовании. Три вещи стоит делать сразу: `prog` (иначе в подсказках будет `__main__.py`), `description` и `required=True` у подкоманд (иначе вызов без команды молча ничего не делает).

Ограничение: `argparse` многословен — на десять параметров уходит сорок строк.

typer: декларативный подход

`typer` строит CLI из аннотаций типов и использует `click` как движок.

python

```
import typer

app = typer.Typer(help="Заявки на места")

@app.command()
def hold(seat: str, owner: str) -> None:
    """Занять место за сотрудником."""
    typer.echo(f"{seat} -> {owner}")
```

Разница в подходе: в `argparse` описывается парсер, в `typer` — функция, а парсер выводится из сигнатуры. `--help` собирается из докстринга, подкоманды — из функций с декоратором.

Критерий	`argparse`	`typer`
Зависимости	нет	`typer`, `click`
Объём кода	растёт линейно	минимален
Подсказки и валидация	вручную	из аннотаций
Прогрев импорта	мгновенный	заметен на холодном старте
Предсказуемость	максимальная	зависит от версии

Утилита, которую человек запускает раз в сутки, выигрывает от `typer`. Скрипт, вызываемый сотни раз в минуту, — от `argparse` без лишних зависимостей.

Подкоманды

Подкоманды — это то, что отличает инструмент от скрипта: `seatctl hold 12A --owner alice` и `seatctl release 12A` с общими флагами наверху. В `argparse` они делаются через `add_subparsers`, и каждый парсер получает свою функцию через `set_defaults(func=...)`:

```python
import argparse
from pathlib import Path

def build_parser():
    p = argparse.ArgumentParser(prog="seatctl")
    p.add_argument("--state", type=Path, default=Path("seats.json"))
    p.add_argument("--version", action="version", version=f"seatctl {__version__}")
    p.add_argument("-v", "--verbose", action="count", default=0)
    p.add_argument("--dry-run", action="store_true")
    sub = p.add_subparsers(dest="cmd", required=True)

    sp = sub.add_parser("list", help="показать заявки")
    sp.set_defaults(func=cmd_list)
    sp = sub.add_parser("hold", help="занять место")
    sp.add_argument("seat")
    sp.add_argument("--owner", required=True)
    sp.set_defaults(func=cmd_hold)
    sp = sub.add_parser("release", help="снять заявку")
    sp.add_argument("seat")
    sp.set_defaults(func=cmd_release)
    return p
```

Диспетчеризация — одна строка `args.func(args)`; точка входа показана в полном листинге ниже.

Общие флаги (`--state`, `-v`, `--dry-run`) объявляются до `add_subparsers` и работают с обеих сторон от имени команды. Диспетчеризация через `args.func`, а не через `if args.cmd == ...`, — новая команда не требует правки диспетчера.

Коды возврата

Код возврата — единственный машинно-читаемый результат программы. По соглашению Unix:

Код	Значение
0	успех
1	ошибка приложения (общая)
2	неверное использование: аргументы, флаги
3	прикладной конфликт: объект занят, не найден
126	файл есть, но нет права на исполнение
127	команда не найдена
128+N	процесс убит сигналом N

Проверим на реальном инструменте:

bash

```
seatctl --state /tmp/seats.json hold 12A --owner alice; echo "rc=$?"
seatctl --state /tmp/seats.json hold 12A --owner bob;  echo "rc=$?"
seatctl --state /tmp/seats.json bogus;                  echo "rc=$?"
```

text

```
rc=0
seatctl: место 12A уже занято
rc=3
seatctl: error: argument cmd: invalid choice: 'bogus'
rc=2
```

Разница между 2 и 3 принципиальна для автоматизации: 2 означает «вызвали неправильно, чинить надо вызывающему», 3 — «вызвали правильно, но состояние не позволило». Скрипт, который видит 2, должен падать и звать человека; скрипт, который видит 3, может обработать ситуацию сам.

Из `main` код возвращается числом, а не через `sys.exit` в глубине вызовов: тогда функция тестируется как обычная, а выход происходит в одной точке. Отдельный случай — смерть от

сигнала: `SIGTERM` (15) даёт оболочке 143, Ctrl-C (`SIGINT`, 2) — 130. Писать эти коды руками не нужно, но в логах CI их приходится отличать от прикладной ошибки.

stdout и stderr

Правило: **данные — в stdout, диагностика — в stderr**. Тогда `seatctl list | wc -l` считает заявки, а сообщение «место занято» не попадает в счётчик и остаётся на экране.

```python
def cmd_list(args):
    for seat, owner in sorted(load(args.state)["holds"].items()):
        print(f"{seat}\t{owner}")          # данные
    return 0

def cmd_hold(args):
    if args.seat in state["holds"]:
        print(f"seatctl: место {args.seat} уже занято", file=sys.stderr)
        return EXIT_CONFLICT
```

`print` с `file=sys.stderr` — минимальный вариант; в инструменте с логированием то же делается через `logging.StreamHandler(sys.stderr)`. Смешивать нельзя: при перенаправлении в файл данные и шум оказываются в одном потоке, и парсер ломается.

Ещё два нюанса. Буферизация: при выводе в терминал stdout построчный, при выводе в пайп — блочный, поэтому в `tail -f` прогресс появляется рывками; помогает `print(..., flush=True)` или запуск с `python -u`. И закрытый пайп: `seatctl list | head -1` убивает читателя, следующий `print` получает `BrokenPipeError` и трейсбек в stderr. Этот случай обрабатывают явно — перехватывают `BrokenPipeError`, закрывают stdout и выходят с нулём: для `head` это нормальный конец работы.

--dry-run и --verbose

`--dry-run` — обещание «ничего не изменится». Реализация должна это гарантировать: не «записать, а потом откатить», а не писать вовсе. В коде это одна ветка в начале `save()` (см. листинг ниже): если флаг выставлен, будущее состояние печатается в stderr, и `return` происходит до любого обращения к диску. Откат здесь непригоден — он сам является записью, а падение между записью и откатом оставляет мусор.

Проверка: после `--dry-run` состояние на диске не меняется, а предполагаемое изменение видно на stderr.

bash

```
seatctl --state /tmp/seats.json --dry-run hold 14C --owner carol
seatctl --state /tmp/seats.json list
```

text

```
{
  "holds": {
    "12A": "alice",
    "14C": "carol"
  }
}
12A alice
```

`--verbose` принято делать счётным (`action="count"`): `-v` — INFO, `-vv` — DEBUG. Уровень вычисляется из числа повторов: `logging.WARNING - 10 * min(args.verbose, 2)` даёт WARNING, INFO и DEBUG для нуля, одного и двух флагов, а по умолчанию не печатается ничего, кроме предупреждений. Вывод при `-v` выглядит так:

text

```
INFO hold 14C -> carol
```

Правило: `-v` не меняет данные в stdout — диагностика идёт в stderr, иначе флаг ломает пайпы.

Сборка wheel

Упаковка начинается с `pyproject.toml`. Минимальный файл для инструмента из одного модуля:

toml

```
[build-system]
requires = ["setuptools>=77"]
build-backend = "setuptools.build_meta"

[project]
name = "seatctl"
version = "1.4.0"
description = "Управление заявками на места"
requires-python = ">=3.12"
dependencies = []

[project.scripts]
seatctl = "seatctl:main"

[tool.setuptools]
py-modules = ["seatctl"]
```

Секция `[project.scripts]` создаёт точку входа (entry point): при установке в `bin/` появляется файл `seatctl`, который вызывает `seatctl.main()` и возвращает его код. Это лучше ручного `#!/usr/bin/env python`: правильный интерпретатор и работа в venv.

Сборщик `build` ставится отдельно (`python -m pip install build`) и работает в изолированном окружении: зависимости из `[build-system]` подтягиваются во временный venv, и локальный setuptools на результат не влияет.

bash

```
python -m build --wheel
```

text

```
Successfully built seatctl-1.4.0-py3-none-any.whl
```

bash

```
python -m venv /tmp/venv-check
/tmp/venv-check/bin/pip install dist/seatctl-1.4.0-py3-none-any.whl
/tmp/venv-check/bin/seatctl --version
```

text

```
seatctl 1.4.0
```

`py3-none-any` — тег совместимости: чистый Python, любая платформа. Wheel — zip с метаданными, при установке ничего не собирается, в отличие от `sdist`.

Версионирование

Версия должна быть в одном месте. Практика: она живёт в `pyproject.toml`, а код читает её через метаданные пакета:

python

```
from importlib.metadata import PackageNotFoundError, version

try:
    __version__ = version("seatctl")
except PackageNotFoundError:
    __version__ = "0.0.0+dev"
```

Второй вариант нужен для запуска из исходников без установки: `importlib.metadata` ищет распределённый пакет и без него бросает исключение — падать из-за этого `--version` не должен.

Схема — семантическое версионирование `MAJOR.MINOR.PATCH`: MAJOR — несовместимое изменение интерфейса, MINOR — новая функциональность, PATCH — исправления. Для CLI интерфейс включает и флаги, и формат вывода: удаление флага ломает чужие скрипты не меньше, чем смена API.

Приватный индекс и установка из git

Внутренние пакеты не публикуются на PyPI. Их кладут в приватный индекс (devpi, Nexus, Artifactory, GitLab Package Registry):

```bash
pip install --index-url https://pip.internal.example/simple/ seatctl==1.4.0
```

Настройку лучше держать в конфиге, а не в каждой команде:

Файл `~/.config/pip/pip.conf`:

```ini
[global]
index-url = https://pip.internal.example/simple/
trusted-host = pip.internal.example
```

`trusted-host` отключает проверку TLS для хоста. Это компромисс: правильнее поставить внутренний CA в системное хранилище, а `trusted-host` держать как временную меру.

Версии в индексе фиксируются точно (`seatctl==1.4.0`), а для критичных зависимостей — ещё и хешем: `--require-hashes` заставляет `pip` сверять контрольные суммы и отказываться от подменённого файла. Это защита от компрометации зеркала, а не от ошибки в пакете.

Установка прямо из git нужна, когда пакет ещё не собран:

```bash
pip install "git+https://git.internal.example/platform/seatctl.git@v1.4.0"
```

Тег или коммит обязателен: `@main` означает, что завтра у коллег будет другая версия того же кода. Для приватных репозиториев pip ходит по SSH — `git+ssh://git@git.internal.example/...` — и берёт ключи из ssh-agent.

Инструмент целиком

Соберём `seatctl`: три подкоманды, общие флаги, коды возврата, dry-run. Первая часть — работа с состоянием.

python

```
"""seatctl — заявки на места: list / hold / release."""
import argparse
import json
import logging
import sys
from importlib.metadata import PackageNotFoundError, version
from pathlib import Path

EXIT_OK, EXIT_ERROR, EXIT_USAGE, EXIT_CONFLICT = 0, 1, 2, 3
log = logging.getLogger("seatctl")


def load(path: Path) -> dict:
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except FileNotFoundError:
        return {"holds": {}}


def save(path: Path, state: dict, dry_run: bool) -> None:
    if dry_run:
        print(json.dumps(state, ensure_ascii=False, indent=2), file=sys.stderr)
        return
    tmp = path.with_suffix(path.suffix + ".tmp")
    tmp.write_text(json.dumps(state, ensure_ascii=False, indent=2),
        encoding="utf-8")
    tmp.replace(path)
```

Вторая часть — команды (функция `cmd_list` приведена выше в разделе о потоках) и точка входа.

python

```
def cmd_hold(args: argparse.Namespace) -> int:
    state = load(args.state)
    if args.seat in state["holds"]:
        print(f"seatctl: место {args.seat} уже занято", file=sys.stderr)
        return EXIT_CONFLICT
    state["holds"][args.seat] = args.owner
    save(args.state, state, args.dry_run)
    log.info("hold %s -> %s", args.seat, args.owner)
    return EXIT_OK


def cmd_release(args: argparse.Namespace) -> int:
    state = load(args.state)
    if state["holds"].pop(args.seat, None) is None:
        print(f"seatctl: заявки на {args.seat} нет", file=sys.stderr)
        return EXIT_CONFLICT
    save(args.state, state, args.dry_run)
    return EXIT_OK
```

```
def main(argv: list[str] | None = None) -> int:
    args = build_parser().parse_args(argv)
    logging.basicConfig(
        level=logging.WARNING - 10 * min(args.verbose, 2),
        format="%(levelname)s %(message)s", stream=sys.stderr)
    try:
        return args.func(args)
    except OSError as exc:
        log.error("%s", exc)
        return EXIT_ERROR


if __name__ == "__main__":
    sys.exit(main())
```

`build_parser()` собирает описанные выше подкоманды; `__version__` берётся из метаданных пакета — об этом в разделе о версионировании. Обратите внимание на `main(argv=None)`: аргумент позволяет тестировать разбор без подмены `sys.argv`, а `sys.exit(main())` в единственном месте превращает код возврата в результат процесса.

Типичные ошибки

1. **Данные в stderr или диагностика в stdout.** Механизм: `print` по умолчанию пишет в stdout, и сообщение об ошибке попадает в файл, который парсят. Пайп `seatctl list | jq` падает на строке «место занято».
2. **Всегда возвращать 0.** Исключение перехвачено, но код не выставлен. CI видит успех, деплой продолжается после неудачной миграции.
3. `sys.exit` **в глубине кода.** Функция становится нетестируемой — она убивает процесс теста; обработка ошибок размазывается по слоям.
4. `--dry-run`**, который пишет.** Флаг проверяется после записи или не проверяется в одной из ветвей. Флаг, которому нельзя верить, хуже отсутствующего.
5. **Зависимость от** `@main` **при установке из git.** Механизм: ветка двигается, у двух машин оказываются разные версии одного пакета, и баг «воспроизводится только у меня» становится нормой.

6. **Версия в трёх местах.** Механизм: `__version__` в коде, версия в `pyproject.toml` и тег в git расходятся. `--version` врёт, а релиз нельзя сопоставить с коммитом.
7. **Относительные пути по умолчанию.** Механизм: `Path("seats.json")` зависит от текущего каталога. При запуске из systemd с другим `WorkingDirectory` инструмент читает не тот файл.

Чек-лист

- [ ] У программы есть `prog`, `description` и осмысленный `--help`.
- [ ] Подкоманды объявлены с `required=True`, диспетчеризация через `func`.
- [ ] Успех — 0, ошибка приложения — 1, неверное использование — 2, конфликт — отдельный код.
- [ ] Данные идут в stdout, все сообщения и логи — в stderr.
- [ ] `--dry-run` гарантирует отсутствие записи, а не откат.
- [ ] `-v` считается, уровни логов растут, вывод в stdout не меняется.
- [ ] Точка входа объявлена в `[project.scripts]`, wheel собирается и ставится в чистый venv.
- [ ] Версия читается из метаданных пакета, а не дублируется в коде.

Упражнения

1. Добавьте в `seatctl` команду `status` с флагом `--json`, которая печатает данные в stdout в двух форматах, не меняя коды возврата.
2. Сделайте `--dry-run` для `release` и напишите тест, который проверяет, что файл состояния не изменился ни на байт.
3. Перепишите `seatctl` на `typer`, сохранив коды возврата и разделение потоков, и сравните объём кода.
4. Соберите wheel, установите его в чистый venv и в контейнер `python:3.14-slim`, затем проверьте `seatctl --version` и `seatctl --help`

без исходников рядом.

5. Настройте публикацию в приватный индекс через CI: сборка wheel, проверка версии, загрузка и установка из индекса в отдельном job.

Ответы и разбор к этим упражнениям — в приложении D, часть 3.

Глава 24a. Makefile: контракт репозитория

Репозиторий, в котором нет единой точки входа, обрастает знанием, живущим в головах: «собери через `go build ./cmd/...`, тесты гоняй с `-race`, линтер поставь такой-то, деплой — скриптом из `scripts/`». Через полгода этот набор знает один человек, а CI повторяет его третьей версией. Makefile решает именно это: он описывает, **как** собрать, проверить и выкатить проект, в файле, который лежит рядом с кодом и одинаково работает у разработчика, в CI и на сервере. Глава — про make как про контракт репозитория: правила, переменные, `.PHONY`, параллелизм и полный рабочий Makefile для Go-сервиса. Заодно разберём, где make перестаёт быть удобным и честнее взять `just` или `task`.

Зачем make, если есть bash

Bash-скрипт исполняет команды сверху вниз. Он не знает, что `build` зависит от `generate`, что пересобирать нужно только изменившиеся файлы и что `test` и `lint` можно запустить одновременно. Make знает: он строит граф зависимостей и выполняет только то, что устарело.

Три свойства, которых нет у скрипта:

- **Декларативность.** Вы описываете результат и его зависимости, а не последовательность шагов. «`bin/tasksvc` зависит от `main.go` и `util.go`» — дальше make сам решает, что делать.
- **Инкрементальность.** Рецепт выполняется, если цель отсутствует или хотя бы одна зависимость новее цели по mtime. Второй `make` подряд не делает ничего — и это не оптимизация, а корректность: пересборка всего на каждый чих съедает минуты на каждом шаге CI.
- **Параллелизм.** Граф зависимостей — это готовый план распараллеливания: `make -j8` выполнит независимые цели одновременно, не нарушая порядок там, где он важен.

Плюс бесплатно: `make` есть в любом образе, где есть компилятор, и не требует установки интерпретатора вроде Python или Node.

Цели, зависимости, команды

Правило (rule) состоит из трёх частей: цель (target), зависимости (prerequisites) и рецепт (recipe) — команды, которые создают цель.

make

```
app: main.o util.o
    cc -o app main.o util.o

main.o: main.c util.h
    cc -c main.c

util.o: util.c util.h
    cc -c util.c
```

Здесь `app` зависит от двух объектных файлов, а те — от исходников и заголовка. Соберём и посмотрим, что происходит при повторном запуске и после правки заголовка:

bash

```
make            # cc -c main.c; cc -c util.c; cc -o app ...
make            # make: 'app' is up to date.
touch util.h
make            # пересоберутся оба .o и app
```

Первое правило в файле — цель по умолчанию, если не задан `.DEFAULT_GOAL`. Порядок правил в файле не задаёт порядок выполнения: make идёт по графу от запрошенной цели.

Табы, а не пробелы

Рецепты должны начинаться с символа табуляции, и это не суеверие. В синтаксисе Makefile пробелы значимы иначе: они разделяют цель и зависимости, продолжают строку, отделяют директивы. Таб был выбран в 1976 году как однозначный маркер «здесь начинается команда», и с тех пор менять его не стали, чтобы не сломать существующие файлы.

Симптом — `Makefile:2: *** missing separator. Stop.` Четыре пробела вместо таба выглядят неотличимо от таба в большинстве шрифтов, поэтому ошибка считается классикой.

make

```
build:
    go build ./...     # четыре пробела — missing separator
```

Что помогает: `.editorconfig` в корне с `indent_style = tab` для `Makefile`, режим `makefile` в редакторе (vim и neovim включают его автоматически), проверка в pre-commit хуке. С версии GNU make 3.82 маркер можно сменить директивой `.RECIPEPREFIX = >`, но это редкий приём: он ломает совместимость с чужими ожиданиями и подсветкой.

.PHONY и когда он обязателен

Make определяет «устарело ли» по файлам. Цель `test` файла не создаёт, поэтому make считает её всегда устаревшей и выполняет — пока в каталоге не появится файл или каталог с именем `test`.

bash

```
mkdir test
make test        # make: 'test' is up to date.
```

Вот это и есть тихая катастрофа: тесты не запускаются, CI зелёный, релиз с багом. `.PHONY` объявляет цель фиктивной — всегда выполнять, никогда не искать файл:

make

```
.PHONY: build test lint docker deploy clean
```

Правило простое: **любая цель, которая не создаёт файл со своим именем, должна быть в** `.PHONY`. Это `test`, `lint`, `fmt`, `deploy`, `clean`, `help` — почти весь набор точек входа. Обратная сторона тоже верна: файловые цели (`bin/app`, `docs/index.html`) в `.PHONY` добавлять нельзя, иначе инкрементальность исчезнет и сборка будет полной каждый раз.

Переменные: `=`, `:=`, `?=`

Make знает несколько операторов присваивания, и разница между ними — источник половины загадочных багов.

Оператор	Когда вычисляется	Типичное применение
`=`	при каждом использовании (лениво)	значения, зависящие от поздних переменных
`:=`	сразу, в момент присваивания	всё, где важен момент вычисления
`?=`	сразу, только если переменная не задана	значения по умолчанию
`+=`	дописывает к текущему значению	накопление флагов

Классическая ловушка — ленивое `=` с побочным эффектом:

make

```
NOW  = $(shell date +%Y-%m-%d)   # вычисляется при каждом обращении
STAMP := $(shell date +%Y-%m-%d) # вычисляется один раз
```

`$(NOW)` в двух рецептах может дать разные даты (если сборка перешла через полночь), `$(STAMP)` — всегда одну. Второй эффект ленивого присваивания хуже: `$(shell ...)` внутри `=` выполняется столько раз, сколько раз переменную раскрыли, — и медленная команда незаметно тормозит весь make.

Автопеременные доступны внутри рецепта и избавляют от повторов:

Переменная	Значение
`$@`	имя цели
`$<`	первая зависимость
`$^`	все зависимости через пробел
`$?`	зависимости новее цели
`$*`	стебель шаблонного правила

Они и делают правила переносимыми: `cc -o $@ $^` работает для любой цели, а не для конкретной.

Шаблонные правила

Шаблон (pattern rule) описывает семейство целей: `%.o: %.c` читается как «любой `.o` собирается из одноимённого `.c`». `%` — стебель, он же `$*`.

make

```
PROTOS := $(wildcard proto/*.proto)
GEN    := $(PROTOS:.proto=.pb.go)

%.pb.go: %.proto
    protoc -I proto --go_out=. --go-grpc_out=. $<

gen: $(GEN)
```

Что здесь происходит: `wildcard` собирает список исходников, подстановка `.proto` → `.pb.go` даёт список целей, шаблонное правило описывает, как получить любую из них, а `gen` — это phony-точка входа, которая требует все сгенерированные файлы. Добавили новый `.proto` — он попал в `PROTOS`, и `make gen` его сгенерирует; ничего в правилах менять не нужно.

Параллелизм и независимость целей

`make -j8` запускает до восьми рецептов одновременно. Это ускоряет сборку в разы, но обнажает ошибки проектирования: цели, которые пишут в один файл, гоняются за него, а порядок их выполнения непредсказуем.

Требование: **цели должны быть независимы по данным**. Если два рецепта пишут в `coverage.out` или в один каталог `bin/`, при `-j` получите то `permission denied`, то битый файл, то зелёный прогон — в зависимости от планировщика. Лечится разнесением артефактов по целям или явной зависимостью.

bash

```
make -j"$(nproc)" test build    # параллельно, если цели независимы
make -j1 deploy                 # последовательно там, где важен порядок
```

Для редких случаев есть `.NOTPARALLEL:`, который отключает параллелизм для указанных целей. Соблазн поставить его на всё — признак того, что граф зависимостей описан неправильно.

Makefile как точка входа

Ниже — полный Makefile для Go-сервиса: сборка, тесты, линт, образ, деплой с `DRY_RUN`. Он рабочий целиком, не фрагмент.

make

```
SHELL   := /bin/bash
.DEFAULT_GOAL := help
.DELETE_ON_ERROR:

BINARY  := tasksvc
PKG     := ./...
```

```
IMAGE   ?= registry.example.com/tasksvc
TAG     := $(shell git rev-parse --short HEAD)
ENV     ?= staging

.PHONY: help build test lint docker deploy clean

help:           ## список целей
    @grep -hE '^[a-z]+:.*## ' $(MAKEFILE_LIST) | awk -F':.*## ' '{printf "  %-8s %s\n", $$1, $$2}'

build:          ## бинарник в bin/ с версией из git
    CGO_ENABLED=0 go build -trimpath -ldflags "-X main.version=$(TAG)" -o bin/$(BINARY) ./cmd/$(BINARY)

test:           ## тесты с детектором гонок
    go test -race -count=1 -coverprofile=coverage.out $(PKG)

lint:           ## gofmt и go vet
    @test -z "$$(gofmt -l .)" || { gofmt -l .; echo "gofmt: файлы не отформатированы"; exit 1; }
    go vet $(PKG)

docker: build   ## образ с тегом коммита
    docker build --build-arg VERSION=$(TAG) -t $(IMAGE):$(TAG) -t $(IMAGE):latest .

deploy: docker  ## выкатка в ENV, DRY_RUN=1 — только план
ifdef DRY_RUN
    @echo "[dry-run] kubectl -n $(ENV) set image deploy/$(BINARY) $(BINARY)=$(IMAGE):$(TAG)"
else
    kubectl -n $(ENV) set image deploy/$(BINARY) $(BINARY)=$(IMAGE):$(TAG)
    kubectl -n $(ENV) rollout status deploy/$(BINARY) --timeout=120s
endif

clean:          ## удалить артефакты
    rm -rf bin coverage.out
```

Разбор решений, которые здесь не случайны:

- `SHELL := /bin/bash` — make по умолчанию вызывает `/bin/sh`, а в нём нет `[[ ]]`, `pipefail` и процессов подстановки. Явный bash убирает разницу между вашей машиной и Alpine-образом.
- `TAG := $(shell git rev-parse --short HEAD)` — версия попадает в бинарник через `-ldflags`, и `tasksvc --version` показывает коммит, а не «dev».
- `IMAGE ?=` и `ENV ?=` — значения по умолчанию, переопределяемые из окружения: `make deploy ENV=prod`.
- `docker: build` — образ собирается только после успешной сборки; `deploy: docker` — только после образа. Цепочка зависимостей вместо `&&` в скрипте.
- `ifdef DRY_RUN` — деплой без переменной выполняет kubectl, с переменной печатает команду. Проверка «что я собираюсь сделать» становится одним флагом, а не отдельным скриптом.

.DELETE_ON_ERROR и .DEFAULT_GOAL

`.DELETE_ON_ERROR:` решает проблему, которую легко не заметить: если рецепт упал на середине, недописанный файл остаётся на

диске и выглядит новее исходников. Следующий `make` считает цель готовой и молча использует обрезанный бинарник или наполовину сгенерированный файл.

```
make
```

```
.DELETE_ON_ERROR:
```

Эта директива удаляет цель, если рецепт завершился с ошибкой, — следующая сборка начнётся с чистого места. Ставить её стоит в каждом Makefile, где цели — файлы.

`.DEFAULT_GOAL := help` задаёт цель по умолчанию. Без неё `make` выполнит первое правило в файле — обычно это оказывается `build` или, хуже, `deploy`. Соглашение «`make` без аргументов печатает справку» безопасно и самодокументируемо: строки `## описание` рядом с целями собираются в список, как в примере выше.

Где make плох

Make — не язык программирования, хотя им пытаются пользоваться как языком. Его функции (`$(foreach)`, `$(if)`, `$(call)`, `$(eval)`) существуют, но строковые: массивов нет, области видимости нет, отладчика нет, а сообщения об ошибках указывают на строку, где раскрылось выражение, — то есть почти всегда не туда.

Признаки, что логика переросла make:

- рецепты длиннее трёх строк с `if`/`for` на bash;
- появляются функции `$(call ...)` с пятью аргументами;
- знание о том, «какой сервис куда деплоить», зашито в Makefile, а не в конфиге;
- чтобы добавить одну операцию, нужно править три места.

Инструмент	Сильные стороны	Цена
make	есть везде, граф зависимостей, инкрементальность	синтаксис с табами, слабые строки
just	рецепты без зависимостей-файлов, нормальный синтаксис	нужен бинарник, нет инкрементальности
task	YAML, переменные, зависимости между задачами	нужен бинарник, многословнее

Практическое разделение: make — когда важна инкрементальная сборка по файлам (компиляция, генерация кода, сборка документации). `just`/`task` — когда нужен просто список команд («подними окружение», «прогони миграции»), а зависимостей по mtime нет. Держать оба в одном репозитории нормально: Makefile для сборки, `just` для разработнических действий.

Интеграция с CI и pre-commit

Главная ценность Makefile — одинаковые команды везде. В CI не должно быть `go test -race ./...`, написанного заново: там должен быть `make test`.

yaml

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: "1.26"
      - run: make lint test
```

Проверка перед коммитом — тот же вызов, только локально:

bash

```
cat > .git/hooks/pre-commit <<'EOF'
#!/bin/sh
exec make lint
EOF
chmod +x .git/hooks/pre-commit
```

Важно понимать ограничение: хуки в `.git/hooks` не клонируются вместе с репозиторием, поэтому такой файл — удобство для

автора, а не гарантия для команды. Гарантию даёт фреймворк `pre-commit` (файл `.pre-commit-config.yaml` в репозитории, `pre-commit install` ставит хук у каждого) или серверный гейт в CI. Makefile в этой схеме остаётся единственным источником команд: и хук, и CI вызывают `make lint`.

Второе правило CI: версия инструментов должна быть зафиксирована. `go-version: "1.26"` в пайплайне и `go 1.26` в `go.mod` — две записи одного факта, и расхождение между ними даёт «локально работает».

Типичные ошибки

1. **Пробелы вместо таба.** `missing separator` возникает потому, что строка с рецептом не распознана как рецепт, и make пытается прочитать её как правило. Лечится `.editorconfig` и режимом `makefile`, а не поиском пробелов глазами.
2. **Цель без `.PHONY`, совпавшая с файлом.** Каталог `test` или файл `build` в корне отключает цель: make видит «файл есть, зависимости старше». Тесты не идут, CI зелёный.
3. **`=` там, где нужно `:=`.** `NOW = $(shell date)` вычисляется на каждое обращение: два рецепта получают разные значения, а медленная команда выполняется многократно. Ошибка видна не сразу — только когда сборка переходит через полночь или начинает тормозить.
4. **Скрытая зависимость от порядка при `-j`.** Два рецепта пишут в один каталог; при последовательном запуске работает, при `make -j8` — то ошибка, то битый артефакт. Гонка воспроизводится через раз, поэтому её списывают на «флаки».
5. **Нет `.DELETE_ON_ERROR`.** Упавшая на середине сборка оставляет недописанный файл новее исходников, и следующий запуск молча использует мусор.
6. **Логика в рецептах.** Ветвление на bash внутри make, деплой «по имени сервиса» через `$(call ...)` — через месяц это нечитаемо. Логика должна уходить в скрипт, make — только вызывать.

7. **Деплой по умолчанию.** Первое правило в файле — `deploy`, и `make` без аргументов в чужой руке выкатывает в прод. `.DEFAULT_GOAL := help` стоит одну строку.

Чек-лист

- [ ] `make` без аргументов печатает справку, `.DEFAULT_GOAL := help` задан.
- [ ] Все нефайловые цели перечислены в `.PHONY`.
- [ ] В Makefile нет строк с рецептами, начинающихся с пробелов; `.editorconfig` это гарантирует.
- [ ] `.DELETE_ON_ERROR` включён, файловые цели не попадают в `.PHONY`.
- [ ] `make -j"$(nproc)"` проходит с первого раза; общих выходных файлов у целей нет.
- [ ] CI вызывает `make lint test`, а не дублирует команды; версия Go совпадает с `go.mod`.
- [ ] Деплой требует явного `ENV` и поддерживает `DRY_RUN=1`.
- [ ] Секреты приходят из окружения, а не записаны в Makefile.

Упражнения

1. Напишите Makefile с целями `build`, `test`, `clean` для проекта на Go 1.26. Добейтесь, чтобы повторный `make build` ничего не делал, а `touch` любого исходника пересобирал бинарник.
2. Сломайте свой Makefile тремя способами: пробелы вместо таба, отсутствие `.PHONY` при существующем каталоге `test`, `=` вместо `:=` с `$(shell date)`. Зафиксируйте текст каждой ошибки.
3. Добавьте в Makefile цель `gen`, которая собирает все `proto/*.proto` шаблонным правилом. Проверьте, что новый файл подхватывается без правки Makefile.
4. Запустите `make -j8` на наборе из четырёх целей, две из которых пишут в один файл. Воспроизведите гонку, затем устраните её через разнесение артефактов или явную зависимость.
5. Перепишите деплой так, чтобы `make deploy` без переменных печатал план и завершался с кодом 0, а `make deploy ENV=prod`

`CONFIRM=yes` выполнял выкатку. Обоснуйте, почему защита должна быть в Makefile, а не в скрипте.

Ответы и разбор к этим упражнениям — в приложении D, часть 5.

Глава 25. Шелл: переменные, кавычки, подстановки

Шелл (shell) — это программа, которая читает строки, разбирает их на слова и запускает другие программы. Всё остальное — переменные, кавычки, подстановки — обслуживает этот единственный цикл. Большинство «загадочных» багов в скриптах автоматизации сводится к тому, что автор представляет себе строку как текст, а шелл видит в ней набор токенов, которые он обрабатывает в строго определённом порядке. В этой главе мы разберём этот порядок: как шелл хранит переменные, чем окружение процесса отличается от переменных оболочки, как работают три вида кавычек и почему отсутствие кавычек ломает скрипт на файле с пробелом. После главы вы сможете читать чужой скрипт и заранее говорить, что он сделает с конкретным именем файла.

Что такое шелл и чем bash отличается от sh

Шелл — интерпретатор с очень маленьким языком: присваивание, подстановки, условные конструкции, циклы, функции, конвейеры. Никаких типов, классов и сборщика мусора здесь нет, зато есть прямая связь с ядром через системные вызовы `fork`, `execve`, `pipe`, `wait`. Когда вы пишете `ls -l /tmp`, шелл делает `fork`, в дочернем процессе ищет `ls` в `PATH` и вызывает `execve`. Родительский процесс ждёт завершения и получает код возврата — целое число от 0 до 255, о котором подробно пойдёт речь в следующей главе.

`sh` — это не «старый bash», а имя POSIX-совместимой оболочки. В Debian, Ubuntu и производных `/bin/sh` — символическая ссылка на `dash`, в Alpine — на `busybox ash`, в RHEL-семействе исторически на `bash` в режиме совместимости. Отсюда главное практическое правило: если в первой строке скрипта написано `#!/bin/sh`, скрипт обязан быть переносимым и не использовать расширения bash. В `dash` нет `[[ ]]`, нет массивов, нет `${var//a/b}`, нет `<<<`, нет `function`, нет `source` (вместо неё — `.`), нет `set -o pipefail`. Скрипт, который

прекрасно работал у вас, на сервере с `dash` упадёт на первой же строке с `[[`.

Возможность	POSIX sh (dash)	bash 5.x
`[[ ... ]]`, регулярные выражения	нет	да
Массивы, ассоциативные массивы	нет	да
`${var//шаблон/замена}`	нет	да
`set -o pipefail`	нет	да
`local` в функциях	да (не POSIX, но есть)	да
`$(...)`, `${var:-default}`	да	да

Правило: для Linux-сервера берите `#!/usr/bin/env bash`, для initramfs и Alpine-контейнеров держитесь POSIX и проверяйте запуском под `dash`.

Переменные: оболочка и окружение

Присваивание в bash не содержит пробелов вокруг `=`:

bash

```
RELEASE="2026.10"
BUILD=42
```

`RELEASE = "2026.10"` с пробелами — это не присваивание, а попытка запустить команду с именем `RELEASE` и аргументами `=` и `2026.10`. Шелл честно ответит `RELEASE: command not found`. Это первая ошибка, которую делает каждый.

Переменная, созданная присваиванием, существует только в текущей оболочке. Чтобы она попала в окружение (environment) дочерних процессов, её нужно экспортировать:

bash

```
export RELEASE
export RELEASE="2026.10"   # экспорт и присваивание одной строкой
```

Разница принципиальная. Окружение — это массив строк вида `KEY=value`, который ядро передаёт в `execve` и который новый процесс получает в `environ`. Переменные, не помеченные на экспорт, в этот массив не попадают, и дочерний процесс о них не знает. Проверить, что видит конкретная команда, можно так:

bash

```
$ APP_ENV=staging bash -c 'echo "env=$APP_ENV"'
env=staging
$ APP_ENV=staging
$ bash -c 'echo "env=$APP_ENV"'
env=
```

В первом случае присваивание стоит перед командой и действует только на время её выполнения, при этом переменная автоматически экспортируется. Во втором — переменная осталась в текущей оболочке и в дочернюю не попала.

Посмотреть окружение — `env`, все переменные оболочки включая неэкспортированные — `set`. Удалить переменную — `unset NAME`, запретить изменение — `readonly NAME`.

`PATH` — обычная переменная окружения, список каталогов через двоеточие. Шелл перебирает их слева направо и запускает первый найденный файл, поэтому команда либо не находится (код 127), либо находится не та, что вы ждали. Bash кэширует поиск в хеш-таблице, сбросить кэш — `hash -r`.

Специальные переменные, которые шелл ведёт сам:

Переменная	Значение
`$0`	имя скрипта или оболочки
`$#`	число позиционных параметров
`$@`, `$*`	все параметры (различаются в кавычках)
`$?`	код возврата последней команды
`$$`	PID текущей оболочки
`$!`	PID последнего фонового процесса
`$1`, `$2`, …	позиционные параметры

Кавычки: три режима и цена ошибки

Это ядро главы. У шелла есть три режима заключения текста, и они влияют на то, что произойдёт с содержимым до запуска команды.

Без кавычек. Результат подстановки проходит разбиение на слова (word splitting) по символам из `IFS`, а затем — подстановку

имён файлов (globbing) по `*`, `?`, `[...]`. Пустая строка после разбиения исчезает вовсе: аргумент не передаётся.

Двойные кавычки `"..."`**.** Внутри работают `$`, `` ` ``, `\` и `!` (в интерактивном режиме), но нет разбиения на слова и нет globbing. Это режим по умолчанию для любой переменной, которую вы подставляете в команду.

Одинарные кавычки `'...'`**.** Внутри не работает вообще ничего, кроме закрывающей кавычки. Идеальны для литералов: регулярных выражений, `awk`-программ, JSON.

Есть ещё `$'...'` — ANSI-C quoting, где работают escape-последовательности: `$'строка\nс переводом'`, `$'\t'`, `$'\x41'`. Это расширение bash, в `dash` его нет.

Практическое следствие: **подставляйте переменные в двойных кавычках всегда**, кроме случаев, когда вам осознанно нужно разбиение на слова. Сравните:

bash

```
$ touch "report 2026.txt"
$ f="report 2026.txt"
$ ls $f
ls: cannot access 'report': No such file or directory
ls: cannot access '2026.txt': No such file or directory
$ ls "$f"
report 2026.txt
```

Во втором случае шелл получил два аргумента вместо одного. Команда `ls` не виновата — она получила ровно то, что ей передали.

Подстановки: $(), ${}, арифметика

Подстановка команд. `$(команда)` заменяется на stdout команды с удалёнными завершающими переводами строк. Обратные кавычки делают то же, но не вкладываются без экранирования — используйте `$()`.

bash

```
$ now=$(date +%F)
$ echo "Сегодня $now, файлов в /etc: $(find /etc -maxdepth 1 -type f | wc -l)"
Сегодня 2026-10-29, файлов в /etc: 81
```

Внутри `$()` можно писать любой конвейер, включая кавычки и вложенные подстановки.

Подстановка параметров — способ задать значение по умолчанию, обрезать путь, заменить подстроку, не вызывая внешних программ:

bash

```
$ unset PORT
$ echo "${PORT:-8080}"
8080
$ echo "после: '${PORT:-}'"
после: ''
$ : "${PORT:=8080}"
$ echo "$PORT"
8080
$ file="/var/log/nginx/access.log.1"
$ echo "${file%/*}  ${file##*/}  ${file%.*}  ${file%%.*}"
/var/log/nginx  access.log.1  /var/log/nginx/access.log  /var/log/nginx/access
```

Ключевые формы:

Форма	Смысл
`${v:-x}`	значение v, а если пусто — x (не присваивает)
`${v:=x}`	то же, но присваивает v
`${v:?сообщение}`	ошибка и выход, если v пусто
`${v:+x}`	x, только если v непусто
`${#v}`	длина строки
`${v#шаблон}`, `${v##шаблон}`	срезать кратчайший/длиннейший префикс
`${v%шаблон}`, `${v%%шаблон}`	срезать кратчайший/длиннейший суффикс
`${v/шаблон/замена}`, `${v//шаблон/замена}`	первое/все вхождения
`${v:смещение:длина}`	подстрока
`${!имя}`	значение переменной, имя которой лежит в имя

`${v:?}` — самый недооценённый инструмент. Вместо того чтобы через двадцать строк упасть на пустом пути и удалить не тот каталог, вы падаете сразу и с внятным сообщением:

bash

```
: "${BACKUP_DIR:?BACKUP_DIR не задан}"
```

Арифметика. `$(( выражение ))` считает в целых числах со знаком, 64 бита, с C-подобным синтаксисом. Деление — целочисленное, с усечением к нулю: `$(( 17 / 5 ))` даёт `3`, `$(( -17 / 5 ))` даёт `-3`. Дробных чисел в bash нет; для них нужен `awk`, `bc` или python.

bash

```
$ n=17; m=5
$ echo "частное $(( n / m )), остаток $(( n % m ))"
частное 3, остаток 2
$ (( n > m )) && echo "n больше m"
n больше m
```

Конструкция `(( выражение ))` без `$` возвращает код 0, если результат не ноль, и 1, если ноль — именно поэтому она удобна как условие.

IFS и разбиение на слова

`IFS` (Internal Field Separator) — переменная, по символам которой шелл режет результат подстановки. По умолчанию это пробел, табуляция и перевод строки. Разбиение применяется **только** к неэкранированным подстановкам: `$var`, `$(...)`, `$((...))`. Литеральный текст в командной строке не разбивается — он уже разбит парсером.

Отсюда два следствия.

Первое: `for x in $(cat list.txt)` — плохая конструкция. Она разобьёт строки по пробелам, а не по строкам, и выполнит globbing для каждого слова. Правильно — `while IFS= read -r line; do ...; done < list.txt`. `IFS=` отключает обрезку ведущих и хвостовых пробелов, `-r` запрещает интерпретацию обратных слэшей.

Второе: если вы меняете `IFS` для своих нужд, делайте это локально для команды или сохраняйте старое значение. `IFS=`, на всю оболочку ломает разбор `$@` и всё, что ниже.

Отключить globbing для фрагмента можно через `set -f` (вернуть — `set +f`); это нужно, когда имена приходят из данных и содержат `*` или `?`.

Массивы

Индексированный массив объявляется скобками, элементы разделяются пробелами:

bash

```
$ hosts=(web-01 "web 02" db-01)
$ echo "${#hosts[@]} элемента, первый ${hosts[0]}"
3 элемента, первый web-01
$ hosts+=(cache-01)
$ printf '<%s>\n' "${hosts[@]}"
<web-01>
<web 02>
<db-01>
<cache-01>
```

`"${hosts[@]}"` разворачивается в отдельные слова с сохранением границ, `"${hosts[*]}"` — в одну строку, склеенную первым символом `IFS`. Разница та же, что между `"$@"` и `"$*"`. Обращение к несуществующему индексу при `set -u` — ошибка, поэтому для «может быть пусто» используйте `${hosts[0]:-}`.

Ассоциативные массивы (bash 4+) требуют `declare -A` — без него `arr[key]=value` молча создаст индексированный массив с индексом 0:

bash

```
$ declare -A codes=([200]=ok [404]=missing [500]=error)
$ for k in "${!codes[@]}"; do echo "$k=${codes[$k]}"; done | sort
200=ok
404=missing
500=error
```

shebang и PATH

Первая строка скрипта — `#!` плюс путь к интерпретатору. Это не комментарий: ядро читает эти байты и запускает указанную программу, передавая ей путь к скрипту как аргумент. Отсюда два варианта:

bash

```
#!/bin/bash          # фиксированный путь, bash точно есть в системе
#!/usr/bin/env bash  # найти bash в PATH
```

`env`-вариант удобен, когда bash стоит в `/usr/local/bin`, но он же и опаснее: `PATH` может указать на другую версию, и скрипт с `declare -A` упадёт на bash 3.2 из системного каталога macOS. Для юнитов systemd и cron надёжнее явный путь.

Скрипт должен быть исполняемым (`chmod +x`), иначе оболочка попробует выполнить его как текст и получит `Permission denied` (код 126). Запуск `bash script.sh` shebang игнорирует полностью — это частая причина «локально работает, в cron нет», если локально вы запускаете через `bash`, а cron — напрямую.

Пример: почему пробел в имени файла ломает скрипт

Соберём минимальный полигон. В каталоге два файла с пробелами в именах:

bash

```
$ mkdir -p /tmp/demo && cd /tmp/demo
$ printf 'alpha\nbeta\n' > "my report.txt"
$ printf 'gamma\n' > "my notes.txt"
$ ls -1
my notes.txt
my report.txt
```

Плохой вариант — «перебрать все txt через `ls`»:

bash

```
for f in $(ls *.txt); do
    echo "== $f"
    wc -l $f
done
```

Реальный вывод:

```
== my
wc: my: No such file or directory
== notes.txt
wc: notes.txt: No such file or directory
== my
wc: my: No such file or directory
== report.txt
wc: report.txt: No such file or directory
```

Что произошло: `$(ls *.txt)` вернул строку `my notes.txt\nmy report.txt`; разбиение по `IFS` разрезало её на четыре слова; цикл получил четыре итерации; `wc` получил по одному неверному аргументу. Скрипт не сообщил об ошибке внятно, а код возврата цикла — 0, потому что последняя команда `echo` успешна.

Правильный вариант:

bash

```
for f in *.txt; do
```

```
    printf '== %s\n' "$f"
    wc -l < "$f"
done
```

```
== my notes.txt
1
== my report.txt
2
```

Здесь `*.txt` разворачивает сам шелл (globbing), каждое имя остаётся одним словом, а `"$f"` защищает его от повторного разбиения. Редирект `< "$f"` избавляет от вывода имени файла в `wc`.

Для имён из `find` используйте NUL как разделитель: `find ... -print0` и `read -r -d ''` — NUL не может встретиться в имени файла, это единственный полностью надёжный способ передать произвольный список.

Типичные ошибки

1. **Отсутствие кавычек вокруг переменной.** Механизм: результат подстановки проходит word splitting и globbing. Имя с пробелом превращается в несколько аргументов, имя с `*` — в список файлов. Лечится `"$var"` везде, где разбиение не нужно осознанно.
2. `var = value` **с пробелами.** Механизм: парсер видит команду `var` с аргументами, а не присваивание. Bash печатает `command not found`, а в скрипте с `set -e` просто завершается.
3. **Расчёт на** `export` **по умолчанию.** Механизм: дочерний процесс получает только то, что попало в `environ`. Переменная, заданная без `export`, невидима для `kubectl`, `envsubst` и любого скрипта, вызванного из вашего.
4. `for i in $(cat file)`**.** Механизм: разбиение идёт по `IFS`, а не по строкам, плюс globbing. Строки с пробелами и `*` обрабатываются неверно. Лечится `while IFS= read -r`.
5. `sh` **вместо** `bash` **при bash-синтаксисе.** Механизм: `/bin/sh` — это `dash` или `ash`, где `[[ ]]` и массивов нет. Скрипт падает на разборе, иногда после частичного выполнения.
6. `echo` **для данных.** Механизм: `echo` по-разному трактует `-n`, `-e` и обратные слэши в разных реализациях. Для предсказуемого вывода используйте `printf`.

7. **Сравнение чисел как строк.** Механизм: `[ "$a" \> "$b" ]` сравнивает лексикографически, поэтому `10` меньше `9`. Для чисел — `(( a > b ))` или `[ "$a" -gt "$b" ]`.

Чек-лист

- Все подстановки переменных в командах заключены в двойные кавычки, кроме мест, где разбиение нужно намеренно.
- В первой строке указан интерпретатор, и он соответствует синтаксису: `#!/usr/bin/env bash` для bash-конструкций, `#!/bin/sh` только для POSIX.
- Обязательные переменные проверены через `${VAR:?сообщение}` или явный `if [ -z ... ]`.
- Для обхода файлов используется glob или `find -print0`, а не `$(ls)` и не `$(find)`.
- Для чтения строк из файла — `while IFS= read -r line`, для списков в массиве — `"${arr[@]}"`.
- Для вывода данных используется `printf`, а не `echo` с флагами.
- Арифметика выполняется через `$(( ))` или `(( ))`, а не через `expr` и не строковыми сравнениями.
- Скрипт прогнан `shellcheck` и запущен под тем интерпретатором, который указан в shebang.

Упражнения

1. Напишите скрипт, который принимает путь к файлу и печатает имя каталога, имя файла без расширения и расширение, используя только подстановки параметров (`${var%...}`, `${var##...}`) без вызова `basename` и `dirname`. Проверьте на путях без слэшей и без расширения.
2. Есть файл со списком хостов, по одному в строке, некоторые строки содержат пробелы и символ `*`. Напишите цикл, который печатает каждый хост в угловых скобках, не интерпретируя содержимое. Объясните, почему вариант с `for h in $(cat hosts)` даёт другой результат.

3. Объявите индексированный массив из пяти элементов, добавьте шестой через `+=`, выведите количество и все элементы по одному на строку. Затем сделайте то же для ассоциативного массива «код HTTP → описание».
4. Напишите функцию `die`, которая печатает сообщение в stderr и завершает скрипт с заданным кодом, используя `${2:-1}` как значение по умолчанию. Примените её для проверки двух обязательных переменных окружения.
5. Возьмите любой свой скрипт, где есть `$(ls ...)` или неквотированные переменные, и перепишите его так, чтобы он корректно работал в каталоге с именами вида `a b.txt`, `*star`, `-n`. Проверьте `shellcheck` до и после.

Ответы и разбор к этим упражнениям — в приложении D, часть 3.

Глава 26. Управляющие конструкции и коды возврата

В шелле нет логического типа. Есть целое число от 0 до 255, которое возвращает каждая команда, и есть конструкции, которые это число читают. Вся ветвящаяся логика скрипта — `if`, `while`, `&&`, `||`, `case` — это проверка кода возврата, а не проверка истины. Кто это понимает, тот перестаёт писать `if [ $? -eq 0 ]` после каждой команды и начинает пользоваться языком так, как он задуман. В главе разобраны `test` и `[[ ]]`, числовые и строковые сравнения, `case`, циклы, функции с локальными переменными, `set -euo pipefail` вместе с его реальными ловушками, `trap` для очистки и полный пример скрипта с разбором аргументов и валидацией.

Код возврата — единственный способ общения

Каждый процесс завершается с кодом выхода (exit status). Соглашение в Unix описано в `sysexits.h` и в документации bash:

Код	Значение
0	успех
1–125	ошибка приложения, смысл задаёт сама программа
126	команда найдена, но не исполняема
127	команда не найдена
128+N	процесс убит сигналом N (например, 143 = 128+15, SIGTERM)

Шелл хранит код последней команды в `$?`. Прочитать его можно один раз: следующая команда, включая `echo`, его перезапишет. Отсюда классическая ошибка:

bash

```
$ grep -q pattern file
$ echo "нашлось: $?"
нашлось: 0
$ if [ $? -eq 0 ]; then echo yes; fi
```

Во второй строке `$?` уже относится к `echo`, а не к `grep`. Правильно — проверять команду напрямую: `if grep -q pattern file; then ...`.

Конструкции `&&` и `||` — это тоже проверки кода возврата, сокращённые до одной строки. `a && b` выполнит `b`, только если `a` вернула 0. `a || b` выполнит `b`, только если `a` вернула не 0. Комбинировать их как `a && b || c` опасно: `c` выполнится и тогда, когда `a` успешна, а `b` упала. Для «сделай или умри» есть идиома `cmd || { echo "ошибка" >&2; exit 1; }` — фигурные скобки группируют команды в текущей оболочке, в отличие от `( )`, которая создаёт подоболочку.

test, [и [[

`test` — встроенная команда, `[` — её синоним с обязательной закрывающей `]`. Обе возвращают 0 при истине, 1 при лжи и 2 при ошибке разбора. Поскольку `[` — обычная команда, пробелы обязательны: `[ -f "$f" ]`, а `[-f "$f"]` даст `command not found`.

`[[ ]]` — не команда, а ключевое слово bash. Отличия существенны:

Поведение	`[ ]`	`[[ ]]`
Разбиение на слова внутри	да	нет
Globbing внутри	да (опасно)	нет
&&, `\`	\	, !`
Регулярные выражения =~	нет	да
Сравнение <, >	нужно экранировать	работает как есть
Пустая переменная	`[ $x = a ]` — синтаксическая ошибка	безопасно

Практический вывод: в bash используйте `[[ ]]`, в POSIX-скриптах — `[ ]` с обязательными кавычками. Смешивать нельзя: `[[` в `dash` — синтаксическая ошибка.

Основные проверки файлов: `-e` существует, `-f` обычный файл, `-d` каталог, `-L` символическая ссылка, `-r`/`-w`/`-x` доступен на чтение/запись/исполнение, `-s` непустой, `-nt`/`-ot` новее/старее.

bash

```
$ f=/etc/hostname
$ [[ -f $f && -r $f ]] && echo "читаемый файл, $(wc -c < "$f") байт"
```

```
читаемый файл, 7 байт
$ [[ $f == /etc/* ]] && echo "лежит в /etc"
лежит в /etc
$ s="release-2026.10.tar.gz"
$ [[ $s =~ ^release-([0-9.]+)\.tar\.gz$ ]] && echo "версия: ${BASH_REMATCH[1]}"
версия: 2026.10
```

`=~` срабатывает на регулярном выражении POSIX ERE, а захваченные группы попадают в массив `BASH_REMATCH`. Регулярное выражение не нужно брать в кавычки — в кавычках оно станет литералом.

Числа и строки

Внутри `[ ]` и `[[ ]]` действуют разные операторы, и путать их — источник тихих ошибок.

Числа	Строки
`-eq`, `-ne`, `-lt`, `-le`, `-gt`, `-ge`	`=`, `==`, `!=`, `<`, `>`

`[ "$a" -gt "$b" ]` сравнивает числа. `[[ $a > $b ]]` сравнивает строки лексикографически: `10` окажется меньше `9`, потому что `1` < `9`. Если хотя бы одна сторона не число, `-gt` печатает `integer expression expected` и возвращает 2, что в `if` трактуется как «ложь» — скрипт молча пойдёт по ветке `else`.

Надёжнее арифметический контекст: `(( a > b ))`. Он не требует `$`, понимает `+`, `*`, `**`, битовые операции, поддерживает присваивание `(( count += 1 ))`. Ограничение — 64-битные целые со знаком и никаких дробей.

bash

```
$ used=87; total=100
$ (( used * 100 / total >= 85 )) && echo "пора чистить диск: ${used}%"
пора чистить диск: 87%
```

Проверка «строка непустая» пишется как `[[ -n $var ]]`, «пустая» — `[[ -z $var ]]`. Сокращение `[[ $var ]]` эквивалентно `-n` и удобно, но менее явно.

case

`case` сопоставляет строку с glob-шаблонами и читается лучше длинной цепочки `if/elif`. Ветка `*)` — это `default`, она должна идти последней.

bash

```
case "$1" in
    start|up)    systemctl start app ;;
    stop|down)   systemctl stop app ;;
    restart)     systemctl restart app ;;
    status)      systemctl status --no-pager app ;;
    *)           echo "использование: $0 {start|stop|restart|status}" >&2; exit 2 ;;
esac
```

В bash 4+ есть `;&` (провалиться в следующую ветку без проверки) и `;;&` (продолжить проверку следующих шаблонов). В POSIX их нет, поэтому в переносимых скриптах — только `;;`.

Циклы

`for` перебирает список слов; без списка — позиционные параметры (`for a` то же, что `for a in "$@"`).

bash

```
for f in /var/log/*.log; do
    printf '%s\t%s\n' "$(stat -c %s "$f")" "$f"
done
```

Если файлов нет, glob останется литералом `/var/log/*.log`, и цикл выполнится один раз с несуществующим путём. Защита — `shopt -s nullglob` (пустой результат вместо литерала) или проверка `[[ -e $f ]]` внутри.

C-подобный цикл нужен, когда важны индексы: `for ((i = 0; i < n; i++)); do ...; done`.

`while` выполняется, пока команда-условие возвращает 0. Главный случай — чтение файла построчно:

bash

```
while IFS= read -r line; do
    [[ -z $line || $line == \#* ]] && continue
    printf '%s\n' "$line"
done < /etc/hosts
```

`until` — то же самое с инвертированной проверкой: тело выполняется, пока условие ложно. `break` и `continue` принимают уровень: `break 2` выходит из двух вложенных циклов.

Функции и локальные переменные

Функция объявляется двумя способами: `name() { ...; }` (POSIX) и `function name { ...; }` (bash). Тело — составная команда в текущей оболочке, поэтому функция может менять переменные вызывающего.

Возвращает функция не значение, а код: `return N`, где N — 0–255. Вернуть строку можно только через stdout и подстановку команд, и это надо делать осознанно:

bash

```
count_logs() {
    local dir=${1:?каталог не задан}
    find "$dir" -maxdepth 1 -name '*.log' -type f | wc -l
}
n=$(count_logs /var/log)
echo "логов: $n"
```

`local` объявляет переменную в области функции и восстанавливает прежнее значение при выходе. Без `local` функция затрёт глобальную переменную с тем же именем — классический баг в скриптах, где функции писались позже основного кода. В bash `local` можно комбинировать: `local -r name=value`, `local -a arr`, `local -n ref=$1` (nameref, ссылка на переменную по имени).

set -euo pipefail и его подводные камни

Стандартная шапка продакшен-скрипта:

bash

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
```

- `-e` (`errexit`) — завершить скрипт при первой команде с ненулевым кодом.
- `-u` (`nounset`) — ошибка при обращении к необъявленной переменной.
- `-o pipefail` — код конвейера равен коду самой правой упавшей команды, а не последней.

Каждый из этих флагов имеет исключения, и именно на них ломаются скрипты, которые «работали без set -e».

`-e` **не срабатывает там, где код проверяется.** В условиях `if`, `while`, `until`, в левой части `&&` и `||`, а также с унарным `!` errexit отключается. Это задумано: вы же сами проверяете код. Но следствие опаснее, чем кажется: функция, вызванная в условии, выполняется целиком без errexit, и упавшая в середине команда не остановит её.

bash

```
$ cat p3.sh
#!/usr/bin/env bash
set -euo pipefail
f() { echo "шаг1"; false; echo "шаг2"; }
if f; then echo "f ok"; else echo "f failed"; fi
$ bash p3.sh
шаг1
шаг2
f ok
```

`false` внутри `f` проигнорирована, `шаг2` выполнился, функция вернула 0 (последняя команда успешна), и скрипт считает, что всё хорошо. Если такая функция делает деплой, вы получите «успешный» деплой без половины шагов.

`-e` **не наследуется в подстановку команд.** `$(...)` выполняется в подоболочке, где errexit выключен, поэтому `x=$(false; echo продолжаем)` напечатает `продолжаем` и вернёт 0. Включается это через `shopt -s inherit_errexit` (bash 4.4+), но тогда ломаются идиомы вида `x=$(grep ... || true)`.

`local` **съедает код возврата.** `local out=$(false)` всегда возвращает 0, потому что код даёт сама `local`, а не подстановка. Проверено:

bash

```
$ cat p7.sh
#!/usr/bin/env bash
set -euo pipefail
f() { local out=$(false); echo "внутри f, out='$out'"; }
f
echo "после f"
$ bash p7.sh
внутри f, out=''
после f
```

Правильно — разделять объявление и присваивание: `local out; out=$(false)` — тогда errexit сработает и скрипт остановится.

Арифметика возвращает код по значению. `((i++))` при `i=0` возвращает 1, потому что результат выражения — ноль:

bash

```
$ cat pitfalls.sh
#!/usr/bin/env bash
set -euo pipefail
i=0
echo "до ((i++))"
((i++))
echo "после: i=$i"
$ bash pitfalls.sh
до ((i++))
$ echo $?
1
```

Скрипт молча умер на первой итерации счётчика. Лечится `((i++)) || true`, `i=$((i+1))` или `(( ++i ))`, где результат не ноль.

`-u` **и пустые массивы.** В bash 4.4+ `"$@"` при отсутствии аргументов безопасен, в более старых версиях `set -u` на нём падал. Для массивов используйте `${arr[@]:-}`, для необязательных параметров — `${1:-}`.

`pipefail` **и штатный ненулевой код.** `grep` возвращает 1, когда ничего не нашёл, — это не ошибка. Конвейер `... | grep -q pattern` под `pipefail` уронит скрипт. Осознанно: `if ! grep -q pattern file; then ....`

Вывод: `set -euo pipefail` полезен, но не заменяет явные проверки. Ошибки в условиях, в подстановках и в арифметике он не поймает; зато поймает опечатку в имени переменной (`-u`) и падение середины конвейера (`pipefail`).

trap и очистка

`trap` регистрирует команду, которая выполнится при получении сигнала или при выходе. Синтаксис: `trap 'команды' СПИСОК_СИГНАЛОВ`, где `EXIT` — псевдосигнал завершения.

bash

```
#!/usr/bin/env bash
set -euo pipefail
WORK=$(mktemp -d)
cleanup() {
    local rc=$?
    rm -rf "$WORK"
    echo "cleanup: удалил каталог, код=$rc" >&2
```

```
}
trap cleanup EXIT
echo "работаю в $WORK"
exit 0
```

```
работаю в /tmp/tmp.Zepla6JscR
cleanup: удалил каталог, код=0
```

Внутри обработчика `$?` ещё содержит код завершения — поэтому `local rc=$?` нужно первой строкой, иначе его перезапишет `rm`. `trap` на `EXIT` выполняется и при `exit 1`, и при падении из-за `set -e`, и при SIGINT, если для него нет отдельного обработчика.

`SIGKILL` перехватить нельзя, `SIGTERM` — можно и нужно: это штатный способ попросить процесс завершиться. Подробно сигналы разобраны в главе 28. Снять обработчик — `trap - EXIT`.

Пример: скрипт с аргументами и валидацией

Скрипт разбирает флаги через `getopts`, проверяет обязательные параметры и возвращает разные коды для разных классов ошибок.

bash

```
#!/usr/bin/env bash
set -euo pipefail

usage() {
    cat <<'USAGE'
Использование: args.sh -e ОКРУЖЕНИЕ [-d КАТАЛОГ] [-n]
  -e  окружение: dev, stage, prod (обязательно)
  -d  каталог релиза (по умолчанию /srv/app)
  -n  dry-run: только показать план
USAGE
}

ENV_NAME=""
TARGET="/srv/app"
DRY_RUN=0

while getopts ":e:d:nh" opt; do
    case "$opt" in
        e) ENV_NAME="$OPTARG" ;;
        d) TARGET="$OPTARG" ;;
        n) DRY_RUN=1 ;;
        h) usage; exit 0 ;;
        :) echo "ошибка: -$OPTARG требует значения" >&2; usage >&2; exit 2 ;;
        *) echo "ошибка: неизвестный флаг -$OPTARG" >&2; usage >&2; exit 2 ;;
    esac
done
shift $((OPTIND - 1))

case "$ENV_NAME" in
    dev|stage|prod) ;;
```

```
    "") echo "ошибка: не задан -e" >&2; usage >&2; exit 2 ;;
    *)  echo "ошибка: неизвестное окружение '$ENV_NAME'" >&2; exit 2 ;;
esac

[[ -d "$TARGET" ]] || { echo "ошибка: каталога $TARGET нет" >&2; exit 3; }

echo "окружение=$ENV_NAME каталог=$TARGET dry_run=$DRY_RUN"
if (( DRY_RUN )); then
    echo "план: выложить релиз в $TARGET/$ENV_NAME"
else
    echo "выкладываю релиз в $TARGET/$ENV_NAME"
fi
```

Разбор: `":e:d:nh"` — ведущее двоеточие переводит `getopts` в «тихий» режим: неизвестный флаг приходит как `?`, отсутствующее значение — как `:`. Это позволяет печатать свои сообщения и отделять код 2 (ошибка использования) от кода 3 (неверное состояние среды). `OPTIND` после цикла указывает на первый неопознанный аргумент.

```
$ ./args.sh
ошибка: не задан -e
Использование: args.sh -e ОКРУЖЕНИЕ [-d КАТАЛОГ] [-n]
...
$ echo $?
2
$ ./args.sh -e prod -d /tmp -n
окружение=prod каталог=/tmp dry_run=1
план: выложить релиз в /tmp/prod
$ ./args.sh -e qa
ошибка: неизвестное окружение 'qa'
$ echo $?
2
```

Типичные ошибки

1. `if [ $? -eq 0 ]` **после команды.** Механизм: `$?` перезаписывается каждой командой, и между проверкой и интересующей командой почти всегда что-то стоит. Проверяйте команду напрямую в `if`.
2. `[ $var = x ]` **при пустой переменной.** Механизм: подстановка исчезает, остаётся `[ = x ]` — `unary operator expected`, код 2, ветка `else`. Лечится кавычками или `[[ ]]`.
3. `-gt` **для строк или** `>` **для чисел.** Механизм: `-gt` требует целых и печатает ошибку, `>` сравнивает лексикографически. Для чисел — `(( ))`.
4. `set -e` **и функция в условии.** Механизм: errexit отключается на всё время выполнения команды, чей код проверяется.

Внутренние сбои игнорируются, функция возвращает код последней команды.

5. `local x=$(cmd)`**.** Механизм: код возврата даёт `local`, а не подстановка, поэтому ошибка теряется. Объявляйте и присваивайте раздельно.
6. `((i++))` **под** `set -e`**.** Механизм: результат 0 даёт код 1, скрипт завершается. Используйте `((++i))` или `((i++)) || true`.
7. `grep` **в конвейере под** `pipefail`**.** Механизм: «не найдено» — это код 1, а не ошибка. Оборачивайте в `if` или добавляйте `|| true` осознанно.

Чек-лист

- Каждая команда, код которой важен, проверяется напрямую (`if cmd; then`), а не через `$?`.
- В сравнениях используются правильные операторы: `-eq`/`-lt` для чисел, `=`/`==` для строк, `(( ))` для арифметики.
- Все раскрытия переменных закавычены, в bash используется `[[ ]]`.
- Скрипт начинается с `set -euo pipefail`, и в нём нет мест, где эти флаги молча не работают (условия, `local x=$(...)`, `((i++))`).
- Каждый `exit` имеет осмысленный код, а `usage` печатается в stderr при ошибке использования.
- Временные файлы создаются через `mktemp`, а удаляются в `trap ... EXIT`.
- Функции используют `local` для всех своих переменных.
- Скрипт проверен `shellcheck` и запущен с пустыми, неверными и граничными аргументами.

Упражнения

1. Напишите функцию `is_number`, возвращающую 0, если аргумент — целое (в том числе отрицательное), и 1 иначе. Реализуйте через `[[ =~ ]]` и через `(( ))`, сравните поведение на строке `12abc`.

2. Перепишите разбор аргументов из примера так, чтобы поддерживался `--env=prod` (длинные опции). Подсказка: `getopts` длинные опции не умеет, нужен ручной цикл по `"$@"` с `case`.
3. Возьмите скрипт с `set -e` и функцией, вызываемой в `if`, и докажите на практике, что внутренняя ошибка не останавливает выполнение. Затем перепишите функцию так, чтобы она возвращала явный код и проверяла каждый шаг.
4. Напишите `case`, который по расширению файла выбирает инструмент обработки (`*.json` → `jq`, `*.yaml` → `yq`, `*.csv` → `awk`), а для неизвестного расширения завершается с кодом 3.
5. Сделайте скрипт, который в цикле опрашивает URL через `curl -fsS` до успеха или до истечения таймаута, с задержкой между попытками, и корректно возвращает 0/1. Убедитесь, что `set -e` не убивает его на первой неудачной попытке.

Ответы и разбор к этим упражнениям — в приложении D, часть 3.

Глава 27. Текст: grep, sed, awk, jq, xargs, find

Девяносто процентов работы DevOps-инженера с текстом — это «найти, вырезать, посчитать, заменить». Логи, конфиги, вывод `kubectl`, JSON из API, списки файлов: всё это потоки строк, которые нужно превратить в ответ на конкретный вопрос. Unix даёт для этого набор маленьких инструментов, каждый из которых делает одно действие и умеет читать stdin и писать в stdout. В этой главе — grep и sed для поиска и замены, awk для полей и агрегатов, jq для JSON, find и xargs для работы с файлами, а также конвейеры, которые решают реальные задачи и не ломаются на пробелах в именах. Отдельно разберём, чего этими инструментами делать не стоит.

grep: поиск и коды возврата

`grep` печатает строки, совпавшие с шаблоном. Базовый шаблон — BRE; с `-E` включается расширенный (ERE), с `-P` — PCRE, если сборка его поддерживает. Все примеры ниже работают с `access.log` в формате combined плюс время ответа: поле 1 — IP, 7 — путь, 9 — код ответа, 10 — байты, 11 — секунды.

bash

```
$ grep -c '" 5[0-9][0-9] ' access.log
2
$ grep -ci 'timeout' /var/log/syslog
0
```

Ключевые флаги:

Флаг	Действие
`-i`	игнорировать регистр
`-v`	инвертировать: строки без совпадения
`-c`	только число совпавших строк
`-n`	номер строки
`-o`	печатать только совпавшую часть
`-r` / `-R`	рекурсивно по каталогу (R идёт по симлинкам)
`-l`	только имена файлов
`-A n` / `-B n` / `-C n`	контекст после/до/вокруг
`-I`	пропускать двоичные файлы

Коды возврата — часть контракта: 0 — совпадения есть, 1 — совпадений нет, 2 — ошибка (нет файла, битое регулярное выражение). Поэтому `grep` в условии работает естественно:

bash

```
if grep -q '^PermitRootLogin yes' /etc/ssh/sshd_config; then
    echo "root-логин по паролю разрешён" >&2
fi
```

`-o` превращает grep в экстрактор. Из лога доступа можно достать методы и пути:

bash

```
$ grep -oE '(GET|POST|DELETE) [^ ]+' access.log | sort | uniq -c | sort -nr
      4 GET /api/orders
      1 POST /api/orders
      1 GET /static/app.js
      1 GET /healthz
      1 GET /api/users/7
      1 DELETE /api/orders/41
```

`grep -r` без ограничений опасен: он заходит в `.git`, `node_modules` и виртуальные окружения. Ограничивайте через `--include='*.yaml'` и `--exclude-dir={.git,node_modules,venv}`. Для поиска по коду лучше `rg` (ripgrep), но в минимальном контейнере часто есть только grep — тогда эти флаги обязательны.

sed: замена и диапазоны

`sed` — потоковый редактор. Программа состоит из адресов и команд: `[адрес[,адрес]]команда`. Главная команда — `s/что/на что/флаги`, где `g` заменяет все вхождения в строке, а не первое.

bash

```
$ sed -E 's/^([0-9.]+) [^"]*"(GET|POST|DELETE) ([^ ]+) [^"]*" ([0-9]{3}).*/\4 \2
    \3/' access.log
200 GET /api/orders
404 GET /api/orders/42
201 POST /api/orders
200 GET /healthz
500 GET /api/orders
304 GET /static/app.js
500 GET /api/orders
200 GET /api/users/7
204 DELETE /api/orders/41
200 GET /api/orders
```

Здесь `-E` включает ERE, группы `(...)` подставляются как `\1`…`\9`. Тот же результат без `-E` потребовал бы экранировать каждый `(`, `)`, `{`, `}` и `|`.

Диапазоны адресов выбирают фрагмент файла. `sed -n '/10:13:/,/10:14:02/p'` печатает всё от первой строки, содержащей `10:13:`, до строки с `10:14:02` включительно; `-n` подавляет автоматический вывод, иначе строки напечатались бы дважды.

Типичные операции:

bash

```
sed -n '1,20p' file              # первые 20 строк
sed -n '$p' file                 # последняя строка
sed '/^#/d; /^$/d' file          # убрать комментарии и пустые строки
sed -i.bak 's/listen 80;/listen 8080;/' nginx.conf   # замена с бэкапом
sed -E 's/[[:space:]]+$//' file  # обрезать хвостовые пробелы
```

`-i` редактирует файл на месте. GNU sed создаёт временный файл и переименовывает его, поэтому права и владелец сохраняются, а вот жёсткие ссылки рвутся, а симлинк заменяется обычным файлом. На BSD/macOS `-i` требует аргумент: `sed -i '' 's/a/b/' file`. В скриптах, которые должны работать везде, либо не используйте `-i`, либо проверяйте платформу.

Символ `&` в замене означает всё совпадение, `\&` — литеральный амперсанд. Это ловушка при замене URL с query-строкой.

awk: поля, условия, агрегаты

`awk` — это маленький язык программирования, а не «grep с колонками». Программа состоит из пар `шаблон { действие }`; выполнение идёт по строкам, `$0` — вся строка, `$1`…`$NF` — поля, разделитель задаётся `-F` или переменной `FS`.

bash

```
$ awk '{b += $10; n++} END {printf "%d запросов, %.1f KB, средний %.0f B\n", n,
    b/1024, b/n}' access.log
10 запросов, 2.5 KB, средний 251 B
```

Здесь нет ни одной внешней команды: awk сам прочитал файл, сложил байты из десятого поля и посчитал среднее. Блок `END` выполняется один раз после последней строки.

Подсчёт по группам — через ассоциативный массив:

bash

```
$ awk '{c[$9]++} END {for (s in c) printf "%s %d\n", s, c[s]}' access.log | sort
    -k2 -nr
200 4
500 2
404 1
304 1
204 1
201 1
```

Порядок обхода массива в awk не определён — поэтому `sort` в конце обязателен, иначе вывод будет меняться от запуска к запуску.

Условие в шаблоне заменяет `if`. Медленные запросы — те, где одиннадцатое поле больше 0,5 с:

bash

```
$ awk '$11 > 0.5 {printf "%s %s %ss %s\n", $1, $6, $11, $7}' access.log
198.51.100.90 "GET 0.872s /api/orders
203.0.113.7 "GET 1.204s /api/orders
```

Полезные встроенные переменные: `NR` — номер строки во входе, `FNR` — номер строки в текущем файле, `NF` — число полей, `FS`/`OFS` — разделители на входе и выходе. `awk -F: '{print $1}' /etc/passwd` печатает имена пользователей, `awk -F, 'NR>1 {sum+=$3} END {print sum}' report.csv` суммирует третий столбец CSV без заголовка.

Важная оговорка про реализации: в Debian и Ubuntu по умолчанию стоит `mawk`, он быстрее, но не имеет расширений GNU:

`gensub()`, `asort()`, `strftime()` могут отсутствовать. Если нужен `gawk`, его ставят явно и вызывают по имени.

jq: JSON как поток

`jq` — фильтр для JSON: на вход документ, на выход результат выражения. Точка `.` — весь документ, `.items` — поле, `.items[]` — поток элементов массива, `.a.b[0]` — вложенный доступ.

bash

```
$ jq -r '.items[] | select(.status=="failed") | "\(.id) \(.dur)s"' deploy.json
r102 44s
r104 31s
```

`-r` выводит сырые строки без кавычек — это то, что нужно для передачи в другие команды. `-c` печатает компактно, одной строкой на документ — удобно для `while read`.

Агрегаты собираются почти как в функциональном языке:

bash

```
$ jq '[.items[].dur] | {n: length, total: add, avg: (add/length)}' deploy.json
{
  "n": 4,
  "total": 95,
  "avg": 23.75
}
$ jq '[.items[]] | group_by(.status) | map({status: .[0].status, n: length})'
    deploy.json
[
  {
    "status": "failed",
    "n": 2
  },
  {
    "status": "ok",
    "n": 2
  }
]
```

`--arg name value` передаёт строку из шелла в фильтр без риска инъекции: `jq -r --arg e prod '.app + "@" + $e' deploy.json` вернёт `orders@prod`. `--argjson` передаёт уже готовый JSON.

Проверка в скрипте: `jq -e '.items | length > 0' deploy.json` вернёт 0, если выражение истинно и не `null`/`false`, и 1 иначе — это готовое условие для `if`. Для потоковой обработки NDJSON используйте `jq -c '. | {id, status}'` построчно; `-s` (slurp), наоборот, собирает весь вход в один массив, что на гигабайтах съест память.

find и xargs

`find` обходит дерево и печатает или обрабатывает найденное. Порядок аргументов важен: выражения объединяются неявным `-a` (И), `-o` (ИЛИ) требует скобок.

bash

```
$ find logs -name 'app*.log' -type f
logs/app 1.log
logs/app 3.log
logs/app 2.log
$ find logs -name '*.log' -mtime +7 -print
logs/old.log
```

`-mtime +7` — «изменён более 7 суток назад» (целые сутки), `-mmin -30` — минуты, `-size +100M` — размер. `-delete` удаляет найденное; обязательно ставьте `-type f` и узкий `-name`, иначе одна опечатка в выражении удалит лишнее. Безопасный порядок — сначала посмотреть `-print`, потом заменить на `-delete`.

`-exec команда {} \;` запускает команду на каждый файл, `-exec команда {} +` собирает файлы в один вызов — второй вариант на порядок быстрее и почти всегда предпочтительнее. Разница как между `xargs -n1` и `xargs` без ограничения.

`xargs` превращает stdin в аргументы. Без `-0` он режет вход по пробелам и ломается на именах с пробелами; с `-0` читает NUL-разделённый поток от `find -print0`:

bash

```
$ find logs -name 'app*.log' -print0 | xargs -0 -n1 -P3 sh -c 'printf "[%s]\n"
    "$1"' _
[logs/app 1.log]
[logs/app 3.log]
[logs/app 2.log]
```

`-P3` — три параллельных процесса, `-n1` — один аргумент на запуск, `_` — подстановка `$0` для `sh -c`. Параллелизм полезен для сетевых операций (скачивание, `curl`), но не для записи в один файл: порядок вывода не гарантирован. `xargs` возвращает 123, если хоть один вызов завершился с ошибкой, и 124, если команда вернула 255 — это надо учитывать в скриптах с `set -e`.

Конвейер реальной задачи

Задача: по логу доступа за минуту найти долю ошибок 5xx, самый медленный путь и общий трафик.

bash

```
#!/usr/bin/env bash
set -euo pipefail

LOG=${1:?укажите путь к логу}
total=$(wc -l < "$LOG")
err=$(grep -cE '" 5[0-9]{2} ' "$LOG" || true)
pct=$(awk -v e="$err" -v t="$total" 'BEGIN {printf "%.2f", (t ? e*100/t : 0)}')

echo "всего запросов: $total"
echo "ошибок 5xx: $err (${pct}%)"
echo "самый медленный:"
awk '{print $11, $7}' "$LOG" | sort -nr | head -1

echo "трафик по кодам:"
awk '{b[$9] += $10} END {for (s in b) printf "  %s: %.1f KB\n", s, b[s]/1024}'
    "$LOG" | sort
```

```
всего запросов: 10
ошибок 5xx: 2 (20.00%)
самый медленный:
1.204 /api/orders
трафик по кодам:
  200: 2.0 KB
  201: 0.3 KB
  204: 0.0 KB
  304: 0.0 KB
  404: 0.1 KB
  500: 0.0 KB
```

`grep -c` возвращает 1, когда совпадений нет, и `set -e` убил бы скрипт — поэтому `|| true` здесь не костыль, а осознанная обработка штатного кода. `awk -v` передаёт значения из шелла в программу, не собирая её конкатенацией строк, — это защищает от поломки на кавычках в данных.

Чего не делать

Не парсите `ls`. Вывод `ls` зависит от локали, флагов и типа терминала; имена с пробелами и переводами строк ломают разбор. Для обхода файлов есть glob и `find -print0`.

Не grep-айте двоичные файлы без нужды. `grep -r` по каталогу с `.so` и архивами печатает `Binary file ... matches` и тратит время. `-I` или `--include` решают проблему.

Не используйте `cat file | grep pattern`**.** Это лишний процесс: `grep pattern file` делает то же самое. `cat` оправдан только при склейке нескольких файлов.

Не считайте `wc -l` **числом строк в файле без завершающего перевода.** Он считает символы `\n`: файл из одной строки без `\n` в конце даст 0.

Не доверяйте `sort` **по умолчанию для чисел.** `sort` лексикографический, поэтому `10` идёт перед `9`. Нужен `sort -n`, для версий — `sort -V`, для людей — `sort -h`.

Не редактируйте конфиги регуляркой без проверки. `sed -i` не валидирует результат. Сначала `sed` в stdout, потом `diff`, потом `-i`.

Типичные ошибки

1. `grep` **под** `set -e`**.** Механизм: «не найдено» — код 1. Скрипт падает там, где должен был продолжить. Лечится `if grep -q`, `|| true` или `grep -c ... || true`.
2. `xargs` **без** `-0`**.** Механизм: вход режется по пробелам и кавычкам, имя `app 1.log` становится двумя аргументами. Нужен `find -print0 | xargs -0`.
3. `awk` **без явного разделителя на CSV.** Механизм: по умолчанию `FS` — пробелы, вся строка становится одним полем `$1`. Нужен `-F`, и учёт кавычек в значениях (для настоящего CSV — специализированный инструмент).
4. **Порядок вывода ассоциативного массива.** Механизм: awk не хранит порядок ключей, jq сохраняет порядок входа, но `group_by` сортирует. Без `sort` вывод нестабилен и «прыгает» между запусками.
5. `sed -i` **на симлинке или в контейнере с read-only слоем.** Механизм: GNU sed заменяет файл новым, симлинк превращается в обычный файл; в read-only ФС — `Device or resource busy`. Проверяйте `-L` и точку монтирования.
6. `find -delete` **без** `-type f`**.** Механизм: под выражение попадают каталоги, и `-delete` удаляет их рекурсивно в порядке обхода.

Сначала `-print`.

7. `jq` **без** `-e` **в условии.** Механизм: `jq` возвращает 0, даже если результат `null` или `false`, — проверка «поле есть и истинно» молча проходит. Нужен `jq -e`.

Чек-лист

- Для поиска указаны ограничения: `--include`, `--exclude-dir`, `-I`.
- Коды возврата `grep` (0/1/2) учтены в скрипте, а не подавлены бездумно.
- `sed -i` применяется после проверки вывода в stdout и, при необходимости, с бэкапом.
- В awk задан `FS`, а вывод отсортирован, если порядок важен.
- JSON обрабатывается через `jq`, а не `grep`/`sed`; в условиях используется `-e`.
- Списки файлов передаются через `-print0` и `xargs -0` (или `-exec ... +`).
- `find -delete` предварён прогоном с `-print` и содержит `-type f`.
- Числовая сортировка выполняется с `-n`/`-V`/`-h`, а не по умолчанию.

Упражнения

1. Из файла `/etc/passwd` получите список имён пользователей с UID ≥ 1000, отсортированный по UID. Сделайте это одной командой `awk` без `sort` и объясните, почему без `sort` порядок не гарантирован.
2. Постройте отчёт по логу: для каждого IP — число запросов, число 5xx и максимальное время ответа. Отсортируйте по числу 5xx по убыванию.
3. Напишите `sed`-выражение, которое превращает `key = value` в `KEY=value` (ключ в верхнем регистре, пробелы убраны). Проверьте на строках с комментариями и пустыми значениями.

4. Есть NDJSON-файл, по одному объекту на строку. Через `jq` выведите суммарное время по каждому `env`, затем через `xargs` передайте список `id` упавших сборок в команду `curl -fsS -X POST`.
5. Найдите в `/var/log` все файлы старше 14 дней, посчитайте их суммарный размер, затем удалите их, используя только `find` и `xargs` с NUL-разделителем. Оформите как скрипт с `DRY_RUN`.

Ответы и разбор к этим упражнениям — в приложении D, часть 3.

Глава 28. Процессы, сигналы, перенаправления

Скрипт — это процесс, у которого есть родитель, идентификатор, файловые дескрипторы и обработчики сигналов. Пока вы пишете линейные команды, эта механика невидима; как только появляется фоновая задача, демон, таймаут или «почему скрипт завис на конвейере», без неё не обойтись. В главе разобрано, как ядро создаёт процессы и кто их хоронит, как работает job control, что означают сигналы `SIGTERM`, `SIGKILL`, `SIGHUP` и `SIGINT`, как устроены перенаправления и дескрипторы, чем опасен конвейер без `pipefail`, зачем нужна подстановка процессов и почему `nohup` — не супервизор. В конце — рабочий пример graceful-останова с таймаутом и эскалацией до `SIGKILL`.

Процесс, PID, PPID

Каждый запущенный процесс получает идентификатор (PID) — целое число, уникальное в момент жизни процесса и переиспользуемое после выхода. У процесса есть родитель (PPID): тот, кто вызвал `fork`. Корень дерева — процесс с PID 1, в современных дистрибутивах это `systemd`, в контейнере — команда, указанная в `ENTRYPOINT`.

bash

```
$ echo "мой pid=$$, родитель=$(ps -o ppid= -p $$ | tr -d ' ')"
мой pid=541909, родитель=541907
$ ps -o pid,ppid,stat,etime,cmd --ppid $$
    PID    PPID STAT     ELAPSED CMD
 541911  541909 S          00:00 sleep 0.2
```

`ps` показывает состояние (`STAT`): `R` — выполняется, `S` — спит в ожидании, `D` — непрерываемый сон (обычно ждёт диск), `Z` — зомби, `T` — остановлен. Зомби — это завершившийся процесс, чей код возврата ещё не забрал родитель: запись в таблице процессов осталась, ресурсов нет. Лечится не убийством зомби (его уже нет), а тем, чтобы родитель вызвал `wait`. Если родитель — ваш скрипт, следите, чтобы он дожидался фоновых задач.

Полезные команды: `ps aux` — всё, `pgrep -af nginx` — найти по имени с полной командной строкой, `pstree -p` — дерево, `kill -0 PID` — проверить существование и право послать сигнал (сигнал не отправляется). Каталог `/proc/PID` даёт всё остальное: `cmdline`, `environ`, `fd`, `status`.

Job control

Job control — механизм интерактивной оболочки. `cmd &` запускает команду в фоне и печатает номер задачи и PID, `jobs` показывает список, `fg %1` возвращает задачу на передний план, `bg %1` продолжает остановленную в фоне. `Ctrl-Z` посылает `SIGTSTP` — приостановить, `Ctrl-C` — `SIGINT`, `Ctrl-\` — `SIGQUIT` с дампом памяти.

В скриптах job control обычно выключен (`set +m`), и `&` означает просто «не ждать». Управлять такими задачами нужно через `$!` и `wait`:

bash

```
./worker-a.sh & a=$!
./worker-b.sh & b=$!
wait "$a"; rc_a=$?
wait "$b"; rc_b=$?
```

`wait` без аргументов ждёт все дочерние процессы и возвращает 0. `wait "$pid"` возвращает код именно этого процесса — это единственный надёжный способ узнать, чем закончилась фоновая задача. `disown` убирает задачу из таблицы оболочки: при выходе из терминала ей не пошлют `SIGHUP`, но и `wait` на неё уже не подействует.

Важно: `kill %1` работает только в интерактивной оболочке с job control. В скрипте адресуйте процессы по PID.

Сигналы

Сигнал — асинхронное уведомление процессу от ядра или другого процесса. У каждого сигнала есть действие по умолчанию, и его можно изменить: перехватить (обработчик в коде) или игнорировать.

Сигнал	Номер	Смысл	Ловится
SIGHUP	1	терминал закрыт; у демонов — «перечитай конфиг»	да
SIGINT	2	Ctrl-C, прервать	да
SIGQUIT	3	Ctrl-\, завершить с дампом	да
SIGKILL	9	безусловное убийство	**нет**
SIGPIPE	13	запись в закрытый конец конвейера	да
SIGTERM	15	штатное завершение	да
SIGCHLD	17	потомок завершился	да
SIGSTOP	19	приостановить	**нет**
SIGTSTP	20	Ctrl-Z, приостановить	да

Номера зависят от архитектуры; в Linux на x86-64 они такие, как в таблице. В скриптах используйте имена (`kill -TERM`), а не числа: имена переносимы.

`SIGKILL` и `SIGSTOP` нельзя перехватить или проигнорировать — это гарантия ядра. Всё остальное можно. Поэтому «убейте меня аккуратно» — это всегда `SIGTERM`, а `SIGKILL` — крайняя мера, после которой не выполнятся ни `trap`, ни `flush` буферов, ни удаление временных файлов.

`SIGHUP` исторически означал обрыв линии с терминалом. Сейчас его используют демоны как «перечитай конфигурацию» (так делают `nginx`, `haproxy`, `sshd`), а оболочка посылает его фоновым задачам при выходе. `nohup` ровно это и делает — ставит `SIGHUP` в игнор.

`SIGPIPE` — причина кода 141. Он приходит, когда процесс пишет в трубу, читатель которой уже закрылся: `yes | head -1` заставляет `yes` получить `SIGPIPE` после первой строки.

bash

```
$ bash -c 'set -o pipefail; yes | head -1; echo "код=$?
```

```
    PIPESTATUS=${PIPESTATUS[*]}"'
у
код=141 PIPESTATUS=141 0
```

Для многих утилит (grep, head, awk) такой исход штатный, и в скрипте с `pipefail` его нужно либо ожидать, либо не использовать `pipefail` на этом конвейере.

Перенаправления

У процесса три стандартных дескриптора: 0 — stdin, 1 — stdout, 2 — stderr. Перенаправление меняет, куда они указывают, и делает это шелл до запуска программы.

bash

```
cmd > file          # stdout в файл, перезапись
cmd >> file         # stdout в файл, дозапись
cmd 2> err.log      # только stderr
cmd &> all.log      # stdout и stderr (bash; в POSIX — > all.log 2>&1)
cmd < input.txt     # stdin из файла
cmd 2>&1 | less     # stderr в тот же поток, что stdout, и в конвейер
```

Порядок имеет значение. `команда > file 2>&1` и `команда 2>&1 > file` дают разный результат, потому что перенаправления применяются слева направо. В первом случае stdout уходит в файл, затем stderr дублируется в текущий stdout — то есть тоже в файл. Во втором stderr дублируется в текущий stdout (терминал), и только потом stdout меняется на файл:

bash

```
$ ls /nope > a.txt 2>&1; cat a.txt
ls: cannot access '/nope': No such file or directory
$ ls /nope 2>&1 > b.txt
ls: cannot access '/nope': No such file or directory
$ wc -c < b.txt
0
```

Во втором случае сообщение ушло на терминал, а файл пуст. Это самая частая ошибка при написании cron-задач: лог пустой, а письма от cron приходят.

`/dev/null` — устройство, которое принимает всё и возвращает пустоту. `cmd > /dev/null 2>&1` — полностью тихий запуск. `cmd 2>/dev/null` — скрыть только ошибки (опасно: вы теряете диагностику).

Дескрипторы можно открывать вручную, и это полезно в циклах, где нельзя переоткрывать файл на каждой итерации:

bash

```
exec 3> report.txt
for h in web-01 web-02 web-03; do
    printf '%s %s\n' "$h" "$(date +%T)" >&3
done
exec 3>&-
```

`exec 3> file` открывает дескриптор 3 в текущей оболочке, `>&3` пишет в него, `3>&-` закрывает. Без закрытия дескриптор живёт до конца скрипта; при передаче его дочерним процессам это иногда нужно намеренно.

Пайпы и pipefail

Конвейер `a | b` соединяет stdout `a` со stdin `b` через буфер в ядре (по умолчанию 64 КиБ). Обе команды запускаются одновременно, поэтому `tail -f app.log | grep ERROR` работает в реальном времени, а не после завершения `tail`.

Код возврата конвейера по умолчанию — код последней команды. `true | false | true` вернёт 0, хотя середина упала:

bash

```
$ bash -c 'true | false | true; echo "без pipefail: $?"'
без pipefail: 0
$ bash -c 'set -o pipefail; true | false | true; echo "с pipefail: $?
    PIPESTATUS=${PIPESTATUS[*]}"'
с pipefail: 1 PIPESTATUS=0 1 0
```

`PIPESTATUS` — массив кодов всех команд конвейера, доступный сразу после него. Он перезаписывается следующей командой, поэтому читать его нужно немедленно.

`pipefail` — почти всегда правильный выбор для скриптов: иначе `curl ... | jq .` вернёт 0, даже если `curl` не смог соединиться и `jq` разобрал пустоту. Исключения — конвейеры, где ненулевой код штатен: `grep`, `diff`, `head` с `SIGPIPE`.

Подстановка процессов

`<(команда)` создаёт временный файловый дескриптор (в Linux — `/dev/fd/63`) и подставляет его как имя файла. Это позволяет передать вывод команды туда, где ожидается путь:

bash

```
$ diff <(printf 'a\nb\nc\n') <(printf 'a\nc\nd\n')
2d1
< b
3a3
> d
$ while read -r user; do echo "пользователь: $user"; done < <(awk -F: '$3 >= 1000
    {print $1}' /etc/passwd)
```

Второй пример важен: `while read` в конвейере выполняется в подоболочке, и переменные, изменённые внутри, не видны снаружи. Подстановка процессов сохраняет цикл в текущей оболочке. `>(команда)` работает в обратную сторону — пишет в stdin команды: `cmd > >(logger -t app)`.

nohup и почему systemd лучше

`nohup cmd &` игнорирует `SIGHUP`, перенаправляет вывод в `nohup.out` и оставляет процесс жить после закрытия терминала. Этого достаточно, чтобы процесс пережил сессию SSH, но не достаточно, чтобы им управлять: никто не перезапустит его после падения, не ограничит память, не соберёт логи в journal, не остановит при выключении машины и не покажет статус.

`systemd` даёт всё это через unit-файл: перезапуск по `Restart=on-failure`, лимиты через `MemoryMax`, логи в journal автоматически (stdout и stderr процесса), остановка через `systemctl stop` с корректной последовательностью сигналов: `SIGTERM` всей контрольной группе, ожидание `TimeoutStopSec` (по умолчанию 90 секунд), затем `SIGKILL`. Юниты и таймеры разобраны в главе 29; здесь важен принцип: `nohup` — способ отвязаться от терминала, `systemd` — способ управлять жизненным циклом.

Пример: graceful-остановка процесса по SIGTERM

Задача: остановить воркер так, чтобы он успел дописать текущую задачу, а если не успел за отведённое время — добить. Сначала сам воркер с обработчиком:

bash

```
#!/usr/bin/env bash
set -euo pipefail
PIDFILE=${1:?pidfile}
```

```
echo $$ > "$PIDFILE"
stopping=0
on_term() {
    echo "[worker $$] получил SIGTERM, завершаю текущую задачу" >&2
    stopping=1
}
trap on_term TERM INT
while (( ! stopping )); do
    sleep 0.2
done
echo "[worker $$] очистка и выход" >&2
rm -f "$PIDFILE"
exit 0
```

Обработчик не завершает процесс сразу: он выставляет флаг, и цикл выходит сам, когда текущая итерация закончена. Прерывать работу посреди записи файла нельзя — это и есть смысл graceful-останова. `sleep` при этом прерывается сигналом, поэтому реакция быстрая.

Теперь останавливающий скрипт с таймаутом и эскалацией:

bash

```
#!/usr/bin/env bash
set -euo pipefail
PIDFILE=${1:?укажите pid-файл}
TIMEOUT=${2:-10}
pid=$(cat "$PIDFILE")
kill -TERM "$pid"
for ((i = 0; i < TIMEOUT * 10; i++)); do
    if ! kill -0 "$pid" 2>/dev/null; then
        echo "процесс $pid завершился штатно за ~$((i / 10)) с"
        exit 0
    fi
    sleep 0.1
done
echo "процесс $pid не ответил за ${TIMEOUT} с, отправляю SIGKILL" >&2
kill -KILL "$pid" 2>/dev/null || true
exit 1
```

Запуск в фоне, ожидание и остановка дают такой результат:

```
worker pid=541909, ppid=541907
процесс 541909 завершился штатно за ~0 с
stop rc=0
[worker 541909] получил SIGTERM, завершаю текущую задачу
[worker 541909] очистка и выход
```

Разбор деталей. `kill -0` не посылает сигнал, а проверяет, существует ли процесс и есть ли право ему сигналить; это стандартный способ polling-ожидания. `2>/dev/null` скрывает `No such process` после выхода. `|| true` после `kill -KILL` нужен потому, что процесс мог умереть между проверкой и сигналом, а `set -e` на

этом бы остановил скрипт с невнятной ошибкой. `exit 1` сигнализирует вызывающему, что штатного останова не было, — это важно для `systemctl`, который в таком случае зафиксирует проблему.

Типичные ошибки

1. `cmd > file 2>&1` **в неправильном порядке.** Механизм: перенаправления применяются слева направо; `2>&1 > file` отправляет stderr на старый stdout (терминал), а stdout — в файл. Диагностика теряется, лог пуст.
2. `SIGKILL` **как первый выбор.** Механизм: сигнал нельзя перехватить, `trap` не выполнится, временные файлы останутся, буферы не сброшены, соединения не закрыты. Сначала `SIGTERM` и ожидание.
3. `kill %1` **в скрипте.** Механизм: job control в неинтерактивной оболочке выключен, `%1` не разрешается. Адресуйте по PID из `$!` или pid-файла.
4. **Переменные из** `while read` **в конвейере.** Механизм: конвейер создаёт подоболочку, изменения переменных не видны снаружи. Лечится подстановкой процессов `< <(...)` или `mapfile`.
5. **Нет** `pipefail`**.** Механизм: код конвейера — код последней команды, падение середины маскируется. `curl | jq` вернёт 0 при неудачном соединении.
6. `nohup` **вместо супервизора.** Механизм: `nohup` только игнорирует `SIGHUP`; перезапуска, лимитов и ротации логов нет. Для сервисов — systemd unit.
7. **Ожидание** `kill` **без проверки.** Механизм: сигнал асинхронен, после `kill` процесс ещё жив. Нужен polling через `kill -0` с таймаутом.

Чек-лист

- Для остановки используется `SIGTERM`, `SIGKILL` — только после таймаута.
- В скрипте-супервизоре есть ожидание с `kill -0` и ограничением по времени.

- Порядок перенаправлений проверен: `> file 2>&1`, а не наоборот.
- В шапке скрипта есть `set -o pipefail`, а случаи штатного ненулевого кода обработаны явно.
- Фоновые задачи адресуются по `$!`/pid-файлу, результат забирается через `wait`.
- PID-файл создаётся после успешного старта и удаляется в `trap ... EXIT`.
- Для сервисов, которые должны жить после выхода из SSH, используется systemd, а не `nohup`.
- Проверено поведение при `Ctrl-C`: временные файлы удаляются, блокировки снимаются.

Упражнения

1. Напишите скрипт, который запускает три команды параллельно, ждёт все и печатает коды возврата каждой. Убедитесь, что общий код скрипта ненулевой, если упала хотя бы одна.
2. Объясните и проверьте на практике разницу между `cmd > log 2>&1` и `cmd 2>&1 > log`, перенаправив вывод в файл и в терминал. Сделайте таблицу «куда попал stdout / stderr» для четырёх вариантов.
3. Реализуйте цикл, который читает список хостов из команды и накапливает результат в массив, не теряя его после завершения цикла.
4. Модифицируйте пример graceful-останова: добавьте обработку `SIGHUP` как «перечитать конфиг без остановки», обработку повторного `SIGTERM` как немедленный выход и логирование в файл через дополнительный дескриптор.
5. Сравните `nohup ./worker.sh &` и systemd-юнит с `Restart=on-failure`: убейте процесс `SIGKILL` в обоих случаях и опишите, что произойдёт через 5 секунд, какие логи останутся и как узнать причину падения.

Ответы и разбор к этим упражнениям — в приложении D, часть 3.

Глава 29. Практика: скрипты бэкапа, healthcheck, деплоя, systemd timers

Знание синтаксиса не превращается в надёжную автоматизацию само. Разница между учебным скриптом и тем, который можно поставить в cron на боевом сервере, — в проверках, блокировках, таймаутах, кодах возврата и возможности безопасно повторить запуск. В этой главе четыре законченных примера: бэкап с проверкой места и ротацией, healthcheck с алертом и режимом `DRY_RUN`, деплой с идемпотентностью и откатом, запуск по расписанию через systemd timer. Все скрипты рабочие, вывод показан реальный. В конце — чек-лист, по которому можно проверить любой свой скрипт перед тем, как отдать его дежурному.

Скрипт бэкапа: место, атомарность, ротация

Требования к ночному бэкапу формулируются быстро: упаковать каталог, проверить, что места хватает, не оставить битый архив при обрыве, хранить N последних копий, уметь посчитать контрольную сумму. Плохой бэкап опаснее отсутствующего: он создаёт иллюзию защиты.

bash

```
#!/usr/bin/env bash
set -euo pipefail

SRC=${1:?укажите каталог-источник}
DEST=${2:?укажите каталог для архивов}
KEEP=${KEEP:-3}
MIN_FREE_MB=${MIN_FREE_MB:-256}
DRY_RUN=${DRY_RUN:-0}

log() { printf '%s %s\n' "$(date -u +%FT%TZ)" "$*" >&2; }
die() { log "ОШИБКА: $*"; exit 1; }

[[ -d $SRC ]] || die "нет каталога $SRC"
mkdir -p "$DEST"

avail=$(df -Pm "$DEST" | awk 'NR==2 {print $4}')
(( avail >= MIN_FREE_MB )) || die "мало места: ${avail} МБ < ${MIN_FREE_MB} МБ"

ts=$(date -u +%Y%m%dT%H%M%SZ)
```

```
archive="$DEST/backup-$ts.tar.gz"

if (( DRY_RUN )); then
    log "DRY_RUN: создал бы $archive и оставил $KEEP последних"
    exit 0
fi

tmp=$(mktemp "$DEST/.part.XXXXXX")
trap 'rm -f "$tmp"' EXIT
tar -C "$(dirname "$SRC")" -czf "$tmp" "$(basename "$SRC")"
mv "$tmp" "$archive"
trap - EXIT
sha256sum "$archive" > "$archive.sha256"
log "создан $archive, $(du -h "$archive" | cut -f1)"

mapfile -t old < <(ls -1t "$DEST"/backup-*.tar.gz 2>/dev/null | tail -n +$((KEEP +
    1)))
if (( ${#old[@]} )); then
    for f in "${old[@]}"; do
        rm -f "$f" "$f.sha256"
        log "удалён старый архив $f"
    done
fi
```

Прогон с `KEEP=2` и четырьмя запусками подряд:

```
2026-10-29T14:52:08Z создан bk/dest/backup-20261029T145208Z.tar.gz, 4.0K
2026-10-29T14:52:09Z создан bk/dest/backup-20261029T145209Z.tar.gz, 4.0K
2026-10-29T14:52:10Z создан bk/dest/backup-20261029T145210Z.tar.gz, 4.0K
2026-10-29T14:52:10Z удалён старый архив bk/dest/backup-20261029T145208Z.tar.gz
2026-10-29T14:52:11Z создан bk/dest/backup-20261029T145211Z.tar.gz, 4.0K
2026-10-29T14:52:11Z удалён старый архив bk/dest/backup-20261029T145209Z.tar.gz
```

Проверка целостности и отказы:

```
$ sha256sum -c backup-20261029T145211Z.tar.gz.sha256
backup-20261029T145211Z.tar.gz: OK
$ ./backup.sh /nope dest; echo $?
2026-10-29T14:52:12Z ОШИБКА: нет каталога /nope
1
```

Что здесь принципиально. `df -Pm` даёт машиночитаемый вывод в мегабайтах: `-P` отключает перенос строк, из-за которого в длинных именах устройств поле `$4` уезжает на другую строку. Архив пишется во временный файл `.part.XXXXXX` в том же каталоге и переименовывается только после успешного `tar` — `mv` в пределах одной ФС атомарен, поэтому читатель никогда не увидит недописанный `.tar.gz`. `trap ... EXIT` удаляет огрызок при любом сбое, а после успешного `mv` снимается через `trap - EXIT`, чтобы не удалить уже готовый файл. Контрольная сумма лежит рядом: восстановление без проверки — лотерея.

Ротация сделана по количеству, а не по возрасту: `ls -1t` сортирует по времени изменения, `tail -n +N` отбрасывает первые N.

Альтернатива — `find "$DEST" -name 'backup-*.tar.gz' -mtime +7 -delete`, она лучше, когда политика «неделя копий», а не «семь копий». Ограничение текущего варианта: имена с пробелами и переводами строк в `DEST` сломают разбор, поэтому каталог бэкапов должен быть под вашим контролем.

Healthcheck: повторы, коды, DRY_RUN

Healthcheck отвечает на один вопрос: приложение действительно обслуживает запросы. Проверка TCP-порта этого не показывает — процесс может слушать порт и возвращать 500. Поэтому проверяется HTTP-код и содержимое ответа.

bash

```
#!/usr/bin/env bash
set -euo pipefail

URL=${1:?укажите URL проверки}
TIMEOUT=${TIMEOUT:-5}
RETRIES=${RETRIES:-3}
DELAY=${DELAY:-2}
WEBHOOK=${WEBHOOK:-}
DRY_RUN=${DRY_RUN:-0}
EXPECT=${EXPECT:-ok}

alert() {
    local msg=$1
    if (( DRY_RUN )); then
        printf 'DRY_RUN: алерт "%s" не отправлен\n' "$msg" >&2
        return 0
    fi
    if [[ -z $WEBHOOK ]]; then
        printf 'АЛЕРТ: %s\n' "$msg" >&2
        return 0
    fi
    curl -fsS --max-time 5 -H 'Content-Type: application/json' \
        -d "$(printf '{"text":"%s"}' "$msg")" "$WEBHOOK" >/dev/null \
        || printf 'не удалось отправить алерт\n' >&2
}

check() {
    local body
    body=$(curl -fsS --max-time "$TIMEOUT" "$URL") || return 1
    printf '%s' "$body" | jq -e --arg e "$EXPECT" '.status == $e' >/dev/null ||
        return 2
    return 0
}

for ((i = 1; i <= RETRIES; i++)); do
    rc=0
    check || rc=$?
    if ((rc == 0)); then
        echo "OK: $URL отвечает status=$EXPECT (попытка $i)"
```

```
        exit 0
    fi
    echo "попытка $i/$RETRIES неудачна (код $rc)" >&2
    if ((i < RETRIES)); then sleep "$DELAY"; fi
done

alert "healthcheck $URL провален после $RETRIES попыток"
exit 1
```

Реальные прогоны против локального сервера:

```
== успех
OK: http://127.0.0.1:18080/health.json отвечает status=ok (попытка 1)
== неверное поле
попытка 1/2 неудачна (код 2)
попытка 2/2 неудачна (код 2)
DRY_RUN: алерт "healthcheck http://127.0.0.1:18080/health.json провален после 2
    попыток" не отправлен
== сервер выключен
curl: (7) Failed to connect to 127.0.0.1 port 18080 after 0 ms: Could not connect
    to server
попытка 1/2 неудачна (код 1)
```

Два кода различимы не случайно: 1 — сеть или HTTP-ошибка (`curl -f` возвращает 22 на 4xx/5xx), 2 — ответ получен, но контент не тот. Это позволяет в алерте разделить «сервис лежит» и «сервис отвечает неправильно». `rc=0; check || rc=$?` — обязательный приём: запись `if check; then` съедает код, а `check` без обработки убьёт скрипт из-за `set -e` на первой же неудаче, и повторов не будет.

`DRY_RUN` реализован так, что и в «сухом» режиме видно точный текст алерта, но наружу ничего не уходит. Это стандарт для скриптов, которые запускаются по расписанию: сначала прогон вручную с `DRY_RUN=1`, потом включение таймера.

Деплой: идемпотентность и откат

Схема «релизы + симлинк» проста и переживает сбои: каждый деплой создаёт новый каталог, `current` — симлинк на активный, откат — переключение симлинка, отдача статики и запуск приложения идут через `current`. Идемпотентность обеспечивается сравнением хеша содержимого: если развёрнуто ровно то же, что собрано, деплой ничего не делает.

bash

```
#!/usr/bin/env bash
set -euo pipefail

APP_DIR=$(realpath -m "${APP_DIR:-/srv/app}")
KEEP=${KEEP:-3}
```

```
SRC=${SRC:-}
RELEASES="$APP_DIR/releases"
CURRENT="$APP_DIR/current"

log() { printf '[deploy] %s\n' "$*" >&2; }
die() { log "ОШИБКА: $*"; exit 1; }
src_hash() { (cd "$1" && find . -type f -print0 | sort -z | xargs -0 sha256sum |
    sha256sum | cut -d' ' -f1); }

mkdir -p "$RELEASES"
exec 9>"$APP_DIR/.deploy.lock"
flock -n 9 || die "деплой уже выполняется (lock занят)"

rollback() {
    mapfile -t rels < <(find "$RELEASES" -mindepth 1 -maxdepth 1 -type d -printf
        '%f\n' | sort)
    ((${#rels[@]} >= 2)) || die "нечего откатывать: релизов ${#rels[@]}"
    local cur idx=-1 target
    cur=$(basename "$(readlink -f "$CURRENT")")
    for i in "${!rels[@]}"; do
        [[ ${rels[i]} == "$cur" ]] && idx=$i
    done
    ((idx > 0)) || die "нет предыдущего релиза для отката"
    target=${rels[idx - 1]}
    ln -sfn "$RELEASES/$target" "$CURRENT"
    log "откат: $cur -> $target"
}

if [[ ${1:-} == --rollback ]]; then
    rollback
    exit 0
fi

[[ -n $SRC ]] || die "не задан SRC"
[[ -d $SRC ]] || die "нет каталога $SRC"

hash=$(src_hash "$SRC")
if [[ -L $CURRENT ]] && [[ $(cat "$(readlink -f "$CURRENT")/.release-hash"
    2>/dev/null) == "$hash" ]]; then
    log "эта версия уже развёрнута, изменений нет"
    exit 0
fi

rel="$RELEASES/$(date -u +%Y%m%dT%H%M%S.%N)"
mkdir "$rel"
cp -a "$SRC/." "$rel/"
printf '%s\n' "$hash" > "$rel/.release-hash"
log "создан релиз $(basename "$rel")"

ln -sfn "$rel" "$CURRENT"
log "current -> $rel"

if [[ -x $CURRENT/app.sh ]]; then
    "$CURRENT/app.sh" || { rollback; die "проверка после переключения провалена"; }
fi

mapfile -t old < <(find "$RELEASES" -mindepth 1 -maxdepth 1 -type d -printf '%f\n'
    | sort | head -n -"$KEEP")
if ((${#old[@]})); then
    for r in "${old[@]}"; do
```

```
        [[ $CURRENT -ef "$RELEASES/$r" ]] && continue
        rm -rf "${RELEASES:?}/$r"
        log "удалён старый релиз $r"
    done
fi
```

Транскрипт: деплой 1.0.0, повторный запуск без изменений, деплой 1.0.1, откат.

```
[deploy] создан релиз 20261029T145333.760190615
[deploy] current -> /srv/app/releases/20261029T145333.760190615
app 1.0.0 запущен
[deploy] эта версия уже развёрнута, изменений нет
[deploy] создан релиз 20261029T145333.791244330
[deploy] current -> /srv/app/releases/20261029T145333.791244330
app 1.0.1 запущен
[deploy] откат: 20261029T145333.791244330 -> 20261029T145333.760190615
```

Детали, без которых это не работает. `realpath -m` превращает относительный `APP_DIR` в абсолютный до создания симлинков — иначе `ln -sfn` запишет относительный путь, который будет разрешаться от каталога ссылки, и `current` окажется битым. `flock -n 9` на дескрипторе 9 берёт неблокирующую блокировку: два одновременных деплоя из CI и с ноутбука не будут мешать друг другу, второй просто выйдет с ошибкой. Имя релиза содержит наносекунды: секундного разрешения не хватает, два деплоя в одну секунду создают каталоги с одинаковым именем или ломают сортировку, если добавлять случайный суффикс. Проверка после переключения (`$CURRENT/app.sh`) — минимальный smoke-тест: если приложение не стартует, откат автоматический, до того как об этом узнают пользователи. Удаление старых релизов проверяет `[[ $CURRENT -ef ... ]]`: активный релиз не удаляется, даже если вышел за `KEEP`.

systemd unit и timer

Cron умеет запускать по расписанию и ничего больше. systemd timer даёт журнал, зависимости, таймауты, `Persistent=true` (догнать пропущенный запуск после выключения машины) и случайный разброс, чтобы сотня серверов не била по хранилищу одновременно.

Файл `/etc/systemd/system/backup.service`:

```ini
[Unit]
```

```
Description=Nightly backup of /srv/data
After=local-fs.target

[Service]
Type=oneshot
User=backup
Group=backup
Environment=KEEP=7 MIN_FREE_MB=2048
ExecStart=/usr/local/bin/backup.sh /srv/data /var/backups/app
TimeoutStartSec=30min
Nice=10
IOSchedulingClass=idle
StandardOutput=journal
StandardError=journal
```

Файл `/etc/systemd/system/backup.timer`:

```
ini

[Unit]
Description=Run backup daily at 02:30

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true
RandomizedDelaySec=10m
Unit=backup.service

[Install]
WantedBy=timers.target
```

Проверка расписания и включение:

```
$ systemd-analyze calendar '*-*-* 02:30:00'
Normalized form: *-*-* 02:30:00
    Next elapse: Fri 2026-10-30 02:30:00 +05
       (in UTC): Thu 2026-10-29 21:30:00 UTC
       From now: 6h left
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now backup.timer
$ systemctl list-timers backup.timer
NEXT                        LEFT    LAST                        PASSED  UNIT          ACTIVATES
Fri 2026-10-30 02:30:00 +05 6h left n/a                         n/a     backup.timer  backup.service
```

`Type=oneshot` — для задач, которые завершаются: systemd ждёт выхода и записывает код возврата. `TimeoutStartSec=30min` убивает зависший бэкап: без него `tar` на огромном каталоге может висеть сутками и блокировать следующий запуск. `Nice=10` и `IOSchedulingClass=idle` снижают нагрузку на диск в час пик. Логи идут в journal автоматически: `journalctl -u backup.service --since yesterday` покажет и вывод скрипта, и код возврата, а `systemctl status backup.service` — последний результат.

Типичная ошибка при переходе с cron: оставить в скрипте относительные пути. В cron рабочий каталог — домашний каталог пользователя, в systemd — `/` (если не задан

`WorkingDirectory`). Абсолютные пути в `ExecStart` и внутри скрипта обязательны.

Чек-лист

Проверка перед тем, как отдать скрипт дежурному:

- Шапка `set -euo pipefail`, интерпретатор в shebang соответствует синтаксису, все пути абсолютные или вычислены от расположения скрипта.
- Обязательные параметры проверены через `${VAR:?}`, неверные значения дают код 2 и сообщение в stderr.
- Есть `DRY_RUN` для всего, что меняет состояние, и он реально прогонялся.
- Запись атомарна: временный файл в том же каталоге плюс `mv`; при сбое `trap` убирает мусор.
- Повторный запуск безопасен: блокировка (`flock`), проверка уже выполненного действия, отсутствие дублей; путь отката проверен хотя бы один раз вручную.
- Внешние вызовы (`curl`, `kubectl`, `psql`) имеют таймаут и проверку кода возврата.
- Ошибки логируются с временной меткой в stderr, успех — одной строкой; секреты не попадают в лог и в `ps`.
- Скрипт прогнан `shellcheck`, запущен с пустыми и неверными аргументами, проверен под тем пользователем, который будет его выполнять.

Типичные ошибки

1. **Бэкап сразу в целевой файл.** Механизм: при обрыве (OOM, диск, сигнал) остаётся валидный по имени, но битый архив, а ротация считает его успешным. Лечится записью в `.part` и `mv`.
2. **Healthcheck без таймаута.** Механизм: `curl` без `--max-time` висит до TCP-таймаута (минуты), проверка не успевает отработать за интервал запуска и накладывается на следующую. Таймаут обязателен.

3. `if check; then` **и потерянный код.** Механизм: `if` возвращает 0, если выполнилась ветка, а `check` под `set -e` в условии не завершает скрипт — повторов не будет, диагностика теряется. Нужен `check || rc=$?`.
4. **Относительный путь в** `ln -sfn`**.** Механизм: цель симлинка разрешается относительно каталога ссылки, а не текущего каталога. `realpath` до создания ссылки.
5. **Имя релиза с секундным разрешением.** Механизм: два деплоя в одну секунду создают одинаковое имя (`mkdir: File exists`) или ломают порядок сортировки при откате. Нужны наносекунды или монотонный счётчик.
6. **Деплой без блокировки.** Механизм: два параллельных запуска создают релизы одновременно, `current` переключается дважды, старые релизы удаляются под работающим процессом. `flock` на файле блокировки.
7. `Persistent=true` **без идемпотентности.** Механизм: после простоя systemd догоняет пропущенный запуск, а следом идёт штатный; скрипт выполняется дважды подряд. Второй запуск должен быть безвредным.

Упражнения

1. Добавьте в скрипт бэкапа шифрование (`age` или `gpg`) и проверку, что ключ доступен до начала архивации. Ошибка должна приводить к коду 3 и не создавать файлов.
2. Переделайте healthcheck так, чтобы он проверял несколько эндпоинтов из файла конфигурации, суммировал результаты и отправлял один алерт со списком упавших.
3. Реализуйте в деплое «канареечную» проверку: после переключения `current` отправьте 10 запросов на новый релиз напрямую по порту, и откатитесь, если доля ошибок выше 10%.
4. Напишите пару unit + timer для healthcheck: запуск каждые 5 минут, `AccuracySec=30s`, алерт в journal при провале. Проверьте `systemd-analyze verify` и `systemctl list-timers`.

5. Возьмите любой свой скрипт и прогоните его по чек-листу из этой главы. Для каждого непройденного пункта опишите конкретный сценарий отказа, который он допускает, и исправьте.

Ответы и разбор к этим упражнениям — в приложении D, часть 3.

Глава 30. Go: начало

Go появился в 2009 году как ответ на боль внутренних сервисов Google: медленная сборка C++, сложность управления памятью и потоками, отсутствие единого способа ставить зависимости. Для DevOps-инженера Go ценен тремя свойствами: он компилируется в один статический бинарник, который запускается на любом Linux без интерпретатора и зависимостей; он даёт конкурентность на уровне языка, а не библиотек; и его инструментальная цепочка едина — форматирование, тесты, сборка и проверка кода встроены в один бинарник `go`. В этой главе мы установим Go, разберёмся с модулями и типичной структурой проекта, соберём минимальный HTTP-сервер и научимся собирать бинарник под другую архитектуру. Синтаксис подробно разбирается в следующей главе, поэтому здесь мы смотрим на рабочий процесс, а не на язык.

Зачем Go в инфраструктуре

Типичная задача DevOps-инженера, для которой Go подходит лучше Python: небольшой контроллер, экспортёр метрик, вебхук для приёма алертов, sidecar-прокси, CLI для внутреннего API. У Python в такой роли два слабых места. Первое — поставка: нужен интерпретатор нужной версии, виртуальное окружение, зависимости, и всё это надо доставить в контейнер или на хост. Второе — старт и потребление памяти: интерпретатор поднимается десятки миллисекунд и занимает десятки мегабайт до первой строки полезной работы.

Go решает обе проблемы одним движением. `go build` создаёт один файл, внутри которого лежат и ваш код, и рантайм Go (сборщик мусора, планировщик горутин, сетевой поллинг). Файл не требует ни libc (если отключить cgo), ни переменных окружения, ни установленных пакетов. Его можно положить в `scratch`-образ Docker (пустой, без оболочки) или в `FROM gcr.io/distroless/static`, и он заработает.

Свойство	Python	Go
Артефакт поставки	исходники + интерпретатор + venv	один бинарник
Старт процесса	десятки мс	единицы мс
Базовая память	десятки МБ	единицы–десятки МБ
Конкурентность	потоки ОС / asyncio	горутины, тысячи на процесс
Сборка	нет этапа сборки	`go build`, секунды

Цифры в таблице — порядки величин, а не бенчмарк. Важна разница в модели: у Go нет отдельного этапа «подготовить окружение» — окружение упаковано в бинарник.

Установка и проверка

Дистрибутивные пакеты отстают от релизов. В Debian и Ubuntu репозиторный `golang-go` обычно на несколько минорных версий позже текущей, а вместе с языком отстают и исправления безопасности в стандартной библиотеке. Для рабочих машин и CI правильнее ставить официальный архив с go.dev.

bash

```
curl -LO https://go.dev/dl/go1.26.0.linux-amd64.tar.gz
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.26.0.linux-amd64.tar.gz
export PATH=$PATH:/usr/local/go/bin
go version
```

Вывод последней команды на момент написания — `go version go1.26.0 linux/amd64`. Актуальная ветка Go — 1.26 (1.26.0 вышла 10 февраля 2026 года, последний патч 1.26.10 — 19 октября 2026); предыдущая — 1.25. Поддерживаются две последние минорные ветки, поэтому обновляться стоит каждый релиз, а не раз в несколько лет: как только ветка выпадает из поддержки, патчи безопасности для неё не выходят.

Строку `export PATH=...` нужно добавить в `~/.profile` или `~/.bashrc`, иначе после перезапуска оболочки `go` исчезнет. В CI то же самое делают в шаге установки или берут готовый образ `golang:1.26`.

go env: где Go хранит своё

`go env` печатает конфигурацию тулчейна. Полный вывод длинный, полезно смотреть выборочно:

bash

```
go env GOROOT GOPATH GOMODCACHE GOCACHE GOPROXY CGO_ENABLED
```

```
/usr/local/go
/home/user/go
/home/user/go/pkg/mod
/home/user/.cache/go-build
https://proxy.golang.org,direct
1
```

`GOROOT` — каталог с самим тулчейном и стандартной библиотекой; его не надо трогать. `GOPATH` — рабочая область пользователя, по умолчанию `~/go`; внутри `bin` (куда `go install` кладёт бинарники), `pkg/mod` (кэш скачанных модулей) и `src` (историческое место, в модульном режиме не используется). `GOCACHE` — кэш сборки: объектные файлы и результаты тестов. Именно он делает повторную сборку почти мгновенной, и именно его теряют, когда в CI не настраивают кэш между запусками. `GOPROXY` — откуда брать модули; `direct` в конце означает «если прокси не ответил, иди в git напрямую».

`CGO_ENABLED=1` разрешает вызовы C-кода. Для статических бинарников его выключают.

Модули

До версии 1.11 зависимости искались по путям внутри `GOPATH`, и проект нельзя было собрать вне этой структуры. Модуль (module) заменил её: теперь у проекта есть явный список зависимостей и версий, а расположение на диске не важно. Модуль — это набор пакетов с общим префиксом импорта и файлом `go.mod` в корне.

bash

```
mkdir -p ~/src/metrics-exporter && cd ~/src/metrics-exporter
go mod init example.com/platform/metrics-exporter
go get github.com/prometheus/client_golang@v1.20.5
go mod tidy
```

После этого в каталоге два файла. `go.mod` описывает модуль (сокращённо):

```
module example.com/platform/metrics-exporter

go 1.26.0
```

```
require github.com/prometheus/client_golang v1.20.5
```

Первая строка — путь модуля. Для внутреннего проекта он может быть любым, лишь бы уникальным: `example.com/platform/metrics-exporter` или `git.company.internal/team/exporter`. Для публикуемого в открытый доступ модуля путь обязан совпадать с URL репозитория, иначе `go get` у пользователей не найдёт код. Строка `go 1.26.0` фиксирует минимальную версию языка, которую требует код; тулчейн новее её соберёт, старее — откажется с внятной ошибкой. Начиная с Go 1.21 в этой строке допустима полная версия с патчем, а не только `1.26`.

Файл `go.sum` — контрольные суммы скачанных модулей. Он защищает от подмены кода в кэше или на прокси: если хеш не совпал, сборка остановится. `go.sum` коммитят в репозиторий вместе с `go.mod`, иначе воспроизводимость теряется.

Ключевая команда повседневной работы — `go mod tidy`. Она проходит по импортам, добавляет отсутствующие зависимости, удаляет неиспользуемые и обновляет `go.sum`. Запускать её нужно после каждого изменения импортов, а в CI — проверять, что она не даёт диффа: это ловит ситуацию «на машине разработчика работает, потому что в кэше лежит пакет, которого нет в `go.mod`».

Версии Go выбирает по принципу минимальной достаточной версии (minimal version selection, MVS): если ваши зависимости требуют `v1.4.0` и `v1.2.0`, будет выбрана `v1.4.0` — наибольшая из минимально необходимых, а не последняя выпущенная. Это делает сборки предсказуемыми: обновление происходит только тогда, когда вы явно попросили. `go get package@latest` — явная просьба, и она меняет `go.mod`; `go get` без версии в современном Go ведёт себя как `@latest` для указанного пакета, поэтому в скриптах версию лучше писать всегда.

Структура проекта

Формально Go не навязывает структуру каталогов: пакеты — это каталоги, и всё. Но несколько соглашений сложились де-факто и поддерживаются инструментами.

```
metrics-exporter/
```

```
├── go.mod
├── go.sum
├── cmd/
│   └── exporter/
│       └── main.go
├── internal/
│   ├── collect/
│   │   └── collect.go
│   └── httpserver/
│       └── server.go
└── pkg/
    └── client/
        └── client.go
```

`cmd/<имя>/main.go` — точка входа. Если бинарников несколько (`cmd/exporter`, `cmd/migrate`), у каждого свой каталог и свой `package main`. Это позволяет одному репозиторию собирать несколько программ: `go build ./cmd/exporter`.

`internal/` — пакеты, которые нельзя импортировать снаружи модуля. Это не соглашение, а правило компилятора: если пакет лежит в `internal`, импорт из другого модуля не соберётся с ошибкой `use of internal package not allowed`. Всё, что не является публичным API, стоит держать здесь — тогда рефакторинг внутренностей не ломает чужие сборки.

`pkg/` — необязательный каталог для кода, который предполагается переиспользовать внешними модулями. Многие команды его не заводят вовсе: пока нет внешних потребителей, `internal` честнее. Пустой `pkg/` только вводит в заблуждение.

Тесты лежат рядом с кодом: `collect.go` и `collect_test.go` в одном каталоге и одном пакете (`package collect`, не `collect_test`, если нужен доступ к неэкспортируемым функциям). Отдельного каталога `tests/` в идиоматичном Go нет.

Рабочий цикл: run, build, test, vet, fmt

`go run` компилирует во временный каталог и сразу запускает — удобно для проверки, но каждый запуск это сборка.

bash

```
go run ./cmd/exporter -listen=:9101
```

`go build` создаёт бинарник в текущем каталоге (или в `$GOBIN` при `go install`). Флаг `-o` задаёт имя, `-trimpath` убирает локальные пути из

бинарника — полезно для воспроизводимых сборок и чтобы не светить структуру домашних каталогов.

bash

```
go build -trimpath -ldflags="-s -w" -o bin/exporter ./cmd/exporter
```

`-ldflags="-s -w"` убирает таблицу символов и отладочную информацию: на простом сервисе бинарник уменьшается примерно с 8,4 до 5,4 МБ (треть), но перестаёт работать `delve`.

`go test ./...` запускает тесты во всех пакетах модуля, `go vet ./...` — статический анализ на подозрительный код: забытый `Printf` с неверным числом аргументов, потерянный `cancel` у контекста, некорректное копирование мьютекса, недостижимый код. `go vet` входит в стандартную поставку и не требует установки; в CI его гоняют до тестов, потому что он быстрее.

`gofmt` — форматтер: `gofmt -l .` печатает файлы, отличающиеся от канонического формата, `gofmt -w .` переписывает их. Отступы — табы, выравнивание — пробелы. В CI `gofmt -l` ставят отдельным шагом: непустой список валит пайплайн.

Минимальный HTTP-сервер

Пятнадцать строк, которые уже работают, но по таймаутам некорректны — об этом в главе 33; здесь важен каркас.

go

```
package main

import (
    "fmt"
    "log"
    "net/http"
)

func main() {
    http.HandleFunc("/healthz", func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "text/plain; charset=utf-8")
        fmt.Fprintln(w, "ok")
    })

    log.Println("listening on :8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Разбор по строкам. `package main` и функция `main` — обязательная пара для исполняемой программы; пакет с другим именем

компилируется в библиотеку. `http.HandleFunc` регистрирует обработчик в глобальном мультиплексоре (`http.DefaultServeMux`) по пути `/healthz`. Обработчик — функция с сигнатурой `func(http.ResponseWriter, *http.Request)`: первый аргумент пишет ответ, второй содержит разобранный запрос. `w.Header().Set` добавляет заголовок до первого `Write` — после записи тела заголовки уже отправлены, и попытка их поменять молча игнорируется. `fmt.Fprintln(w, "ok")` пишет тело и неявно ставит статус 200, если он не был задан раньше. `http.ListenAndServe(":8080", nil)` слушает все интерфейсы на порту 8080; `nil` означает «взять `DefaultServeMux`».

Важная деталь: `log.Fatal` завершает процесс с кодом 1, если сервер не смог запуститься (порт занят, нет прав). Без него программа молча завершится, и в systemd вы увидите только `status=0/SUCCESS` — классическая ловушка при отладке юнита.

Проверить:

bash

```
go run . &
curl -sS -i http://127.0.0.1:8080/healthz
```

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Date: Thu, 29 Oct 2026 14:12:03 GMT
Content-Length: 3

ok
```

Обратите внимание на `Content-Length: 3`: Go буферизует небольшие ответы и вычисляет длину сам, если вы не вызвали `Flush`.

Кросс-компиляция

Компилятор Go умеет собирать под другую платформу без дополнительных тулчейнов: достаточно переменных `GOOS` и `GOARCH`. Сборка под Linux ARM64 с машины на amd64 — одна команда.

bash

```
CGO_ENABLED=0 GOOS=linux GOARCH=arm64 \
  go build -trimpath -ldflags="-s -w" -o bin/exporter-linux-arm64 ./cmd/exporter
file bin/exporter-linux-arm64
```

```
bin/exporter-linux-arm64: ELF 64-bit LSB executable, ARM aarch64, statically
    linked, Go BuildID=..., stripped
```

Слова `statically linked` — результат `CGO_ENABLED=0`. Если cgo включён и код использует, например, системный DNS через glibc, бинарник окажется динамически слинкованным и потребует совместимую версию libc в контейнере. Поэтому в Docker-сборках для Go почти всегда ставят `CGO_ENABLED=0`; следствие — DNS резолвится встроенным резолвером Go, который читает `/etc/resolv.conf` сам, и это поведение отличается от системного в редких случаях (например, при `nsswitch.conf` с нестандартными источниками).

Список поддерживаемых пар — `go tool dist list`. Полноценная сборка и поставка разбираются в главе 36.

Типичные ошибки

Зависимость от локального пути вместо модуля. Импорт вида `import "metrics-exporter/internal/collect"` не работает: Go ищет пакет по пути модуля из `go.mod`, а не по каталогу на диске. Правильно — `example.com/platform/metrics-exporter/internal/collect`, префикс модуля обязателен. Ошибка возникает у тех, кто пришёл из Python, где пакет ищется относительно текущего файла.

`internal` **за пределами модуля.** Пакет в `internal/` импортируется только кодом внутри родительского каталога модуля. Попытка импортировать его из соседнего репозитория даёт ошибку компиляции, а не предупреждение. Это защита, а не баг: не переносите `internal` в `pkg`, если не готовы поддерживать API.

`go.sum` **в** `.gitignore`**.** Без него сборка на чистой машине либо падает на проверке контрольных сумм, либо тянет то, что оказалось в кэше. `go.sum` — часть исходников, как `package-lock.json` или `poetry.lock`.

`go get` **без версии в скрипте.** `go get github.com/x/y` обновляет зависимость до последней версии, и сборка, которая вчера работала, завтра может не собраться из-за несовместимого минорного релиза. Версию фиксируют в `go.mod` и меняют осознанно, отдельным коммитом.

Сборка без `-trimpath` **в CI.** В бинарник попадают абсолютные пути сборочной машины (`/home/runner/work/...`), из-за чего два

бинарника из одного коммита имеют разные хеши. Это ломает проверку воспроизводимости и подпись артефактов.

`go build` **без указания пакета.** Команда собирает пакет текущего каталога: в корне с несколькими `cmd/` и без собственных `.go`-файлов она падает с `no Go files in ...`. Явно указывайте `./cmd/<имя>`.

Игнорирование `go vet`**.** `fmt.Printf("%d", name)` с `name string` компилируется, но печатает `%!d(string=...)`. `go vet` ловит это до запуска; без него ошибка доживает до продакшна и проявляется в логах.

Чек-лист

- [ ] `go version` показывает поддерживаемую ветку (1.26 или 1.25), а не репозиторную сборку двухлетней давности.
- [ ] В корне проекта есть `go.mod`, в нём осмысленный путь модуля, а `go.sum` закоммичен.
- [ ] `go mod tidy` не даёт диффа на чистом клоне.
- [ ] Точки входа лежат в `cmd/<имя>`, приватный код — в `internal/`.
- [ ] `gofmt -l .` возвращает пустой список.
- [ ] `go vet ./...` и `go test ./...` проходят локально и в CI.
- [ ] Релизная сборка идёт с `-trimpath` и `CGO_ENABLED=0`, бинарник проверен через `file` на `statically linked`.
- [ ] `go build` запускается из чистого клона без сети (при заполненном кэше модулей) — проверка воспроизводимости.

Упражнения

1. Создайте модуль `example.com/lab/hello`, добавьте `cmd/hello/main.go`, который печатает версию Go из `runtime.Version()` и количество CPU из `runtime.NumCPU()`. Соберите его с `-trimpath -ldflags="-s -w"` и сравните размер с обычной сборкой.
2. Добавьте в проект пакет `internal/greet` с функцией `Hello(name string) string` и покройте его тестом в том же каталоге. Убедитесь, что `go test ./...` находит тест, а импорт пакета из другого модуля невозможен.

3. Расширьте HTTP-сервер: пусть `/healthz` отвечает `ok`, а `/version` возвращает JSON с версией Go и временем сборки, переданным через `-ldflags "-X main.buildTime=..."`. Проверьте оба эндпоинта через `curl`.
4. Соберите тот же сервер под `linux/arm64` и `windows/amd64`, проверьте результаты командой `file`. Объясните, почему для `windows` расширение `.exe` добавляется автоматически.
5. Настройте в проекте шаг CI, который падает, если `gofmt -l .` непуст или `go mod tidy` изменяет файлы. Опишите, какие ещё проверки стоит добавить в этот же шаг и почему именно в таком порядке.

Ответы и разбор к этим упражнениям — в приложении D, часть 3.

Глава 31. Синтаксис: срезы, мапы, указатели, интерфейсы, ошибки

Go намеренно маленький язык: в нём 25 ключевых слов, нет наследования, нет исключений, нет перегрузки операторов. Эта минимальность — не бедность, а решение: код на Go читается одинаково у разных команд, потому что способов выразить одну мысль мало. Но несколько механизмов устроены не так, как ожидает человек из Python или Java, и именно на них ломаются новички: срез — это не массив, мапа обходится в случайном порядке, интерфейс может быть не nil при nil-значении внутри, а ошибка — обычное значение, которое надо возвращать, а не бросать. В этой главе мы разберём эти механизмы на уровне модели памяти, чтобы поведение программ перестало быть сюрпризом. Все примеры проверены компилятором Go 1.26: добавив `package main` и импорты, любой можно запустить через `go run`.

Массивы и срезы

Массив в Go — значение фиксированной длины, известной на этапе компиляции. `[4]int` и `[8]int` — разные типы, и присваивание копирует все элементы целиком.

go

```
var a [4]int
b := a          // полная копия, 32 байта
b[0] = 7
fmt.Println(a[0], b[0]) // 0 7
```

Срез (slice) — это дескриптор массива, лежащего в куче или на стеке. Он состоит из трёх машинных слов: указатель на первый элемент, длина (len) и ёмкость (cap). На 64-битной платформе — 24 байта независимо от количества элементов. Копирование среза копирует дескриптор, а не данные: обе переменные смотрят в один и тот же массив.

go

```
s := []int{1, 2, 3, 4}
t := s[1:3]      // len=2, cap=3, указывает на s[1]
t[0] = 99
fmt.Println(s)   // [1 99 3 4] — изменился исходный срез
```

`len(t)` равен 2, а `cap(t)` — 3, потому что от индекса 1 до конца исходного массива остаётся три элемента. Отсюда правило: `cap` среза после `s[i:j]` уменьшается на `i`, но не на `j-i`. Понимание этого объясняет большинство «магических» багов с неожиданно изменившимися данными.

append и переаллокация

`append` добавляет элементы в конец. Если в ёмкости есть место, он пишет в тот же массив и возвращает новый дескриптор с увеличенной длиной. Если места нет — выделяет новый массив, копирует туда старые элементы и возвращает дескриптор на него.

go

```
func main() {
    s := make([]int, 0, 2)
    s = append(s, 1, 2)
    fmt.Printf("len=%d cap=%d %p\n", len(s), cap(s), s) // len=2 cap=2 0xc000...
    s = append(s, 3)
    fmt.Printf("len=%d cap=%d %p\n", len(s), cap(s), s) // len=3 cap=4 0xc000...
        (новый адрес)
}
```

При росте (`runtime.growslice`) ёмкость увеличивается примерно вдвое для небольших срезов и примерно на четверть для больших. Это амортизирует стоимость копирования: серия из N добавлений даёт O(N) работы, а не $O(N^2)$. Практический вывод: если заранее известно количество элементов, `make([]T, 0, n)` избавит от нескольких копирований — на миллионе элементов это заметно.

Ловушка, которая следует из общей памяти: `append` в срез, полученный через `s[:2]`, может перезаписать элемент `s[2]`, если ёмкости хватило.

go

```
s := []int{1, 2, 3}
t := s[:2]
t = append(t, 10)  // ёмкости хватило — пишем в s[2]
fmt.Println(s, t)  // [1 2 10] [1 2 10]
```

Если такая запись нежелательна, срез копируют явно: `t := slices.Clone(s[:2])` или `append([]int(nil), s[:2]...)`. Пакет `slices` (стандартная библиотека с Go 1.21) содержит `Clone`, `Contains`, `Index`, `Sort`, `Equal`, `Delete`, `Insert` и избавляет от ручных циклов.

Мапы

Мапа (map) — хеш-таблица. Объявление `map[string]int` создаёт nil-мапу: читать из неё можно (вернётся нулевое значение), писать — паника `assignment to entry in nil map`. Создавать нужно через `make` или литерал.

```go
m := map[string]int{"a": 1}
m["b"] = 2
delete(m, "a")

v, ok := m["missing"]   // v==0, ok==false
_, exists := m["b"]     // проверка наличия без значения
```

Идиома `v, ok := m[k]` — единственный способ отличить отсутствующий ключ от ключа с нулевым значением. Для `map[string]struct{}` (множество) значение не нужно вовсе: `m[k] = struct{}{}` занимает ноль байт.

Порядок обхода мапы не определён и намеренно рандомизирован в рантайме. Две итерации по одной и той же неизменённой мапе дадут разный порядок.

```go
for k, v := range m {
    fmt.Println(k, v)
}
```

Это сделано специально, чтобы код не опирался на порядок, который может измениться при росте мапы и между версиями рантайма. Если нужен детерминированный обход — сортируйте ключи: `slices.Sorted(maps.Keys(m))` возвращает отсортированный срез (итераторы `maps.Keys` появились в Go 1.23). Кстати, `fmt.Println` мапы печатает ключи отсортированными — это особенность форматирования, а не свойство мапы, и полагаться на неё в логике нельзя.

Мапа — ссылочный тип: копия переменной указывает на ту же таблицу. И мапы не безопасны для одновременной записи из разных горутин: параллельная запись даёт фатальную ошибку `concurrent map writes`, которую нельзя перехватить через `recover`. Для конкурентного доступа нужен `sync.Mutex` или `sync.Map` (глава 32).

Указатели

Указатель хранит адрес значения. `&x` берёт адрес, `*p` разыменовывает. В Go нет арифметики указателей: `p++`, `p+1`, приведение произвольного числа к указателю запрещены на уровне языка. Это убирает целый класс ошибок работы с памятью, который есть в C, и позволяет сборщику мусора точно знать, какие адреса живые.

```go
type Counter struct{ n int }

func (c *Counter) Inc()       { c.n++ }      // меняет оригинал
func (c *Counter) Value() int { return c.n }  // тот же получатель

func main() {
    c := Counter{}
    c.Inc()
    fmt.Println(c.Value()) // 1
}
```

Указатель нужен в трёх случаях: чтобы метод менял получателя; чтобы не копировать большую структуру (условно — больше нескольких десятков байт); чтобы отличать «значение отсутствует» от нулевого значения (`*int` со значением nil). Во всех остальных случаях значение проще и безопаснее.

`new(T)` возвращает `*T`, указывающий на нулевое значение; на практике чаще пишут `&T{}`. Разыменование nil-указателя — паника, и это нормально: программа, которая дошла до nil там, где ожидалось значение, уже сломана, и падение с трассировкой лучше, чем тихая порча данных.

Структуры и методы

Структура — набор полей. Метод — функция с получателем (receiver), объявленная в том же пакете, что и тип.

Ключевое решение при проектировании — value или pointer receiver. Правила простые: если хотя бы один метод меняет состояние, все методы типа делают pointer receiver; если тип содержит `sync.Mutex` или другой некопируемый примитив, — тоже pointer; если тип маленький и неизменяемый (`time.Time`, `url.URL`), — value. Смешивать нельзя: методы с разными получателями попадают в разные наборы методов, и тип может перестать удовлетворять интерфейсу.

```go
type Buffer struct {
    mu   sync.Mutex
    data []byte
}

func (b *Buffer) Write(p []byte) (int, error) {
    b.mu.Lock()
    defer b.mu.Unlock()
    b.data = append(b.data, p...)
    return len(p), nil
}
```

Встраивание (embedding) заменяет наследование: поле без имени продвигает методы встроенного типа наружу.

```go
type Logger struct{ prefix string }

func (l Logger) Log(msg string) { fmt.Println(l.prefix, msg) }

type Service struct {
    Logger       // встроено: s.Log("...") работает
    name   string
}
```

Это композиция, а не подтипизация: `Service` не является `Logger` в смысле присваивания, но получает его методы. Переопределение делается объявлением метода с тем же именем у внешнего типа.

Интерфейсы

Интерфейс — набор сигнатур методов. Реализация неявная: тип удовлетворяет интерфейсу, если у него есть все эти методы, и объявлять `implements` не нужно. Отсюда принцип «интерфейсы объявляет потребитель»: пакет, которому нужно писать в поток, объявляет у себя `type writer interface{ Write([]byte) (int, error) }` — и

любой тип с методом `Write` подходит, включая `os.File`, `bytes.Buffer` и `net.Conn`.

Стандартная библиотека построена на маленьких интерфейсах: `io.Writer` (один метод), `io.Reader`, `fmt.Stringer`, `error`. Маленький интерфейс легко реализовать и легко подменить в тесте — в этом главная практическая ценность.

Значение интерфейса — пара (динамический тип, значение). Пустой интерфейс `any` (алиас `interface{}` с Go 1.18) хранит что угодно. Отсюда знаменитая ловушка:

```
go

type MyErr struct{}
func (e *MyErr) Error() string { return "my error" }

func do() error {
    var e *MyErr = nil
    return e            // вернули интерфейс с типом *MyErr и nil-значением
}

func main() {
    err := do()
    fmt.Println(err == nil) // false!
}
```

Интерфейс равен nil только если nil и тип, и значение. Здесь тип — `*MyErr`, поэтому `err != nil`, и проверка `if err != nil` сработает, хотя внутри пусто. Лечится тем, что функция возвращает `nil` явно, а не nil-указатель конкретного типа.

Приведение типа: `v, ok := x.(io.Writer)` — безопасная форма, `v := x.(io.Writer)` паникует при несоответствии. Множественное приведение удобно делать через `switch v := x.(type)`.

Ошибки как значения

`error` — встроенный интерфейс с единственным методом `Error() string`. Ошибки возвращаются последним значением и проверяются сразу — исключений в языке нет.

```
go

var ErrNotFound = errors.New("not found")

func find(id string) (*Item, error) {
    it, ok := db[id]
    if !ok {
        return nil, fmt.Errorf("find %q: %w", id, ErrNotFound)
```

```
    }
    return &it, nil
}
```

`fmt.Errorf` с глаголом `%w` оборачивает ошибку, сохраняя цепочку. `errors.Is(err, ErrNotFound)` проходит по цепочке и сравнивает с sentinel-ошибкой, `errors.As(err, &target)` ищет в цепочке ошибку конкретного типа и записывает её в target. С Go 1.20 `errors.Join` объединяет несколько ошибок в одну, а `errors.Is/As` умеют обходить и такое дерево.

Разделение ответственности: низкоуровневый код возвращает ошибку с контекстом и обёрткой, вызывающий решает, что с ней делать — повторить, вернуть 500 или 404. Логировать нужно один раз, на верхнем уровне; логирование и возврат одной и той же ошибки на каждом слое даёт тройные записи об одном сбое.

go

```
it, err := find("42")
switch {
case errors.Is(err, ErrNotFound):
    http.Error(w, "not found", http.StatusNotFound)
case err != nil:
    log.Printf("find: %v", err)
    http.Error(w, "internal error", http.StatusInternalServerError)
}
```

Такой `switch` — типичный обработчик на границе сервиса: наружу отдаётся минимальная информация, внутрь лога — полная цепочка.

defer, panic, recover

`defer` откладывает вызов до выхода из функции. Отложенные вызовы выполняются в обратном порядке (LIFO), как стек. Аргументы вычисляются в момент `defer`, а не в момент выполнения:

go

```
func f() {
    i := 1
    defer fmt.Println("deferred:", i) // запомнит 1
    i = 2
    fmt.Println("now:", i)            // 2
}
// now: 2
// deferred: 1
```

Отсюда правило: `defer f.Close()` безопасен, а `defer os.Remove(name)` с переменной, которая потом меняется, — нет. `defer` внутри цикла копит отложенные вызовы до конца функции; для файлов в цикле нужен либо явный `Close()`, либо функция-обёртка на итерацию. После open-coded defers (Go 1.14) накладные расходы `defer` почти нулевые — экономить на нём не нужно.

`panic` разворачивает стек и выполняет отложенные вызовы. `recover` в отложенной функции останавливает панику и возвращает её значение. Применять `recover` стоит в двух местах: на границе процесса (HTTP-обработчик, воркер очереди, gRPC-интерцептор), чтобы одна паника не убила весь сервис, и в тестовых хелперах. Превращать панику в ошибку внутри бизнес-логики не нужно: если функция может не справиться — она возвращает `error`.

```go
func safeHandler(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if v := recover(); v != nil {
                log.Printf("panic: %v\n%s", v, debug.Stack())
                http.Error(w, "internal error", http.StatusInternalServerError)
            }
        }()
        next.ServeHTTP(w, r)
    })
}
```

Важно: `recover` работает только при непосредственном вызове из отложенной функции. `defer func() { helper() }()`, где `helper` вызывает `recover`, панику не остановит.

Типичные ошибки

Ожидание, что срез — это копия. `t := s[1:3]` даёт дескриптор на ту же память; изменение `t[0]` меняет `s[1]`. Механизм — общий массив и разные заголовки. Копия делается `slices.Clone` или `copy(dst, src)`.

append в под-срез портит исходные данные. Если у под-среза осталась ёмкость, `append` пишет в общий массив за пределами его длины. Механизм тот же: `cap` считается от начала под-среза до конца буфера. Лечится трёхиндексным срезом `s[i:j:k]`, который ограничивает ёмкость, или клонированием.

Порядок обхода мапы как контракт. Код, который печатает элементы мапы и надеется на стабильный вывод, ломается при добавлении ключа или смене версии. Рантайм рандомизирует стартовую позицию итерации намеренно. Нужен порядок — сортируйте ключи.

Запись в nil-мапу. `var m map[string]int; m["a"] = 1` — паника, потому что таблица не выделена. Чтение при этом работает и возвращает нулевое значение, из-за чего ошибка обнаруживается позже, чем хотелось бы.

nil-указатель внутри интерфейса. Функция возвращает `error`, внутри `var e *MyErr`, и `err != nil` истинно при пустом значении. Механизм — интерфейс хранит пару (тип, значение); nil-интерфейс требует nil в обоих полях.

Смешанные получатели. Часть методов объявлена на `T`, часть на `*T`. В набор методов `T` попадают только value-методы, поэтому `T` перестаёт удовлетворять интерфейсу, а `*T` — удовлетворяет. Компилятор скажет об этом только в месте присваивания интерфейсу, что затрудняет диагностику.

Потеря обёртки при возврате ошибки. `return err` без `%w` разрывает цепочку, и `errors.Is` наверху не находит sentinel. Обёртка должна быть на каждом слое, где добавляется контекст.

Чек-лист

- [ ] Перед `append` в срез, полученный нарезанием, вы понимаете, есть ли у него запас ёмкости.
- [ ] Для известного заранее количества элементов используется `make([]T, 0, n)`.
- [ ] Ни одна проверка не зависит от порядка обхода мапы.
- [ ] Все методы типа имеют одинаковый вид получателя.
- [ ] Функции, возвращающие `error`, возвращают литерал `nil`, а не nil-указатель конкретного типа.
- [ ] Ошибки оборачиваются через `%w`, а проверяются через `errors.Is/As`, а не сравнением строк.

- [ ] `defer` внутри циклов либо отсутствует, либо вынесен в отдельную функцию.
- [ ] `recover` есть только на границах (HTTP-обработчик, воркер), не в бизнес-логике.

Упражнения

1. Напишите функцию `dedup([]string) []string`, которая удаляет дубликаты, сохраняя порядок первого вхождения, и не мутирует входной срез. Проверьте тестом, что исходный срез не изменился.
2. Реализуйте тип `Matrix` с методами `At(row, col int) float64`, `Set(row, col int, v float64)` и `Transposed() *Matrix`. Решите, где нужны pointer-получатели, и объясните выбор для каждого метода.
3. Объявите свой интерфейс `Store` с методами `Get(string) ([]byte, error)` и `Put(string, []byte) error`. Реализуйте его поверх мапы и поверх временного каталога, затем напишите функцию, работающую с любым `Store`.
4. Создайте иерархию ошибок: `ErrNotFound`, `ErrConflict` и тип `ValidationError` с полем `Field`. Напишите функцию, возвращающую разные ошибки, и обработчик, который через `errors.Is/As` отдаёт 404, 409 и 422.
5. Добавьте в HTTP-обработчик из главы 30 middleware, который перехватывает панику, логирует стек и возвращает 500. Проверьте его, намеренно вызвав панику по специальному пути, и убедитесь, что сервер продолжает обслуживать запросы.

Ответы и разбор к этим упражнениям — в приложении D, часть 3.

Глава 32. Конкурентность: горутины, каналы, context

Конкурентность в Go — не библиотека, а часть языка: горутина создаётся словом `go`, канал — встроенный тип, `select` — конструкция. Это меняет масштаб задачи: сервис, который держит десять тысяч соединений, пишется как обычный последовательный код, где на каждое соединение приходится одна горутина ценой в несколько килобайт стека. Но у простоты есть цена — цена дисциплины: утечки горутин, гонки данных и забытые отмены не видны компилятору. В этой главе мы разберём, как устроен планировщик и почему горутины дешевле потоков, как правильно работать с каналами и `select`, что делает `context` и чего он делать не должен, как выглядит рабочая очередь задач и какими инструментами ловить проблемы — от `runtime.NumGoroutine` до pprof и race detector.

Горутины и планировщик

Горутина (goroutine) — функция, запущенная словом `go` и исполняемая планировщиком Go. Начальный стек — 2 КБ, он растёт и сжимается динамически, поэтому сотни тысяч горутин в одном процессе реальны. Поток ОС имеет стек фиксированного размера (обычно 8 МБ виртуальной памяти), и переключение между потоками делает ядро; переключение горутин делает рантайм в пространстве пользователя, без системного вызова.

```go
func main() {
    var wg sync.WaitGroup
    for i := 0; i < 5; i++ {
        wg.Add(1)
        go func(id int) {          // параметр передаётся явно
            defer wg.Done()
            fmt.Println("worker", id)
        }(i)
    }
    wg.Wait()
}
```

Планировщик устроен по модели GMP. `G` — горутина со своим стеком и состоянием. `M` — поток ОС (machine), который исполняет

код. `P` — логический процессор (processor) с локальной очередью горутин; их количество равно `GOMAXPROCS` и по умолчанию — числу доступных CPU. Поток может исполнять код, только удерживая `P`. Это даёт локальные очереди без блокировок и работу кражи задач (work stealing): простаивающий `P` забирает половину очереди у занятого.

Планировщик вытесняющий: с Go 1.14 горутина может быть прервана в любой точке через сигнал, а не только на вызовах функций. До этого цикл без вызовов функций мог занять поток навсегда. Блокирующие операции не занимают поток: сетевой ввод-вывод обслуживается поллером (`epoll` в Linux), и горутина, ждущая сокет, просто снимается с `P`, освобождая поток для других. Системный вызов, который нельзя разбить (например, чтение файла), блокирует `M`, и рантайм создаёт дополнительный поток — поэтому «дешёвая» горутина может стоить потока, если она делает блокирующие syscall.

Практическое следствие для контейнеров: `GOMAXPROCS` влияет на параллелизм. С Go 1.25 значение по умолчанию учитывает лимиты cgroup (`cpu.max` в Kubernetes), а не только `runtime.NumCPU()`, — раньше под с лимитом 2 CPU на 64-ядерной ноде получал `GOMAXPROCS=64` и страдал от троттлинга и лишних переключений. Если вы задаёте `GOMAXPROCS` переменной окружения, помните: она отключает новое поведение по умолчанию.

Каналы

Канал — типизированная очередь с синхронизацией. Небуферизованный (`make(chan int)`) работает как рандеву: отправитель блокируется, пока получатель не готов, и наоборот. Буферизованный (`make(chan int, 10)`) принимает до `cap` значений без получателя; отправка в полный канал блокируется, чтение из пустого — тоже.

```go
jobs := make(chan int, 3)   // буфер на 3 задачи
jobs <- 1                   // не блокируется, пока есть место
close(jobs)                 // закрывает отправитель, когда задач больше не будет

for j := range jobs {       // цикл завершится, когда канал закрыт и пуст
    fmt.Println(j)
```

```
}
```

Семантика, которую нужно помнить целиком:

Операция	nil-канал	закрытый канал
Отправка `ch <- v`	блокируется навсегда	паника
Получение `<-ch`	блокируется навсегда	сразу нулевое значение, `ok=false`
`close(ch)`	паника	паника

Закрывать канал должен отправитель, и только он: получателей может быть несколько, и никто из них не знает, закончили ли остальные. Закрытие — это сигнал «данных больше не будет», а не «я больше не читаю». Если получатель хочет сообщить, что ему надоело, для этого есть отдельный канал или `context`.

Проверка `v, ok := <-ch` отличает закрытый канал от нулевого значения в потоке данных. Правило: если по каналу передаются значения, которые могут быть нулевыми, проверяйте `ok`; если канал используется только как сигнал завершения — достаточно `range` или `<-done`.

Ещё одно правило, которое экономит отладку: не смешивайте «данные» и «завершение» в одном канале. Отдельный `done chan struct{}` и отдельный канал данных читаются проще и не требуют различать «пришёл ноль» и «канал закрыт».

select

`select` ждёт готовности любого из нескольких каналов и выполняет соответствующую ветку. Если готовы несколько — выбор случайный, что защищает от starvation.

```go
select {
case v := <-results:
    handle(v)
case <-ctx.Done():
    return ctx.Err()
case <-time.After(2 * time.Second):
    return errors.New("timeout")
}
```

Ветка `default` делает `select` неблокирующим — удобно для «попробуй забрать, не жди». Но `select` с `default` в цикле

превращается в busy-loop, который съедает CPU; если нужен опрос, добавьте паузу или используйте тикер.

`time.After` в горячем цикле раньше был известной утечкой: таймер жил до срабатывания, даже если ветка уже не нужна. С Go 1.23 неиспользуемые таймеры освобождаются сборщиком мусора, но привычка `timer := time.NewTimer(d); defer timer.Stop()` всё равно полезна — она делает намерение явным и не зависит от версии.

sync: WaitGroup, Mutex, Once

`sync.WaitGroup` ждёт завершения группы горутин. `Add` вызывается **до** запуска горутины, иначе `Wait` может проскочить раньше, чем счётчик увеличится. `Done` обычно ставят через `defer`.

`sync.Mutex` защищает критическую секцию. `sync.RWMutex` даёт параллельное чтение, но пишет всё равно по одному; он выигрывает только при заметно преобладающем чтении и коротких секциях, иначе накладные расходы съедают выгоду.

```go
type Metrics struct {
    mu     sync.Mutex
    counts map[string]int
}

func (m *Metrics) Inc(key string) {
    m.mu.Lock()
    defer m.mu.Unlock()
    if m.counts == nil {
        m.counts = make(map[string]int)
    }
    m.counts[key]++
}
```

`sync.Once` выполняет инициализацию ровно один раз, даже при одновременных вызовах; `sync.OnceValue` (с Go 1.21) оборачивает функцию и возвращает её результат, сохраняя его.

Отдельно стоит запомнить: мьютекс нельзя копировать после первого использования. Поэтому структура с мьютексом передаётся по указателю, а `go vet` предупреждает о копировании (`copylocks`).

context: отмена, дедлайны, значения

`context.Context` — стандартный способ передать сигнал отмены, дедлайн и небольшую порцию метаданных через границы вызовов. Контекст всегда первый параметр функции, всегда называется `ctx`, никогда не хранится в структуре и никогда не равен nil (если сомневаетесь — `context.Background()`).

```go
func fetchAll(ctx context.Context, urls []string) ([]string, error) {
    ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
    defer cancel() // обязательно, иначе утечка таймера и горутины

    results := make([]string, len(urls))
    errCh := make(chan error, len(urls))

    var wg sync.WaitGroup
    for i, u := range urls {
        wg.Add(1)
        go func() {
            defer wg.Done()
            body, err := fetchOne(ctx, u)
            if err != nil {
                errCh <- err
                return
            }
            results[i] = body
        }()
    }
    wg.Wait()
    close(errCh)
    for err := range errCh {
        return nil, err
    }
    return results, nil
}
```

Каждая горутина пишет в свой индекс `results`, поэтому мьютекс не нужен: непересекающаяся запись в разные элементы среза безопасна. Буфер `errCh` равен числу URL — гарантия, что ни одна горутина не заблокируется на отправке ошибки, даже если читатель уже ушёл. Тот же паттерн с автоматической отменой остальных запросов при первой ошибке даёт `errgroup` из `golang.org/x/sync`.

Правила, которые нарушают чаще всего. `defer cancel()` нужен всегда, даже если контекст с дедлайном и вы уверены, что он истечёт: до истечения таймер и дочерний контекст держат память. Отмену надо проверять: функция, которая получила `ctx`, но ни разу не посмотрела `ctx.Done()` или `ctx.Err()`, отмену игнорирует. И `context.WithValue` — не для передачи параметров: это

для request-scoped метаданных (идентификатор запроса, идентификатор трассировки, данные аутентификации), которые нужны на всех уровнях, но не являются аргументами бизнес-логики. Ключ должен быть неэкспортируемым типом, а не строкой, иначе два пакета перетрут друг друга.

go

```
type ctxKey struct{}

func WithRequestID(ctx context.Context, id string) context.Context {
    return context.WithValue(ctx, ctxKey{}, id)
}

func RequestID(ctx context.Context) string {
    id, _ := ctx.Value(ctxKey{}).(string)
    return id
}
```

Утечки горутин

Горутина не собирается сборщиком мусора, пока выполняется: она держит стек и всё, на что ссылается. Утечка — горутина, заблокированная навсегда. Три типовых сценария: отправка в канал, который никто не читает; чтение из канала, в который никто не пишет; цикл, у которого нет условия выхода при отмене.

go

```
func leak() {
    ch := make(chan int)
    go func() { ch <- 1 }() // никто не прочитает — горутина висит вечно
    // return без чтения
}
```

Ловится это профилем горутин. В сервисе поднимают `net/http/pprof` и смотрят:

bash

```
go tool pprof -http=:8081 http://127.0.0.1:6060/debug/pprof/goroutine
```

Профиль показывает стеки всех горутин, сгруппированные по месту создания; растущее число одинаковых стеков на одном канале — почти всегда утечка. Быстрая проверка без pprof: `runtime.NumGoroutine()` в метрике; стабильный сервис держит примерно постоянное число, линейный рост при постоянной нагрузке — сигнал.

Worker pool

Канонический паттерн: фиксированное число воркеров читает задачи из канала. Число воркеров ограничивает параллелизм — важно, когда работа упирается во внешний ресурс (БД, API), а не в CPU.

go

```
func Run(ctx context.Context, n int, jobs <-chan Job) error {
    var wg sync.WaitGroup
    errCh := make(chan error, n)

    for i := 0; i < n; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for {
                select {
                case <-ctx.Done():
                    return
                case j, ok := <-jobs:
                    if !ok {
                        return
                    }
                    if err := process(ctx, j); err != nil {
                        select {
                        case errCh <- err:
                        default: // не блокируемся, если ошибку уже никто не ждёт
                        }
                        return
                    }
                }
            }
        }()
    }

    wg.Wait()
    close(errCh)
    for err := range errCh {
        return err // наружу — первая из собранных ошибок
    }
    return nil
}
```

Здесь важны три детали. Буфер `errCh` равен числу воркеров, поэтому ни один воркер не блокируется навсегда, даже если никто не читает. `default` в отправке ошибки гарантирует, что поздние ошибки не подвесят горутину. `select` с `ctx.Done()` в каждом цикле — единственный способ остановить воркер, который ждёт задачу. Побочный эффект схемы: воркер, вернувший ошибку, выбывает, и пул продолжает работу меньшим числом горутин — для «первая ошибка отменяет всё» нужен ещё один `cancel`.

Race detector

Гонка данных — два обращения к одной памяти, хотя бы одно из них запись, без синхронизации. Компилятор её не видит, тесты часто тоже: результат зависит от планирования.

bash

```
go test -race ./...
go run -race ./cmd/exporter
```

Детектор инструментирует код и отслеживает доступы к памяти; при гонке печатает два стека — где читали и где писали. Цена: замедление в 5–10 раз и кратный рост памяти, поэтому в продакшне его не включают. И главное ограничение: детектор находит только те гонки, которые реально произошли в этом запуске. Отсутствие отчёта не доказывает отсутствие гонки, поэтому `-race` гоняют в CI на тестах с параллельными сценариями. Бенчмарки и профилирование подробно разбираются в главе 35.

Типичные ошибки

Захват переменной цикла без параметра. До Go 1.22 все итерации делили одну переменную, и горутины видели последнее значение. С Go 1.22 переменная своя на каждой итерации, и старый баг исчез — но код, написанный до этого, всё ещё встречается, а в модулях с `go 1.21` в `go.mod` старое поведение сохраняется. Механизм: версия языка в `go.mod` определяет семантику цикла.

`wg.Add` **внутри горутины.** `Wait` может выполниться до первого `Add` и вернуться сразу, после чего `main` завершит процесс с работающими горутинами. `Add` должен вызываться в порождающем коде, до `go`.

Чтение из закрытого канала как из живого. Закрытый канал отдаёт нулевые значения бесконечно, и цикл `for { v := <-ch }` без проверки `ok` крутится вхолостую, потребляя CPU. Механизм: закрытие — не ошибка, а нормальное состояние, и отличить его можно только вторым значением.

Отправка в канал без получателя. Воркер, который пишет в небуферизованный канал после того, как читатель ушёл по таймауту, висит навсегда. Лечится `select` с `ctx.Done()` или буфером, достаточным для всех отправителей.

`defer cancel()` **забыт.** Контекст с таймаутом держит таймер и родительскую ссылку до истечения срока; в цикле на каждый запрос это накопление. `go vet` ловит часть таких случаев (`lostcancel`), но не все — например, когда `cancel` присвоен в поле структуры.

Мьютекс вокруг долгой операции. Блокировка, удерживаемая на время сетевого вызова, сериализует весь сервис и превращает параллельный обработчик в однопоточный. Критическая секция должна содержать только работу с памятью.

Проверка `err == context.Canceled` **вместо** `errors.Is`**.** Отменённый контекст почти всегда возвращается обёрнутым (`fmt.Errorf("fetch: %w", err)`), и прямое сравнение не срабатывает. Нужен `errors.Is`.

Чек-лист

- [ ] `Add` у `WaitGroup` вызывается до запуска горутины.
- [ ] Каждый канал закрывает ровно один отправитель, и это не получатель.
- [ ] Все горутины, запущенные на запрос, завершаются по `ctx.Done()` или по закрытию канала.
- [ ] `cancel` вызывается через `defer` в той же функции, где создан контекст.
- [ ] `context.WithValue` используется только для метаданных, ключи — неэкспортируемые типы.
- [ ] В тестах есть `-race`, и хотя бы один тест работает параллельно.
- [ ] `runtime.NumGoroutine()` не растёт линейно под нагрузкой (проверено нагрузочным прогоном).
- [ ] Мьютекс не удерживается во время сетевых вызовов и не копируется.

Упражнения

1. Напишите функцию `Merge(chans ...<-chan int) <-chan int`, которая сливает несколько каналов в один и закрывает результат, когда закрыты все источники. Проверьте на трёх каналах с разной скоростью.
2. Реализуйте worker pool из раздела про пул, добавив ограничение «не более N одновременных HTTP-запросов» и сбор всех ошибок через `errors.Join`. Прогоните под `-race`.
3. Сделайте намеренную утечку горутин и найдите её через `pprof` по адресу `/debug/pprof/goroutine`. Опишите, как по стеку понять место блокировки.
4. Напишите HTTP-обработчик, который отменяет работу при разрыве соединения клиентом (`r.Context()`), и проверьте через `curl --max-time`, что горутина завершается, а не продолжает считать.
5. Сравните два варианта счётчика: под `sync.Mutex` и на атомарных операциях `sync/atomic`. Замерьте `go test -bench` на 1, 4 и 8 горутинах и объясните разницу.

Ответы и разбор к этим упражнениям — в приложении D, часть 3.

Глава 33. HTTP-сервер и клиент

Стандартная библиотека Go содержит полноценный HTTP-сервер и клиент: `net/http` не требует фреймворка, поддерживает HTTP/1.1 и HTTP/2 (второй — автоматически при TLS), пул соединений, таймауты и корректное завершение. Именно поэтому на Go пишут экспортёры, вебхуки, sidecar-прокси и внутренние API: сервис на 300 строк без единой внешней зависимости — норма, а не подвиг. Но у `net/http` есть свойство, о котором узнают в инциденте: значения по умолчанию рассчитаны на «работает и не падает», а не на «устойчив к враждебной сети». В этой главе мы соберём сервис с обработчиками, middleware, идентификатором запроса, таймаутами и остановом по `SIGTERM`, а затем — клиент с пулом соединений, таймаутами и осмысленными повторами.

Сервер: мультиплексор и обработчики

Обработчик (handler) — это всё, что умеет `ServeHTTP(http.ResponseWriter, *http.Request)`. `http.HandlerFunc` превращает обычную функцию в обработчик. Мультиплексор (`ServeMux`) сопоставляет путь и метод с обработчиком.

С Go 1.22 шаблоны стали выразительнее: в них можно указать метод и переменные пути.

```go
mux := http.NewServeMux()
mux.HandleFunc("GET /healthz", handleHealth)
mux.HandleFunc("GET /items/{id}", func(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")   // значение из пути
    fmt.Fprintf(w, "item %s\n", id)
})
mux.HandleFunc("POST /items", handleCreate)
mux.HandleFunc("GET /files/{path...}", handleFiles) // {path...} — остаток пути
```

Шаблон `GET /items/{id}` не совпадёт с `POST /items/42` — вернётся 405 с заголовком `Allow`, а не 404. Это заметно упрощает API: не нужно вручную разбирать `r.Method` в каждом обработчике.

Глобальный `http.DefaultServeMux` и функции `http.HandleFunc` удобны в примерах, но в сервисе лучше создавать свой мультиплексор.

Причина в том, что `DefaultServeMux` — глобальная переменная, и любой импортированный пакет может зарегистрировать в ней путь при инициализации (так делают `net/http/pprof` и некоторые библиотеки). В результате маршрут появляется в вашем API неявно, а тесты, поднимающие два сервера, конфликтуют за одни и те же пути.

Middleware

Middleware — функция, принимающая обработчик и возвращающая обработчик. Так как обработчик — это интерфейс с одним методом, обёртки складываются в цепочку без магии.

```go
type Middleware func(http.Handler) http.Handler

func Chain(h http.Handler, mws ...Middleware) http.Handler {
    for i := len(mws) - 1; i >= 0; i-- { // последний в списке — ближе всего к
        обработчику
        h = mws[i](h)
    }
    return h
}
```

Порядок применения обратный порядку перечисления: `Chain(h, A, B)` даёт `A(B(h))`, то есть `A` видит запрос первым, а `B` — первым ответ. Это важно для логирования: middleware, снимающий метрики по времени, должен стоять снаружи, иначе он не учтёт время, потраченное внешними обёртками.

Логирование требует знать код ответа, а `http.ResponseWriter` его не отдаёт. Решение — обёртка, запоминающая статус:

```go
type statusRecorder struct {
    http.ResponseWriter
    status int
    bytes  int
}

func (r *statusRecorder) WriteHeader(code int) {
    r.status = code
    r.ResponseWriter.WriteHeader(code)
}

func (r *statusRecorder) Write(p []byte) (int, error) {
    if r.status == 0 {
        r.status = http.StatusOK // неявный 200 при первой записи
    }
```

```
    n, err := r.ResponseWriter.Write(p)
    r.bytes += n
    return n, err
}
```

Без такой обёртки в логах будет `status=0` для всех успешных ответов, потому что обработчик не вызывает `WriteHeader` явно. И ещё: обёртка ломает доступ к `http.Flusher` и `http.Hijacker`, если не реализовать их проброс — для SSE и WebSocket это критично.

Таймауты: почему сервер без них уязвим

`http.Server` с нулевыми таймаутами не имеет ограничений по времени вообще. Клиент может открыть соединение, отправить один байт заголовка и молчать — сервер будет держать соединение и горутину, пока клиент не отвалится по своему таймауту (или никогда). Это атака Slowloris: несколько тысяч таких соединений исчерпывают лимит файловых дескрипторов (типичный `LimitNOFILE` в systemd-юните — 1024, если не поднят), и сервер перестаёт принимать новые подключения.

go

```
srv := &http.Server{
    Addr:              ":8080",
    Handler:           handler,
    ReadHeaderTimeout: 5 * time.Second,   // защита от Slowloris
    ReadTimeout:       15 * time.Second,  // весь запрос: заголовки + тело
    WriteTimeout:      30 * time.Second,  // от заголовков до конца записи
    IdleTimeout:       90 * time.Second,  // keep-alive простаивающего соединения
    MaxHeaderBytes:    1 << 20,           // 1 МиБ на заголовки
}
```

Смысл каждого параметра разный, и путать их — типичная ошибка. `ReadHeaderTimeout` ограничивает только чтение заголовков и закрывает самый дешёвый вектор Slowloris. `ReadTimeout` — всё чтение запроса целиком, включая тело; если сервис принимает большие загрузки, его надо увеличивать или использовать `http.MaxBytesReader` вместе с ним. `WriteTimeout` отсчитывается от конца чтения заголовков запроса; для долгих стриминговых ответов он станет ограничением, и тогда его поднимают или управляют дедлайном через `http.ResponseController` (появился в Go 1.20). `IdleTimeout` не даёт простаивающим keep-alive соединениям копиться; если он ноль, используется `ReadTimeout`.

Таймаут сервера — грубая защита уровня соединения. Точная — дедлайн на обработку запроса:

```
go

func withTimeout(d time.Duration, h http.Handler) http.Handler {
    return http.TimeoutHandler(h, d, `{"error":"timeout"}`)
}
```

`http.TimeoutHandler` возвращает 503, если обработчик не успел, но не прерывает его: горутина продолжает работу, пока не завершится сама. Настоящая отмена делается через `r.Context()`, который отменяется при разрыве соединения клиентом и при `Shutdown`.

Graceful shutdown

При деплое под приходит `SIGTERM`. Если процесс завершится немедленно, клиенты получат оборванные соединения, а балансировщик — ошибки 502. Правильное поведение: перестать принимать новое, доработать текущие запросы, затем выйти.

```
go

func run(srv *http.Server) {
    ctx, stop := signal.NotifyContext(context.Background(), syscall.SIGTERM,
        os.Interrupt)
    defer stop()

    go func() {
        if err := srv.ListenAndServe(); err != nil && !errors.Is(err,
            http.ErrServerClosed) {
            log.Fatalf("listen: %v", err)
        }
    }()

    <-ctx.Done() // пришёл SIGTERM
    stop()       // восстанавливаем обработку сигналов: второй SIGTERM убьёт
        процесс

    shutdownCtx, cancel := context.WithTimeout(context.Background(),
        15*time.Second)
    defer cancel()
    if err := srv.Shutdown(shutdownCtx); err != nil {
        log.Printf("shutdown: %v", err)
    }
}
```

`Shutdown` закрывает слушающие сокеты, закрывает простаивающие соединения и ждёт завершения активных, пока не истечёт переданный контекст. `ListenAndServe` при этом возвращает

`http.ErrServerClosed` — это нормальный код выхода, поэтому его нельзя логировать как ошибку и нельзя вызывать `log.Fatal` на нём.

Таймаут в `shutdownCtx` должен быть меньше, чем `terminationGracePeriodSeconds` в Kubernetes (по умолчанию 30 секунд), иначе под получит `SIGKILL` до того, как сервер закончит. Практика: время на остановку 15–20 секунд при grace period 30 — остаётся запас на сбор логов и выход.

Отдельная деталь: `SIGKILL` перехватить нельзя, поэтому «мы ловим все сигналы» — неверная формулировка. Ловят `SIGTERM` и `SIGINT`; `SIGKILL` — последняя мера ядра.

Клиент

`http.DefaultClient` не имеет таймаута. Запрос к недоступному хосту может висеть минутами, удерживая горутину и соединение; в сервисе с высокой нагрузкой это исчерпывает пул и память. Свой клиент — обязателен.

```
go


var httpClient = &http.Client{
    Timeout: 10 * time.Second, // весь запрос: соединение, редиректы, чтение тела
    Transport: &http.Transport{
        DialContext:           (&net.Dialer{Timeout: 3 * time.Second, KeepAlive: 30 *
            time.Second}).DialContext,
        TLSHandshakeTimeout:   3 * time.Second,
        ResponseHeaderTimeout: 5 * time.Second,
        MaxIdleConns:          100,
        MaxIdleConnsPerHost:   20, // по умолчанию 2
        IdleConnTimeout:       90 * time.Second,
        ForceAttemptHTTP2:     true,
    },
}
```

`Client.Timeout` покрывает всё: установку соединения, TLS-рукопожатие, отправку, ожидание заголовков, чтение тела. Это удобно, но у него есть следствие: таймаут срабатывает и во время чтения тела, поэтому для больших ответов нужен либо больший `Timeout`, либо отказ от него в пользу `context.WithTimeout` на конкретный запрос.

`MaxIdleConnsPerHost` по умолчанию равен 2. Для сервиса, который делает десятки параллельных запросов к одному API, это означает постоянные новые TCP-соединения и TLS-рукопожатия; поднятие до 20–50 обычно даёт кратный выигрыш по задержке. `Transport` нужно переиспользовать: создание нового на каждый

запрос лишает вас пула и оставляет висеть простаивающие соединения до сборки мусора.

Повторы

Повтор запроса — операция с побочными эффектами, если метод не идемпотентен. Повторять `GET`, `HEAD`, `PUT`, `DELETE` при сетевой ошибке и кодах 502/503/504 разумно; `POST` — только если сервис поддерживает ключ идемпотентности. Ответ 429 повторяют, уважая `Retry-After`.

go

```
func doWithRetry(ctx context.Context, req *http.Request) (*http.Response, error) {
    backoff := 200 * time.Millisecond
    for attempt := 0; attempt < 4; attempt++ {
        resp, err := httpClient.Do(req.Clone(ctx))
        switch {
        case err != nil && ctx.Err() != nil:
            return nil, ctx.Err() // отмена важнее повтора
        case err != nil:
            // сетевая ошибка — повторяем
        case resp.StatusCode < 500 && resp.StatusCode !=
            http.StatusTooManyRequests:
            return resp, nil
        default:
            io.Copy(io.Discard, io.LimitReader(resp.Body, 1<<20)) // дочитать и
                закрыть
            resp.Body.Close()
        }

        select {
        case <-ctx.Done():
            return nil, ctx.Err()
        case <-time.After(backoff):
        }
        backoff *= 2
    }
    return nil, errors.New("retries exhausted")
}
```

Три вещи, без которых повтор вредит. Тело ответа нужно дочитать и закрыть перед повтором, иначе соединение не вернётся в пул. `req.Clone(ctx)` нужен, потому что тело запроса одноразовое: повторная отправка того же `*http.Request` с непустым `Body` отправит пустое тело. И задержка должна расти, иначе при массовом сбое все клиенты синхронно добьют уже падающий сервис — к экспоненте добавляют джиттер.

Полный сервис

Соберём всё вместе: маршруты, middleware с request id и логом, таймауты, `/healthz` и `/readyz`, останов по `SIGTERM`.

go

```
func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK) // liveness: процесс жив
    })
    mux.HandleFunc("GET /readyz", func(w http.ResponseWriter, r *http.Request) {
        ctx, cancel := context.WithTimeout(r.Context(), time.Second)
        defer cancel()
        if err := checkDeps(ctx); err != nil { // готовность принимать трафик
            http.Error(w, "not ready", http.StatusServiceUnavailable)
            return
        }
        w.WriteHeader(http.StatusOK)
    })

    h := Chain(mux, RequestID, AccessLog, Recoverer)

    srv := &http.Server{
        Addr:              envOr("ADDR", ":8080"),
        Handler:           h,
        ReadHeaderTimeout: 5 * time.Second,
        ReadTimeout:       15 * time.Second,
        WriteTimeout:      30 * time.Second,
        IdleTimeout:       90 * time.Second,
        BaseContext:       func(net.Listener) context.Context { return context.Background() },
    }
    run(srv) // функция с signal.NotifyContext и Shutdown из раздела выше
}
```

Разница между проверками принципиальна для Kubernetes. `healthz` (liveness) отвечает «процесс жив и не завис»: если она падает, kubelet перезапускает под. `readyz` (readiness) отвечает «можно слать трафик»: если недоступна БД, под выводится из балансировки, но не перезапускается. Ошибка новичков — проверять в liveness зависимости, из-за чего падение БД вызывает лавину перезапусков всех подов и полный отказ сервиса.

Middleware `RequestID` берёт идентификатор из заголовка `X-Request-ID` (если его прислал прокси) или генерирует новый, кладёт в контекст и в заголовок ответа — тогда один и тот же идентификатор виден и в логах сервиса, и в логах балансировщика, и в ответе клиенту.

Проверка работоспособности:

bash

```
go build -o /tmp/svc . && /tmp/svc &
curl -sS -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8080/healthz
```

```
kill -TERM %1
sleep 1; jobs
```

Ожидаемый вывод — `200`, после `kill` процесс выходит с кодом 0 и освобождает порт: активные запросы доработаны, новые не принимаются.

Типичные ошибки

Сервер без таймаутов. Нулевые значения означают «ждать вечно»: Slowloris держит горутины и дескрипторы, пока не кончится `LimitNOFILE`. Механизм — `net.Conn` без дедлайна; лечится `ReadHeaderTimeout` плюс `ReadTimeout`.

`log.Fatal` **на** `http.ErrServerClosed`**.** При штатном завершении `ListenAndServe` возвращает эту ошибку, и `log.Fatal` превращает нормальный выход в код 1. В systemd это выглядит как падение; проверяйте `errors.Is(err, http.ErrServerClosed)`.

Отсутствие `defer resp.Body.Close()`**.** Тело ответа держит соединение; без закрытия транспорт не возвращает его в пул, и после нескольких сотен запросов клиент упирается в лимит сокетов. Механизм — соединение освобождается только после `Close` и полного чтения тела.

Повтор неидемпотентного запроса. Повтор `POST` создаёт дубликат заказа или платежа. Механизм — сервер не отличает повтор от нового запроса без ключа идемпотентности. Повторяйте только идемпотентные методы или передавайте `Idempotency-Key`.

Новый `http.Client` **на каждый вызов.** Каждый клиент создаёт свой `Transport` со своим пулом; соединения не переиспользуются, растёт число рукопожатий и TIME_WAIT сокетов. Клиент и транспорт создаются один раз и живут всё время работы процесса.

`Shutdown` **без таймаута.** `srv.Shutdown(context.Background())` ждёт активные запросы бесконечно; зависший обработчик не даст процессу выйти, и под получит `SIGKILL` от kubelet. Контекст остановки обязан иметь дедлайн меньше grace period.

Обёртка `ResponseWriter`, теряющая интерфейсы. `statusRecorder` без `Flush`/`Hijack` ломает SSE и WebSocket: обработчик приводит `w` к `http.Flusher` и получает `ok=false`. Нужно реализовать проброс или использовать `http.ResponseController`.

Чек-лист

- [ ] У `http.Server` заданы `ReadHeaderTimeout`, `ReadTimeout`, `WriteTimeout`, `IdleTimeout`.
- [ ] Используется собственный `ServeMux`, а не `DefaultServeMux`.
- [ ] Есть `/healthz` (liveness, без зависимостей) и `/readyz` (readiness, с проверкой зависимостей).
- [ ] `SIGTERM` обрабатывается через `signal.NotifyContext`, `Shutdown` ограничен таймаутом меньше grace period.
- [ ] `http.ErrServerClosed` не логируется как ошибка.
- [ ] Клиент создан один раз, `Timeout` задан, `MaxIdleConnsPerHost` поднят под нагрузку.
- [ ] Тело каждого ответа закрывается, а перед повтором — дочитывается.
- [ ] Каждый запрос имеет идентификатор, он есть и в логах, и в заголовке ответа.

Упражнения

1. Добавьте в сервис маршрут `GET /items/{id}` с валидацией: если `id` не число, вернуть 400 с JSON-ошибкой. Проверьте через `curl`, что `POST /items/1` даёт 405 и заголовок `Allow`.
2. Напишите middleware, который ограничивает частоту запросов с одного IP до 10 в секунду (token bucket на `time.Ticker`), и проверьте его `hey` или простым циклом в `curl`.
3. Реализуйте клиент с повторами и джиттером, затем протестируйте его против локального сервера, который отвечает 503 первые два запроса. Убедитесь по логам, что задержки растут.
4. Запустите сервис под нагрузкой, отправьте `SIGTERM` в момент активных запросов и убедитесь, что все они завершились

успешно. Затем поставьте `WriteTimeout` в 100 мс и повторите — объясните результат.

5. Подключите `net/http/pprof` на отдельный порт (не на публичный мультиплексор) и снимите профиль горутин под нагрузкой. Найдите, сколько горутин занято обработкой запросов и сколько ждёт в `select`.

Ответы и разбор к этим упражнениям — в приложении D, часть 4.

Глава 34. Базы данных в Go

Проблемы в продакшене Go-сервиса почти всегда приходят из базы: «слишком много соединений», «запрос висит двадцать минут», «после переключения на реплику всё сломалось». В главе разбирается `database/sql`: почему это не драйвер, а пул с неочевидными дефолтами. Дальше — `context` в каждом запросе, защита от инъекций, миграции и деградация при недоступной базе.

database/sql: что это на самом деле

`database/sql` — не драйвер и не ORM. Это прослойка: единый интерфейс (`QueryContext`, `ExecContext`, `BeginTx`), пул соединений, кэш подготовленных выражений. Протокол реализует драйвер, который регистрируется в `init()` и подключается пустым импортом:

```
go

import (
    "database/sql"

    _ "github.com/jackc/pgx/v5/stdlib" // регистрирует драйвер "pgx"
)
```

Для PostgreSQL разумен `jackc/pgx/v5/stdlib` (`lib/pq` — в режиме поддержки), для SQLite — `modernc.org/sqlite`, чистый Go без cgo: важно для статических бинарников (глава 36).

Главное следствие: `*sql.DB` — это **пул**, а не соединение. Он создаётся один раз на процесс и передаётся в репозитории. `sql.Open` не устанавливает соединение — только разбирает DSN, поэтому ошибка в пароле проявится на первом запросе, а не при старте.

```
go

func openDB(ctx context.Context, dsn string) (*sql.DB, error) {
    db, err := sql.Open("pgx", dsn)
    if err != nil {
        return nil, fmt.Errorf("open db: %w", err)
    }
    db.SetMaxOpenConns(20)
    db.SetMaxIdleConns(20)
    db.SetConnMaxLifetime(30 * time.Minute)
```

```
    db.SetConnMaxIdleTime(5 * time.Minute)

    pingCtx, cancel := context.WithTimeout(ctx, 3*time.Second)
    defer cancel()
    if err := db.PingContext(pingCtx); err != nil {
        db.Close()
        return nil, fmt.Errorf("ping db: %w", err)
    }
    return db, nil
}
```

Пул соединений и почему дефолты опасны

Параметр	Дефолт	Что это значит
`MaxOpenConns`	0 (без лимита)	Число соединений ограничено только нагрузкой
`MaxIdleConns`	2	В пуле простаивает максимум два соединения
`ConnMaxLifetime`	0 (вечно)	Соединение живёт до закрытия процесса
`ConnMaxIdleTime`	0 (вечно)	Простаивающее соединение не закрывается

Неограниченный `MaxOpenConns` — первая мина. PostgreSQL по умолчанию принимает 100 клиентских соединений, и каждый бэкенд — процесс ОС. Двадцать подов под всплеском легко дают две тысячи соединений, и база отвечает `FATAL: sorry, too many clients already`, причём отказывает и старым подам.

`MaxIdleConns = 2` при `MaxOpenConns = 20` — вторая. На всплеске открывается двадцать соединений, после спада восемнадцать закрываются, на следующем всплеске снова открываются. Каждое новое — TCP-хендшейк, TLS, аутентификация и `fork` бэкенда: миллисекунды, которые складываются в задержку. `MaxIdleConns` ставят равным `MaxOpenConns`.

Бесконечный `ConnMaxLifetime` — третья. Соединение может прожить сутки; за это время pgbouncer или балансировщик закроет его по таймауту простоя, и приложение получит `unexpected EOF`. При failover соединения смотрят на старый адрес. 15–30 минут гарантируют периодическое пересоздание и повторный резолв DNS.

Бюджет соединений базы делят между всеми экземплярами с запасом на миграции: при 100 соединениях и десяти подах — 8–10 на под.

Как увидеть, что делает пул

`db.Stats()` отдаёт `Open`, `InUse`, `Idle`, `WaitCount`, `WaitDuration` — их выводят в метрики. Растущий `WaitCount` при `Open == MaxOpenConns` значит, что пул исчерпан и запросы стоят в очереди.

context в каждом запросе

Нужны `QueryContext`, `QueryRowContext`, `ExecContext`, `BeginTx`, `PingContext`. Версии без контекста означают «ждать вечно»: если база зависла, горутина висит вместе с соединением, двадцать таких исчерпывают пул, и сервис умирает, хотя база жива.

go

```
func (s *Store) Order(ctx context.Context, id int64) (*Order, error) {
    ctx, cancel := context.WithTimeout(ctx, 2*time.Second)
    defer cancel()

    row := s.db.QueryRowContext(ctx,
        `SELECT id, user_id, total_cents, status, created_at
           FROM orders WHERE id = $1`, id)

    var o Order
    err := row.Scan(&o.ID, &o.UserID, &o.TotalCents, &o.Status, &o.CreatedAt)
    if errors.Is(err, sql.ErrNoRows) {
        return nil, ErrNotFound
    }
    if err != nil {
        return nil, fmt.Errorf("query order %d: %w", id, err)
    }
    return &o, nil
}
```

Таймаут ограничивает время удержания соединения и берётся из бюджета запроса (глава 33). Для `Query` обязателен `defer rows.Close()`: незакрытые строки держат соединение.

Prepared statements и SQL-инъекции

Склейка SQL строками — самый распространённый способ получить инъекцию:

go

```
q := fmt.Sprintf("SELECT id FROM users WHERE email = '%s'", email)
// email = "x@y.z' OR '1'='1"  → вернёт всех пользователей
```

Правильно — плейсхолдер: драйвер отправляет запрос и значения отдельно, поэтому значение не может стать частью SQL.

```go
row := db.QueryRowContext(ctx, `SELECT id FROM users WHERE email = $1`, email)
```

Синтаксис зависит от драйвера: `$1`, `$2` в PostgreSQL, `?` в MySQL и SQLite; `database/sql` их не переводит. Плейсхолдер работает только для **значений**: `$1` на месте имени колонки PostgreSQL не примет, поэтому динамическая сортировка идёт через белый список:

```go
col, ok := map[string]string{"created": "created_at", "total":
    "total_cents"}[r.URL.Query().Get("sort")]
if !ok {
    col = "created_at"
}
q := "SELECT id FROM orders ORDER BY " + col + " LIMIT $1"
```

Транзакции и уровни изоляции

```go
func (s *Store) Transfer(ctx context.Context, from, to, amount int64) error {
    tx, err := s.db.BeginTx(ctx, &sql.TxOptions{Isolation: sql.LevelSerializable})
    if err != nil {
        return fmt.Errorf("begin: %w", err)
    }
    defer tx.Rollback() // после Commit вернёт sql.ErrTxDone и ничего не сделает

    if _, err := tx.ExecContext(ctx,
        `UPDATE accounts SET balance = balance - $1 WHERE id = $2`, amount, from);
            err != nil {
        return fmt.Errorf("debit: %w", err)
    }
    if _, err := tx.ExecContext(ctx,
        `UPDATE accounts SET balance = balance + $1 WHERE id = $2`, amount, to);
            err != nil {
        return fmt.Errorf("credit: %w", err)
    }
    return tx.Commit()
}
```

`Read Committed` — дефолт PostgreSQL: видны только закоммиченные данные, но между двумя запросами одной транзакции они могут измениться. `Repeatable Read` даёт снимок на всю транзакцию и ловит потерянные обновления ошибкой сериализации. `Serializable` ведёт себя так, будто транзакции шли по очереди, и требует

повтора при ошибке `40001`; в MySQL дефолт иной — `Repeatable Read` с next-key блокировками.

Миграции: goose

Схема версионируется в репозитории и применяется автоматически, а не руками. `pressly/goose/v3` хранит пары Up/Down в SQL-файлах:

```sql
-- migrations/00002_add_orders_status.sql
-- +goose Up
ALTER TABLE orders ADD COLUMN status text NOT NULL DEFAULT 'new';

-- +goose Down
ALTER TABLE orders DROP COLUMN status;
```

Применение из кода — через `embed`, чтобы миграции попали в бинарник:

```go
//go:embed migrations/*.sql
var migrationsFS embed.FS

func Migrate(db *sql.DB) error {
    goose.SetBaseFS(migrationsFS)
    if err := goose.SetDialect("postgres"); err != nil {
        return err
    }
    return goose.Up(db, "migrations")
}
```

Альтернатива — `golang-migrate/migrate/v4` с CLI `migrate -path ./migrations -database "$DSN" up`. Правила: миграции только вперёд; одна миграция — одно изменение; DDL и заполнение данных не смешивать; удаление колонки — отдельным релизом, когда код перестал её читать.

N+1 и как его увидеть

Код выглядит линейным, а запросов делает N+1:

```go
orders, _ := s.ordersByUser(ctx, userID) // 1 запрос
for _, o := range orders {
    items, _ := s.itemsByOrder(ctx, o.ID) // N запросов
    o.Items = items
}
```

На тысяче заказов это тысяча round-trip по 1–3 мс — секунды и тысяча удержаний соединения. Лечится `JOIN` или двумя запросами с массивом:

```go
rows, err := s.db.QueryContext(ctx,
    `SELECT order_id, sku, qty FROM order_items WHERE order_id = ANY($1)`, ids)
```

Увидеть N+1 можно счётчиком запросов в middleware (обёртка над `*sql.DB` логирует HTTP-вызовы, где запросов больше пяти), через `pg_stat_statements` или по трассировке с десятками одинаковых спанов `SELECT`.

Healthcheck и деградация

Проверка базы не должна быть дорогой. `PingContext` с таймаутом 1–2 секунды — для readiness: под выводится из балансировки, но не перезапускается. В liveness проверку базы не ставят, иначе Kubernetes перезапустит здоровые поды и устроит шторм.

```go
func (h *Health) Ready(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 1500*time.Millisecond)
    defer cancel()
    if err := h.db.PingContext(ctx); err != nil {
        http.Error(w, "db unavailable", http.StatusServiceUnavailable)
        return
    }
    w.WriteHeader(http.StatusOK)
}
```

Деградация — это поведение, когда база не отвечает: таймауты на всё, ограниченные ретраи с backoff, ответ 503 с `Retry-After` вместо зависшего запроса. Для критичных зависимостей добавляют circuit breaker: после серии ошибок запросы не отправляются вовсе, и база не захлёбывается ретраями.

Типичные ошибки

1. `sql.Open` **без настройки пула.** Дефолты дают неограниченные соединения и вечный `ConnMaxLifetime`; проблема проявляется не на тесте, а на сотнях RPS.
2. **Новый** `*sql.DB` **в хендлере.** Каждый `Open` создаёт свой пул, соединения не переиспользуются.

3. `db.Query` **вместо** `QueryContext`**.** Бесконечное ожидание и удержание соединения: при зависании базы пул выедается.
4. **Забытый** `rows.Close()`**.** `Query` держит соединение, пока строки не прочитаны или не закрыты.
5. **Склейка SQL через** `fmt.Sprintf`**.** Любое пользовательское значение в тексте запроса — потенциальная инъекция.
6. **Проверка базы в liveness.** При недоступности базы перезапускаются все поды сразу; восстановление превращается в шторм.
7. **Миграции вручную и без версий.** Окружения расходятся, и никто не знает, какая схема на проде.

Чек-лист

- [ ] `MaxOpenConns` задан числом из бюджета соединений базы, а не нулём.
- [ ] `MaxIdleConns` равен `MaxOpenConns`, `ConnMaxLifetime` — 15–30 минут.
- [ ] `PingContext` с таймаутом вызывается при старте, сервис не поднимается при ошибке.
- [ ] Все запросы идут через `*Context`-методы, у каждого есть таймаут.
- [ ] `rows.Close()` стоит в `defer` сразу после успешного `Query`.
- [ ] Значения передаются плейсхолдерами, идентификаторы — из белого списка.
- [ ] В транзакции есть `defer tx.Rollback()`.
- [ ] Есть `/readyz` с проверкой базы и метрики `db.Stats()` в мониторинге.

Упражнения

1. Запустите PostgreSQL с `max_connections=20` и сервис с `MaxOpenConns=0` под 200 параллельными запросами. Поймайте «too many clients» и почините конфигурацией пула.
2. Напишите обёртку над `*sql.DB`, считающую запросы на HTTP-вызов, найдите N+1 и перепишите его на `ANY($1)`.

3. Реализуйте `Transfer` с `LevelSerializable` и тремя повторами при ошибке `40001` с jitter; проверьте под нагрузкой, что баланс не уходит в минус.
4. Добавьте миграции на goose: колонку `status`, индекс по `user_id`, удаление старой колонки; прогоните `up`, `down`, `up`.
5. Сделайте `/readyz` и проверьте поведение при остановленной базе: под должен выйти из балансировки и вернуться в неё без перезапуска.

Ответы и разбор к этим упражнениям — в приложении D, часть 4.

Глава 34a. Очереди и асинхронная обработка

Сервис, который при оформлении заказа сам отправляет письмо, выставляет счёт и обновляет склад, падает вместе с любой из трёх систем. Очередь разрывает эту связь: заказ фиксируется в базе, а побочные действия идут позже и независимо. Дальше начинается инженерия — семантика доставки, повторы, порядок, отставание потребителя. В главе — зачем нужен брокер, чем отличаются RabbitMQ, Kafka и NATS и как написать потребителя, не боящегося повторной доставки.

Зачем сервису очередь

Разрыв связи по времени. Производитель и потребитель не работают одновременно: заказ создан в 23:59, письмо ушло в 00:01. При синхронных вызовах доступность заказа равна произведению доступностей зависимостей: три сервиса по 99,9% дают 99,7%.

Сглаживание пиков. Нагрузка почти всегда пиковая: распродажа, рассылка, ночной пересчёт. Очередь превращает пик в запас работы, но задержка при этом растёт вместе с длиной очереди — её нужно измерять.

Повторы. Упавший вызов можно повторить позже: сообщение не потеряно, у него есть счётчик попыток и задержка.

Fan-out. Событие `order.created` нужно биллингу, складу, аналитике и поиску. С брокером producer публикует один раз, а подписки заводят сами потребители.

Очередь не нужна, когда ответ нужен немедленно (проверка баланса при оплате) и когда объём — сотни сообщений в сутки: `SELECT ... FOR UPDATE SKIP LOCKED` по таблице задач держит тысячи в секунду.

Производитель, брокер, потребитель

Модель общая, различаются названия. Производитель (producer) публикует сообщение с ключом маршрутизации, брокер (broker) хранит его, потребители (consumers) объединяются в группу (consumer group): внутри группы сообщение достаётся одному участнику, между группами дублируется — так два сервиса читают поток независимо.

Различие, определяющее остальное, — кто кого тянет. RabbitMQ работает по push-модели: брокер сам отправляет сообщения в канал, а `prefetch` ограничивает число неподтверждённых в полёте. Kafka — по pull: потребитель запрашивает батч long poll'ом, поэтому медленный потребитель просто реже спрашивает. После ack RabbitMQ удаляет сообщение, а Kafka лишь сдвигает offset: прочитанное остаётся в логе, и его можно перечитать — в RabbitMQ реплея нет.

RabbitMQ, Kafka, NATS

RabbitMQ — умный брокер с маршрутизацией. Producer публикует в обменник (exchange) типа `direct`, `topic`, `fanout` или `headers`, а тот по ключу раскладывает копии в очереди согласно привязкам. Есть приоритеты, TTL, dead-letter exchange и отложенная доставка через TTL плюс DLX. Сообщение живёт, пока его не подтвердили, поэтому миллион сообщений в очереди — это память брокера; зеркальные очереди заменены кворумными (quorum queues) на Raft.

Kafka — распределённый журнал (log), а не очередь. Топик делится на партиции, каждая — упорядоченный append-only лог на диске, потребитель хранит не подтверждение, а позицию (offset). Записи не удаляются при чтении: действует retention по времени или размеру, типично 7 дней, поэтому поток перечитывают после исправления бага. Пропускную способность дают батчинг и последовательная запись — сотни тысяч сообщений в секунду на брокер; плата — сложность эксплуатации и `linger.ms` в 5–20 мс.

NATS — минималистичный брокер с моделью «subject — подписка». Базовый NATS доставляет at-most-once и ничего не хранит: быстрый транспорт для команд с задержкой в доли

миллисекунды. JetStream добавляет persistence: потоки с retention, at-least-once, дедупликация по заголовку `Nats-Msg-Id` в пределах окна дубликатов (около двух минут), key-value и object store. Кластер собирается на Raft, эксплуатация проще кафкиной.

Критерий	RabbitMQ	Kafka	NATS JetStream
Модель	Очередь с маршрутизацией	Партиционированный лог	Стрим по subject
Порядок	В пределах очереди	В пределах партиции	В пределах subject
Реплей	Нет	Да, до истечения retention	Да, до лимита потока
Когда брать	Маршрутизация, задачи	Потоки событий, аудит	Команды, низкая задержка

Семантика доставки

At-most-once — подтверждение до обработки или его отсутствие. Сообщение теряется при падении потребителя, зато не дублируется: годится для метрик и логов.

At-least-once — подтверждение после обработки. Упал процесс между записью в базу и `ack` — брокер доставит сообщение снова. Дубликат неизбежен, потому что «записать и подтвердить» — две операции без общей транзакции; это режим по умолчанию и правильный выбор.

Effectively-once — не свойство брокера, а свойство системы: at-least-once плюс идемпотентная обработка. Эффект наблюдается один раз, хотя доставок было несколько.

Exactly-once требует, чтобы чтение, обработка и запись были одной транзакцией. Kafka умеет это внутри себя: идемпотентный producer плюс транзакции позволяют читать из топика, писать в топик и коммитить offset атомарно. Как только на выходе появляется PostgreSQL, транзакция его не накрывает — распределённого коммита между брокером и базой нет. Рабочих вариантов два: транзакционный outbox (событие пишется в ту же транзакцию, что и данные) и идемпотентный приёмник.

Идемпотентный потребитель

Идемпотентность — свойство операции, повтор которой не меняет результат. Три способа, от лучшего к худшему.

Естественная идемпотентность: `UPDATE orders SET status = 'paid' WHERE id = $1` при повторе даёт то же состояние. Годится для «установить значение», не годится для «увеличить баланс».

Ключ идемпотентности и таблица обработанных: потребитель вставляет ключ с уникальным индексом и, если вставка не прошла, считает сообщение дубликатом.

```sql
CREATE TABLE processed_events (
    idempotency_key text PRIMARY KEY,
    event_type      text NOT NULL,
    processed_at    timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX processed_events_ts ON processed_events (processed_at);
```

Таблица должна жить дольше максимального окна повторов: если события повторяются через сутки, а записи чистятся через час, дедупликация не работает. Практика — 7–30 дней и партиционирование по `processed_at`, чтобы чистить через `DROP PARTITION`.

Upsert по версии нужен, когда важен и порядок: `ON CONFLICT (id) DO UPDATE ... WHERE excluded.version > t.version` отбрасывает устаревшее событие.

Порядок и партиционирование

Глобального порядка в распределённой очереди нет: он противоречит параллелизму. Гарантия локальная — очередь в RabbitMQ, партиция в Kafka, subject в NATS — и только при одном потребителе на эту единицу.

Ключ выбирают по объекту, а не по типу события: `order_id` отправляет все события заказа в одну партицию, и `created → paid → shipped` придут по порядку. Случайный ключ даёт равномерность без порядка, что нормально для независимых событий.

Две ловушки. Горячий ключ: крупный продавец даёт половину трафика, его партиция отстаёт. Requeue: возвращённое

сообщение встаёт в конец и ломает порядок, поэтому ошибки уводят в retry-очередь.

Подтверждения, видимость, redelivery

Подтверждение (ack) — сигнал, что работу можно забыть. `nack` с `requeue=true` возвращает сообщение, с `requeue=false` отправляет в DLX. В RabbitMQ неподтверждённые висят в состоянии unacked, пока не закроется соединение: тогда они возвращаются с флагом `redelivered`. Зависший потребитель держит их бесконечно, поэтому нужен таймаут обработки.

В Kafka подтверждения нет, есть коммит offset. Пока offset не закоммичен, при ребалансе партиция уходит другому потребителю, и тот начинает с последнего коммита. Отсюда правило: коммитить после обработки, а `max.poll.interval.ms` держать больше времени обработки.

В SQS есть таймаут видимости (visibility timeout): сообщение невидимо N секунд. Если обработка дольше, его начнёт параллельно обрабатывать второй потребитель, поэтому таймаут берут больше p99 обработки.

DLQ и повторная обработка с backoff

Повторять бесконечно в основной очереди нельзя: отравленное сообщение (poison message) крутится в цикле, съедает CPU и раздувает лаг.

Работающая схема: три попытки с задержкой, затем dead-letter queue. В RabbitMQ задержку делают через retry-очередь без потребителей с `x-message-ttl` и DLX обратно в рабочую: 5 с → 30 с → 5 мин. В Kafka — топики `retry.5s`, `retry.1m`, `retry.10m`; после исчерпания попыток сообщение уходит в DLQ на ручной разбор.

DLQ без алерта — тихая потеря данных. Минимум: метрика глубины, алерт на рост, дашборд с телами сообщений и процедура повторной публикации. Полезно писать причину (`x-error`) и исходный ключ маршрутизации.

Consumer lag как главная метрика

Отставание (consumer lag) — разница между последним записанным и последним обработанным сообщением. В Kafka это `log-end-offset − committed offset` по партиции, в RabbitMQ — длина очереди плюс unacked.

Читать её нужно как время: `lag / скорость обработки` — оценка задержки. Лаг в 10 000 сообщений при 50 000/с — это 200 мс; тот же лаг при 10/с — 16 минут.

bash

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group payments-worker
```

Вывод содержит `LOG-END-OFFSET` и `LAG` по партициям; одна отстающая партиция — почти всегда горячий ключ.

Тревоги ставят на производные: лаг растёт монотонно 10–15 минут, время обработки превысило SLO, число неподтверждённых упёрлось в prefetch. Алерт «lag > 10 000» срабатывает на здоровой системе и молчит на больной.

Backpressure

Backpressure (противодавление) — механизм, при котором медленный потребитель заставляет производителя ждать, а не копить очередь. Без него очередь превращается в утечку памяти с задержкой — если producer быстрее consumer, разница оседает в брокере или, хуже, в памяти сервиса, где `go func() { ch <- msg }()` накапливает горутины, а буфер на миллион значений — гигабайты.

Давление нужно вернуть источнику. В RabbitMQ: `prefetch` в десятки сообщений, `x-max-length` с политикой `reject-publish`, publisher confirms. В Kafka ограниченный `buffer.memory`: при заполнении `send` блокируется на `max.block.ms`. В приложении — ограниченный канал (глава 32), семафор на N обработок и 429 с `Retry-After` на входе.

Схема события и её эволюция

Событие — публичный контракт, менять его так же больно, как API. Минимальный конверт:

```json
{
  "id": "0f3c1e9a-6b1f-4d7e-9a55-2c8f0b7d1e44",
  "type": "payment.captured",
  "version": 1,
  "occurred_at": "2026-10-29T18:04:11.512Z",
  "idempotency_key": "payment:88213:captured",
  "payload": {"order_id": 1042, "amount_cents": 150000, "currency": "EUR"}
}
```

`id` — идентификатор сообщения, по нему ловят дубликаты. `type` — имя факта в прошедшем времени, по нему маршрутизируют. `version` — версия схемы. `occurred_at` — когда факт произошёл, а не когда отправлен: при разборе инцидента разница в минуты критична. `idempotency_key` — бизнес-ключ, устойчивый к повторной публикации.

Правила эволюции: добавлять только необязательные поля, не переименовывать и не удалять, не менять тип, новую версию вводить рядом. Потребитель игнорирует неизвестные поля. Строгий вариант — Avro или Protobuf с реестром схем (schema registry).

Пример: потребитель с ручным ack, дедупом и DLQ

Очередь объявляется сразу с dead-letter exchange: приложение не пересылает сообщения, достаточно `nack` без requeue.

```go
args := amqp.Table{
    "x-dead-letter-exchange":    "dlx",
    "x-dead-letter-routing-key": "payments.dlq",
}
_, err := ch.QueueDeclare("payments.q", true, false, false, false, args)
err = ch.Qos(20, 0, false) // не больше 20 неподтверждённых на потребителя
```

Ниже — обработчик: подтверждение после коммита, дедупликация и запись одним оператором, любая ошибка уводит сообщение в DLQ.

```go
package worker

import (
    "context"
    "database/sql"
```

```
    "encoding/json"
    "fmt"
    "log/slog"

    amqp "github.com/rabbitmq/amqp091-go"
)

type Event struct {
    Version        int    `json:"version"`
    IdempotencyKey string `json:"idempotency_key"`
    OrderID        int64  `json:"order_id"`
    AmountCents    int64  `json:"amount_cents"`
}

type Handler struct{ db *sql.DB }

func (h *Handler) Run(ctx context.Context, deliveries <-chan amqp.Delivery) error {
    for d := range deliveries {
        if err := h.handle(ctx, d); err != nil {
            slog.Error("handle", "err", err, "msg_id", d.MessageId)
            _ = d.Nack(false, false) // requeue=false → DLX → DLQ
            continue
        }
        if err := d.Ack(false); err != nil {
            return fmt.Errorf("ack: %w", err)
        }
    }
    return nil
}

func (h *Handler) handle(ctx context.Context, d amqp.Delivery) error {
    var ev Event
    if err := json.Unmarshal(d.Body, &ev); err != nil {
        return fmt.Errorf("decode: %w", err) // повторы не помогут
    }
    if ev.IdempotencyKey == "" || ev.Version != 1 {
        return fmt.Errorf("unsupported: %s v%d", d.Type, ev.Version)
    }

    // Один оператор = одна транзакция: дедуп и запись применяются вместе.
    _, err := h.db.ExecContext(ctx, `
        WITH dedup AS (
            INSERT INTO processed_events (idempotency_key, event_type)
            VALUES ($1, $2) ON CONFLICT (idempotency_key) DO NOTHING
            RETURNING 1
        )
        INSERT INTO payments (id, order_id, amount_cents)
        SELECT $3, $4, $5 FROM dedup
        ON CONFLICT (id) DO UPDATE SET amount_cents = EXCLUDED.amount_cents`,
        ev.IdempotencyKey, d.Type, d.MessageId, ev.OrderID, ev.AmountCents)
    if err != nil {
        return fmt.Errorf("apply: %w", err)
    }
    return nil
}
```

`Ack` стоит после `handle`: падение между записью и подтверждением даёт повторную доставку, и её отбросит `processed_events`. CTE с

`INSERT ... ON CONFLICT DO NOTHING RETURNING 1` — атомарная проверка: если ключ уже есть, CTE не вернёт строк, вставка в `payments` не выполнится, а обработчик завершится успешно. Дедуп и запись попадают в одну неявную транзакцию, поэтому «половины» не бывает.

Типичные ошибки

1. `ack` **до обработки.** At-most-once по ошибке: падение после подтверждения теряет сообщение без следа.
2. **Идемпотентность «на словах».** Проверка и запись — две операции, между ними сбой; без уникального ключа в базе дедупликации нет.
3. `requeue=true` **без backoff.** Отравленное сообщение возвращается мгновенно и бесконечно, съедая CPU.
4. **Ожидание глобального порядка.** События заказа лежат в разных партициях, `paid` приходит раньше `created`.
5. **Алерт по абсолютному лагу.** Порог в сообщениях не связан с задержкой: одно и то же число — норма и катастрофа.
6. **Unbounded-буферы.** Канал без ограничения превращает перегрузку в OOM через десять минут.
7. **Ожидание exactly-once до базы.** Kafka-транзакции не покрывают PostgreSQL; нужен outbox.

Чек-лист

- [ ] У сообщения есть `id`, `type`, `version`, `occurred_at` и `idempotency_key`.
- [ ] Потребитель подтверждает сообщение после коммита, а не после чтения.
- [ ] Дедупликация опирается на уникальный индекс в базе, а не на проверку в коде.
- [ ] Ключ партиционирования привязан к объекту, порядок по которому важен.
- [ ] Ограничены `prefetch` и размер очереди; переполнение возвращает ошибку producer'у.

- [ ] Есть retry с ростом задержки и DLQ, на рост DLQ настроен алерт.
- [ ] Лаг измеряется в секундах обработки, а не только в сообщениях.
- [ ] Схема версионируется, потребитель игнорирует неизвестные поля.

Упражнения

1. Запустите RabbitMQ 4 через `docker run -d -p 5672:5672 -p 15672:15672 rabbitmq:4-management` и опубликуйте 1000 сообщений в очередь с `x-max-length=100` и `reject-publish`; объясните поведение producer.
2. Возьмите consumer из главы, убейте процесс между коммитом и `ack` и добейтесь, чтобы дубликат не изменил баланс. Покажите строки в `processed_events`.
3. Настройте retry-очереди с TTL 5 с, 30 с и 5 мин и DLQ после трёх попыток; проверьте по `x-death`, что счётчик попыток растёт.
4. Соберите Kafka из трёх брокеров, создайте топик с тремя партициями и ключом `order_id`; покажите, что события заказа идут по порядку, а горячий ключ перегружает партицию.
5. Смоделируйте медленного потребителя: добавьте `time.Sleep(200 * time.Millisecond)` в обработчик и постройте график лага; обоснуйте порог алерта через SLO.

Ответы и разбор к этим упражнениям — в приложении D, часть 5.

Глава 34b. SQL: индексы, планы, блокировки

Запрос на 3 мс и он же на 40 секунд отличаются одной строкой SQL. Причина почти всегда одна: база исполняет не то, что написано, а то, что выбрал планировщик по своей статистике и стоимости. В главе разбирается путь запроса от парсера до executor'а, как устроен B-tree и почему индекс иногда вредит, как читать `EXPLAIN ANALYZE` и ловить расхождение оценки с фактом, что происходит с данными при параллельных транзакциях и как разбирать deadlock по логу. Примеры проверены на PostgreSQL 16 с таблицей в миллион строк.

Как база исполняет запрос

Запрос проходит четыре стадии. Парсер строит синтаксическое дерево и проверяет имена таблиц и колонок. Переписывание (rewrite) разворачивает представления (views), применяет правила и row-level security. Планировщик (planner) перебирает варианты доступа и соединения и выбирает самый дешёвый по своей модели стоимости. Executor исполняет выбранный план узлами: каждый узел берёт строки у дочерних и отдаёт результат наверх.

Стоимость — условные единицы, а не время. По умолчанию `seq_page_cost = 1.0` (чтение страницы последовательно), `random_page_cost = 4.0` (случайное чтение), `cpu_tuple_cost = 0.01` за строку. Отсюда следует главное: планировщик сравнивает «прочитать 8000 страниц подряд» с «прочитать 20 страниц индекса и 20 страниц таблицы вразброс». На SSD разница между случайным и последовательным чтением меньше, чем заложено в дефолтах, и `random_page_cost = 1.1` часто ближе к реальности.

Решения планировщик принимает по статистике: гистограммы, число различных значений, корреляция физического порядка с логическим. Собирает её `ANALYZE` — вручную, автоанализом или `VACUUM ANALYZE`, по выборке `300 × default_statistics_target` строк (30 000

при дефолтном значении 100). Нет статистики — план строится по константам.

B-tree: почему поиск логарифмический

B-tree — сбалансированное дерево, где все листья на одной глубине и связаны в список. Внутренние страницы содержат разделительные ключи, листья — пары «ключ → ссылка на строку» (TID: номер блока и смещение). Поиск спускается от корня к листу за высоту дерева: для миллиона строк это обычно три уровня, то есть три страницы вместо восьми тысяч. Диапазон (`BETWEEN`, `>`, `ORDER BY ... LIMIT`) читается как последовательный проход по списку листьев, без повторного спуска.

Индекс хранит ключи отсортированными, поэтому ускоряет и `ORDER BY`: чтение по индексу уже упорядочено. Плата — размер и запись: каждая вставка обновляет все индексы, а случайные ключи (UUID v4) дают случайные страницы и расщепление листьев.

Селективность: когда индекс вредит

Селективность — доля строк, которую отбирает условие. Индекс выгоден, когда строк мало: прочитать 20 страниц таблицы дешевле, чем 8000. Когда строк много, дешевле прочитать таблицу подряд и отфильтровать на месте.

Живой пример: в таблице миллион заказов, у `status` четыре значения, `paid` — четверть строк.

```sql
EXPLAIN (ANALYZE, BUFFERS) SELECT id, total_cents FROM orders WHERE status =
    'paid';
```

```
Seq Scan on orders  (cost=0.00..20631.00 rows=249333 width=12)
                    (actual time=0.010..86.978 rows=250000 loops=1)
  Filter: (status = 'paid'::text)
  Rows Removed by Filter: 750000
  Buffers: shared hit=8131
Execution Time: 94.429 ms
```

Индекс по `status` существует, но запрос, отбирающий 75% строк (`status <> 'paid'`), всё равно идёт seq scan'ом: вернуть придётся 750 000 строк, и индекс только добавит случайных чтений.

```
->  Parallel Seq Scan on orders  (cost=0.00..13339.33 rows=312764 width=0)
```

```
                                (actual time=0.021..34.223 rows=250000 loops=3)
      Filter: (status <> 'paid'::text)
      Rows Removed by Filter: 83333
Execution Time: 50.926 ms
```

Практическое правило: индекс оправдан при выборке до нескольких процентов таблицы и почти бесполезен на 20–30%. Вредные индексы не бесплатны: каждый добавляет запись при `INSERT` и `UPDATE` и мешает autovacuum, поэтому индекс на низкоселективной колонке в одиночку — почти всегда ошибка.

Составные индексы и левый префикс

Индекс по `(user_id, created_at DESC)` отсортирован сначала по `user_id`, внутри — по `created_at`. Поэтому он работает для условий на `user_id`, для `user_id` + диапазон по `created_at`, но бесполезен для одного `created_at`: нужные строки размазаны по всему дереву.

sql

```
CREATE INDEX idx_orders_user_created ON orders (user_id, created_at DESC);
EXPLAIN (ANALYZE, BUFFERS) SELECT id FROM orders WHERE user_id = 42;
```

```
Bitmap Heap Scan on orders  (cost=4.58..81.85 rows=20 width=12)
                            (actual time=0.075..0.095 rows=20 loops=1)
  Recheck Cond: (user_id = 42)
  Heap Blocks: exact=20
  Buffers: shared hit=23 read=3
  ->  Bitmap Index Scan on idx_orders_user_created  (cost=0.00..4.58 rows=20
      width=0)
        Index Cond: (user_id = 42)
Execution Time: 0.131 ms
```

Тот же запрос без индекса шёл 25 мс параллельным seq scan'ом по миллиону строк — выигрыш в двести раз на двадцати строках. А запрос `WHERE created_at > now() - interval '1 hour'` этот индекс не использует: планировщик берёт отдельный `idx_orders_created` и делает `Index Scan ... Index Cond: (created_at > ...)` за 2,5 мс. Правило левого префикса: индекс `(a, b)` обслуживает `a`, `a AND b`, но не `b`.

Порядок колонок: сначала сравнение на равенство, потом одна диапазонная, потом сортировка. `(user_id, created_at)` и `(user_id, status, created_at)` — разные индексы; второй покрывает `user_id AND status AND диапазон`. Колонки с равенством можно менять местами почти без потерь, диапазонную двигать влево нельзя.

Покрывающий индекс

Если все нужные запросу колонки есть в индексе, executor не идёт в таблицу вовсе — это `Index Only Scan`. Синтаксис Postgres — `INCLUDE` для «полезной нагрузки»:

```sql
CREATE INDEX idx_orders_user_covering ON orders (user_id) INCLUDE (total_cents);
VACUUM ANALYZE orders;
```

```
Index Only Scan using idx_orders_user_covering on orders  (cost=0.42..4.78 rows=20
    width=12)
                                                          (actual
               time=0.037..0.039 rows=20 loops=1)
  Index Cond: (user_id = 42)
  Heap Fetches: 0
  Buffers: shared hit=1 read=3
Execution Time: 0.056 ms
```

`Heap Fetches: 0` — таблица не читалась. Условие: страницы помечены в карте видимости (visibility map), а это делает `VACUUM`; без него в плане появится `Heap Fetches: 20` и выигрыш исчезнет. Колонки в `INCLUDE` не участвуют в сортировке индекса, но занимают место в листьях.

EXPLAIN и EXPLAIN ANALYZE

`EXPLAIN` показывает план и оценку, не выполняя запрос. `EXPLAIN ANALYZE` выполняет и добавляет фактические строки и время — на продакшене это запрос, который реально пойдёт в базу, поэтому для `UPDATE` и `DELETE` его оборачивают в `BEGIN ... ROLLBACK`.

Читать план нужно как дерево: строки с большим отступом выполняются первыми и отдают результат родителю. В строке узла: `cost=старт..итог` в условных единицах, `rows` — оценка числа строк на выходе, `width` — средняя ширина строки в байтах, `actual time=первая_строка..все_строки`, `rows` после `actual` — фактические, `loops` — сколько раз узел запускался.

Ключевой навык — сравнивать оценку с фактом. Вот запрос с приведением типа, где статистики для выражения нет:

```
Gather  (cost=1000.00..16922.67 rows=5000 width=8) (actual time=11.734..48.227
    rows=11 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  ->  Parallel Seq Scan on orders  (cost=0.00..15422.67 rows=2083 width=8)
                                   (actual time=13.702..42.582 rows=4 loops=3)
        Filter: ((total_cents)::text = '4999'::text)
        Rows Removed by Filter: 333330
```

```
Execution Time: 48.239 ms
```

Планировщик ждал 5000 строк, получил 11 — ошибка в 450 раз. Причина: гистограммы колонки `total_cents` для выражения `total_cents::text` не применимы, и он взял селективность по умолчанию (0,5%). Тот же запрос без приведения — `WHERE total_cents = 4999` — идёт через `Bitmap Index Scan` за 0,1 мс. В параллельных узлах `rows` — среднее на один воркер, поэтому фактические строки умножают на `loops`: 4 × 3 ≈ 12 при реальных 11, средние округляются.

Оценки врут по трём причинам: планировщик считает условия независимыми (умножает селективности, хотя значения коррелированы), статистики нет для выражений и для `LIKE` с ведущим `%`, и данные успели измениться после `ANALYZE`. Лечится `CREATE STATISTICS`, индексами по выражению, `ALTER COLUMN ... SET STATISTICS` и честным автоанализом.

N+1 и как его увидеть

Код с циклом по заказам делает один запрос за списком и по запросу на каждый элемент — это 1 + N round-trip. На тысяче заказов при 1–2 мс на запрос получается 2 секунды вместо 10 мс. В плане это не видно: каждый запрос по отдельности идеален и попадает в индекс. Видно по счётчику запросов в middleware, по логу с тысячей одинаковых `SELECT` или по `pg_stat_statements`, где `calls` простого запроса растёт синхронно с RPS. Лечение — `JOIN` или `WHERE order_id = ANY($1)` (глава 34).

Транзакции: ACID и уровни изоляции

ACID на пальцах: атомарность — либо всё, либо ничего; согласованность — ограничения схемы соблюдаются; изолированность — параллельные транзакции не видят полурезультатов; долговечность — закоммиченное выживает падение. Первые два свойства обеспечивает приложение и схема, два последних — журнал (WAL) и MVCC.

MVCC означает: пишущий не блокирует читающего, потому что читатель видит снимок данных на момент начала запроса, а старые версии строк живут в таблице, пока их не уберёт `VACUUM`.

Отсюда главная аномалия PostgreSQL — распухание таблиц при долгих транзакциях: они держат горизонт, и мёртвые версии не удаляются.

Уровень	Грязное чтение	Неповторяющееся чтение	Фантомы
Read Uncommitted	нет (работает как RC)	да	да
Read Committed	нет	да	да
Repeatable Read	нет	нет	нет
Serializable	нет	нет	нет

`Read Committed` — дефолт: каждый оператор видит свой снимок, поэтому два одинаковых `SELECT` в одной транзакции могут вернуть разное. `Repeatable Read` даёт один снимок на транзакцию и в PostgreSQL дополнительно отлавливает потерянные обновления ошибкой `40001`. `Serializable` строит граф зависимостей (SSI) и откатывает транзакции, которые дали бы неверный результат при последовательном исполнении, — тоже `40001`, и его нужно повторять.

Практика: держать `Read Committed` и защищать конкретные инварианты явно — `SELECT ... FOR UPDATE` на строку, `UPDATE ... WHERE balance >= $1` с проверкой `RowsAffected`, уникальные индексы. `Serializable` берут, когда инвариант распределён по нескольким строкам и его неудобно защищать руками; тогда обязателен ретрай с jitter, иначе часть запросов будет падать под нагрузкой.

Блокировки: row-level и deadlock

Запись строки берёт эксклюзивную блокировку до конца транзакции; читатель в MVCC не блокируется. `SELECT ... FOR UPDATE` берёт блокировку явно, `NOWAIT` падает сразу вместо ожидания, `SKIP LOCKED` пропускает занятые строки — так строят очередь задач на таблице. Блокировки строк не эскалируются в табличные, но каждая ждущая транзакция занимает соединение.

Deadlock (взаимная блокировка) возникает, когда две транзакции берут блокировки в разном порядке. PostgreSQL находит цикл

через `deadlock_timeout` (по умолчанию 1 с) и убивает одну транзакцию:

```
ERROR:  deadlock detected
DETAIL:  Process 158 waits for ShareLock on transaction 845; blocked by process
    151.
Process 151 waits for ShareLock on transaction 846; blocked by process 158.
HINT:  See server log for query details.
CONTEXT:  while updating tuple (0,10) in relation "orders"
```

Читается так: процесс 158 ждёт транзакцию 845 (её держит 151), а 151 ждёт 846 (её держит 158) — цикл. `CONTEXT` показывает, на какой строке всё встало. Дальше в логе ищут два запроса этих сессий и видят разный порядок `UPDATE`: один блокировал заказ 10, потом 11; второй — наоборот. Лечится не ретраем, а порядком: обновлять строки в одном порядке (например, `ORDER BY id` перед циклом), укорачивать транзакции и не держать в них сетевые вызовы. Для разбора на живой базе — `SELECT pid, pg_blocking_pids(pid), query FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid)) > 0`.

Типовые убийцы производительности

Функция от колонки в `WHERE`. `WHERE date(created_at) = current_date - 3` не может использовать индекс по `created_at`: индекса по `date(created_at)` нет, и планировщик читает всю таблицу — 54 мс и `Rows Removed by Filter: 304533`. Диапазон по голой колонке даёт `Index Only Scan` за 16 мс.

```sql
-- плохо: индекс по created_at не используется
SELECT count(*) FROM orders WHERE date(created_at) = current_date - 3;
-- хорошо: тот же смысл, но sargable-условие
SELECT count(*) FROM orders
 WHERE created_at >= current_date - 3 AND created_at < current_date - 2;
```

`LIKE '%x%'`**.** Ведущий `%` не даёт сузить диапазон в B-tree: индекс упорядочен по началу строки. Даже `LIKE 'user1%'` в общем случае не использует обычный индекс — нужен `text_pattern_ops` или collation `C`; для поиска по подстроке ставят `pg_trgm` с GIN-индексом.

Неявное приведение типов. `WHERE total_cents::text = '4999'` — тот же случай, что выше: 48 мс против 0,1 мс. В MySQL хуже: сравнение строковой колонки с числом приводит колонку к числу молча, и индекс перестаёт работать без всякой ошибки в коде.

`OFFSET` **на больших таблицах.** `LIMIT 20 OFFSET 800000` заставляет прочитать и отсортировать 800 020 строк: `Sort Method: external merge Disk: 25488kB`, 412 мс и 25 МБ временных файлов. Keyset-пагинация — `WHERE created_at < $1 ORDER BY created_at DESC LIMIT 20` — те же 20 строк за 54 мс без сортировки.

Типичные ошибки

1. **Индекс «на всякий случай».** Каждый индекс замедляет запись и занимает место; индекс на низкоселективной колонке не используется и вредит.
2. `EXPLAIN` **без** `ANALYZE`**.** Оценка не равна факту: план выглядит дешёвым, а запрос идёт секунды.
3. **Вера в** `rows` **без сравнения с** `actual rows`**.** Расхождение в десятки раз — сигнал о плохой статистике, а не о медленном диске.
4. **Функция или приведение типа на колонке в** `WHERE`**.** Условие перестаёт быть sargable, и индекс не применяется.
5. **Долгие транзакции.** Держат горизонт MVCC, мешают `VACUUM`, копят мёртвые строки и блокировки.
6. **Разный порядок блокировок.** Причина большинства deadlock'ов; ретрай лечит симптом, а не цикл.
7. `OFFSET` **для глубокой пагинации.** Стоимость растёт линейно с номером страницы, хотя нужны те же 20 строк.

Чек-лист

- [ ] У каждого частого запроса есть план из `EXPLAIN (ANALYZE, BUFFERS)` на объёме, близком к продакшену.
- [ ] Оценка `rows` сопоставлена с `actual rows`; расхождение больше 10 раз объяснено.
- [ ] Условия в `WHERE` sargable: нет функций и приведений над колонкой.
- [ ] Составные индексы начинаются с колонок равенства, диапазонная — последняя.

- [ ] Для горячих точечных запросов рассмотрен покрывающий индекс и `Heap Fetches: 0`.
- [ ] Транзакции короткие, порядок блокировок единый, на `40001` есть ретрай.
- [ ] Глубокая пагинация сделана по ключу, а не через `OFFSET`.
- [ ] `autovacuum` и `ANALYZE` работают; статистика не устарела после массовой загрузки.

Упражнения

1. Создайте таблицу на миллион строк и снимите `EXPLAIN (ANALYZE, BUFFERS)` для запроса без индекса, с индексом и с индексом на низкоселективной колонке; объясните выбор планировщика.
2. Постройте составной индекс `(user_id, status, created_at)` и найдите три запроса, для которых он работает, и три, для которых бесполезен.
3. Добейтесь расхождения оценки и факта в сто раз через приведение типа или коррелированные колонки; почините план через `CREATE STATISTICS` или индекс по выражению.
4. Воспроизведите deadlock двумя сессиями `psql` с обратным порядком `UPDATE`; найдите виновника по `pg_blocking_pids` и исправьте порядок блокировок.
5. Перепишите пагинацию с `OFFSET 800000` на keyset и сравните `Buffers`, `temp` и время на двух планах.

Ответы и разбор к этим упражнениям — в приложении D, часть 5.

Глава 35. Тесты, бенчмарки, pprof, race detector

В Go инструменты тестирования встроены в тулчейн: `go test` знает про покрытие, профилирование и детектор гонок, и всё это не требует сторонних библиотек. Из-за этого тесты в Go пишут чаще, а профилируют реже, чем следовало бы: `pprof` кажется чем-то для избранных. В главе — табличные тесты, бенчмарки и `-benchmem`, чтение вывода `pprof`, поимка гонки и пределы покрытия.

Табличные тесты

Идиома Go — один тест, много случаев в срезе структур, каждый прогоняется через `t.Run`:

```
go
```

```
func TestParsePort(t *testing.T) {
    tests := []struct {
        name    string
        in      string
        want    int
        wantErr bool
    }{
        {"valid", "8080", 8080, false},
        {"lower bound", "0", 0, true},
        {"too big", "65536", 0, true},
        {"not a number", "http", 0, true},
        {"empty", "", 0, true},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got, err := ParsePort(tt.in)
            if (err != nil) != tt.wantErr {
                t.Fatalf("err = %v, wantErr %v", err, tt.wantErr)
            }
            if got != tt.want {
                t.Errorf("got %d, want %d", got, tt.want)
            }
        })
    }
}
```

Подтест с именем даёт адресную фильтрацию: `go test -run 'TestParsePort/too_big'`. Новый случай — одна строка вместо функции. Сообщения пишут в стиле «получили — ожидали», а `t.Fatalf` используют, когда продолжать нет смысла (иначе паника на нулевом значении).

t.Helper и t.Cleanup

Проверки, вынесенные в функции, без `t.Helper()` ломают номера строк: ошибка указывает внутрь хелпера, а не в место вызова.

```go
func assertStatus(t *testing.T, got, want int) {
    t.Helper()
    if got != want {
        t.Fatalf("status = %d, want %d", got, want)
    }
}

func newTestDB(t *testing.T) *sql.DB {
    t.Helper()
    db, err := sql.Open("sqlite", "file:"+t.TempDir()+"/test.db")
    if err != nil {
        t.Fatalf("open db: %v", err)
    }
    t.Cleanup(func() { db.Close() })
    return db
}
```

`t.Cleanup` регистрирует уборку в обратном порядке регистрации и выполняется даже при `t.Fatal` — в отличие от `defer` в хелпере, который сработает при выходе из хелпера, а не из теста. `t.TempDir()` создаёт каталог, который Go удалит сам, и не требует `defer os.RemoveAll`.

httptest

Для хендлеров есть два уровня. Юнит-уровень — `httptest.NewRecorder()`: запрос проходит через хендлер в память, без сети.

```go
func TestHealth(t *testing.T) {
    req := httptest.NewRequest(http.MethodGet, "/healthz", nil)
    rec := httptest.NewRecorder()
    HealthHandler(rec, req)
    if rec.Code != http.StatusOK {
        t.Fatalf("code = %d, want 200", rec.Code)
    }
    if body := rec.Body.String(); body != "ok\n" {
        t.Errorf("body = %q, want %q", body, "ok\n")
    }
}
```

Интеграционный — `httptest.NewServer(handler)`: поднимается настоящий HTTP-сервер на localhost со случайным портом, и

клиент проходит весь путь через сетевой стек, включая заголовки, таймауты и keep-alive. Так проверяют middleware, редиректы и graceful shutdown; `srv.Close()` вызывают через `t.Cleanup`.

Бенчмарки и -benchmem

go

```
func BenchmarkEncode(b *testing.B) {
    payload := make([]byte, 1024)
    b.ReportAllocs()
    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        if _, err := Encode(payload); err != nil {
            b.Fatal(err)
        }
    }
}
```

`b.N` подбирает сам `go test`: он увеличивает число итераций, пока измерение не станет статистически устойчивым. Подготовку данных делают до `b.ResetTimer()`, иначе она попадёт в измерение. Запуск:

bash

```
go test -run '^$' -bench . -benchmem -count 5 ./...
```

text

```
BenchmarkEncode-8   1248390    941.2 ns/op    2048 B/op    3 allocs/op
```

Здесь важны не наносекунды, а `B/op` и `allocs/op`: они не зависят от загрузки машины и показывают, сколько памяти и объектов на итерацию создаёт код. Рост аллокаций — почти всегда рост нагрузки на GC, и это видно даже на шумном ноутбуке. `-count 5` даёт пять прогонов для оценки разброса; сравнение версий удобно через `benchstat`.

pprof: где на самом деле уходит время

Профиль CPU снимают прямо с теста:

bash

```
go test -run '^$' -bench BenchmarkEncode -cpuprofile cpu.out -memprofile mem.out
go tool pprof -top -nodecount=10 cpu.out
```

text

```
      flat  flat%   sum%        cum   cum%
     1.42s 28.40%  28.40%      1.42s 28.40%  runtime.memmove
     0.88s 17.60%  46.00%      2.10s 42.00%  encoding/json.Marshal
     0.41s  8.20%  54.20%      0.41s  8.20%  runtime.mallocgc
```

`flat` — время в самой функции, `cum` — включая вызываемые. Если сверху `memmove` и `mallocgc`, проблема не в логике, а в копировании и аллокациях: лечится переиспользованием буферов и `sync.Pool`. Команда `go tool pprof -list Encode cpu.out` показывает время по строкам функции:

text

```
ROUTINE ======================== main.Encode in /src/encode.go
     0.88s      2.10s (flat, cum) 42.00% of Total
         .          .     12: func Encode(v any) ([]byte, error) {
     0.20s      0.20s     13:     buf := new(bytes.Buffer)
     0.50s      1.80s     14:     enc := json.NewEncoder(buf)
```

Интерактивный режим `go tool pprof cpu.out` даёт те же `top` и `list`, плюс `web` — граф вызовов, для него нужен graphviz. Для живого сервиса профиль снимают через `net/http/pprof`, подключённый пустым импортом на отдельном порту, — `go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30`. Порт наружу не публикуют.

Race detector

`go test -race` включает детектор гонок — инструментирование доступа к памяти. Классическая гонка:

go

```
var counter int

func TestRace(t *testing.T) {
    var wg sync.WaitGroup
    for i := 0; i < 100; i++ {
        wg.Add(1)
        go func() { defer wg.Done(); counter++ }()
    }
    wg.Wait()
}
```

text

```
WARNING: DATA RACE
Write at 0x00c00001a0a8 by goroutine 8:
  main.TestRace.func1()
      /src/main_test.go:12 +0x30
Previous write at 0x00c00001a0a8 by goroutine 7:
  main.TestRace.func1()
```

Детектор не «иногда» находит, а сообщает о реальном пересечении доступа, но только о том, который случился в прогоне: гонка в редкой ветке может не проявиться. Поэтому `-race` гоняют в CI на каждом коммите, а не раз в месяц. Цена — замедление в 5–10 раз и рост потребления памяти; в продакшене сборку с `-race` не выпускают. Лечение: `atomic.Int64`, `sync.Mutex` или передача данных через канал.

Покрытие и его пределы

bash

```
go test -coverprofile=cover.out ./...
go tool cover -func=cover.out | tail -1
go tool cover -html=cover.out
```

Покрытие считает, какие строки выполнялись, а не какие проверены. Строка `if err != nil { return err }` покрыта, если ошибка произошла, но никто не проверял её содержимое. Отсюда типичная картина: 85% покрытия при нулевой проверке граничных случаев. Ориентир для сервисного кода — 60–80% на пакетах с логикой; гнаться за 100% в хендлерах и сгенерированном коде бессмысленно. Полезнее смотреть на покрытие диффа: не изменил ли новый PR ветку без теста.

Тесты с таймаутами

По умолчанию `go test` убивает весь пакет через 10 минут. Отдельный тест ограничивают `-timeout`:

bash

```
go test -timeout 60s -run TestSlowIntegration ./...
```

Внутри теста полезен `t.Deadline()`: он возвращает момент, когда пакет будет убит, и позволяет выделить операции бюджет.

go

```
func TestWorkerShutdown(t *testing.T) {
    dl, ok := t.Deadline()
    ctx := context.Background()
    if ok {
        var cancel context.CancelFunc
        ctx, cancel = context.WithDeadline(ctx, dl.Add(-5*time.Second))
        defer cancel()
    }
    if err := worker.Shutdown(ctx); err != nil {
```

```
        t.Fatalf("shutdown: %v", err)
    }
}
```

Запас в пять секунд оставляет время на `t.Fatalf` и уборку, иначе тест падает по общему таймауту. `t.Parallel()` ускоряет пакет, но требует независимых тестов: общий `temp`-каталог или общая БД превратят параллельный прогон в источник флаки.

Фаззинг

С версии 1.18 в тулчейне есть нативный фаззинг: `go test -fuzz` подставляет в тест случайные строки и ищет панику или нарушение инварианта.

go

```
func FuzzParsePort(f *testing.F) {
    f.Add("8080") // начальный корпус
    f.Fuzz(func(t *testing.T, s string) {
        port, err := ParsePort(s)
        if err == nil && (port < 1 || port > 65535) {
            t.Fatalf("out of range: %d", port)
        }
    })
}
```

bash

```
go test -fuzz FuzzParsePort -fuzztime 30s
```

Фаззер держит найденный вход в `testdata/fuzz/`, и он превращается в обычный регрессионный кейс, который гоняется при каждом `go test`. Тридцати секунд обычно хватает, чтобы найти панику в парсере; в CI фаззинг запускают по расписанию, а не на каждый коммит.

Полезные флаги go test

Флаг	Что делает
`-run`	фильтр тестов по регулярному выражению
`-bench`	фильтр бенчмарков
`-benchmem`	добавляет `B/op` и `allocs/op`
`-count`	число прогонов для оценки разброса
`-race`	включает детектор гонок
`-shuffle=on`	случайный порядок тестов и пакетов
`-timeout`	предел времени на пакет

`-shuffle=on` стоит включить в CI хотя бы раз в неделю: он вскрывает тесты, которые проходят только благодаря порядку запуска и общему состоянию.

Типичные ошибки

1. **Тест проверяет реализацию, а не поведение.** Моки на каждый вызов ломаются от любого рефакторинга; тест должен фиксировать контракт — код ответа, тело, побочный эффект.
2. **Хелпер без `t.Helper()`.** Ошибка указывает внутрь хелпера, и в большом пакете непонятно, какой тест упал.
3. `defer` **вместо** `t.Cleanup` **в хелпере.** `defer` срабатывает при выходе из хелпера, то есть до конца теста: ресурс закрывается раньше, чем тест им пользуется.
4. **Бенчмарк с подготовкой внутри цикла.** `b.N`-цикл меряет генерацию данных вместе с кодом; результат зависит от машины и вводит в заблуждение.
5. **Оценка по** `ns/op` **без** `-benchmem`**.** Наносекунды шумят, а аллокации стабильны; оптимизация без счёта аллокаций часто делает хуже.
6. `-race` **только локально.** Гонка в редкой ветке не воспроизводится, а в CI с параллельными тестами всплывает; детектор должен быть в пайплайне.
7. **Погоня за 100% покрытия.** Время уходит на тесты геттеров, а необработанная ошибка в середине остаётся незамеченной.

Чек-лист

- [ ] Каждый тест-кейс — подтест с осмысленным именем, фильтруемый через `-run`.
- [ ] Все хелперы начинаются с `t.Helper()`, уборка — через `t.Cleanup`.
- [ ] Для хендлеров есть и юнит-тест на `httptest.NewRecorder`, и хотя бы один на `httptest.NewServer`.
- [ ] Бенчмарки запускаются с `-benchmem`, подготовка вынесена до `b.ResetTimer()`.
- [ ] `go test -race ./...` выполняется в CI на каждый коммит.
- [ ] Профили CPU и heap снимаются воспроизводимой командой и читаются через `-top` и `list`.
- [ ] У интеграционных тестов есть `-timeout` и запас на уборку через `t.Deadline()`.

Упражнения

1. Перепишите набор из пяти отдельных тестов в табличный и добавьте флаг `-run` для одного случая.
2. Найдите в своём коде функцию с аллокациями внутри цикла, снимите `-benchmem` до и после `sync.Pool`, сравните `allocs/op` через `benchstat`.
3. Снимите CPU-профиль теста и найдите в `-top` три самые дорогие функции; проверьте догадку командой `list`.
4. Внесите гонку в счётчик, поймайте её `-race`, затем исправьте тремя способами: мьютекс, атомик, канал. Сравните сложность.
5. Сделайте тест, который без `t.Deadline()` падает по общему таймауту, и почините его, выделив бюджет операции.

Ответы и разбор к этим упражнениям — в приложении D, часть 4.

Глава 36. Сборка и поставка

Go компилируется в один самодостаточный бинарник, и это меняет всю цепочку поставки: нет интерпретатора, нет виртуального окружения, нет `node_modules`. Но у этой простоты есть цена — размер, воспроизводимость и то, что версию в бинарник никто не положит за вас. В главе разбирается, как собрать статический бинарник, вшить в него версию и коммит, собрать под чужую архитектуру, сделать сборку воспроизводимой, упаковать в контейнер на десяток мегабайт и подписать результат.

Статический бинарник: CGO_ENABLED=0

По умолчанию `go build` на Linux линкует бинарник с libc, если в проекте есть cgo (например, `net` с системным резолвером или `os/user`). Такой бинарник зависит от версии glibc на хосте, и перенос в контейнер на Alpine даёт классическую ошибку `not found or not executable`, потому что musl и glibc несовместимы.

bash

```
CGO_ENABLED=0 go build -o bin/api ./cmd/api
file bin/api
ldd bin/api
```

text

```
bin/api: ELF 64-bit LSB executable, x86-64, statically linked, Go BuildID=...
    not a dynamic executable
```

`CGO_ENABLED=0` отключает cgo и заставляет стандартную библиотеку использовать чистые Go-реализации: DNS-резолвер читает `/etc/resolv.conf` сам, `os/user` разбирает `/etc/passwd`, вместо системного `getaddrinfo`. Компромисс: `/etc/nsswitch.conf` и экзотические источники имён (LDAP, SSSD) перестают работать, а разрешение имён идёт по `resolv.conf` с таймаутами Go, а не системными. Для Kubernetes этого достаточно; если нужна интеграция с корпоративным NSS — оставляйте cgo и собирайте на том же дистрибутиве, что и рантайм.

Исключение — драйверы с обязательным cgo (например, `github.com/mattn/go-sqlite3`). Тогда либо статическая линковка с musl, либо чистый Go-аналог `modernc.org/sqlite` (глава 34).

Версия в бинарнике: ldflags -X

Версия, которую видно в `api --version` и в логах старта, — первый вопрос при разборе инцидента. Вшивается она линкером в строковую переменную пакета `main`:

go

```
var (
    version = "dev"
    commit  = "none"
    date    = "unknown"
)

func main() {
    if len(os.Args) > 1 && os.Args[1] == "--version" {
        fmt.Printf("api %s (%s, %s)\n", version, commit, date)
        return
    }
    slog.Info("starting", "version", version, "commit", commit)
    // запуск сервера
}
```

Сборка подставляет значения вместо дефолтов:

bash

```
go build -trimpath -buildvcs=false \
  -ldflags "-s -w \
    -X main.version=$(git describe --tags --always --dirty) \
    -X main.commit=$(git rev-parse --short HEAD) \
    -X main.date=$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
  -o bin/api ./cmd/api
```

`-X` работает только со строковыми переменными и только если они не константы. Проверить, что попало в бинарник, можно без запуска:

bash

```
go version -m bin/api
```

text

```
bin/api: go1.26.10
    path    example.com/api/cmd/api
    mod example.com/api (devel)
    build   -trimpath
    build   CGO_ENABLED=0
    build   GOOS=linux
    build   GOARCH=amd64
```

```
    build   vcs.revision=3f9c1ab...
```

`go version -m` печатает модуль, зависимости и настройки сборки — этого часто достаточно, чтобы понять, из какого коммита собран образ. Дублирующая информация есть и в рантайме: `debug.ReadBuildInfo()` отдаёт `vcs.revision` и `vcs.time`, если сборка шла с `-buildvcs=true` (по умолчанию — да, если каталог под git).

Кросс-компиляция

Кросс-компиляция в Go не требует тулчейнов: стандартная библиотека собирается под целевую платформу тем же компилятором.

bash

```
GOOS=linux   GOARCH=arm64 CGO_ENABLED=0 go build -o bin/api-linux-arm64 ./cmd/api
GOOS=darwin  GOARCH=arm64 CGO_ENABLED=0 go build -o bin/api-darwin-arm64 ./cmd/api
GOOS=windows GOARCH=amd64 CGO_ENABLED=0 go build -o bin/api.exe ./cmd/api
```

GOOS	GOARCH	**Куда**
linux	amd64	x86-серверы, большинство облаков
linux	arm64	Graviton, Ampere, Raspberry Pi
darwin	arm64	Apple Silicon, ноутбуки разработчиков
windows	amd64	утилиты для админов

Ограничение ровно одно: с `CGO_ENABLED=1` кросс-компиляция требует кросс-линкера под целевую платформу, и это уже не «одна команда». Для `linux/amd64` полезно знать про `GOAMD64`: уровень `v1` — базовый и совместим со всем, `v2` добавляет SSE4.2 и POPCNT, `v3` — AVX2 и BMI2; собирать с `v3` в публичный образ нельзя: часть инстансов упадёт с `SIGILL`.

Воспроизводимые сборки

Воспроизводимая сборка (reproducible build) — та, что из одного коммита даёт побайтово одинаковый бинарник. Это позволяет проверить, что в реестре лежит именно собранное из исходников, а не что-то ещё.

bash

```
go build -trimpath -buildvcs=false -ldflags "-s -w" -o /tmp/api1 ./cmd/api
go build -trimpath -buildvcs=false -ldflags "-s -w" -o /tmp/api2 ./cmd/api
sha256sum /tmp/api1 /tmp/api2
go version -m /tmp/api1 | head -3
```

```
text
```

```
6d1f5b1e0c2f9a7...  /tmp/api1
6d1f5b1e0c2f9a7...  /tmp/api2
```

Три вещи ломают воспроизводимость: абсолютные пути в отладочной информации (лечится `-trimpath`), VCS-стамп с временем и грязным деревом (`-buildvcs=false` или сборка из чистого архива) и переменная `-ldflags "-X main.date=..."` с текущим временем. Последнее противоречие кажется неразрешимым, но решается передачей времени коммита: `date -u -d @$(git log -1 --format=%ct)`.

Размер бинарника и почему UPX вреден

Типичный HTTP-сервис на Go — 12–20 МБ. `-ldflags="-s -w"` убирает таблицу символов и DWARF и даёт минус 25–30%:

```
bash
```

```
go build -o /tmp/api-full ./cmd/api
go build -ldflags "-s -w" -o /tmp/api-stripped ./cmd/api
ls -lh /tmp/api-full /tmp/api-stripped
```

```
text
```

```
-rwxr-xr-x 1 dev dev 18M api-full
-rwxr-xr-x 1 dev dev 13M api-stripped
```

Дальше обычно вспоминают про UPX. Делать этого не стоит. UPX сжимает бинарник самомодифицирующимся распаковщиком: при старте он разворачивает код в память, из-за чего ломаются core dumps и профилирование, растёт потребление памяти (страницы становятся анонимными и не делятся между процессами), антивирусы в корпоративных контурах ругаются на упакованный ELF, а выигрыш в реестре почти нулевой — образ и так сжимается gzip при передаче, и сжатый UPX-бинарник весит примерно столько же. Экономия в 30% размера образа не окупает потерю диагностики в инциденте.

Multistage Dockerfile

Компилятор в образ не нужен: собираем в одном слое, копируем бинарник в пустой.

dockerfile

```
FROM golang:1.26-alpine AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -trimpath -buildvcs=false \
    -ldflags "-s -w -X main.version=${VERSION}" -o /out/api ./cmd/api

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/api /api
USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/api"]
```

`COPY go.mod go.sum` отдельным слоем до `COPY . .` — главный трюк: пока зависимости не менялись, слой с `go mod download` берётся из кэша, и пересборка после правки одной строки кода занимает секунды. `distroless/static` содержит корневые сертификаты и `/etc/passwd`, но не содержит шелла — это одновременно меньше поверхность атаки и меньше 5 МБ. Если нужен `scratch`, сертификаты копируют руками:

dockerfile

```
FROM scratch
COPY --from=build /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=build /out/api /api
ENTRYPOINT ["/api"]
```

buildx и несколько платформ

Обычный `docker build` собирает только под архитектуру хоста. Для мультиарх-образа нужен buildx с эмуляцией через QEMU:

bash

```
docker buildx create --name multi --use
docker buildx build --platform linux/amd64,linux/arm64 \
  --build-arg VERSION=1.4.2 \
  --cache-from type=gha --cache-to type=gha,mode=max \
  -t registry.example.com/api:1.4.2 --push .

docker buildx imagetools inspect registry.example.com/api:1.4.2
```

text

```
Name:      registry.example.com/api:1.4.2
MediaType: application/vnd.oci.image.index.v1+json
Manifests:
```

```
  Platform:    linux/amd64
    Digest:    sha256:3b1e...
  Platform:    linux/arm64
    Digest:    sha256:9f2c...
```

Сборка под arm64 на amd64-хосте идёт через эмуляцию и потому медленная; в CI её ускоряют нативными раннерами под каждую платформу. Результат — манифест-список, и `docker pull` сам выберет нужный образ. Тег `:latest` для продакшена не используют: деплоят по digest `@sha256:...`.

Подпись и SBOM в двух словах

Поставка не заканчивается сборкой образа. Минимум: сгенерировать SBOM (список компонентов) и подписать образ.

bash

```
syft registry.example.com/api:1.4.2 -o spdx-json > sbom.json
cosign sign --yes registry.example.com/api@sha256:...
cosign attest --yes --predicate sbom.json --type spdxjson
    registry.example.com/api@sha256:...
```

Подпись привязывается к digest, а не к тегу, поэтому подменить образ под тем же тегом незаметно не получится. Проверка в кластере — через admission policy (глава 43), которая отклоняет образы без валидной подписи.

Типичные ошибки

1. `CGO_ENABLED=1` **и базовый образ** `scratch`**.** Бинарник требует libc, которой в образе нет; контейнер падает с `no such file or directory` при старте. Собирайте статически или берите образ с libc.
2. **Версия только в теге образа.** Тег перезаписывается, и через месяц невозможно понять, что работало в инциденте; версия должна быть в бинарнике и в `go version -m`.
3. `-ldflags "-X"` **для не-строки.** Линкер молча не подставит значение в `int` или `const`, и в бинарнике останется дефолт.
4. **Сборка без** `-trimpath`**.** Пути вида `/home/ivan/work/api` попадают в бинарник: сборки не совпадают побайтово, а в стектрейсах светится структура каталогов разработчика.
5. **UPX «для экономии».** Ломаются core dumps и профилирование, растёт потребление памяти, а размер образа

в реестре почти не меняется.

6. `COPY . .` **до** `go mod download`**.** Любая правка кода инвалидирует слой с зависимостями, и сборка каждый раз качает модули заново.
7. `:latest` **в деплое.** Откат невозможен, потому что непонятно, на что откатываться; деплойте по digest.

Чек-лист

- [ ] `CGO_ENABLED=0` и проверка `ldd` показывают «not a dynamic executable».
- [ ] `--version` печатает версию, коммит и дату сборки.
- [ ] `go version -m` на готовом бинарнике показывает нужный `vcs.revision`.
- [ ] Сборка идёт с `-trimpath`, две сборки из одного коммита совпадают по `sha256sum`.
- [ ] Размер бинарника измерен, `-s -w` применён, UPX не используется.
- [ ] В Dockerfile `go.mod`/`go.sum` копируются до исходников.
- [ ] Образ собирается под `linux/amd64` и `linux/arm64`, деплой идёт по digest.
- [ ] Для образа есть SBOM и подпись, привязанная к digest.

Упражнения

1. Соберите сервис с `CGO_ENABLED=0` и с `CGO_ENABLED=1`, сравните `ldd`, размер и поведение при запуске в `scratch`.
2. Вшейте версию через `-X`, проверьте `--version` и `go version -m`; затем попробуйте подставить время коммита вместо текущего и добейтесь совпадения `sha256sum` у двух сборок.
3. Соберите мультиарх-образ через buildx и проверьте манифест командой `docker buildx imagetools inspect`.
4. Возьмите бинарник 15 МБ, примените `-s -w` и UPX, замерьте размер, потребление RSS при старте и возможность снять heap-профиль. Объясните результат.

5. Настройте в CI генерацию SBOM и подпись cosign, затем проверьте подпись командой `cosign verify` перед деплоем.

Ответы и разбор к этим упражнениям — в приложении D, часть 4.

Глава 37. Git: модель данных, ветки, rebase, хуки

Git изучают с команд — `commit`, `push`, `pull` — и поэтому он кажется набором ритуалов, которые иногда ломаются. На самом деле это маленькая объектная база с четырьмя типами объектов и указателями на них; почти любая «страшная» ситуация сводится к вопросу «на какой коммит сейчас смотрит ссылка». Дальше — модель объектов, merge и rebase, reflog, конфликты и хуки, которые не дают мусору попасть в историю.

Четыре типа объектов

Содержимое репозитория хранится в `.git/objects` как объекты, адресуемые по хешу содержимого. Типов четыре: **blob** — содержимое файла без имени и прав; **tree** — каталог со списком имён, режимов и хешей; **commit** — tree корня, автор, время, сообщение и родители; **tag** — аннотированная метка с сообщением.

Имя файла живёт в tree, а не в blob, поэтому одинаковое содержимое в двух каталогах — один объект. Внутренности смотрят без рабочих файлов:

bash

```
git cat-file -p HEAD
git cat-file -t HEAD
git cat-file -p HEAD^{tree}
git hash-object -w README.md
git rev-parse HEAD
```

text

```
tree 4b825dc642cb6eb9a060e54bf8d69288fbee4904
parent 8a1c0f3e2b7d9c4a5e6f708192a3b4c5d6e7f809
author Ivan Petrov <ivan@example.com> 1772000000 +0300
committer Ivan Petrov <ivan@example.com> 1772000100 +0300

Add retry to the HTTP client
```

Хеш — SHA-1 от заголовка объекта и его содержимого; формат выбирается при создании репозитория (`git init --object-format=sha256`), совместимости между форматами нет. Из неизменяемости следует главное: изменить коммит нельзя,

можно только создать новый. Вся «правка истории» — создание новых объектов и перестановка указателя.

Ссылки и HEAD

Ссылка (ref) — файл с хешем коммита: `.git/refs/heads/main`, `.git/refs/tags/v1.4.2`, `.git/refs/remotes/origin/main`. Старые упаковываются в `.git/packed-refs`. `HEAD` — символическая ссылка на текущую ветку:

bash

```
cat .git/HEAD
git rev-parse --abbrev-ref HEAD
```

text

```
ref: refs/heads/main
main
```

Отсюда «detached HEAD»: если `HEAD` содержит хеш, а не имя ветки, новые коммиты ни к какой ветке не привязаны; `git switch -c hotfix` из этого состояния сохраняет работу. Ветка — указатель, поэтому создание занимает микросекунды.

merge и rebase

Merge сохраняет обе линии истории и создаёт коммит с двумя родителями; rebase переносит коммиты на новую базу, создавая их заново с другими хешами.

bash

```
git switch feature/retry
git merge main          # коммит слияния, если история разошлась
git rebase main         # переписывает коммиты feature поверх main
git log --graph --oneline --decorate --all
```

text

```
* 9a0b1c2 (HEAD -> feature/retry) Extract backoff helper
* 5d6e7f8 Add retry to HTTP client
* 3c4d5e6 (main) Bump deps
```

	merge	**rebase**
История	сохраняет ветвление	линейная
Хеши коммитов	не меняются	меняются
Конфликты	один раз	могут повторяться
Общие ветки	безопасен	ломает чужие клоны

Правило: rebase — для своей локальной ветки до пуша, merge — для интеграции в общую. Переписанная общая ветка означает, что у коллег остались коммиты, которых нет в origin, и `pull` принесёт дубли. `git merge --no-ff` применяют, когда факт слияния ветки нужно сохранить в истории.

Интерактивный rebase и squash

Перед пушем коммиты приводят в порядок: склеивают «fix typo», переписывают сообщения, разбивают крупное изменение. `git rebase -i HEAD~4` открывает список:

```text
pick 1a2b3c4 Add retry to HTTP client
squash 5d6e7f8 Fix typo
reword 9a0b1c2 Extract backoff helper
drop 3c4d5e6 Debug print
```

`pick` оставляет коммит, `squash` склеивает с предыдущим и открывает редактор общего сообщения, `fixup` склеивает, выбрасывая сообщение, `reword` меняет текст, `edit` останавливает rebase для правки, `drop` удаляет. Ветка переписана и требует `git push --force-with-lease`: он откажется пушить, если в origin появились чужие коммиты, в отличие от `--force`.

cherry-pick

Бэкпорт хотфикса в релизную ветку:

```bash
git switch release/1.4
git cherry-pick -x 9a0b1c2
```

Флаг `-x` добавляет в сообщение строку `(cherry picked from commit 9a0b1c2)`, и происхождение правки видно через полгода. Cherry-pick создаёт копию с другим хешем: при последующем мерже ветки изменение приедет ещё раз.

revert вместо reset

`git revert` создаёт новый коммит, отменяющий изменения указанного; `git reset` двигает указатель ветки назад. Это разные операции по смыслу.

bash

```
git revert --no-edit 9a0b1c2   # безопасно для запушенной ветки
git reset --soft HEAD~1        # отменить коммит, оставив изменения в индексе
git reset --mixed HEAD~1       # отменить коммит и индекс, файлы целы
git reset --hard HEAD~1        # выбросить коммит И правки в рабочих файлах
```

`--hard` опасен тем, что удаляет незакоммиченные правки без возможности восстановления: reflog хранит коммиты, но не содержимое каталога.

bash

```
git stash push -m wip
git reset --hard HEAD~1
git stash pop
```

На общей ветке `reset` запрещён: у остальных останутся «удалённые» коммиты, и следующий push вернёт их.

reflog как спасение

Reflog — локальный журнал перемещений `HEAD` и веток: 90 дней для достижимых коммитов, 30 для недостижимых.

bash

```
git reflog
git log -g --oneline -n 5
```

text

```
9a0b1c2 HEAD@{0}: reset: moving to HEAD~1
1a2b3c4 HEAD@{1}: commit: Add retry to HTTP client
5d6e7f8 HEAD@{2}: checkout: moving from main to feature/retry
```

bash

```
git reset --hard HEAD@{1}
```

Этого хватает для большинства случаев «я всё потерял»: неудачный rebase, `reset --hard`, удалённая ветка. Если коммит не виден в reflog, работает `git fsck --lost-found`:

bash

```
git fsck --lost-found
git show --stat 3c4d5e6
git branch --contains 9a0b1c2
```

text

```
dangling commit 3c4d5e6b7a8f9e0d1c2b3a4f5e6d7c8b9a0f1e2d
```

Найденный хеш открывают через `git show` и возвращают в ветку `git branch rescue 3c4d5e6`.

Конфликты и их разрешение

Конфликт возникает, когда обе стороны изменили одни и те же строки. Git оставляет маркеры в файле:

```text
<<<<<<< HEAD
timeout: 5 * time.Second
=======
timeout: 10 * time.Second
>>>>>>> feature/retry
```

```bash
git status --short
git diff --name-only --diff-filter=U
git log --merge --oneline -p
```

```text
interactive rebase in progress; onto 3c4d5e6
You are currently rebasing branch 'feature/retry' on '3c4d5e6'.
  (fix conflicts and then run "git rebase --continue")

UU internal/client/client.go
```

`UU` значит «изменён обеими сторонами». После правки — `git add` и продолжение: `git rebase --continue` или `git merge --continue`. Полезны `git checkout --ours` / `--theirs` (взять сторону целиком) и `git rerere` — он запоминает разрешения и применяет их повторно при rebase той же ветки.

Хуки

Хуки — исполняемые скрипты, которые git вызывает на события. Каталог `.git/hooks` не клонируется, поэтому команда либо раскладывает хуки установочным скриптом, либо задаёт общий путь:

```bash
git config core.hooksPath .githooks
chmod +x .githooks/*
git config --get core.hooksPath
```

`.githooks/pre-commit` не даёт закоммитить неотформатированный Go:

```bash
```

```
#!/bin/sh
set -e
files=$(git diff --cached --name-only --diff-filter=ACM | grep '\.go$' || true)
[ -z "$files" ] && exit 0
unformatted=$(gofmt -l $files)
if [ -n "$unformatted" ]; then
    echo "gofmt required for:"
    echo "$unformatted"
    exit 1
fi
go vet ./...
```

`.githooks/commit-msg` требует номер задачи в первой строке:

bash

```
#!/bin/sh
head -1 "$1" | grep -qE '^[A-Z]+-[0-9]+: .+' || {
    echo "commit message must look like: API-142: add retry"
    exit 1
}
```

Хуки обходятся флагом `--no-verify`, поэтому они не заменяют CI, а экономят время: ошибка видна до пуша.

.gitignore и секреты в истории

`.gitignore` действует только на неотслеживаемые файлы. Если `.env` уже закоммичен, правило ничего не изменит: файл убирают из индекса (`git rm --cached .env`) и коммитят удаление. История при этом сохранит содержимое, и любой, кто клонирует репозиторий, получит секрет.

bash

```
git log --all -p -S 'AKIA' --oneline
git log --all --diff-filter=D --name-only -- '*.env'
```

Если утечка подтверждена: сначала ротация ключа, затем вычистка истории (`git filter-repo --path .env --invert-paths` или BFG), форс-пушнуть и попросить коллег переклонировать. Вычистка без ротации бессмысленна: ключ уже был в чужом клоне, в бэкапе и в логах CI.

Типичный рабочий цикл

bash

```
git switch main
git pull --ff-only
git switch -c feature/retry
git status --short
```

```
git add -p                  # по кускам, а не всё подряд
git commit -m "API-142: add retry with backoff to HTTP client"
git fetch origin
git rebase origin/main
go test ./... && go vet ./...
git push -u origin feature/retry
git push --force-with-lease
```

`pull --ff-only` не создаёт коммит слияния в локальной main и сразу сообщает о расхождении. `add -p` даёт осмысленные коммиты, а `rebase origin/main` перед пушем делает Pull Request линейным.

Типичные ошибки

1. `git reset --hard` **без сохранения работы.** Reflog восстанавливает коммиты, но не незакоммиченные правки — они теряются навсегда.
2. **Rebase запушенной общей ветки.** Хеши меняются, у коллег остаются осиротевшие коммиты, следующий `pull` приносит дубли.
3. `push --force` **вместо** `--force-with-lease`**.** Флаг без lease затирает чужие коммиты, появившиеся в origin после вашего fetch.
4. **Секрет в истории.** `.gitignore` не удаляет уже отслеживаемый файл, а вычистка без ротации ключа ничего не даёт.
5. **Хуки только локально.** `.git/hooks` не клонируется, у нового разработчика проверок нет — нужен `core.hooksPath`.
6. `git add .` **в спешке.** В коммит попадают отладочные файлы и локальные конфиги; `add -p` дешевле разбора.
7. **Merge-коммиты на общей ветке по умолчанию.** История заполняется «Merge branch main», и найти причину изменения труднее.

Чек-лист

- [ ] `git config core.hooksPath` указывает на версионируемый каталог с хуками.
- [ ] Хуки `pre-commit` и `commit-msg` исполняемые и проверены на `--no-verify`.
- [ ] В `.gitignore` есть `.env`, `bin/`, локальные конфиги; отслеживаемые секреты убраны из индекса.

- [ ] Force-push только через `--force-with-lease`.
- [ ] В shared-ветках нет rebase и `reset`.
- [ ] Сообщения коммитов содержат номер задачи, squash перед PR выполнен.
- [ ] `git log --all -S` по паттернам ключей не находит утечек.
- [ ] Pull Request — линейная ветка, `go test` и `go vet` зелёные до пуша.

Упражнения

1. Создайте коммит, посмотрите `git cat-file -p HEAD` и `HEAD^{tree}`; убедитесь, что `git hash-object` того же содержимого даёт тот же хеш.
2. Сделайте ветку с тремя коммитами, склейте их через `git rebase -i`, затем верните исходное состояние через `git reflog`.
3. Смоделируйте конфликт в двух ветках, разрешите его, включите `git rerere` и повторите rebase — конфликт должен разрешиться автоматически.
4. Напишите `pre-commit` с проверкой `gofmt` и `commit-msg` с номером задачи; проверьте обход через `--no-verify`.
5. Заведите коммит с фальшивым ключом, найдите его через `git log -S`, вычистите `git filter-repo` и проверьте, что объект исчез.

Ответы и разбор к этим упражнениям — в приложении D, часть 4.

Глава 38. Docker: слои, Dockerfile, compose

Контейнер — это не «лёгкая виртуальная машина», а обычный процесс на хосте, которому ядро ограничило видимость и потребление ресурсов. Понимание этого факта снимает половину проблем эксплуатации: непонятные OOMKilled, «данные пропали после перезапуска», «образ весит два гигабайта». В главе разбирается, как устроены образ и его слои, как писать Dockerfile, который не тащит в прод компилятор, как хранить данные и как описать весь стек в compose. После главы вы сможете собрать образ меньше 100 МБ с non-root пользователем и объяснить, почему правка одной строки обнулила кэш всей сборки.

Контейнер и виртуальная машина

Виртуальная машина эмулирует железо: гипервизор даёт гостевой ОС собственное ядро, собственный планировщик, собственные драйверы. Контейнер ядро не получает — он использует ядро хоста и изолирован двумя механизмами Linux, которые появились задолго до Docker.

Namespaces ограничивают видимость. Каждому контейнеру выдаётся свой набор пространств имён: `pid` (своё дерево процессов, внутри контейнера приложение видит себя как PID 1), `net` (свой сетевой стек, свои интерфейсы и таблица маршрутов), `mnt` (своя точка монтирования корневой файловой системы), `uts` (своё имя хоста), `ipc` (свои очереди SysV и разделяемая память), `user` (свой диапазон UID, что позволяет root внутри быть nobody снаружи), `cgroup` (своё представление иерархии), `time` (смещение часов).

cgroups ограничивают потребление. Контроллеры `cpu`, `memory`, `io`, `pids` считают и режут ресурсы: сколько ядер доступно, сколько памяти можно занять до OOM-kill, какая пропускная способность диска, сколько процессов разрешено. В cgroup v2 всё это —

единая иерархия файлов, где лимит памяти лежит в `memory.max`, а текущее потребление — в `memory.current`.

Свойство	Виртуальная машина	Контейнер
Ядро	своё, гостевое	общее с хостом
Запуск	десятки секунд	доли секунды
Размер образа	гигабайты	десятки-сотни мегабайт
Изоляция	сильная (аппаратная)	процессная, слабее
Плотность на хосте	единицы	десятки-сотни

Практическое следствие: контейнер не может иметь другое ядро, чем хост. Linux-контейнер на Windows или macOS работает внутри служебной виртуальной машины, которую Docker Desktop поднимает прозрачно, — и именно поэтому на macOS bind mount медленнее, чем на Linux.

Образ и слои

Образ (image) — неизменяемый стек слоёв только для чтения плюс конфиг: переменные окружения, рабочая директория, команда запуска, порт. Слой — это tar-архив с разницей файловой системы относительно предыдущего слоя. Инструкции `RUN`, `COPY` и `ADD` создают по слою каждая; `FROM`, `ENV`, `WORKDIR`, `CMD`, `ENTRYPOINT`, `EXPOSE`, `USER` меняют только конфиг и места не занимают.

При запуске поверх стека монтируется тонкий записываемый слой через overlayfs. Любая запись в файл из нижнего слоя не меняет его, а копирует файл наверх — механизм copy-on-write. Отсюда два следствия: удаление файла в следующем слое не уменьшает образ (файл остаётся в нижнем слое), а записанные внутрь контейнера данные живут ровно до `docker rm`.

bash

```
docker build -t shop-api:1.4.2 .
docker image ls shop-api
docker history --no-trunc shop-api:1.4.2 | head -20
```

`docker history` показывает размер каждого слоя — первое, что стоит смотреть, когда образ распух: строка `RUN apt-get install ...` на 400 МБ сразу объясняет, куда ушло место.

Dockerfile: инструкции, которые важно понимать

Минимальный рабочий Dockerfile для Python-сервиса выглядит так:

dockerfile

```
FROM python:3.14-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /app

RUN groupadd --system --gid 1001 app \
 && useradd --system --uid 1001 --gid app --no-create-home app

COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

COPY --chown=app:app . .

USER app
EXPOSE 8000

CMD ["python", "-m", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--port",
    "8000"]
```

Разберём по строкам. `FROM` задаёт базовый слой; `-slim` — это Debian без компиляторов и документации, около 130 МБ против 1 ГБ у полного образа. `ENV` пишет переменные в конфиг образа, а не в отдельный слой: `PYTHONUNBUFFERED=1` заставляет Python не буферизовать stdout, иначе логи появляются в `docker logs` с задержкой и теряются при падении контейнера. `WORKDIR` создаёт директорию и делает её текущей для последующих инструкций. `RUN groupadd/useradd` создаёт пользователя с фиксированным UID 1001 — от него зависят права на смонтированные тома.

`COPY requirements.txt ./` и `RUN pip install` стоят отдельно от `COPY . .` намеренно: зависимости меняются редко, код — часто. `USER app` переключает пользователя для следующих инструкций и для самого процесса. `EXPOSE 8000` — документация: наружу порт выводит только `-p`. `CMD` в exec-форме (JSON-массив) запускает процесс напрямую, без промежуточного `/bin/sh -c`.

CMD и ENTRYPOINT

`ENTRYPOINT` задаёт исполняемый файл, `CMD` — аргументы по умолчанию. Если в образе только `CMD`, то `docker run image bash` полностью заменит команду. Если задан `ENTRYPOINT ["python", "-m", "uvicorn"]`, то `docker run image app.main:app` добавит аргумент к entrypoint.

	`CMD`	`ENTRYPOINT`
Роль	команда или аргументы по умолчанию	фиксированный исполняемый файл
Переопределение	`docker run image <cmd>` заменяет целиком	аргументы дописываются
Типичное использование	самостоятельный образ	образ-обёртка, CLI-утилита

Главная ловушка — shell-форма. `CMD uvicorn app.main:app` выполняется как `/bin/sh -c "uvicorn ..."`, и PID 1 становится shell. Shell не пробрасывает SIGTERM дочернему процессу, поэтому `docker stop` ждёт 10 секунд и добивает контейнер SIGKILL: соединения рвутся, а не закрываются штатно. Всегда используйте exec-форму и убедитесь, что приложение обрабатывает SIGTERM (см. главу 28).

Кэш слоёв

Docker собирает образ последовательно и после каждой инструкции ищет в кэше слой с точно таким же родителем и точно такой же инструкцией. Совпало — слой берётся из кэша, не совпало — все последующие слои пересобираются. Кэш инвалидируется не только содержимым файла, но и его метаданными: `COPY . .` учитывает mtime и права.

Отсюда правило порядка: от самого стабильного к самому изменчивому. Сначала базовый образ, потом системные пакеты, потом манифест зависимостей, потом установка зависимостей, и только в конце — исходный код. Если поставить `COPY . .` перед `pip install`, то правка одного пробела в README пересоберёт все зависимости: полторы минуты вместо двух секунд.

Кэш сбрасывается `docker build --no-cache`, а в CI его надо выносить наружу (`--cache-to`/`--cache-from` с `type=registry` в BuildKit), иначе каждый запуск собирает образ с нуля.

Multi-stage сборка

Компилятор нужен, чтобы собрать бинарник, но не нужен, чтобы его запускать: multi-stage оставляет его в сборочном stage, а в финальный копирует только результат.

dockerfile

```
FROM golang:1.26-alpine AS builder

WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download

COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build \
    -trimpath -ldflags="-s -w" \
    -o /out/server ./cmd/server

FROM gcr.io/distroless/static-debian12:nonroot

COPY --from=builder /out/server /server
USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/server"]
```

Первый stage весит около 300 МБ и остаётся в кэше сборки, но не попадает в результат. Второй — distroless: только бинарник и сертификаты, без shell и пакетного менеджера, около 15–20 МБ. `CGO_ENABLED=0` даёт статически слинкованный бинарник, которому не нужна glibc; `-ldflags="-s -w"` убирает отладочные символы, экономя 20–30%. Отсутствие shell в образе — защита: если в приложение просочится RCE, выполнить `curl | sh` будет нечем.

Тот же приём работает для Python: stage `builder` собирает wheel-пакеты, финальный stage ставит их `pip install --no-cache-dir --no-index --find-links=/wheels` в чистый `python:3.14-slim`.

Non-root и .dockerignore

Процесс от root внутри контейнера — это root на хосте при монтировании каталога, при работе с сокетом Docker и при эксплойте ядра. Поэтому `USER` в Dockerfile обязателен. Исключение — образы, которые сами сбрасывают привилегии, но и там предпочтительнее unprivileged-вариант.

`.dockerignore` работает как `.gitignore`, но для контекста сборки. Без него `docker build .` отправляет демону весь каталог, включая `.git`,

`node_modules`, `venv` и локальные `.env`. Это замедляет сборку, ломает кэш (изменился `.git/index` — изменился слой) и рискует утащить секреты в образ.

text

```
.git
.gitignore
.venv
__pycache__
*.pyc
.pytest_cache
.mypy_cache
.env
.env.*
docker-compose*.yml
tests/
docs/
*.md
```

Данные: volumes и bind mounts

Файловая система контейнера эфемерна. Для данных есть три варианта.

Тип	Где живёт	Когда использовать
named volume	`/var/lib/docker/volumes/<name>`	данные БД, загрузки пользователей
bind mount	произвольный путь хоста	конфиги, разработка с live-reload
tmpfs	RAM	кэш, временные файлы, секреты

bash

```
docker volume create pgdata
docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:17-alpine
docker run --rm -it -v "$PWD":/src -w /src python:3.14-slim python script.py
```

Named volume управляется Docker и переживает `docker rm`. Bind mount связывает путь хоста: удобно в разработке, опасно в проде, потому что файлы приходят с чужими UID и правами, а SELinux на Fedora или RHEL потребует суффикс `:z`. Опция `:ro` должна быть по умолчанию для конфигов.

Сети

По умолчанию контейнер попадает в bridge-сеть `docker0` и получает адрес вида `172.17.0.1/16`. В дефолтном bridge нет DNS по

именам — контейнеры находят друг друга только по IP, что для compose непригодно. Поэтому создаётся пользовательская bridge-сеть: в ней работает встроенный DNS, и имя контейнера резолвится в его адрес.

bash

```
docker network create shop-net
docker run -d --name db --network shop-net postgres:17-alpine
docker run -d --name api --network shop-net -p 127.0.0.1:8000:8000 shop-api:1.4.2
```

Теперь `api` подключается к `db:5432`. `--network host` убирает изоляцию: контейнер видит порты хоста напрямую, `-p` игнорируется. Это быстрее, но занимает порт хоста и ломает модель «много реплик на одной машине». `--network none` — для задач без сети.

Публикуйте порты на `127.0.0.1`, а не на `0.0.0.0`, если перед контейнером стоит reverse proxy: иначе приложение окажется доступным напрямую, минуя TLS и rate limiting.

Compose: сервис, БД, healthcheck

Compose описывает несколько контейнеров как один проект: общая сеть, порядок запуска, тома, переменные окружения.

yaml

```
services:
  api:
    build: .
    image: registry.example.com/shop/api:1.4.2
    restart: unless-stopped
    environment:
      DATABASE_URL: postgresql://shop:${POSTGRES_PASSWORD}@db:5432/shop
      APP_ENV: production
    ports:
      - "127.0.0.1:8000:8000"
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request,sys;
          r=urllib.request.urlopen('http://127.0.0.1:8000/healthz', timeout=2);
          sys.exit(0 if r.status==200 else 1)"]
      interval: 10s
      timeout: 3s
      retries: 5
      start_period: 15s
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 384M
```

```
    volumes:
      - uploads:/app/uploads

  db:
    image: postgres:17-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: shop
      POSTGRES_USER: shop
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shop -d shop"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  pgdata:
  uploads:
```

Ключевые детали. `${POSTGRES_PASSWORD}` подставляется из `.env` в каталоге проекта; сам `.env` в git не попадает. `depends_on` с `condition: service_healthy` ждёт не запуска контейнера, а успешного healthcheck — иначе приложение стартует раньше, чем Postgres начнёт принимать соединения, и падает. `start_period` даёт время на инициализацию, не считая эти секунды ошибками. `deploy.resources.limits` работает и без Swarm — это те же cgroup-лимиты. `restart: unless-stopped` перезапускает упавший контейнер, но не остановленный руками.

Ресурсные лимиты

bash

```
docker run -d --name api \
  --memory 384m --memory-swap 384m \
  --cpus 1.5 --pids-limit 256 \
  shop-api:1.4.2
```

`--memory` задаёт жёсткий лимит: при превышении ядро убивает процесс с кодом 137 (128 + SIGKILL 9). `--memory-swap`, равный `--memory`, запрещает swap — иначе контейнер «уходит в swap» и деградирует вместо быстрого падения. `--cpus 1.5` — это квота, а не привязка к ядрам; для привязки есть `--cpuset-cpus`.

Лимит памяти бесполезен, если приложение о нём не знает. JVM до недавних версий считала heap от памяти хоста; `-XX:MaxRAMPercentage=75` решает проблему, но только при корректном

cgroup-лимите. Go-рантайм читает `memory.max` сам. Python с NumPy или большим кэшем нужно ограничивать на уровне приложения, иначе OOM-kill придёт внезапно.

Типичные ошибки

1. **Гигантский последний слой.** `COPY . .` после установки зависимостей кладёт в образ `.git`, тесты и venv. Причина — отсутствие `.dockerignore` и неверный порядок инструкций; лечится и тем, и другим.
2. **Секреты в образе.** `ENV DATABASE_PASSWORD=...` или `COPY .env .` оставляет секрет в слое навсегда: он виден в `docker history` и в registry, даже если следующей инструкцией файл удалён. Секреты передаются в рантайме через переменные окружения, файлы-монтирования или секрет-хранилище.
3. **Тег `latest`.** `FROM python:latest` и `image: shop-api:latest` означают, что завтрашняя сборка даст другой результат, а откат невозможен — откатывать некуда. Фиксируйте минорную версию базы и immutable-тег приложения (хэш коммита или semver).
4. `CMD` **в shell-форме.** Процесс не получает SIGTERM, `docker stop` всегда ждёт таймаут и убивает SIGKILL. Причина — промежуточный `/bin/sh`, который не форвардит сигналы.
5. **Данные внутри контейнера.** Запись в `/app/uploads` без volume: после `docker compose up --force-recreate` файлов нет. Контейнер — не диск.
6. `RUN apt-get update` **отдельной строкой.** Кэш списка пакетов устаревает, и `apt-get install` падает на 404. `update` и `install` должны быть в одном `RUN` через `&&`, а в конце — `rm -rf /var/lib/apt/lists/*`.

Чек-лист

- [ ] В Dockerfile есть `USER` с не-root пользователем, и процесс запускается от него.
- [ ] `CMD`/`ENTRYPOINT` в exec-форме, приложение обрабатывает SIGTERM.
- [ ] `.dockerignore` исключает `.git`, `.env`, тесты, кэши.

- [ ] Зависимости копируются и ставятся до копирования исходного кода (кэш работает).
- [ ] Образ собран multi-stage, в финальном слое нет компилятора и пакетного менеджера.
- [ ] Тег образа — версия или хэш, не `latest`; базовый образ зафиксирован по минорной версии.
- [ ] Данные, которые должны пережить перезапуск, лежат в named volume.
- [ ] `docker compose config` проходит без ошибок, у зависимых сервисов есть healthcheck и `depends_on: condition: service_healthy`.

Упражнения

1. Соберите образ по Dockerfile из раздела про инструкции. Посмотрите `docker history` и найдите три самых больших слоя. Уменьшите итоговый размер хотя бы вдвое, не меняя функциональность.
2. Добавьте в Dockerfile слой, который ставит `curl` и удаляет его в следующей инструкции. Убедитесь, что размер образа не уменьшился, и объясните почему.
3. Переведите Python-сервис на multi-stage сборку с wheel-пакетами. Сравните размер и время холодной сборки до и после.
4. Напишите compose-файл для сервиса, Postgres и Redis с healthcheck у всех трёх. Добейтесь, чтобы `docker compose up` поднимал стек без единого рестарта приложения.
5. Ограничьте контейнер 256 МБ памяти и добейтесь, чтобы приложение при переполнении завершалось с понятной ошибкой в логах, а не молча.

Ответы и разбор к этим упражнениям — в приложении D, часть 4.

Глава 39. CI/CD: пайплайны, кэш, артефакты

Пайплайн — исполняемая документация о том, как код превращается в сервис. Когда он есть, вопрос «какая версия сейчас в проде» имеет ответ в виде хэша коммита, а не воспоминаний дежурного. Когда его нет, каждый релиз — ручная операция, и цена ошибки растёт вместе с командой. В главе разбирается анатомия пайплайна: стадии, кэш, артефакты, матрицы, секреты, окружения и стратегии выката. После неё вы сможете написать workflow для Python и Go, который укладывается в десять минут и не теряет кэш между запусками.

Что такое пайплайн и из чего он состоит

Пайплайн (pipeline) — последовательность стадий, каждая из которых превращает вход в выход и может упасть. Минимальный набор: проверка стиля и типов, тесты, сборка артефакта, публикация образа, выкат, проверка после выката. Стадии состоят из jobs — единиц работы на отдельном исполнителе (runner). Jobs внутри стадии идут параллельно, стадии — последовательно, и переход возможен только при успехе предыдущей.

Пайплайн должен быть воспроизводимым: один коммит, прогнанный дважды, даёт один результат. Всё, что нарушает это свойство — незафиксированные версии зависимостей, «последняя» версия тула, состояние раннера от прошлого запуска, — превращает его в генератор случайных чисел.

Быстрый фидбек: порядок стадий

Разработчик узнаёт о поломке через минуты, а не через полчаса, только если дешёвые проверки стоят первыми. Линтер на 8 секунд должен падать раньше, чем тесты на 6 минут: нет смысла гонять интеграционные тесты, если код не компилируется.

Порядок — по возрастанию стоимости:

1. Синтаксис и форматирование (`ruff format --check`, `gofmt -l`) — секунды.
2. Статический анализ и типы (`ruff check`, `mypy`, `go vet`) — десятки секунд.
3. Юнит-тесты без внешних зависимостей — секунды-минуты.
4. Интеграционные тесты с базой и сетью — минуты.
5. Сборка, публикация, выкат, smoke-тесты — минуты.

Бюджет на pull request — до 10 минут: дольше — и люди начинают мержить «на глаз». Дорогие проверки (полный e2e, нагрузочные) выносятся на ночь или на релизную ветку.

Кэш зависимостей

Установка зависимостей — самая предсказуемая статья расходов: `pip install -r requirements.txt` на чистом раннере занимает 40–90 секунд, `go mod download` — 10–30, и это время умножается на число jobs и матричных комбинаций.

Кэш строится по ключу — обычно это хэш файла с точными версиями (`requirements.txt`, `poetry.lock`, `go.sum`). Пока хэш не меняется, раннер распаковывает архив с пакетами вместо скачивания. Изменилась одна зависимость — ключ другой, кэш промахнулся, пакеты качаются заново, зато старые версии не подмешаются.

В GitHub Actions кэш включается одной строкой у `setup-python` или `setup-go`, и это почти всегда правильнее, чем писать `actions/cache` руками:

```yaml
- uses: actions/setup-python@v5
  with:
    python-version: "3.14"
    cache: pip
    cache-dependency-path: requirements-dev.txt
```

Три правила, без которых кэш вредит. Не кэшируйте venv целиком: пути внутри архива абсолютные, и на другом раннере окружение окажется нерабочим — кэшируйте `~/.cache/pip` и `~/go/pkg/mod`. Не давайте кэшу из pull request перезаписать кэш

основной ветки, иначе форк подложит в него свой пакет. И задавайте срок жизни: кэш месячной давности чаще замедляет, чем ускоряет.

Артефакты между стадиями

Кэш — оптимизация, артефакт (artifact) — контракт. Артефакт — файл или набор файлов, который одна стадия передаёт другой: бинарник, wheel-пакет, Docker-образ, отчёт о тестах.

Принцип «собирай один раз» снимает целый класс расхождений. Если тесты гоняются на одном бинарнике, а в прод уезжает другой, собранный позже с другими флагами, проверяли вы не то, что выкатываете. Поэтому сборка идёт в отдельном job, а выкат берёт загруженный артефакт.

yaml

```
- uses: actions/upload-artifact@v4
  with:
    name: server-linux-amd64
    path: bin/server
    retention-days: 14
```

yaml

```
- uses: actions/download-artifact@v4
  with:
    name: server-linux-amd64
    path: bin/
- run: sha256sum bin/server
```

Отчёты о тестах загружаются с `if: always()`, иначе при падении тестов отчёта не будет — а нужен он как раз тогда. Для образов артефакт — сам реестр: следующие стадии ссылаются на неизменяемый тег, а не пересобирают.

Матрица тестов

Матрица (matrix) — способ прогнать один job с разными параметрами, не дублируя YAML: версия языка, версия базы, операционная система.

yaml

```
strategy:
  fail-fast: false
  matrix:
    python: ["3.13", "3.14"]
```

```
    database: ["postgres:16-alpine", "postgres:17-alpine"]
```

Комбинации идут параллельно, и падение одной не отменяет остальные (`fail-fast: false`) — иначе вы увидите первую ошибку и не узнаете про остальные три. Матрица не должна разрастаться: 2 версии языка на 2 версии базы — это 4 job, а 5 на 5 — уже 25, и очередь съест всю экономию.

Секреты в CI

Секрет в CI — ключ доступа к инфраструктуре или учётные данные реестра. Хранится он в секрет-хранилище платформы (`secrets.*` в GitHub Actions), подставляется в переменные окружения шага и маскируется в логах. Маскирование не абсолютно: закодированное значение в лог попадёт.

Что закрывает большинство утечек:

- **Минимальные права токена.** `permissions: contents: read` в начале workflow. `GITHUB_TOKEN` с правами `write-all` — готовый вектор атаки через скомпрометированную зависимость.
- **OIDC вместо статических ключей.** Вместо долгоживущего `AWS_SECRET_ACCESS_KEY` workflow получает короткоживущий токен (`permissions: id-token: write`), и облако доверяет конкретному репозиторию и ветке. Утёкший токен живёт минуты, а не годы.
- **Никаких секретов в `pull_request` из форков.** Такой workflow доступа к секретам не получает, и это правильно. `pull_request_target` доступ даёт, но выполняет код форка в контексте базовой ветки — известный путь к краже секретов.
- **Секреты не передаются в аргументах командной строки.** `docker login -p $PASSWORD` виден в `ps` и в логах; используйте `--password-stdin`.

Окружения и продвижение

Окружение (environment) — среда, куда выкатывается артефакт: dev, stage, prod. Продвижение (promotion) означает, что в следующее окружение едет тот же артефакт, а не пересобранный заново.

```text
```

```
коммит → lint → test → build(образ :sha) → dev → stage → prod
                                              ↑       ↑       ↑
                                            авто    авто   ручное подтверждение
```

В GitHub Actions окружение задаётся блоком `environment`, к нему привязываются правила защиты: обязательные ревьюеры, запрет выката из чужих веток, свой набор секретов. Секреты прода не должны быть доступны job, собирающему образ из pull request.

Артефакт помечается неизменяемым тегом (хэш коммита, например `:a1b2c3d`), а `:1.4.2` — метка поверх него. Откатиться с `:latest` невозможно: непонятно, на что откатываться.

Стратегии деплоя

Стратегия	Простой	Ресурсы	Скорость отката	Когда применять
Recreate	есть, секунды-минуты	1x	быстрый	dev, внутренние сервисы
Rolling	нет	1x + запас	средний	большинство stateless-сервисов
Blue-green	нет	2x	мгновенный	критичный прод, есть бюджет
Canary	нет	1x + доля	быстрый	рискованные изменения, много трафика

Recreate останавливает старую версию и запускает новую: дёшево, но между остановкой и стартом сервис недоступен, а незавершённые запросы обрываются.

Rolling заменяет экземпляры по одному: подняли новый, дождались готовности, погасили старый. Требует совместимости версий: реплики работают параллельно и видят одну базу.

Blue-green держит два одинаковых окружения и переключает трафик целиком: не подошло — вернули за секунды. Цена — двойные ресурсы и общее состояние (база, миграции).

Canary отправляет на новую версию 1–5% трафика и смотрит на ошибки и задержки. Ловит деградацию, которую не поймали тесты, но требует метрик по версиям и автоматики отката.

Откат

Откат (rollback) — не скрипт, а свойство процесса. Он работает при трёх условиях: артефакт предыдущей версии существует, конфигурация версионирована, изменения схемы совместимы в обе стороны.

Первое обеспечивают неизменяемые теги и политика хранения в реестре: удалять старые образы можно, но не все и не сразу.

Второе — конфиги лежат в git и применяются тем же пайплайном (глава 40), а не правятся руками на сервере.

Третье — самое болезненное. Миграция «удалить колонку» делает откат кода невозможным: старый код её ждёт. Поэтому миграции делаются в два шага с релизом между ними: добавить новую колонку и писать в обе, через релиз перестать читать старую, затем удалить. Схема называется expand/contract.

bash

```
kubectl rollout undo deployment/api -n shop
kubectl rollout status deployment/api -n shop --timeout=120s
```

Пример: Python в GitHub Actions

yaml

```
name: ci

on:
  push:
    branches: [main]
  pull_request:

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

env:
  PYTHON_VERSION: "3.14"

jobs:
  lint:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ env.PYTHON_VERSION }}
          cache: pip
          cache-dependency-path: requirements-dev.txt
      - run: pip install -r requirements-dev.txt
```

```
      - run: ruff format --check . && ruff check .
      - run: mypy app

  test:
    runs-on: ubuntu-24.04
    strategy:
      fail-fast: false
      matrix:
        python: ["3.13", "3.14"]
    services:
      postgres:
        image: postgres:17-alpine
        env:
          POSTGRES_USER: shop
          POSTGRES_PASSWORD: shop
          POSTGRES_DB: test
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U shop -d test"
          --health-interval 5s
          --health-timeout 3s
          --health-retries 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
          cache: pip
          cache-dependency-path: requirements-dev.txt
      - run: pip install -r requirements-dev.txt
      - run: pytest -q --junitxml=report.xml --cov=app --cov-fail-under=80
        env:
          DATABASE_URL: postgresql://shop:shop@localhost:5432/test
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: reports-py${{ matrix.python }}
          path: report.xml
          retention-days: 7

  build:
    needs: [lint, test]
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-24.04
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
```

```
          tags: ghcr.io/${{ github.repository }}:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  deploy-stage:
    needs: build
    runs-on: ubuntu-24.04
    environment:
      name: stage
      url: https://stage.example.com
    steps:
      - uses: actions/checkout@v4
      - run: ./scripts/deploy.sh ghcr.io/${{ github.repository }}:${{ github.sha }}
        env:
          KUBE_CONFIG: ${{ secrets.KUBE_CONFIG_STAGE }}
```

Что здесь важно. `concurrency` отменяет предыдущий запуск на той же ветке: пока вы пушите третий коммит, первые два неинтересны. `if: github.ref == 'refs/heads/main'` не даёт собирать образ из pull request. `permissions` сужены до чтения кода и записи пакетов. Тег — `github.sha`, поэтому деплой ссылается на конкретный коммит. `cache-from/to: type=gha` кэширует слои Docker, `deploy-stage` опирается на окружение с секретом `KUBE_CONFIG_STAGE`; в прод-окружении тот же job потребует подтверждения.

Пример: Go в GitHub Actions

Go собирается быстрее и не требует виртуального окружения, но требует явной матрицы по платформам и `-race` в тестах.

yaml

```
name: go-ci

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: "1.26"
          cache-dependency-path: go.sum
      - run: go vet ./...
      - run: go test -race -covermode=atomic -coverprofile=cover.out ./...
      - run: go tool cover -func=cover.out | tail -1

  build:
    needs: test
```

```
    runs-on: ubuntu-24.04
    strategy:
      matrix:
        include:
          - goos: linux
            goarch: amd64
          - goos: linux
            goarch: arm64
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: "1.26"
          cache-dependency-path: go.sum
      - run: |
          CGO_ENABLED=0 GOOS=${{ matrix.goos }} GOARCH=${{ matrix.goarch }} \
          go build -trimpath -ldflags="-s -w" -o bin/server-${{ matrix.goarch }}
              ./cmd/server
      - uses: actions/upload-artifact@v4
        with:
          name: server-${{ matrix.goos }}-${{ matrix.goarch }}
          path: bin/server-${{ matrix.goarch }}
```

`-race` включает детектор гонок и замедляет тесты в 2–10 раз — приемлемая цена: гонка, пойманная в CI, на порядки дешевле, чем в проде. Кросс-компиляция в матрице даёт два артефакта из одного коммита.

Типичные ошибки

1. **Деплой без тестов.** Пайплайн из одного шага `docker build && kubectl apply` экономит десять минут и стоит одного инцидента в месяц. Причина — спешка при первом релизе, которая фиксируется в YAML навсегда.
2. **«Зелёный» пайплайн при падающих тестах.** `pytest || true`, `continue-on-error: true` на тестах или `allow_failure` в GitLab делают шаг незаметным. Через месяц никто не помнит, почему там стоит флаг, а тесты не проверяют ничего.
3. `latest` **в деплое.** Деплой по плавающему тегу означает, что откат невозможен, а «та же версия» на stage и prod — разные образы. Причина — удобство ручного `docker push` вне пайплайна.
4. **Пересборка на каждой стадии.** Каждый job собирает свой бинарник — и в прод уезжает не то, что тестировали. Лечится артефактом и `needs`.
5. **Секреты в логах.** `echo $TOKEN`, `set -x` вокруг команд с паролями, передача ключа аргументом. Маскирование в CI не спасает от

base64 и посимвольного вывода.

6. **Ручные правки на проде.** `kubectl edit` в обход git создаёт дрейф: пайплайн при следующем выкате затрёт правку, а причина инцидента останется невидимой. Любое изменение проходит через репозиторий.

Чек-лист

- [ ] Пайплайн запускается на pull request и на push в основную ветку.
- [ ] Дешёвые проверки (формат, линт) идут раньше тестов; общий бюджет PR — до 10 минут.
- [ ] Кэш зависимостей настроен по хэшу lock-файла, а не по имени ветки.
- [ ] Артефакт собирается один раз и передаётся между стадиями; в прод едет он же.
- [ ] Секреты берутся из хранилища платформы, токену выданы минимальные `permissions`.
- [ ] У окружений есть правила защиты; прод требует ручного подтверждения.
- [ ] Образы тегируются хэшем коммита, старые версии хранятся достаточно долго для отката.
- [ ] Откат проверен на практике, а не описан в вики.

Упражнения

1. Возьмите репозиторий без CI и добавьте workflow из трёх job: линт, тесты с Postgres-сервисом, сборка образа. Добейтесь, чтобы повторный запуск на неизменённом коде занимал вдвое меньше времени за счёт кэша.
2. Добавьте матрицу по двум версиям Python и намеренно сломайте тест только на одной. Убедитесь, что пайплайн показывает обе ветки, а не останавливается на первой.
3. Переведите деплой на схему dev → stage → prod с ручным подтверждением прода. Проверьте, что на всех трёх окружениях работает один и тот же тег образа.

4. Реализуйте canary-выкат: 10% трафика на новую версию, автоматический откат при росте доли 5xx выше 1% за пять минут.

Ответы и разбор к этим упражнениям — в приложении D, часть 4.

Глава 39a. Бэкапы и восстановление

Резервная копия существует не тогда, когда файл записан, а когда из него подняли сервис. Разница между «у нас есть бэкапы» и «мы восстанавливаемся за час» стоит нескольких инцидентов. В главе — как перевести требование бизнеса в RPO и RTO, чем логический дамп Postgres отличается от физического, как устроены restic и borg и почему проверка восстановления обязательна. В конце — скрипт с systemd-таймером и учения.

Бэкап — это не копия

Копия отвечает на вопрос «где лежат те же байты», бэкап — «из чего я подниму сервис». У бэкапа три свойства: полнота (база, файлы, конфиги, секреты, роли СУБД), изолированность (другая ФС, машина, учётка, аккаунт) и проверенность (восстановление выполнялось).

`cp -a /srv/data /srv/data.bak` не проходит ни по одному пункту: тот же диск, та же учётка, и никто не проверял, что из `data.bak` что-то поднимается.

Реплика тоже не бэкап: `DELETE` без `WHERE` приезжает на неё за миллисекунды, ошибка приложения повторяется на всех узлах, повреждение страницы копируется потоком WAL. Реплика спасает от отказа железа, но не от ошибки человека.

RPO и RTO: числа, о которых договариваются

RPO (recovery point objective) — сколько данных допустимо потерять: «два часа» значит, что после аварии в базе будут все транзакции, кроме последних двух часов. RTO (recovery time objective) — за сколько сервис снова начнёт обслуживать запросы; считается от обнаружения аварии до прохождения smoke-теста.

Согласуют их с владельцем продукта и в деньгах: час простоя магазина с оборотом 4 млн рублей в сутки — около 170 тысяч выручки, после чего «сделаем PITR» превращается в «PITR

нужен двум сервисам из двенадцати, остальным хватит ночного дампа».

Класс сервиса	RPO	RTO	Что требуется
Платежи, биллинг	≤ 5 мин	1 ч	потоковый WAL, PITR, прогретый стенд
Основной API	15 мин	2 ч	WAL + ежедневный дамп, IaC для подъёма
Внутренний инструмент	24 ч	8 ч	ночной дамп в объектное хранилище
Аналитика	7 дней	48 ч	недельный дамп

RPO задаёт частоту и способ, RTO — процедуру: два часа не уложить, если подъём окружения — двадцать шагов из вики, поэтому RTO меньше суток почти всегда означает инфраструктуру как код и учения. Числа «на глазок» и есть причина, по которой бэкапы не восстанавливаются.

Правило 3-2-1 и его современная версия

Классика: три копии (рабочая и две резервные), два разных носителя, одна копия вне площадки — чтобы одна причина не уничтожила всё: сбой массива, ошибка оператора, пожар в стойке, шифровальщик.

Современная формулировка добавляет два пункта — 3-2-1-1-0: одна копия неизменяемая (immutable) или офлайн, и ноль ошибок при проверке. Неизменяемость закрывает сценарий, которого в 3-2-1 нет: атакующий с доступом к учётке бэкапа удаляет все три копии. Способы:

- S3 с Object Lock в режиме compliance и учётка только на запись и чтение, без `s3:DeleteObject` (`prune` в таком бакете не работает);
- `borg serve --append-only`: сервер не даёт удалить архив, чистка идёт отдельным ключом с другой машины;
- офлайн-копия — лента или диск на время записи.

Что бэкапить и что не надо

Список объектов важнее выбора инструмента: половина провалов — забытые роли Postgres, отсутствующий `terraform.tfstate` или секрет, который «и так есть в Vault».

Объект	Бэкапить	Чем
PostgreSQL	да, логически и физически	`pg_dump`, `pg_basebackup` + WAL
Файлы пользователей	да	restic
`/etc`, конфиги nginx	да, но источник правды — git	git + restic для срочного отката
Секреты	да, отдельно и шифрованно	снапшот Vault, SOPS
Terraform state	да, обязательно	версионирование бакета + копия
Кэши, индексы, `venv`	нет	пересобираются за минуты
Логи	обычно нет	централизованное хранилище (глава 42)

Три метода: логический дамп, физический, снапшот

Логический дамп

`pg_dump` читает базу через обычное соединение и выгружает SQL или архив в своём формате. Дамп согласован: снимается в одном снапшоте транзакции, таблицы не разъезжаются по времени.

bash

```
pg_dump -h 127.0.0.1 -U backup -Fd -j 4 -f /var/backups/pg-dump/shop shop
pg_dumpall -h 127.0.0.1 -U backup --globals-only > /var/backups/pg-dump/globals.sql
createdb -h 127.0.0.1 -U postgres shop_restore
pg_restore -h 127.0.0.1 -U postgres -d shop_restore -j 4 \
  --no-owner --exit-on-error /var/backups/pg-dump/shop
```

`-Fd` — формат каталога, по файлу на таблицу, поэтому `-j 4` распараллеливает выгрузку. `--globals-only` выгружает роли и табличные пространства, которых в дампе базы нет: без них восстановление упадёт на первом `GRANT`. `--exit-on-error` обязателен: по умолчанию `pg_restore` печатает ошибки и возвращает нуль — выходит «успешная» база без половины таблиц.

Физический бэкап и WAL

`pg_basebackup` копирует файлы кластера, а `archive_command` непрерывно отправляет сегменты журнала предзаписи в архив. База плюс журналы дают восстановление на произвольный момент — PITR (point-in-time recovery).

bash

```
pg_basebackup -h 127.0.0.1 -U replicator -D /var/backups/pg/base -Fp -Xs -P -R
```

`-Xs` передаёт WAL вместе с копией, `-R` прописывает параметры старта; пользователю `replicator` нужны `REPLICATION` и строка в `pg_hba.conf`.

ini

```
archive_mode = on
archive_command = 'test ! -f /var/lib/pgwal/%f && cp %p /var/lib/pgwal/%f'
```

`test ! -f` не даёт перезаписать уже заархивированный сегмент: Postgres считает архив корректным по нулевому коду и повторяет вызов при ошибке. Физическая копия совместима только с той же мажорной версией — копию 17-й на 18-й не поднять.

Снапшот файловой системы

LVM, ZFS, btrfs и EBS-снапшоты копируют том за секунды. Для базы это crash-consistent копия: страницы могут быть в середине записи, и Postgres восстановится по журналу — но только если журнал на том же томе. Снапшот на том же массиве не резервная копия.

Метод	Время на 500 ГБ	RPO	Переносимость
`pg_dump`	1–3 ч	момент запуска	между мажорными версиями
`pg_basebackup` + WAL	минуты + журналы	секунды	только та же мажорная
снапшот ФС	секунды	момент снапшота	нужен тот же тип тома

restic: инкремент, дедуп, шифрование

restic разбивает поток на блоки переменного размера по содержимому: вставка байта в начало файла не сдвигает

последующие блоки, поэтому дедупликация работает и после правок, в отличие от `tar`. Репозиторий — pack-файлы, индексы, снапшоты и ключи; шифрование включено всегда (AES-256 с Poly1305).

bash

```
export RESTIC_REPOSITORY=s3:https://s3.example.com/backups/shop
export RESTIC_PASSWORD_FILE=/etc/restic/password
restic init
restic backup /srv/data /var/backups/pg-dump \
  --tag daily --exclude-caches --exclude-file=/etc/restic/exclude.txt
restic forget --keep-last 3 --keep-daily 7 --keep-weekly 4 --keep-monthly 12
restic prune
restic check --read-data-subset=5%
```

Дедуп виден в отчёте: первый прогон по 62 ГБ добавляет 1,2 ГиБ данных и занимает 413 МиБ, второй — 386 МиБ новых при 94 МиБ записанных. Ретенция — политики: `--keep-daily 7` оставляет последний снапшот за каждый из семи дней, `--keep-weekly 4` — за четыре недели, `--keep-monthly 12` — за год. `prune` переписывает pack-файлы — операция тяжёлая по вводу-выводу.

`check` проверяет структуру и хэши, `--read-data-subset=5%` читает 5% данных — единственный способ узнать, что провайдер не потерял объекты; полный `--read-data` раз в квартал. borg устроен похоже, но умеет `--append-only` на сервере, чего restic не даёт.

Расписание и ротация

Расписание выводится из RPO: 15 минут — журналы уходят непрерывно, 24 часа — таймер раз в сутки. Копия должна укладываться в окно: дамп на 5 часов при интервале 4 часа накладывается сам на себя, отсюда `flock`. Мониторинг строится не на ошибке, а на отсутствии успеха: скрипт пишет отметку времени, а внешняя проверка кричит, если отметке больше суток.

Проверка восстановления

Три уровня, и настоящим считается только третий. Первый — `restic check`: репозиторий цел, хэши сходятся. Второй — пробное восстановление и сверка: `pg_restore --list` видит все таблицы, `count(*)` по ключевым совпадает с продом.

Третий — учения (drill) на чистом окружении: новая машина, никаких прод-зависимостей, восстановление с нуля, запуск приложения, smoke-тест. Время до успешного ответа — измеренный RTO; если он больше заявленного, политика не выполнена.

bash

```
#!/usr/bin/env bash
# /usr/local/bin/restore-drill.sh — проверка восстановления на чистом хосте
set -euo pipefail
export RESTIC_REPOSITORY=${RESTIC_REPOSITORY:-s3:https://s3.example.com/backups/sho
    p}
export RESTIC_PASSWORD_FILE=${RESTIC_PASSWORD_FILE:-/etc/restic/password}
RTO_MIN=${RTO_MIN:-120}
start=$(date +%s)
work=$(mktemp -d /var/tmp/drill.XXXXXX)
trap 'pg_ctl -D "$work/pgdata" stop -m immediate >/dev/null 2>&1 || true; rm -rf
    "$work"' EXIT
log() { printf '%s %s\n' "$(date -u +%FT%TZ)" "$*" >&2; }

restic restore latest --target "$work" --include /var/backups/pg-dump
dump="$work/var/backups/pg-dump/shop"
[[ -f $dump/toc.dat ]] || { log "ОШИБКА: в снапшоте нет дампа shop"; exit 2; }

initdb -D "$work/pgdata" -U postgres >/dev/null
pg_ctl -D "$work/pgdata" -o "-p 55432 -k $work -c fsync=off" -l "$work/pg.log"
    start >/dev/null
createdb -h "$work" -p 55432 -U postgres shop_restore
pg_restore -h "$work" -p 55432 -U postgres -d shop_restore -j 4 \
  --no-owner --exit-on-error "$dump"
rows=$(psql -h "$work" -p 55432 -U postgres -d shop_restore -tAc 'select count(*)
    from orders')
log "восстановлено строк в orders: $rows"
(( rows > 0 )) || { log "ОШИБКА: таблица orders пуста"; exit 2; }

elapsed=$(( ($(date +%s) - start) / 60 ))
log "RTO drill: $elapsed мин при цели $RTO_MIN мин"
(( elapsed <= RTO_MIN )) || exit 3
```

Коды различимы не случайно: 2 — данные неполные, 3 — превышен RTO; дежурный сразу видит, «бэкап плохой» или «мы медленные». Учения проводят раз в квартал и пишут результат в журнал.

Шифрование и где хранить ключ

Шифрование на клиенте означает, что провайдер хранилища видит только шифротекст. Ключ живёт отдельно от репозитория: Vault, KMS, HSM, в крайнем случае конверт в сейфе (глава 43a).

Пароль restic нельзя сбросить: в репозитории его нет, без пароля данные — мусор. Копий ключа две, в разных местах, одна — не в той системе, где бэкапы. Ротация не перешифровывает данные: `restic key add` создаёт пароль, `restic key remove` удаляет старый.

Полный пример: скрипт и таймер

Каркас тот же, что в главе 29 (блокировка, проверки, логи, коды возврата), но хранение делегировано restic, а ротация — политике `forget`.

bash

```
#!/usr/bin/env bash
# /usr/local/bin/pg-backup.sh — запуск: pg-backup.sh daily|weekly
set -euo pipefail
export RESTIC_REPOSITORY=${RESTIC_REPOSITORY:-s3:https://s3.example.com/backups/sho
    p}
export RESTIC_PASSWORD_FILE=${RESTIC_PASSWORD_FILE:-/etc/restic/password}
MODE=${1:-daily}
DB=${DB:-shop}
DUMP_DIR=${DUMP_DIR:-/var/backups/pg-dump}
EXCLUDES=${EXCLUDES:-/etc/restic/exclude.txt}
LOCK_FILE=${LOCK_FILE:-/run/pg-backup/backup.lock}
excl=()
if [[ -r $EXCLUDES ]]; then excl=(--exclude-file="$EXCLUDES"); fi
log() { printf '%s %s\n' "$(date -u +%FT%TZ)" "$*" >&2; }
die() { log "ОШИБКА: $*"; exit 1; }

exec 9>"$LOCK_FILE"
flock -n 9 || die "бэкап уже выполняется"
install -d -m 0700 "$DUMP_DIR"
rm -rf "${DUMP_DIR:?}/${DB:?}" "$DUMP_DIR/globals.sql"
pg_dump -h 127.0.0.1 -U backup -Fd -j 4 -f "$DUMP_DIR/$DB" "$DB" \
  || die "pg_dump завершился с ошибкой"
pg_dumpall -h 127.0.0.1 -U backup --globals-only > "$DUMP_DIR/globals.sql" \
  || die "pg_dumpall завершился с ошибкой"
log "дамп готов: $(du -sh "$DUMP_DIR" | cut -f1)"

restic backup "$DUMP_DIR" /srv/data --tag "$MODE" \
  --exclude-caches "${excl[@]}" || die "restic backup"

if [[ $MODE == weekly ]]; then
    restic forget --keep-last 3 --keep-daily 7 \
        --keep-weekly 4 --keep-monthly 12 || die "restic forget"
    restic prune || die "restic prune"
    restic check --read-data-subset=5% || die "restic check"
    log "ретенция применена, репозиторий проверен"
fi

date -u +%FT%TZ > /var/lib/pg-backup/last-success
log "бэкап завершён"
```

`rm -rf "${DUMP_DIR:?}/${DB:?}"` — обязательная подстановка: `set -u` не спасает от пустой переменной. Отметка `last-success` пишется после всех проверок. Флаг `--exclude-file` добавляется только при читаемом файле: restic падает с ошибкой, если файла нет.

ini

```
# /etc/systemd/system/pg-backup.service
[Unit]
Description=Ночной бэкап PostgreSQL и /srv/data в restic
After=network-online.target postgresql.service
Wants=network-online.target
RequiresMountsFor=/srv/data

[Service]
Type=oneshot
User=backup
Group=backup
RuntimeDirectory=pg-backup
RuntimeDirectoryMode=0750
StateDirectory=pg-backup
Environment=RESTIC_REPOSITORY=s3:https://s3.example.com/backups/shop
ExecStart=/usr/local/bin/pg-backup.sh daily
TimeoutStartSec=4h
Nice=10
IOSchedulingClass=idle

# /etc/systemd/system/pg-backup.timer
[Unit]
Description=Ежедневный бэкап в 02:15

[Timer]
OnCalendar=*-*-* 02:15:00
Persistent=true
RandomizedDelaySec=20m
Unit=pg-backup.service

[Install]
WantedBy=timers.target
```

`RuntimeDirectory` создаёт `/run/pg-backup` с владельцем-сервисом — иначе `flock` упадёт на правах; `StateDirectory` даёт `/var/lib/pg-backup` для отметки. `TimeoutStartSec` убивает зависший бэкап, `RandomizedDelaySec` разводит серверы. Пароль в `Environment` не передаётся, только путь к файлу `0400`.

Типичные ошибки

1. **Бэкап на тот же диск.** Механизм: копия лежит рядом с данными, и отказ массива, `rm -rf` или шифровальщик уносят обе.

2. **Бэкап без проверки восстановления.** Механизм: нулевой код `pg_dump` читается как «бэкап есть», хотя проверена только запись; отсутствие ролей или расширений выяснится в аварии.
3. **Дамп без прав и владельцев.** Механизм: дамп снят суперпользователем, восстановление идёт под другой учёткой — `pg_restore` падает на `ALTER ... OWNER TO`. Лечится `--no-owner` и выгрузкой ролей.
4. **Несовместимая версия Postgres.** Механизм: `pg_dump` должен быть не старше сервера, физическая копия между мажорными версиями не переносится; после обновления кластера клиент в бэкап-контейнере остался старым, и бэкапы тихо перестали создаваться.
5. **Ключ шифрования рядом с бэкапом.** Механизм: пароль положили в тот же репозиторий «чтобы не забыть»; при потере хоста теряется и репозиторий.
6. **Тихий отказ расписания.** Механизм: юнит отключили, `Persistent` ничего не догнал, а алерта на «бэкапа нет сутки» нет.

Чек-лист

- [ ] RPO и RTO записаны и согласованы с владельцем продукта, интервал бэкапа не больше RPO.
- [ ] Три копии, два носителя, одна вне площадки; минимум одна неизменяемая или офлайн.
- [ ] В бэкап входят база, файлы, конфиги, секреты и Terraform state; кэши — нет.
- [ ] Для базы есть и логический дамп, и физическая копия с журналами, если RPO меньше часа.
- [ ] `restic check --read-data-subset` идёт по расписанию, полный `--read-data` — раз в квартал.
- [ ] Восстановление на чистом окружении проводили за последние 90 дней, время уложилось в RTO.
- [ ] Ключ лежит отдельно от репозитория, есть вторая копия и break-glass.

- [ ] Настроен алерт на отсутствие успешного бэкапа, а не только на ошибку.

Упражнения

1. Посчитайте RTO для базы 200 ГБ: `pg_dump` отдаёт 40 МБ/с, `pg_restore` принимает 25 МБ/с, канал до хранилища 100 Мбит/с. Сравните с восстановлением из `pg_basebackup` при копировании тома 300 МБ/с и объясните разницу.
2. Разверните локальный restic-репозиторий, сделайте три снапшота каталога с 2 ГБ файлов, меняя по 1% содержимого. Покажите через `restic stats --mode raw-data`, как работает дедупликация.
3. Настройте PITR: `pg_basebackup`, `archive_command`, затем удалите таблицу и восстановите кластер на момент за минуту до `DROP`.
4. Доработайте скрипт учений: сравнение `count(*)` по трём таблицам с продом, допуск на суточный прирост, отдельные коды для «данные не сходятся» и «превышен RTO».
5. Составьте политику для 12 сервисов: RPO/RTO, объём, ретенция, стоимость. Обоснуйте, почему для шести из них PITR не нужен.

Ответы и разбор к этим упражнениям — в приложении D, часть 5.

Глава 40. IaC: Terraform и Ansible

Инфраструктура, собранная руками в консоли облака, существует в единственном экземпляре и только в памяти того, кто её собирал. Воспроизвести её в другом регионе нельзя, откатить изменение нечем, а причина ночного инцидента выясняется сравнением скриншотов. Infrastructure as Code (IaC) переносит описание инфраструктуры в файлы, которые лежат в git, проходят ревью и применяются автоматически. В главе два инструмента с разной природой: Terraform описывает, какие ресурсы должны существовать, Ansible — каким должно быть состояние машин. После главы вы сможете написать конфигурацию с remote state и блокировками, playbook с handlers и шаблонами и объяснить, почему `plan` в CI важнее, чем `apply` на ноутбуке.

Декларативный и императивный подход

Императивный подход отвечает на вопрос «что делать»: создать каталог, если его нет, установить пакет, запустить сервис. Скрипт из десяти команд ломается на второй, если каталог уже существует.

Декларативный подход отвечает на вопрос «что должно быть»: три виртуальные машины такого типа, такой server-блок в конфиге nginx, установленный пакет. Инструмент сам вычисляет разницу между желаемым и текущим и выполняет недостающее.

Terraform декларативен по отношению к набору ресурсов: он строит граф зависимостей и создаёт недостающее. Ansible формально тоже декларативен, но применяется последовательно по машинам и задачам, поэтому в нём легко скатиться к императивному сценарию из `command` и `shell`.

	Terraform	Ansible
Модель	желаемый набор ресурсов	желаемое состояние хостов
Область	облако, сеть, DNS, кластеры	ОС, пакеты, конфиги, деплой
Состояние	хранит state-файл	без состояния, опрашивает хост
Порядок	граф зависимостей	порядок задач в playbook
Агент на хосте	не нужен	не нужен (SSH)

Terraform: провайдеры, ресурсы, state

Конфигурация Terraform состоит из блоков: `terraform` (версии и backend), `provider` (плагин, который умеет говорить с API), `resource` (объект, которым управляем), `variable`, `local`, `output`, `module`.

hcl

```
terraform {
  required_version = ">= 1.9"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
  backend "s3" {
    bucket         = "shop-tfstate"
    key            = "prod/storage/terraform.tfstate"
    region         = "eu-central-1"
    dynamodb_table = "tf-locks"
    encrypt        = true
  }
}

provider "aws" {
  region = "eu-central-1"
  default_tags {
    tags = local.tags
  }
}

variable "environment" {
  type        = string
  description = "Имя окружения"
  validation {
    condition     = contains(["dev", "stage", "prod"], var.environment)
    error_message = "Допустимы только dev, stage и prod."
  }
}

variable "vpc_id" {
```

```
  type = string
}

locals {
  name = "shop-${var.environment}"
  tags = {
    Project     = "shop"
    Environment = var.environment
    ManagedBy   = "terraform"
  }
}

resource "aws_s3_bucket" "uploads" {
  bucket = "${local.name}-uploads"
  tags   = local.tags
}

resource "aws_s3_bucket_versioning" "uploads" {
  bucket = aws_s3_bucket.uploads.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_public_access_block" "uploads" {
  bucket                  = aws_s3_bucket.uploads.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_security_group" "api" {
  name        = "${local.name}-api"
  description = "HTTPS для API"
  vpc_id      = var.vpc_id
  tags        = local.tags

  ingress {
    description = "HTTPS"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

output "uploads_bucket" {
  value       = aws_s3_bucket.uploads.bucket
  description = "Имя бакета для загрузок"
}
```

Здесь важно несколько вещей. `required_providers` фиксирует диапазон `~> 6.0`: патчи приезжают, мажорные изменения — нет. `default_tags` избавляет от копирования тегов в каждый ресурс — без тегов через полгода не понять, за что платит счёт. `validation` ловит опечатку `prd` на этапе `plan`. `aws_s3_bucket_public_access_block` вынесен отдельным ресурсом намеренно: так видно, что именно меняется.

State: что это и почему его не хранят в git

State — JSON-файл с соответствием между адресами в конфигурации (`aws_s3_bucket.uploads`) и реальными идентификаторами в облаке. Без state Terraform не знает, что бакет уже создан, и попытается создать второй.

В state попадают все атрибуты, включая чувствительные: пароли баз, приватные ключи, строки подключения. Это первая причина не коммитить его. Вторая — конкурентность: два инженера с разными копиями применят изменения независимо, один затрёт состояние другого, и Terraform «потеряет» ресурсы, создав дубликаты. Третья — конфликты слияния, которые невозможно разрешить осмысленно.

Поэтому state лежит в remote backend: S3 с шифрованием, GCS, Terraform Cloud — с блокировкой на время операции. Блокировка (locking) не даёт двум `apply` идти одновременно. В AWS исторически использовали DynamoDB-таблицу с ключом `LockID`; современные версии умеют блокировку через сам S3, но DynamoDB-вариант всё ещё распространён.

plan и apply

Рабочий цикл состоит из трёх команд.

bash

```
terraform init
terraform plan -out=tfplan
terraform apply tfplan
```

`init` скачивает провайдеры и инициализирует backend. `plan` сравнивает конфигурацию, state и реальное состояние облака и показывает, что будет создано (`+`), изменено (`~`) и уничтожено (`-`).

`apply tfplan` применяет ровно тот план, который вы видели, — даже если конфигурацию успели поменять.

Читать план нужно целиком, особенно строки с `-`. Случайное удаление выглядит как одна строчка `# aws_db_instance.main will be destroyed`, а последствия — потеря данных. `forces replacement` означает удаление и создание заново: для бакета это потеря содержимого, для балансировщика — смена IP.

Схема в команде: `plan` на каждый pull request с публикацией комментарием, `apply` — только из CI после мержа, с сохранённым артефактом плана.

Модули

Модуль — переиспользуемый набор ресурсов с входами (`variable`) и выходами (`output`). Модуль вызывается блоком `module`:

```hcl
module "network" {
  source = "../../modules/network"

  name = local.name
  cidr = "10.20.0.0/16"
  azs  = ["eu-central-1a", "eu-central-1b"]
}

module "storage" {
  source      = "../../modules/storage"
  name        = local.name
  environment = var.environment
  vpc_id      = module.network.vpc_id
}
```

Модуль `modules/network` внутри выглядит как обычная конфигурация: `variable "cidr"`, `resource "aws_vpc" "this"`, `output "vpc_id"`. Ссылка `module.network.vpc_id` создаёт неявную зависимость: storage не начнёт создаваться, пока сеть не готова.

Правила, которые делают модули полезными: одна задача и не больше 8–10 входных переменных; не оборачивайте в модуль каждый ресурс — модуль из одного бакета добавляет абстракцию без пользы; версионируйте модули тегом, иначе «последняя версия» сломает прод в чужой день.

Ansible: инвентарь и playbook

Ansible работает по SSH и не требует агента. Инвентарь (inventory) описывает хосты и группы, playbook — что на них должно быть.

yaml

```
all:
  children:
    web:
      hosts:
        web1.example.com:
        web2.example.com:
      vars:
        server_name: shop.example.com
        app_port: 8000
    db:
      hosts:
        db1.example.com:
      vars:
        postgres_version: "17"
```

Playbook — список play, каждый play применяется к группе хостов и состоит из задач.

yaml

```
---
- name: Настройка веб-серверов
  hosts: web
  become: true
  vars_files:
    - vars/secrets.yml
  handlers:
    - name: reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
  tasks:
    - name: Установить nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
        cache_valid_time: 3600

    - name: Разложить конфиг виртуального хоста
      ansible.builtin.template:
        src: templates/site.conf.j2
        dest: /etc/nginx/sites-available/shop.conf
        owner: root
        group: root
        mode: "0644"
        validate: nginx -t -c /etc/nginx/nginx.conf
      notify: reload nginx

    - name: Включить сайт
      ansible.builtin.file:
        src: /etc/nginx/sites-available/shop.conf
```

```
        dest: /etc/nginx/sites-enabled/shop.conf
        state: link
      notify: reload nginx

    - name: Открыть 443
      community.general.ufw:
        rule: allow
        port: "443"
        proto: tcp

    - name: Создать пользователя деплоя
      ansible.builtin.user:
        name: deploy
        shell: /bin/bash
        groups: www-data
        append: true

    - name: Разрешить деплой-пользователю перезапуск сервиса
      ansible.builtin.copy:
        dest: /etc/sudoers.d/deploy-restart
        content: "deploy ALL=(root) NOPASSWD: /bin/systemctl restart shop\n"
        mode: "0440"
        validate: visudo -cf %s
```

Шаблон `templates/site.conf.j2` — обычный Jinja2 с доступом к переменным инвентаря:

```jinja
server {
    listen 443 ssl;
    server_name {{ server_name }};

    ssl_certificate     /etc/letsencrypt/live/{{ server_name }}/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/{{ server_name }}/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:{{ app_port }};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Идемпотентность модулей

Идемпотентность (idempotency) — свойство операции давать тот же результат при повторном выполнении. `apt` с `state: present` либо ставит пакет, либо сообщает `ok`. `template` сравнивает контрольную сумму и перезаписывает файл, только если содержимое отличается.

Проблема в том, что `command` и `shell` идемпотентными не являются: они выполняются всегда и всегда помечают задачу как `changed`. Лечится это явными условиями:

yaml

```
- name: Инициализировать базу приложения
  ansible.builtin.command:
    cmd: /usr/local/bin/shop migrate
    creates: /var/lib/shop/.migrated

- name: Проверить версию
  ansible.builtin.command: /usr/local/bin/shop --version
  register: shop_version
  changed_when: false
  failed_when: shop_version.rc != 0
```

`creates` пропускает задачу, если файл уже существует, — типичный маркер выполненной миграции. `changed_when: false` для проверок, которые ничего не меняют. `failed_when` вместо `ignore_errors`: последний прячет ошибку навсегда, а первый позволяет описать точное условие отказа.

Handlers и перезапуск сервисов

Handler — задача, которая выполняется один раз, если хотя бы одна задача её уведомила. Так пять изменённых конфигов дают один перезапуск, а не пять. Срабатывают handlers в конце play, поэтому следующая задача увидит ещё старую конфигурацию; если это важно, вызывайте `meta: flush_handlers`.

`validate: nginx -t -c ...` — страховка: сломанный конфиг не попадёт на диск. `visudo -cf %s` для sudoers по той же причине: ошибка в файле лишает root-доступа.

Vault: секреты в Ansible

Пароли и ключи не должны лежать в git в открытом виде. `ansible-vault` шифрует файл целиком симметричным ключом:

bash

```
ansible-vault create vars/secrets.yml
ansible-vault edit vars/secrets.yml
ansible-playbook -i inventory.yml site.yml --ask-vault-pass
```

yaml

```
db_password: !vault |
```

```
$ANSIBLE_VAULT;1.1;AES256
623133653966623430613934643361633837643737646136336536343062313864336264366233613
613433366535396636353433363266653533376166613162 0a663537646436643839616531643561
633962653339663861663736326265393261663539653632626330303336303133386463353036303
343862666666613765 0a353638643435666633633964366338633066623234616432373231333331
6564
```

Ключ передаётся файлом `--vault-password-file` (права `0400`, в git не попадает). Пароль в командной строке виден в истории shell и в `ps`. Задачи с секретами помечаются `no_log: true`, иначе значение попадёт в вывод.

Когда Terraform, когда Ansible

Граница проходит по жизненному циклу. Terraform управляет сущностями провайдера: VPC, подсети, балансировщики, управляемые базы, DNS-записи, IAM-роли, кластеры Kubernetes. Ansible — состоянием внутри машин: пакеты, файлы, сервисы, пользователи, деплой.

Пересечение — виртуальные машины: Terraform создаёт её и передаёт в Ansible через inventory, конфигурацию внутри делает Ansible. Альтернатива — cloud-init и готовые образы (Packer), тогда Ansible нужен только для деплоя. Признак неверного выбора: `null_resource` с `local-exec`, вызывающий shell-скрипты, — попытка делать конфигурацию инструментом для инфраструктуры. Обратный признак — Ansible, создающий через API балансировщики: надёжно он этого не умеет, потому что не хранит состояние.

Типичные ошибки

1. **State в git.** Секреты попадают в историю навсегда, а параллельные `apply` затирают друг друга. Причина — простота первого запуска: локальный файл появился сам и был закоммичен «на всякий случай».
2. `apply` **без просмотра плана.** Одна строка `- aws_db_instance.main will be destroyed` в плане означает потерю базы. Причина — привычка запускать `apply -auto-approve` в цикле отладки.
3. **Два инженера с локальным state.** Terraform у одного не видит ресурсы, созданные другим, и создаёт дубликаты с другими именами; счёт за месяц удваивается. Лечится remote

backend с блокировкой.

4. `count` **вместо** `for_each`**.** При удалении второго элемента из списка `count = 3` Terraform пересоздаёт все последующие, потому что адреса `[0]`, `[1]`, `[2]` сдвигаются. `for_each` привязывает ресурс к ключу, и удаление одного не трогает остальные.
5. `shell`/`command` **вместо модулей в Ansible.** Задача всегда `changed`, идемпотентности нет, повторный прогон делает то же самое ещё раз. Причина — модуль для задачи не найден или лень читать документацию.
6. `ignore_errors: true` **и** `failed_when: false`**.** Ошибка маскируется, playbook зелёный, а сервис не настроен. Через месяц выясняется, что половина хостов конфигурировалась не так.
7. **Ручные правки после применения.** Изменение в консоли облака или на сервере создаёт дрейф (drift): следующий `plan` покажет расхождение, а `apply` его затрёт. Причина — отсутствие запрета на ручные изменения и доступа к консоли без аудита.

Чек-лист

- [ ] State в remote backend с шифрованием и блокировкой, в git только конфигурация.
- [ ] Версии Terraform и провайдеров зафиксированы диапазоном в `required_providers`.
- [ ] `plan` выполняется в CI на каждый pull request, `apply` — только из CI по сохранённому плану.
- [ ] У всех ресурсов есть теги проекта и окружения (через `default_tags` или `locals`).
- [ ] Модули используются там, где есть повторение, и версионируются.
- [ ] Playbook проходит повторный прогон с `changed=0` на неизменённой конфигурации.
- [ ] Секреты в Ansible зашифрованы vault, ключ не в git; задачи с секретами помечены `no_log`.

- [ ] Есть проверка конфигов перед применением: `terraform validate`, `ansible-playbook --check --diff`.

Упражнения

1. Опишите в Terraform бакет объектного хранилища с версионированием, запретом публичного доступа и тегами. Добейтесь, чтобы повторный `plan` показывал `No changes`.
2. Перенесите state из локального файла в S3 с блокировкой. Убедитесь, что второй параллельный `apply` завершается ошибкой блокировки, а не повреждает state.
3. Напишите playbook, который ставит nginx, раскладывает конфиг из шаблона и перезапускает сервис только при изменении. Проверьте идемпотентность повторным прогоном.
4. Добавьте в playbook секрет из vault и убедитесь, что при `-vvv` значение не попадает в вывод.

Ответы и разбор к этим упражнениям — в приложении D, часть 4.

Глава 41. Kubernetes: поды, деплойменты, сервисы, probes

Kubernetes решает одну задачу: поддерживать заданное состояние набора контейнеров на наборе машин, даже когда машины падают. Всё остальное — сервисы, деплойменты, probes — следствия из этого принципа. В главе разбираются объекты, которых достаточно для 90% рабочих задач: под, Deployment, Service, Ingress, ConfigMap, Secret, probes и requests/limits. Отдельно — диагностика через `kubectl` и четыре аварии: CrashLoopBackOff, ImagePullBackOff, OOMKilled и Pending. Манифесты соответствуют API, актуальному для поддерживаемых версий 1.35–1.37.

Зачем оркестратор

Один сервер с `docker compose` живёт, пока живы железо и Docker. Упал диск — сервис лежит, пока дежурный не поднимет. Нагрузка выросла вдвое — нужно вручную перенести контейнеры на вторую машину и проверить, что база видит новую реплику.

Оркестратор берёт на себя три вещи. Размещение: выбирает узел, где хватает CPU и памяти, с учётом меток, taint'ов и anti-affinity. Самовосстановление: перезапускает контейнеры, пересоздаёт поды на других узлах, если узел перестал отвечать. Декларативное управление: вы описываете желаемое состояние, контроллеры бесконечно приводят к нему текущее.

Цена — сложность: Kubernetes несёт собственный API, сетевую модель, планировщик и набор абстракций, которые надо понимать, чтобы не выстрелить в ногу. В проекте с одним сервисом он часто избыточен, в проекте с десятком сервисов — почти неизбежен.

Под и контейнер

Под (Pod) — минимальная единица планирования: один или несколько контейнеров, которые делят сетевой namespace и

тома. У пода один IP, контейнеры видят друг друга через `localhost`, запускаются на одном узле.

В продакшене поды не создают напрямую — ими управляет контроллер, иначе после падения узла никто не создаст замену. Для разовой отладки сети хватает `kubectl run debug --image=curlimages/curl:8.10.1 --restart=Never -- sleep infinity`.

Следствия из «один IP на под»: порты контейнеров внутри пода не могут конфликтовать, а sidecar обязан слушать другой порт; имя пода содержит случайный суффикс, поэтому обращаться к нему по имени нельзя.

Deployment и ReplicaSet

Deployment описывает желаемое состояние приложения: образ, число реплик, стратегию обновления, шаблон пода. Он создаёт ReplicaSet, а тот — поды. Новый шаблон даёт новый ReplicaSet, старый масштабируется вниз, а история ревизий остаётся — это и есть механизм отката.

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: shop
  labels:
    app.kubernetes.io/name: api
spec:
  replicas: 3
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app.kubernetes.io/name: api
  template:
    metadata:
      labels:
        app.kubernetes.io/name: api
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 1001
        fsGroup: 1001
        seccompProfile:
```

```
          type: RuntimeDefault
      containers:
        - name: api
          image: ghcr.io/example/shop-api:a1b2c3d
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8000
          envFrom:
            - configMapRef:
                name: api-config
            - secretRef:
                name: api-secrets
          resources:
            requests:
              cpu: 100m
              memory: 192Mi
            limits:
              memory: 256Mi
          readinessProbe:
            httpGet:
              path: /healthz
              port: http
            initialDelaySeconds: 3
            periodSeconds: 5
            timeoutSeconds: 2
            failureThreshold: 2
          livenessProbe:
            httpGet:
              path: /livez
              port: http
            initialDelaySeconds: 10
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3
          startupProbe:
            httpGet:
              path: /healthz
              port: http
            periodSeconds: 2
            failureThreshold: 30
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
---
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: shop
spec:
  type: ClusterIP
  selector:
    app.kubernetes.io/name: api
  ports:
    - name: http
      port: 80
      targetPort: http
```

Разберём ключевые поля. `selector.matchLabels` и `template.metadata.labels` должны совпадать: иначе ReplicaSet не найдёт ни одного пода и будет плодить новые. `image` с хэшем коммита вместо `latest` обязателен: при `IfNotPresent` узел с закэшированным образом оставит старую версию. `runAsNonRoot` роняет под на старте, если образ пытается запуститься от root.

Service: ClusterIP, NodePort, LoadBalancer

Service даёт стабильную точку входа поверх нестабильных подов: постоянное имя в DNS кластера, постоянный виртуальный IP и `Endpoints`, которые контроллер обновляет при каждом изменении набора подов.

Тип	Доступность	Когда использовать
ClusterIP	только внутри кластера	общение сервисов, база
NodePort	порт на каждом узле	отладка, свой балансировщик
LoadBalancer	внешний LB провайдера	публикация наружу
ExternalName	CNAME на внешний DNS	сервис вне кластера

`ClusterIP` — тип по умолчанию и правильный выбор для внутренних зависимостей: база не должна быть доступна через `NodePort`. `NodePort` открывает порт 30000–32767 на каждом узле. `LoadBalancer` создаёт облачный LB на каждый сервис — дорого, поэтому наружу публикуют один Ingress-контроллер.

Балансировка внутри Service не учитывает загрузку подов: kube-proxy раскидывает соединения, sticky sessions требуют `sessionAffinity: ClientIP`. Учёт HTTP — задача ingress или mesh.

Service находит поды по меткам, поэтому `kubectl get endpoints api` — быстрая проверка селектора. Пустой список объясняет 503 от ingress без единой ошибки в логах.

Ingress

Ingress — правила маршрутизации HTTP на уровне L7: хост, путь, TLS, сервис. Сам объект ничего не делает: нужен контроллер (ingress-nginx, Traefik), который читает объекты и настраивает прокси.

yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  namespace: shop
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - shop.example.com
      secretName: shop-tls
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  name: http
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

`pathType: Prefix` совпадает по сегментам: `/api` поймает `/api/v1/orders`, но не `/apiv2`. `tls.secretName` ссылается на Secret с сертификатом, который создаёт cert-manager. Аннотации специфичны для контроллера: при переносе между nginx и Traefik их придётся переписать.

ConfigMap и Secret

ConfigMap хранит конфигурацию, Secret — чувствительные значения. Технически Secret — тот же объект, но значения закодированы base64 и по умолчанию не шифруются: в etcd они лежат открыто, если не включено шифрование at-rest. Base64 — кодирование, а не защита.

bash

```bash
kubectl -n shop create configmap api-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=info
```

```
kubectl -n shop create secret generic api-secrets \
  --from-literal=DATABASE_URL="$DATABASE_URL"
```

Секреты из внешних хранилищ предпочтительнее: External Secrets Operator или Secrets Store CSI Driver создают Secret из значений Vault, так что в git не попадает ничего, а ротация делается в одном месте.

`envFrom` подставляет все ключи ConfigMap переменными окружения, но изменения не подхватываются без перезапуска подов. `volumeMount` обновляет файл в поде автоматически (с задержкой до минуты), однако приложение должно читать файл, а не окружение.

requests, limits и QoS

`requests` — сколько ресурсов планировщик резервирует для пода. `limits` — потолок: превышение CPU-лимита ведёт к троттлингу, памяти — к OOMKilled.

Правило: **requests определяют размещение, limits определяют выживание**. Заниженные requests — планировщик напихает на узел больше подов, чем он тянет. Лимит памяти, равный request, не даст использовать всплеск, и под будет убит при пике.

Из соотношения requests и limits Kubernetes выводит класс QoS:

Класс	Условие	Кого убивают первым
Guaranteed	requests = limits у всех контейнеров	последним
Burstable	requests < limits	по потреблению сверх requests
BestEffort	ничего не задано	первым

Базе и критичным сервисам нужен Guaranteed: одинаковые requests и limits. Веб-приложениям разумно Burstable с memory limit в 2–3 раза выше request. BestEffort допустим только для отладки.

Probes

Три проверки отвечают на три разных вопроса, и путать их — самая частая причина «странных» рестартов.

livenessProbe — «процесс жив?». Провал ведёт к перезапуску. Проверять надо сам факт ответа, а не доступность базы: иначе падение БД вызовет рестарт всех подов и полный отказ вместо деградации.

readinessProbe — «можно отправлять трафик?». Провал убирает под из Endpoints, но не перезапускает — это правильное место для проверки зависимостей: база недоступна, под не готов, трафик идёт на остальные.

startupProbe — «приложение уже загрузилось?». Пока она не прошла, остальные probes не выполняются. Нужна сервисам с долгим стартом: без неё liveness убьёт приложение на 30-й секунде инициализации, которая занимает минуту.

Бюджеты в манифесте выше: старт — до 60 секунд (30 попыток по 2), рестарт после 30 секунд провалов liveness, вывод из балансировки после двух провалов readiness с периодом 5 секунд.

Эндпоинты должны быть дешёвыми: проверка с тяжёлым запросом в базу каждые 5 секунд на 50 подах создаёт нагрузку больше, чем сам сервис.

Rolling update: maxSurge и maxUnavailable

При обновлении Deployment создаёт новый ReplicaSet и переносит реплики: `maxSurge` — сколько подов можно создать сверх желаемого, `maxUnavailable` — сколько может быть недоступно.

По умолчанию 25% и 25%: для трёх реплик один под недоступен, один создаётся дополнительно. С `maxUnavailable: 0` и `maxSurge: 1` обновление идёт без потери ёмкости, но требует свободных ресурсов — иначе новые поды уйдут в Pending.

bash

```
kubectl -n shop rollout status deployment/api --timeout=180s
kubectl -n shop rollout history deployment/api
kubectl -n shop rollout undo deployment/api --to-revision=3
```

Выкат «завис» почти всегда потому, что новый под не становится Ready: старые поды продолжают работать, и это правильно — плохая версия не должна вытеснить хорошую.

HPA в двух словах

HorizontalPodAutoscaler меняет `replicas` по метрике. В `autoscaling/v2` это CPU, память или внешние метрики.

yaml

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
  namespace: shop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 3
  maxReplicas: 12
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
```

HPA работает только при заданных `requests` по CPU: утилизация считается в процентах от request. `stabilizationWindowSeconds` не даёт масштабироваться вниз при коротком провале трафика.

Диагностика через kubectl

Порядок действий почти всегда одинаков:

bash

```bash
kubectl -n shop get pods -o wide
kubectl -n shop describe pod api-7d9f8c6b5-x2l4q
kubectl -n shop logs api-7d9f8c6b5-x2l4q --tail=100
kubectl -n shop logs api-7d9f8c6b5-x2l4q --previous
kubectl -n shop exec -it api-7d9f8c6b5-x2l4q -- sh
kubectl -n shop port-forward svc/api 8080:80
kubectl -n shop get events --sort-by=.lastTimestamp | tail -20
kubectl top pods -n shop
```

`get pods -o wide` показывает узел и IP. `describe` — главный инструмент: в конце есть `Events`, где видно, что решил планировщик и почему kubelet не смог запустить контейнер. `logs --previous` читает логи убитого контейнера — единственный способ увидеть причину падения.

Четыре аварии

CrashLoopBackOff. Контейнер стартует, падает и перезапускается с растущей паузой (10с, 20с, 40с… до 5 минут). Причина в приложении или конфигурации: нет переменной, неверная строка подключения, падение на миграции. `describe` покажет `Exit Code: 1`, `logs --previous` — стектрейс. Увеличивать `initialDelaySeconds` бессмысленно: проблема не в скорости старта.

ImagePullBackOff. Kubelet не может скачать образ: неверный тег, приватный registry без `imagePullSecrets`, истёкший токен. `describe` покажет `unauthorized` или `manifest unknown`. Проверять надо доступ, а не кластер.

OOMKilled. Контейнер превысил memory limit, ядро убило процесс, exit code 137. Причины: лимит меньше реального потребления, утечка, всплеск на большом запросе. Повышать лимит вслепую не стоит — смотрите `kubectl top pod` в динамике.

Pending. Под не размещён ни на одном узле: `Insufficient cpu`, `untolerated taint` или `didn't match node affinity`. Причины — нехватка ресурсов, метки, непривязанный PVC. Приложение не запускалось, искать ошибку в нём бессмысленно.

Типичные ошибки

1. `latest` **в образе.** `imagePullPolicy: IfNotPresent` означает, что узел с закэшированным `latest` не обновится, а откат некуда делать. Причина — удобство ручных пушей; лечится хэшем коммита.
2. **liveness проверяет базу.** Падение базы вызывает каскадный рестарт всех подов и полный отказ. Причина — желание «проверить всё»; зависимости проверяет readiness.
3. **Нет requests.** Планировщик считает под нулевым по ресурсам, напихивает их на узел, и при первой нагрузке всё

уходит в OOM и троттлинг. Причина — манифест «чтобы запустилось быстрее».

4. **limit = request для всех подряд.** Сервис не может использовать всплеск, а при малейшем превышении его убивают. Для stateless-сервисов разумнее Burstable.
5. **Секреты в ConfigMap.** Пароли в ConfigMap видны всем, у кого есть `get configmaps`, и попадают в `kubectl describe`. Причина — «Secret всё равно base64, какая разница».
6. `kubectl edit` **в проде.** Изменение живёт до следующего выката, потом исчезает; причина инцидента теряется. Всё идёт через git.

Чек-лист

- [ ] Образ зафиксирован по хэшу коммита, `imagePullPolicy` осознан.
- [ ] У всех контейнеров заданы requests и limits; для критичных — Guaranteed.
- [ ] Readiness проверяет зависимости, liveness — только живость процесса.
- [ ] Для медленно стартующих сервисов есть startupProbe.
- [ ] `maxUnavailable` не равен 100%, выкат не роняет ёмкость до нуля.
- [ ] Секреты не в ConfigMap и не в git; доступ к ним ограничен RBAC.
- [ ] Есть `PodDisruptionBudget`, чтобы drain узла не убил все реплики сразу.
- [ ] Манифесты применяются из git, а не через `kubectl edit`/`apply` руками.

Упражнения

1. Разверните Deployment с тремя репликами и Service типа ClusterIP. Убедитесь, что `kubectl get endpoints` содержит три адреса, затем сломайте селектор и объясните, что изменилось.

2. Настройте три probe для приложения, которое стартует 40 секунд. Добейтесь, чтобы без startupProbe поды рестартовали, а с ней — нет.
3. Задайте memory limit 64 МБ приложению, которое потребляет 100 МБ. Найдите в `describe` причину завершения и объясните разницу между exit code 137 и 1.
4. Сделайте rolling update с `maxUnavailable: 0`, `maxSurge: 1` на кластере без свободных ресурсов. Разберитесь, почему поды ушли в Pending, и предложите два способа исправить.

Ответы и разбор к этим упражнениям — в приложении D, часть 4.

Глава 41a. Helm и GitOps

В главе 41 манифесты применялись руками через `kubectl apply`. На одном сервисе это работает, на десяти сервисах в трёх окружениях — нет: одинаковые YAML расходятся по копиям, правки в кластере теряются, откатить «весь релиз» нечем. Дальше два инструмента и один принцип: Helm упаковывает манифесты в параметризованный чарт, Kustomize решает ту же задачу патчами, а GitOps делает git единственным источником правды. Примеры — полный чарт сервиса и Application для Argo CD.

Проблема сырых манифестов

Первый рефакторинг обычно выглядит так: `manifests/dev/api.yaml`, `staging/api.yaml`, `prod/api.yaml`. Файлы совпадают на 90%, различаются репликами, тегом образа и хостом в Ingress. Дальше кто-то правит лимиты в проде и забывает про staging, кто-то во время инцидента делает `kubectl edit` и не переносит изменение в git.

Три поломки. Копипаста: изменение в общей части нужно внести трижды, и одна из копий всегда отстаёт. Дрейф (drift): живое состояние расходится с git, потому что `kubectl edit`, `scale` и правки руками никем не отслеживаются. Нет единицы отката: `kubectl rollout undo` возвращает только Deployment, а релиз — это ещё Service, Ingress, ConfigMap и миграция.

Helm и Kustomize решают первую и третью проблемы, GitOps — вторую.

Helm: chart, шаблоны, values

Чарт (chart) — каталог с шаблонами и значениями (Helm 3, актуальная ветка на момент написания):

```
charts/shop-api/
├── Chart.yaml          # метаданные: имя, версия чарта, версия приложения
├── values.yaml         # значения по умолчанию
├── values-prod.yaml    # переопределения для прода
└── templates/
    ├── _helpers.tpl    # именованные шаблоны, не рендерятся сами
    └── app.yaml        # Deployment + Service + Ingress
```

`helm template` рендерит шаблоны локально и печатает YAML, ничего не отправляя в кластер, — это главный инструмент отладки. `helm upgrade --install` отправляет результат в API-сервер и записывает релиз (release) в Secret внутри namespace: имя, ревизию, отрендеренный манифест. Из этих записей работают `helm history` и `helm rollback`.

Чарт целиком: три файла

`Chart.yaml`:

```yaml
apiVersion: v2
name: shop-api
description: HTTP API магазина
type: application
version: 1.8.0
appVersion: "2.14.3"
kubeVersion: ">=1.35.0-0"
```

`values.yaml` — все ручки с осмысленными значениями по умолчанию:

```yaml
replicaCount: 3
image:
  repository: ghcr.io/example/shop-api
  tag: ""          # пусто → тег берётся из appVersion
service:
  port: 80
ingress:
  enabled: true
  className: nginx
  host: shop.example.com
  tlsSecret: shop-tls
resources:
  requests: {cpu: 100m, memory: 192Mi}
  limits: {memory: 256Mi}
extraEnv: []
```

`templates/app.yaml` — три объекта в одном файле, разделённые `---`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-api
  labels:
    app.kubernetes.io/name: shop-api
    app.kubernetes.io/instance: {{ .Release.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
```

```
  selector:
    matchLabels:
      app.kubernetes.io/name: shop-api
      app.kubernetes.io/instance: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app.kubernetes.io/name: shop-api
        app.kubernetes.io/instance: {{ .Release.Name }}
    spec:
      containers:
        - name: api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default
              .Chart.AppVersion }}"
          ports:
            - name: http
              containerPort: 8000
          {{- with .Values.extraEnv }}
          env:
            {{- toYaml . | nindent 12 }}
          {{- end }}
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
---
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-api
spec:
  selector:
    app.kubernetes.io/name: shop-api
    app.kubernetes.io/instance: {{ .Release.Name }}
  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: http
---
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ .Release.Name }}-api
spec:
  ingressClassName: {{ .Values.ingress.className }}
  tls:
    - hosts: [{{ .Values.ingress.host | quote }}]
      secretName: {{ .Values.ingress.tlsSecret }}
  rules:
    - host: {{ .Values.ingress.host | quote }}
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: {{ .Release.Name }}-api
                port:
                  name: http
{{- end }}
```

Разбор. `.Release.Name` — имя релиза из `helm upgrade`, а не из чарта: один чарт можно поставить дважды в разные namespace без конфликта имён. `image.tag | default .Chart.AppVersion` делает appVersion значением по умолчанию для тега, но в проде тег задаётся явно. `{{- with }}` пропускает блок, если список пуст, а минус в `{{-` съедает перевод строки слева, иначе в YAML остаётся пустая строка. `toYaml | nindent 12` вставляет структуру с отступом 12 пробелов — самый частый источник ошибок «неверный отступ в отрендеренном манифесте».

Окружения: values-prod.yaml

`values-prod.yaml` содержит только отличия:

yaml

```
replicaCount: 6
image:
  tag: "2.14.3"
resources:
  requests:
    cpu: 500m
    memory: 512Mi
  limits:
    memory: 1Gi
ingress:
  host: shop.example.com
```

Файлы применяются по порядку, побеждает последний: `-f values.yaml -f values-prod.yaml`. Вложенные словари сливаются по ключам, а списки заменяются целиком — это ломает `extraEnv`, добавленный в prod-файле: он не дополняет, а вытесняет базовый список.

Правило, которое экономит инциденты: **в values.yaml есть все ключи с рабочими значениями по умолчанию, env-файлы только переопределяют**. Тогда `helm template` без `-f` даёт валидный манифест, а не пустые поля.

helpers, условия, range

Повторяющиеся метки выносят в `templates/_helpers.tpl`. Файлы с подчёркиванием не рендерятся в манифесты, из них только берут `include`:

```
{{- define "shop-api.labels" -}}
app.kubernetes.io/name: {{ .Chart.Name }}
```

```
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end -}}
```

Использование в шаблоне:

```yaml
metadata:
  name: {{ include "shop-api.fullname" . }}
  labels:
    {{- include "shop-api.labels" . | nindent 4 }}
```

`include` возвращает строку, `template` печатает её на месте — поэтому в пайплайнах с `nindent` используется только `include`. Точка в конце (`include ... .`) передаёт текущий контекст, без неё внутри хелпера не будет `.Chart` и `.Values`.

Условия и циклы закрывают вариативность без копирования файлов: `{{- if .Values.ingress.enabled }}` убирает Ingress целиком, `{{- range .Values.extraEnv }}` разворачивает список, `required` валит рендер с понятным сообщением вместо пустого хоста.

Команды: template, diff, upgrade, rollback

```bash
helm lint ./charts/shop-api
helm template shop ./charts/shop-api -f values-prod.yaml | head -40
helm diff upgrade shop ./charts/shop-api -n shop -f values-prod.yaml
helm upgrade --install shop ./charts/shop-api -n shop --create-namespace \
  -f values-prod.yaml --atomic --wait --timeout 5m
helm history shop -n shop
helm rollback shop 4 -n shop --wait
```

`helm diff` — плагин, не часть Helm, но без него `upgrade` превращается в лотерею: он показывает, какие поля изменятся до применения. `--atomic` включает `--wait` и откатывает релиз, если поды не стали Ready за `--timeout`; без него неудачный выкат остаётся `deployed`, а половина объектов — в новой версии. `--wait` ждёт готовности Deployment и Job, но не проверяет, что приложение отвечает правильно: за это отвечает smoke-тест.

`helm rollback` возвращает манифест предыдущей ревизии. Важная деталь: он не пересобирает образ и не откатывает миграции БД. Если тег образа в values не менялся, откат вернёт тот же самый тег и ничего не изменит.

Версии: chart version и appVersion

Два разных поля, которые постоянно путают. `version` — версия чарта по semver, обязательна, увеличивается при любом изменении шаблонов или values. `appVersion` — версия приложения, строка в кавычках; Helm её нигде не использует, кроме `.Chart.AppVersion`, которую вы сами подставляете в тег.

Отсюда правило релиза: тег образа — неизменяемый (хэш коммита или semver), `version` чарта инкрементируется в том же коммите, `appVersion` совпадает с тегом. Публикация в OCI-реестр — `helm package` и `helm push shop-api-1.8.0.tgz oci://ghcr.io/example/charts`.

Почему Helm не любят и Kustomize

Претензия к Helm честная: шаблоны — это язык программирования на YAML, с циклами, условиями и ошибками, которые вылезают только при рендере. Отладка идёт через `--debug`, типов нет, `values.schema.json` пишут редко. Второе: чарт — ещё один артефакт, который надо версионировать и публиковать.

Kustomize идёт другим путём: базовые манифесты остаются обычным Kubernetes YAML в `base/`, а окружения — каталоги `overlays/prod/` с патчами (`patches`) и подменой тега (`images`). Шаблонов нет, есть наложение изменений на структуру.

Критерий	Helm	Kustomize
Изменения	шаблоны и values	патчи поверх base
Сложная логика	условия, range, хелперы	нет, только структура
Версионирование	`version` чарта, реестр	версия = коммит git
Отладка	`helm template --debug`	`kustomize build`
Когда выбирать	много похожих сервисов, публикация	несколько окружений одного сервиса

Они не конкуренты: Kustomize умеет подключать чарты (`helmCharts`), Argo CD поддерживает оба рендерера.

GitOps: git как источник правды

GitOps — это не инструмент, а правило: желаемое состояние кластера описано в git, и только там. Изменение в кластере

делается коммитом, а не `kubectl`. Из правила следуют три свойства.

История: `git log` по каталогу приложения — история деплоев, с автором и причиной. Откат: `git revert` возвращает и манифесты, и образ, и конфигурацию. Безопасность: кластер сам тянет изменения из git, поэтому CI не хранит kubeconfig и не имеет прав в продакшене — самая частая дыра в схеме «CI делает `kubectl apply`».

Механика — цикл сверки (reconcile loop): контроллер раз в несколько минут или по вебхуку из git сравнивает желаемое состояние с живым и применяет расхождения. Это и есть drift detection — отличие не «замечается» отдельным скриптом, а исчезает при сверке.

Argo CD: Application и ApplicationSet

Application — объект, который связывает каталог в git с namespace в кластере (Argo CD 2.x на момент написания):

yaml

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop-api
  namespace: argocd
spec:
  project: shop
  source:
    repoURL: https://github.com/example/shop-deploy.git
    targetRevision: main
    path: charts/shop-api
    helm:
      valueFiles: [values-prod.yaml]
  destination:
    server: https://kubernetes.default.svc
    namespace: shop
  syncPolicy:
    automated: {prune: true, selfHeal: true}
    syncOptions: [CreateNamespace=true]
```

`automated` включает автоприменение при расхождении. `selfHeal` возвращает украденные ресурсы к состоянию git — он и делает `kubectl edit` бессмысленным. `prune` удаляет объекты, исчезнувшие из git: без него удалённый Deployment остаётся в кластере навсегда, а с ним есть риск снести лишнее.

Когда приложений десятки, их не пишут руками — их генерирует ApplicationSet:

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: shop-api-envs
  namespace: argocd
spec:
  goTemplate: true
  generators:
    - list:
        elements:
          - {env: staging, namespace: shop-staging}
          - {env: production, namespace: shop}
  template:
    metadata:
      name: 'shop-api-{{ .env }}'
    spec:
      project: shop
      source:
        repoURL: https://github.com/example/shop-deploy.git
        targetRevision: main
        path: charts/shop-api
      destination:
        server: https://kubernetes.default.svc
        namespace: '{{ .namespace }}'
```

Генератор `list` — простейший; `git` создаёт Application на каждый каталог в репозитории, и добавление окружения сводится к созданию каталога.

Дрейф и откат через git revert

`argocd app diff shop-api` показывает расхождение между живым состоянием и тем, что рендерится из git: какие поля изменены, какие объекты лишние, каких не хватает. Пустой вывод — единственный надёжный признак, что кластер соответствует репозиторию.

Откат делается `git revert`, а не кнопкой в UI: `selfHeal` вернёт состояние git через минуты, поэтому «откат в Argo» живёт до следующей сверки. Последовательность: revert коммита, `Synced`, `argocd app diff`, smoke-тест.

Секреты в GitOps

Правило «всё в git» упирается в секреты: пароль в открытом виде — утечка, а без него чарт не рендерится. Решения: SOPS с age или KMS (шифрованный файл в git, расшифровка на стороне контроллера), Sealed Secrets (можно коммитить, расшифрует только контроллер), External Secrets Operator (в git ссылка на запись Vault). Подробно — в главе 43а.

Порядок выката: sync waves и hooks

Обычно Argo применяет объекты параллельно, и это ломает сценарии, где миграция должна пройти до нового пода. Порядок задаёт аннотация `argocd.argoproj.io/sync-wave`: объекты с меньшим номером применяются раньше, с одинаковым — параллельно.

```yaml
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "-1"
```

Для разовых действий (миграция схемы, прогрев кэша) есть хуки: Job с `argocd.argoproj.io/hook: PreSync` выполняется до применения манифестов, `PostSync` — после, `hook-delete-policy` убирает его за собой. Практика: миграция отдельным PreSync Job, Deployment в волне 0, Ingress — в волне 1.

Типичные ошибки

1. `latest` **или** `appVersion` **вместо тега.** Механизм: удобно не менять values, но `helm rollback` вернёт манифест с тем же тегом, и откат ничего не откатит.
2. `helm upgrade` **без** `--atomic`**.** Механизм: релиз помечается `deployed`, даже если поды не поднялись; часть объектов остаётся в новой версии, часть — в старой.
3. **Секреты в values.yaml.** Механизм: «потом зашифруем», а репозиторий уже склонирован, и история git хранит значение навсегда.
4. `kubectl edit` **при включённом** `selfHeal`**.** Механизм: правка живёт до следующей сверки, потом молча исчезает — это выглядит как «изменения не применяются».

5. `prune` **без разбора.** Механизм: удалили каталог из git или переименовали объект — Argo удалил ресурс, включая PVC с данными.
6. **Один Application на весь монорепозиторий.** Механизм: любой коммит перекатывает всё сразу, `sync waves` не помогают, откат одного сервиса тянет за собой остальные.

Чек-лист

- [ ] `version` чарта увеличивается в том же коммите, что и изменение шаблонов; тег образа неизменяем.
- [ ] В values.yaml есть все ключи с рабочими значениями, env-файлы только переопределяют.
- [ ] `helm lint` и `helm template` прогоняются в CI, диф чарта показывается в PR.
- [ ] Выкат идёт с `--atomic --wait --timeout`, откат проверен хотя бы раз.
- [ ] Секретов в git нет: SOPS, Sealed Secrets или External Secrets.
- [ ] После синка `argocd app diff` пуст, `selfHeal` включён, `prune` осознан.
- [ ] Порядок выката задан sync waves, миграции — отдельным PreSync Job.
- [ ] Откат описан как `git revert`, а не как кнопка в UI.

Упражнения

1. Соберите чарт из трёх файлов для своего сервиса, отрендерите `helm template` с разными values и сравните вывод: какие поля меняются, какие остаются.
2. Добавьте helpers для меток и `required` для хоста Ingress. Добейтесь, чтобы рендер без `ingress.host` падал с понятным сообщением.
3. Переведите те же манифесты на Kustomize (base + два overlay) и опишите, какие изменения дались проще, а какие — сложнее, чем в Helm.

4. Разверните Argo CD, заведите Application на свой репозиторий с `selfHeal: true`. Измените реплики через `kubectl scale` и объясните, что произошло.
5. Сделайте релиз из двух коммитов: в первом ошибка в образе, во втором исправление. Откатитесь через `git revert` и покажите в `argocd app history`, что состояние вернулось.

Ответы и разбор к этим упражнениям — в приложении D, часть 5.

Глава 42. Наблюдаемость: метрики, логи, трейсы, SLO

Наблюдаемость (observability) — это способность ответить на вопрос «почему сервис ведёт себя именно так» по внешним данным, не подключаясь отладчиком к живому процессу. Три столпа — метрики, логи и трейсы — отвечают на разные вопросы и стоят по-разному: метрики дешёвые и агрегированные, логи подробные и дорогие, трейсы связывают всё в один запрос. В главе разбираются типы метрик и почему среднее врёт, кардинальность и её цена, exposition format, рабочий PromQL, структурные логи, propagation через `traceparent`, а также SLI/SLO с error budget. Заканчивается глава алертами по симптомам и дашбордами на четырёх золотых сигналах.

Три столпа и цена каждого

Метрика — число с метками, посчитанное внутри процесса и отданное наружу. Метрики агрегируют: одна серия `http_requests_total` заменяет миллион строк лога. Поэтому они дешёвы в хранении (сэмпл — несколько байт плюс индекс) и хороши для трендов, алертов и ёмкостного планирования. Плохи для разбора единичного случая: по метрике видно, что 3% запросов медленные, но не видно, какие именно.

Лог — событие с полезной нагрузкой. Он отвечает на «что произошло с конкретным запросом», но объём растёт линейно от трафика. Трейс — дерево операций одного запроса с длительностями; он отвечает на «где именно ушло время», пересекая границы сервисов.

Столп	Вопрос	Объём	Retention
Метрики	сколько, как быстро растёт	~1–2 байта на сэмпл	15 дней — 1 год
Логи	что случилось с запросом	килобайты на событие	7–30 дней
Трейсы	где ушло время	~1 КБ на трейс, с sampling	3–14 дней

Выбор инструмента диктуется вопросом, а не модой: если нужен график — метрика, если нужен идентификатор запроса — лог, если нужен виновник задержки — трейс.

Метрики: три типа, которые нужны

Counter — монотонно растущее число: запросы, ошибки, байты. Уменьшаться не может (кроме рестарта процесса), поэтому по нему всегда берут `rate()` или `increase()`. Gauge — значение, которое ходит туда-сюда: занятая память, длина очереди, число соединений. Histogram — распределение наблюдений по бакетам: латентность, размер ответа.

Histogram в exposition format разворачивается в набор серий: `_bucket` с меткой `le` (меньше или равно), `_sum` и `_count`. Бакеты накопительные: значение в `le="0.1"` включает всё, что попало в `le="0.05"`. Отсюда главное правило — границы бакетов подбирают под SLO, а не «от 1 мс до 10 с»: если SLO 300 мс, нужен бакет на 0.3, иначе квантиль интерполируется через дыру в полсекунды.

Summary считает квантили внутри процесса и отдаёт готовые `{quantile="0.95"}`. Он точен на одной реплике и бесполезен при агрегации: сложить или усреднить квантили нескольких подов нельзя. Поэтому в кластере по умолчанию берут histogram.

Квантили: почему среднее врёт

Средняя латентность 120 мс звучит нормально, но может означать, что 99% запросов идут за 80 мс, а 1% висит по 4 секунды — и это те самые запросы, которые платят пользователи. Среднее скрывает хвост, а хвост и есть пользовательский опыт. Среднее по квантилям тоже неверно: среднее из p95 двух реплик — не p95 системы.

Бакеты дают приближение. `histogram_quantile(0.95, ...)` линейно интерполирует внутри бакета, куда попал ранг, поэтому при редких бакетах ошибка достигает десятков процентов. Практический вывод: квантиль из histogram — для алертов и трендов, точные перцентили — из логов или трейсов, где есть сырые длительности.

Кардинальность

Кардинальность — число уникальных комбинаций меток. Prometheus хранит отдельную серию на каждую комбинацию, и каждая серия стоит памяти, индекса и CPU при запросе. Метка `user_id` на сервисе с миллионом пользователей даёт миллион серий на одну метрику; вместе с `code` и `route` — миллионы. Классический сценарий: добавили `request_id` «для отладки», и через сутки Prometheus съел 32 ГБ и начал отдавать 502 на запросы дашборда.

Правило: метки — только те, у которых ограниченное и известное число значений (метод, код, роут из фиксированного списка, версия, регион). Всё, что похоже на идентификатор — в логи и трейсы. Топ по кардинальности смотрят через `promtool tsdb analyze`, а не запросом из дашборда: такой запрос сам по себе перегружает хранилище.

Exposition format и эндпоинт /metrics

Prometheus забирает (scrape) метрики по HTTP в текстовом формате. Клиентская библиотека отдаёт их на `/metrics`:

```
# HELP http_requests_total Total HTTP requests.
# TYPE http_requests_total counter
http_requests_total{job="api",route="/cart",code="200"} 184213
http_requests_total{job="api",route="/cart",code="503"} 27
# HELP http_request_duration_seconds Request latency.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{job="api",route="/cart",le="0.05"} 171004
http_request_duration_seconds_bucket{job="api",route="/cart",le="0.1"} 180991
http_request_duration_seconds_bucket{job="api",route="/cart",le="+Inf"} 184240
http_request_duration_seconds_sum{job="api",route="/cart"} 3371.4
http_request_duration_seconds_count{job="api",route="/cart"} 184240
```

Формат прост: `имя{метка="значение"} значение [timestamp_ms]`. Имя — `[a-zA-Z_:][a-zA-Z0-9_:]*`, метка — `[a-zA-Z_][a-zA-Z0-9_]*`. Метка `le="+Inf"` обязательна и равна `_count`. Один и тот же набор меток на одном эндпоинте нельзя отдать дважды — это ошибка парсинга, и скрейп целиком отбрасывается.

Проверка руками перед тем, как винить Prometheus:

bash

```
curl -s localhost:8000/metrics | grep -E '^(http_requests_total|# TYPE)'
curl -s -o /dev/null -w '%{http_code} %{time_total}\n' localhost:8000/metrics
```

Если `/metrics` отвечает 200, но занимает 2 секунды — проблема в приложении (метрики считаются на лету), а не в сборщике. Тяжёлые вычисления выносят в фоновый поток.

PromQL на реальных вопросах

Скорость запросов по роутам. `rate` считает прирост в секунду на окне и сам разбирается с рестартами и сбросами счётчика:

```
sum by (route) (rate(http_requests_total{job="api"}[5m]))
```

Доля ошибок — отношение двух агрегатов, а не среднее отношений:

```
sum(rate(http_requests_total{job="api",code=~"5.."}[5m]))
/
sum(rate(http_requests_total{job="api"}[5m]))
```

p95 латентности. Ключевое — `sum by (le)`: агрегировать надо бакеты, сохраняя границу, и только потом считать квантиль:

```
histogram_quantile(0.95,
  sum by (le, route) (rate(http_request_duration_seconds_bucket{job="api"}[5m]))
)
```

Пропавший сервис. `absent()` возвращает пустой результат, если серия ещё есть, и `1`, если её нет; именно поэтому алерт на «тишину» пишут через него, а не через `up == 0`, который сам исчезает вместе с целью:

```
absent(up{job="api"})
```

Насыщение по подам:

```
topk(5, sum by (pod) (rate(container_cpu_usage_seconds_total{namespace="shop",
    container="api"}[5m])))
```

Окно `[5m]` при scrape 15 с даёт 20 точек — минимум для устойчивого `rate`. Окно меньше четырёх интервалов даёт рваные графики и ложные пики: `rate` экстраполирует на краях.

Логи: структура важнее объёма

Текстовый лог `2026-10-29 12:01:33 ERROR something failed` не парсится без регулярки на каждый формат. Структурный лог — JSON или logfmt с фиксированными полями: время, уровень, сообщение, сервис, версия, `trace_id`, `request_id`, пользователь, длительность.

```
json
```

```
{"ts":"2026-10-29T12:01:33.412Z","level":"error","service":"api",
    "version":"a1b2c3d","msg":"payment failed",
    "trace_id":"4bf92f3577b34da6a3ce929d0e0e4736","request_id":"01J9Z3K7",
    "route":"/pay","status":502,"dur_ms":1830,"err":"upstream timeout"}
```

Уровни нужны, чтобы фильтровать до записи, а не после. INFO — бизнес-события, WARN — деградация без отказа, ERROR — запрос не выполнен. `DEBUG` в проде включают на время, иначе объём вырастет на порядок. Отдельно про `ERROR` на каждый 4xx: клиентские ошибки — не авария, иначе алерт на ERROR никогда не замолчит.

`request_id` генерируется на входе (edge) и пробрасывается дальше; `trace_id` приходит из заголовка, если он есть. Оба — ключ к сборке истории запроса из разных сервисов:

bash

```
jq -r 'select(.level=="error") | [.ts,.service,.request_id,.status] | @tsv'
    api.log | tail -20
```

Стоимость. Пять миллионов событий в сутки по 1.5 КБ — это ~7.5 ГБ в день, 225 ГБ в месяц до сжатия. Сжатие в Loki даёт порядок 10x, в Elasticsearch — 2–3x, но память и CPU индексации остаются. Поэтому тела запросов не логируют целиком, healthcheck отбрасывают на входе, а retention задают по потокам: ошибки — 30 дней, debug — 3 дня.

Трейсы: span, trace id и propagation

Трейс — дерево span'ов. У каждого span есть `trace_id` (общий на весь запрос), `span_id`, родитель, имя операции, время начала и длительность, атрибуты. Корневой span создаёт входной сервис, дочерние — вызовы БД, кэша, других сервисов.

Связь между сервисами держится на заголовке. Стандарт W3C Trace Context задаёт `traceparent` из четырёх частей: версия, `trace_id` (32 hex), `parent_id` (16 hex), флаги. `01` в конце означает «сэмплировать».

bash

```
curl -s -H 'traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01' \
     http://localhost:8000/pay -o /dev/null -w '%{http_code}\n'
```

Сервис обязан принять `traceparent`, использовать его `trace_id` для своих span'ов и передать дальше с новым `parent_id`. Если

библиотека не подключена, трейс рвётся на первом же хопе, и в Jaeger видно дерево из одной ветки — типичный симптом.

Sampling неизбежен: хранить все трейсы при 2000 rps — это гигабайты в час. Head-based (решение на входе, обычно 1–10%) дёшев, но может не поймать редкую ошибку. Tail-based (решение после сборки трейса) сохраняет все медленные и все с ошибкой — дороже по CPU, зато полезнее. Компромисс: 100% для ошибок и запросов дольше SLO, 1% для остального.

SLI, SLO, SLA и error budget

SLI (indicator) — измерение: доля успешных запросов, p95 латентности, свежесть данных. SLO (objective) — целевое значение SLI на периоде: 99.9% успешных за 30 дней. SLA — юридический документ с санкциями, и он всегда слабее SLO: обещать клиенту 99.9%, а внутри держать 99.95%.

Бюджет ошибок считается прямо. 30 дней — это 43 200 минут; 0.1% от них — 43 минуты 12 секунд полного отказа в месяц. 99.95% — 21 минута 36 секунд. 99.99% — 4 минуты 19 секунд, и это уже требует дежурства, автоматического отката и отказоустойчивости на уровне зон. Недельный бюджет при 99.9% — около 10 минут.

Error budget превращает надёжность в инженерную величину: пока бюджет цел, можно катить релизы и рисковать; бюджет сожжён — фичи замораживаются, команда занимается надёжностью. Без этого разговор «нам нужно надёжнее» бесконечен, потому что цель не имеет цены. SLO 100% при этом не ставят: он недостижим, обесценивает бюджет и создаёт «зелёные» графики при реальных сбоях.

Алерты: симптомы, причины и runbook

Алерт по причине — «CPU выше 80%». Причина может быть безвредной (прогрев кэша), а может быть следствием; главное — она не говорит, страдает ли пользователь. Алерт по симптому — «доля 5xx выше 1% пять минут» или «p95 выше 300 мс» — говорит прямо. Правило: страницу (page) получают только

симптомы, влияющие на пользователя или на скорое исчерпание бюджета. Всё остальное — тикет на утро.

yaml

```
groups:
  - name: api.slo
    interval: 30s
    rules:
      - alert: ApiErrorBudgetBurnFast
        expr: |
          (sum(rate(http_requests_total{job="api",code=~"5.."}[5m]))
           / sum(rate(http_requests_total{job="api"}[5m]))) > 14.4 * 0.001
        for: 2m
        labels:
          severity: page
          team: shop
        annotations:
          summary: "API сжигает месячный бюджет за час"
          runbook: "wiki/runbooks/api-5xx"
      - alert: ApiTargetDown
        expr: absent(up{job="api"})
        for: 5m
        labels:
          severity: page
        annotations:
          summary: "Prometheus не видит ни одной цели api"
```

Порог 14.4 — не магия: при SLO 99.9% бюджет 0.1%, поэтому 14.4x означает сгорание 2% месячного бюджета за час. Второй, медленный алерт ставят на 6x за 6 часов, чтобы ловить не пики, а устойчивую деградацию.

`for` обязателен: без него один пропущенный скрейп или всплеск при деплое будит человека. Пять минут — разумный минимум для page, кроме burn-rate алертов: там окно само длинное, и короткий `for` не ловит шум.

Runbook — не формальность. В нём: что означает алерт, какие три команды выполнить первыми, как откатить, кого звать. Если runbook нет, дежурный ночью изобретает его заново.

Дашборды: четыре золотых сигнала

Google SRE предлагает четыре сигнала, которых достаточно для любого сервиса: латентность (отдельно успешных и неуспешных запросов), трафик (rps), ошибки (доля и абсолютное число), насыщение (CPU, память, соединения, очередь — насколько близко к пределу).

Верхний ряд — SLO и error budget, второй — четыре сигнала, третий — ресурсы и зависимости. Дашборд не должен требовать скролла, чтобы понять «всё плохо или нет», а у каждой панели должен быть ответ на вопрос «какое значение означает проблему»: график без единиц и порога — украшение. Дашборды дежурного и разбирающего разные: первому нужны симптомы и текущее состояние, второму — детализация по роутам, версиям и подам.

Типичные ошибки

1. **Метка с идентификатором.** `user_id`, `request_id`, `session_id` в метках дают взрывную кардинальность: память растёт с числом пользователей, запросы замедляются, Prometheus падает по OOM. Причина — желание «отфильтровать конкретный запрос» инструментом, который для этого не предназначен.
2. **Средняя латентность в SLO.** Среднее скрывает хвост, поэтому деградация p99 не видна до полного отказа. Причина — привычка считать средние по умолчанию.
3. **Усреднение квантилей.** `avg(http_request_duration_seconds{quantile="0.95"})` даёт число, не означающее ничего. Причина — путаница между gauge и распределением.
4. **Алерт на причину, а не на симптом.** «CPU > 80%» горит при прогреве и молчит, когда сервис отдаёт 500 из-за ошибки в коде. Причина — метрику проще найти, чем сформулировать пользовательский симптом.
5. **Алерт без `for`.** Один пропущенный скрейп — страница в три ночи; через месяц дежурные начинают игнорировать алерты. Причина — страх пропустить событие, который приводит к пропуску всех.
6. **Логирование всего.** Тела запросов, healthcheck, debug в проде: счёт за хранение растёт быстрее трафика, а полезных записей не прибавляется. Причина — «пусть будет, пригодится».

7. **SLO 100%.** Недостижим технически, обесценивает бюджет и провоцирует «зелёные» графики при реальных сбоях. Причина — SLO пишут как обещание, а не как инструмент.

Чек-лист

- [ ] У сервиса есть `/metrics` с counter, histogram и gauge; сбор идёт не реже 30 с.
- [ ] Границы бакетов histogram соответствуют SLO, а не выбраны случайно.
- [ ] В метках нет идентификаторов; топ кардинальности проверен.
- [ ] Латентность отслеживается квантилями по бакетам, а не средним.
- [ ] Логи структурные, с `trace_id` и `request_id`, уровень настраивается без пересборки.
- [ ] `traceparent` принимается и пробрасывается во все исходящие вызовы.
- [ ] Для каждого SLO описан SLI, окно и владелец; error budget считается автоматически.
- [ ] Page-алерты — только по симптомам, у каждого есть runbook и владелец.

Упражнения

1. Поднимите Prometheus и приложение с `/metrics`. Напишите запрос доли 5xx за 5 минут и убедитесь, что при остановке приложения он даёт пустой результат, а не ноль. Объясните, почему для «сервис пропал» нужен `absent()`.
2. Добавьте в приложение histogram латентности с бакетами под SLO 300 мс. Сравните `histogram_quantile(0.95, ...)` с точным p95 по логам и оцените ошибку интерполяции.
3. Введите метку с `user_id` на тестовом стенде и снимите потребление памяти Prometheus до и после. Найдите момент, когда кардинальность становится проблемой.

4. Настройте два алерта на один SLO: быстрый (14.4x за 1 ч) и медленный (6x за 6 ч). Прогоните нагрузку, дающую 2% ошибок, и объясните, какой сработает и почему.
5. Соберите трейс из двух сервисов через `traceparent`. Сломайте propagation в одном из них и опишите, как изменится дерево в интерфейсе трейсинга.

Ответы и разбор к этим упражнениям — в приложении D, часть 4.

Глава 42a. Нагрузочное тестирование

Нагрузочный тест отвечает на вопрос, который продакшн-метрики задать не могут: что будет, если трафика станет вдвое больше. В работающем сервисе видна одна точка — текущая, и она почти всегда далека от предела, пока не случится распродажа. В главе разбирается, какие величины имеют смысл под нагрузкой, чем открытая модель отличается от закрытой, как ступеньками найти точку насыщения и как провести тест, не уронив прод. В конце — сценарий на k6 с порогами и таблица результатов для отчёта.

Зачем нагружать сервис

Целей три. Первая — узнать предел раньше пользователей: сколько запросов в секунду сервис держит при приемлемой латентности. Вторая — проверить SLO: «99.9% запросов быстрее 300 мс» проверяется не одним `curl`, а распределением на устойчивой нагрузке. Третья — найти точку насыщения (knee point) и заложить запас; без неё ёмкостное планирование превращается в гадание по графикам.

Четыре величины, которые измеряют

Пропускная способность (throughput) — сколько запросов сервис обрабатывает в секунду. Это не то, что отправлено: при перегрузке генератор шлёт 300 rps, а сервис отвечает на 180, и 180 и есть его пропускная способность.

Латентность измеряется квантилями: p50 (типичный запрос), p95, p99, реже p99.9. p50 отвечает на «нормально ли большинству», p99 — «что почувствует человек, которому не повезло». SLO пишут по p95 или p99, потому что именно хвост формирует жалобы.

Доля ошибок — отношение ответов 5xx и таймаутов к общему числу. Отдельно считают бизнес-ошибки: ответ 200 с телом `{"error": "out of stock"}` для нагрузки тоже отказ.

Насыщение (saturation) — близость ресурса к пределу: CPU, память, пул соединений, дескрипторы, очередь в БД. Оно объясняет, почему латентность выросла, а пропускная способность — нет.

Связывает их закон Литтла: `L = λ × W`, где `L` — число одновременных запросов, `λ` — интенсивность, `W` — время ответа. При 1000 rps и 200 мс в системе одновременно 200 запросов. Если пул к БД — 50 соединений и каждый запрос держит соединение 150 мс, потолок около 330 rps, и оптимизация кода его не обойдёт: запросы встанут в очередь.

Почему средняя латентность врёт

Распределение латентности имеет тяжёлый хвост (heavy tail) и обычно близко к логнормальному: большинство запросов укладывается в узкий диапазон, но редкие события — промах кэша, GC-пауза, ретрай, блокировка — в десятки раз медленнее. Среднее складывает всё в сумму, и хвост в ней растворяется.

Пример на 1000 запросах: 900 по 40 мс, 90 по 120 мс, 10 по 3 с. Среднее — 77 мс, что выглядит прилично, а p99 — 3 секунды: каждый сотый пользователь ждёт в 75 раз дольше среднего. Отчёт со средним скрывает ровно то, из-за чего пишут жалобы.

Иногда это объясняют законом Амдала — он здесь ни при чём: Амдал говорит о пределе ускорения при распараллеливании работы с последовательной частью, а форма распределения латентности от доли последовательного кода не зависит. Причина проще: среднее арифметическое, а опыт пользователя определяется квантилями. Второе следствие: `avg` из p95 пяти подов не равен p95 системы (глава 42).

Открытая и закрытая модель нагрузки

Закрытая модель (closed model): фиксированное число виртуальных пользователей, каждый шлёт запрос, ждёт ответ и шлёт следующий. Пропускная способность равна `VU / latency`: 100 потоков при 200 мс дают 500 rps, при деградации до 2 с — 50 rps.

Отсюда главная ловушка: когда сервис замедляется, закрытый генератор замедляется вместе с ним и снижает давление —

ровно тогда, когда его нужно было сохранить. Очередь не растёт, и тест показывает «всё нормально» на нагрузке, которую сам же и уменьшил. Это называется согласованным пропуском (coordinated omission): пауза сервера не попадает в статистику, потому что генератор в этот момент молчал.

Открытая модель (open model) задаёт темп прихода независимо от ответов: 300 rps — это 300 rps, даже если сервис отвечает за пять секунд. Так ведёт себя реальный трафик: пользователи приходят сами. При превышении предела запросы копятся в очереди, латентность растёт нелинейно, и на графике видно колено.

Модель	Как задаётся	Что показывает	Исполнитель в k6
Закрытая	N воркеров, think time	предел при фиксированном пуле клиентов	`constant-vus`, `ramping-vus`
Открытая	rps, не зависит от ответа	предел системы и рост очереди	`constant-arrival-rate`, `ramping-arrival-rate`

Правило: предел ищут открытой моделью, конкретный клиентский пул (приложение с 50 соединениями) эмулируют закрытой.

Инструменты

Инструмент	Модель	Сильная сторона	Ограничение
k6	обе	сценарии, пороги, проверки, вывод в Prometheus	JS-рантайм, память на VU
vegeta	открытая	ровный темп, отчёт и график	один запрос на цель
wrk	закрытая	очень высокий rps на нескольких ядрах	сценарии только через Lua
ab	закрытая	есть везде, один флаг	один URL, слабый отчёт

k6 — выбор для сценариев с порогами как гейтом в CI; vegeta — «долбить один эндпоинт с заданным rps»:

bash

```
echo "GET http://api.stage.internal/api/v1/products?limit=20" \
  | vegeta attack -rate=200 -duration=60s -timeout=5s \
  | tee results.bin | vegeta report
```

`vegeta report` печатает `rate` (что просили), `throughput` (что получилось) и латентности по квантилям. Расхождение между первыми двумя — признак того, что темп исчерпал сервис или сам генератор.

wrk даёт максимум «из коробки» (epoll, десятки тысяч rps на четырёх ядрах), но задаёт не темп, а число соединений: `wrk -t4 -c200 -d30s` — закрытая модель, и полученный rps упирается в этот пул.

Методика: прогрев, ступеньки, knee point

Прогрев. Первые 30–120 секунд показывают не сервис, а его запуск: пустые кэши, TLS-хендшейки, наполнение пула, JIT, масштабирование. Эти данные выбрасывают, в отчёт идёт только полка: разница между холодным и тёплым прогоном в p99 достигает двукратной.

Базовый прогон (baseline) — один прогон на заведомо безопасной нагрузке, около 30% ожидаемого пика. Он проверяет сам сценарий, даёт точку отсчёта для сравнения версий и показывает, что стенд вообще работает.

Ступеньки. Нагрузку поднимают шагами по 20–30% и держат каждый 2–5 минут: мельче — тест затягивается, крупнее — колено теряется между ступенями. Долго нужно потому, что `rate()` и автоскейлер реагируют с задержкой в минуты.

Колено — точка, где латентность начинает расти быстрее нагрузки. До 200 rps p99 растёт линейно, после 250 уходит в секунды при CPU приложения 60%: упёрлись не в процессор, а в очередь — пул соединений, блокировку, диск, лимит upstream. Дальше измеряться будет длина очереди, а не сервис. Колен два: мягкое (латентность) и жёсткое (ошибки); между ними — зона, в которой прод и оказывается в час пик.

Полка на 70–80% от колена в течение часа-двух — тест на выносливость (soak): он ловит утечки памяти и рост числа соединений. Спад (ramp-down) показывает, восстанавливается ли

сервис и успевает ли автоскейлер сжать реплики. Повтор обязателен: разброс p99 больше 20% между прогонами означает, что стенд не контролируется.

Полный сценарий на k6

```
js
```

```
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Rate } from 'k6/metrics';

const businessErrors = new Rate('business_errors');

export const options = {
  scenarios: {
    ramp: {
      executor: 'ramping-arrival-rate',
      startRate: 20,
      timeUnit: '1s',
      preAllocatedVUs: 50,
      maxVUs: 400,
      stages: [
        { target: 50,  duration: '1m' },  // прогрев, в отчёт не идёт
        { target: 100, duration: '3m' },
        { target: 150, duration: '3m' },
        { target: 200, duration: '3m' },
        { target: 200, duration: '5m' },  // полка
        { target: 0,   duration: '1m' },  // спад
      ],
    },
  },
  thresholds: {
    http_req_failed:   [{ threshold: 'rate<0.01', abortOnFail: true, delayAbortEval: '60s' }],
    http_req_duration: ['p(95)<300', 'p(99)<800'],
    business_errors:   ['rate<0.005'],
    checks:            ['rate>0.99'],
  },
};

const BASE = __ENV.BASE_URL || 'http://api.stage.internal';

export default function () {
  const list = http.get(`${BASE}/api/v1/products?limit=20`, {
    tags: { name: 'list_products' },
  });
  check(list, {
    'list 200': (r) => r.status === 200,
    'list < 150ms': (r) => r.timings.duration < 150,
  });

  const order = http.post(
    `${BASE}/api/v1/orders`,
    JSON.stringify({ sku: 'SKU-1042', qty: 1 }),
    { headers: { 'Content-Type': 'application/json' }, tags: { name: 'create_order' } },
  );
  businessErrors.add(!check(order, { 'order 201': (r) => r.status === 201 }));

  sleep(1 + Math.random() * 2);
}
```

`ramping-arrival-rate` — открытая модель: k6 держит темп прихода итераций, добавляя виртуальных пользователей, а не наоборот. `preAllocatedVUs` и `maxVUs` — буфер исполнителей; если упирается в `maxVUs`, появляется метрика `dropped_iterations`, тест недогружает сервис и даёт заниженный результат.

Пороги — это SLO в терминах теста: прогон с нарушенным порогом завершается кодом 99 и годится гейтом в пайплайн. `abortOnFail` с `delayAbortEval: '60s'` — стоп-кран: тест встаёт при доле ошибок выше 1%, но не раньше конца прогрева.

Метка `name` фиксирует имя запроса; без неё URL с идентификатором заказа даёт серию на каждый запрос — ошибка кардинальности из главы 42, только в отчёте генератора. `business_errors` нужна потому, что встроенные метрики считают только транспорт: ответ 200 с телом-ошибкой для них успех.

bash

```
k6 run --out experimental-prometheus-rw load/orders.js

http_req_duration........: avg=71.2ms min=3.1ms med=48ms max=2.1s p(90)=140ms
    p(95)=210ms p(99)=740ms
http_req_failed..........: 0.42%  ✓ rate<0.01
business_errors..........: 0.11%  ✓ rate<0.005
dropped_iterations.......: 0      ✓
http_reqs................: 265412 276.5/s
```

`http_reqs` — средняя интенсивность по всему прогону, включая разгоны и спад.

Как не положить прод

Стенд. Идеал — копия прода по репликам, конфигурации и объёму данных: база на 500 строк не воспроизводит поведение индекса на 50 миллионах, и запрос, который на стенде идёт 2 мс, в проде идёт 200.

Доля трафика. Если стенда нет, тестируют прод — по одному поду (service с селектором на реплику), 1–5% трафика, вне пика, с заголовком `X-Load-Test: k6`, чтобы отделить нагрузку в метриках. Write-сценарии опасны отдельно: тест создаёт реальные заказы, если не переключён на песочницу провайдера.

Стоп-кран. Три уровня: `abortOnFail` в тесте, внешний watchdog по доле 5xx (читать Prometheus каждые 15 с и убивать процесс при

превышении) и ручной выключатель у дежурного, который знает о тесте. Без третьего тест однажды совпадёт с инцидентом, и никто не поймёт, что чинить. Ограничения генератора — тоже предохранитель: `maxVUs`, потолок rps, лимит времени.

Что смотреть по ходу

- CPU приложения и отдельно троттлинг: `rate(container_cpu_cfs_throttled_seconds_total[1m])` — процесс с лимитом 500m на 100% утилизации ведёт себя иначе, чем свободные ядра.
- Память и RSS: рост на полке без спада — утечка или растущий кэш.
- GC: частота и длительность пауз; совпадение пиков p99 с паузами — почти всегда ответ.
- Пул соединений: занятые, свободные, ожидающие; очередь за соединением видна как рост латентности при низком CPU. В БД — `max_connections`, взаимные блокировки, медленные запросы.
- Upstream: таймауты и ошибки внешних вызовов — сервис может деградировать из-за чужого.
- Дескрипторы, conntrack, сеть: при десятках тысяч соединений упираются раньше CPU.

Почему тест на ноутбуке врёт

Генератор и сервис на одной машине делят ядра, кэш и сетевой стек; на loopback нет сетевой карты, реального RTT и потерь, поэтому латентность занижена, а пропускная способность завышена. Turbo Boost и тепловой троттлинг делают результат зависимым от температуры корпуса, Docker Desktop добавляет NAT, Wi-Fi — ретрансмиты.

В облаке то же с другой стороны: соседи по гипервизору дают `%steal`, а «одинаковые» инстансы различаются фактическим CPU. Минимум: отдельная машина в той же сети, CPU генератора ниже 70%, контроль троттлинга, никаких фоновых сборок.

Как зафиксировать результат

Сырые JSON-отчёты (`--out json`) полезнее скриншотов: из них пересчитывают любые срезы. Решение принимают по таблице, где строка — ступень:

RPS	p50, мс	p95, мс	p99, мс	Ошибки, %	CPU app	CPU БД
50	41	88	140	0.00	12	8
100	44	105	190	0.00	24	15
150	48	150	420	0.10	38	22
200	61	290	900	0.40	52	31
250	95	780	2400	2.80	61	44
300	210	1900	6000	11.0	63	58

Колено — между 200 и 250 rps: p99 вырос в 2.7 раза при росте нагрузки на 25%, а CPU почти не изменился, то есть упёрлись в очередь. Рабочая точка — 150 rps на реплику: при пике 400 rps нужны три реплики с запасом 12%.

К отчёту прикладывают скрипт (ссылкой на коммит), digest образа, конфигурацию стенда и объём данных, версию k6 «на момент написания», совмещённый график rps/латентности/ошибок/CPU, таблицу и абзац ограничений.

Связь с error budget и ёмкостным планированием

Порог в тесте — SLO, выраженный числом. При SLO 99.9% на 30 дней бюджет — 43 минуты полного отказа; тест с 0.4% ошибок на 400 rps за десять минут даёт 960 неудачных запросов, около 2.7% месячного бюджета. Арифметика переводит «немного ошибок под нагрузкой» в цену.

Ёмкость считается от колена: `реплики = пиковый_rps / (rps_на_реплику × 0.7)`, вверх, плюс одна на отказ зоны. Прогноз роста применяют к пику, а не к среднему. Если колено ближе 30% к пику — это техдолг с датой: следующая распродажа его вскроет.

Типичные ошибки

1. **Средняя латентность в отчёте.** Хвост растворяется в сумме, деградация не видна до отказа. Причина — среднее считается

по умолчанию и легко объясняется руководству.

2. **Закрытая модель под видом открытой.** «100 потоков держат 500 rps» — не предел, а следствие `VU / latency`; при деградации тест сам снижает давление. Причина — потоки интуитивно понятнее, чем rps.
3. **Прогон без прогрева.** Пустые кэши первых секунд попадают в p99. Причина — тест запускают на минуту, чтобы быстрее.
4. **Пустая база и один под.** На стенде с 500 строками нет ни реального плана запроса, ни конкуренции за соединения. Причина — копия данных дорого.
5. **Насыщенный генератор.** `dropped_iterations` не смотрят, тест недогружает, а выводы делают о сервисе. Причина — генератор считают эталоном, а не узлом с собственным пределом.
6. **Нагрузка в прод без стоп-крана.** Тест совпадает с инцидентом или сам его создаёт, дежурный не знает, что происходит. Причина — «мы аккуратно, по одному поду». Тот же корень у веры в один прогон: разброс больше 20% — это шум стенда.

Чек-лист

- [ ] Цель сформулирована (предел, SLO, регрессия), и под неё выбрана модель нагрузки.
- [ ] Есть прогрев, ступеньки по 20–30%, полка и спад; прогрев исключён из отчёта.
- [ ] Колено записано числом, рабочая точка не выше 70–80% от него.
- [ ] В отчёте квантили, доля ошибок и насыщение, а не среднее.
- [ ] `dropped_iterations` равен нулю, CPU генератора ниже 70%.
- [ ] Данные на стенде по объёму близки к проду; есть `abortOnFail`, watchdog и дежурный, знающий о тесте.
- [ ] Прогон повторён дважды, разброс p99 меньше 20%.

Упражнения

1. Прогоните один эндпоинт через `ab`, `wrk` и k6 с одинаковым числом клиентов. Объясните расхождение в rps и p99.
2. Перепишите сценарий с `constant-vus` на `ramping-arrival-rate` при той же пиковой интенсивности. Какая модель раньше показывает деградацию и почему?
3. Постройте таблицу «RPS → p99 → ошибки» для сервиса с задержкой 50 мс и пулом соединений на 10. Найдите колено и ресурс, который его создал.
4. Замедлите БД через `pg_sleep` при неизменном темпе. Что произойдёт с `dropped_iterations` и что показал бы тот же тест в закрытой модели?
5. Сравните прогон с генератора на той же машине и с отдельного хоста в той же сети. Оцените разницу p50 и p99 и объясните её источники.

Ответы и разбор к этим упражнениям — в приложении D, часть 5.

Глава 43. Безопасность и supply chain

Безопасность сервиса — это не отдельный этап перед релизом, а набор решений, принятых в Dockerfile, в манифесте, в пайплайне и в коде обработчика. Большинство реальных взломов не экзотичны: утёкший ключ из переменной окружения, контейнер под root с лишними capabilities, зависимость с известной CVE, образ, подменённый по тегу. В главе разбирается модель угроз типового сервиса, хранение и ротация секретов, минимальные привилегии в контейнере, проверка цепочки поставки (lock-файлы, Trivy, SBOM, cosign), OIDC вместо долгоживущих ключей в CI, TLS внутри кластера и обязательный минимум по OWASP для API. Заканчивается главой чек-лист «сервис перед публикацией в интернет».

Модель угроз: от чего защищаемся

Прежде чем что-то настраивать, полезно выписать, кто и как может навредить. Для типового веб-сервиса угрозы укладываются в пять групп.

Утечка секретов — ключи, пароли, токены попадают в git, в образ, в лог или в вывод ошибки. Компрометация зависимости или образа — атакующий публикует пакет с похожим именем либо подменяет тег в registry. Избыточные права — процесс внутри контейнера может больше, чем нужно, поэтому ошибка в коде превращается в компромисс узла. Атака через входные данные — инъекции, SSRF, доступ к чужим объектам по идентификатору. И, наконец, слабая наблюдаемость: атаку просто не замечают, потому что нет логов доступа и алертов на аномалии.

Модель угроз отвечает на вопрос «что мы защищаем и от кого», а не «какие инструменты модны». Внутренний сервис в закрытом контуре и публичный API с платежами имеют разные приоритеты: первому важнее сегментация и аудит, второму — аутентификация, rate limit и защита от инъекций.

Секреты: почему не в git и не в образе

Секрет в git — это секрет навсегда: история клонируется, форкается, попадает в бэкапы и в CI-кэш. Удаление коммита не помогает, `git filter-repo` переписывает историю только у тех, кто её перетянул, а старый объект может жить в чужом клоне. Поэтому единственная реакция на секрет в git — ротация, а не «удалили и забыли». То же с образом: слой с `ENV API_KEY=...` остаётся в манифесте даже если следующий слой его «удаляет».

Переменные окружения — компромисс. Их видно в `docker inspect`, в `/proc/<pid>/environ` (для того же пользователя или root), в дампах, в логах исключений при старте, и они наследуются всеми дочерними процессами, включая сторонние утилиты. Файл с правами `0400` внутри тома лучше: он не наследуется и не попадает в `ps e`. Vault (HashiCorp Vault, AWS Secrets Manager, Yandex Lockbox) лучше всего: секрет выдаётся по короткоживущему токену, имеет TTL, а его выдача логируется.

Практичная схема для Kubernetes: секреты живут в хранилище, в кластер попадают через External Secrets Operator или CSI-драйвер, монтируются файлом в `emptyDir`/`tmpfs`, а не в env. Приложение читает файл при старте и умеет перечитывать при изменении.

yaml

```
volumes:
  - name: db-creds
    csi:
      driver: secrets-store.csi.k8s.io
      readOnly: true
      volumeAttributes:
        secretProviderClass: shop-db
containers:
  - name: api
    volumeMounts:
      - name: db-creds
        mountPath: /run/secrets/db
        readOnly: true
```

Ротация должна быть процессом, а не событием: два активных ключа одновременно, приложение умеет читать новый, старый отзывается через сутки. Если ротация требует ручного изменения конфига и перезапуска — её не будут делать, и ключ проживёт три года.

Минимальные привилегии в контейнере

Контейнер — это процесс с namespaces и cgroups (глава 38), а не виртуальная машина. Всё, что процесс может внутри, он может и в ядре: смонтировать, отправить сырой пакет, загрузить модуль. Ограничения ставятся явно.

yaml

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]
  seccompProfile:
    type: RuntimeDefault
```

`runAsNonRoot` не даст запуститься от uid 0 — типичная ошибка «в Dockerfile нет USER, значит root». `allowPrivilegeEscalation: false` закрывает `setuid`-бинари. `drop: ALL` убирает 14 capabilities, которые Docker даёт по умолчанию, включая `NET_RAW` (сырые сокеты, спуфинг) и `CHOWN`; нужные возвращаются точечно, например `NET_BIND_SERVICE` для порта 80. `readOnlyRootFilesystem` превращает попытку записать webshell в ошибку ввода-вывода; каталоги под запись монтируются отдельно.

seccomp — фильтр системных вызовов. Профиль `RuntimeDefault` блокирует около 40 опасных вызовов (`mount`, `ptrace`, `kexec_load`, `bpf`) и не требует написания политики. В Kubernetes его задают явно, потому что неявные умолчания зависят от версии и от настройки kubelet.

Проверить, что получилось, можно до выката: `kubectl auth can-i --list` и `kubectl exec ... -- id` покажут реальные права, а `kubectl get pod -o yaml | grep -A5 securityContext` — то, что применилось, а не то, что задумано. Pod Security Admission в режиме `restricted` отклонит манифест без этих полей — полезный предохранитель на уровне namespace.

Supply chain: зависимости, образы, SBOM, подпись

Цепочка поставки — всё, что попадает в артефакт помимо вашего кода. Начните с воспроизводимости: lock-файл (`poetry.lock`, `uv.lock`, `go.sum`, `package-lock.json`) фиксирует точные версии и хэши, а `pip install --require-hashes -r requirements.txt` не даст поставить пакет, чей хэш не совпал. Без lock-файла сборка вчера и сегодня — разные сборки, и «у нас ничего не менялось» перестаёт быть аргументом.

CVE-сканирование образа — второй слой. Trivy сверяет пакеты из слоёв с базами уязвимостей и умеет валить сборку:

bash

```
trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 \
  ghcr.io/example/shop-api@sha256:9f2c1e7b4a5d6c8e0f1a2b3c4d5e6f708192a3b4c5d6e7f80
      91a2b3c4d5e6f70

ghcr.io/example/shop-api (debian 13.1)
Total: 14 (HIGH: 9, CRITICAL: 5)
┌──────────────┬────────────────┬──────────┬──────────────────────┐
│   Library    │ Vulnerability  │ Severity │ Fixed Version        │
├──────────────┼────────────────┼──────────┼──────────────────────┤
│ libssl3      │ CVE-2026-1234  │ CRITICAL │ 3.5.2-1              │
│ libcurl4     │ CVE-2026-5678  │ HIGH     │ 8.14.1-2             │
└──────────────┴────────────────┴──────────┴──────────────────────┘
```

`--ignore-unfixed` отсекает уязвимости без исправления: чинить их всё равно нечем, а шум в пайплайне обесценивает проверку. Сканировать надо не только финальный образ, но и базовый: `debian:13-slim` и `distroless` имеют разный размер поверхности атаки, и distroless-образ без shell и пакетного менеджера выигрывает просто потому, что атакующему нечем воспользоваться.

SBOM (software bill of materials) — машинный список компонентов: имя, версия, лицензия, хэш. Генерируется на сборке (`syft`, `trivy --format cyclonedx`) в форматах SPDX или CycloneDX и хранится рядом с образом. Через полгода, когда объявят CVE в библиотеке, SBOM отвечает на вопрос «какие из наших 40 сервисов её содержат» за минуты, а не за неделю grep'а по репозиториям.

Тег — не идентификатор. `latest` и даже `v1.4.2` перезаписываемы: registry не запрещает запушить другой образ под тем же тегом, а кэш узла оставит старый. Digest — хэш манифеста, он неизменен. Поэтому в продакшн-манифесте пишут `image: ghcr.io/example/shop-api@sha256:...`, а тег оставляют для читаемости.

Подпись замыкает цепочку: cosign подписывает digest ключом или ключом из OIDC-идентичности CI, а кластер проверяет подпись при допуске (Kyverno, Sigstore policy-controller, Gatekeeper).

bash

```
cosign verify ghcr.io/example/shop-api@sha256:9f2c1e7b... \
  --certificate-identity-regexp '^https://github.com/example/shop/\.github/workflow
      s/build\.yml@refs/heads/main$' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
```

Проверка `--certificate-identity` важна не меньше самой подписи: без неё валидна любая подпись любого аккаунта GitHub, включая ваш личный форк с тем же именем workflow.

CI: OIDC вместо долгоживущих ключей

Ключ, лежащий в секретах CI, живёт годами и работает откуда угодно — утечка означает доступ, который никто не заметит. OIDC меняет модель: пайплайн получает подписанный токен от провайдера (GitHub, GitLab), облако проверяет его и выдаёт временные креды на 15–60 минут.

yaml

```
permissions:
  id-token: write
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-24.04
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/gh-deploy-shop
          aws-region: eu-central-1
```

Роль в облаке доверяет конкретному репозиторию, ветке и workflow — в условии trust policy, а не «любому токену GitHub». Тогда компрометация форка не даёт доступа к продакшн-аккаунту. Тот же принцип применим к registry и к Kubernetes: короткоживущий токен вместо kubeconfig с сертификатом на три года.

TLS внутри кластера

Сеть внутри Kubernetes плоская: любой под по умолчанию может обратиться к любому другому по IP или имени сервиса. Если база принимает соединения без TLS, то компромисс любого пода в namespace — это доступ к трафику и к данным.

Минимум: TLS на всех соединениях сервис-сервис и сервис-БД, сертификаты через cert-manager с автоматическим продлением, проверка цепочки и имени хоста на стороне клиента, а не `verify=False`. Дальше — mTLS: обе стороны предъявляют сертификат, и под без валидного клиентского сертификата не подключится. Это даёт service mesh (Istio, Linkerd) почти без изменений в коде, но добавляет sidecar и сложность; для небольшого контура достаточно cert-manager плюс NetworkPolicy.

NetworkPolicy важна не меньше TLS: политика «разрешено только с подов с меткой `app=api` на порт 5432» сужает доступ даже при украденном пароле. По умолчанию в namespace стоит `default-deny` на ingress и egress, а разрешения выдаются явно.

Аудит и логирование доступа

Логи доступа отвечают на вопрос «кто что сделал»: аутентификация (успешная и неуспешная), авторизация (отказ по правам), изменения объектов (кто удалил секрет, кто создал RoleBinding). В Kubernetes для этого включают audit policy с уровнями `Metadata` для обычных запросов и `Request` для критичных ресурсов, в облаке — audit trail провайдера.

Что логировать обязательно: неуспешные логины, смену прав и ролей, доступ к секретам, административные операции, выдачу токенов. Чего нельзя: пароли, токены, полные номера карт, содержимое cookie. Маскирование — на уровне логгера, а не «мы аккуратно передаём».

Логи аудита должны быть отдельно от прикладных: другое хранилище, другой retention (обычно год), запрет на удаление приложением. Иначе атакующий с доступом к поду просто сотрёт следы.

OWASP-минимум для API

Инъекции. SQL собирается только параметризованными запросами; конкатенация строк с вводом — запрещена код-ревью. То же для shell: `subprocess.run(["git", "log", ref])` вместо `shell=True`. Валидация входа — вторая линия, а не первая.

SSRF. Если сервис ходит по URL, переданному клиентом (вебхук, импорт по ссылке, превью изображения), атакующий попросит `http://169.254.169.254/latest/meta-data/` и заберёт креды инстанса. Защита: allowlist схем и хостов, запрет link-local и приватных диапазонов после резолва, отдельный egress-прокси, IMDSv2 с ограничением hop limit.

IDOR. Проверка прав должна быть на уровне объекта, а не только на уровне эндпоинта. `GET /orders/12345` обязан проверить, что заказ принадлежит текущему пользователю, — иначе аутентификация есть, а авторизации нет. Это самая частая уязвимость в API, потому что фильтр «по текущему пользователю» легко забыть в одном из десяти обработчиков.

Rate limit. Лимит по ключу (пользователь, токен, IP) с ответом `429` и заголовком `Retry-After`. Без него перебор паролей, перебор OTP и скрейпинг каталога идут со скоростью вашей инфраструктуры. Отдельно лимитируют дорогие операции: поиск, экспорт, отправку писем.

Аутентификация и токены. Короткоживущий access token (15 минут), refresh с ротацией и отзывом, пароли — bcrypt cost 12 или argon2id. JWT проверяется по алгоритму из конфигурации, а не из заголовка `alg` — иначе `alg: none` проходит проверку.

Типичные ошибки

1. **Секрет в `.env`, закоммиченный «на время».** Файл попадает в историю, в CI-кэш и в образ; ротация не делается, потому что «никто не видел». Причина — соблазн запустить локально за 10 секунд.
2. `USER root` **по умолчанию.** В Dockerfile нет `USER`, и процесс получает uid 0 внутри контейнера; при монтировании docker.sock или privileged-режиме это root на узле. Причина — базовые образы не всегда задают пользователя.

3. `drop: ALL` **без проверки.** После ужесточения сервис падает на `bind: permission denied` для порта 80, и правку откатывают целиком, возвращая все capabilities. Причина — ужесточение без теста в staging.
4. **Сканирование только финального слоя.** `trivy` без `--ignore-unfixed` даёт сотни «неисправимых» находок, пайплайн краснеет, проверку отключают. Причина — порог выставлен без учёта реальной исправимости.
5. **Тег вместо digest.** Откат «на предыдущий тег» поднимает не тот образ, а supply-chain-атака через перезапись тега не детектируется. Причина — тег читаемее, и об этом не думают до инцидента.
6. `curl | bash` **в CI.** Установка инструмента из интернета без проверки подписи и версии — исполнение произвольного кода в пайплайне с доступом к секретам. Причина — «так быстрее, чем пиновать версию».
7. `verify=False` **в клиенте к внутреннему сервису.** Отключает проверку цепочки и имени, и mTLS превращается в декорацию. Причина — самоподписанный сертификат в тестовом контуре.

Чек-лист: сервис перед публикацией в интернет

- [ ] Все секреты приходят из хранилища, не из git и не из образа; ротация описана и проверена.
- [ ] Контейнер: non-root, `drop: ALL`, `allowPrivilegeEscalation: false`, `readOnlyRootFilesystem`, seccomp `RuntimeDefault`.
- [ ] В манифесте digest, а не тег; образ подписан, подпись проверяется при допуске.
- [ ] Trivy на HIGH/CRITICAL с `--ignore-unfixed` валит сборку; SBOM сохранён рядом с образом.
- [ ] Зависимости зафиксированы lock-файлом, установка с проверкой хэшей.
- [ ] В CI нет долгоживущих ключей: только OIDC и временные креды.

- [ ] TLS на всех внутренних соединениях, `default-deny` NetworkPolicy, mTLS там, где нужен.
- [ ] Аутентификация, авторизация на уровне объекта, rate limit; аудит-логи отдельно и с retention не меньше года.

Упражнения

1. Соберите образ без `USER` и проверьте `id` внутри контейнера. Затем добавьте non-root пользователя, `drop: ALL` и `readOnlyRootFilesystem` — и найдите, какие каталоги приложению всё-таки нужны на запись.
2. Прогоните один и тот же образ через `trivy image` с `--ignore-unfixed` и без него. Сравните число находок и объясните, почему порог «ноль уязвимостей» на практике неработоспособен.
3. Сгенерируйте SBOM в формате CycloneDX для образа, найдите в нём конкретную библиотеку и её версию. Оцените, за сколько вы ответите на вопрос «где ещё используется эта версия».
4. Подпишите образ через cosign и проверьте его с неверным `--certificate-identity`. Объясните, почему подпись без проверки идентичности не защищает от подмены.
5. Напишите обработчик, который принимает URL и скачивает его. Реализуйте защиту от SSRF и проверьте её на `http://169.254.169.254/` и на домене, резолвящемся в приватный адрес.

Ответы и разбор к этим упражнениям — в приложении D, часть 4.

Глава 43a. Секреты: Vault, SOPS, age

«Не храните секреты в git» — верное правило, из которого не следует, что делать вместо. Секрет нужен приложению в момент старта, CI — в момент деплоя, инженеру — в момент отладки, и все три пути надо не только закрыть, но и сделать короче, чем небезопасный. В главе разбирается классификация секретов и модель угроз, способы хранения от переменных окружения до Vault, шифрование значений внутри git через SOPS и age, ротация без простоя и порядок действий при утечке. В конце — рабочие примеры: политика Vault для приложения, динамические креды БД и расшифровка values.yaml в CI.

Почему «не в git» — половина ответа

Секрет, попавший в git, считается скомпрометированным навсегда: история клонируется, форкается, кэшируется в CI, попадает в бэкапы и в чужие рабочие копии. `git rm` и даже `git filter-repo` не отменяют того, что объект уже был скачан. Отсюда единственная разумная реакция — ротация, а не удаление коммита.

Но запрет без альтернативы рождает обходные пути: секрет пересылают в мессенджере, пишут в вики, кладут в переменную окружения CI и называют это «хранилищем». Вопрос надо ставить иначе: где секрет живёт, кто и по какому праву его читает, как он меняется и как узнать, кто его читал. Ответ на эти четыре вопроса и есть система управления секретами.

Классификация и модель угроз

Разные секреты требуют разной цены защиты. Пароль к тестовой базе и ключ подписи релизов не должны лежать в одном месте с одинаковыми правами.

Тип	Живёт	Кто читает	Цена утечки
Пароль БД	долго	приложение	доступ к данным, часто ко всем тенантам
API-ключ внешнего сервиса	месяцы	приложение	деньги, репутация, данные клиентов
TLS-ключ	90 дней	балансировщик	подмена трафика, обход проверок
Токен CI/CD	часы-дни	пайплайн	деплой произвольного кода в прод
Ключ шифрования данных	годы	сервис	расшифровка бэкапов и архива

Модель угроз начинается с перечня, от кого защищаемся: разработчик с доступом к репозиторию, но без доступа к проду; скомпрометированный под, читающий своё окружение; инсайдер с правами администратора кластера; резервная копия в незащищённом хранилище; лог с заголовком авторизации.

Граница: хранилище не защищает от root на хосте и от того, у кого есть право читать память процесса. Оно сокращает поверхность и делает доступ видимым, но не отменяет компромисс узла.

Три способа хранения и их цена

Переменные окружения (environment variables) — самый распространённый способ и самый утечкоопасный. Значение видно в `/proc/<pid>/environ` любому процессу того же пользователя, в `docker inspect`, в `ps e`, в дампах и логах падений; оно наследуется всеми дочерними процессами, включая чужие утилиты. Плюс один — простота: ничего не надо монтировать.

bash

```
tr '\0' '\n' < /proc/1/environ | grep -i pass
```

Файл с правами `0400`, смонтированный в контейнер, лучше: он не наследуется, не попадает в `ps e` и читается только владельцем. Минусы — жизненный цикл: кто обновит файл при ротации, увидит ли приложение изменение, не останется ли копия в образе.

Secret-хранилище (Vault, облачные менеджеры секретов) даёт то, чего нет у первых двух: короткоживущую выдачу, права на конкретный путь, аудит каждого чтения и ротацию в одном месте. Цена — ещё один компонент, который может упасть, и необходимость аутентифицировать приложение, а не человека.

Vault: движки, политики, токены

Vault — сервер, который выдаёт секреты по токену. Всё остальное — следствия: у токена есть TTL и политика, у секрета — путь и версия, у каждого чтения — запись в аудите.

Движок KV версии 2 хранит версии значений. Путь в API отличается от логического: `secret/shop/api` в CLI — это `secret/data/shop/api` в API и в политиках, потому что `data` — часть движка, а не имени секрета.

bash

```
vault secrets enable -path=secret kv-v2
vault kv put secret/shop/api db_password='S3cure-...' stripe_key='sk_live_...'
vault kv get -field=db_password secret/shop/api
vault kv metadata put -max-versions=5 secret/shop/api
```

Политика — единственный способ ограничить доступ; токен без политики не умеет ничего. Минимальные права означают `read` на конкретный путь, без `list` на весь движок и без доступа к `metadata`, если версии не нужны.

hcl

```
path "secret/data/shop/api" {
  capabilities = ["read"]
}

path "secret/metadata/shop/api" {
  capabilities = ["read"]
}
```

Приложение аутентифицируется не паролем, а методом, привязанным к платформе: Kubernetes-auth (под предъявляет service account token), AppRole (role_id и secret_id), облачный IAM. Дальше — токен с коротким TTL, который продлевается, пока жив процесс:

bash

```
vault policy write shop-api shop-api.hcl
```

```
vault token create -policy=shop-api -ttl=1h -renewable=true
curl -s -H "X-Vault-Token: $VAULT_TOKEN" \
  "$VAULT_ADDR/v1/secret/data/shop/api" | jq -r '.data.data.db_password'
```

`renewable=true` важен: без продления приложение умрёт через час после старта. TTL в час означает, что утёкший токен живёт час, а не три года.

Динамические креды вместо статичного пароля

Статичный пароль БД живёт в Vault, но остаётся одним и тем же для всех реплик: утечка требует ручной ротации, а в логах БД все запросы выглядят как один пользователь. Движок database создаёт пользователя на время: Vault подключается к БД административной учёткой, выполняет `CREATE ROLE`, отдаёт логин и пароль с TTL и удаляет роль по истечении.

bash

```
vault secrets enable database
vault write database/config/shop \
  plugin_name=postgresql-database-plugin \
  allowed_roles="shop-api" \
  connection_url="postgresql://{{username}}:{{password}}@db:5432/shop?sslmode=requi
      re" \
  username="vault_admin" password="$VAULT_DB_ADMIN_PASS"

vault write database/roles/shop-api db_name=shop default_ttl=1h max_ttl=24h \
  creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}'
    VALID UNTIL '{{expiration}}';
    GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO \"{{name}}\";"
```

Теперь у каждой реплики свой пользователь, в `pg_stat_activity` видно, кто держит соединение, а срок жизни креды ограничен часом. Обратная сторона: приложение обязано переподключаться при истечении, а пул — обновлять пароль, иначе динамические креды дают отказы раз в час.

Agent, injector и запечатывание

Заставлять каждое приложение ходить в API Vault — лишний код. Vault Agent сайдкаром аутентифицируется сам, держит и обновляет токен, рендерит секреты в файл по шаблону; приложение просто читает файл. Injector делает то же через admission webhook, подменяя поды с аннотациями.

Отдельно — режим запечатывания (seal). После старта Vault запечатан: данные на диске зашифрованы, а ключ шифрования

защищён мастер-ключом, который собирается из кворума (например, 3 из 5). Пока три хранителя не введут свои части через `vault operator unseal`, секреты не выдаются.

В продакшене ручное распечатывание — точка отказа: после перезагрузки узла сервис не поднимется, пока не соберутся три человека. Поэтому используют авто-распечатывание (auto-unseal) через KMS или transit-движок другого кластера: кворум людей меняется на доверие к облачному KMS.

SOPS и age: шифрование значений в git

Если секреты должны лежать в git (например, в GitOps-репозитории), их шифруют так, чтобы расшифровать мог только нужный круг. SOPS шифрует не файл целиком, а значения, оставляя ключи и структуру читаемыми: диффы остаются осмысленными.

Ключи для SOPS бывают разные (age, PGP, KMS); age — самый простой: одна строка приватного ключа, один публичный адрес.

bash

```
age-keygen -o age.key
```

```
Public key: age1qyqszqgpqyqszqgpqyqszqgpqyqszqgpqyqszqgpqyqszqgpqyqs2m6s9
```

bash

```
age-keygen -y age.key
```

Правила шифрования задаются в `.sops.yaml` в корне репозитория, чтобы не перечислять получателей в каждой команде:

yaml

```
creation_rules:
  - path_regex: environments/prod/.*\.yaml$
    encrypted_regex: ^(data|stringData|password|token|key)$
    age: age1qyqszqgpqyqszqgpqyqszqgpqyqszqgpqyqszqgpqyqszqgpqyqs2m6s9
```

`encrypted_regex` шифрует только значения с этими именами: остальное остаётся открытым, и в ревью видно, что изменился, например, только тег образа.

bash

```
sops --encrypt --in-place environments/prod/values.yaml
sops --decrypt environments/prod/values.yaml | head -20
```

В зашифрованном файле значения выглядят как `ENC[AES256_GCM,data:...,type:str]`, а в конце появляется блок `sops` с получателями, версией формата и MAC. Правка идёт через `sops environments/prod/values.yaml`: редактор получает расшифрованный текст, а на диск возвращается шифрованный.

Расшифровать может тот, чей приватный ключ указан получателем. Значит, круг доступа к прод-секретам — это список age-адресов в `.sops.yaml`, и его изменение видно в диффе. Отзыв доступа — перевыпуск файла с новым списком, но старые версии в истории остаются читаемыми для того, у кого был ключ.

Расшифровка в CI и GitOps

В CI приватный ключ лежит в секрете пайплайна (в GitHub — Actions secret, в GitLab — masked variable) и передаётся SOPS через переменную окружения, а не файлом на диске:

bash

```
export SOPS_AGE_KEY="$AGE_PRIVATE_KEY"
sops --decrypt environments/prod/values.yaml > /tmp/values.yaml
helm upgrade --install shop ./chart -f /tmp/values.yaml --namespace shop
```

`/tmp/values.yaml` после деплоя надо удалять, а лучше не создавать: `sops exec-file` и `sops exec-env` расшифровывают во временный файл или переменные на время команды и убирают за собой. Печатать расшифрованное в лог нельзя: маскирование в CI не спасает от `set -x` и `helm --debug`.

В GitOps-схеме (глава 41а) кластер сам тянет манифесты из git. Flux расшифровывает SOPS при применении: ключ лежит в Secret в namespace контроллера, в манифесте указан `spec.decryption.provider: sops`; ArgoCD требует для этого плагин или sidecar. Альтернатива — Sealed Secrets: приватный ключ хранит контроллер в кластере, а в git попадает `SealedSecret`, который может расшифровать только этот кластер. Разница принципиальная: SOPS-файл прочитает любой, у кого есть age-ключ, SealedSecret в другом кластере бесполезен — удобно для мультикластера, неудобно для восстановления из бэкапа.

Ротация без простоя

Ротация — процесс, а не событие. Минимальная схема — два активных ключа и перекрытие (overlap): новый создаётся, распространяется, приложения переключаются, старый живёт ещё сутки и только потом отзывается. Иначе реплики с кэшированной конфигурацией начнут получать отказы.

Порядок для API-ключа: создать второй ключ в панели провайдера, положить оба в Vault (`stripe_key` и `stripe_key_next`), выкатить код, читающий новый, дождаться обновления всех подов, отозвать старый. Окно перекрытия должно быть больше времени жизни пода и TTL кэша конфигурации.

С паролем БД сложнее: PostgreSQL и MySQL допускают один пароль на роль. Варианты — отдельная роль на релиз (старую удаляют после переключения), PgBouncer с двумя пользователями или динамические креды, где перекрытие получается само.

Автоматическая ротация снимает ручную работу, но добавляет следствия: приложение должно перечитывать секрет без рестарта, пул соединений — переподключаться, мониторинг — отличать плановую ротацию от аварии. Худший результат — ротация, которая ломает сервис каждые 30 дней и которую поэтому отключают.

Что делать при утечке

Порядок важен, и он не совпадает с интуицией «сначала разобраться».

1. Отозвать. Немедленно отключить ключ, сменить пароль, отозвать токен. Разбирательство — после; пока ключ жив, утечка продолжается.
2. Заменить. Выпустить новый секрет и выкатить его; проверить, что все потребители перешли.
3. Найти источник. `gitleaks detect --source . --redact` по репозиторию и истории, поиск по логам CI, по бакетам, по образам, по локальным копиям.
4. Проверить использование. По логам провайдера и БД — были ли обращения из незнакомых мест, какие данные читались,

что создано.

5. Записать. Краткая хронология, кто и когда узнал, что сделано; для инцидент-ревью и для регуляторов, если данные персональные.

Поиск по истории не должен светить сам секрет: `git log -S 'sk_live_' --oneline` покажет коммиты, где строка появлялась, не печатая её. Удаления недостаточно: переписывание истории ломает клоны и не гарантирует, что объект исчез у всех, поэтому ротация обязательна независимо от чистки.

Аудит: кто читал и когда

Аудит отличает управление секретами от их хранения. В Vault включают audit device — файл или syslog, куда пишется каждое обращение: путь, метод, идентичность, адрес, результат.

bash

```
vault audit enable file file_path=/var/log/vault/audit.log
jq -r 'select(.request.path=="secret/data/shop/api")
  | [.time, .auth.display_name, .request.remote_address] | @tsv' \
  /var/log/vault/audit.log | tail -20
```

Значения секретов в аудит не попадают, только метаданные; лог надо защищать от удаления приложением и хранить отдельно от прикладных. В Kubernetes к этому добавляется audit policy с уровнем `Metadata` на `secrets`, в облаке — audit trail провайдера.

Аудит отвечает на вопросы: кто читал прод-секрет за 30 дней, были ли чтения из незнакомого namespace, сколько раз выдавались динамические креды. Если ответить нельзя, хранилище используют как дорогой файл.

Типичные ошибки

1. **Секрет в `.env`, закоммиченный «на время».** Файл попадает в историю, кэш CI и образ; ротации нет, потому что «никто не видел». Причина — соблазн запустить локально быстро.
2. `ENV API_KEY=...` **в Dockerfile.** Слой остаётся в манифесте даже если следующий слой «удаляет» переменную. Причина — путаница между изменением файловой системы и историей слоёв.

3. **Токен Vault без TTL и с политикой** `root`**.** Утечка даёт доступ ко всем секретам навсегда, а аудит показывает только `root`. Причина — так быстрее отладить.
4. **Ручное распечатывание без плана.** После перезагрузки кластер не поднимается, пока не соберутся три человека с ключами. Причина — seal воспринимают как формальность.
5. **Отзыв ключа до переключения.** Реплики с кэшированной конфигурацией начинают отдавать 500, и инцидент выглядит как последствие ротации. Причина — экономия на окне перекрытия.
6. **Расшифровка в CI с печатью в лог.** `set -x` или `helm --debug` выводят секрет в лог, доступный всем, у кого есть доступ к сборке. Причина — отладка на живых секретах.

Чек-лист

- [ ] Секреты не в git и не в образе; в репозитории только шифротекст.
- [ ] Приложение получает секрет по короткоживущему токену с минимальной политикой.
- [ ] У каждого секрета есть владелец, срок жизни и описанная процедура ротации.
- [ ] Ротация проверена на стенде с окном перекрытия, а не только описана.
- [ ] Аудит-логи включены, хранятся отдельно и не удаляются приложением; запечатывание покрыто планом: кто и за какое время распечатает.
- [ ] В CI секрет не печатается, временные файлы удаляются, ключ — в masked-переменной.
- [ ] Есть порядок действий при утечке: отзыв, замена, аудит, запись.

Упражнения

1. Сравните на одном контейнере три способа доставки секрета: env, файл `0400` и том `emptyDir` с `medium: Memory`. Покажите, где

значение видно снаружи контейнера и кому.

2. Поднимите Vault в dev-режиме, включите KV v2, создайте политику только на чтение одного пути и токен с TTL 10 минут. Убедитесь, что после истечения токена чтение возвращает 403.
3. Зашифруйте `values.yaml` через SOPS с `encrypted_regex`, измените одно значение и объясните, почему дифф остаётся читаемым. Затем расшифруйте файл ключом, которого нет в списке получателей.
4. Настройте динамические креды PostgreSQL в Vault с TTL 5 минут. Посмотрите, что происходит с открытым соединением после истечения, и предложите, как приложение должно себя вести.
5. Проведите учебную утечку: положите ключ в репозиторий, найдите `gitleaks`, отзовите, замените и соберите хронологию по аудит-логу Vault.

Ответы и разбор к этим упражнениям — в приложении D, часть 5.

Глава 44. Инциденты, postmortem, on-call

Всё, что описано в предыдущих главах, рано или поздно ломается в три часа ночи, и тогда работают не знания вообще, а заранее отрепетированный порядок действий. Инцидент — это не «что-то упало», а событие, влияющее на пользователей или на бизнес и требующее немедленного вмешательства. В главе разбирается, как классифицировать инцидент, зачем нужен incident commander, как остановить кровь до поиска причины, что делать в шести типовых сценариях, как писать postmortem без обвинений и как строить on-call без выгорания, — вплоть до полного разбора учебной аварии.

Что считается инцидентом и уровни severity

Не всякая ошибка — инцидент. Одиночный 500-й ответ, который не повторился, — запись в логе. Инцидент начинается там, где есть влияние: пользователи не могут войти, платежи не проходят, данные теряются, SLO сгорает быстрее обычного. Формулировка «влияние на пользователя» важнее технической «CPU 95%»: второе может быть нормой.

Уровень	Влияние	Реакция	Обновления
SEV1	Полный отказ, потеря данных, утечка	Все на связи, IC немедленно	каждые 15 мин
SEV2	Ключевая функция недоступна или деградировала	IC и владелец сервиса	каждые 30 мин
SEV3	Ограниченное влияние, есть обход	Дежурный, в рабочее время	по мере
SEV4	Косметика, внутренние неудобства	Тикет в очередь	нет

Классификация нужна не для бюрократии, а чтобы понять, будить ли директора и сколько людей отвлекать. Ошибка в обе

стороны дорога: занижение оставляет одного человека тонуть, завышение приучает игнорировать алерты.

Роли: почему без incident commander хуже

В первые минуты самое ценное — не скорость набора команд, а отсутствие хаоса. Без назначенной роли десять инженеров одновременно лезут в прод и спорят в голосовом канале; два человека независимо откатывают разные сервисы, третий перезапускает базу, и состояние системы меняется быстрее, чем его успевают осмыслить.

Incident commander (IC) не чинит, а управляет: держит картину целиком, знает, кто что делает, какая гипотеза проверяется, что исключено, когда следующее обновление для пользователей. IC не обязан быть самым опытным инженером и не должен быть автором последнего деплоя — иначе он уйдёт в код и потеряет обзор.

Остальные роли: оператор (руки), коммуникатор (статус-страница и заказчики), секретарь (таймлайн). В маленькой команде один человек совмещает две роли, но IC выделяется отдельно — иначе его нет. «IC не чинит» соблюдать тяжело, но открывший терминал IC перестаёт замечать, что происходит вокруг.

Канал, таймлайн и фиксация

Инцидент обслуживается в отдельном канале: `#inc-2026-10-29-api-5xx`. Не в личных сообщениях и не в общем чате команды — иначе контекст размазан, а подключившиеся через двадцать минут не понимают, что уже проверено.

Первое сообщение — шаблон: что сломано, кто заметил, уровень, кто IC, ссылка на дашборд. Дальше каждое действие фиксируется с временем: «10:47 — откатили api», «10:52 — 5xx упали с 32% до 4%», «10:58 — гипотеза: миграция заблокировала таблицу orders». Таймлайн ведут руками или ботом; важно не качество инструмента, а привычка. Без таймлайна postmortem через три дня — реконструкция по памяти. Отдельно

фиксируются отвергнутые гипотезы, иначе следующий дежурный проверит то же заново.

Сначала остановить кровь

Главный принцип: остановить влияние, потом искать причину. Пока сервис отдаёт 500, каждая минута — потерянные заказы, а не интересный эксперимент. Диагностика, которая ничего не меняет, безопасна; эксперимент, который может ухудшить, допустим только если он быстрее отката.

Три инструмента, в порядке предпочтения. Откат: если деградация началась в пределах часа после выката, гипотеза «виноват релиз» проверяется дешевле всего — `kubectl rollout undo deployment/api -n shop`, затем `kubectl rollout status deployment/api -n shop --timeout=180s`. Откат не требует понимания причины и почти всегда безопасен, если миграции обратно совместимы. Feature flag: если релиз не при чём или откат невозможен, выключается конкретная функция — быстрее деплоя. Масштабирование и изоляция: добавить реплики, снять нагрузку с зоны, включить rate limit, отключить тяжёлый отчёт.

Чего не делать: перезапускать всё подряд, менять конфиг «на удачу», лезть в базу с `ALTER`, удалять поды, чтобы «освежить». Каждое действие меняет состояние и уничтожает улики; перезапуск кластера целиком — почти всегда худшее из решений.

Типичные сценарии и первые команды

5xx сразу после деплоя. Первым делом — корреляция по времени. `kubectl rollout history deployment/api -n shop` и `kubectl get rs -n shop` показывают ревизии. Если 5xx начались в пределах десяти минут после выката — откат, не дожидаясь анализа. Параллельно `kubectl logs -n shop -l app=api --tail=100 --since=10m | grep -i error` даёт материал для postmortem, но не задерживает откат.

Рост latency без ошибок. Смотреть, где именно: сервис, база, внешняя зависимость. `kubectl top pods -n shop` покажет CPU и память, p95 по роутам — какой эндпоинт тянет вниз. Частые

причины: исчерпан пул соединений, запрос без индекса, троттлинг из-за CPU limit, ретраи, умножающие нагрузку.

OOM. `kubectl get pods -n shop` покажет `OOMKilled` и растущее число рестартов, `kubectl describe pod` — `Last State: Terminated, Reason: OOMKilled, Exit Code: 137`. Быстрая мера — поднять limit, если на узле есть память; настоящая причина — утечка или запрос, тянущий слишком много.

Диск. `df -h` по всем точкам монтирования, затем `du -xh --max-depth=1 /var | sort -h | tail -10`, чтобы найти, что выросло. Виновники: логи без ротации, docker-слои от частых сборок (`docker system df`), WAL базы, core-дампы. Быстрое освобождение — `journalctl --vacuum-size=500M`; при 100% заполнения сначала освобождают хотя бы гигабайт, иначе процесс не сможет писать.

DNS. Симптом коварный: часть запросов падает с `no such host` или висит по 5 секунд. `dig +short api.example.com`, `dig @10.96.0.10 api.example.com` и `dig +trace` показывают, на каком уровне ломается. В Kubernetes отдельно проверяют `kubectl -n kube-system logs -l k8s-app=kube-dns --tail=50` и conntrack: при переполнении таблицы DNS-запросы теряются.

Сертификат. Ошибка TLS у клиентов, метрика «истекает через 0 дней». Проверка:

bash

```
openssl s_client -connect shop.example.com:443 -servername shop.example.com
    </dev/null 2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates
```

```
subject=CN=shop.example.com
issuer=C=US, O=Let's Encrypt, CN=R11
notBefore=Jul 29 10:00:00 2026 GMT
notAfter=Oct 27 10:00:00 2026 GMT
```

`notAfter` в прошлом — подтверждение. Лечение: перевыпуск (cert-manager делает это сам, если не сломан ClusterIssuer), установка, проверка на клиентах с закэшированным старым сертификатом. Профилактика — алерт за 21 и за 7 дней, а не за час.

Коммуникация с пользователями

Молчание во время инцидента воспринимается как «они не знают, что делают». Первое сообщение публикуется, когда влияние подтверждено, даже если причина неизвестна: что не работает, кого касается, когда будет следующее обновление.

Шаблон: «Мы наблюдаем проблемы с оформлением заказов: часть запросов завершается ошибкой. Платёжные данные не затронуты. Следующее обновление в 11:15». Без предположений о причине, без обещаний сроков, без технических деталей, которые ничего не говорят пользователю.

Частота задаётся уровнем: SEV1 — каждые 15 минут, даже если новостей нет. Пропущенное обновление хуже плохой новости: люди начинают писать в поддержку и в соцсети, и вы получаете второй фронт. После восстановления — сообщение о факте и обещание разбора; публичный postmortem нужен для SEV1 и для всего, о чём знали пользователи.

Postmortem без обвинений

Цель postmortem — изменить систему так, чтобы этот класс отказов не повторился, а не найти виноватого. Обвинительный разбор гарантирует, что в следующий раз люди будут скрывать детали: «я не деплоил», «мне не сказали». Blameless не означает «без ответственности» — он означает, что ответственность за исправление лежит на системе и владельцах процессов, а не на человеке, который нажал кнопку в 2:00, потому что процесс позволял нажимать её без проверки.

Структура документа: заголовок и метаданные (дата, длительность, severity, сервисы, автор); влияние в числах (пользователи, запросы, минуты, ущерб); таймлайн с точностью до минуты; корневая причина — техническая и конкретная; способствующие факторы — почему ошибка стала инцидентом; что сработало; action items с владельцем, сроком и типом.

Корневая причина почти всегда одна, способствующих факторов несколько. «Инженер ошибся в конфиге» — не корневая причина, а её отсутствие: если конфиг проходил валидацию, значит валидация не покрывала этот случай. «Миграция

добавляла NOT NULL без значения по умолчанию, из-за чего запись блокировалась на 40 секунд» — корневая причина.

Action items делятся на три типа, и деление важнее формулировок. Предотвращение — чтобы не повторилось (обратно совместимая миграция в два шага). Обнаружение — чтобы заметить быстрее (алерт на p99 записи, а не только на 5xx). Смягчение — чтобы уменьшить ущерб (автоматический откат по неуспешному healthcheck). Пункт «быть внимательнее» не является action item: его нельзя выполнить и нельзя проверить. Хороший признак — пункты попадают в обычный трекер с дедлайнами; плохой — их двадцать, и через месяц не сделано ни одного.

Метрики инцидентов

MTTD (mean time to detect) — от начала влияния до первого сигнала; показывает качество мониторинга, а не скорость людей. Если MTTD 40 минут, никакой героизм не поможет. MTTR (mean time to recover) — от сигнала до восстановления, включает диагностику и откат. MTBF — среднее время между инцидентами, показатель зрелости.

Средние обманчивы: один инцидент на 8 часов сдвигает MTTR всего квартала, поэтому смотрят распределение и худшие 10%. И отдельно — **повторяемость**: доля инцидентов, у которых корневая причина уже встречалась. Если она растёт, postmortem пишутся, а action items не делаются, и процедура превратилась в ритуал.

On-call: ротация, эскалация, усталость

Дежурство — не «быть на телефоне», а обязательство отреагировать за фиксированное время (обычно 5–15 минут) и иметь право и возможность действовать. Без права откатывать и без доступа к системам дежурный — приёмщик жалоб.

Ротация — обычно неделя, с передачей смены голосом: что горит, что в процессе, какие изменения запланированы. Эскалация описана заранее: не ответил за 5 минут — звонок

второму, через 10 — третьему, через 15 — руководителю. Автоматически, а не «если что, звоните мне».

Усталость — главный враг. Человек, которого разбудили трижды за ночь, утром принимает решения хуже, чем трезвый новичок. Поэтому: не больше недели подряд, отгул после тяжёлой ночи, и жёсткое правило — **ночью будят только то, что требует действий человека прямо сейчас**.

Что не должно будить: CPU выше 80% без влияния на сервис, тикет от сканера уязвимостей, диск на 85% при работающей автоочистке, ошибки в неиспользуемом сервисе, «сертификат истекает через 30 дней». Что должно: 5xx выше порога, полный отказ, потеря данных, быстрое сжигание error budget. Если дежурного будят больше двух раз за смену без необходимости — проблема в алертах, а не в дежурном.

Разбор учебного инцидента

Магазин, три реплики API, деплой в 10:20 добавил индекс на таблицу `orders` через миграцию. Алерт сработал в 10:44.

10:44 — `ApiErrorBudgetBurnFast`: доля 5xx 18% за 5 минут. Дежурный подтверждает по дашборду: ошибки на `/orders`, p95 выросла с 180 мс до 6 с.

10:46 — открыт канал `#inc-2026-10-29-orders`, назначен IC, уровень SEV2: оформление заказов недоступно, каталог и оплата работают.

10:48 — первая гипотеза: деплой в 10:20. `kubectl rollout history` показывает ревизию 42 от 10:19. IC принимает решение откатывать, не дожидаясь анализа логов.

10:50 — `kubectl rollout undo deployment/api -n shop` → ревизия 41, `rollout status` завершается через 70 секунд.

10:53 — 5xx падают до 2%, но не до нуля: соединения к базе висят в блокировке. В `pg_stat_activity` — 12 запросов в состоянии `active` с ожиданием `Lock: relation`.

10:57 — отменён зависший `CREATE INDEX` (выполнялся без `CONCURRENTLY` и держал `ACCESS EXCLUSIVE` на `orders`). 5xx → 0, p95 → 200 мс.

11:05 — статус-страница: «Оформление заказов восстановлено. Ошибки наблюдались с 10:24 до 10:57. Причина — блокировка таблицы заказов при миграции. Разбор опубликуем в течение трёх дней».

Postmortem. Влияние: 33 минуты, 4 100 неудачных запросов, 2 700 пользователей, около 180 незавершённых корзин. Корневая причина: `CREATE INDEX` без `CONCURRENTLY` на таблице с активной записью; миграция шла в одном шаге с деплоем, поэтому откат приложения не снимал блокировку. Способствующие факторы: миграции не проходят ревью на блокировки, нет алерта на `pg_locks`, откат приложения не предусматривает откат миграции, `lock_timeout` не задан. Что сработало: алерт по симптому, решение откатывать за 4 минуты, канал и IC с самого начала.

Action items: `lock_timeout = 3s` для миграций (платформа, две недели); правило «индексы только `CONCURRENTLY`» в линтере миграций (бэкенд, неделя); алерт на блокировки дольше 30 секунд (SRE, две недели); шаг «проверить блокировки после миграции» в runbook деплоя (дежурный, три дня).

Типичные ошибки

1. **Диагностика вместо остановки крови.** Команда двадцать минут читает логи, пока сервис лежит, хотя откат занял бы три минуты. Причина — желание понять причину до действия и страх «откат не поможет».
2. **Нет назначенного IC.** Все чинят, никто не сводит картину; двое одновременно меняют конфиг. Причина — уверенность, что «мы маленькие, у нас и так все в курсе».
3. **Разбор с поиском виноватого.** После первого такого разбора люди скрывают детали, и следующие postmortem бесполезны. Причина — путаница между ответственностью за исправление и наказанием.

4. **Action items без владельцев и сроков.** «Нужно улучшить мониторинг» живёт в документе годами. Причина — разбор воспринимается как отчёт, а не как план работ.
5. **Алерты на причины.** Ночные звонки из-за CPU 85% при здоровом сервисе; через месяц дежурные спят под алерты. Причина — метрику проще подключить, чем сформулировать симптом.
6. **Спор «это база» / «это приложение».** Время уходит на переписку вместо локализации. Причина — метрики принадлежат командам, а не сервису.
7. **Отсутствие таймлайна.** Через три дня никто не помнит порядок событий, postmortem строится на догадках. Причина — канал заводится, но записи не ведутся.

Чек-лист

- [ ] Уровни severity и критерии влияния определены и известны всей команде.
- [ ] Роль IC назначается в первые минуты и не совмещается с тушением.
- [ ] Для каждого сервиса есть канал инцидента и шаблон первого сообщения.
- [ ] Таймлайн ведётся по ходу, включая отвергнутые гипотезы.
- [ ] Откат автоматизирован и проверен на staging; миграции обратно совместимы.
- [ ] Runbook есть для каждого page-алерта, и в нём первые три команды.
- [ ] Postmortem проводится в течение недели, action items — с владельцем и сроком в трекере.
- [ ] On-call: ротация не длиннее недели, эскалация автоматическая, ночью — только симптомы.

Упражнения

1. Составьте таблицу severity для своего сервиса: критерий влияния, кто реагирует, частота обновлений. Проверьте её на

трёх прошедших инцидентах.

2. Проведите учебный инцидент: отключите одну реплику API и добейтесь, чтобы её отсутствие замечалось только по метрикам. Засеките MTTD и MTTR.
3. Напишите postmortem по инциденту из упражнения 2: влияние в числах, таймлайн, корневая причина, три action items с владельцами и сроками.
4. Доведите диск на тестовом узле до 100% и восстановите сервис. Опишите команды и то, что помешало бы вам на реальном узле без консоли.
5. Разберите текущие page-алерты: для каждого ответьте, требует ли он действий ночью. Всё, что не требует, переведите в тикеты и посчитайте, сколько звонков это убирает.

Ответы и разбор к этим упражнениям — в приложении D, часть 4.

Приложение А. Чек-лист продакшн-сервиса

Чек-лист читают выборочно: перед первым деплоем, на ревью архитектуры, при разборе инцидента. Поэтому каждый пункт — проверяемое утверждение с обоснованием: не «используйте таймауты», а «таймаут задан на каждом внешнем вызове, иначе зависшая зависимость исчерпает пул воркеров». Пункт, который нельзя защитить на ревью, в список не попадает.

Блоки идут в порядке пути изменения: код → образ → конфигурация → сеть → данные → наблюдаемость → эксплуатация → безопасность.

Код и зависимости

- [ ] **Версии зафиксированы lock-файлом** (`uv.lock`, `go.sum`, `package-lock.json`) — сборка без него через месяц возьмёт другие версии, и в прод поедет код, которого не было в тестах.
- [ ] **Lock-файл лежит в репозитории и обновляется отдельным PR** — обновление зависимости должно быть видно в истории и откатываться одним revert.
- [ ] **В репозитории нет секретов**: ни `.env` с паролями, ни ключей в коде, ни токенов в истории — попавший в git секрет считается скомпрометированным, помогает только ротация.
- [ ] **Secret scanning работает в CI** (`gitleaks detect --redact`, secret scanning хостинга) — человек забудет про ключ, автоматика проверит на каждом push.
- [ ] **Линтер и форматтер закреплены версией и запускаются в CI** — иначе ревью обсуждает отступы, а разные версии линтера дают разные приговоры одному коду.
- [ ] **Статический анализ проходит**: `mypy --strict` для Python, `go vet` и `golangci-lint` для Go — часть дефектов дешевле ловить до запуска, чем в инциденте.

- [ ] **Сборка воспроизводится из чистого клона** (`git clone … && make build`) — если нужны локальные доустановки, у команды нет сборки, а есть устная традиция.
- [ ] **Мёртвый код и фича-флаги с постоянным значением удалены** — флаг, который всегда включён, это ветка, которую никто не тестирует.
- [ ] **Правка закрыта тестом, который падает без неё** — тест, написанный после кода и не проверенный на «красном», фиксирует текущее поведение, а не требуемое.
- [ ] **Зависимости берутся из доверенного registry** с проверкой хэшей — подмена пакета в публичном индексе это атака на цепочку поставок, а не теория.

Сборка и образ

- [ ] **Multi-stage сборка**: компилятор и dev-зависимости остаются в стадии `builder`, в финальный образ копируется только артефакт — иначе компилятор, `.git` и кэш пакетов уезжают в прод.
- [ ] **Базовый образ закреплён по digest** (`python:3.14-slim@sha256:…`), а не `latest` — тег перезаписывается, digest нет.
- [ ] **Процесс работает не от root**: `USER app` в Dockerfile, `runAsNonRoot` в поде — компромисс приложения не должен сразу давать root в контейнере.
- [ ] **Корневая ФС только для чтения**, запись — в явно смонтированные каталоги (`--read-only --tmpfs /tmp`) — так ловится и код, который пишет в случайные места.
- [ ] **Образ собирается в CI, а не на ноутбуке**, и тот же digest едет в стейджинг и прод — «собрал локально» ломает и воспроизводимость, и подпись.
- [ ] **SBOM генерируется на сборке** (`syft`, `trivy image --format cyclonedx`) и хранится рядом с образом — при новой CVE нужно за минуты узнать, затронуты ли вы.

- [ ] `.dockerignore` **исключает** `.git`**,** `node_modules`**,** `.venv`**, тесты** — контекст сборки определяет и скорость, и размер слоёв.
- [ ] **Образ подписан** (`cosign sign`), в кластере проверяется подпись — тег можно перезаписать, подпись привязана к digest.
- [ ] **В слоях нет секретов и мусора**: `docker history --no-trunc` не содержит команд с ключами, а `.git` и тестовые данные не попали в контекст — иначе секрет уезжает туда же, куда и образ.

Конфигурация

- [ ] **Конфиг приходит из окружения или файла-секрета, а не из кода** — один образ должен работать в стейджинге и проде без пересборки.
- [ ] **Обязательные параметры валидируются на старте** (pydantic-settings, `go-playground/validator`) — упасть при запуске дешевле, чем на первом запросе.
- [ ] **Fail-closed**: нет пароля к БД, ключа подписи или сертификата — сервис не стартует, а не работает в «режиме без проверки».
- [ ] **Различие сред — только в значениях, не в наборе ключей** — иначе прод исполняет ветку кода, которой не было в тестах.
- [ ] **Секреты не видны в** `docker inspect` **и в env образа** — переменные читает любой с доступом к API; для секретов нужен файл или vault.
- [ ] **У дефолтов один источник правды** — дефолт, продублированный в трёх местах, разъезжается молча.
- [ ] **Изменение конфига не требует пересборки**, но требует рестарта или релоада, и это записано в runbook.
- [ ] **Файлы секретов монтируются с правами 0400 и владельцем процесса** — секрет, читаемый всем в контейнере, бесполезен как секрет.

Сеть и доступ

- [ ] **TLS терминируется на границе, внутри — mTLS или приватная сеть** — «внутри доверенная сеть» перестаёт быть правдой в первый день компромисса.
- [ ] **TLS не ниже 1.2, лучше 1.3**, слабые шифры отключены — сканеры и комплаенс спрашивают об этом первыми.
- [ ] **Таймаут задан на каждом внешнем вызове** (connect, read, total) и меньше таймаута вызывающего — иначе зависшая зависимость съест пул воркеров.
- [ ] **Есть общий бюджет времени запроса**: дедлайн на входе, context/таймаут вниз по стеку — иначе каскад ретраев превращает деградацию в отказ.
- [ ] **Rate limit на входе** (token bucket по IP или токену) — без него один клиент с циклом `while true` кладёт сервис.
- [ ] **Ограничены размер тела и число одновременных соединений** — иначе память кончается от нескольких медленных клиентов.
- [ ] **Healthcheck разделён**: liveness не зависит от БД, readiness проверяет готовность принимать трафик — иначе падение БД вызывает лавину перезапусков.
- [ ] **Порт слушается на нужном интерфейсе** (`127.0.0.1` для админки и метрик) — «0.0.0.0 по привычке» публикует отладку наружу.
- [ ] **Исходящие соединения ограничены политиками** — сервису нужны БД и два API, а не весь интернет.
- [ ] **Клиент проверяет сертификат сервера**, отключение проверки не зашито в код и не приходит из конфига — `verify=False` в проде обнуляет TLS.

Данные

- [ ] **Бэкап есть, и восстановление проверено** на чистом окружении за последние 30 дней — непроверенный бэкап — это гипотеза, а не бэкап.

- [ ] **RPO и RTO записаны числами** — «бэкап раз в сутки» отвечает, сколько потеряем, но не за сколько поднимемся.
- [ ] **Копия хранится вне основного отказа**: другой регион или аккаунт, отдельные креды — иначе одна авария уносит и данные, и бэкапы.
- [ ] **Миграции совместимы в обе стороны** (expand/contract): сначала новая колонка, потом код, потом удаление старой — иначе откат приложения ломает схему.
- [ ] **Миграции версионированы и идемпотентны** (Alembic, goose, golang-migrate) — повторный запуск не должен падать.
- [ ] **Долгие миграции не блокируют таблицу** (`CREATE INDEX CONCURRENTLY`, батчи) — `ALTER TABLE` на миллионе строк останавливает сервис целиком.
- [ ] **Данные, которые нельзя потерять, реплицируются синхронно** — асинхронная реплика теряет последние транзакции при падении мастера.
- [ ] **Удаление данных предусмотрено** (retention, запрос на удаление персональных данных) — «храним всё вечно» тоже решение, только со штрафом.
- [ ] **Пул соединений к БД ограничен и имеет таймаут ожидания** — без лимита всплеск трафика открывает тысячи соединений и роняет БД раньше, чем приложение.

Наблюдаемость

- [ ] **Метрики RED на каждом сервисе** (rate, errors, duration) и USE для ресурсов — без них инцидент обсуждают по ощущениям.
- [ ] **Латентность считается по перцентилям, а не средним** — среднее скрывает хвост, который и чувствуют пользователи.
- [ ] **Логи структурные (JSON) и с correlation id** — grep по тексту работает до первого масштабирования, дальше нужны поля.

- [ ] **Уровень логирования меняется без пересборки**, `debug` не включён в проде — это и объём, и утечка данных.
- [ ] **Алерты по симптомам, а не по причинам**: «5xx выше 2% пять минут», а не «CPU выше 80%» — высокий CPU может быть нормой.
- [ ] **У каждого алерта есть действие**: ссылка на runbook, владелец и порог, при котором он правда требует вмешательства.
- [ ] **Дашборд открывается за один клик** и показывает золотые сигналы, а не сорок графиков.
- [ ] **Трейсинг включён на границах** (HTTP, БД, очередь) с сэмплированием — логи говорят «что», трейс говорит «где».
- [ ] **Логи и метрики живут дольше инцидента**: 14–30 дней минимум — разбор обычно начинается на следующий день.
- [ ] **Лейблы метрик не кардинальны**: `user_id`, `request_id` и сырой URL в лейблах взрывают хранилище — их место в логах и трейсах.

Эксплуатация

- [ ] **Runbook на каждый сервис**: как проверить, как перезапустить, как откатить, кого звать — в три часа ночи читают runbook, а не архитектурный документ.
- [ ] **On-call расписан** с эскалацией и вменяемым горизонтом — «отвечают все» значит «не отвечает никто».
- [ ] **Requests и limits заданы**: requests для планировщика, limits для защиты соседей — без них один под съедает ноду.
- [ ] **Graceful shutdown реализован**: SIGTERM → перестать принимать → доработать → выйти, и укладывается в `terminationGracePeriodSeconds`.
- [ ] **Есть preStop-пауза или задержка вывода из балансировки** — иначе трафик приходит в уже умирающий процесс.

- [ ] **Несколько реплик и PodDisruptionBudget** — обновление и обслуживание ноды не должны быть заметны.
- [ ] **Откат проверен на практике**, а не описан: предыдущий образ запускается и совместим с текущей схемой БД.
- [ ] **Дежурный набор команд лежит в одном месте** (`rollout undo`, `drain`, `scale`) — импровизация под давлением даёт вторую аварию.
- [ ] **Версия и commit видны снаружи** (`/healthz` или метрика `build_info`) — иначе первый вопрос инцидента «что сейчас развёрнуто» остаётся без ответа.

Безопасность

- [ ] **CVE-скан блокирует сборку** при critical/high с доступным фиксом (trivy, grype) — скан, который только пишет отчёт, ни на что не влияет.
- [ ] **Секреты ротируются по расписанию и после ухода человека** — неограниченный срок жизни превращает утечку в вечный доступ.
- [ ] **Минимальные привилегии**: сервисный аккаунт с нужными verbs, пользователь БД с правами на свою схему, `privileged: false`, capabilities срезаны.
- [ ] **Доступ к проду именной и аудируемый** (SSO, `kubectl` через прокси с логами) — общий root-пароль делает ответ на вопрос «кто это сделал» невозможным.
- [ ] **Зависимости проверяются не только на CVE, но и на поддержку** — заброшенный пакет — это будущая CVE без патча.
- [ ] **Секреты не попадают в логи и трейсы**: маскирование на уровне логгера, а не «мы аккуратно».
- [ ] **Артефакты проверены на утечку**: `.env`, ключи и дампы не попадают ни в образ, ни в бэкап.
- [ ] **Образ пересканируется и после публикации**, а не только на сборке — CVE появляются у неизменного образа, потому что меняется база уязвимостей.

Минимум перед первым деплоем в прод

1. Образ собран в CI, закреплён по digest, работает не от root.
2. Lock-файл в репозитории, секретов в git нет.
3. Обязательные параметры валидируются, без секрета сервис не стартует.
4. Таймауты заданы на всех внешних вызовах.
5. Liveness и readiness разделены и настроены.
6. Миграции совместимы с предыдущей версией приложения.
7. Бэкап снят и хотя бы раз восстановлен вручную.
8. Метрики, структурные логи и один алерт на всплеск 5xx.
9. Лимиты ресурсов и graceful shutdown.
10. Runbook с процедурой отката и фамилия дежурного.

Что проверить при инциденте

Сначала сузить область, потом менять. Любое изменение до диагностики уничтожает улики, а «на всякий случай перезапустил» стирает причину.

1. **Масштаб и начало**: с какого времени, все пользователи или часть, какая доля запросов падает — это отсекает «наш сервис» от «сеть и провайдер».
2. **Что менялось за последние два часа**: деплой, миграция, конфиг, фича-флаг, сертификат.
3. **Ресурсы**: CPU throttling, память и OOMKilled, диск и inodes, число соединений.
4. **Зависимости**: БД (пул, медленные запросы, блокировки), DNS, внешние API, очередь.
5. **Сеть**: код ответа на балансировщике, срок сертификата, потери пакетов до пода.
6. **Логи и трейсы** по correlation id конкретного упавшего запроса.

7. **Решение**: причина коррелирует с изменением — откат; не коррелирует — стабилизация (масштаб, лимиты, отключение фичи).
8. **Коммуникация**: статус, оценка, следующий апдейт через 30 минут.

Симптом	Первое, что смотреть
5xx на балансировщике, поды живы	логи приложения, `kubectl logs --previous`, ошибки зависимостей
502/504	readiness, таймаут upstream, порт в Service и targetPort
поды в CrashLoopBackOff	`kubectl describe pod` → Last State, exit code, лимит памяти
латентность выросла, ошибок нет	CPU throttling, пул соединений к БД, GC-паузы, медленный запрос
трафик не доходит	пустые Endpoints, селектор, NetworkPolicy, DNS-имя
внезапные 401/403	ротация секретов, истёкший токен или сертификат, RBAC
диск заполнен	логи, `/tmp`, WAL, старые образы, inodes

Первые команды при разборе:

bash

```
kubectl get pods -o wide --sort-by=.status.startTime | tail -5
kubectl describe pod api-7d9f-x2 | sed -n '/Events/,$p'
kubectl logs api-7d9f-x2 --previous --tail=100
kubectl rollout history deploy/api | tail -3
ss -s; df -h /; df -i /
```

Приложение B. Шпаргалки

Команда, флаг и одна строка «когда звать». Всё, что здесь есть, выполняется локально без интернета.

bash

Параметры и подстановки

Конструкция	Что делает и когда нужна
`"$@"`	все аргументы отдельными словами; `$*` склеивает их в одно — почти всегда ошибка
`${v:-def}`	значение по умолчанию, если пусто или не задано
`${v:=def}`	то же, но записывает результат в переменную
`${v:?msg}`	прервать скрипт с сообщением, если пусто
`${#v}`	длина строки
`${v%.*}` / `${v%%.*}`	срезать с конца первое / последнее расширение
`${v#*/}` / `${v##*/}`	срезать с начала до первого / последнего слэша (basename)
`${v/a/b}` / `${v//a/b}`	заменить первое / все вхождения без sed
`${v,,}` / `${v^^}`	нижний / верхний регистр (bash 4+)
`$(cmd)`	подстановка вывода; обратные кавычки не использовать
`((n++))`	арифметика; под `set -e` при результате 0 вернёт 1 — пишите `((++n))`

Условия и тесты

Проверка	Когда
`[[ -f $p ]]`	файл существует; `-d` каталог, `-e` существует, `-s` непустой, `-x` исполняемый
`[[ -z $s ]]` / `[[ -n $s ]]`	строка пустая / непустая
`[[ $a == *.log ]]`	glob-сравнение; справа от `==` кавычки не ставят
`[[ $s =~ ^v([0-9]+)$ ]]`	regex, совпадение в `${BASH_REMATCH[1]}`
`(( a > b ))`	числовое сравнение; строки — `[[ $a < $b ]]`
`case $x in a) … ;; *) … ;; esac`	ветвление по шаблонам; варианты объединяются вертикальной чертой, читается лучше лестницы `elif`
`command -v jq >/dev/null`	проверить наличие команды, а не `which`

Редиректы и конвейеры

Запись	Смысл
`> f` / `>> f`	перезаписать / дописать stdout
`2> f`	stderr в файл
`> f 2>&1`	stdout и stderr в файл; порядок важен, `2>&1 > f` ошибочен
`&> f`	то же короче (bash)
`2>&1` перед конвейером	stderr тоже уйдёт в конвейер
`2>/dev/null`	выбросить ошибки
`<<<"$s"`	here-string: строка на stdin без echo и конвейера
`cmd < <(other)`	process substitution: подставить вывод как файл
`tee -a log`	раздвоить поток: и на экран, и в файл
`find . -print0` + `xargs -0 -r -n1`	по одному аргументу, безопасно с пробелами в именах
`exec 3>f` … `>&3`	свой дескриптор, чтобы не смешивать выводы

Оператор `cmd |& next` — короткая запись `cmd 2>&1 | next`: в конвейер уходят оба потока.

Каркас скрипта

bash

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
trap 'echo "fail at line $LINENO" >&2' ERR
```

`-e` не срабатывает внутри `if`, `while` и после `&&` — там код проверяют явно. `pipefail` нужен, иначе `false | true` вернёт 0.

git

Состояния и просмотр

Команда	Когда
`git status -sb`	короткий статус плюс ветка и расхождение с upstream
`git diff`	рабочая копия против индекса
`git diff --staged`	индекс против HEAD — ровно то, что уйдёт в коммит
`git diff HEAD`	всё незакоммиченное сразу
`git log --oneline --graph --decorate -20`	история с ветками и тегами
`git show v1.4.2:app/main.py`	файл из коммита без переключения
`git stash -u`	отложить правки вместе с новыми файлами

Отмена

Команда	Когда
`git restore f`	выбросить правки в файле, вернув версию из индекса
`git restore --staged f`	убрать из индекса, правки оставить
`git commit --amend --no-edit`	дописать в последний коммит без правки сообщения
`git revert <sha>`	коммит-отмена; единственный безопасный способ в общей ветке
`git reset --soft HEAD~1`	снять коммит, изменения оставить в индексе
`git reset --hard <sha>`	уничтожить рабочую копию — только для локального и с `reflog` в уме
`git reflog`	журнал перемещений HEAD: спасает после неудачного reset или rebase
`git restore --source=<sha> -- path`	вернуть файл из старого коммита

Поиск

Команда	Когда
`git log -S'secret_key'`	коммиты, изменившие число вхождений строки (pickaxe)
`git log -G'regex'`	коммиты, где diff подходит под regex
`git log -L 40,60:app/main.py`	история конкретного диапазона строк
`git blame -w -C -L 10,20 f.py`	кто и когда, игнорируя пробелы и переносы
`git grep -n pattern v1.4.2`	поиск в старом теге
`git bisect start && git bisect run ./test.sh`	бинарный поиск ломающего коммита

rebase и публикация

bash

```
git fetch --prune
git rebase -i origin/main                  # squash/fixup/reword
git rebase --autosquash -i HEAD~5          # подхватит fixup! и squash!
git rebase --onto main old-base feature    # перенести хвост на другую базу
git push --force-with-lease                # после rebase; не --force
```

`--force-with-lease` откажется пушить, если ветку успели обновить. Конфликт: правка → `git add` → `git rebase --continue`; выйти — `--abort`, пропустить — `--skip`.

docker

Жизненный цикл

Команда	Когда
`docker run -d --name api -p 8080:8000 api:1.4.2`	запуск в фоне с публикацией порта
`docker ps -a`	все контейнеры; без `-a` только живые
`docker logs -f --tail 100 api`	логи; `--since 10m` — за последние 10 минут
`docker exec -it api sh`	зайти внутрь; `sh` есть всегда, `bash` не факт
`docker inspect -f '{{.State.ExitCode}} {{.State.OOMKilled}}' api`	почему контейнер умер
`docker stats --no-stream`	CPU, память, сеть на текущий момент
`docker stop -t 30 api`	SIGTERM и 30 секунд до SIGKILL
`docker rm -f api`	удалить, даже если работает
`docker update --memory 512m api`	поменять лимит без пересоздания

Отладка

bash

```
docker run --rm -it --entrypoint sh api:1.4.2   # осмотреться без CMD
docker run --rm -it --network container:api alpine sh  # сеть пода
docker cp api:/app/log.txt ./                   # достать файл
docker diff api                                 # правки в слое
docker history --no-trunc api:1.4.2             # команды и слои
```

Exit code 137 — SIGKILL или OOM, 143 — SIGTERM, 1 — ошибка приложения. Если контейнер падает сразу, `--entrypoint sh` покажет, есть ли вообще бинарник в образе.

Очистка

Команда	Когда
`docker system df`	сколько занимают образы, контейнеры, тома, кэш
`docker system prune -af --volumes`	снести всё неиспользуемое, включая тома
`docker image prune -a --filter until=168h`	образы старше недели
`docker builder prune -af`	кэш сборки — обычно самый большой потребитель
`docker volume ls -f dangling=true`	бесхозные тома

compose

Команда	Когда
`docker compose up -d --build`	поднять и пересобрать изменённые сервисы
`docker compose ps` / `logs -f api`	состояние и логи одного сервиса
`docker compose exec api sh`	зайти в запущенный сервис
`docker compose config`	итоговый конфиг после merge и подстановок — проверять перед деплоем
`docker compose run --rm api pytest`	разовая команда в новом контейнере
`docker compose down -v`	остановить и удалить тома: данные пропадут

kubectl

Чтение

Команда	Когда
`kubectl get pods -o wide`	поды с нодой и IP
`kubectl get pods -A -l app=api --sort-by=.status.startTime`	фильтр по метке и сортировка по времени
`kubectl describe pod api-7d9f-x2`	события и причины: ImagePullBackOff, Unhealthy
`kubectl logs -f --tail=200 deploy/api`	логи по деплойменту
`kubectl logs api-7d9f-x2 --previous`	логи предыдущего, уже упавшего контейнера
`kubectl logs -l app=api --max-log-requests=10`	логи всех подов по селектору
`kubectl exec -it deploy/api -- sh`	внутрь пода
`kubectl port-forward svc/api 8080:80`	локально до сервиса без внешнего адреса
`kubectl cp api-7d9f-x2:/tmp/dump ./dump`	скопировать файл из пода
`kubectl top pods` / `top nodes`	потребление, нужен metrics-server
`kubectl auth can-i delete pods -n prod`	проверить права, а не гадать
`kubectl config current-context`	куда вы сейчас стреляете — смотреть перед `apply`

rollout

Команда	Когда
`kubectl rollout status deploy/api --timeout=120s`	дождаться завершения выкатки в скрипте
`kubectl rollout history deploy/api`	ревизии; `--revision=3` — детали
`kubectl rollout undo deploy/api`	откат на предыдущую ревизию
`kubectl rollout restart deploy/api`	перезапуск без правки манифеста, чтобы подхватить ConfigMap
`kubectl scale deploy/api --replicas=5`	ручное масштабирование

Форматирование и события

bash

```
 # имена, статусы и рестарты без шапки
kubectl get pods -o custom-columns=\
NAME:.metadata.name,PHASE:.status.phase,\
RESTARTS:.status.containerStatuses[0].restartCount --no-headers

 # образы всех контейнеров в namespace
kubectl get deploy -o custom-columns=\
NAME:.metadata.name,'IMAGE:.spec.template.spec.containers[*].image'

 # одно поле без YAML
```

```
kubectl get pod api-7d9f-x2 -o jsonpath='{.status.podIP}'

 # свежие события внизу
kubectl get events --sort-by=.lastTimestamp -A | tail -30
kubectl get events --field-selector reason=FailedScheduling
kubectl explain deploy.spec.strategy --recursive | head -30
```

`jsonpath` молча пропускает отсутствующие поля, а `-o yaml` надёжнее, когда структура незнакома.

Отладка

bash

```
kubectl debug -it api-7d9f-x2 --image=busybox --target=api  # в под
kubectl debug node/node-1 -it --image=busybox               # осмотр ноды
kubectl describe node node-1 | sed -n '/Allocated resources/,/Events/p'
```

systemd

Управление

Команда	Когда
`systemctl status api.service`	состояние, последние строки журнала, PID
`systemctl restart api` / `reload api`	перезапуск / перечитать конфиг без обрыва
`systemctl enable --now api`	в автозапуск и запустить сразу
`systemctl list-units --failed`	что сломано на машине прямо сейчас
`systemctl list-timers --all`	таймеры и время следующего запуска
`systemctl cat api`	юнит и все drop-in'ы одним выводом
`systemctl edit api`	свой drop-in, не трогая чужой файл
`systemctl daemon-reload`	после правки юнита, иначе изменений не будет
`systemctl show api -p MainPID -p ExecMainStatus`	конкретные свойства для скриптов

journalctl

Команда	Когда
`journalctl -u api -f`	живой лог сервиса
`journalctl -u api --since "10 min ago"`	за период; `-b -1` — за прошлую загрузку
`journalctl -u api -p err -o json-pretty`	только ошибки, структурой
`journalctl -u api -g 'timeout'`	regex по журналу, быстрее конвейера с grep
`journalctl -k -n 100`	ядро: OOM, сеть, диск
`journalctl --disk-usage` / `--vacuum-size=500M`	сколько занимает и как урезать

Шаблон юнита

ini

```
[Unit]
Description=Shop API
After=network-online.target
Wants=network-online.target

[Service]
Type=notify
User=app
WorkingDirectory=/opt/api
EnvironmentFile=/etc/api/env
ExecStart=/opt/api/api --config /etc/api/config.yaml
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=2s
TimeoutStopSec=30
LimitNOFILE=65535
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/var/lib/api

[Install]
WantedBy=multi-user.target
```

`Type=notify` требует поддержки в приложении, иначе `simple`. Шаблон `api@.service` с `%i` даёт экземпляры: `systemctl start api@blue`.

сеть

Команда	Когда
`ss -tlnp`	кто слушает TCP и какой процесс
`ss -tanp state established`	установленные соединения по процессам
`ss -s`	сводка: сколько сокетов в каком состоянии
`ss -tin dst 10.0.0.5`	rtt, cwnd, ретрансмиты по конкретному пиру

Команда	Когда
`ip -br a` / `ip r` / `ip neigh`	адреса, маршруты, ARP-таблица кратко
`dig +short api.example.com`	только ответ
`dig api.example.com @1.1.1.1 +noall +answer`	спросить конкретный резолвер
`dig -x 10.0.0.5 +short`	обратная запись
`dig +trace example.com`	цепочка от корня: где ломается делегирование
`nc -zv host 5432`	жив ли порт, быстрее telnet
`mtr -rwc 20 host`	потери и задержка по хопам
`tcpdump -ni any port 443 -c 20 -A`	20 пакетов с содержимым; `-w f.pcap` — в файл
`ethtool -S eth0`	счётчики ошибок и дропов; фильтруйте вывод по `err`

curl с таймингами

bash

```
FMT='code=%{http_code} dns=%{time_namelookup} conn=%{time_connect}'
FMT="$FMT tls=%{time_appconnect} ttfb=%{time_starttransfer}"
FMT="$FMT total=%{time_total}\n"
curl -sS -o /dev/null -w "$FMT" https://api.example.com/health
```

`conn - dns` — сеть до сервера, `tls - conn` — рукопожатие (сертификат, CPU), `ttfb - tls` — работа бэкенда.

bash

```
curl -v --http2 https://api.example.com/          # версия и заголовки
curl --resolve api.example.com:443:10.0.0.5 \
  https://api.example.com/                        # обойти DNS
curl -sS -H 'Content-Type: application/json' -d '{"a":1}' https://api/x
curl -sS --retry 3 --retry-delay 1 --max-time 10 https://api/health
curl -sSI https://api.example.com/ | head -1       # только заголовки и код
```

TLS

bash

```
openssl s_client -connect api.example.com:443 \
  -servername api.example.com </dev/null 2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates

openssl s_client -connect api.example.com:443 -tls1_3 \
  -servername api.example.com </dev/null 2>&1 | grep -E 'Protocol|Cipher'

echo | openssl s_client -connect api.example.com:443 2>/dev/null \
  | openssl x509 -noout -checkend 604800 && echo "живёт ещё неделю"
```

`-servername` обязателен: без SNI сервер отдаст не тот сертификат или дефолтный.

python

Команда	Когда
`python -m venv .venv && . .venv/bin/activate`	окружение проекта
`uv sync --frozen`	поставить ровно то, что в `uv.lock`
`uv add httpx` / `uv add --dev pytest`	добавить зависимость и обновить lock
`uv run pytest -q`	запуск в окружении проекта без активации
`uv lock --upgrade-package httpx`	точечное обновление одной зависимости
`python -m pip list --outdated`	что устарело
`python -X dev -W error app.py`	режим разработки: предупреждения как ошибки
`export PYTHONUNBUFFERED=1`	логи сразу в stdout, без буфера — обязательно в контейнере

pytest

Флаг	Когда
`-x -q`	остановиться на первой ошибке
`-k "login and not slow"`	выбрать тесты по имени
`-m "not integration"`	выбор по маркерам
`--lf` / `--ff`	только упавшие / сначала упавшие
`-n auto`	параллельно, нужен pytest-xdist
`--cov=app --cov-report=term-missing`	покрытие с непокрытыми строками
`--durations=10`	самые медленные тесты
`-s --pdb`	показать print и встать в отладчик на падении
`--setup-show`	порядок фикстур, когда непонятно, что течёт

go

Команда	Когда
`go build -trimpath -ldflags="-s -w" -o api ./cmd/api`	бинарник без путей сборки и отладочных символов
`GOOS=linux GOARCH=arm64 go build ./cmd/api`	кросс-компиляция под другую платформу
`go vet ./...`	подозрительный код до запуска тестов
`go test ./... -race -count=1`	тесты с детектором гонок, без кэша
`go test -run TestPay -v -count=1`	один тест
`go test -bench . -benchmem -run '^$'`	бенчмарки с аллокациями
`go test ./... -coverprofile=c.out` + `go tool cover -func=c.out`	покрытие; последняя строка — `total`
`go tool pprof -http=:8080 localhost:6060/debug/pprof/profile`	CPU-профиль за 30 секунд (`?seconds=30`)
`go tool pprof -http=:8080 localhost:6060/debug/pprof/heap`	топ по памяти
`go mod tidy` / `go mod why -m x`	привести зависимости в порядок / понять, кто тянет модуль
`go list -m -u all`	доступные обновления модулей

bash

```
 # стеки всех горутин: искать зависшие в select или на мьютексе
curl '127.0.0.1:6060/debug/pprof/goroutine?debug=2'
```

Однострочники

bash

```
ps -eo pid,pcpu,pmem,rss,cmd --sort=-pcpu | head -10  # кто ест CPU
du -xh --max-depth=1 /var | sort -h | tail -10        # что занимает диск
find /var/log -type f -size +100M -printf '%s %p\n'   # крупные файлы
lsof -i :8080 -sTCP:LISTEN                            # кто держит порт
jq -r '.items[] | [.name, .status] | @tsv' data.json  # JSON в таблицу
awk -F'"' '{print $2}' access.log | sort | uniq -c | sort -rn  # топ URL
tar -C /srv -czf - app | ssh host 'tar -C /srv -xzf -'  # каталог по SSH
journalctl --since -1h -p warning | wc -l             # предупреждения за час
```

Приложение C. Глоссарий

Термин — определение в одну строку и глава, где он разобран. Порядок алфавитный.

A/B-тест — сравнение двух версий на случайных группах по заранее выбранной метрике; без критерия результат неотличим от шума. (гл. 42)

ACK — подтверждение в TCP с номером следующего ожидаемого байта; его отсутствие запускает ретрансмиссию. (гл. 7)

API — договор между программами: какие запросы можно послать и что вернётся; стабильность этого договора важнее реализации. (гл. 12a)

API gateway — единая точка входа: терминирует TLS, маршрутизирует запросы, применяет лимиты и аутентификацию. (гл. 14)

API key — долгоживущий секрет, которым вызывающая сторона подтверждает, что имеет право звать API; передаётся заголовком, а не в URL. (гл. 12a)

ARP — сопоставление IP-адреса и MAC в пределах одного L2-сегмента; таблица живёт в `ip neigh`. (гл. 6)

At-least-once — семантика доставки «хотя бы один раз»: сообщение не потеряется, но может прийти повторно — потребитель обязан быть идемпотентным. (гл. 34a)

Availability — доля успешных запросов за окно; 99,9% в месяц — это около 43 минут недоступности. (гл. 42)

B-tree index — сбалансированное дерево, которое превращает поиск строки в O(log n) вместо O(n) полного сканирования. (гл. 34b)

Backpressure — механизм, при котором медленный потребитель заставляет производителя ждать, а не копить очередь. (гл. 32)

Base image — образ, от которого наследуется слой; его размер и CVE определяют итоговый образ. (гл. 38)

Bearer token — токен доступа в заголовке `Authorization`, который сервер проверяет без дополнительного запроса; живёт ограниченное время. (гл. 12a)

Blue-green — два одинаковых окружения и переключение трафика между ними целиком, а не по долям. (гл. 39)

Branch — подвижный указатель на коммит, а не копия файлов; поэтому ветки почти бесплатны. (гл. 37)

Cache-Control — директивы кэширования (`max-age`, `no-store`, `s-maxage`) для браузера и промежуточных кэшей. (гл. 11)

Canary — выкатка на малую долю трафика с наблюдением метрик перед раскаткой на всех. (гл. 39)

CDN — сеть кэширующих узлов ближе к пользователю; снимает нагрузку с origin и сокращает RTT. (гл. 11)

CFS — планировщик Linux: делит CPU по весам групп, а не поровну между процессами. (гл. 4)

cgroup — механизм ядра, который ограничивает и учитывает CPU, память, I/O и число процессов. (гл. 5c)

CIDR — запись подсети «адрес/длина префикса»: `/24` — 256 адресов, `/32` — один хост. (гл. 6)

Circuit breaker — предохранитель: после серии ошибок вызовы к зависимости прекращаются на время. (гл. 21)

CNI — плагин, который выдаёт подам IP и настраивает маршрутизацию между ними. (гл. 41)

Commit — неизменяемый снимок дерева с метаданными, адресуемый хэшем SHA-1 или SHA-256. (гл. 37)

Consumer lag — отставание потребителя от производителя в сообщениях; главная метрика здоровья очереди. (гл. 34a)

Container image — стек слоёв только для чтения плюс конфиг запуска: env, команда, порт. (гл. 38)

Context — объект Go с дедлайном и отменой, который протягивают через все вызовы запроса. (гл. 32)

Copy-on-write — копирование страницы или файла только при записи; основа `fork` и overlayfs. (гл. 5)

Correlation ID — идентификатор запроса, который проносят через сервисы и логи для склейки событий. (гл. 42)

CPU throttling — принудительная пауза cgroup, исчерпавшей квоту; видна как рост latency при низком CPU. (гл. 5c)

DaemonSet — контроллер, который держит по поду на каждой или на выбранных нодах. (гл. 41)

Data race — одновременный доступ к памяти без синхронизации, где хотя бы одна операция — запись. (гл. 32)

Dead letter queue (DLQ) — очередь для сообщений, которые не удалось обработать; сохраняет их для разбора вместо бесконечного ретрая. (гл. 34a)

Deadlock — взаимная блокировка: каждый участник ждёт ресурс, который держит другой. (гл. 32)

Defer — отложенный вызов Go: выполняется при выходе из функции, в обратном порядке. (гл. 31)

Deployment — контроллер, управляющий ReplicaSet'ами и стратегией обновления подов. (гл. 41)

Digest — хэш образа (`sha256:…`); в отличие от тега однозначно определяет содержимое. (гл. 38)

DNS — распределённая иерархия имён; ответ кэшируется на TTL, поэтому смена адреса видна не сразу. (гл. 8)

Dockerfile — декларация сборки образа; каждая инструкция `RUN`, `COPY`, `ADD` создаёт слой. (гл. 38)

Drain — выселение подов с ноды перед обслуживанием с учётом PodDisruptionBudget. (гл. 41)

Drift — расхождение реальной инфраструктуры с описанной в коде; лечится планом и применением. (гл. 40)

eBPF — программа, которая исполняется в ядре по событию (syscall, пакет, функция) без пересборки ядра и перезапуска сервиса. (гл. 5d)

Edge-triggered — режим epoll: уведомление приходит на изменение состояния, а не пока данные есть. (гл. 15)

Entry point — точка входа процесса: `ENTRYPOINT` в образе или `console_scripts` в Python-пакете. (гл. 24)

epoll — механизм ядра, отдающий готовые дескрипторы списком за один системный вызов. (гл. 15)

ETag — версия представления для условного запроса `If-None-Match`, ответ — `304 Not Modified`. (гл. 11)

Event loop — цикл, который ждёт готовности дескрипторов и раздаёт колбэки; один поток обслуживает тысячи соединений. (гл. 15)

Exit code — число, возвращаемое процессу родителю: 0 успех, 127 команда не найдена, 128+N — смерть от сигнала N. (гл. 26)

Expand/contract — схема миграции без простоя: сначала добавить совместимое, потом переключить код, потом удалить старое. (гл. 34)

EXPLAIN ANALYZE — команда, которая выполняет запрос и показывает фактический план: узлы, rows, время и расхождение с оценкой планировщика. (гл. 34b)

Fail-closed — при нехватке конфигурации отказ: без пароля сервис не стартует, а не работает открытым. (гл. 43)

File descriptor — номер в таблице открытых файлов процесса; сокет — тоже дескриптор. (гл. 5)

FQDN — полностью определённое доменное имя с точкой в конце, без поиска в списке доменов. (гл. 8)

fsync — сброс данных и метаданных файла на носитель; без него запись живёт только в page cache. (гл. 5)

GIL — глобальная блокировка интерпретатора CPython: байткод исполняет один поток за раз. (гл. 20)

GitOps — деплой, при котором источник правды — git-репозиторий, а кластер непрерывно приводит себя к описанному состоянию. (гл. 41a)

goroutine — легковесный поток Go, планируемый рантаймом на пуле OS-потоков; стартует с 2 КБ стека. (гл. 32)

Graceful shutdown — прекратить приём новых запросов, доработать текущие, закрыть ресурсы, выйти по коду 0. (гл. 33)

gRPC — RPC поверх HTTP/2 с protobuf-контрактом и кодогенерацией клиентов. (гл. 13)

Head-of-line blocking — задержка одних сообщений из-за того, что очередь блокирует потерянный пакет предыдущего. (гл. 10)

Helm chart — пакет kubernetes-манифестов с шаблонами и values: один чарт — много окружений без копипасты. (гл. 41a)

HPA — автомасштабирование подов по CPU, RPS или кастомной метрике. (гл. 41)

HPACK — сжатие заголовков HTTP/2 со статической и динамической таблицами. (гл. 10)

HTTP/2 — бинарный протокол с мультиплексированием потоков в одном TCP-соединении. (гл. 10)

HTTP/3 — HTTP поверх QUIC: потоки в UDP, поэтому потеря пакета не блокирует остальные. (гл. 10)

Idempotency — свойство операции, повтор которой не меняет результат; основа безопасных ретраев. (гл. 13)

Ingress — правило маршрутизации внешнего HTTP-трафика внутрь кластера по хосту и пути. (гл. 41)

inode — структура метаданных файла: права, размер, блоки, время; имени внутри неё нет. (гл. 5)

iptables — правила фильтрации и трансляции пакетов в ядре; на них построен Service в kube-proxy. (гл. 6)

Journald — бинарный журнал systemd с индексами по юниту, приоритету и загрузке. (гл. 5b)

JWT — подписанный токен `header.payload.signature`; payload только закодирован, но не зашифрован. (гл. 13)

k6 — нагрузочный инструмент со сценариями на JS: stages, thresholds, checks — падает, если p99 выше порога. (гл. 42a)

Kafka — распределённый журнал: сообщения хранятся с диска, читаются по offset, ordering гарантирован внутри партиции. (гл. 34a)

Keep-alive — переиспользование одного TCP-соединения для нескольких HTTP-запросов. (гл. 9)

kube-proxy — компонент ноды, реализующий Service через iptables, IPVS или eBPF. (гл. 41)

kubectl — клиент API-сервера: любая команда — это HTTP-запрос к `/api` или `/apis`. (гл. 41)

kubelet — агент на ноде: запускает поды по спецификации и сообщает их статус в API. (гл. 41)

Layer — tar-архив с разницей файловой системы; кэшируется по хэшу инструкции и входных файлов. (гл. 38)

Least privilege — минимально достаточные права: только нужные вызовы API, файлы и порты. (гл. 43)

Liveness probe — проверка «процесс завис, перезапусти»; не должна зависеть от внешних сервисов. (гл. 41)

Load average — среднее число задач в очереди на CPU и в непрерываемом I/O за 1, 5 и 15 минут. (гл. 4)

Lock-файл — зафиксированное дерево зависимостей с хэшами для воспроизводимой установки. (гл. 16)

MAC-адрес — 48-битный адрес интерфейса, действующий в пределах L2-сегмента. (гл. 6)

Makefile — файл с целями и зависимостями для `make`: `make test` работает одинаково у всех и в CI. (гл. 24a)

mTLS — взаимная аутентификация: сертификат предъявляют и клиент, и сервер. (гл. 12)

MTU — максимальный размер полезной нагрузки кадра; Ethernet — 1500 байт, IPv6 требует не меньше 1280. (гл. 6)

Mutex — примитив взаимного исключения; в Go не реентерабелен, повторный захват — дедлок. (гл. 32)

Namespace (K8s) — логическое деление кластера для имён, квот и прав доступа. (гл. 41)

Namespace (Linux) — изоляция видимости: `pid`, `net`, `mnt`, `uts`, `ipc`, `user`, `cgroup`, `time`. (гл. 5c)

NAT — трансляция адресов и портов; маскарадинг исходящего трафика контейнеров — её частный случай. (гл. 6)

Node — машина кластера, на которой kubelet запускает поды. (гл. 41)

Non-root — запуск процесса не от UID 0: `USER` в Dockerfile и `runAsNonRoot` в поде. (гл. 38)

Observability — способность понять состояние системы по её метрикам, логам и трейсам, без нового кода. (гл. 42)

OOM killer — механизм ядра, убивающий процесс при нехватке памяти; выбирает по `oom_score`. (гл. 5)

OOMKilled — статус контейнера, убитого лимитом cgroup, а не глобальным OOM хоста. (гл. 41)

OpenTelemetry — стандарт генерации и экспорта метрик, логов и трейсов с единым SDK. (гл. 42)

Overlayfs — union-ФС контейнера: слои только для чтения плюс записываемый верхний слой. (гл. 38)

P99 — латентность, ниже которой укладывается 99% запросов; хвост распределения, который и определяет пользовательский опыт. (гл. 42a)

Page cache — кэш страниц файлов в свободной памяти; из-за него `free` показывает почти нулевую свободную память. (гл. 5)

Page fault — обращение к странице, которой нет в RAM; minor — из кэша, major — с диска. (гл. 5)

PDB — PodDisruptionBudget: сколько подов можно вывести одновременно при обслуживании. (гл. 41)

PID — идентификатор процесса; в PID-namespace контейнера приложение обычно становится PID 1. (гл. 4)

Pod — минимальная единица Kubernetes: контейнеры с общей сетью и томами. (гл. 41)

pprof — профилировщик Go: CPU, heap, goroutine и блокировки по `/debug/pprof`. (гл. 35)

Probe — периодическая проверка контейнера kubelet'ом: `exec`, `httpGet`, `tcpSocket`. (гл. 41)

QUIC — транспорт поверх UDP со встроенным TLS 1.3 и независимыми потоками. (гл. 10)

Race detector — инструмент Go (`-race`), находящий гонки по модели happens-before. (гл. 35)

Rate limit — ограничение числа запросов в окне; на практике — token bucket или leaky bucket. (гл. 14)

RBAC — ролевая модель Kubernetes: Role или ClusterRole плюс Binding субъекту. (гл. 41)

Readiness probe — проверка «можно слать трафик»; её провал убирает под из Endpoints. (гл. 41)

Rebase — перенос коммитов на другую базу с новыми хэшами и линейной историей. (гл. 37)

Reflog — журнал перемещений HEAD и веток; спасает коммиты после неудачного reset. (гл. 37)

ReplicaSet — контроллер, поддерживающий заданное число одинаковых подов. (гл. 41)

Reverse proxy — посредник перед сервисами: терминирует TLS, кэширует, распределяет запросы и скрывает топологию. (гл. 14)

Rollback — возврат к предыдущей версии; в Kubernetes — `kubectl rollout undo`. (гл. 41)

Rolling update — постепенная замена подов с параметрами `maxSurge` и `maxUnavailable`. (гл. 41)

RPO/RTO — сколько данных допустимо потерять и за какое время восстановиться после отказа. (гл. 42)

RSS — resident set size: страницы процесса, реально находящиеся в физической памяти. (гл. 5)

Runbook — пошаговая инструкция на случай отказа: как проверить, перезапустить, откатить и кого звать. (гл. 44)

Saturation — степень загрузки ресурса (CPU, пул соединений, диск); при насыщении латентность растёт нелинейно. (гл. 42a)

SBOM — машиночитаемый перечень компонентов образа и их версий для проверки на CVE. (гл. 43)

Selectivity — доля строк, которые отбирает условие; низкая селективность делает индекс бесполезным. (гл. 34b)

Service — стабильный виртуальный IP и DNS-имя для набора подов по селектору. (гл. 41)

SIGTERM — сигнал вежливой остановки, который можно перехватить; SIGKILL (9) перехватить нельзя. (гл. 28)

SLO — цель по уровню сервиса; SLI — измеряемый показатель, error budget — допустимая доля нарушений. (гл. 42)

SO_REUSEPORT — опция сокета: несколько процессов слушают один порт, ядро распределяет соединения. (гл. 15)

Socket — пара адрес и порт плюс протокол; в Unix — файловый дескриптор для сетевого ввода-вывода. (гл. 7)

SOPS — шифрование значений в YAML/JSON прямо в git: файл читаем, секреты зашифрованы ключом age или KMS. (гл. 43a)

Span — операция внутри трейса с длительностью, атрибутами и ссылкой на родителя. (гл. 42)

SSH tunnel — проброс порта через ssh-соединение: `-L` (локально), `-R` (удалённо), `-D` (socks) — доступ к сервису без публикации в сеть. (гл. 5e)

StatefulSet — контроллер для подов с устойчивыми именами, порядком запуска и своими дисками. (гл. 41)

Stuck — состояние, когда прогресса нет, но процесс жив: ждёт блокировку, I/O или дедлайн. (гл. 44)

sudo — запуск команды от другого пользователя по правилам `/etc/sudoers`. (гл. 5a)

Swap — вытеснение страниц памяти на диск; в контейнерах обычно отключён. (гл. 5)

syscall — переход в ядро для привилегированной операции: чтения, записи, открытия сокета. (гл. 4)

systemd — PID 1 в большинстве дистрибутивов: юниты, зависимости, таймеры, журнал. (гл. 5b)

TCP handshake — три шага SYN → SYN/ACK → ACK до передачи данных; добавляет один RTT к первому запросу. (гл. 7)

TIME_WAIT — состояние закрывшей соединение стороны: ожидание 2×MSL для поздних пакетов. (гл. 7)

TLS — шифрование и аутентификация транспорта; в TLS 1.3 рукопожатие занимает один RTT. (гл. 12)

Token bucket — алгоритм лимита: токены копятся с заданной скоростью, запрос тратит один. (гл. 14)

Trace — дерево span'ов одного запроса, прошедшего через несколько сервисов. (гл. 42)

TTL — время жизни записи DNS или счётчик хопов у пакета; уменьшается каждым шагом. (гл. 8)

UID — числовой идентификатор пользователя; ядро проверяет права по нему, а не по имени. (гл. 5a)

ulimit — лимиты ресурсов процесса (`nofile` и другие), задаются в systemd через `LimitNOFILE`. (гл. 5b)

Unit — описание сервиса, таймера, сокета или таргета в systemd. (гл. 5b)

UTF-8 — кодировка Unicode переменной длины: 1 байт для ASCII, до 4 для эмодзи. (гл. 2)

uv — менеджер пакетов и окружений Python с lock-файлом и быстрым резолвом. (гл. 16)

Vault — хранилище секретов с политиками, TTL и аудитом; умеет выдавать динамические креды БД вместо статичных. (гл. 43a)

veth — пара виртуальных интерфейсов: один в контейнере, второй в bridge хоста. (гл. 38)

WAL — журнал предзаписи: изменения пишутся в лог до страниц данных, обеспечивая атомарность. (гл. 34)

WebSocket — двунаправленный канал поверх HTTP после `Upgrade`; прокси должны его пропускать. (гл. 9)

Working tree — файлы на диске; git сравнивает их с индексом, а не с последним коммитом. (гл. 37)

Zero-downtime — обновление без обрыва запросов: несколько реплик, readiness, preStop, graceful shutdown. (гл. 41)

Zombie — завершившийся процесс, которого родитель не собрал через `wait`. (гл. 4)

Приложение D. Ответы и решения — часть 1 (главы 1–8 и Linux)

Глава 1. Что такое вычисление: автоматы, машина Тьюринга, вычислимость

1. Четыре состояния: `red` (30 с), `green` (25 с), `yellow` (3 с), `ped_green` (15 с). Кнопка состояние не меняет, а поднимает флаг `ped_wait`; переход `green → yellow → ?` смотрит на флаг: если он выставлен, дальше `ped_green`, иначе `red`. Критерий: после `ped_green` автомат возвращается в `red`, флаг сброшен, повторное нажатие фазу не продлевает.
2. Добавляются состояния `IN_STRING`, `ESCAPE`, `COMMENT`: в строке `\` переводит в `ESCAPE`, а `"` возвращает назад; `ESCAPE` съедает ровно один символ. Вне строки `#` включает комментарий до `\n`. Память ограничена текущим состоянием, счётчиков и вложенности нет — язык остаётся регулярным; вложенные комментарии потребовали бы стека.
3. Два состояния — `even` и `odd`: головка идёт вправо, на каждом `1` переключает чётность, на `0` не меняет, на пустой ячейке пишет `Y` для чётного и `N` для нечётного. Шагов ровно `n + 1` — n чтений и одна запись, то есть O(n). Проверка: «1011» даёт `N`, «1010» — `Y`; лимит шагов нужен как внешняя страховка от зацикливания.
4. Состояния `starting`, `ready`, `degraded`, `stopping`, `failed`; события — старт процесса, успех и провал пробы, `SIGTERM`, таймаут. Неявных переходов обычно два: `starting → ready` по факту открытого порта без проверки БД (отсюда «готов, но отдаёт 500») и отсутствие ребра `stopping → failed` по таймауту — оно даёт зависший при деплое контейнер рядом со старым.

Глава 2. Биты, числа и кодировки

1. `0xDEADBEEF` = `1101 1110 1010 1101 1011 1110 1110 1111`, восьмеричное `0o33653337357`, десятичное `3735928559`. Нех читается «глазами», потому что 16 — степень двойки: каждая цифра это ровно 4

бита и перевод сводится к подстановке. Для десятичной такой таблицы нет, нужна позиционная арифметика — восемь умножений на 16 и сложений.

2. Знаковые значения идут от −128 до 127, байты `0x80–0xFF` — это −128…−1. Сравнение расходится только когда операнды лежат по разные стороны границы 128: `0xFF` больше `0x01` как беззнаковое (255 > 1) и меньше как знаковое (−1 < 1). Равенство, сложение и вычитание совпадают всегда: они идут по модулю 256, битовая картина одна.

3. Печатаются знак, смещённая экспонента (8 или 11 бит) и мантисса без неявной единицы (23 или 52). `0.1` во float32 — `0x3DCCCCCD`, `1/3` — `0x3EAAAAAB`. `1e300` во float32 даёт `inf` (переполнение), в float64 — `0x7E37E43C8800759C`; `5e-324` — минимальный субнормал double `0x0000000000000001`, во float32 он равен нулю. Соседние целые float32 перестаёт различать сразу за `2^24 = 16 777 216`.

4. Декодер определяет длину последовательности по ведущему байту (`0xxxxxxx` — 1, `110xxxxx` — 2, `1110xxxx` — 3, `11110xxx` — 4), проверяет префикс `10` у продолжений и собирает код по маскам. Обрезанная последовательность даёт ошибку в позиции ведущего байта. Пара `c0 af` — перекодировка нуля: `0xC0` запрещён как ведущий (двухбайтовые начинаются с `C2`), иначе у символа было бы два представления.

5. Не-ASCII ищутся как `grep -P '[^\x00-\x7F]' app.log | head`, кодировка проверяется `file -bi` и `iconv -f utf-8 -t utf-8` (падение — файл не UTF-8). Прогон в cp1251 на кириллице часто даёт `cannot convert`. Двойное кодирование выглядит как `ÐŸÑ€Ð¸Ð²ÐµÑ`, и обратимо; неверная кодировка даёт `?????` или потерю символов — данные утрачены.

Глава 3. Процессор и память: как код превращается в такты

1. На графике «нс на элемент от размера» видны полки: до 32–48 КиБ около 0,3–0,5 нс (L1), до 1–2 МиБ около 1 нс (L2), затем 2–4 нс (L3) и 5–8 нс на последовательном чтении из DRAM. Границы сравниваются с `lscpu | grep -i cache`. Критерий:

перегибы совпадают с `lscpu` в пределах 20–30 %; ступени в сто раз нет — префетчер сглаживает DRAM.

2. Вручную: `char`, 3 паддинга, `int`, `char`, 7 паддинга, `double`, `char`, 7 паддинга — 32 байта. Проверка: `printf("%zu\n", sizeof(struct S));`. Минимум даёт порядок `double, int, char, char, char`: 8 + 4 + 3 = 15, округление до 16. Меньше нельзя: выравнивание структуры 8, размер обязан быть кратен 8, а значащих байт 14.

3. Оба потока пишут в поля одной структуры, то есть в одну 64-байтовую линию, и она по очереди переходит между ядрами — каждое `++` становится промахом когерентности. На миллионе инкрементов разница обычно 5–20 раз: десятки миллисекунд против секунды. После `alignas(64)` на каждый счётчик время почти линейно: линии не пересекаются.

4. Указатель занимает 8 байт, узел с ним — 16, узел с 32-битным индексом — 8–12: обход читает вдвое меньше линий, префетчер лучше предсказывает адрес. Выигрыш 1,3–2 раза и растёт с высотой дерева: каждый уровень — отдельный зависимый промах, при 24 уровнях это 24 латентности DRAM подряд, и лишние байты умножаются на их число.

5. Для однопоточного кода разница в пределах 5–10 %: он упирается в латентность, а не в полосу, и удалённых обращений при прогретом кэше мало. Многопоточный насыщает межсокетный канал, и пропускная способность падает почти вдвое. Смотреть `numactl --hardware`, `numastat -p PID` (доля local/remote) и сравнивать прогоны с `--cpunodebind=N --membind=N`.

Глава 4. Операционная система: процессы, потоки, планировщик

1. `ps -o pid,stat,wchan` даёт `R+` с `wchan -`, `S+` с `ep_poll` или `do_wait`, `T` с `do_signal_stop`, `Z` с `-`. В `T` процесс снят с очереди планировщика и тактов не получает, но остаётся в таблице: память, дескрипторы и открытые файлы за ним сохранены. Продолжает его только `SIGCONT`, а `SIGKILL` для остановленного откладывается до продолжения.

2. При `nice 0` и `nice 10` веса равны 1024 и 110, то есть примерно 9:1 — на двух счётчиках выходит 90 % и 10 %. Две группы с `cpu.weight=100` дают 50/50. Расхождение потому, что nice — свойство задачи внутри группы, а вес — свойство группы: процессор сначала делится между группами, потом внутри. В контейнере nice может не влиять вовсе, если задачи уже в разных группах.
3. Побайтное чтение — миллион `read()`, и каждый при отсутствии данных усыпляет поток, давая добровольное переключение. С буфером 64 КиБ вызовов около шестнадцати при том же объёме. Смотреть `grep voluntary_ctxt_switches /proc/PID/status`: разница в тысячи раз, по времени секунды против миллисекунд, потому что переключение стоит порядка микросекунды.
4. `EMFILE` наступает на 1024-м дескрипторе. В контейнере помогает `--ulimit nofile=4096:4096`, в systemd — `LimitNOFILE=65536` в `[Service]` и `daemon-reload`. `ulimit` в оболочке меняет rlimit только у новых процессов: значение копируется при `fork`/`exec`, у работающего сервиса остаётся своё. Зомби: 100 потомков в `Z`; обработчик `SIGCHLD` с `waitpid(-1, &st, WNOHANG)` в цикле собирает их.

Глава 5. Виртуальная память и файловые системы

1. В `maps` три области: `r-xp` — код, `rw-p` — `[heap]`, `rw-p` — `[stack]`. Код `r-x`, потому что его исполняют, но не меняют: страница, куда можно писать и откуда исполнять, — готовый вектор атаки, поэтому соблюдается W^X. Стек `rw-` без `x`: в него пишут локальные переменные и адреса возврата, исполнять оттуда нечего. Строка `rwxp` — почти всегда JIT.
2. Для разреженного файла 500 МБ: `ls -l` даёт 524288000, `du -h` — 0 или 4,0K, `du -h --apparent-size` — 500M. `du` считает выделенные блоки, `--apparent-size` и `ls` — поле `st_size`. У обычного файла все три совпадут. Практическое следствие: занятое место проверяют `du`/`df`, а не `ls`.
3. Без `fsync` десять тысяч строк уходят в страничный кэш и пишутся одним потоком — около 0,02 с. С `fsync` после каждой

строки это 10 000 обращений к устройству по 0,1–5 мс, от секунды до десятков секунд: разница в 100–1000 раз. `fsync` ждёт подтверждения от накопителя со сбросом его кэша, поэтому цена — латентность диска на вызов.

4. Процесс доходит примерно до 512 МиБ и получает `SIGKILL`: код 137 (`128+9`), в `memory.events` растут `max` и `oom_kill`, в `memory.peak` виден потолок. В `dmesg` строка `Memory cgroup out of memory: Killed process 2417 (python) total-vm:612345kB, anon-rss:510000kB, file-rss:0kB, oom_score_adj:0` — по `anon-rss` видно, что съедена анонимная память, а не кэш.

5. Файл удаляют по имени (`rm /var/log/app.log`), а место возвращают, обрезав открытый дескриптор: `: > /proc/2417/fd/3` или `truncate -s 0 /proc/2417/fd/3`. Дескриптор ищется через `ls -l /proc/2417/fd | grep deleted` и `lsof +L1`. Inode и блоки живут, пока открыт хотя бы один дескриптор — поэтому `df` место показывает, а `du` уже нет.

Глава 5a. Linux: пользователи, права, sudo

1. `chgrp dev /srv/shared && chmod 3775 /srv/shared`. Отвечают три бита: setgid (`2000`) — новые файлы наследуют группу каталога; sticky (`1000`) — удалять и переименовывать чужие файлы может только владелец, владелец каталога и root; групповой `w` — право править содержимое. Проверка: файл другого участника группы правится, а `rm` его же возвращает `Operation not permitted`.

2. Список даёт `find / -xdev -perm -4000 -type f`. `/usr/bin/passwd` пишет в `/etc/shadow`, недоступный на чтение обычному пользователю; без бита смена своего пароля станет невозможна. `/usr/bin/sudo` читает `sudoers` и меняет uid — без setuid перестанут работать все делегированные права. `/usr/bin/mount` монтирует по записи в `fstab`; без бита пользователь не смонтирует разрешённый носитель.

3. `setfacl -m u:deploy:--x /srv/app` даёт только проход по каталогу (без `r` листинга нет), затем `setfacl -m u:deploy:r /srv/app/config.yml`. Строка `mask::r-x` — верхняя граница прав для именованных записей и группы-владельца: эффективные права равны пересечению записи с маской, поэтому при ручной правке

выходит «право есть, а доступа нет». `setfacl` пересчитывает маску сам.

4. Первый способ: `setcap cap_net_bind_service=+ep /usr/sbin/nginx`, проверка `getcap`. Второй: `sysctl -w net.ipv4.ip_unprivileged_port_start=80`, для стойкости — файл в `sysctl.d`. Обновление пакета заменяет бинарник и стирает capability — сервис не стартует до повторного `setcap`; при переносе на другой хост право тоже теряется. Sysctl переезжает с хостом, но открывает младшие порты всем процессам.

5. В `/etc/sudoers.d/ci` (режим `0440`, владелец `root:root`): `ci ALL=(root) NOPASSWD: /usr/bin/systemctl restart app.service`. Ни `ALL`, ни масок, ни оболочки, аргумент фиксирован, поэтому `sudo systemctl restart ssh` и `systemctl edit` отклоняются. Root через запись не получить, пока сам юнит и `ExecStart` принадлежат root: иначе `ci` правит `ExecStart=` и запускает свой код от root. Проверка — `sudo -l -U ci`.

Глава 5b. Linux: пакеты, systemd, журналы

1. `dpkg -S /usr/bin/ss` отвечает `iproute2: /usr/bin/ss`, полный список файлов даёт `dpkg -L iproute2`. В `dpkg -l` две первые буквы — желаемое и текущее состояние: `ii` значит «должен быть установлен» и «установлен»; `rc` — удалён, конфиги остались; `iU` — распакован, но не настроен. Файл, не принадлежащий пакету, даёт `no path found` — он собран вручную.

2. `apt-mark hold nginx`, затем `apt update` и `apt-cache policy nginx`: `Installed` и `Candidate` совпадают, `apt list --upgradable` имени не содержит. Это и есть доказательство — индекс обновился, кандидат прежний. После `unhold` следующий `upgrade` может прыгнуть через несколько версий сразу: конфиги потребуют ответов, зависимости потянут новые библиотеки, и всё в один момент, вместе с накопленными CVE.

3. Юнит: `Type=notify`, `ExecStart=/opt/app/venv/bin/python /opt/app/main.py`, `User=app`, `LimitNOFILE=65536`, `NoNewPrivileges=yes`, `PrivateTmp=yes`, `ProtectSystem=strict`, `ReadWritePaths=/var/lib/app`, `Restart=on-failure`. `systemd-analyze verify app.service` должен дать код 0 и пустой вывод. `Type=notify` требует вызова `sd_notify(READY=1)` из

приложения, иначе старт повиснет до `TimeoutStartSec`.

4. `health.service` (`Type=oneshot`, `ExecStart=/usr/local/bin/healthcheck.sh`) и `health.timer` с `OnCalendar=*:0/5`, `Persistent=true`, `Unit=health.service`, `WantedBy=timers.target`. `Persistent=true` хранит отметку в `/var/lib/systemd/timers`, поэтому пропущенный за время простоя запуск выполнится один раз после загрузки. Проверка — `systemctl list-timers health.timer` и журнал юнита.

5. В `/etc/systemd/journald.conf` — `Storage=persistent` и `SystemMaxUse=500M`, затем `mkdir -p /var/log/journal` и `systemctl restart systemd-journald`; размер показывает `journalctl --disk-usage`, чистят `journalctl --vacuum-size=500M`. Ошибки прошлой загрузки одной командой: `journalctl -b -1 -p err`. Без каталога `/var/log/journal` журнал живёт в `/run` и после перезагрузки исчезает.

Глава 5c. Linux: namespaces, cgroups, диагностика

1. `unshare --fork --pid --mount-proc --uts --net bash`, внутри `hostname c1` и `ps -ef`: две строки — сам `bash` с PID 1 и `ps`. Снаружи `ps -ef` показывает процесс с настоящим PID хоста и весь остальной список. Сеть пуста: только `lo` в состоянии DOWN, адресов нет. Критерий: три проверки — `hostname`, число строк `ps`, `ip addr` — ведут себя одинаково и воспроизводимо.

2. `memory.max` — потолок в байтах (`536870912`) или `max`; `cpu.max` — «квота период», например `50000 100000` = 0,5 CPU; `pids.max` — максимум задач; `memory.events` — счётчики `low`, `high`, `max`, `oom`, `oom_kill`. Текущее потребление — `memory.current`, пик — `memory.peak`. Рост `max` значит упор в потолок, рост `oom_kill` — процессы уже убивают.

3. Процесс с лимитом 64 МиБ, выделяющий 256 МиБ, убивается на подходе к 64 МиБ: код 137, `max` в `memory.events` инкрементируется многократно, `oom_kill` — на единицу, в `memory.events.local` те же счётчики без потомков. В `dmesg` остаётся `Memory cgroup out of memory: Killed process ... anon-rss:...`. От утечки отличается тем, что `memory.current` упирался в потолок.

4. `free -m` читает `/proc/meminfo`, а тот собирается из глобальных счётчиков ядра: в контейнере с лимитом 512 МиБ видно 15,5

ГиБ хоста. `memory.current` берётся из cgroup и включает page cache, поэтому он больше RSS приложения, но отражает реальное потребление. Смотреть надо пару `memory.max`/`memory.current`; `free` внутри контейнера для планирования непригоден.

5. `SIGTERM` перехватывается: приложение закрывает соединения, дописывает буферы и выходит с кодом 143 (`128+15`). `SIGKILL` не перехватывается и не откладывается, код 137. `terminationGracePeriodSeconds` (30 в Kubernetes, 10 в `docker stop`) — срок между ними; если обработчик не успел, процесс убивают и данные теряются. Долгие задачи выносят в `preStop`-хук или увеличивают grace.

Глава 6. Сети: от кабеля до IP-пакета

1. `arping -I eth0 -c 3 192.168.0.250` (или `tcpdump -ni eth0 arp`) даёт `who-has 192.168.0.250 tell 192.168.0.75` с адресом назначения `ff:ff:ff:ff:ff:ff`. Кадр широковещательный, потому что MAC получателя неизвестен, а ARP работает внутри сегмента: узнать адрес можно только спросив всех. Ответ приходит юникастом, и дальше запрос идёт на конкретный MAC.

2. `/22` — маска `255.255.252.0`, шаг сетей по третьему октету 4: 130 попадает в блок 128, значит сеть `10.8.128.0`, broadcast `10.8.131.255`, хостов `1024 − 2 = 1022`, диапазон `10.8.128.1–10.8.131.254`. `ipaddress.ip_network("10.8.130.200/22", strict=False)` даёт те же значения. Вручную достаточно взять 130 = `10000010` и обнулить два младших бита.

3. `ping -M do -s 1472 host` проходит — MTU пути 1500. При `Frag needed and DF set` размер уменьшают: 1452 — IPsec, 1420 — WireGuard, 1412 — OpenVPN, 1392 — VXLAN, 1492 — PPPoE. Интерфейс сравнивают по `ip link show eth0 | grep mtu`. Расхождение означает инкапсуляцию: локально 1500, а путь меньше, и большие пакеты молча теряются при фильтрации ICMP.

4. Заголовок TCP — 20 байт: sport (2), dport (2), seq (4), ack (4), data offset и флаги (2), window (2), checksum (2), urgent (2). В дампе после IP: `1f 90 01 bb` — порты 8080 и 443, два поля по четыре нуля — seq и ack в SYN, `50 02` — смещение 5 слов и флаг

`S`, `ff ff` — окно, далее опции `02 04 05 b4` (MSS 1460), `03 03 07` (wscale 7).

5. Типовая схема: роутер `192.168.0.1/24` (DHCP, DNS, NAT), хост `192.168.0.75`, шлюз по умолчанию — роутер, за ним адрес провайдера. Проверки: `ip -br addr`, `ip route` (строка `default via`), `curl ifconfig.me`, правила DNAT в роутере. Если внешний адрес из `100.64.0.0/10`, проброс портов невозможен из-за CGNAT.

Глава 7. TCP и UDP: надёжность, порты, сокеты

1. `sudo tcpdump -ni any -c 20 'tcp port 443 and tcp[tcpflags] & tcp-syn != 0'`: в первом сегменте видны `seq` (ISN), `win`, опции `mss 1460` и `wscale 7`. ISN случаен и у каждого соединения свой — Linux считает его по RFC 6528 из адресов, портов и таймера, поэтому разница с предыдущим обычно в сотни миллионов. Это защита от подмены и коллизий, а не счётчик.

2. Сервер принимает соединение и не вызывает `recv()`: приёмный буфер заполняется, и в ACK он объявляет нулевое окно. На сервере `ss -tn` показывает `Recv-Q` равным размеру буфера (обычно 131072), на клиенте `ss -tin` — `Send-Q` с непереданными байтами и таймер `persist`. Клиент блокируется, отправив объём буферов с обеих сторон — единицы мегабайт, а не мегабайт «в трубу».

3. Сервер получает FIN и не вызывает `close()`: соединение уходит в `CLOSE-WAIT`, и ядро его не закрывает — таймаута для этого состояния нет. `ss -tan state close-wait` показывает рост, дескрипторы не освобождаются, и через часы сервис упирается в `EMFILE`. Лечится только `close()` в приложении; `tcp_fin_timeout` относится к `FIN-WAIT-2` и здесь не помогает.

4. Эфемерных портов 28 232 (`32768–60999`), порт занят 60 с в `TIME_WAIT`, значит к одному адресу выходит около 470 новых соединений в секунду. Для 30 000/с нужно 1,8 млн портов — в 16-битном поле их нет. Лечение: пул с keep-alive (одно соединение обслуживает сотни запросов), затем расширение `ip_local_port_range`, и только потом `tcp_tw_reuse=1`.

5. UDP-эхо на 50 строк: `socket(AF_INET, SOCK_DGRAM)`, `recvfrom`, `sendto` обратно. При `tc qdisc add dev eth0 root netem loss 10%` теряется примерно каждый десятый ответ, и клиент видит пропуски — повторов нет по построению. TCP на том же канале не теряет данных, но платит повторами: падает пропускная способность и растёт p99 из-за RTO от 200 мс, тогда как задержка UDP ровная.

Глава 8. DNS: как имя становится адресом

1. `dig +short app.example.com` спрашивает DNS напрямую, `getent hosts app.example.com` идёт через NSS и сначала читает `/etc/hosts`, `nslookup` использует тот же резолвер, но в своём формате. Расходятся они, когда в `/etc/hosts` есть строка: `dig` покажет адрес из зоны, `getent` — из файла. Отсюда правило: расхождение `dig` и `getent` — вина hosts или `nsswitch.conf`, а не DNS.
2. TTL понижают заранее и ждут старый TTL: правка TTL чужие кэши не чистит, в них лежит прежнее значение. После смены A-записи локальный резолвер увидит её через 0–60 с, публичные (`1.1.1.1`, `8.8.8.8`) — в пределах нового TTL, корпоративный — до 60 с плюс свой минимум. Мерить: `dig +noall +answer` (колонка TTL убывает) и опрос нескольких резолверов раз в 10 с.
3. Split-horizon на dnsmasq: `address=/app.example.com/10.8.0.10` в локальном конфиге и `local=/example.com/`, чтобы зона не уходила в upstream; на BIND — два `view` с выбором по `match-clients`. Внутренний клиент получает `10.8.0.10`, внешний — публичный адрес. Проверка: `dig @127.0.0.53 app.example.com` против `dig @1.1.1.1 app.example.com`.
4. `ndots:5` означает: имя, в котором меньше пяти точек, сначала пробуется с каждым доменом из `search`. При двух доменах поиска и десяти коротких именах выходит 10 × 3 = 30 запросов вместо 10 — по два `NXDOMAIN` на имя. В дампе видно пачки `api.corp.local`, `api.svc.cluster.local`, затем `api.`. Лечение — `ndots:2` или FQDN с точкой на конце.
5. `dig +trace app.example.com` проходит цепочку: корневые серверы → серверы зоны `com` → делегирование на NS провайдера.

Неожиданность видна там, где `NS` в ответе родительской зоны не совпадает с ожидаемым: регистратор отдаёт `ns1.parking.example` вместо ваших `ns1.yourhost`. Дальше `dig NS example.com @<сервер родителя>` и `dig +norecurse` к найденному серверу показывают, где теряется актуальная зона.

Глава 1а. Алгоритмика: как оценивать алгоритмы и не изобретать велосипед

1. Бинарный поиск: 10^6 запросов × 20 сравнений = $2 \cdot 10^7$ операций, но каждый поиск — по списку, то есть промахи кэша на каждом шаге. `set`: 10^6 вставок при построении плюс 10^6 проверок, каждая O(1) в среднем. По операциям выигрывает `set` примерно в десять раз, по константе — сильнее; цена — память: множество из миллиона int в CPython занимает порядка 40–60 МБ против ~36 МБ у списка (указатели плюс объекты). Если запросов мало, а память в дефиците — бинарный поиск выгоднее.
2. Проход по файлу построчно с `Counter` по IP и `heapq.nlargest(20, counter.items(), key=lambda kv: kv[1])`: один проход O(n) по строкам и O(u log 20) на выборку, где u — число уникальных IP. `sorted(Counter.items())` сортирует по ключу (строке IP), а не по частоте — это не тот ответ; полная сортировка по значению даёт O(u log u) и требует держать все уникальные ключи, а их на 2 ГБ лога могут быть миллионы. Если уникальных слишком много, точный счётчик заменяют на Count-Min Sketch: памяти фиксировано, результат приблизительный с гарантированной ошибкой.
3. Три цвета в DFS: белый — не посещён, серый — в текущем стеке, чёрный — обработан. Ребро в серый узел и есть цикл, а восстановить его позволяет стек пути.

python

```
def find_cycle(graph: dict[str, list[str]]) -> list[str] | None:
    color, stack, pos = {}, [], {}
    def dfs(u: str) -> list[str] | None:
        color[u] = "gray"; pos[u] = len(stack); stack.append(u)
        for v in graph.get(u, []):
            if color.get(v) == "gray":
                return stack[pos[v]:] + [v]          # нашли петлю
            if color.get(v) is None:
```

```
                got = dfs(v)
                if got:
                    return got
        color[u] = "black"; stack.pop(); pos.pop()
        return None
    for node in graph:
        if color.get(node) is None:
            got = dfs(node)
            if got:
                return got
    return None
```

Сложность O(V + E): каждый узел и каждое ребро посещаются один раз.

4. `OrderedDict` даёт O(1) на чтение и вставку: `move_to_end(key)` при обращении, `popitem(last=False)` при переполнении. Связный список нужен затем, что удаление из середины по известному узлу — O(1); в `OrderedDict` эту роль играет внутренний двусвязный список, поэтому перебор ключей в порядке вставки бесплатен. Ключевое условие O(1): доступ к узлу по хешу, а не поиск по списку.

python

```
from collections import OrderedDict

class LRU:
    def __init__(self, cap: int):
        self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d:
            return None
        self.d.move_to_end(k)
        return self.d[k]
    def put(self, k, v):
        self.d[k] = v
        self.d.move_to_end(k)
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

5. Типовой результат: вложенный цикл `for a in A: for b in B: if a in seen_list` — это $O(n^2)$ из-за `in` по списку; замена на `set` даёт O(n) и ускорение в десятки-сотни раз на массивах от 10^4. Если же перебор пар действительно нужен (например, поиск пересечений), сложность остаётся O(n·m), и ускорять надо не константы, а сам алгоритм — сортировкой и двумя указателями или хеш-соединением.

Приложение D. Ответы и решения — часть 2 (главы 9–20)

Номера ответов совпадают с номерами упражнений в главах. Код исполним целиком, если не помечено «фрагмент».

Глава 9. HTTP/1.1: запросы, ответы, заголовки

1. `printf 'GET / HTTP/1.1\r\n\r\n' | nc example.com 80` → `400 Bad Request`; два разных `Host` → тоже `400` (часть серверов отдаёт `421`). `Host` обязателен и единственен: по нему выбирается виртуальный хост, а расхождение значений между фронтендом и бэкендом открывает отравление кэша.
2. Тело `1a\r\nThis is the first chunk.\r\n15\r\nAnd the second one.\r\n0\r\n\r\n` делится на блоки: `0x1a` = 26 байт (текст плюс CRLF), `0x15` = 21 байт, затем нулевой блок. Склейка даёт `This is the first chunk.And the second one.` — ровно это печатает `curl -s`, а `curl -s --raw` показывает блоки с размерами.
3. `return 302` отдаёт `Location`, но `curl -L -X POST -d 'a=1'` повторит запрос как `GET` без тела — операция молча не выполнится. `return 308` сохраняет метод: curl повторяет `POST` с `Content-Length: 3` и телом `a=1`. Поэтому перевод http→https и POST-редиректы делают через 308.
4. С переиспользованием один `curl` с сотней URL делает 1 TCP-рукопожатие (+1 TLS) и 100 запросов по одному соединению. Без него (`curl` в цикле, `Connection: close`) — 100 рукопожатий, 100 сокетов в `TIME_WAIT` на 60 с и ≈4 с лишней задержки при RTT 40 мс.
5. Для формата из главы (`$7` — статус, `$5` — путь, `urt=` — десятое поле), проверено:

```
awk '{split($0,u,/urt=/); t=u[2]+0; c[$7]++;
      if ($7>=500) {e[$5]++; s[$5]+=t; n[$5]++}}
     END {for (k in c) print "code", k, c[k];
          for (p in e) printf "5xx %s %d %.3f\n", p, e[p], s[p]/n[p]}'
        access.log
```

Вывод: `code 502 2, code 200 1, code 504 1, 5xx /v1/orders 2 0.007`. 502 с `urt=0.007` — бэкенд отказал мгновенно; 502 с `urt` около `proxy_read_timeout` — обрыв посреди ответа; 504 — таймаут чтения.

Глава 10. HTTP/2 и HTTP/3: мультиплексирование и QUIC

1. `curl -so /dev/null -w '%{http_version} %{time_starttransfer}\n' --http2 --limit-rate 100k URL` и то же с `--http1.1`. На одиночном запросе разница 5–20 мс, и h2 обычно медленнее: добавляются `SETTINGS`, `WINDOW_UPDATE`, фрейминг. Выигрыш h2 виден на десятках параллельных ресурсов.
2. С `http2 on;` вывод `openssl s_client -alpn h2` содержит `ALPN protocol: h2`, а `curl -w '%{http_version}'` даёт `2`. Без директивы — `ALPN protocol: http/1.1` или `No ALPN negotiated`, версия `1.1`. Директива действует только в `server` с `listen ... ssl`; после правки — `nginx -t` и `reload`.
3. `iptables -I OUTPUT -p udp --dport 443 -j DROP`, затем `curl --http3 -w '%{http_version} %{time_connect}\n' URL`. Не дождавшись ответа на QUIC-пакеты, curl ждёт транспортный таймаут (обычно 2–5 с) и откатывается на TCP: в выводе `2` или `1.1`. `--http3-only` завершится ошибкой без отката.
4. При 2 % потерь h1.1 с шестью соединениями теряет 1/6 пропускной способности на каждую потерю, h2 — все потоки сразу, они в одном TCP. На RTT 50–100 мс h2 медленнее h1.1 на 20–40 % по полной загрузке страницы из 50 ресурсов. Лечится h3/QUIC, где потоки независимы.

Глава 11. Куки, сессии, кэширование, CDN

1. `curl -i` показывает `Set-Cookie: session_id=...; Path=/; HttpOnly; Secure; SameSite=Lax; Max-Age=604800`. От XSS защищает `HttpOnly` (значение не читается из JavaScript), от CSRF — `SameSite`: `Strict` и `Lax` не отправляют куку при кросс-сайтовом POST. `Secure`, `Path`, `Domain`, `Max-Age` задают область и срок.
2. Без `Domain` кука host-only: `curl -b 'sid=1' https://api.example.com/` её не отправит, сервер вернёт анонимный ответ. С `Domain=example.com` она уходит на любой поддомен. Плата — куку видит любой поддомен, поэтому для критичных используют префикс `__Host-`,

требующий отсутствия `Domain` и `Path=/`.

3. Фрагмент:

```
proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=api:10m inactive=60m;
location /api/public/ {
    proxy_pass http://backend; proxy_cache api; proxy_cache_valid 200 10s;
    add_header X-Cache-Status $upstream_cache_status always;
}
```

Первый запрос — `MISS`, второй в течение 10 с — `HIT`, после `sleep 11` — `EXPIRED`: nginx перепроверяет объект у бэкенда. Без `always` статус не виден на 304.

4. У `example.com` HTML приходит с `cache-control: max-age=86000` и сильным `etag`; `If-None-Match` даёт `304` без тела — документ кэшируется, но сутки не обновляется. Разумнее `max-age=60, stale-while-revalidate=600` для HTML и годовой `max-age` с хешем в имени для статики.

5. При `SameSite=Strict` переход по внешней ссылке кросс-сайтовый, поэтому куки нет и пользователь видит форму входа, уже имея сессию. `Lax` отправляет куку при навигации GET, но не при кросс-сайтовом POST. Дополнительно в форму кладут CSRF-токен и проверяют `Origin` на небезопасных методах.

Глава 12. TLS и сертификаты

1. Сравниваем четыре поля: версию и шифр (`New, TLSv1.3, Cipher is ...`), число звеньев в `Certificate chain`, строку `ALPN protocol`. Без флага `-alpn h2` в выводе будет `No ALPN negotiated` у всех трёх сайтов — это незапрошенный ALPN, а не отсутствие h2; с флагом у `cloudflare.com` и `google.com` появляется `h2`. Одно звено в цепочке — промежуточного нет.

2. `location ^~ /.well-known/acme-challenge/ { return 404; }` ломает HTTP-01: `certbot renew --dry-run` печатает `Invalid response from http://example.com/.well-known/acme-challenge/...: 404` и `Type: unauthorized`, код 1. После удаления location и `reload` тот же прогон даёт `all simulated renewals succeeded`.

3. Фрагмент:

```
end=$(openssl s_client -connect "$HOST:443" -servername "$HOST" </dev/null
    2>/dev/null \
      | openssl x509 -noout -enddate | cut -d= -f2)
days=$(( ( $(date -d "$end" +%s) - $(date +%s) ) / 86400 ))
```

```
echo "tls_cert_days_left{host=\"$HOST\"} $days" >
    /var/lib/node_exporter/tls.prom
[ "$days" -lt 21 ] && exit 1
```

Алерт `tls_cert_days_left < 21` с `for: 1h`; при `notAfter` через 20 дней скрипт возвращает 1.

4. Браузер получает `Strict-Transport-Security: max-age=60` и минуту переписывает `http://` на `https://` до отправки запроса — в логах nginx запросов на порт 80 нет. Откат: отдать `max-age=0` по HTTPS и дождаться истечения прежнего срока; в Chrome ещё `chrome://net-internals/#hsts`.

Глава 13. REST и проектирование API

1. У GitHub: `POST /repos/{o}/{r}/merges` (слияние), `POST /markdown` (операция без ресурса), `PUT /user/starred/{o}/{r}`. REST-альтернативы: `POST /v1/merge-requests` и `POST /v1/renders` с телом. При переходе теряется очевидность «одна ручка — одно действие» и идемпотентность, которую даёт семантика `PUT`.

2. Перенос: `PATCH /v1/tasks/{id}`, тело `{"project_id": 9}`, успех `200`, идемпотентен — повтор даёт то же состояние. Слияние: `POST /v1/tasks/{id}/merge`, тело `{"source_ids": [7, 8]}`, успех `200` и `{"merged": 2}`, неидемпотентно — обязателен `Idempotency-Key`, повтор возвращает сохранённый ответ.

3. Схема и алгоритм:

```
CREATE TABLE idem (key text PRIMARY KEY, body_hash text NOT NULL,
  status smallint, response jsonb);
INSERT INTO idem(key, body_hash) VALUES ($1,$2) ON CONFLICT (key) DO NOTHING;
```

Ноль вставленных строк → читаем запись: хеш другой → `422`, `status IS NULL` → `409`, иначе отдаём сохранённые код и тело. Уникальный индекс — единственная защита от гонки двух параллельных запросов.

4. `OFFSET 100000` даёт `Limit -> Index Scan ... rows=100050` и 40–80 мс: база читает и выбрасывает 100 тысяч строк. Cursor `WHERE (created_at, id) < ($1,$2) ORDER BY created_at DESC, id DESC LIMIT 50` — `rows=50`, 0,1–0,3 мс, стоимость не зависит от глубины. Плата — нет перехода на страницу N.

5. В OpenAPI 3.1 описываем путь, схему и `components.responses.Problem` с `application/problem+json`. Клиент генерируем `openapi-python-client`: он разбирает `type/title/status/detail` в исключение и ретраит только `429, 502, 503, 504`, не трогая `400/422`.

Глава 14. Прокси, балансировщики, reverse proxy

1. Цепочка и значения: клиент 30 с → LB idle 60 с → nginx `proxy_connect_timeout 2s`, `proxy_send_timeout 10s`, `proxy_read_timeout 30s` → приложение 10 с → БД `statement_timeout 5s`. Инверсия — когда клиент ждёт 5 с, а приложение считает 30: клиент уходит, nginx пишет 499, работа жжёт пул.
2. При `max_fails=2 fail_timeout=10s` упадут два запроса, дальше узел выведен на 10 с. С `proxy_next_upstream error timeout` пользователи этого не заметят, но два лишних запроса уйдут на живой бэкенд; возврат в пул — пробным запросом после `fail_timeout`.
3. Round robin при смеси 10 мс и 2 с копит длинные запросы на случайных узлах, p99 тянется к худшему узлу. `least_conn` учитывает активные соединения и выравнивает очередь — p99 падает в 1,5–3 раза. На однородных запросах разницы нет, и round robin дешевле.
4. Рабочий конфиг: `set_real_ip_from 10.0.0.0/8; real_ip_header X-Forwarded-For; real_ip_recursive on;`. Запрос с `-H 'X-Forwarded-For: 1.2.3.4'` с внешнего адреса не входит в доверенную сеть, заголовок игнорируется, `$remote_addr` — реальный IP. С `set_real_ip_from 0.0.0.0/0` подстановка проходит.
5. Шаги: включить запись сессий в Redis параллельно с памятью (dual-write), читать сначала из Redis с фолбэком, дождаться нуля промахов, отключить sticky в балансировщике, убрать локальное хранилище. Точки отката — после dual-write и после отключения sticky; TTL сессий перекрывает окно миграции.

Глава 15. nginx и event loop

1. Замер на 1000 простаивающих сокетах: потоки дают +10 МиБ RSS и +8 ГиБ виртуальной памяти (стек 8 МиБ на поток),

asyncio/selectors — рост в пределах сотен КиБ. На 10 000 соединений потоки упираются в лимиты и переключения контекста, а цикл событий добавляет сотни байт на сокет.

2. `worker_connections 8192` при `worker_rlimit_nofile 1024`: в `error.log` `Too many open files`, клиенты получают 500. Починка без правки `nginx.conf` — drop-in юнита: `systemctl edit nginx` с `[Service]` и `LimitNOFILE=65535`, затем `daemon-reload` и `restart`. Проверка — `/proc/<worker>/limits`.

3. При `gzip on` директива `sendfile` неприменима: сжатие требует чтения файла в пользовательское пространство, ядро не может копировать страницы напрямую в сокет. `ab -k -c 100 -n 20000` даёт практически одинаковый RPS для `sendfile on` и `off`, а на несжимаемых типах с `gzip off` разница кратная.

4. Edge-режим (`EPOLLET`) сообщает о готовности один раз; если обработчик прочитал 100 байт из 4096 и не дочитал до `EAGAIN`, второго события не будет и остаток зависнет. Level-режим вернёт дескриптор снова. Вывод: в edge-режиме читать циклом до `EAGAIN`, иначе теряются данные.

5. На 8 ядрах `worker_processes 4` дают максимальный RPS и минимальный p99; `32` — RPS ниже на 10–20 %, p99 выше в разы. `vmstat 1` показывает рост `cs` и `sy`, `pidstat -w` — непроизвольные переключения: воркеры борются за кэш вместо полезной работы.

Глава 16. Python: окружение

1. После `python3.14 -m venv .venv && .venv/bin/pip install httpx` команда `.venv/bin/python -c 'import sys; print(sys.prefix); print(sys.path[-1])'` даёт `.venv` и `.venv/lib/python3.14/site-packages`; у системного интерпретатора `sys.prefix` — `/usr`, и `httpx` там не импортируется.

2. В `requirements.txt` транзитивные зависимости попадают снимком (`pip freeze`), включая случайные версии dev-инструментов. В `pyproject.toml` — только прямые диапазоны, полный граф с пинами лежит в `uv.lock`: пакетов обычно на 20–40 % меньше, потому что разрешатель выбирает одну версию на граф.

3. Цикл даёт `FOR_ITER`, `STORE_FAST`, `CALL` для `append`, `JUMP_BACKWARD` — генератора нет. В варианте с генераторным выражением видно `LOAD_CONST (<code object <genexpr>>)`, затем `MAKE_FUNCTION`: объект создаётся вызовом этого кода, его тело начинается с `RETURN_GENERATOR` и содержит `YIELD_VALUE`.
4. `python -m build && unzip -l dist/tasks_api-0.3.0-py3-none-any.whl` показывает пакет, `METADATA`, `entry_points.txt`. Типичная находка — нет шаблонов и `py.typed`: запуск из корня находил их через текущую директорию в `sys.path`, а после установки в чистый venv — `FileNotFoundError`.
5. Фрагмент workflow:

```
strategy: {matrix: {python: ["3.13", "3.14"]}, fail-fast: false}
steps:
  - run: uv lock --check
  - run: uv sync --frozen
  - run: uv run pytest -q
```

`uv lock --check` падает при расхождении `pyproject.toml` и `uv.lock`, `--frozen` запрещает обновлять lock при установке, матрица прогоняет обе ветки.

Глава 17. Синтаксис и типы: коллекции, срезы, comprehension

1. `set` даёт уникальные сервисы, чтобы comprehension не проходил по дублям:

```
metrics = [{"service": "api", "p95": 120}, {"service": "api", "p95": 300},
           {"service": "web", "p95": 90}]
out = {s: max(m["p95"] for m in metrics if m["service"] == s)
       for s in {m["service"] for m in metrics}}
print(out)
```

Вывод `{'api': 300, 'web': 90}`: `max` по фильтру даёт максимум за один проход, без сортировки.

2. Баг: `acc=[]` вычисляется при `def` и живёт в `func.__defaults__`, поэтому вызовы делят список. Исправление без смены сигнатуры:

```
def batch(items, size=10, acc=None):
    acc = [] if acc is None else acc
    acc.append(items)
    return acc
print(batch("a"), batch("b"))
```

Вывод `['a'] ['b']`; клиенты, передающие `acc` явно, работают как раньше.

3. Аннотации:

```
from typing import Literal
def render(cfg: dict[str, str | int], mode: Literal["fast", "safe"]) ->
    tuple[str, int]:
    return str(cfg), 0
```

`mypy --strict` на вызове `render({}, mode="turbo")` печатает `Argument "mode" to "render" has incompatible type "Literal['turbo']"; expected "Literal['fast', 'safe']"` — ошибка ловится до запуска.

4. Замер на 100 000 экземпляров в `set`: с `__slots__` — 13,0 МиБ, без — 16,9 МиБ (≈41 байт на объект). `__eq__` и `__hash__` по паре полей обязательны вместе: определив только `__eq__`, Python обнулит `__hash__`, и объект перестанет попадать в `set`.

5. Проверка `n-1 in coll`, 1000 повторов: 10 элементов — 0,11 мкс список против 0,05 мкс множество; 1 000 — 6,4 мкс против 0,075 мкс (×86); 1 000 000 — 7,2 мс против 0,074 мкс (×97 000). Список перестаёт быть приемлемым на сотнях элементов в цикле: `set` даёт O(1) вместо O(n).

Глава 18. Функции, замыкания, декораторы

1. `def tag(name, /, *, prefix="app", **attrs)` собирает `prefix:name` и атрибуты; `tag(name="x")` падает с `TypeError: tag() missing 1 required positional argument: 'name'`. Сообщение точное: слэш запрещает передачу по имени, именованный `name` не заполняет позиционный параметр, а `**attrs` его не перехватывает.

2. Фабрика лимитера:

```
import time
def make_limiter(rate):
    interval, state = 1.0 / rate, {"next": 0.0}
    def allow():
        now = time.monotonic()
        if now >= state["next"]:
            state["next"] = max(now, state["next"]) + interval
            return True
        return False
    return allow
```

Состояние живёт в замыкании, `monotonic` не зависит от перевода часов: `[lim() for _ in range(4)]` → `[True, False, False, False]`.

3. В `retry` добавляем `jitter` и `deadline`: задержка `min(base * factor ** (n-1), max_delay) + random.uniform(0, jitter)`, а перед сном сверяем `time.monotonic() < deadline`; если бюджет исчерпан — `raise last_exc`. При `deadline=0.2` и всегда падающей функции выход за ~0,2 с, а не за все попытки.

4. `@lru_cache` на `load_config(path)` держит результат вечно: правка файла не видна до перезапуска — это и есть устаревшие данные. Инвалидация по `mtime`: ключом сделать `(path, os.stat(path).st_mtime_ns)`, тогда изменение файла создаёт новую запись, а старые вытесняются по `maxsize`.

5. Конвейер `read_lines` → `only_errors` (фильтр по уровню) → `count_by_status` (агрегация в словарь). На файле 20 000 строк пик по `tracemalloc` — 22,8 КиБ, вывод `{'200': 6667, '500': 13333}`. Память постоянна: в ней одна строка и аккумулятор, а не весь файл.

Глава 19. ООП, dataclasses, протоколы, MRO

1. `@dataclass(frozen=True, slots=True)` с полями `kind`, `name`, `namespace="default"` и свойством `fqn` возвращает `prod/deployment/api`. `frozen=True` генерирует `__hash__`, поэтому `r in {r}` истинно и хеши равны у одинаковых значений; `slots=True` убирает `__dict__`, и опечатка в имени поля падает сразу.

2. Порядок задаётся объявлением базовых классов снизу вверх: `Logs(Metrics)`, `Metrics(Retry)`, `Retry(Timeout)`, `Timeout(Base)`. `Logs.__mro__` = `['Logs', 'Metrics', 'Retry', 'Timeout', 'Base', 'object']`, `handle()` → `['logs', 'metrics', 'retry', 'timeout', 'call']`. Обратный порядок ломает цепочку: `super()` уходит в следующий класс MRO, и слои пропускаются.

3. `Backoff(base, factor, max_delay)` валидирует в сеттерах-свойствах, поэтому проверка срабатывает уже в `__init__`: `Backoff(0.5, 0.5, 3.0)` падает с `ValueError: factor must be >= 1, got 0.5` при создании. Свойство `delays` для `Backoff(0.5, 2.0, 3.0, attempts=5)` даёт `(0.5, 1.0, 2.0, 3.0, 3.0)`.

4. `class Notifier(Protocol): def send(self, text: str) -> bool: ...`; реализации `LogNotifier` и `HttpNotifier` от протокола не наследуются. Функция `def notify(n: Notifier, items: list[str]) ->`

`int` проходит `mypy --strict` с обеими, а класс без `send` даёт `Argument 1 has incompatible type`. `isinstance` проверяет лишь имена методов.

5. Класс на 150 строк с тремя методами без `self` распадается на `@dataclass(frozen=True)` с данными и три свободные функции; фикстур становится меньше, потому что состояние создаётся конструктором (`Config(env="prod")`) вместо `setup_class` и `mocker.patch` атрибутов. Обычно число фикстур падает с 6–8 до 1–2 на модуль.

Глава 20. Асинхронность: asyncio

1. `fetch_all` оборачивает каждый запрос в `asyncio.timeout(2)` и собирает `(url, ok, elapsed)` через `gather(..., return_exceptions=True)`, чтобы падение одного не рушило остальные. На 20 URL с задержкой 0,2 с последовательный вариант занимает ≈4 с, конкурентный — ≈0,25 с: время равно самому медленному запросу, а не сумме.

2. `asyncio.timeout(5)` вокруг трёх `await` по 2 с: третий отменяется, `TimeoutError` перехватывается, функция возвращает два накопленных результата и `complete=False`. Частичный результат собирают в список по мере готовности, а не после `gather`: при отмене результат `gather` теряется.

3. Семафор берётся внутри задачи, вокруг `async with sem`, счётчик инкрементируется после входа. Для 100 задач по 20 мс при лимите 5: пик 5, гистограмма `[(1, 20), (2, 20), (3, 20), (4, 20), (5, 20)]`, время ≈0,41 с. Семафор вокруг `gather` ограничит создание задач, а не работу.

4. Воркер берёт `async with lock`, меняет метрику и держит её в `try/finally`: `finally` выполняется и при `CancelledError`. После `task.cancel()` через 100 мс проверка даёт `lock.locked() == False`, `inflight == 0`, `task.cancelled() == True`. Глотать `CancelledError` нельзя — задача считается успешной.

5. 10 млн чисел, 4 части по 2,5 млн: один поток — 1,70 с, четыре потока — 2,98 с, четыре процесса — 0,92 с. Потоки медленнее последовательного варианта: GIL не даёт параллелизма, добавляются переключения. Процессы дают ≈1,8× на 8 ядрах;

цена — сериализация аргументов и результатов.

Глава	Упражнений	Ответов
9. HTTP/1.1	5	5
10. HTTP/2 и HTTP/3	4	4
11. Куки, сессии, кэш	5	5
12. TLS и сертификаты	4	4
13. REST и API	5	5
14. Прокси и балансировка	5	5
15. nginx и event loop	5	5
16. Python: окружение	5	5
17. Синтаксис и типы	5	5
18. Функции и декораторы	5	5
19. ООП и протоколы	5	5
20. asyncio	5	5
Итого	**58**	**58**

Приложение D. Ответы и решения — часть 3 (главы 21–32)

Все ответы проверены на локальном стенде: Python 3.13, bash 5.2, Go 1.26.10. Выводы в ответах — реальные результаты запусков, а не ожидаемые.

Глава 21. Сеть и API на Python

1. `fetch(url, timeout=5.0)` держит `urlopen` в `try` с тремя ветками: `HTTPError` → `(e.code, e.read())`, `(URLError, OSError)` → `(0, str(e))`, успех → `(r.status, r.read())`. Проверено на локальном сервере: 200, 404 и таймаут 1 с дают `(200, …)`, `(404, …)`, `(0, …)`, наружу не летит ничего. У httpx то же самое — одна строка `client.get()` при уже настроенном `Timeout(5)`.
2. В `retry_after`: если `raw.isdigit()` — `int(raw)`; иначе `email.utils.parsedate_to_datetime(raw)` и `max(0, int((dt - now_utc).total_seconds()))`. Проверено: `"120"` → 120, дата «сейчас + 60 с» → 59 (дробная часть отбрасывается), мусор → `default`. Без `tzinfo=UTC` у naive-даты вычитание бросает `TypeError`.
3. Бюджет — скользящее окно 60 с: `deque` из `(ts, is_retry)`, перед повтором выбрасываем старые метки и считаем долю. Если `retries/total > 0.1`, повтора нет — сразу `ApiError`. При исчерпании лучше отдать ошибку наверх, чем ждать: бюджет общий на процесс, и пауза останавливает все воркеры.
4. `allow()` пускает, пока `opened_at is None`; пятая подряд неудача в `record()` открывает цепь на 30 с, затем пропускается ровно один пробный запрос, успех сбрасывает счётчики. На `MockTransport` с 503 первые 10 вызовов из 13 попыток до сети дошли 9, остальные отсеклись мгновенно; пробный после паузы получил 200 и закрыл цепь.

Глава 22. Файлы, конфиги, логирование

1. `tail` идёт от конца блоками по 8 КБ: `f.seek(0, 2)`, затем назад, пока `buf.count(b"\n") <= n`, результат — `splitlines()[-n:]`. На файле 1 ГБ (≈21 млн строк) вышло 0.1 мс и maxrss 10.4 МБ, то есть память не зависит от размера файла. Неравенство `<= n` нужно из-за завершающего перевода строки.
2. `merge_configs` делает `out = dict(base)` и рекурсивно сливает только словари: `isinstance(v, dict) and isinstance(out.get(k), dict)`. Списки и скаляры заменяются целиком: `tags: ["c"]` перетёр `["a","b"]`, а `db.opts` стал `{ssl: true, timeout: 5}`. Исходный `base` не изменился — на каждом уровне создаётся новый словарь.
3. Фильтр обходит структуру рекурсивно и заменяет значение, если `k.lower() in {"password","token","authorization"}`; в `logging.Filter` переписывается `record.msg`, а `record.args` обнуляется. Проверено на вложенности и списке: `password`, `nested.token` и `list[0].Authorization` стали `***`, остальные поля не тронуты.
4. Ротация: `shutil.copyfileobj` в `gzip.open(..., "wb")`, затем пустой файл через `os.replace` (атомарнее, чем `truncate`), потом удаление `*.gz` со `st_mtime` старше 14 суток. Проверено: архив за сегодня создан, лог стал 0 байт, файл 20-дневной давности удалён, свежий остался.

Глава 23. Тесты: pytest

1. `@pytest.mark.parametrize` на шесть пар: `0s→0`, `1h1h→7200`, `999999d→86 399 913 600`, `2d3h4m5s`. Отдельный параметризованный тест на `ValueError`: `""`, `"15"`, `"1x"`, `"1h30"`, `"1H"`. Итог — 15 тестов за 0.04 с. `1h1h` проходит, потому что парсер суммирует все токены, а не берёт последний.
2. Фикстура подменяет `time.monotonic` и `time.sleep` через `monkeypatch.setattr`; `time` импортируется самим тестируемым модулем, поэтому подмена видна его коду. `wait_ready(5.0, poll=0.25)` даёт ровно 20 итераций и `expired=True` за миллисекунды: виртуальные часы двигает заглушка `sleep`, реального ожидания нет.
3. `asyncio.sleep` подменяется корутиной, собирающей аргументы, а транспорт считает обращения. Проверено: при трёх 503

`len(calls) == 3` и `slept == [0.5, 1.0]`, тест идёт 0.01 с. Проверять нужно и число попыток, и задержки — иначе тест зелёный и при ретраях без backoff.

4. Строка `max(0, int(...))` в `retry_after` была покрыта, но не проверена: тесты подавали только будущие даты, клампинг не срабатывал. Новый тест с датой в прошлом требует 0 и падает, если убрать `max`. Покрытие ветвей этого не видит: обе ветки исполнены, а поведение не проверено.

Глава 24. CLI и упаковка

1. `cmd_status` печатает `json.dumps({...}, sort_keys=True)` при `--json`, иначе `занято мест: N` и строки `seat -> owner`; `return EXIT_OK` в обеих ветвях. Проверено: stdout `{"holds": {"12A": "alice"}, "seats": 1}`, rc=0. `sort_keys` даёт стабильный вывод для тестов и диффов.
2. Тест: `hold`, затем `before = st.read_bytes()`, `mtime = st.stat().st_mtime_ns`, вызов `--dry-run release 12A` → rc=0, `read_bytes() == before`, `mtime` тот же. Сверять только байты мало: `save` может перезаписать файл идентичным содержимым, и mtime ловит именно это.
3. На `typer` вышло 79 строк против 96: исчезли `build_parser` и `set_defaults`, типы берутся из аннотаций. Коды сохранены явно — `raise typer.Exit(3)`, диагностика — `typer.echo(..., err=True)`. Проверено: конфликт даёт rc=3 и сообщение в stderr, `status --json` отдаёт тот же JSON.
4. `pip wheel . -w dist` → `seatctl-1.4.0-py3-none-any.whl`; в чистом venv `pip install --no-index --find-links dist seatctl`, из `/tmp` команда `seatctl --version` печатает `seatctl 1.4.0`. В контейнере то же плюс `--no-cache-dir`. Работает за счёт `[project.scripts]`: без точки входа файла в `bin/` не будет.
5. Job `build`: `pip wheel`, сверка версии с тегом, `twine upload --repository-url $INDEX`. Job `verify`: `pip install --index-url $INDEX seatctl==$VERSION` и `seatctl --version`. Разделение гарантирует, что проверяется именно опубликованный артефакт, а не локальная сборка из соседнего каталога.

Глава 25. Шелл: переменные, кавычки, подстановки

1. `dir=${p%/*}` (если слэша нет, результат равен `$p` — подставляем `.`), `f=${p##*/}`, дальше `base=${f%.*}` и `ext=${f##*.}` только при `[[ $f == *.* ]]`. Проверено: `/var/log/syslog.bak` → `/var/log|syslog|bak`, `report.txt` → `.|report|txt`, `/etc/hosts` → `/etc|hosts|` (расширения нет).
2. `while IFS= read -r h; do printf '<%s>\n' "$h"; done < hosts` печатает по строке на хост: `<web 01>`, `<db*>`. Вариант `for h in $(cat hosts)` сначала режет вывод по IFS, а потом раскрывает glob: `web 01` превращается в два элемента, а `db*` — в список файлов каталога.
3. `a=(alpha beta gamma delta epsilon); a+=(zeta)` → `${#a[@]}` = 6, `printf '%s\n' "${a[@]}"` печатает по одному в строку. Ассоциативный: `declare -A http=([200]="OK" [404]="Not Found" …); http[429]="Too Many Requests"` даёт 4 элемента, обход `"${!http[@]}"` неупорядочен, поэтому `| sort -n`.
4. `die() { printf '%s\n' "${1:-ошибка}" >&2; exit "${2:-1}"; };` проверка: `[[ -n ${TOKEN:-} ]] || die "нет TOKEN" 2`. `${2:-1}` подставляет код 1, а `exit` внутри функции завершает весь скрипт, а не только функцию — поэтому `die` нельзя вызывать в `$(...)`.
5. Правило: кавычки вокруг каждой подстановки, `--` перед путями, `find … -print0 | xargs -0`, вместо `$(ls)` — glob `for f in ./*`. До правки shellcheck ругается на неквотированное `$(ls)` (SC2046) и переменные без кавычек (SC2086), после — чисто. В каталоге с `a b.txt`, `*star`, `-n` неквотированный цикл видит пять «файлов» вместо трёх.

Глава 26. Управляющие конструкции и коды возврата

1. `is_number() { [[ $1 =~ ^-?[0-9]+$ ]]; }` на `12abc` даёт 1. Арифметический вариант `(( $1 ))` печатает `((: 12abc: value too great for base (error token is "12abc")` и тоже возвращает 1, но шумит в stderr, спотыкается на `08` (восьмеричная запись) и считает `0` ложью. Для валидации входа годится только regex.

2. Ручной цикл: `while (($#)); do case $1 in --env=*) env=${1#*=} ;; --env) env=$2; shift ;; --) shift; rest+=("$@"); break ;; -*) die "флаг $1" 2 ;; *) rest+=("$1") ;; esac; shift; done`. Проверено: `--env=prod` и `--env stage` дают одинаковый результат, неизвестный `-x` → rc=2. `getopts` длинные опции не умеет.

3. В `if f; then … set -e` отключён на всём теле `f`: после `false` печатается «после false внутри f», функция возвращает 0, скрипт идёт дальше. Прямой вызов `g` с тем же `false` завершает скрипт с rc=1, и «НЕ ДОСТИГНУТО» не печатается. Лечится `|| return 1` после каждого шага.

4. `case ${f##*.} in json) tool=jq ;; yaml|yml) tool=yq ;; csv) tool=awk ;; *) echo "нет инструмента: $f" >&2; exit 3 ;; esac`. Проверено: три файла разобраны (`jq`, `yq`, `awk`), `notes.txt` даёт rc=3 и сообщение в stderr. Явный `exit 3` нужен потому, что код по умолчанию — 1, а 3 отличает «неизвестный формат» от прочих сбоев.

5. `until code=$(curl -fsS -o /dev/null -w '%{http_code}' --max-time 2 "$url"); do (( SECONDS < deadline )) || exit 1; sleep 1; done`. `set -e` не срабатывает: команда стоит в условии `until`. Проверено на сервере с 503/503/200: два сообщения curl об ошибке, успех через 2.0 с, rc=0.

Глава 27. Текст: grep, sed, awk, jq, xargs, find

1. Сортировка вставками прямо в `END`:

bash

```
awk -F: '$3>=1000{n++;u[n]=$1;id[n]=$3}
END{for(i=2;i<=n;i++){t=id[i];s=u[i];for(j=i-1;j>0&&id[j]>t;j--){id[j+1]=id[j];
    u[j+1]=u[j]} id[j+1]=t;u[j+1]=s}
for(i=1;i<=n;i++)print u[i],id[i]}' /etc/passwd
```

Вывод совпал с `sort -k2,2n`: `nextdoor 1000`, `nobody 65534`. POSIX awk не имеет `sort`, поэтому «без sort» — ручная сортировка (в gawk есть `asort`, но он непортируем). Порядок у `awk` не гарантирован, потому что он обходит вход последовательно и индекс не хранит.

2. Поля формата combined: 1 — IP, 9 — код, 11 — секунды.

bash

```
awk '{n[$1]++; if($9>=500) e[$1]++; if($11>m[$1]) m[$1]=$11}
END{for(ip in n) printf "%-12s %4d %4d %7.3f\n", ip, n[ip], e[ip]+0, m[ip]}'
```

```
    access.log | sort -k3,3nr
```

`e[ip]+0` обязателен: у IP без 5xx поле пустое. Вывод: `10.0.0.2 2 2 2.500`, `10.0.0.1 3 1 1.204`, `10.0.0.3 1 0 0.008`.

3. `sed -E '/^[[:space:]]*#/d; s/^[[:space:]]*([A-Za-z0-9_.]+)[[:space:]]*=[[:space:]]*(.*)$/\U\1\E=\2/'`. Проверено: `key = value` → `KEY=value`, `empty =` → `EMPTY=`, комментарий удалён, `DB_HOST = db.local` → `DB_HOST=db.local`. `\U…\E` — расширение GNU sed, в BSD/macOS нужен perl или `tr`.

4. `jq -sc 'group_by(.env)[] | {env: .[0].env, total: (map(.dur)|add)}' builds.ndjson` даёт `{"env":"prod","total":24}` и `{"env":"stage","total":3.0}`. Дальше `jq -r 'select(.ok|not) | .id' builds.ndjson | xargs -r curl -fsS -X POST https://ci.example/builds/` — id уходят аргументами по одному.

5. `find /var/log -type f -mtime +14 -print0 > list`; размер — `xargs -0 -r stat -c %s < list | awk '{s+=$1} END{print s}'`; удаление — `xargs -0 -r rm -f -- < list` под `(( DRY_RUN ))`. NUL обязателен из-за имён с переводом строки. Проверено на каталоге с `b c.log`: 2 файла, 4000 байт.

Глава 28. Процессы, сигналы, перенаправления

1. Запуск в фоне с накоплением PID, затем `for p in "${pids[@]}"; do if wait "$p"; then echo "$p: 0"; else echo "$p: $?"; rc=1; fi; done; exit "$rc"`. Проверено: `586838: 0`, `586839: 7`, `586840: 0` и общий rc=1. Ждать нужно по PID: `wait` без аргументов вернул бы код только последнего задания.

2. Порядок редиректов решает всё, потому что `2>&1` копирует текущее значение fd 1:

вариант	stdout	stderr
`>log 2>&1`	файл	файл
`2>&1 >log`	файл	терминал
`>log`	файл	терминал
`2>log`	терминал	файл

Во втором случае `2>&1` сработал до `>log`, поэтому stderr остался на терминале; в файле только `out`.

3. `arr=(); while IFS= read -r h; do arr+=("$h"); done < <(gen_hosts)` — подстановка процесса вместо пайпа. Пайп выполнил бы `while` в подоболочке, и массив умер бы вместе с ней. Проверено: `${#arr[@]}` = 2, `arr[1]` = `b c`, значения с пробелом целы.
4. `trap on_term TERM INT` с флагом `stopping`: первый SIGTERM пишет в fd 3 и ставит флаг, второй делает `exit 143`; `trap on_hup HUP` только перечитывает конфиг. Проверено: в логе старт, HUP, TERM, повторный TERM, rc=143. `exec 3>>log` держит дескриптор отдельно, stdout и stderr остаются свободными.
5. `nohup ./worker.sh &`: после `kill -9` процесс исчезает навсегда, поднимать некому, в `nohup.out` остаётся хвост без причины смерти. systemd с `Restart=on-failure` перезапустит юнит через `RestartSec`; `systemctl status` и journal покажут `code=killed, status=9/KILL`, `systemctl show -p NRestarts` — счётчик.

Глава 29. Практика: скрипты бэкапа, healthcheck, деплоя, systemd timers

1. Проверка ключа идёт первой строкой, до `mkdir` и `tar`: `gpg --batch --list-keys "$RECIPIENT" || { echo "нет ключа" >&2; exit 3; }`. Дальше `tar -czf - dir | gpg --batch --yes -e -r "$RECIPIENT" -o "$tmp"` и `mv` в финальное имя. Проверено: без ключа rc=3 и ни одного файла, с ключом архив расшифровывается в исходное дерево.
2. Цикл по конфигу копит `failed+=("$url ($code)")`, в конце один `logger -t healthcheck "упало N: …"` и `exit 1`. Тонкость: `code=$(curl … || true); code=${code:-000}` — curl сам печатает `000` при отказе, и `|| echo 000` даёт `000000`. Проверено: из четырёх эндпоинтов упали два, оба попали в алерт с кодами.
3. После `ln -sfn releases/$NEW current` — десять запросов на `127.0.0.1:$PORT` (порт релиза, мимо прокси), считаем не-200; `(( bad * 100 > 10 * n ))` при `n=10` → `ln -sfn releases/$OLD current` и `exit 1`. Значит, допустима ровно одна ошибка из десяти — это стоит проговорить в ревью.
4. `hc.timer`: `OnCalendar=*:0/5`, `AccuracySec=30s`, `Unit=hc.service`; `hc.service`: `Type=oneshot`, `ExecStart=/usr/local/bin/healthcheck.sh`, `TimeoutStartSec=20`. `systemd-analyze verify` проходит, ругаясь только на отсутствующий

бинарь, `systemd-analyze calendar '*:0/5'` печатает `*-*-* *:00/5:00`.

5. Частые провалы: нет `flock` — при `Persistent=true` два бэкапа пишут в один файл; нет `TimeoutStartSec` — зависший `tar` блокирует следующий запуск; относительный путь в `ExecStart` даёт `WorkingDirectory=/` и «файл не найден»; нет `trap … EXIT` — `.part` остаётся после обрыва.

Глава 30. Go: начало

1. `go mod init example.com/lab/hello`, `main.go` печатает `runtime.Version()` и `runtime.NumCPU()`: `go: go1.26.10`, `cpus: 8`. Обычная сборка — 2 406 230 Б, с `-trimpath -ldflags="-s -w"` — 1 585 314 Б (−34%). `-s -w` убирает таблицы символов и DWARF, `-trimpath` — абсолютные пути в бинаре.

2. `internal/greet/greet.go` с `func Hello(name string) string` и тест в том же каталоге: `go test ./...` → `ok example.com/lab/hello/internal/greet`. Импорт из чужого модуля даже с `replace` даёт `use of internal package example.com/lab/hello/internal/greet not allowed` — это проверка компилятора, а не соглашение.

3. `mux.HandleFunc("GET /healthz", …)` пишет `ok`; `/version` отдаёт `json.NewEncoder(w).Encode(map[string]string{"go": runtime.Version(), "build": buildTime})`, а `var buildTime = "unknown"` переопределяется `-ldflags "-X main.buildTime=$(date -u +%FT%TZ)"`. Проверено: `ok` и `{"build":"2026-10-29T16:47:40Z","go":"go1.26.10"}`; `POST /healthz` → 405.

4. `CGO_ENABLED=0 GOOS=linux GOARCH=arm64 go build …` → `ELF 64-bit LSB executable, ARM aarch64, statically linked`; `GOOS=windows GOARCH=amd64` → `PE32+ executable (console) x86-64`. `.exe` Go добавляет, только когда имя выбирает сам (`go build ./cmd/hello` → `hello.exe`); при явном `-o hello-windows` суффикса не будет.

5. Шаг CI: `test -z "$(gofmt -l .)" || { gofmt -d .; exit 1; }`, затем `go mod tidy` и `git diff --exit-code go.mod go.sum`. Проверено: неформатированный файл валит первый шаг, после `gofmt -w` проходит. Порядок — от дешёвого к дорогому: fmt, tidy (сеть), vet, тесты, сборка.

Глава 31. Синтаксис: срезы, мапы, указатели, интерфейсы, ошибки

1. `Dedup` пишет в новый срез, поэтому вход не мутируется:

```go
func Dedup(in []string) []string {
    seen := make(map[string]struct{}, len(in))
    out := make([]string, 0, len(in))
    for _, s := range in {
        if _, ok := seen[s]; ok {
            continue
        }
        seen[s] = struct{}{}
        out = append(out, s)
    }
    return out
}
```

Тест сверяет и результат `[b a c]` для `[b a b c a]`, и то, что исходный срез остался из пяти элементов. `make(..., 0, len(in))` даёт одну аллокацию вместо роста.

2. Поля — `rows, cols int` и `data []float64`. `Set` обязан быть на указателе: он меняет данные, и на копии структуры запись потерялась бы. `At` и `Transposed` приёмник не меняют, но указатель дешевле:

```go
func (m *Matrix) At(r, c int) float64     { return m.data[r*m.cols+c] }
func (m *Matrix) Set(r, c int, v float64) { m.data[r*m.cols+c] = v }

func (m *Matrix) Transposed() *Matrix {
    t := &Matrix{rows: m.cols, cols: m.rows, data: make([]float64, len(m.data))}
    for r := 0; r < m.rows; r++ {
        for c := 0; c < m.cols; c++ {
            t.data[c*t.cols+r] = m.data[r*m.cols+c]
        }
    }
    return t
}
```

3. `type Store interface { Get(string) ([]byte, error); Put(string, []byte) error }`. `MemStore` — мапа под `sync.RWMutex`; `DirStore` — `os.ReadFile`/`os.WriteFile` с обёрткой `errors.Is(err, fs.ErrNotExist) → ErrNotFound`. Функция `Copy(dst, src Store, key string) error` работает с любым: обе реализации проходят проверку `var _ Store = …`.

4. `var ErrNotFound, ErrConflict = errors.New("не найдено"), errors.New("конфликт")`, `type ValidationError struct{ Field string }` с методом `Error()`. В обработчике первым идёт `errors.As(err, &ve)` → 422, затем `errors.Is` для 404 и 409, иначе 500. `As` нужен раньше:

тип-ошибка не совпадёт с sentinel по `Is`, но обёрнута через `%w`.

5. `Recover(next)` — `defer func(){ if v := recover(); v != nil { log.Printf("panic: %v\n%s", v, debug.Stack()); w.WriteHeader(500) } }()`. Тест на `httptest`: путь `/boom` даёт 500, следующий запрос на `/live` — 200, сервер жив. Паника в горутине обработчика этот middleware не поймает — она уронит процесс.

Глава 32. Конкурентность: горутины, каналы, context

1. На каждый источник — горутина-пересыльщик, общий `WaitGroup` закрывает результат:

```
go

func Merge(chans ...<-chan int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup
    wg.Add(len(chans))
    for _, ch := range chans {
        go func(c <-chan int) {
            defer wg.Done()
            for v := range c {
                out <- v
            }
        }(ch)
    }
    go func() { wg.Wait(); close(out) }()
    return out
}
```

Проверено на трёх каналах, один из которых отдаёт значение через 20 мс: получены все четыре значения, канал закрылся сам.

2. Ограничение — буферизованный `sem := make(chan struct{}, limit)` с `defer func(){ <-sem }()`; ошибки копятся в срез под `sync.Mutex` и возвращаются `errors.Join(errs...)`. Тест на `httptest` со счётчиком активных запросов под `atomic`: пик не превысил 2 при `limit=2`, оба 503 попали в `errors.Join`, прогон под `-race` чистый.

3. Профиль отдаёт `3 @ 0x… main.main.func1.1 … main.go:18` — три горутины в одной точке. Стек читают снизу вверх: последняя строка — ваш код, выше — стандартная библиотека. `chan send` в этой строке означает горутину, ждущую читателя, которого нет: канал создан, но `range`/`<-` по нему не запущен.

4. `select { case <-time.After(10 * time.Second): fmt.Fprintln(w, "done"); case <-ctx.Done(): return }`. `curl --max-time 1` получил rc=28, а в логе появилось `отменён: context canceled` сразу после разрыва: `r.Context()` закрывается вместе с соединением, и работа прекращается, а не считается вхолостую.
5. На 8-ядерной машине: мьютекс 20.3 / 92.2 / 120.1 нс на 1/4/8 горутинах, атомик 12.8 / 18.0 / 21.0 нс. Мьютекс сериализует горутины и на 8 даёт в 5.7 раза хуже; атомик не блокирует, а крутит CAS на кэш-линии, поэтому деградация мягкая. Разница видна только при высокой частоте операций.

Сводка по главам (упражнений / ответов)

Глава	Упражнений	Ответов
21	4	4
22	4	4
23	4	4
24	5	5
25	5	5
26	5	5
27	5	5
28	5	5
29	5	5
30	5	5
31	5	5
32	5	5
Итого	**57**	**57**

Приложение D. Ответы и решения — часть 4 (главы 33–44)

Глава 33. HTTP-сервер и клиент

1. `mux.HandleFunc("GET /items/{id}", h)` — шаблон с методом сам отдаёт 405 и заголовок `Allow: GET, HEAD`. Валидация: `id, err := strconv.Atoi(r.PathValue("id"))`, при ошибке `Content-Type: application/json`, `w.WriteHeader(400)` и тело `{"error":"id must be an integer"}`. Проверка: `curl -i -X POST localhost:8080/items/1` → `405`, `Allow: GET, HEAD`.
2. `map[string]*bucket` под мьютексом, пополнение ленивое: `tokens = min(burst, tokens+elapsed.Seconds()*10)`; `burst=10` разрешает всплеск, поэтому в секунду проходит больше десяти. Отказ — `429` и `Retry-After: 1`. Проверка циклом `curl` с `-w '%{http_code}'` и `uniq -c`: ~10×200, остальные 429. Записи без обращений чистить, иначе map растёт от сканеров.
3. Повтор только для идемпотентных запросов и только на `503/429/5xx` и сетевые ошибки, три попытки, задержка `base << i` со сном `rand.Int63n(d)` — full jitter против синхронной волны. Перед повтором `io.Copy(io.Discard, resp.Body)` и `Close()`, иначе соединение не вернётся в пул. В логе 180 мс, затем 740 мс, третий запрос 200.
4. `signal.NotifyContext` + `srv.Shutdown` с таймаутом 15 с: новые соединения не принимаются, активные доигрывают — `hey -n 2000 -c 50` даёт ноль ошибок. С `WriteTimeout: 100ms` клиент видит `unexpected EOF`: дедлайн считается от конца чтения заголовков, запись обрывается, обработчик не отменяется и 503 отдать не может.
5. Отдельный мультиплексор на `127.0.0.1:6060` с `pprof.Index`. `go tool pprof -top 'http://localhost:6060/debug/pprof/goroutine?seconds=30'`: `net/http.(*conn).serve` — занятые обработкой (сходится с `-c`), `runtime.gopark` и `selectgo` — ждущие на канале, базе или мьютексе. Сотни ждущих при `-c 50` — утечка горутин.

Глава 34. Базы данных в Go

1. `docker run -e POSTGRES_PASSWORD=secret postgres:17 -c max_connections=20`, `MaxOpenConns=0`, 200 параллельных → `pq: sorry, too many clients already` и 500 у части запросов, страдают и работающие поды. Лечение: `SetMaxOpenConns(15)`, `SetMaxIdleConns(15)`, `SetConnMaxLifetime(30*time.Minute)`; поды × соединений на под ≤ `max_connections` минус резерв.
2. Обёртка держит `atomic.Int64` и инкрементирует его в `QueryContext/ExecContext`, middleware пишет `db_queries_per_request`. N+1 — 101 запрос на страницу; исправление: собрать `ids` и `SELECT id, name FROM users WHERE id = ANY($1)` с `pq.Array(ids)`, разложить в map. Стало два запроса, p99 падает в разы.
3. `BeginTx` с `sql.LevelSerializable`; `UPDATE accounts SET balance = balance - $1 WHERE id = $2 AND balance >= $1`, проверка `RowsAffected()==1` иначе откат. До трёх повторов только на `40001` с паузой 20–100 мс и jitter. Проверка: 100 параллельных переводов, сумма неизменна, отрицательных нет, повторы видны в метрике.
4. `goose -dir migrations create add_order_status sql`: `ALTER TABLE orders ADD COLUMN status text NOT NULL DEFAULT 'new'`; отдельной миграцией `CREATE INDEX idx_orders_user_id` (для `CONCURRENTLY` — `-- +goose NO TRANSACTION`); затем `DROP COLUMN legacy_state`. `goose up`, `down`, `up` — всё Applied, `\d orders` подтверждает.
5. `/readyz` делает `db.PingContext` с таймаутом 2 с и отдаёт 503; liveness базу не проверяет, иначе под рестартует вместо выхода из балансировки. `docker stop pg` → после `failureThreshold: 2` при `periodSeconds: 5` под NotReady, IP уходит из `kubectl get endpoints api`, трафик не идёт. Запуск базы → возврат за ~10 с, `RESTARTS` остаётся 0.

Глава 35. Тесты, бенчмарки, pprof, race detector

1. `tests := []struct{ name, in string; want int; wantErr bool }{...}` и `for _, tt := range tests { t.Run(tt.name, ...) }`. Один случай: `go test ./pkg -run 'TestParse/integer' -v` — слэш отделяет подтест, пробелы в имени заменяются подчёркиваниями. Новый кейс становится

одной строкой в таблице, а не копией тела теста.

2. `go test -bench=Encode -benchmem -count=5 > old.txt`, буфер вынести в `var bufPool = sync.Pool{New: func() any { return new(bytes.Buffer) }}`, затем `benchstat old.txt new.txt`: `allocs/op` 4 → 0–1, падают `B/op` и `ns/op`. Оговорка: для мелких объектов `Pool` медленнее обычной аллокации, выигрыш только на дорогих объектах, уходящих в кучу.

3. `go test -bench=Handle -cpuprofile cpu.out`, затем `go tool pprof -top cpu.out` — верхние flat-строки обычно `runtime.mapaccess2_faststr`, `encoding/json.Marshal`, `compress/flate`. Догадку проверяем `go tool pprof -list=json.Marshal cpu.out`: стоимость разложена по строкам исходника, `-peek` показывает вызывающих, `-http=:8080` рисует граф.

4. `var n int` и сто горутин с `n++`, `go test -race` печатает `WARNING: DATA RACE` с двумя стеками и завершает прогон с ненулевым кодом. Исправления: `sync.Mutex` вокруг `n++` (~20 нс), `atomic.Int64.Add(1)` (~7 нс, лучший вариант для счётчика), канал или горутина-владелец (~100 нс, оправдан, когда нужен ещё и порядок).

5. Бесконечный `for { select { case <-ch: ...; case <-time.After(time.Second): } }` → `go test -timeout 30s` паникует дампом всех горутин. Лечение — бюджет: `ctx, cancel := context.WithTimeout(t.Context(), 5*time.Second)` и проверка `ctx.Err()`; запас на уборку — `context.WithDeadline(ctx, t.Deadline().Add(-2*time.Second))`. Падение становится assertion.

Глава 36. Сборка и поставка

1. `CGO_ENABLED=0 go build` → `ldd` печатает `not a dynamic executable`; сборка с `CGO_ENABLED=1` линкует `libc.so.6`. В `FROM scratch` работает только статический, cgo падает с `no such file or directory`. Разница ~1 МБ, а резолвер разный: libc с `nsswitch.conf` против чистого Go с `/etc/resolv.conf`.

2. `go build -trimpath -buildvcs=false -ldflags "-s -w -X main.version=1.4.2 -X main.date=$(date -u -d @$(git log -1 --format=%ct) +%Y-%m-%dT%H:%M:%SZ)"`. `--version` → 1.4.2, `go version -m` печатает флаги.

Две сборки дают одинаковый `sha256sum`; `date +%s` или отсутствие `-trimpath` ломают совпадение.

3. `docker buildx build --platform linux/amd64,linux/arm64 -t registry.example.com/api:1.4.2 --push .`, затем `docker buildx imagetools inspect registry.example.com/api:1.4.2` — индекс-манифест и два дочерних со своими digest. Мультиарх нельзя `--load`: ошибка про несколько платформ, нужен `--push` или `--output type=oci`.

4. `-s -w` даёт 15 МБ → ~10,5 МБ, `upx -9` — ~4 МБ. Но распаковщик разворачивает код в анонимные страницы: RSS выше на несколько мегабайт, страницы не делятся, core dump и символизация `pprof` ломаются, антивирус ругается, а в реестре выигрыш близок к нулю — образ и так жмётся gzip. Вывод: UPX не применять.

5. `syft "$IMAGE" -o cyclonedx-json=sbom.json` + `cosign attest --type cyclonedx --predicate sbom.json`; `cosign sign --yes "$IMAGE"@sha256:....` Гейт: `cosign verify "$IMAGE"@sha256:... --certificate-identity-regexp '^https://github.com/org/repo/.github/workflows/release.yml@.*$' --certificate-oidc-issuer https://token.actions.githubusercontent.com`, ненулевой код останавливает деплой.

Глава 37. Git: модель данных, ветки, rebase, хуки

1. `git cat-file -p HEAD` печатает `tree`, `parent`, автора, коммиттера и сообщение; `git cat-file -p HEAD^{tree}` — список `mode type hash name`. `git hash-object README.md` возвращает ровно тот же blob-хеш, что в дереве: хеш считается от `"blob <длина>\0"` плюс содержимое, без времени и имени файла. Отсюда же дедупликация блобов между ветками.

2. `git switch -c feat`, три коммита, `git rebase -i HEAD~3` — первый `pick`, остальные `squash`. Возврат: `git reflog` показывает `rebase (finish): returning to refs/heads/feat` и запись до неё, `git reset --hard feat@{1}` восстанавливает три коммита. Reflog живёт 90 дней, объекты через него достижимы — это и есть страховка от неудачного rebase.

3. `git config rerere.enabled true` заранее. Конфликт → правка → `git add` → `git rebase --continue`, решение оседает в `.git/rr-cache`; `git reset --hard ORIG_HEAD`; повторный rebase печатает `Resolved 'app.go' using previous resolution` и ничего не спрашивает. `git rerere status` и `git rerere diff` показывают, что именно запомнено.

4. `pre-commit`: `files=$(gofmt -l .); [ -z "$files" ] || { echo "$files"; exit 1; }`. `commit-msg`: `grep -qE '^[A-Z]+-[0-9]+' "$1" || { echo "нужен ABC-123"; exit 1; }`, оба `chmod +x`. `git commit --no-verify -m wip` проходит: хуки локальны и обходятся флагом, поэтому те же проверки дублируют в CI.

5. `git log -S 'AKIA' --oneline --all` находит коммит. Чистка: `git filter-repo --replace-text <(echo 'AKIA...==>REDACTED')`, затем `git reflog expire --expire=now --all && git gc --prune=now`; `git cat-file -e` на старый blob → `Not a valid object name`. Ключ всё равно отозвать: форки и кэши CI перезапись не получат.

Глава 38. Docker: слои, Dockerfile, compose

1. `docker history --no-trunc --format '{{.Size}}\t{{.CreatedBy}}' api:old | sort -h | tail -3` — база, `apt-get install`, `COPY . .` с `node_modules` и `.git`. Вдвое: multi-stage, alpine или distroless, `--no-install-recommends` с удалением `/var/lib/apt/lists` в том же `RUN`, `.dockerignore`. 1,1 ГБ → 60–90 МБ.

2. Слои неизменяемы: `RUN apt-get install -y curl` добавляет ~12 МБ, следующий `RUN apt-get purge -y curl` пишет whiteout — файлы из нижнего слоя не удаляются, а маскируются. `docker history` показывает оба слоя, размер не меняется. Правильно ставить и удалять в одном `RUN`, а для отладки брать `kubectl debug`, не curl в проде.

3. Builder `python:3.14-slim` с `pip wheel -w /wheels -r requirements.txt`; runtime — тот же slim, `COPY --from=builder /wheels /wheels` и `pip install --no-cache-dir /wheels/*.whl`. Размер 1,1 ГБ → ~180 МБ, холодная сборка дольше из-за двух этапов; `RUN --mount=type=cache,target=/root/.cache/pip` возвращает тёплым пересборкам секунды.

4. Ключ — условие, а не список: `depends_on: {postgres: {condition: service_healthy}, redis: {condition: service_healthy}}`. Healthcheck: `pg_isready -U app -d app`, `redis-cli ping`, `wget -qO- localhost:8080/healthz`. После `up -d` все healthy, `RestartCount` равен 0. Без условий приложение стартует раньше базы и рестартует.

5. `docker run --memory=256m --memory-swap=256m` при 100 МБ даёт OOMKilled, код 137 и пустой лог. Понятная ошибка: `GOMEMLIMIT=200MiB` (80% лимита) плюс опрос `runtime.ReadMemStats`, на 85% — `WARN memory high` и `log.Fatal`, код 1. `docker inspect -f '{{.State.OOMKilled}} {{.State.ExitCode}}'` → `false 1`, сообщение в логах.

Глава 39. CI/CD: пайплайны, кэш, артефакты

1. Три job: `lint` (`golangci-lint run`), `test` с `services.postgres` (`--health-cmd pg_isready`), `build` с `needs: [lint, test]` и buildx. Кэш: `actions/setup-go` с `cache: true` (ключ по `go.sum`) и `cache-from/to: type=gha,mode=max`. Холодный прогон 8 мин, повторный 3–4; в логах `Cache restored from key`. Вторая половина выигрыша — кэш слоёв buildx.

2. `strategy: {fail-fast: false, matrix: {python: ["3.13", "3.14"]}}`, версия подставляется `${{ matrix.python }}`. При `fail-fast: true` падение одной ветки отменяет соседнюю, и в гриде видно `cancelled`; с `false` обе доходят до конца, и видно, что ломается только 3.14, а 3.13 зелёный.

3. Сборка один раз, дальше продвижение по digest: job `build` отдаёт `outputs.digest`, деплой использует `image@${{ needs.build.outputs.digest }}` в dev, stage и prod; прод — `environment: production` с reviewers. Проверка: `kubectl -n dev get pod -o jsonpath='{.items[0].status.containerStatuses[0].imageID}'` и то же в stage и prod — один sha256.

4. Argo Rollouts: `steps: [{setWeight: 10}, {pause: {duration: 5m}}, {setWeight: 100}]`; analysis с PromQL `sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m])) > 0.01`, `interval: 1m`, `count: 5`, `failureLimit: 1`. При ошибках на канареечной версии Rollout уходит в `Degraded` и вес возвращается к нулю сам.

Глава 40. IaC: Terraform и Ansible

1. `aws_s3_bucket` + `aws_s3_bucket_versioning` (`Enabled`) + `aws_s3_bucket_public_access_block` (все четыре `true`) + `tags`. Приёмка: второй `terraform plan` → `No changes. Your infrastructure matches the configuration.` Предложения изменений означают неописанный атрибут или ручную правку; дрейф отдельно виден в `plan -refresh-only`.
2. `backend "s3" { bucket = "tf-state-prod", key = "api/terraform.tfstate", dynamodb_table = "tf-locks", encrypt = true }`, затем `terraform init -migrate-state`. Второй параллельный `apply` → `Error: Error acquiring the state lock` и `ConditionalCheckFailedException` с ID владельца; state цел. В Terraform 1.10+ вместо DynamoDB — `use_lockfile = true`.
3. `apt: name=nginx state=present`; `template: src=site.conf.j2 dest=/etc/nginx/conf.d/site.conf` с `notify: reload nginx`; handler `service: name=nginx state=reloaded`. Первый прогон `changed=2`, второй `changed=0, failed=0` — это и есть критерий идемпотентности. `--check --diff` заранее показывает, что изменится, и не трогает handler.
4. `vars_files: [group_vars/vault.yml]`, файл шифруется `ansible-vault encrypt`, запуск с `--ask-vault-pass`. На задаче записи секрета `no_log: true`: в выводе, включая `-vvv`, значение заменяется на `the output has been hidden`. Проверка: `ansible-playbook site.yml -vvv | grep -c 'super-secret'` → 0. Без `no_log` секрет виден в diff результата.

Глава 41. Kubernetes: поды, деплойменты, сервисы, probes

1. Deployment `replicas: 3`, метка `app: api`; Service `ClusterIP`, `selector: app: api`, `port: 80 → 8080`. `kubectl get endpoints api-svc` → три IP:8080. Селектор `app: api-v2` → `<none>`, endpointslices пуст: ClusterIP жив, но kube-proxy не знает бэкендов, клиент получает отказ. Поды Running — сломана связка Service→Pod.
2. Без startupProbe liveness с `initialDelaySeconds: 5`, `periodSeconds: 10`, `failureThreshold: 3` убивает под на ~35-й секунде при старте 40 с → `CrashLoopBackOff`, RESTARTS растёт. С `startupProbe: {periodSeconds: 5, failureThreshold: 12}` (бюджет 60 с) liveness и readiness не выполняются, пока она не прошла: под Ready один раз, `RESTARTS`

`0`.

3. `kubectl describe pod` → `Last State: Terminated, Reason: OOMKilled, Exit Code: 137`. 137 = 128+9: SIGKILL от ядра по лимиту cgroup, приложение ничего не пишет. Код 1 — собственный выход с ошибкой (паника, конфиг), в `Reason` будет `Error`. `kubectl get pod` при этом показывает `Running` с растущими RESTARTS: под перезапускается.

4. `maxUnavailable: 0` не даёт удалить старый под до Ready нового, новому нужна ёмкость → `Pending`, в describe `0/3 nodes are available: 3 Insufficient memory`. Варианты: снизить `requests` до реального потребления или добавить узел с autoscaler; либо `maxUnavailable: 1`, приняв просадку. Старые поды трафик держат.

Глава 42. Наблюдаемость: метрики, логи, трейсы, SLO

1. `sum(rate(http_requests_total{job="api", code=~"5.."}[5m])) / sum(rate(http_requests_total{job="api"}[5m]))`. При остановке серий нет, результат — пустой вектор, а не ноль: `> 0.05` не даёт данных, алерт молчит при лежащем сервисе. Поэтому `absent(up{job="api"} == 1)` или `absent(sum(rate(...)))`: «нет данных» — отдельное состояние.

2. Бакеты `{.01,.025,.05,.1,.15,.2,.25,.3,.5,1,2,5}`, граница 0.3 обязательна. `histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))`; точный p95 по логам — сортировка и элемент `ceil(0.95*n)-1`. Ошибка от линейной интерполяции внутри бакета 0.3–0.5 достигает 10–20% и у границы SLO меняет вердикт.

3. Серий столько же, сколько пар пользователь×метрика: 10 тыс. × 20 = 200 тыс., при 3–4 КБ на серию это 600–800 МБ RSS. Смотреть `prometheus_tsdb_head_series` и `process_resident_memory_bytes` на шагах 100/1k/10k. Проблема наступает, когда серии растут быстрее трафика: scrape дольше интервала, запросы таймаутят. `user_id` — в логи и трейсы.

4. Burn rate = доля ошибок / бюджет 0,001, то есть 20 при двух процентах. Быстрый (14,4× за 1 ч) сработает через ~43 минуты

устойчивых 2%: окно разбавляет начало. Медленный (6× за 6 ч) — только через часы. Пара даёт скорость и точность: короткое окно ловит пик, длинное — тихую деградацию, которую пик не показывает.

5. А делает `Extract` из `traceparent`, создаёт span и `Inject` в заголовки исходящего запроса. Если B берёт `context.Background()` или забывает `Inject`, он стартует новую трассировку: в Jaeger вместо одного дерева из двух вложенных span'ов появляются две трассы с разными `trace_id` и пустым `parent_id`, waterfall разорван, число трейсов удвоилось.

Глава 43. Безопасность и supply chain

1. `docker run --rm img id` → `uid=0(root)`. Лечение: `RUN useradd -r -u 10001 app` и `USER 10001`; securityContext `runAsNonRoot`, `runAsUser: 10001`, `allowPrivilegeEscalation: false`, `readOnlyRootFilesystem: true`, `drop: ["ALL"]`. Падает с `read-only file system` на `/tmp` и кэше — им нужны `emptyDir`. `kubectl exec -- id` → 10001.

2. `trivy image --severity HIGH,CRITICAL api:1.4.2` даёт 41 находку, с `--ignore-unfixed` — 12: разница это CVE без исправленной версии в базе, обновление пакетов их не уберёт. Порог «ноль» на живом образе недостижим; гейт — `--ignore-unfixed --exit-code 1 --severity CRITICAL` плюс исключения с датой истечения.

3. `syft api:1.4.2 -o cyclonedx-json=sbom.json`, затем `jq '.components[] | select(.name=="golang.org/x/net") | {name, version, purl}'` → `v0.40.0`, purl `pkg:golang/golang.org/x/net@v0.40.0`. По одному образу ответ за секунды; по сорока образам без Dependency-Track — часы на вытягивание манифестов, без SBOM — грепать `go.sum` по репозиториям.

4. `cosign sign --yes api@sha256:...`, затем `cosign verify api@sha256:... --certificate-identity=dev@example.com --certificate-oidc-issuer=https://token.actions.githubusercontent.com`. Неверная идентичность → `Error: no matching signatures`, код 1. Сертификат Fulcio получит любой, привязку к пайплайну дают только identity и issuer; проверять по digest.

5. Только `http`/`https`; резолв через `net.Resolver`; отклонить 127/8, 10/8, 172.16/12, 192.168/16, 169.254/16, `::1`, fc00::/7. Диалектить по разрешённому IP (иначе DNS rebinding обходит проверку), без редиректов или с проверкой каждого хопа, тело через `io.LimitReader`. `169.254.169.254` и домен на 10.0.0.5 → 400 `destination not allowed`.

Глава 44. Инциденты, postmortem, on-call

1. SEV1 — полный отказ, потеря данных или утечка: все на связи, IC сразу, апдейт каждые 15 мин. SEV2 — ключевая функция: IC и владелец, 30 мин. SEV3 — деградация: дежурный, час. SEV4 — тикет. Если на трёх прошлых инцидентах реакция не совпала с уровнем, критерий расплывчат: заменить слова числами.

2. `kubectl scale deploy api --replicas=2`: клиенты не замечают, ошибок нет, латентность +20–30% из-за меньшей ёмкости. Признак — только `up{job="api"} == 0` и `kube_deployment_status_replicas_available`. MTTD — до срабатывания алерта с `for: 2m` (3:20), MTTR — до возврата реплики (4:10). MTTD выше пяти минут означает, что алерта на `up` нет.

3. Влияние: 1240 ошибок за 8 минут, 3% оформлений, данных не потеряно. Таймлайн UTC: 10:02 конфиг, 10:04 5xx, 10:07 алерт, 10:09 IC, 10:16 откат, 10:20 восстановление. Причина: `/health` стал отдавать 500, под выведен, ёмкость — две реплики. Actions: `up==0` (Иванов, 05.11), `minReplicas: 3` и PDB (Петров, 07.11), проба на `/readyz` (Сидоров, 03.11).

4. `fallocate -l 5G /fill`; симптомы — `DiskPressure`, evictions, `no space left on device` при pull. Диагностика: `df -h`, `du -xhd1 /var | sort -h`, `journalctl --disk-usage`, `crictl rmi --prune`. Удалить файл, флаг снимется за ~5 минут. Без консоли помог бы DaemonSet с `hostPath: /` и `nsenter` либо serial console — и то и другое готовят до инцидента.

5. Page — только если есть влияние на пользователей и действие нужно в минуты. «Диск 80%», «сертификат истекает через 30 дней», «рестарты > 5 за час» без нарушения SLO — тикеты. Из 24 page-алертов остаются 15, 9 переводятся в тикеты: звонков

было ~7 в неделю, стало ~2. Пересматривать раз в месяц, иначе тикеты копятся и теряются.

Приложение D. Ответы и решения — часть 5 (дополнительные главы)

Глава 5d. eBPF: наблюдение за ядром без пересборки

1. `sudo bpftrace -e 'tracepoint:syscalls:sys_enter_execve { printf("%-16s pid=%-7d ppid=%-7d %s\n", comm, pid, ppid, str(args->filename)); } interval:s:60 { exit(); }'` — за минуту на рабочем хосте 40–80 строк вида `bash pid=589529 ppid=589501 /usr/bin/docker`. `comm` берётся из `task->comm`: 16 байт, копия basename бинарника, но её переопределяет `prctl(PR_SET_NAME)`, её наследует `fork`, а у скрипта с шебангом там интерпретатор (`python3`, а не `deploy.py`).
2. `sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat /str(args->filename) == "/srv/data/report.csv"/ { @s[tid]=nsecs; } tracepoint:syscalls:sys_exit_openat /@s[tid]/ { @us = hist((nsecs-@s[tid])/1000); delete(@s[tid]); }'`. На тёплом кэше почти всё в бакете `[1]` (2700 из 3200); после `echo 3 > /proc/sys/vm/drop_caches` бакет `[1]` падает до десятков, вырастают `[8,16)`, `[32,64)`, `[128,256)`: путь и inode ищутся на диске, syscall ждёт ввода-вывода.
3. DNS и рукопожатие — две разные точки. DNS: `uprobe:/lib/x86_64-linux-gnu/libc.so.6:getaddrinfo` плюс `uretprobe` на нём, разница по `tid`. Рукопожатие: `tracepoint:sock:inet_sock_set_state` с `newstate==2` (SYN_SENT) и потом `newstate==1` (ESTABLISHED), ключ `args->skaddr`, имя процесса запоминается в первой пробе. Типично 2–40 мс DNS и 30–40 мс рукопожатие при RTT 30 мс; сверка — `curl -w '%{time_namelookup} %{time_connect}'`.
4. Интерактивный `cat` на терминале ждёт человека: гистограмма off-CPU уходит в секунды и минуты, последний бакет `[1024,2048)` и выше — десятки срабатываний. `dd` ждёт диск: узкий пик в 10–50 мс на блок, хвост до сотен миллисекунд при flush. Форма отражает природу ожидания: неограниченное человеческое

против ограниченного по времени устройства.

5. `sudo bpftrace -e 'tracepoint:syscalls:sys_enter_execve { @by_parent[comm] = count(); }'` — на входе в `execve` в `comm` ещё старый образ, то есть родитель. Накладные расходы: `perf stat -e cycles,instructions -a -- sleep 30` до и с пробой — при 5 тыс. exec/с +0,4% CPU, при 50 тыс. +9%. Безопасно до ~10 тыс. событий/с на ядро с агрегатом без `printf`; с `printf` — 2–3 тыс. Выше нужен фильтр в ядре или ringbuf.

Глава 5e. SSH: ключи, конфиг, туннели

1. `ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -C 'nextdoor@workstation'`, `ssh-copy-id -i ~/.ssh/id_ed25519.pub -p 2222 nextdoor@127.0.0.1`, затем `PasswordAuthentication no`, `sudo sshd -t && sudo systemctl reload ssh`. С неверным ключом `ssh -vv` печатает `Offering public key: id_other ED25519 SHA256:...`, строки `Server accepts key` нет, сервер отвечает `Authentications that can continue: publickey`, итог — `Permission denied (publickey)`. Пароль выключен, других методов не осталось.
2. Блоки `Host staging` / `Host prod` с разными `IdentityFile` и `IdentitiesOnly yes`. Проверка: `ssh -G staging | grep -i identityfile` → `identityfile ~/.ssh/id_staging`, для `prod` — свой. `IdentitiesOnly` важен: без него агент предлагает все ключи подряд, сервер считает неудачные попытки и после `MaxAuthTries` (6 по умолчанию, в нашем конфиге 3) рвёт соединение с `Too many authentication failures`, хотя нужный ключ у него есть.
3. `ssh -f -N -p 2222 -L 19080:127.0.0.1:8080 nextdoor@127.0.0.1`; `ss -tlnp | grep 19080` → `users:(("ssh",pid=595634,fd=5))`, `curl` → `200`. Слушает клиентский `ssh`: он держит локальный сокет, а TCP к `127.0.0.1:8080` открывает уже удалённая сторона в своём namespace. Сервер о существовании порта 19080 не знает — он есть только на вашей машине.
4. `command="/usr/bin/head -c 4096 /srv/reports/daily.csv",no-pty,no-port-forwarding,no-agent-forwarding,no-X11-forwarding ssh-ed25519 AAAA... ci@build`. `ssh host 'cat /etc/passwd'` печатает первые 4 КиБ отчёта, `ssh -tt host` → `PTY allocation request failed on channel 0`. Ограничения проверяет сервер: `command=` подменяет запрошенное, `no-pty` отклоняет `pty-req`, поэтому обойти их клиентом нельзя.

5. `ControlMaster auto`, `ControlPath ~/.ssh/cm/%r@%h:%p`, `ControlPersist 10m`: первая команда 106 мс, вторая и третья 8–10 мс против 91/92/98 без мультиплексирования. `ProxyJump bastion` даёт два TCP-соединения в цепочке (клиент→бастион и бастион→цель), два KEX и две аутентификации; бастион видит только шифрованный поток, а ключ предъявляется целевой машине.

Глава 12a. API: что это такое и как им пользоваться

1. Пример на GitHub REST: `Authorization: Bearer <token>`; лимит 5000 запросов в час на токен (60 без токена), заголовки `X-RateLimit-Remaining` и `Reset`; ошибка — `application/json` с `message`, `status`, `documentation_url`; пагинация `per_page`/`page` плюс `Link: rel="next"`; версия — заголовок `X-GitHub-Api-Version: 2022-11-28`; deprecation — `Deprecation` и `Sunset` в ответе и раздел changelog. Критерий: шесть пунктов выписаны из документации, а не угаданы.

2. `curl -i` печатает статусную строку, все заголовки как есть (включая повторы `Set-Cookie`) и тело первого ответа, без перехода по редиректам. httpx редиректы по умолчанию выполняет, отдаёт разобранные заголовки без учёта регистра, склеивает одноимённые через запятую и не показывает статусную строку; `raise_for_status()` бросает `HTTPStatusError` с `.response` внутри, поэтому `Retry-After` и тело ошибки доступны после исключения.

3. Девять записей A…I, страница по три. Offset: `page=1` → A,B,C; `page=2` → D,E,F; вставка N в начало; `page=3&offset=6` → F,G,H — F пришёл дважды, I потерян. Cursor: `?after=<id F>` → G,H,I, потому что условие `(created_at, id) < (:ts, :id)` идёт по индексу и вставка в начало его не сдвигает. Плата за курсор — нельзя прыгнуть на страницу 7.

4. Подпись — `HMAC-SHA256(secret, f"{ts}." + raw_body)`, сравнение через `hmac.compare_digest`, окно `abs(time.time() - int(ts)) <= 300`. Изменённое тело: HMAC не сходится, `401`, событие не сохранено. Подпись недельной давности: HMAC верный, но разница 604 800 с > 300, отказ `401` — окно и есть защита от

реплея.

5. Ночная процедура: в Vault лежат `stripe_key` и `stripe_key_next`, приложение читает `_next` при наличии; выкатка; проверка по метрике успешных вызовов; отзыв старого ключа в панели; запись в аудит. При утечке — сначала отзыв, потом замена: `vault kv get -field=stripe_key`, `revoke`, новый ключ, рестарт подов, `gitleaks detect --source .` по истории.

Глава 24a. Makefile: контракт репозитория

1. Файловая цель с зависимостями от исходников: `BIN:=bin/tasksvc`, `SRC:=$(shell find . -name '*.go' -not -path './vendor/*')`, правило `$(BIN): $(SRC) go.mod` с `go build -trimpath -o $@ ./cmd/tasksvc`, `build: $(BIN)`, `test:`, `clean:`, всё в `.PHONY` кроме `$(BIN)`. Повторный `make build` → `make: 'bin/tasksvc' is up to date.`; `touch cmd/tasksvc/main.go` → пересборка. Работает за счёт сравнения mtime.

2. Пробелы: `Makefile:4: *** missing separator. Stop.`, код 2. Без `.PHONY` при существующем каталоге: `mkdir test; make test` → `make: 'test' is up to date.`, код 0 — тесты не запускаются, CI зелёный. `NOW = $(shell sleep 1; date +%s)` в одном рецепте дважды даёт 1793000001 и 1793000002: ошибки нет, есть молчаливо разные значения в одной сборке.

3. `PROTOS := $(wildcard proto/*.proto)`, `GEN := $(PROTOS:.proto=.pb.go)`, правило `%.pb.go: %.proto` с `protoc ... $<`, цель `gen: $(GEN)`. Новый `proto/refund.proto` попадает в `wildcard` при следующем запуске, и `make gen` собирает только его — остальные `.pb.go` новее своих `.proto`. Правка Makefile не нужна, `%` покрывает семейство.

4. Две цели пишут в `coverage.out`: при `make -j8` второй процесс открывает файл на запись, первый получает обрезанный профиль, `go tool cover -func` показывает часть пакетов, иногда `open coverage.out: no such file`. Разносим артефакты (`coverage.api.out`, `coverage.db.out`, слияние отдельной целью) или ставим явную зависимость и `.NOTPARALLEL: test`; гонка видна по нестабильному числу покрытия.

5. `deploy:` с `ifdef CONFIRM` и проверкой `ifeq ($(ENV),prod)`; без переменных печатает `plans: kubectl -n prod set image ...` и выходит

с нулём, с `ENV=prod CONFIRM=yes` выполняет. Защита в Makefile, потому что `make deploy` — единственная точка входа для человека и CI: скрипт можно вызвать в обход, а цель без `CONFIRM` не выполнится ни у кого.

Глава 34a. Очереди и асинхронная обработка

1. Очередь объявляется с `x-max-length=100` и `x-max-length-behavior=reject-publish`; публикуем 1000. В очереди остаётся 100, новые не вытесняют старые. С publisher confirms producer получает 100 `ack` и 900 `basic.nack` — это и есть сигнал притормозить. Без confirms publish выглядит успешным, сообщения молча теряются: `rabbitmqctl list_queues name messages_ready` показывает 100 при отправленных 1000.
2. Убиваем процесс после `COMMIT`, до `Ack`: брокер вернёт сообщение с `redelivered=true`. CTE `INSERT INTO processed_events ... ON CONFLICT DO NOTHING RETURNING 1` не вернёт строк, вставка в `payments` не выполнится, обработчик завершится успешно. В таблице одна строка на ключ, баланс 1000, а не 2000: дедуп и запись — одна транзакция.
3. Три retry-очереди без потребителей с `x-message-ttl` 5000/30000/300000 и DLX обратно в рабочую; счётчик попыток — в заголовке `x-attempts`, после третьей `nack(requeue=false)` → DLQ. `x-death` растёт: `[{count:1,reason:expired,queue:payments.retry.5s}]`, затем `count:2` с `retry.30s`, затем `count:3` и сообщение в `payments.dlq`. Монотонный `count` — доказательство, что задержка реально применялась.
4. `kafka-topics.sh --create --partitions 3 --replication-factor 3`: ключ `order_id` даёт `hash(key) % 3`, и `created → paid → shipped` одного заказа идут в одну партицию по порядку. Горячий ключ виден в `--describe --group payments-worker`: одна партиция с LAG в тысячи, две по нулю — порядок сохранён, но параллелизм на этом ключе потерян.
5. `time.Sleep(200 * time.Millisecond)` в обработчике даёт 5 сообщений/с на потребителя при входе 50/с: лаг растёт на 45/с, за 10 минут — 27 000. Порог алерта считаем по времени: `lag / rate > SLO/2`, то есть при SLO 60 с сигнал на 30 с лага (150

сообщений), а не на «10 000», которое на здоровой системе срабатывает ложно.

Глава 34b. SQL: индексы, планы, блокировки

1. Миллион заказов, `status` с четырьмя значениями. Без индекса: `Seq Scan ... rows=250000`, `Buffers: shared hit=8131`, 94 мс. Индекс по `user_id`: `Bitmap Heap Scan`, 26 буферов, 0,13 мс на 20 строках. Индекс по `status` планировщик игнорирует: 25% таблицы — это 250 000 случайных чтений против 8131 последовательных, оценка стоимости индекса выше, и запрос снова идёт seq scan.
2. Работает: `WHERE user_id=42`; `WHERE user_id=42 AND status='paid'`; `WHERE user_id=42 AND status='paid' AND created_at > now()-interval '30 days' ORDER BY created_at DESC`. Бесполезен: только `status='paid'` (нет левого префикса); только диапазон по `created_at`; `WHERE user_id > 100 AND status='paid'` — диапазон по первой колонке ломает использование второй, дальше идёт фильтр.
3. `WHERE city='Москва' AND zip='101000'` при коррелированных колонках: планировщик перемножает селективности, оценивает `rows=250`, фактически 25 000 — расхождение в 100 раз, и вместо `Nested Loop` выбирается неверный `Hash Join`. Лечится `CREATE STATISTICS st_city_zip (dependencies, ndistinct) ON city, zip FROM addresses; ANALYZE addresses;` — оценка становится 24 800, план меняется.
4. Сессия 1: `UPDATE orders SET total=1 WHERE id=10;` затем `id=11`. Сессия 2 наоборот. Через секунду (`deadlock_timeout`) одна падает: `ERROR: deadlock detected`, `DETAIL: Process 158 waits for ShareLock on transaction 845; blocked by process 151`. Виновник — `SELECT pid, pg_blocking_pids(pid), query FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid))>0`. Лечение — единый порядок: обновлять `ORDER BY id`.
5. `LIMIT 20 OFFSET 800000`: `Sort Method: external merge Disk: 25488kB`, 412 мс, `temp read/written` около 25 МБ, буферов — десятки тысяч. Keyset `WHERE created_at < $1 ORDER BY created_at DESC LIMIT 20` по индексу: 54 мс, ноль temp, `Buffers: shared hit=25`. Разница в 7,6 раза и в гигабайтах temp на длинной выборке; цена — нельзя прыгнуть на произвольную страницу.

Глава 39a. Бэкапы и восстановление

1. `pg_dump` 40 МБ/с: 200 000/40 = 5000 с ≈ 83 мин; канал 100 Мбит/с = 12,5 МБ/с: 200 000/12,5 = 16 000 с ≈ 267 мин; `pg_restore` 25 МБ/с: 8000 с ≈ 133 мин. RTO ≈ 483 мин ≈ 8 ч, и узкое место — не диск, а канал. `pg_basebackup` копирует файлы: 200 000/300 ≈ 11 мин плюс replay WAL. Приёмка: этапы посчитаны отдельно, узкое место названо, физическая копия укладывается в час.
2. `restic init`, три `restic backup /srv/data` с правкой 1% файлов. `restic stats --mode raw-data` → 2,04 ГБ при 6 ГБ логических данных, коэффициент 2,9×. Второй и третий снапшоты добавили по ~20 МБ: блоки режутся по содержимому (в среднем ~1 МиБ), неизменные 99% ссылаются на уже записанные пакеты. Приёмка: три снапшота в `restic snapshots`, raw-data < 2,2 ГБ, `restic check` без ошибок.
3. `pg_basebackup -D /var/backups/pg/base -Fp -Xs -P -R`, `archive_mode=on`, `archive_command='test ! -f /var/lib/pgwal/%f && cp %p /var/lib/pgwal/%f'`. `DROP TABLE orders`, затем `recovery_target_time='2026-10-29 11:58:00+00'`, `restore_command='cp /var/lib/pgwal/%f %p'`, `recovery_target_action='promote'`. Приёмка: таблица на месте, `pg_is_in_recovery()` = false, потеря ≤ 1 минуты.
4. Скрипт: `count(*)` по трём таблицам против прода, допуск `max(0.5%, суточный прирост × 2)`. Коды: 0 — сходится, 2 — расхождение данных, 3 — превышен `RTO_MIN`, 1 — технический отказ (нет доступа, пустой дамп). Приёмка: коды различимы в systemd, допуск выводится из прироста, время сравнивается с целью после smoke-теста.
5. Двенадцать сервисов: платежи (RPO 5 мин, RTO 1 ч, PITR), API (15 мин/2 ч), внутренние (24 ч/8 ч), аналитика (7 дней/48 ч). PITR не нужен шести: внутренние инструменты и аналитика терпят суточную потерю, данные восстановимы из источника (события в Kafka, выгрузки), а WAL-архив и прогретый стенд стоят денег. Приёмка: у каждого сервиса пять полей, шесть обоснованы, суммарная стоимость посчитана.

Глава 41a. Helm и GitOps

1. `Chart.yaml`, `values.yaml` и `templates/app.yaml` (Deployment + Service + Ingress); `helm template shop ./charts/shop-api -f values-prod.yaml`. Меняются `image.tag`, `replicas`, `resources.limits`, `ingress.host`; остаются `metadata.name` через `fullname`, блок `labels`, порты Service, пути проб. Приёмка: `helm lint` чист, два рендера отличаются только ожидаемыми полями, `diff` объяснён построчно.
2. `_helpers.tpl` с `define "shop-api.labels"`, в шаблоне `{{- include "shop-api.labels" . | nindent 4 }}`; в Ingress `host: {{ required "ingress.host is required" .Values.ingress.host }}`. Без значения рендер падает: `Error: execution error at (shop-api/templates/app.yaml:41:14): ingress.host is required`, код 1. Приёмка: код 1, в тексте имя ключа, с `--set ingress.host=...` рендер проходит.
3. `base/` с общими манифестами, `overlays/staging` и `overlays/prod` с `images:` и `replicas` через патчи. Проще: смена тега, реплик, ресурсов — три строки в kustomization. Сложнее: условий и циклов нет, необязательный Ingress делают отдельным base или патчем с `$patch: delete`, а `required` заменяет `kubeconform` в CI. Приёмка: оба оверлея дают валидный YAML.
4. `argocd app create shop --repo ... --path deploy --dest-namespace shop --sync-policy automated --self-heal`. `kubectl scale deploy/shop-api --replicas=5` живёт до следующей сверки (по умолчанию 3 минуты), затем Argo возвращает значение из git: в UI `OutOfSync → Synced`, в событиях `self-heal`. Приёмка: откат ≤ 3 мин, `argocd app diff` пуст, действие в истории.
5. Коммит `8b2e7d4` с битым тегом → `ImagePullBackOff`, коммит `c3f9a1b` с исправлением → `Synced`. `git revert c3f9a1b` даёт `d41c0e5`, чей отрендеренный манифест равен ревизии до ошибки; `argocd app history shop` показывает три ревизии, у последней — `d41c0e5`. Приёмка: три записи в history, манифест совпадает, smoke-тест зелёный.

Глава 42a. Нагрузочное тестирование

1. При `-c 50` и нулевом think time: `ab` ~3200 rps, p99 190 мс; `wrk -t4 -c50` ~18 500 rps, p99 65 мс; k6 ~9800 rps, p99 95 мс. Расхождение не про сервис: `ab` однопоточный и упирается в

клиент, k6 тратит время на JS-рантайм и GC, `wrk` на epoll почти ничего не стоит. Приёмка: одинаковые `-c`, прогрев отброшен, CPU генератора < 70%, разброс p99 < 20% между прогонами.

2. `constant-vus` с 50 VU и задержкой 50 мс даёт потолок 1000 rps: когда сервис замедляется, VU сами ходят реже, темп падает и деградация маскируется. `ramping-arrival-rate` держит темп независимо от ответов, поэтому латентность и ошибки растут раньше. Приёмка: пиковая интенсивность та же, `dropped_iterations` = 0, колено видно на ступень раньше.
3. Пул на 10 соединений при 50 мс: `10 / 0,05 = 200 rps` — теоретическое колено. Таблица: 100 rps → p99 90 мс; 150 → 300 мс; 200 → 1500 мс; 250 → 6000 мс, ошибки 5%. Ресурс — не CPU (52%), а очередь за соединениями: `wait_count` в пуле, `idle in transaction` в `pg_stat_activity`. Приёмка: рабочая точка ≤ 70% колена, ресурс назван метрикой.
4. `pg_sleep(0.5)` в запросе при том же темпе: VU не успевают, k6 исчерпывает `maxVUs`, растёт `dropped_iterations`, фактический rps падает ниже целевого — тест недогружает сервис. В закрытой модели счётчика dropped нет: запросы копятся в VU, rps падает до `N / latency`, а latency включает ожидание в очереди клиента. Приёмка: dropped > 0, фактический rps приведён рядом с целевым.
5. С той же машины: p50 44 мс, p99 1100 мс; с отдельного хоста в той же сети: p50 46 мс, p99 780 мс. Плюс 2 мс — RTT сети, минус 300 мс хвоста — генератор больше не конкурирует за CPU и не делит conntrack с сервисом; на одной машине добавляются исчерпание эфемерных портов и `TIME_WAIT`. Приёмка: сценарий и данные те же, разница объяснена двумя причинами.

Глава 43a. Секреты: Vault, SOPS, age

1. Env: `tr '\0' '\n' < /proc/1/environ` внутри, `docker inspect` и `ps e` снаружи, значение наследуют дочерние процессы. Файл `0400` из Secret: внутри читает только владелец, снаружи — root узла и любой с `kubectl get secret -o yaml` (base64). `emptyDir` с `medium: Memory`: tmpfs, в `docker inspect` и `/proc/*/environ` значения нет, но оно в

RAM и видно root узла и в core dump. Приёмка: значения нет в трёх местах, ротация без рестарта.

2. `vault secrets enable -path=secret kv-v2`, `vault policy write shop-ro` с `path "secret/data/shop/api" { capabilities = ["read"] }`, `vault token create -policy=shop-ro -ttl=10m`. Через 10 минут `curl -H "X-Vault-Token: $T" .../v1/secret/data/shop/api` → `403 {"errors":["permission denied"]}`. Приёмка: 403 после TTL, политика только `read`, токен не `root`, чтения видны в аудите.

3. `.sops.yaml` с `encrypted_regex: ^(data|stringData|password|token|key)$`; `sops --encrypt --in-place values.yaml`. Меняем `image.tag` — в диффе одна открытая строка, остальные значения остаются `ENC[AES256_GCM,...]`. Расшифровка чужим ключом: `sops --decrypt --age <чужой> values.yaml` → `no identity matched any of the recipients`. Приёмка: дифф читаем, чужой ключ не работает, `.sops.yaml` в git.

4. `vault write database/roles/shop-api db_name=shop default_ttl=5m max_ttl=1h creation_statements="CREATE ROLE ... VALID UNTIL '{{expiration}}'"`. Открытое соединение после истечения живёт: PostgreSQL не рвёт сессии при `DROP ROLE`, но переподключение даёт `FATAL: password authentication failed`. Приёмка: старый пароль не пускает, `ConnMaxLifetime` меньше TTL, отказов в логах нет.

5. `git commit` с ключом → `gitleaks detect --source . --redact` находит `stripe_key` в `config.go:12` и в истории. Хронология: T0 коммит, T+4 мин детект в CI, T+9 мин отзыв в панели, T+15 мин новый ключ в Vault и выкатка, T+40 мин аудит показывает все чтения пути. Приёмка: gitleaks в CI, отзыв ≤ 15 мин, хронология из `vault audit` приложена.